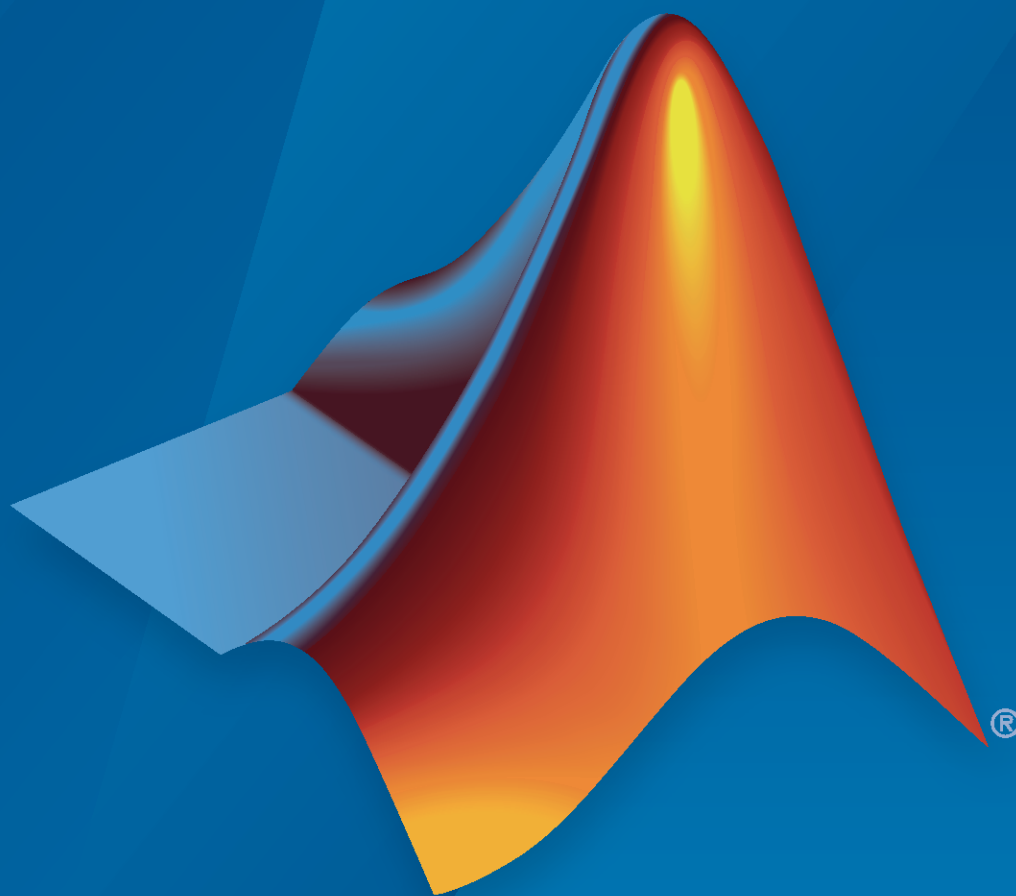


Simulink® Test™

User's Guide



MATLAB® & SIMULINK®

R2025b



How to Contact MathWorks



Latest news: www.mathworks.com

Sales and services: www.mathworks.com/sales_and_services

User community: www.mathworks.com/matlabcentral

Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Simulink® Test™ User's Guide

© COPYRIGHT 2015–2025 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

March 2015	Online only	New for Version 1.0 (Release 2015a)
September 2015	Online only	Revised for Version 1.1 (Release 2015b)
October 2015	Online only	Rereleased for Version 1.0.1 (Release 2015aSP1)
March 2016	Online only	Revised for Version 2.0 (Release 2016a)
September 2016	Online only	Revised for Version 2.1 (Release 2016b)
March 2017	Online Only	Revised for Version 2.2 (Release 2017a)
September 2017	Online Only	Revised for Version 2.3 (Release 2017b)
March 2018	Online Only	Revised for Version 2.4 (Release 2018a)
September 2018	Online Only	Revised for Version 2.5 (Release 2018b)
March 2019	Online Only	Revised for Version 3.0 (Release 2019a)
September 2019	Online Only	Revised for Version 3.1 (Release 2019b)
March 2020	Online Only	Revised for Version 3.2 (Release 2020a)
September 2020	Online Only	Revised for Version 3.3 (Release 2020b)
March 2021	Online Only	Revised for Version 3.4 (Release 2021a)
September 2021	Online Only	Revised for Version 3.5 (Release 2021b)
March 2022	Online Only	Revised for Version 3.6 (Release 2022a)
September 2022	Online Only	Revised for Version 3.7 (Release 2022b)
March 2023	Online Only	Revised for Version 3.8 (Release 2023a)
September 2023	Online Only	Revised for Version 23.2 (R2023b)
March 2024	Online Only	Revised for Version 24.1 (R2024a)
September 2024	Online Only	Revised for Version 24.2 (R2024b)
March 2025	Online Only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Rereleased for Version 25.2 (R2025b)

1	Test Strategies	
	Link to Requirements	1-2
	Requirements Traceability Considerations	1-4
	Establish Requirements Traceability for Testing	1-4
	Requirements-Based Testing for Model Development	1-8

2	Test Harness	
	Test Harness and Model Relationship	2-2
	Harness-Model Relationship for a Model Component	2-3
	Harness-Model Relationship for a Top-Level Model	2-4
	Resolving Parameters	2-5
	Test Harness Considerations	2-5
	Test Harness Construction for Specific Model Elements	2-8
	Signal Conversion	2-8
	Function Calls	2-8
	Physical Signal Connections	2-9
	Bus Signals	2-9
	String Signals	2-9
	Non-Graphical Connections	2-10
	Export Function Models	2-11
	Execution Semantics	2-11
	Sample Time Specification	2-11
	Create or Import Test Harnesses and Select Properties	2-13
	Create a Test Harness For a Top Level Model	2-13
	Create Test Harnesses for Model Components	2-13
	Import a Test Harness	2-14
	Preview and Open Test Harnesses	2-14
	Change Test Harness Properties	2-15
	Considerations for Selecting Test Harness Properties	2-15
	Create Test Harness Dialog Box Properties	2-15
	Import Test Harness Dialog Box Properties	2-19
	Customize Test Harness Creation Default Property Values	2-20
	Refine, Test, and Debug a Subsystem	2-22
	Manage Test Harnesses	2-29
	Manage External Test Harnesses	2-29

Convert Between Internal and External Test Harnesses	2-30
Preview and Open Test Harnesses	2-31
Model and Test Harness Locking	2-32
Find Test Cases Associated with a Test Harness	2-32
Export Test Harnesses to Standalone Models	2-33
Move and Clone Test Harnesses	2-33
Clone and Export a Test Harness to a Separate Model	2-35
Delete Test Harnesses Programmatically	2-37
Export Test Harness to Previous Version	2-39
Customize Test Harnesses	2-40
Callback Function Definition and Harness Information	2-40
Display Harness Information struct Contents	2-42
Share Data Between Callbacks	2-42
Customize a Test Harness to Create Mixed Source Types	2-43
Test Harness Callback Example	2-44
Create Test Harnesses from Standalone Models	2-47
Test Harness Import Workflow	2-47
Component Compatibility for Test Harness Import	2-48
Import a Standalone Model as a Test Harness	2-49
Synchronize Changes Between Test Harness and Model	2-52
Set Synchronization for a New Test Harness	2-52
Change Synchronization of an Existing Test Harness	2-57
Synchronize Configuration Set and Model Workspace Data	2-57
Check for Unsynchronized Component Differences	2-57
Rebuild a Test Harness	2-57
Push Changes from Test Harness to Model	2-58
Check Component and Push Parameter to Main Model	2-58
Test Library Blocks	2-62
Library Testing Workflow	2-62
Library and Linked Subsystem Test Harnesses	2-62
Edit Library Block from a Test Harness	2-63
Testing a Library and a Linked Block	2-63
SIL Testing a Reusable Library Subsystem	2-68

Test Sequences and Assessments

3

Test Sequence Basics	3-2
Test Sequence Hierarchy	3-2
Test Sequence Scenarios	3-2
Transition Types	3-2
Use Stateflow Chart for Test Harness Inputs and Scheduling	3-5
Use a Stateflow Chart for Test Harness Scheduling	3-5
Use a Stateflow Chart as a Test Harness Source	3-6
Stateflow Chart as Test Harness Scheduler and Source	3-6

Assess Simulation and Compare Output Data	3-11
Compare Simulation Data to Baseline Data or Another Simulation	3-11
Post-Process Results With a Custom Script	3-12
Run-Time Assessments	3-12
Logical and Temporal Assessments	3-13
Verify Model Simulation by Using when Decomposition	3-14
Activate verify Statements by Using Signal Conditions	3-14
Activate verify Statements by Using Test Sequence Steps	3-19
Author a Test Sequence and Test Assessment	3-22
Test Sequence Editor	3-29
Define Test Sequences	3-29
Manage Test Steps	3-29
Manage Input, Output, and Data Objects	3-31
Find and Replace	3-32
Automatic Syntax Correction	3-33
Output and View Active Step Data	3-33
Transitions, Temporal Operators, and Messages in Test Sequence Blocks	3-36
Transition Between Steps Using Temporal or Signal Conditions	3-36
Temporal Operators	3-36
Transition Operators	3-37
Use Messages in Test Sequences	3-37
Generate Test Signals	3-42
Signal Generation Functions	3-42
Sinusoidal and Random Number Functions in Test Sequences	3-42
Using an External Function in a Test Sequence Block	3-45
Programmatically Create a Test Sequence	3-47
Programmatically Create and Run Test Sequence Scenarios	3-51
Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager	3-54
Scenario Parameter Section	3-62
Define Test Steps and Assessments	3-63
Assessment Statements	3-63
Temporal Operators	3-63
Transition Operators	3-64
Signal Generation Functions	3-64
Logical Operators	3-65
Relational Operators	3-65
Debug a Test Sequence	3-67
Test Downshift Points of a Transmission Controller	3-70

Examine Model Verification Results by Using Simulation Data Inspector	3-75
.....	
Fix Requirements-Based Testing Issues	3-79
Assess Temporal Logic by Using Temporal Assessments	3-85
Create a Temporal Assessment	3-85
Define Temporal Assessment Conditions	3-86
Evaluate the SUT	3-88
Link Temporal Assessments to Requirements	3-90
Test Traffic Light Control by Using Logical and Temporal Assessments	3-91
.....	
Logical and Temporal Assessment Syntax	3-99
Bounds Check Assessments	3-99
Trigger-Response Assessments	3-99
Custom Assessments	3-101
Logical and Temporal Assessment Conditions	3-101
Define Variables in the Assessment Callback Section	3-102

Observers

4

Access Model Data Wirelessly by Using Observers	4-2
Observer Reference Block	4-3
Connect Signals or Other Model Data Using an Observer Port Block	4-4
Trace Observed Items to Model Signals and Objects	4-6
Simulate a System Model with an Observer Reference Block	4-6
Create Observers for Heat Pump Temperature Model	4-7
Convert Verification Subsystem to an Observer Reference	4-9
Observer Considerations and Limitations	4-11
Observe Messages	4-13
Message Bus Elements	4-13
Add a Message Observer	4-13
Observe a Message Signal	4-14
Observe Conditional Subsystem Signals	4-17
Add an Observer for a Conditional Subsystem	4-17
Observe a Signal in a Conditional Subsystem	4-18
Limitations	4-20
Observe Internal Variables of an FMU	4-21
Observe Internal Variables to Tune PID Controller Inside an FMU	4-21

SIL Verification for a Subsystem	5-2
Create a SIL Verification Harness for a Controller	5-2
Configure and Simulate a SIL Verification Harness	5-4
Compare the SIL Block and Model Controller Outputs	5-4
Use SIL/PIL to Verify Generated Code from an Earlier Release	5-6
Create Test Harness That Reuses Generated Code	5-6
Create Test Harness for Normal Simulation	5-7
Create Equivalence Test Case	5-7
Run Test and Analyze Results	5-8
Code Generation Verification Workflow with Simulink Test	5-10
Import Test Cases for Equivalence Testing	5-16
Settings for Test Case Simulations	5-16
Top-Level Model	5-16
Model Block in SIL/PIL Mode	5-17
Model Block or Reusable Library Subsystem in a Test Harness	5-18
Back-to-Back Testing a Model Using the SIL/PIL Manager App	5-19
Test Integrated Code	5-24
Test Integrated C Code	5-24
Testing Code in an S-Function Block	5-24

Test Manager Test Cases

Manage Test File Dependencies	6-3
Package a Test File Using Projects	6-3
Find Test File Dependencies and Impact	6-4
Share a Test File with Dependencies	6-5
Compare and Merge Test Files	6-6
Create Multiple Baseline Tests in Batch Mode	6-12
Test a Simulation for Run-Time Errors	6-14
Configure the Model	6-14
Create the Test Case	6-14
Run the Test Case	6-15
View the Error	6-15
Automatically Create a Set of Test Cases	6-17
Creating Test Cases from Model Elements	6-17
Generating Test Cases from a Model	6-17
Generate Tests and Test Harnesses for a Model or Components	6-20
Open the Create Test for Component Wizard	6-20

Select Model or Component to Test	6-21
Set Up Test Inputs	6-22
Test Method	6-23
Save Test Data	6-24
Generate the Test Harness and Test Case	6-25
Override Model Parameters in a Test Case	6-27
Override the Simulation Mode During Test Execution	6-32
Back-to-Back Equivalence Testing	6-36
Back-to-Back (MIL/SIL) Equivalence Testing an Atomic Subsystem	6-50
Testing AUTOSAR Compositions	6-59
Automate Testing for Highway Lane Following	6-64
Synchronize Tests	6-75
Use External File Data in Test Cases	6-76
Data Mapping	6-76
Create a Test Case from an Excel Spreadsheet	6-77
Import an Excel Spreadsheet into an Existing Test Case	6-78
Add Multiple Microsoft Excel Spreadsheets as Input to a Test Case	6-78
Include Microsoft Excel Test Data in Test Results	6-79
Importing Test Inputs from Microsoft Excel	6-79
Add a MAT-File or MATLAB Script (.m) File as External Input	6-82
Using External Files Programmatically	6-83
Create External Data Files to Use in Test Cases	6-84
Generate an Excel File	6-84
Input, Baseline, and Parameter Override Test Case Data Formats in Excel	6-89
Create a MAT File for Test Case Inputs	6-91
MATLAB Script Format for Test Case Data	6-92
Capture Simulation Data in a Test Case	6-93
Add Logged Signals When Creating a Test Harness	6-93
Add Logged Signals in the Test Manager	6-93
Capture Data from Local and Global Data Stores	6-95
Log Leaf Signals of a Bus	6-96
Use Triggers to Start and Stop Signal Logging	6-97
Run Tests in Multiple Releases of MATLAB	6-102
Considerations for Testing in Multiple Releases	6-102
Add Releases Using Test Manager Preferences	6-103
Run Baseline Tests in Multiple Releases	6-103
Run Equivalence Tests in Multiple Releases	6-104
Run Simulation Tests in Multiple Releases	6-105
Assess Temporal Logic in Multiple Releases	6-106
Collect Coverage in Multiple-Release Tests	6-107
Examine Test Failures and Modify Baselines	6-110
Examine Test Failure Signals and Update Baseline Test	6-110

Manually Update Signal Data in a Baseline	6-112
Create and Run Test Cases with Scripts	6-115
Create and Run a Baseline Test Case	6-115
Create and Run an Equivalence Test Case	6-116
Run a Test Case and Collect Coverage	6-117
Create and Run Test Case Iterations	6-117
Test Models Using MATLAB-Based Simulink Tests	6-119
Classes and Methods	6-119
Creating a Baseline MATLAB-Based Simulink Test	6-120
Linking to Requirements from a MATLAB-Based Simulink Test File	6-122
Limitations of MATLAB- Based Tests	6-123
Using MATLAB-Based Simulink Tests in the Test Manager	6-124
Collect Coverage Using MATLAB-Based Simulink Tests	6-128
Create and Run a RoadRunner Test Case	6-133
Set Up RoadRunner and Simulink Model	6-133
Use the Test Manager to Run a RoadRunner Test	6-133
Use Simulink Test API Commands to Create and Run a RoadRunner Test	6-137
RoadRunner Test Case Limitations	6-137
Test Iterations	6-139
Create Table Iterations	6-139
Create Scripted Iterations	6-143
Sweep Through a Set of Parameters	6-145
Capture Baseline Data from Iterations	6-148
Collect Coverage in Tests	6-151
Set Up Coverage Collection Using the Test Manager	6-151
View Coverage Results in the Test Manager	6-153
Add Tests for Missing Coverage	6-155
Coverage Filtering Using the Test Manager	6-156
Test Coverage for Requirements-Based Testing	6-158
Increase Test Coverage for a Model	6-163
Run Tests Using Parallel Execution	6-167
When Do Tests Benefit from Using Parallel Execution?	6-167
Limitations of Running Tests Using Parallel Execution	6-167
Use Parallel Execution	6-167
Set Signal Tolerances	6-169
Modify Criteria Tolerances	6-169
Change Leading Tolerance in a Baseline Comparison Test	6-169
Specify Test Properties in the Test Manager	6-173
Test Case, Test Suite, and Test File Sections Summary	6-173
Create Test Case from External File	6-175
Tags	6-175

Description	6-175
Requirements	6-175
RoadRunner	6-176
System Under Test	6-176
Simulation 1 and Simulation 2	6-178
Parameter Overrides	6-178
Callbacks	6-179
Inputs	6-181
Simulation Outputs	6-182
Configuration Settings Overrides	6-183
Fault Settings	6-184
Sequence Diagram Assessment	6-184
Baseline Criteria	6-184
Equivalence Criteria	6-186
Iterations	6-186
Logical and Temporal Assessments	6-187
Custom Criteria	6-188
Coverage Settings	6-188
Test File Options	6-190
Test File Content	6-190
Preferences	6-191
Increase Coverage by Generating Test Inputs	6-192
Overall Workflow	6-192
Generate Test Cases Using Simulink Design Verifier	6-193
Process Test Results with Custom Scripts	6-197
MATLAB Testing Framework	6-197
Define a Custom Criteria Script	6-197
Reuse Custom Criteria and Debug Using Breakpoints	6-198
Custom Criteria Programmatic Interface Example	6-200
Assess the Damping Ratio of a Flutter Suppression System	6-203
Create, Store, and Open MATLAB Figures	6-206
Create a Custom Figure for a Test Case	6-206
Include Figures in a Report	6-207
Test Models Using MATLAB Unit Test	6-209
Overall Workflow	6-209
Considerations	6-209
Comparison of Test Nomenclature	6-209
Basic Workflow Using MATLAB® Unit Test	6-210
Output Results for Continuous Integration Systems	6-212
Test a Model for Continuous Integration Systems	6-212
Model Coverage Results for Continuous Integration	6-214
Parametric Sweep for a Simscape Thermal Model	6-216
Projector Controller Testing Using verify and Real-Time Tests	6-222
Test Execution Order	6-227
Single Test Case on a Single Model	6-227

Multiple Test Cases on Multiple Models	6-227
Multiple Test Cases in a Single Test Suite on a Single Model	6-228
Multiple Test Cases in Multiple Test Suites on a Single Model	6-229
Test Case with Parameter Overrides	6-230
Filter Test Execution, Results, and Coverage	6-231
Add Tags	6-231
Filter Tests and Results	6-231
Run Filtered Tests	6-231
Filter Coverage	6-231
Write and Run Tests that Inject Faults by Using the Simulink Test Manager	6-232
Create Fault Sets in the Simulink Test Manager	6-232
Programmatically Create Fault Sets	6-233
Run Fault Injection Tests on an Example Model	6-233
Baseline Testing	6-240
Author a Baseline Test Case	6-242
Compare Simulation to Baseline Data	6-246
Create a Test Results Report	6-249

Test Manager Results and Reports

7

View Test Case Results	7-2
View Results Summary	7-2
Visualize Test Case Comparison and Criteria Results	7-3
Debugging Baseline Test Failures Using Model Slicer	7-6
Export Test Results	7-16
Create, Set Options, and Use Templates for Test Result Reports	7-17
Create a Test Results Report	7-17
Save Reporting Options with a Test File	7-17
Generate Reports Using Templates	7-17
Generating a Test Results Report	7-20
Customize Test Results Reports	7-21
Customize Reports Using a Custom Class	7-21
Customize Reports Using a Custom Template File	7-29
Customize a Test Results Report	7-29
Append Code to a Test Report	7-34
Results Sections	7-37
Summary	7-38

Test Requirements	7-38
Iteration Settings	7-38
Errors	7-39
Logs	7-39
Description	7-39
Variant Configuration Details	7-39
Fault Settings	7-39
Parameter Overrides	7-40
Coverage Results	7-40
Aggregated Coverage Results	7-40
Scope coverage results to linked requirements	7-40
Add Tests for Missing Coverage	7-40
Applied Coverage Filters	7-40
Generate Test Specification Reports	7-41
Customize Test Specification Reports	7-45
Update PDF or HTML Reports	7-45
Update Word Reports	7-48
Add Content to Reports	7-49
Debugging Equivalence Test Failures Using Model Slicer	7-54

8 | **Process Automation for Simulink Test**

Use Process Advisor to Streamline Requirements-Based Testing of Models and Generated Code	8-2
Set Up the Model Testing Process	8-3
Open the Process Advisor by Using Simulink Test Manager	8-4
Run Tasks and View Results in the Model Testing Process	8-5

9 | **Real-Time Testing**

Install and Set Up the Simulink Test Support Package for ASAM XIL Standard	9-2
Install the Support Package	9-2
Set Up the Test Bench	9-2
Configure and Build Your Model	9-2
Create and Run Tests Using the ASAM XIL Standard	9-4
Configure the Test Bench	9-4
Create the Test Body	9-6
Run the Test	9-7
Limitations	9-7
Troubleshooting	9-8

Create Tests That Use Third-Party Test Benches	9-9
Create ASAM XIL Test with Trigger that Controls Data Acquisition	9-9
Create ASAM XIL Test with Two Test Benches	9-11
Create Tests Using Simulink Model Simulation as a Test Bench	9-14
ASAM XIL Classes and Methods Supported by Simulink Test	9-17
Testbench Class	9-17
MAPort Class	9-18
MAPortVariableInfo Class	9-18
MAPortConfig Class	9-19
MAPortFactory Class	9-19
Capture Class	9-19
CaptureEvent Class	9-20
CaptureResult Class	9-20
CaptureResultMemoryWriter Class	9-20
CaptureResultMDFWriter Class	9-20
CycleNumberDuration Class	9-20
CapturingFactory Class	9-21
DurationWatcher Class	9-21
DurationFactory Class	9-21
CycleDurationNumber Class	9-21
TimeSpanDuration Class	9-21
WatcherFactory Class	9-22
TaskInfo Class	9-22
VendorSpecificConfiguration Class	9-22
Test Models in Real Time	9-23
Prepare, Create, and Run Real-Time Tests	9-23
Real-Time Testing Considerations	9-25
Real-Time Testing Limitations	9-25
Reuse Desktop Test Cases for Real-Time Testing	9-27
Convert Desktop Test Cases to Real-Time	9-27
Use External Data for Real-Time Tests	9-27
Reuse Desktop Test Case for Real-Time Testing	9-28
Prepare for Real-Time Testing	9-31
Complete Model Simulation Testing	9-31
Set Up the Target Computer	9-31
Configure the Model for Testing	9-31
Set Configuration Parameters	9-32
Create and Run Real-Time Tests	9-34
Add a Real-Time Test Case	9-34
Specify the System Under Test	9-34
Assign Code to Real-Time Callbacks	9-36
Create Test Iterations	9-36
Run Test and View Real-Time Test Results	9-38

Importing and Testing Custom C/C++ Code	10-2
Import Code Using the Wizard or the API	10-2
Code Importer Generated Artifacts	10-2
Limitations and Workarounds	10-3
Import Custom Code for Unit Testing Using API Commands	10-5
Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard	10-11

Verify and Validate a Model and Code	11-2
Test Model Against Requirements and Report Results	11-4
Display the Requirements	11-4
Link Requirements to Tests	11-5
Run the Test	11-6
Report the Results	11-7
Analyze Models for Standards Compliance and Design Errors	11-9
Check Model for MAB Style Guideline Violations	11-9
Check Model for Design Errors	11-10
Perform Functional Testing and Analyze Test Coverage	11-11
Open the Test Harness and Model	11-11
Measure Model Coverage	11-12
Generate Tests to Increase Model Coverage	11-12
Analyze Code and Perform Software-in-the-Loop Testing	11-14
Analyze Code for Defects, Metrics, and MISRA C:2012	11-14
Test Code Against Model Using Software-in-the-Loop Testing	11-19

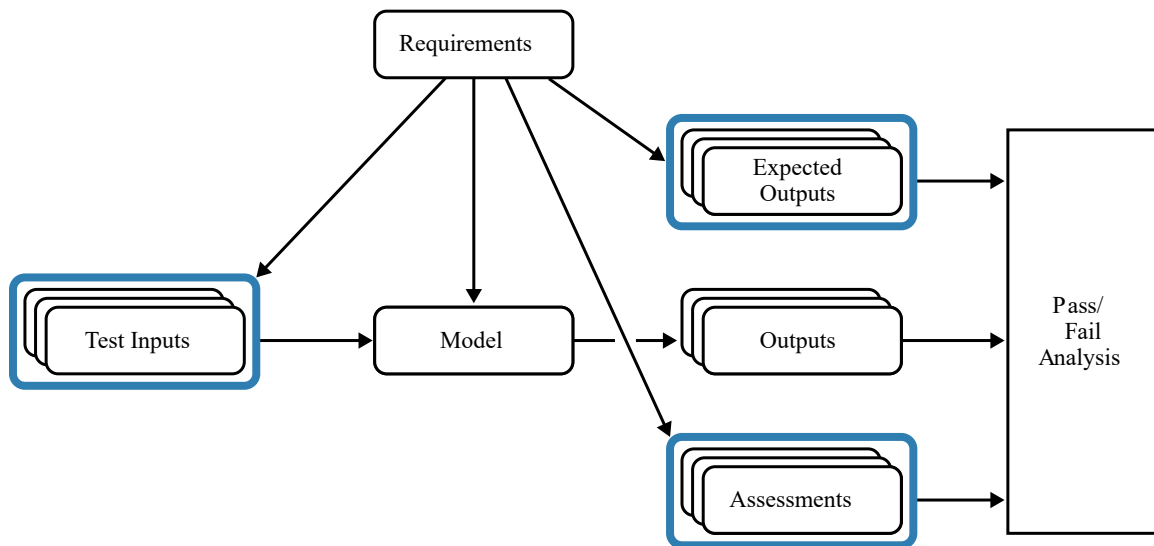
Test Strategies

- “Link to Requirements” on page 1-2
- “Requirements-Based Testing for Model Development” on page 1-8

Link to Requirements

In this section...
“Requirements Traceability Considerations” on page 1-4
“Establish Requirements Traceability for Testing” on page 1-4

Since requirements specify behavior in response to particular conditions, you can develop test inputs, expected outputs, and assessments from the model requirements.



Requirements Traceability Considerations

Consider the following limitations working with requirements links in test harnesses:

- Some blocks and subsystems are recreated during test harness rebuild operations. Requirements linking is not supported for these blocks and subsystems in a test harness:
 - Conversion subsystems between the component under test and the sources or sinks
 - Test Sequence blocks that schedule function calls
 - Blocks that drive control input signals to the component under test
 - Blocks that drive Goto or From blocks that pass component under test signals
 - Data Store Read and Data Store Write blocks
- If you use external requirements storage, performing the following operations requires reestablishing requirements links to model objects inside test harnesses:
 - Cut/paste or copy/paste a subsystem with a test harness
 - Clone a test harness
 - Move a test harness from a linked block to the library block

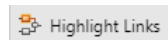
Establish Requirements Traceability for Testing

If you have a Simulink Test and a Requirements Toolbox™ license, you can link requirements to test harnesses, test sequences, and test cases.

Requirements Traceability for Test Harnesses

When you edit requirements links to the component under test, the links immediately synchronize between the test harness and the main model. Other changes to the component under test, such as adding a block, synchronize when you close the test harness. If you add a block to the component under test, close and reopen the harness to update the main model before adding a requirement link.

To view items with requirements links, on the **Apps** tab, under Model Verification, Validation, and Test, click **Requirements Manager**. In the **Requirements** tab, click **Highlight Links**



Requirements Traceability for Test Sequences

In test sequences, you can link to test steps. To create a link, first find the model item, test case, or location in the document you want to link to. Right-click the test step, select **Requirements**, and add a link or open the link editor.

To highlight or remove the highlighting from test steps that have requirements links, toggle the

requirements links highlighting button  in the Test Sequence Editor toolstrip. Highlighting test steps also highlights the model block diagram.

Requirements Traceability for Test Cases

If you use many test cases with a single test harness, link to each specific test case to distinguish which blocks and test steps apply to it. To link test steps or test harness blocks to test cases,

- 1 Open the test case in the Test Manager.
- 2 In the left pane, in the **Test Browser** tab, select the test case.
- 3 In Simulink in the **Apps** tab, click **Requirements Manager**.
- 4 To link a test case to a:
 - Simulink block, right-click the block and select **Requirements > Link to Current Test Case** from the context menu.
 - Test step, double-click the test sequence block in the test harness to open the Test Sequence Editor. Right-click the test step and select **Requirements > Link to Current Test Case** from the context menu.

Requirements Traceability Example

This example demonstrates adding requirements links to a test harness and test sequence. The model is a component of an autopilot roll control system. This example requires Simulink Test and Requirements Toolbox.

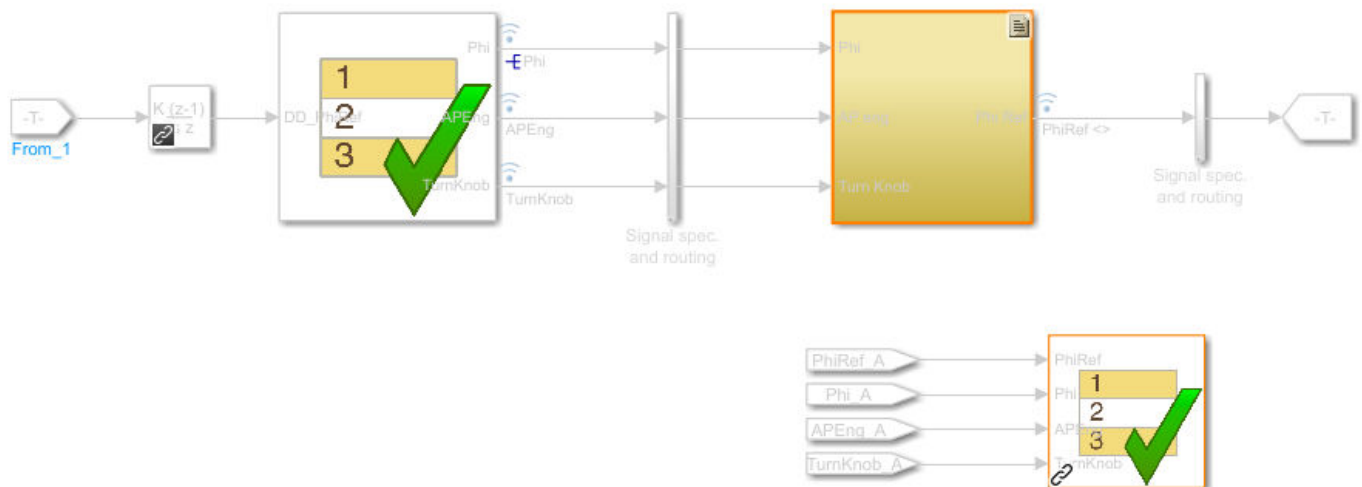
- 1 Open the model, the test file, and the harness.

```
openExample("simulinktest/ModelCoverageMATLABUnitExample", ...
    supportingFile="RollAutopilotMdlRef.slx")
openExample("simulinktest/ModelCoverageMATLABUnitExample", ...
    supportingFile="AutopilotTestFile.mldatx")
sltest.harness.open("RollAutopilotMdlRef/Roll Reference",...
    "RollReference_Requirement1_3")
```

- 2 In the test harness, on the **Apps** tab, under Model Verification, Validation, and Test, click **Requirements Manager**. In the **Requirements** tab, click **Highlight Links**



The test harness highlights the Test Sequence block, component under test, and Test Assessment block.



- 3 Add traceability to the Discrete Derivative block.

- a Right-click the Discrete Derivative block and select **Requirements > Open Outgoing Links dialog**.
- b In the **Requirements** tab, click **New**.
- c Enter the following to establish the link:
 - Description: DD link
 - Document type: Text File (legacy)
 - Document: RollAutopilotRequirements.txt
 - Location: 1.3 Roll Hold Reference

The screenshot shows a software dialog box titled 'Requirements' with a 'Document Index' tab selected. On the left side, there is a vertical column of buttons: 'New', 'Up', 'Down', 'Delete', and 'Copy'. To the right of these buttons is a list box containing the text 'DD Link'. Below the list box, there are several input fields and buttons: 'Description:' with the value 'DD Link'; 'Document type:' with a dropdown menu set to 'Text file' and a 'Use current' button; 'Document:' with a dropdown menu set to 'RollAutopilotRequirements.txt' and a 'Browse...' button; 'Location: (Type/Identifier)' with a dropdown menu set to 'Search text' and a text input field containing '1.3 Roll Hold Reference'; and 'User tag:' with an empty dropdown menu.

- d Click **OK**. The Discrete Derivative block highlights.
- 4 To trace to the requirements document, right-click the Discrete Derivative block, and select **Requirements > DD Link**. The requirements document opens in the editor and highlights the linked text.

1.3 Roll Hold Reference

Navigate to test harness using MATLAB command:

```
web('http://localhost:31415/matlab/feval/rmiobjnavigate?argu
```

REQUIREMENT

1.3.1 When roll hold mode becomes the active mode the roll hold

Navigate to test step using MATLAB command:

```
web('http://localhost:31415/matlab/feval/rmiobjnavigate?argu
```

1.3.1.1. The roll hold reference shall be set to zero if the act

Navigate to test step using MATLAB command:

```
web('http://localhost:31415/matlab/feval/rmiobjnavigate?argu
```

- 5 In the test harness, open the Test Sequence block. Add a requirements link that links the InitializeTest step to the test case.
 - a In the Test Manager, in the left pane, in the **Test Browser** tab, select Requirement 1.3 Test.
 - b In the test harness, double-click the test sequence block to open the Test Sequence Editor. Right-click the InitializeTest step and select **Requirements > Link to Current Test Case** from the context menu.

When the requirements link is added, the Test Sequence Editor highlights the step.

Step	Transition
InitializeTest Phi = 0; APEng = false; TurnKnob = 0; % Initializes test sequence outputs	1. true

See Also

More About

- “Test Model Against Requirements and Report Results” on page 11-4
- “Link Directly to Requirements in Third-Party Applications” (Requirements Toolbox)

Requirements-Based Testing for Model Development

This example shows how to test an autopilot subsystem against a requirement.

This example shows how to test a subsystem against a requirement by using the test manager, test harness, Test Sequence block, and Test Assessment block. The requirements document links to the test case and test harness, and `verify` statements assess the component under test.

As you build your model, you can add test cases to verify the model against requirements. Subsequent users can run the same test cases, then add test cases to accomplish further verification goals such as achieving 100% coverage or verifying generated code.

This example tests the Roll Reference subsystem against a requirement using three scenarios. A Test Sequence block provides inputs, and a Test Assessment block evaluates the component. The Roll Reference subsystem is one component of an autopilot control system. Roll Reference controls the reference angle of the aircraft's roll control system. The subsystem fails one assessment, prompting a design change that limits the subsystem output at high input angles.

Paths and Example Files

Enter the following to store paths and file names for the example:

```
topModel = 'TestAndVerificationAutopilotExample';  
rollModel = 'RollAutopilotMdlRef';  
testHarness = 'RollReference_Requirement1_3';  
testFile = 'AutopilotTestFile.mldatx';  
reqDoc = 'RollAutopilotRequirements.txt';
```

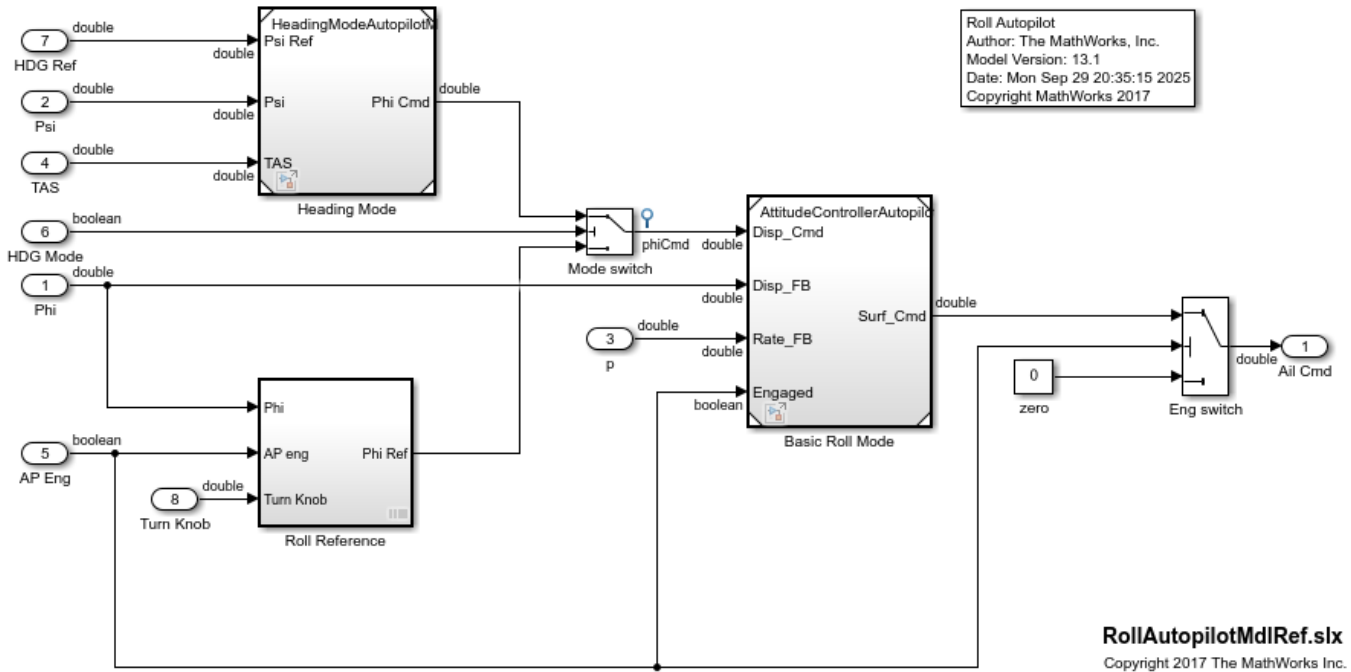
Open the Test File and Model

Open the RollAutopilotMdlRef model. The full control system TestAndVerificationAutopilotExample references this model.

```
open_system(rollModel)
```

Requirements-based Testing for Controller Development

This model is used to show how to perform requirements-based testing using test harnesses, Test Sequence blocks, and the test manager.
To view the demo, enter `sltestRequirementsTestingAutopilotDemo` in MATLAB(R).



Open the test file in the Test Manager.

```
tf = sltest.testmanager.load(testFile);
sltest.testmanager.view;
```

Open the Requirements Editor to see the requirements associated with the Roll Reference block. In the **Test Browser** pane, expand **AutopilotTestFile** and **Basic Design Test Cases** in the tree, and click **Requirement 1.3 test**. In the Requirement 1.3 test tab, expand **Requirements**. Double-click on any of the requirements links to open the Requirements Editor. Highlighting a requirement displays its details in the Description section of the Properties pane.

Requirement 1.3 Test

[AutopilotTestFile](#) » [Basic Design Test Cases](#) » [Requirement 1.3 Test](#)

Simulation Test

☐ Create Test Case from External File

► TAGS

► DESCRIPTION

▼ REQUIREMENTS*

[1.3.1.1 phiref = 0 if phi < 6](#)

[1.3.1.2 phiref = 30 if phi > 30](#)

[1.3.1.3 phiref = tk if tk >= 3](#)

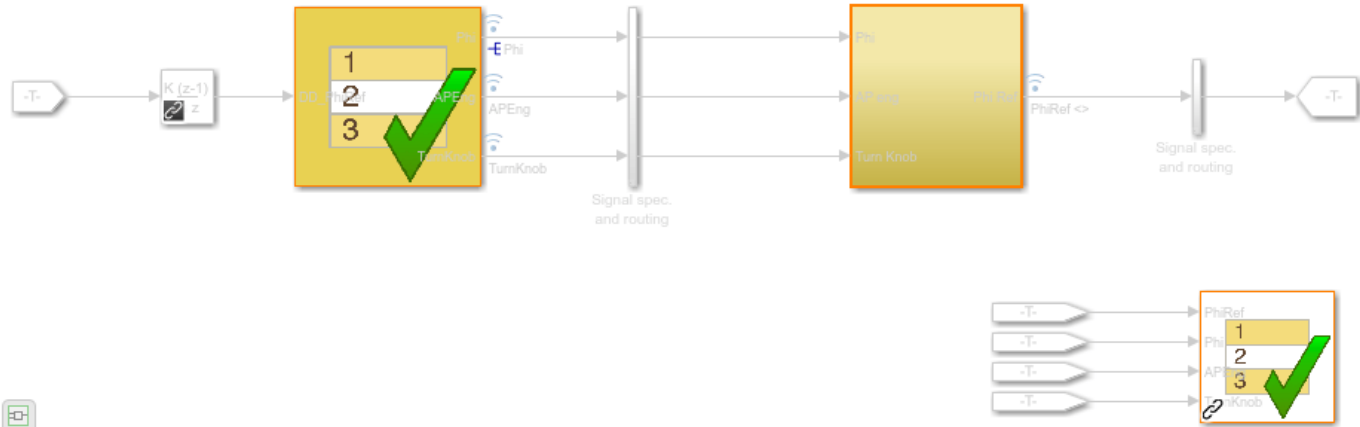
Requirement 1.3.1 specifies that when roll hold mode becomes the active mode, the roll hold reference shall be set to the actual roll angle of the aircraft, except under the conditions described in the child requirements.

- Child requirement 1.3.1.1 specifies that the roll hold reference shall be set to zero if the actual roll angle is less than 6 degrees, in either direction, at the time of roll hold engagement.
- Child requirement 1.3.1.2 specifies that the roll hold reference shall be set to 30 degrees in the same direction as the actual roll angle if the actual roll angle is greater than 30 degrees at the time of roll hold engagement.
- Child requirement 1.3.1.3 specifies that the roll reference shall be set to the cockpit turn knob command, up to a 30 degree limit, if the turn knob is commanding 3 degrees or more in either direction.

The test case creates a scenario for each child requirement to test the normal conditions and exceptions in the requirement.

The requirements document traces to the test harness using URLs that map to the Test Sequence block and test steps. Open the test harness and highlight the component associated with reference requirement 1.3.

```
sltest.harness.open([rollModel '/Roll Reference'],testHarness)  
rmi('highlightModel','RollReference_Requirement1_3')
```



The Test Sequence block, Test Assessment block, and component under test link to the requirements document. Highlight requirements links by selecting **Apps > Requirements Manager** and then, clicking **Highlight Links** in the test harness model. You can also highlight links in the Test Sequence Editor by clicking **Toggle requirements links highlighting** in the toolstrip. For information on creating requirements links, see “View and Create Links” (Requirements Toolbox).

Test Sequence

Open the Test Sequence block.

```
open_system('RollReferenceRequirement1_3/Test Sequence')
```

Step	Transition	Next Step
InitializeTest Phi = 0; APEng = false; TurnKnob = 0; % Initializes test sequence outputs	1. true	AttitudeLevels ▼
☐ AttitudeLevels TurnKnob = 0; EndTest = 0; % Tests correct PhiRef for several attitude levels Add step after • Add sub-step	1. EndTest == 1	TurnKnobLevels ▼
☐ APEngage_LowRoll % Tests low attitude	1. duration(DD_PhiRef == 0,sec) >= DurationLimit % transitions when the discrete derivative of PhiRef % is equal to 0 for a certain time limit. This means the % signal is not changing.	APEngage_MedRoll ▼
SetLowPhi Phi = 4; APEng = false;	1. true	EngageAP_Low ▼

The Test Sequence block creates test inputs for three scenarios:

In each test, the test sequence sets a signal level, then engages the autopilot. The test sequence checks that PhiRef is stable for a minimum time *DurationLimit* before it transitions to the next signal level. For the first two scenarios, the test sequence sets the *EndTest* local variable to 1, triggering the transition to the next scenario.

These scenarios check basic component function, but do not necessarily achieve objectives such as 100% coverage.

Test Assessments

Open the Test Assessment block.

```
open_system('RollReference_Requirement1_3/Test Assessment')
```

Step	
1	GlobalAssessment
2	NormalRange when (abs(PHI) >= 6 && PHI <= 30) && APEng == true && abs(TurnKnob) <= 0.001 verify(abs(PHIRef - PHI) < 0.001, 'Simulink:verify_normal', 'PHIRef must equal PHI for normal operation')
3	BelowLowLimit when abs(PHI) < 6 && APEng == true && abs(TurnKnob) <= 0.001 verify(abs(PHIRef - 0) < 0.001, 'Simulink:verify_low', 'PHIRef must equal 0 for low angle operation')
4	ExceedPosLimit when PHI > 30 && APEng == true && abs(TurnKnob) <= 0.001 verify(abs(PHIRef - 30) < 0.001, 'Simulink:verify_high_pos', 'PHIRef must equal 30 for high pos angle operation')
5	ExceedNegLimit when PHI < -30 && APEng == true && abs(TurnKnob) <= 0.001 verify(abs(PHIRef - (-30)) < 0.001, 'Simulink:verify_high_neg', 'PHIRef must equal -30 for high neg angle operation')

The Test Assessment block evaluates Roll Reference. The assessment block is a library linked subsystem, which facilitates test assessment reuse between multiple test harnesses. The block contains verify statements covering:

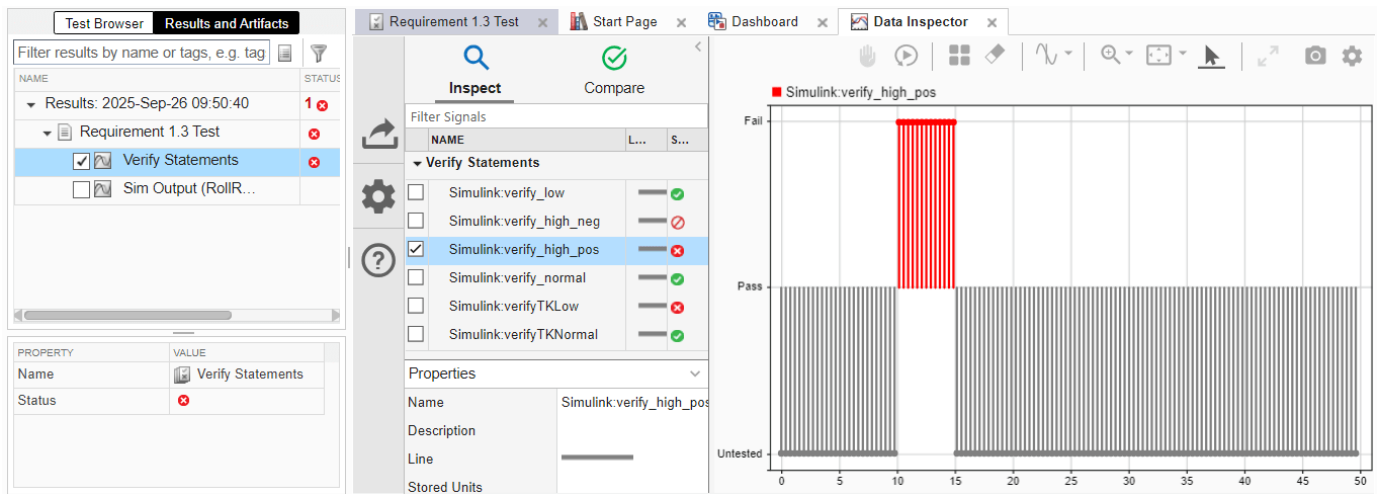
- The requirement that $\text{PHIRef} = \text{PHI}$ when PHI operates inside the low and high limits.
- The requirement that $\text{PHIRef} = 0$ when $\text{PHI} < 6$ degrees.
- The requirement that $\text{PHIRef} = 30$ when $\text{PHI} > 30$ degrees.
- The requirement that when TurnKnob is engaged, $\text{PHIRef} = \text{TurnKnob}$ if $\text{TurnKnob} \geq 3$ degrees.

Verify the Subsystem

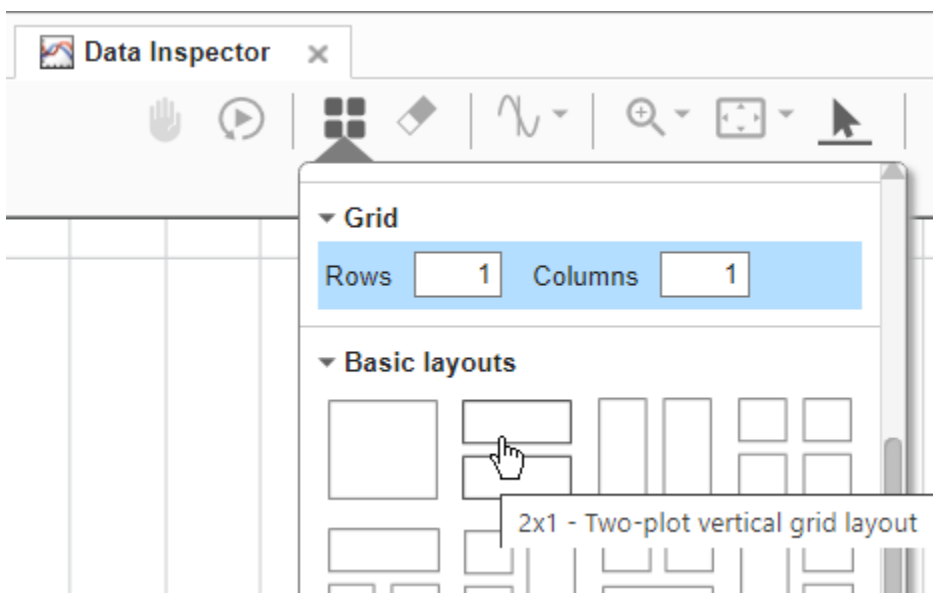
To run the test, in the Test Manager, right-click **Requirement 1.3 Test** in the Test Browser pane, and click **Run**.

The simulation returns verify statement results and simulation output in the Test Manager. The `verify_high_pos` statement fails.

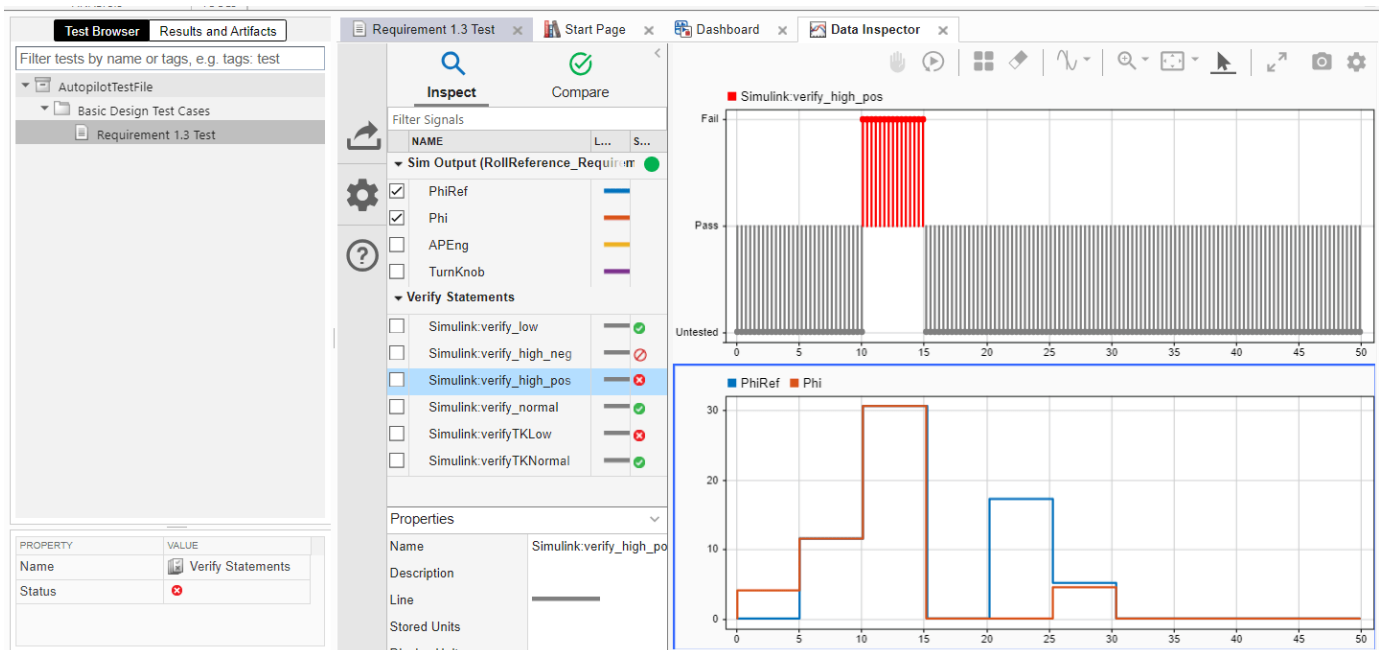
- 1 In the **Results and Artifacts** pane in the Test Manager, expand the **Results**.
- 2 Select **Verify Statements**.
- 3 In the Inspect pane of the **Data Inspector** tab, select **Simulink: verify_high_pos**. The plot shows when the statement fails.



- 1 In the "Data Inspector*" tab, click Visualizations and layouts and under **Basic layouts** select **2x1 - Two-plot vertical grid layout**.



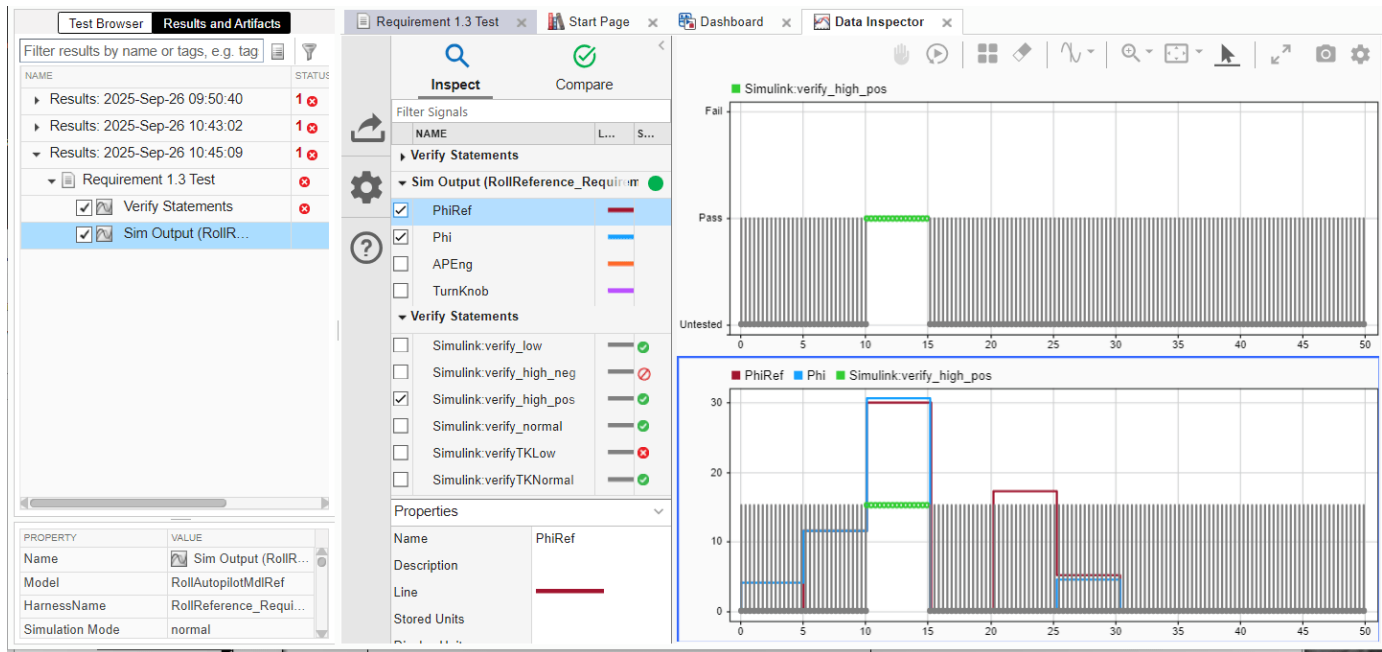
- 1 Click the lower plot, expand the **Sim Output** signals in the Inspect pane, and select PhiRef and Phi. The output traces align with the verify results in the upper plot. Observe that PhiRef exceeds 30 degrees when Phi exceeds 30 degrees.



Update RollReference to limit the PhiRef signal.

- 1 Close the test harness.
- 2 In the main model, double-click the Roll Reference block and add a Saturation block to the model as shown.
- 3 Set the upper limit to 30 and the lower limit to -30.
- 4 Link the block to its requirement. Click the Saturation block to select it. In the Requirements editor, right-click requirement 1.1.2 and click **Link from Saturation**.

Run the test again. The verify statement passes, and the output in the test manager shows that PhiRef does not exceed 30 degrees.



See Also

Requirements Manager

More About

- “Link to Requirements” on page 1-2
- “Test Model Against Requirements and Report Results” on page 11-4
- “Link Directly to Requirements in Third-Party Applications” (Requirements Toolbox)

Test Harness

- “Test Harness and Model Relationship” on page 2-2
- “Test Harness Construction for Specific Model Elements” on page 2-8
- “Create or Import Test Harnesses and Select Properties” on page 2-13
- “Refine, Test, and Debug a Subsystem” on page 2-22
- “Manage Test Harnesses” on page 2-29
- “Customize Test Harnesses” on page 2-40
- “Create Test Harnesses from Standalone Models” on page 2-47
- “Synchronize Changes Between Test Harness and Model” on page 2-52
- “Test Library Blocks” on page 2-62

Test Harness and Model Relationship

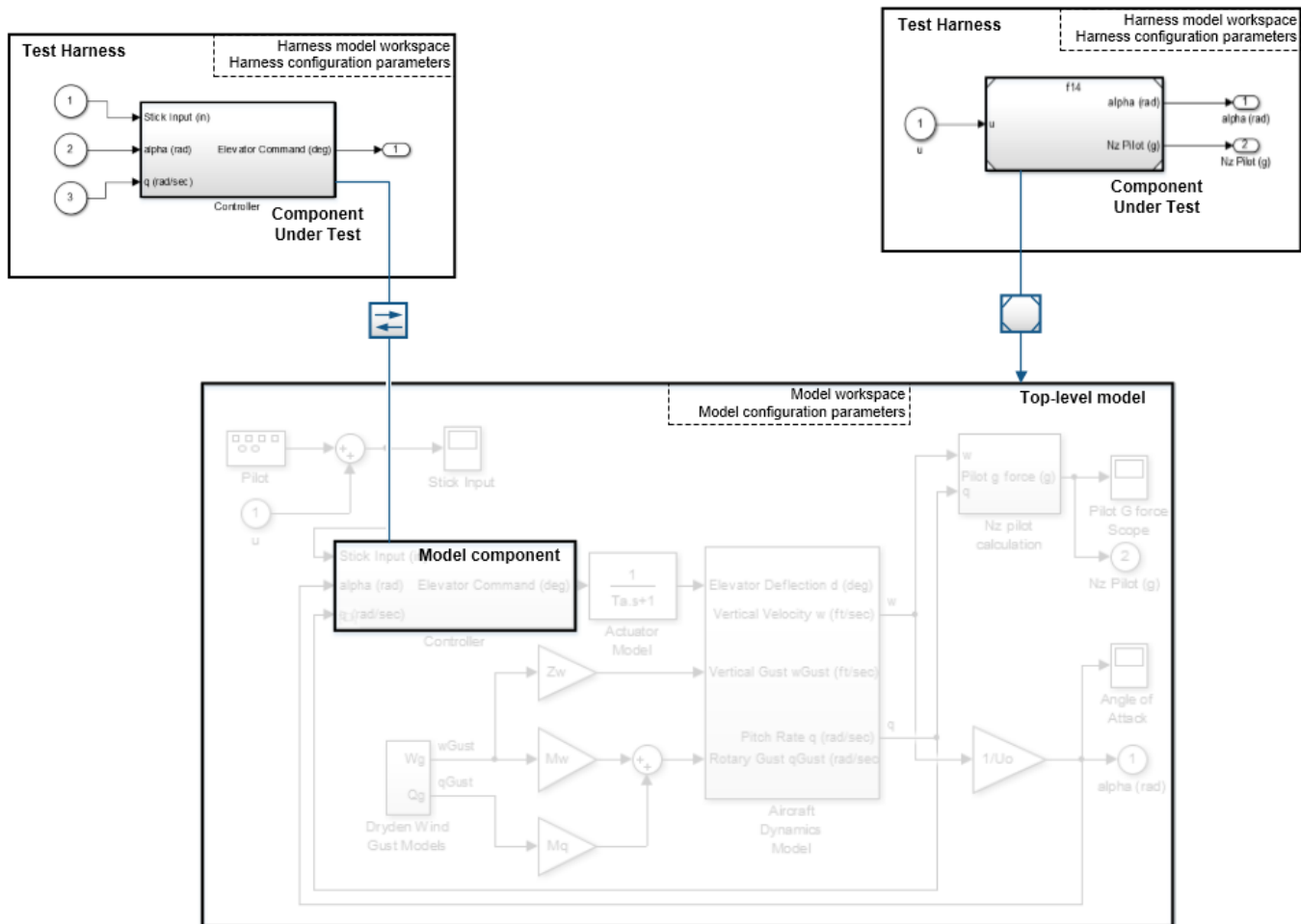
In this section...
“Harness-Model Relationship for a Model Component” on page 2-3
“Harness-Model Relationship for a Top-Level Model” on page 2-4
“Resolving Parameters” on page 2-5
“Test Harness Considerations” on page 2-5

A test harness is a model block diagram that you can use to test, edit, or debug a Simulink model. In the main model, you associate a harness with a model component or the top-level model. The test harness contains a separate model workspace and configuration set. The test harness is associated with the main model and can be accessed through the model canvas.

When you create an external harness, a metadata XML file is also created. The XML file contains the unique ID of the design model, which maintains the association between the model and its harness. The metadata file does not need to be in the same folder as the model, as long as they are both on the MATLAB® path.

You build the test harness model around the component under test, which links the harness to the main model. If you edit the component under test in the harness, the main model updates when you close the harness. You can generate a test harness for:

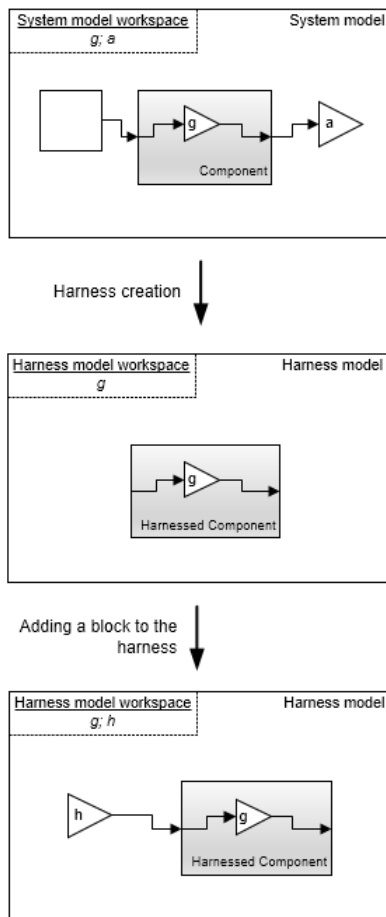
- A model component, such as a subsystem, library block, Subsystem Reference block, Model block, or System Composer™ component. The test harness isolates the component in a separate simulation environment. If you convert a Subsystem or Subsystem Reference block to a Model block, the test harnesses are transferred to the model reference (see “Test Harness Considerations” on page 2-5).
- A top-level model. The component under test is a Model block referencing the main model. You can also build a test harness in a subsystem model.



Harness-Model Relationship for a Model Component

When you associate a test harness with a model component, the harness model workspace contains copies of the parameters associated with the component. For example, suppose that you create a test harness for a component that contains a Gain block and then add a second Gain block to the harness.

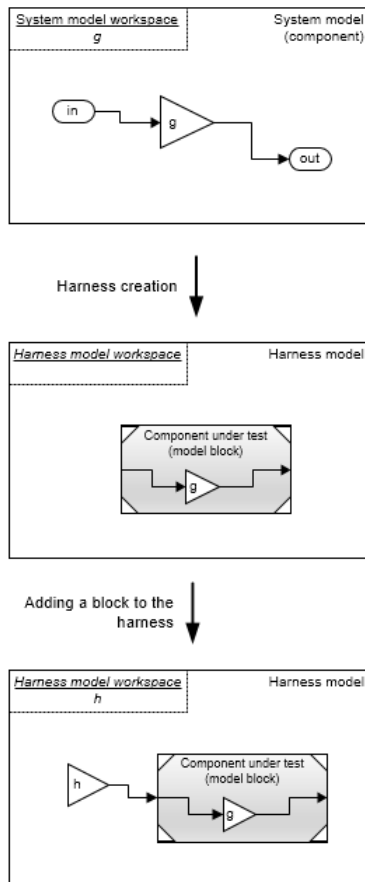
- The parameter g defines part of the component under test, so the harness model workspace contains a copy of g .
- The parameter a defines part of the main model outside of the component under test, so the harness model workspace does not contain a copy of a .
- The parameter h is the gain of the Gain block that you added to the harness. Because this block is outside the component under test, h exists only in the harness model workspace.



Harness-Model Relationship for a Top-Level Model

When you associate a harness with the top level of the main model, the harness model workspace does not contain copies of parameters relevant to the component. The component under test is a Model block that references the main model, so the parameters remain in the main model workspace. For example, suppose that you create a test harness for a top-level model that contains a Gain block and then add a second Gain block to the harness.

- The component under test references the main model, and the parameter g exists in the main model workspace. The harness model workspace does not contain a copy of g .
- The parameter h is the gain of the Gain block that you added to the harness. Because this block is outside the component under test, h exists only in the harness model workspace.



Resolving Parameters

Parameters in the test harness resolve to the most local workspace. Parameters resolve to the harness model workspace, then the system model workspace, then the base MATLAB workspace.

Test Harness Considerations

- You can build a test harness for these types of model components:
 - Model blocks
 - Subsystem Reference blocks
 - Subsystem blocks
 - Subsystem and Subsystem Reference block components
 - Reusable library subsystems that have function interfaces and are at the top-level of the library
 - Stateflow® blocks, including Stateflow charts, Truth Table blocks, State Transition Table blocks, and Test Sequence blocks
 - System Composer components
 - C Caller blocks
 - MATLAB Function blocks

- User-defined function blocks
- Test harnesses are not supported for these types of Stateflow objects:
 - Atomic subcharts
 - Simulink based states
 - Simulink functions
 - MATLAB functions
- The presence of test harnesses, whether saved internally or externally, has no impact on simulation or code generation for a model.
- Multiple harnesses owned by a Simulink model can be open at the same time. When multiple harnesses are open, you cannot synchronize changes from the harness to the model using push synchronization.
- You cannot delete a test harness unless all test harnesses are closed.
- Do not comment out the component under test in the test harness. Commenting out the component under test might cause unexpected behavior.
- Model and test harness locking is specific to each type of synchronization. For information on synchronization, see “Synchronization Mode” on page 2-19.
- The signal names used in the component under test propagate from the model to the test harness. For subsystem harnesses, some propagated signal names might be visible only after you compile the harness. For block diagram harnesses, signal names are propagated even if you do not select **Show propagated signals** in the Signal Properties dialog box.
- Subsystem and Subsystem Reference blocks and components within those blocks
 - Test harnesses attached to Subsystem models:
 - Always synchronize with the underlying model
 - Are created without compiling
 - Do not support post-build callbacks
 - Do not auto shape inputs
 - If a subsystem has a test harness, you cannot expand the subsystem contents into the model that contains the subsystem. Delete the test harness before expanding the subsystem. For more information, see “Expand Subsystem Contents”.
 - When you convert a Subsystem or Subsystem Reference block to a Model block, the test harnesses are transferred to the model reference. Harnesses on the Subsystem block are converted to block diagram harnesses. Nested harnesses within the subsystem are copied to identical blocks in the model reference. All transferred harnesses are internal harnesses in the model reference. Test harnesses might be renamed when they are transferred. You can see feedback about the harness transfer in the **Complete Conversion** pane of the Conversion Advisor or at the MATLAB Command line.

These limitations apply to converting a Subsystem or Subsystem Reference block to a Model block:

- SIL and PIL harnesses are not transferred.
- Requirements in a test harness for a subsystem are not transferred. You must copy them manually.

- If your test harness contains a To Workspace block, the block variable is not saved in the base workspace after the test finishes running. Upon test completion, the base workspace is restored to its original state.
- The Upgrade Advisor and XML differencing are not supported for test harness models.
- A test harness with a Signal Editor block source does not support:
 - Frame-based signals
 - Variable-dimension signals
- For a test harness with a Test Sequence block or Stateflow chart as the source, all inputs to the component under test must operate with the same sample time.
- These considerations apply to collecting coverage in a test harness:
 - Loading coverage results to a model, or aggregating coverage results across models, requires a model consistent with the coverage results. Therefore, to perform aggregated coverage collection, use test harnesses configured to automatically synchronize the component under test. Set **Synchronization Mode** to Synchronize on harness open and close. For more information, see “Synchronize Changes Between Test Harness and Model” on page 2-52.
 - If the test harness is configured to synchronize the component under test when you open or close the harness, coverage results from the test harness are associated with the main model. When you close the test harness, the coverage results remain active in memory. You can aggregate coverage with additional results collected from the main model or another synchronized test harness.
 - If the test harness is configured to only synchronize the component under test when you manually push or rebuild, the coverage results are associated with the test harness.
 - When you close the test harness, the coverage results are removed from memory.
 - If the component under test design differs between test harness and main model, you cannot aggregate coverage results.
 - You can aggregate coverage results with the main model if the component under test design does not differ, but you must manually load the coverage results into the main model. See the function cvload (Simulink Coverage).

For information on coverage, see “Collect Coverage in Tests” on page 6-151

See Also

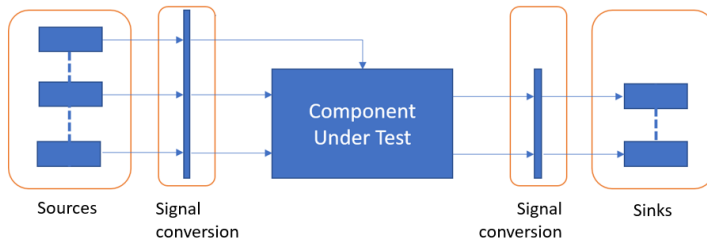
“Create a Test Harness” | Model Reference Conversion Advisor

More About

- “Compare Capabilities of Model Components”

Test Harness Construction for Specific Model Elements

A test harness consists of one or more source blocks that drive the component under test, which drives one or more sink blocks. Test harness construction configures signal attributes, function calls, data stores, and execution semantics. When possible, the test harness matches signal attributes at the sources, sinks, and component interface. For more information on selecting sources and sinks, see “Sources and Sinks” on page 2-16.



Signal Conversion

Signal conversion subsystems adapt the signal interface of the source and sink blocks to the graphical interface of the component. The graphical interface of the component includes input signals, output signals, and action, trigger, or enable inputs. The test harness compiles the main model to determine signal attributes:

- Data type
- Dimensions
- Complexity

Signal attributes are adapted to the sources during harness construction in one of two ways:

- Source blocks that can generate signals with the compiled attributes are configured to do so.
- If a source block cannot generate signals with the compiled attributes, signal attribute blocks in the signal conversion subsystem adapt the output of the source blocks. Signal attribute blocks include Reshape, Rate Transition and Data Type Conversion blocks.

By default, signal conversion subsystems are locked from editing.

Function Calls

Function Call Drivers

If the component under test has function call inputs, a Test Sequence block, MATLAB Function block, or Stateflow chart source generates function call inputs to the component, even if you select a different source during harness creation. To override this behavior and connect function call inputs to your selected source type, create the test harness with the `sltest.harness.create` function, and set 'DriveFcnCallWithTestSequence' to false. For example:

```
sltest.harness.create('Model/FcnCallSubsystem', 'Source', 'From File', ...
'DriveFcnCallWithTestSequence', false)
```

Function Call Outputs

Function call outputs of the component under test connect to Terminator blocks.

Physical Signal Connections

Components that accept or output physical signals are supported during harness construction, but sources and sinks are not generated. You can add physical modeling blocks to the test harness after construction.

Bus Signals

Test harness configuration for bus inputs and outputs depends on the bus connection ability of the source or sink blocks:

- Sources and sinks that can accept a bus signal are directly connected to the component without modification.
- If a source cannot output a bus signal, bus signals are automatically constructed from individual bus elements in the signal conversion subsystem.
- If a sink cannot accept a bus signal, bus signal elements are expanded from the bus signal in the signal conversion subsystem. For a top-level model, subsystem, or model reference that receives or outputs a bus of messages, if you select a Scope block as the sink, the test harness adds a terminator instead of the Scope block because Scope blocks do not support multiple messages.

String Signals

If the component under test uses string data inputs, and your test harness source does not support string data, string inputs are connected to Ground blocks.

String Inputs

Harness Source Selection	Source Block for String Inputs
Inport	Inport
Signal Editor	Ground
From Workspace	Ground
From File	Ground
Test Sequence	Ground
Playback	Ground
Chart	Ground
Constant	String Constant (individual string input) Ground (bus containing string)
Ground	Ground

If the component under test uses string data outputs, and your test harness sink does not support string data, string outputs are connected to Terminator blocks.

String Outputs

Harness Sink Selection	Sink Block for String Outputs
Output	Output
Scope	Terminator
To Workspace	Terminator
To File	Terminator
Terminator	Terminator

Non-Graphical Connections

In addition to the graphical interface of a component, Simulink supports several non-graphical connections. Test harness construction also supports non-graphical connections.

Goto-From Connections

Goto-From block pairs that cross the component boundary are considered component inputs or outputs.

- A From block without a corresponding Goto block in the component is considered a component input signal. The test harness includes a source block with a corresponding Goto block.
- A Goto block without the corresponding From block in the component is considered a component output signal. The test harness includes a sink block with a corresponding From block.

Data Store Memory

Data Store Read and Data Store Write blocks require a complete data store definition in the test harness.

- If a Data Store Read or Data Store Write block lacks a corresponding Data Store Memory block in the component, the test harness adds a Data Store Memory block.
- For a component containing only Data Store Read blocks, the test harness adds a source block driving a Data Store Write block.
- For a component containing only Data Store Write blocks, the test harness adds a Data Store Read block driving a sink block.
- These blocks inside a referenced model can access data stored in a Data Store Memory block defined at a higher level in the model hierarchy: Data Store Read, Data Store Write, S-Function, MATLAB Function, MATLAB System, and Chart blocks. To allow these blocks to access the data stored in the Data Store Memory block at a higher level in the model hierarchy:
 - Place a Data Store Memory block inside the referenced model. For more information about supported locations of the Data Store Memory block, see **Description** in Data Store Memory.
 - In the Data Store Memory block dialog box, select **Data store reference**.
 - On the **Signal Attributes** tab, specify the **Data type**, **Dimensions** and **Signal type**. When you select **Data store reference**, these options are not available: Inherit for **Data type**, -1 for **Dimensions** and auto for **Signal type**.

If global data store memory read or write usage cannot be determined, then Data Store Read and Data Store Write blocks are not included in the test harness.

Simulink Function Definitions

If the component calls a Simulink Function that is not defined in the component, the test harness adds a stub Simulink Function block matching the function call signature.

Export Function Models

Test harnesses contain a function-call scheduler for components that use the export-function modeling style. The scheduler is a Test Sequence block, MATLAB Function block, or Stateflow chart that contains prototype calls to the functions in your model.

The scheduler Test Sequence block includes a test step containing:

- A catalog of globally scoped Simulink Function blocks in the component.
- A list of function-call triggers accessible at the component interface.

Harness construction honors periodic function-call triggers with appropriate decimation of the function-call event in the Test Sequence block, MATLAB Function block, or Stateflow chart.

Test harnesses include `Initialize`, `Terminate`, and `Reset` steps for models that contain `Initialize`, `Terminate`, and `Reset` event subsystems. You can include `Initialize`, `Terminate`, and `Reset` steps for other export-function models using the `'ScheduleInitTermReset'` property of `sltest.harness.create`.

Execution Semantics

The execution behavior of a component depends on factors such as computed sample times, solver settings, model configuration, and parameter settings. Execution behavior also depends on run-time events such as function-call triggers and asynchronous events. To handle these execution semantics, test harness construction:

- Copies configuration parameter settings from the main model into the test harness.
- Copies required parameter definitions from the main model workspace into the test harness model workspace.
- Copies data dictionary settings from the main model into the test harness if the harness owner is not a Subsystem Reference block, or if the harness owner is a Subsystem Reference block that does not have an attached data dictionary.
- Honors a limited subset of sample time settings using explicit source block specifications and Rate Transition blocks.

Other factors, such as additional blocks in the harness and solver heuristics, can cause test harness execution to differ from the main model. The graphical and compiled interface of the component takes precedence over other execution semantics.

Sample Time Specification

Simulink supports an array of sample times, including types that are derived during model compilation. Test harness construction supports periodic discrete, continuous, and fixed-in-minor-step sample times with these considerations:

- Source blocks that support the desired rate are configured to do so, and the signal conversion subsystem contains a Signal Specification block with the rate specification.

- Test harness construction does not configure source blocks that cannot support the desired rate.
 - If the desired rate is periodic discrete or fixed-in-minor-step, the test harness contains a Rate Transition block in the signal conversion subsystem.
 - If the desired rate is continuous, the execution semantics are determined by the solver. The signal conversion subsystem does not contain a Rate Transition block.

Other sample time specifications are ignored during test harness construction. In those cases, solver settings determine execution behavior.

See Also

“Create or Import Test Harnesses and Select Properties” on page 2-13

Create or Import Test Harnesses and Select Properties

In this section...

“Create a Test Harness For a Top Level Model” on page 2-13
 “Create Test Harnesses for Model Components” on page 2-13
 “Import a Test Harness” on page 2-14
 “Preview and Open Test Harnesses” on page 2-14
 “Change Test Harness Properties” on page 2-15
 “Considerations for Selecting Test Harness Properties” on page 2-15
 “Create Test Harness Dialog Box Properties” on page 2-15
 “Import Test Harness Dialog Box Properties” on page 2-19
 “Customize Test Harness Creation Default Property Values” on page 2-20

Test harnesses are separate workspaces you can use to isolate tests from your design model. You can create test harnesses for your whole model or for one or more components in your model. You can also import a test harness from a standalone test model.

Create a Test Harness For a Top Level Model

To create a test harness for a top-level model (including subsystem and model reference models):

- 1 Right-click in the Simulink model and click **Test Harness > Create for Model** to open the Create Test Harness dialog box.
- 2 After selecting the desired options, click **OK** to create the test harness.

For information on the Create Test Harness dialog box properties, see “Create Test Harness Dialog Box Properties” on page 2-15.

Create Test Harnesses for Model Components

To create a test harness for one or multiple model components:

- 1 On the **Apps** tab, under Model Verification, Validation, and Test, click **Simulink Test**.
- 2 On the **Tests** tab, click **Simulink Test Manager** to open the Test Manager.
- 3 Create a new test file in the Test Manager.
- 4 Click **New > Test for Model Component**, which opens the Create Test for Model Component workflow wizard.
- 5 On the first page of the wizard, specify the model and select the component or components. If you select multiple components, the wizard creates a test harness for each component.
- 6 Create the test harness or harnesses by completing the wizard pages.

Note The Create Test for Model Component workflow wizard exposes a subset of test harness options. If your test harness does not need to use non-default options, use the wizard to create a harness quickly. If you need to change other options, use the Test Manager for the test harness you

created with the wizard. For information on using the wizard and the properties it sets, see “Generate Tests and Test Harnesses for a Model or Components” on page 6-20.

Import a Test Harness

To import a test harness from a standalone test model:

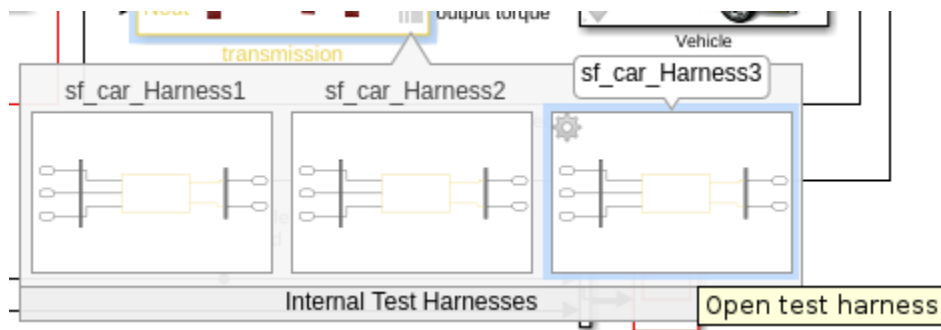
- 1 Right-click in the Simulink model or on a model component and click **Test Harness > Import for Model** or **Test Harness > Import for Component**, respectively, to open the Import Test Harness dialog box.
- 2 After selecting the desired options, click **OK** to import the test harness.

Note Importing test harnesses for Software Architecture components that have client-server ports is not supported.

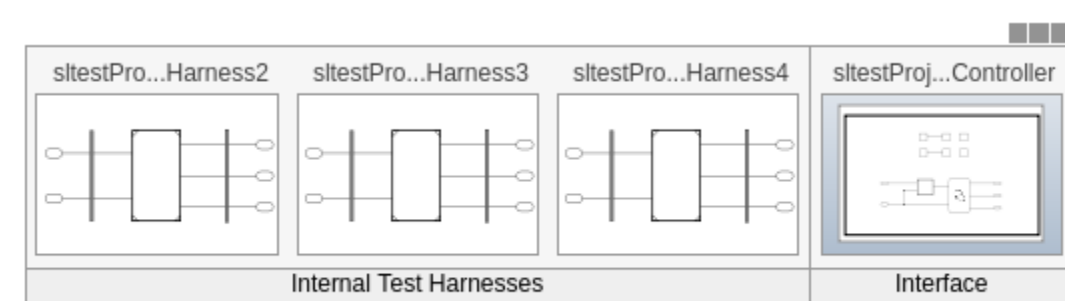
For information on the Import Test Harness dialog box properties, see “Import Test Harness Dialog Box Properties” on page 2-19.

Preview and Open Test Harnesses

When a model component has a test harness, a badge appears in the lower right of the block. To view the test harnesses, click the badge. To open a test harness, click a tile.



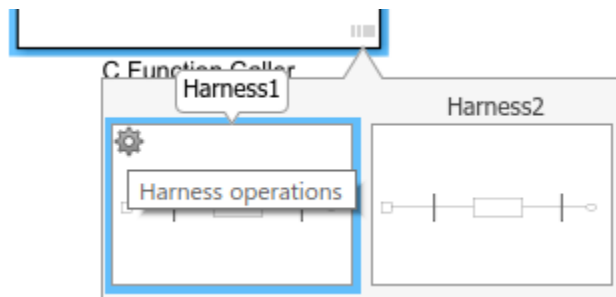
To view test harnesses for a model block diagram, click the pullout icon in the model canvas. To open a test harness, click a tile.



Change Test Harness Properties

To change properties of an open test harness, click the badge  in the test harness block diagram and click **Test harness properties** to open the harness properties dialog box.

To change properties of test harnesses from the main model, click the **Harness operations** icon from the test harness preview.



Considerations for Selecting Test Harness Properties

Before selecting test harness properties, consider the following:

- What data source you want to use for your test case input
- How you want to view or store test output
- Whether you want to copy parameters and workspaces from the main model to the harness
- Whether you plan to edit the component under test
- How you want to synchronize changes between the test harness and model

Except for sources and sinks, you can change harness properties later using the harness properties dialog box. To change sources and sinks after harness creation, manually remove the blocks from the test harness and replace them with new sources and sinks.

Note The following sections describe the test harness properties in the Create Test Harness dialog box. For information on the test harness properties in the Create Test for Model Component wizard, see “Generate Tests and Test Harnesses for a Model or Components” on page 6-20.

Create Test Harness Dialog Box Properties

Harness Name

Test harnesses must use valid MATLAB file names.

Save test harnesses externally

Use this check box to select whether the model stores test harnesses internally or externally:

- When cleared, the model saves test harnesses internally, as part of the model SLX or MDL file.

- When selected, the model saves test harnesses externally, in separate SLX files in the current working folder, and adds a harness information XML file to the model folder. The harness information file can be in any location on the MATLAB path.

If a model already has test harnesses, the dialog box does not display the check box. Instead the dialog box displays the message **Harnesses saved internally** or **Harnesses saved externally**, which indicates how previously created test harnesses are stored. All test harnesses for a model are stored either internally or externally so when you create an additional harness, it is stored in the same way as previous harnesses.

See “Manage Test Harnesses” on page 2-29.

Select Function Interface

Select the function interface to associate with the reusable library subsystem test harness. This option appears only if the component under test is a reusable library subsystem with a function interface.

Sources and Sinks

In the Create Test Harness dialog box, under **Sources and Sinks**, select the source and sink from the respective menus. The menus provide common sources and sinks.

In addition to the options in the dropdown menus, you can use source and sink blocks from the Simulink Sources or Sinks library. Select Custom source or sink, and enter the path to the block. For example:

`simulink/Sources/Sine Wave`

`simulink/Sinks/Terminator`

Custom sources and sinks build the test harness with one block per port.

Create scalar inputs

When you select this property, the test harness creates scalar inputs for multidimensional signals. The individual scalar inputs are reshaped to match the dimension of the input signals to the component under test. Scalar expansion is performed only when the harness owner model can be compiled before harness generation. Libraries, subsystem models, and harnesses created with the **Create without compiling the model** option set do not compile first so **Create scalar inputs** does not apply to them. **Create scalar inputs** applies to test harnesses with Inport, Constant, Signal Editor, From Workspace, or From File source blocks.

Add scheduler for function-calls and rates / Generate function-call signals using

The title of this option depends on whether the component under test is a subsystem or a model. Use a scheduler to control the number of times and the order in which blocks or subsystems are executed. To include a scheduler block in your test harness, select a block from the drop-down list. You can use a Test Sequence block, a MATLAB Function block, or a Stateflow chart as the scheduler.

- **Add scheduler for function-calls and rates:** For a model, you can use the block to call functions and set sample times for model inputs and outputs.

- **Generate function-call signals using:** For a subsystem, you can use the block to call functions in the subsystem.

For information on schedulers, see “Rate Transitions” (Embedded Coder), “Asynchronous Rate Transitions” (Embedded Coder), or “Schedule Simulink Functions by Using Stateflow” (Stateflow)

Enable Initialize, Reset, and Terminate ports

Selecting this option exposes initialize, terminate, or reset function-call ports in the component under test and connects the scheduler block to the ports.

This option appears when you create a test harness for a top-level model and select a block for the **Add scheduler for function-calls and rates** option.

When running the test harness, if you encounter an error about executing a function call at an initialize, reset, or terminate port, use these commands to hide and disconnect the ports.

`Subsystem_name` is the system under test in the test harness.

```
set_param(<Subsystem_name>, 'ShowModelInitializePort', 'off');
set_param(<Subsystem_name>, 'ShowModelResetPorts', 'off');
set_param(<Subsystem_name>, 'ShowModelTerminatePort', 'off');
```

Add Separate Assessment Block

Select **Add separate assessment block** to include a separate Test Assessment block in the test harness.

A Test Assessment block is a separate Test Sequence block configured with properties commonly used for verifying the component under test. For more information, see “Assess Simulation and Compare Output Data” on page 3-11 and “Verify Model Simulation by Using when Decomposition” on page 3-14.

Log Output Signals

Select **Log output signals** to log all output signals of the component under test. You can use this option only when creating a new harness. Signals are logged during test case execution and return test results. If an output signal does not have a name or a propagated name, it is assigned one in the harness using the format `<component under test name>:<output port number>`. To remove a signal from being logged, open the harness, right-click the signal, and select **Stop Logging Selected Signals**.

Open Harness After Creation

Clear **Open Harness After Creation** to create the test harness without opening it. This can be useful creating multiple test harnesses in succession.

Create without compiling the model

Creating a test harness without compiling the model can be useful if you are prototyping a design that cannot yet compile. When you create a test harness without compiling the main model:

- Parameters are not copied to the test harness workspace.
- The main model configuration is not copied to the test harness.
- The test harness does not contain conversion subsystems.

You may need to add blocks such as signal conversion blocks to the test harness. You can rebuild the harness when you are ready to compile the main model. For more information, see “Synchronize Changes Between Test Harness and Model” on page 2-52.

Test harnesses for Subsystem models are created without compiling the model.

Verification Modes

The test harness verification mode determines the type of block generated in the test harness.

- **Normal**: A Simulink block diagram.
- **Software-in-the-Loop (SIL)**: The component under test references generated code, operating as software-in-the-loop. Requires Embedded Coder®.
- **Processor-in-the-Loop (PIL)**: The component under test references generated code for a specific processor instruction set, operating as processor-in-the-loop. Requires Embedded Coder.

Subsystem model test harnesses do not support SIL or PIL verification.

Note Keep the SIL or PIL code in the test harness synchronized with the latest component design. If you select SIL or PIL verification mode without selecting **Rebuild harness on open**, your SIL or PIL block code might not reflect recent updates to the main model design. To regenerate code for the SIL or PIL block in the test harness, select **Rebuild Harness > Update Harness Configuration Settings and Model Workspace**.

Use generated code to create SIL/PIL block

If generated code for the SIL/PIL block already exists, select this property to use that existing code instead of regenerating the code. This option is available only for subsystem harnesses. It does not apply to Subsystem model test harnesses.

Build folder

Specify the folder that contains the generated code for the SIL/PIL block. This option is available only if you selected **Use generated code to create SIL/PIL block**.

Post-create callback method

You can customize your test harness using one or more post-create callbacks. A post-create callback is a function that runs after the harness is created. For example, your callback can set up signal logging, add custom blocks, or change the harness simulation times. If you specify more than one callback, separate them using commas. The callbacks run in the order that they are listed. For more information, see “Customize Test Harnesses” on page 2-40. This option does not apply to Subsystem model test harnesses.

Rebuild harness on open

When you select this property, the test harness rebuilds every time you open it. If you specified to use existing generated code for a SIL/PIL subsystem using `sltest.harness.create` or `sltest.harness.set`, the harness rebuild uses that code instead of regenerating it. For details on the rebuild process, see “Synchronize Changes Between Test Harness and Model” on page 2-52. This option does not apply to Subsystem model test harnesses.

Update Configuration Parameters and Model Workspace data on rebuild

When you select this property, configuration parameters and model workspace data update when you rebuild the harness. For details on the rebuild process, see “Synchronize Changes Between Test Harness and Model” on page 2-52. This option does not apply to Subsystem model test harnesses.

Post-rebuild callback method

You can customize your test harness using a post-rebuild callback. A post-rebuild callback is a function that runs after the harness is rebuilt. For example, your callback can set up signal logging, add custom blocks, or change the harness simulation times. For more information, see “Customize Test Harnesses” on page 2-40. This option does not apply to Subsystem model test harnesses.

Synchronization Mode

Synchronization mode controls when changes to the component under test are synced to the main model, and when changes to the harness owner are synced into a test harness. The synchronization mode also affects model and harness locking. For information and limitations, see “Synchronize Changes Between Test Harness and Model” on page 2-52,.

Import Test Harness Dialog Box Properties

Name

Name to use for the imported test harness. Test harnesses must use valid MATLAB file names.

Save Test Harnesses Externally

This option controls how the model stores test harnesses. A model stores all its test harnesses either internally or externally.

If a model already has test harnesses, the dialog box does not display the check box. The dialog box displays either **Harnesses saved internally** or **Harnesses saved externally**, which indicates how previously created test harnesses are stored. Since all test harnesses for a model are stored either internally or externally, when you create an additional harness, it is stored in the same way as previous harnesses.

- When cleared, the model saves test harnesses as part of the model SLX file.
- When selected, the model saves test harnesses in separate SLX files in the current working folder, and adds a harness information XML file to the model folder. The harness information file can be in any location that is on the MATLAB path.

See “Manage Test Harnesses” on page 2-29.

Simulink model to import

Name of or full path to the Simulink standalone test model to import.

Component under Test in imported model

Path to the component under test in the imported Simulink standalone test model.

Rebuild harness on open

When you select this property, the test harness rebuilds every time you open it. If you specified to use existing generated code for a SIL/PIL subsystem using `sltest.harness.create` or `sltest.harness.set`, the harness rebuild uses that code instead of regenerating it. For details on the rebuild process, see “Synchronize Changes Between Test Harness and Model” on page 2-52. This option does not apply to Subsystem model test harnesses.

Update Configuration Parameters and Model Workspace data on rebuild

When you select this property, configuration parameters and model workspace data update when you rebuild the harness. For details on the rebuild process, see “Synchronize Changes Between Test Harness and Model” on page 2-52. This option does not apply to Subsystem model test harnesses.

Customize Test Harness Creation Default Property Values

To set default property values for the creation of new test harnesses, use an `sl_customization` file or the `setHarnessCreateDefaults` function. All newly created test harnesses use the new default values.

To see the current default test harness property values, use `sltest.harness.getHarnessCreateDefaults`.

For an individual test harness, you can change the property values from the default values by using the Create Test Harness dialog box or the `sltest.harness.create` function. Using either of these options does not change the default values used when creating a new test harness. See also “Create Test Harness Dialog Box Properties” on page 2-15 and “Change Test Harness Properties” on page 2-15.

Set Defaults by Using an `sl_customization.m` File

To change the default property values used when creating new test harnesses, you can create an `sl_customization.m` file.

- 1 Create an `sl_customization.m` file and specify the new default property values.

You can set all name-value arguments of `sltest.harness.create`, except where noted.

This sample `sl_customization.m` file sets the harness name to `myTestHarness`, sets a post-create callback to use the `addHarnessAnnotation` function, saves the harness internally, and logs outputs:

```
function sl_customization(cm)
    % Create the struct with the harness options
    myStruct.Name="myTestHarness"
```

```

myStruct.PostCreateCallback = "addHarnessAnnotation";
myStruct.SaveExternally = false;
myStruct.LogOutputs = true;

% Invoke harness customization
cObj = cm.SimulinkTestCustomizer;
cObj.setHarnessCreateDefaults(myStruct);
end

```

- 2 Save the `sl_customization.m` file.
- 3 Add the file to the MATLAB path.
- 4 Register the new customizations by reloading Simulink or by using `sl_refresh_customizations`. For more information, see “Register Customizations with Simulink”.

Note When you register a file, its values become the default property values. All new test harnesses use the default property values and all previously registered values are cleared.

To view the customized default values, use `sltest.harness.getHarnessCreateDefaults`.

Set Defaults by Using the `setHarnessCreateDefaults` Function

You can also use `sltest.harness.setHarnessCreateDefaults` to set the default property values. You can set any of the name-value pair properties described in `sltest.harness.create`, except where noted. Using `sltest.harness.setHarnessCreateDefaults` saves and registers the default property values. However, if you already set and registered values using an `sl_customization.m` file, using `sltest.harness.setHarnessCreateDefaults` overwrites the values specified in the file.

See Also

`sltest.harness.setHarnessCreateDefaults` |
`sltest.harness.getHarnessCreateDefaults` | Test Sequence

Related Examples

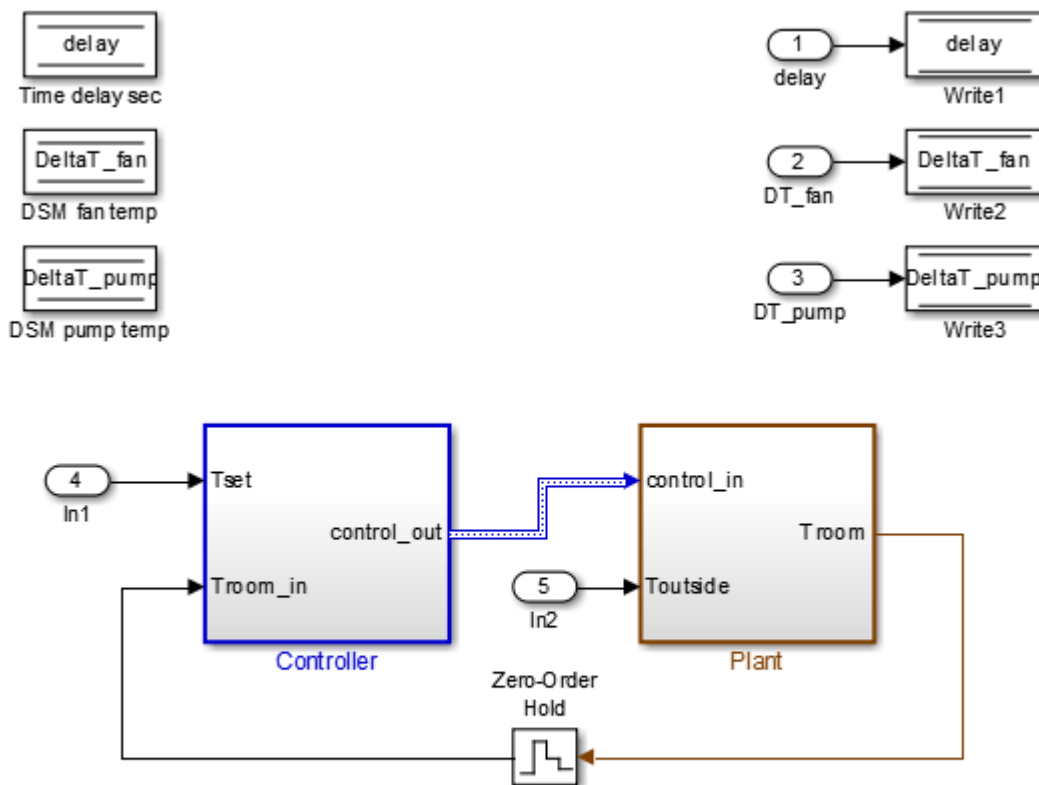
- “Test Harness and Model Relationship” on page 2-2
- “Synchronize Changes Between Test Harness and Model” on page 2-52
- “Register Customizations with Simulink”
- “Manage Test Harnesses” on page 2-29
- “Customize Test Harnesses” on page 2-40

Refine, Test, and Debug a Subsystem

This example shows how to refine and test a controller subsystem by using a test harness. Test harnesses provide a development and testing environment that leaves the main model design intact. You can test a functional unit of your model in isolation without altering the main model. In the example, the main model is a controller-plant model of an air conditioning/heat pump unit. The controller must operate according to several requirements.

Open the Model

sltestHeatpumpExample



Copyright 1990-2014 The MathWorks, Inc.

In the example model:

- The controller accepts the room temperature and specified temperature inputs.
- The controller output is a bus with signals that control the fan, heat pump, and the direction of the heat pump (heat or cool).
- The plant accepts the control bus. The heat pump and the fan signals are Boolean, and the heat pump direction is specified by +1 for cooling and -1 for heating.

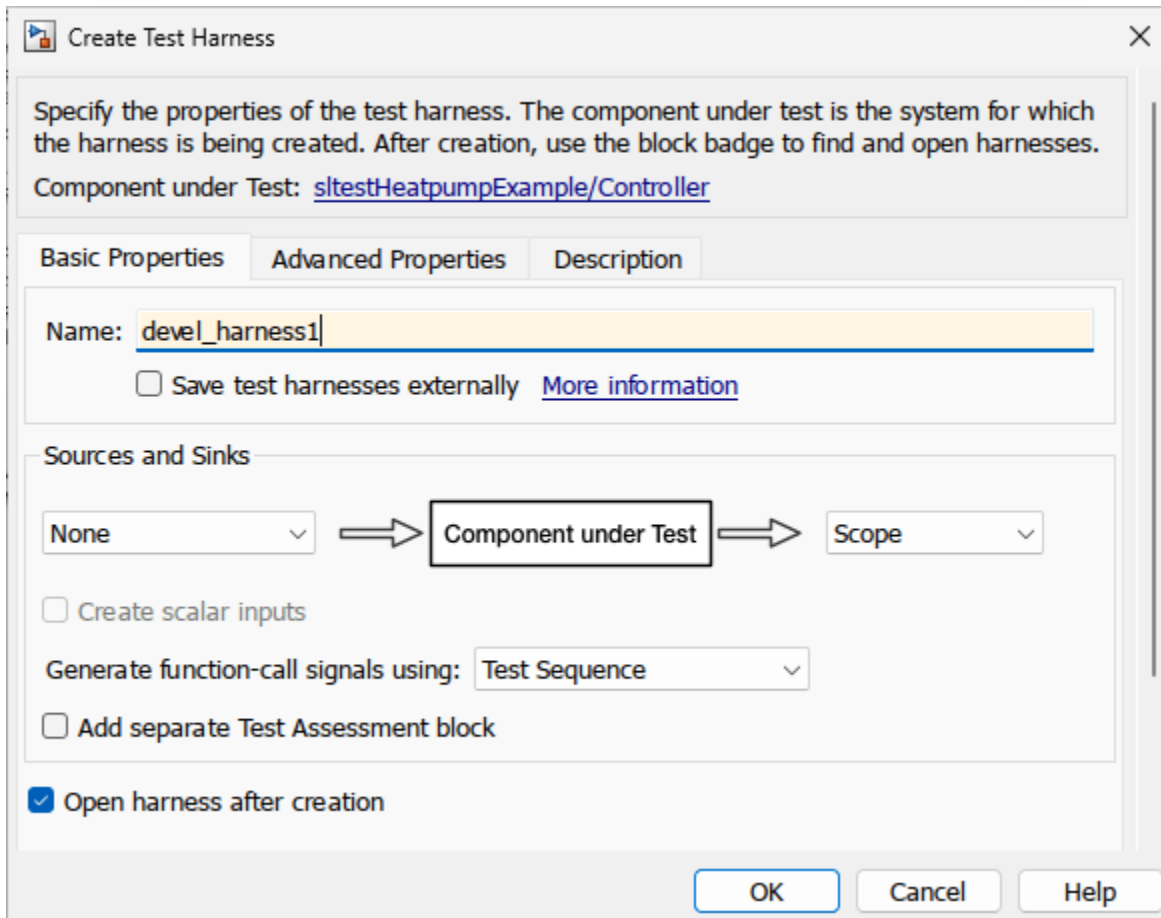
Temperature Condition States

The test covers four temperature conditions. Each condition corresponds to one operating state with fan, pump, and pump direction signal outputs.

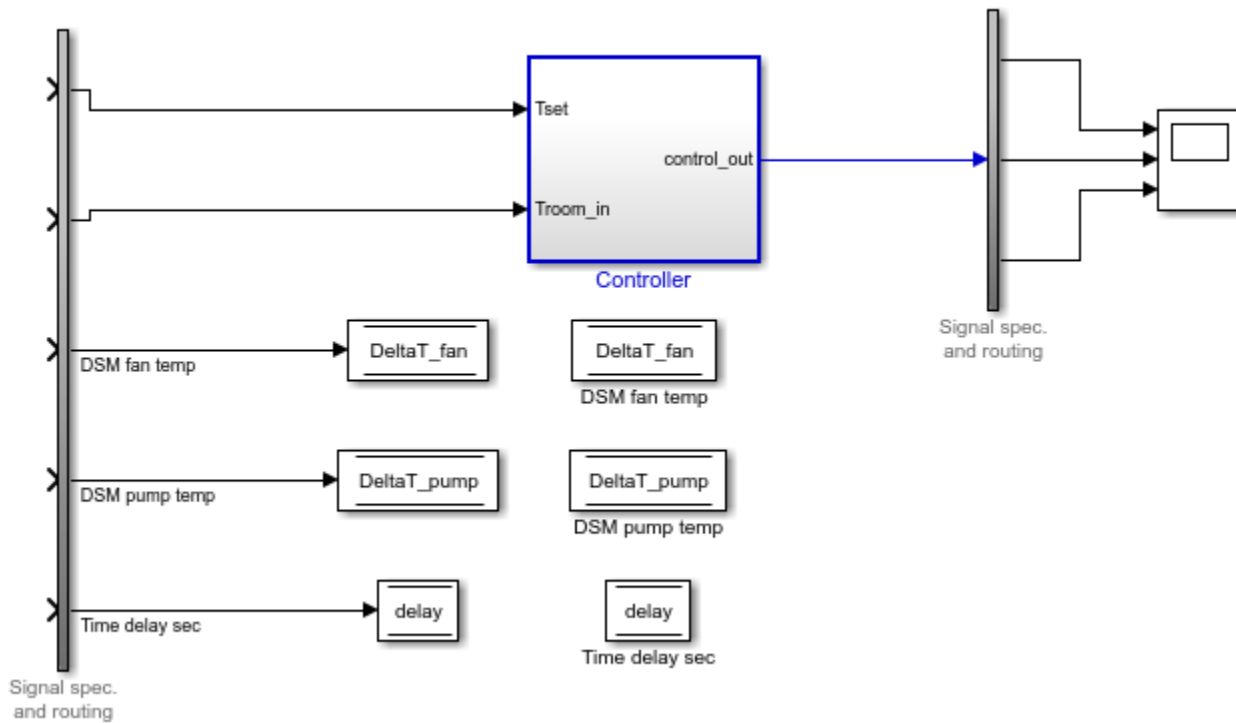
Temperature Condition	System State	Fan Command	Pump Command	Pump Direction
$ T_{room_in} - T_{set} < \Delta T_{fan}$	idle	0	0	0
$\Delta T_{fan} \leq T_{room_in} - T_{set} < \Delta T_{pump}$	fan only	1	0	0
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} < T_{room_in}$	cooling	1	1	-1
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} > T_{room_in}$	heating	1	1	1

Create a Harness for the Controller

1. Right-click the Controller subsystem and select **Test Harness > Create for 'Controller'**.
2. Set the harness properties in the **Basic Properties** tab:
 - **Name:** devel_harness1
 - Clear **Save test harness externally**
 - **Sources and Sinks:** None and Scope
 - Clear **Add separate assessment block**
 - Select **Open harness after creation**

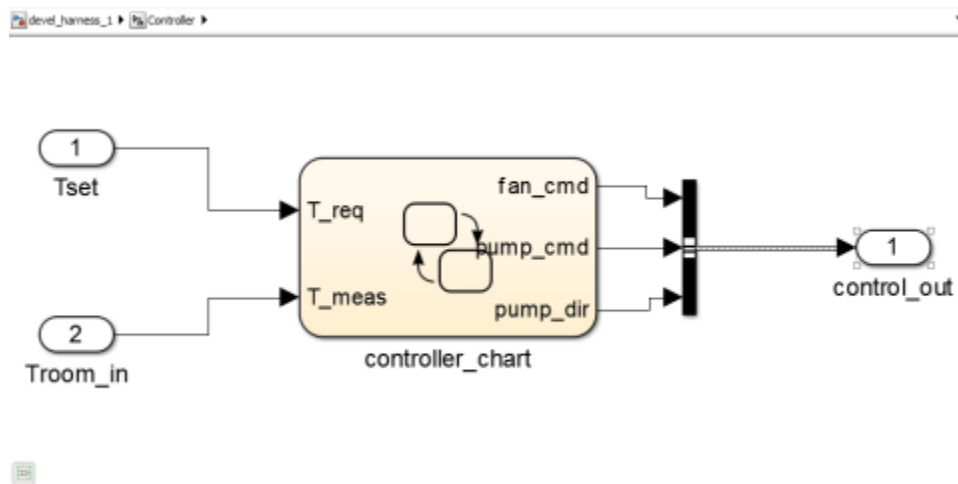


3. Click **OK** to create the harness.



Inspect and Refine the Controller

1. In the test harness, double-click Controller to open the subsystem.
2. Connect the chart to the Inport blocks.



3. In the test harness, click **Save** to save the test harness and model.

Add Test Inputs and Test the Controller

1. Navigate to the top level of devel_harness1.

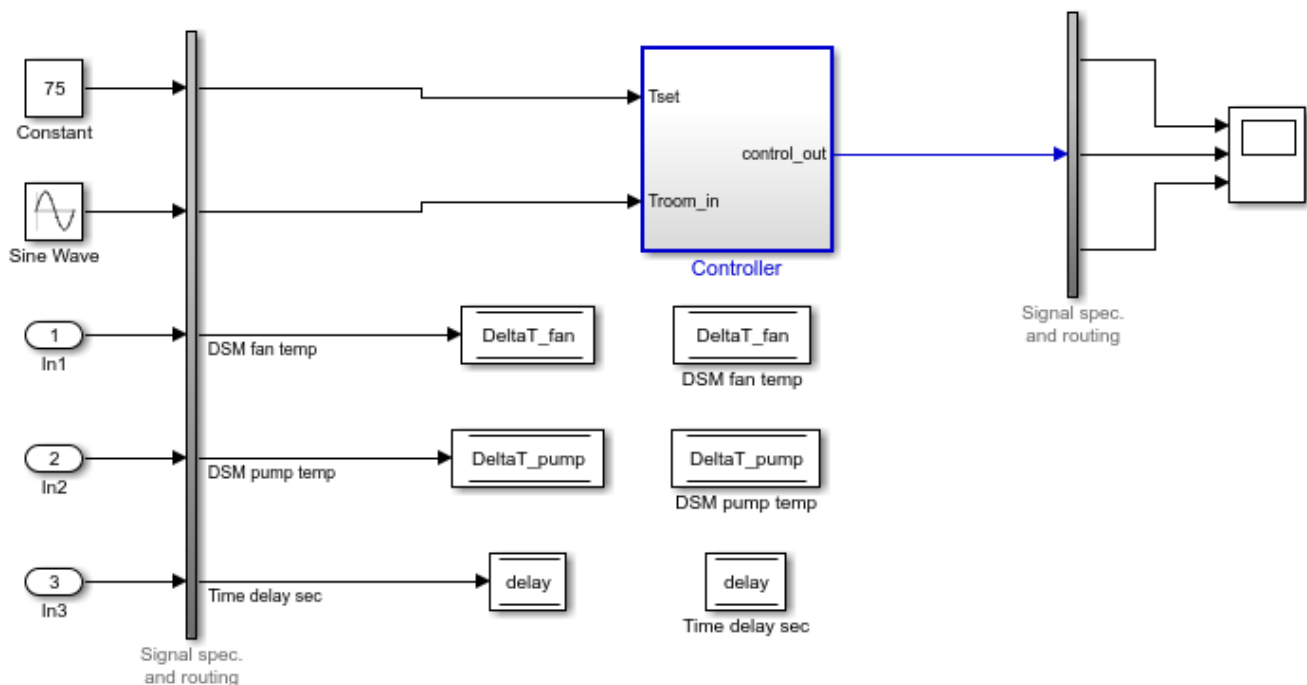
2. Create a test input for the harness with a constant **Tset** and a time-varying **Troom**. Connect a Constant block to the **Tset** input and set the value to 75.

3. Add a Sine Wave block to the harness model to simulate a temperature signal. Connect the Sine Wave block to the conversion subsystem input **Troom_in**.

4. Double-click the Sine Wave block and set the parameters:

- **Amplitude:** 15
- **Bias:** 75
- **Frequency:** $2\pi/3600$
- **Phase (rad):** 0
- **Sample time:** 1
- Select **Interpret vector parameters as 1-D**

5. Connect Inport blocks to the Data Store Write inputs.



6. Click **Model Settings** to open the Configuration Parameters dialog box, in the **Data Import/Export** pane, select **Input** and enter **u**. **u** is an existing structure in the MATLAB base workspace. Then, click **Apply**.

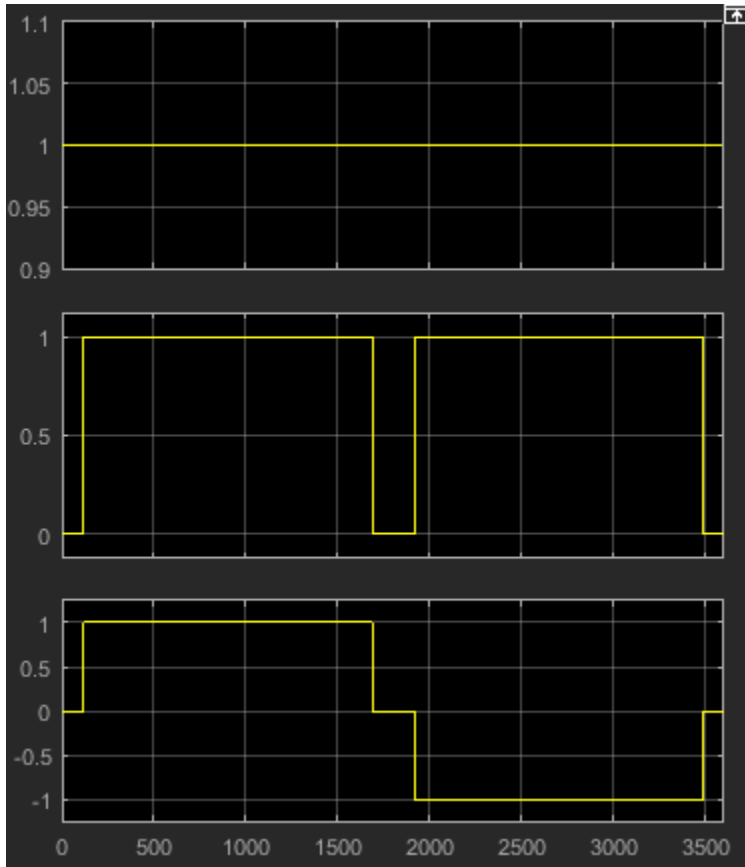
7. In the **Solver** pane, set **Stop time** to 3600. Click **OK**.

8. Open the Scope in the test harness and change the layout to show three plots.

9. Click **Run** to simulate.

Debug the Controller

1. Observe that the controller output, `fan_cmd`, is 1 during the IDLE condition where $|T_{room} - T_{set}| < \Delta T_{fan}$. This is a bug because `fan_cmd` should be 0 at IDLE. The `fan_cmd` control output must be changed for IDLE.



2. In the harness model, open the Controller subsystem.

3. Open `controller_chart`.

4. In the IDLE state, `fan_cmd` is set to return 1. Change `fan_cmd` to return 0. IDLE is now:

IDLE

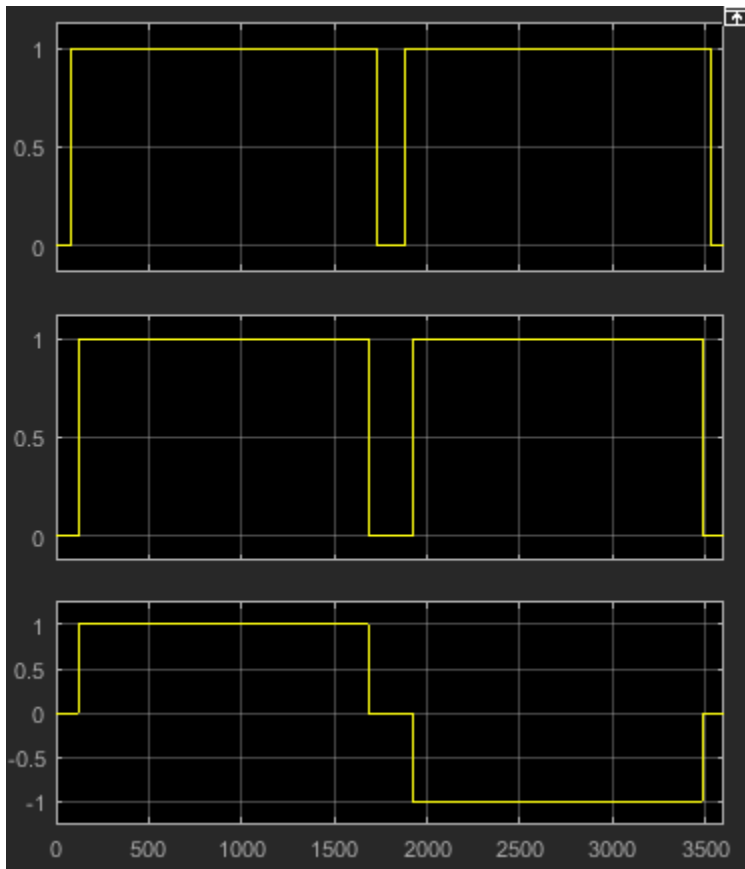
entry:

```
fan_cmd = 0;
```

```
pump_cmd = 0;
```

```
pump_dir = 0;
```

5. Simulate the harness model again and observe the outputs.



fan_cmd now meets the requirement to equal 0 at IDLE.

See Also

Related Examples

- “Create or Import Test Harnesses and Select Properties” on page 2-13
- “Set Up Test Inputs” on page 6-22

Manage Test Harnesses

In this section...

“Manage External Test Harnesses” on page 2-29
 “Convert Between Internal and External Test Harnesses” on page 2-30
 “Preview and Open Test Harnesses” on page 2-31
 “Model and Test Harness Locking” on page 2-32
 “Find Test Cases Associated with a Test Harness” on page 2-32
 “Export Test Harnesses to Standalone Models” on page 2-33
 “Move and Clone Test Harnesses” on page 2-33
 “Clone and Export a Test Harness to a Separate Model” on page 2-35
 “Delete Test Harnesses Programmatically” on page 2-37
 “Export Test Harness to Previous Version” on page 2-39

You can export, clone, move, and delete the test harnesses associated with your model, or change where the model stores the test harnesses. You can save test harnesses internally as part of the model SLX or MDL file or externally in separate SLX files. Because all test harnesses must be either internal or external, you cannot use both types of harness storage in one model.

If you store your model in a configuration management system, consider using external test harnesses. You can use external test harnesses to create or change a harness without changing the model file. If you plan to share your model often, consider using internal test harnesses. Creating or changing an internal test harness changes your model SLX or MDL file. Both internal and external test harnesses offer the same synchronization, push, rebuild, and badge interface functionality.

To obtain the name of the model that is the harness owner, use

```
get_param(<harness name or handle>,"OwnerBDName")
```

Manage External Test Harnesses

Models with external test harnesses use a separate SLX file for each harness, and an XML file with the name `<modelName>_harnessInfo.xml` to store the metadata that links the model and the harnesses. Changing test harnesses can change the `harnessInfo.xml` file. The metadata and the model are linked using a unique ID for the model. By default, the metadata XML file is created in the same folder as the model.

Follow these guidelines for external test harnesses:

Warning Do not delete or make any manual changes to the `harnessInfo.xml` file. Deleting the `harnessInfo.xml` file might sever the relationship between the model and harnesses, which cannot be regenerated from the model.

- The `harnessInfo.xml` file must be writable to save changes to the test harness or the main model.
- Folders containing test harness SLX files must be on the MATLAB path.

- If the test harness `harnessInfo.xml` file is not in the same folder as the model, the XML file or its folder must be on the MATLAB path.
- If you convert internal test harnesses to external test harnesses, the new SLX files are saved in the current working folder.
- If you convert external test harnesses to internal test harnesses, the external SLX files can be anywhere on the MATLAB path.
- If your model uses external test harnesses, only create a copy of your model using **Save > Save as**. Using **Save as** copies external test harnesses to the destination folder of the new model, renames the harnesses, and keeps the harness information current.

Copying the model file on disk will not copy external harnesses associated with the model.

- Only change or delete test harnesses using the Simulink UI or commands:
 - To delete test harnesses, use the thumbnail UI or the `sltest.harness.delete` command.
 - To rename test harnesses, use the harness properties UI or the `sltest.harness.set` command.
 - To make a copy of an externally saved test harness, use the `sltest.harness.clone` command or save the test harness to a new name using **Save > Save as**.

Deleting or renaming harness files outside of Simulink causes an inaccurate `harnessInfo.xml` file and problems loading test harnesses.

Convert Between Internal and External Test Harnesses

You can change how your model stores test harnesses at different phases of your model lifecycle. For example:

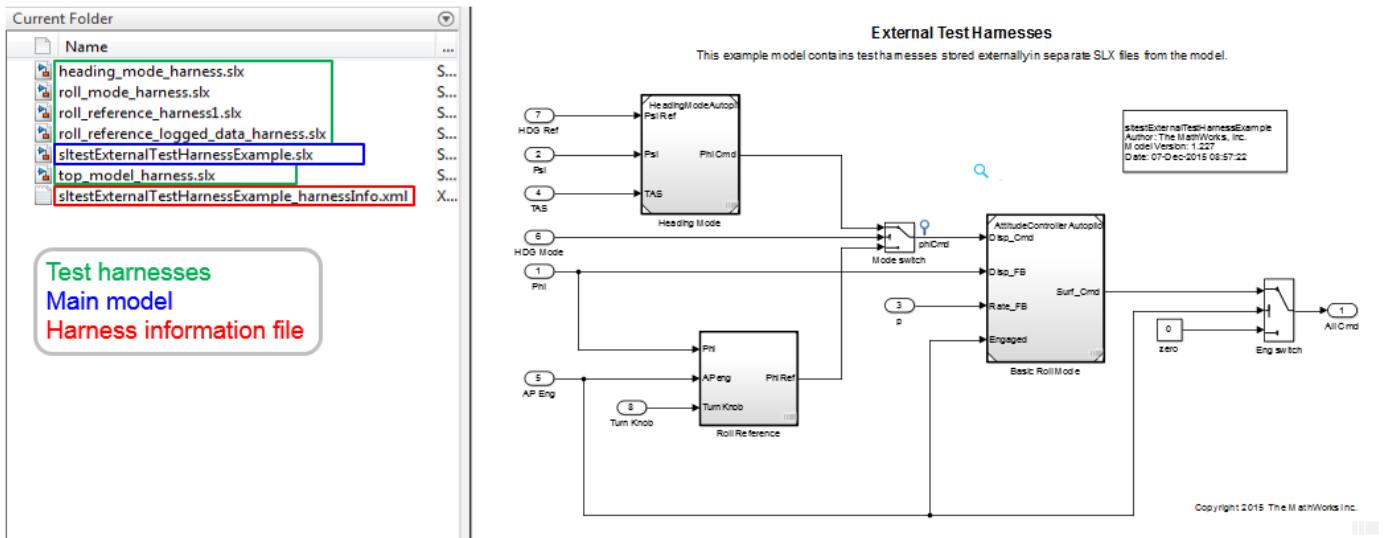
- Develop your model using internal test harnesses so that you can more easily share the model for review. When you complete your design and place the model under change control, convert to external harnesses.
- Use the configuration management model as the starting point for a new design. Test the existing model with external harnesses to avoid modifying it. Then, create a copy of the existing model. Convert to internal harnesses for the new development phase.

To change the test harness storage to external (or internal):

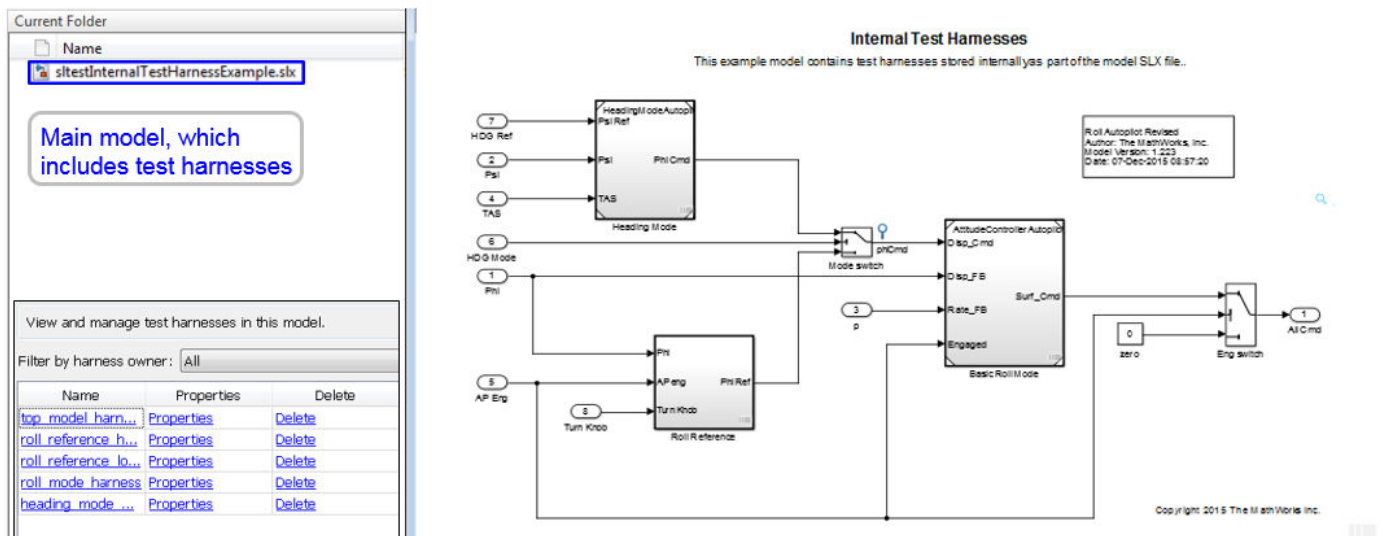
- 1 Navigate to the top of the main model.
- 2 On the **Apps** tab, under Model Verification, Validation, and Test, click **Simulink Test**. Then, on the **Tests** tab, click **Manage Test Harnesses > Convert to External Harnesses** or **Convert to Internal Harnesses**.
- 3 A dialog box provides information on the conversion procedure and the affected test harnesses. Click **Yes** to continue.

The harnesses are converted.

- 4 The conversion to external test harnesses creates an SLX file for each test harness and a harness information XML file `<modelName>_harnessInfo.xml`.

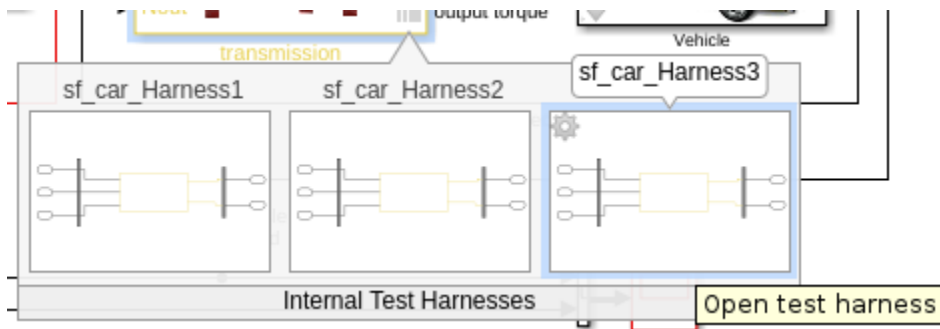


Inversely, conversion to internal test harnesses moves the test harness SLX files and the `harnessInfo.xml` file.

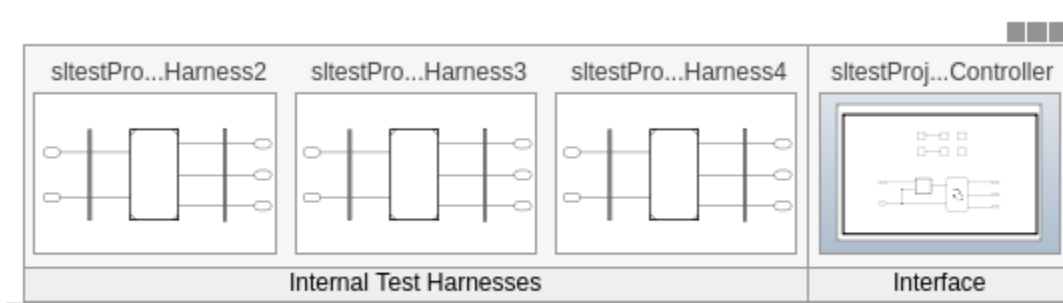


Preview and Open Test Harnesses

When a model component has a test harness, a badge appears in the lower right of the block. To view the test harnesses, click the badge. To open a test harness, click a tile.



To view test harnesses for a model block diagram, click the pullout icon in the model canvas. To open a test harness, click a tile.

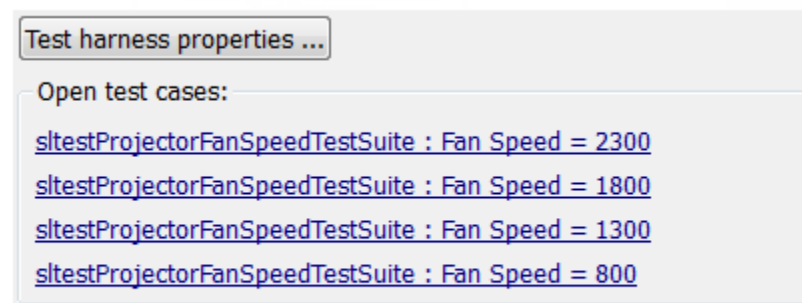


Model and Test Harness Locking

Model and test harness locking is specific to each type of synchronization. For information, see “Synchronization Mode” on page 2-19.

Find Test Cases Associated with a Test Harness

To list open test cases that refer to the test harness, click the badge  in the test harness canvas. You can click a test case name and navigate to the test case in the Test Manager.



Export Test Harnesses to Standalone Models

You can export test harnesses to standalone models, which is useful for archiving test harnesses or sharing a test harness design without sharing the model.

- To export an individual test harness:
 - 1 From the individual harness model, on the **Apps** tab, under **Model Verification, Validation, and Test**, click **Simulink Test**.
 - 2 In the **Harness** tab, click **Detach and Export**.
 - 3 In the Export Test Harness to Independent Model dialog box, click **OK**.
 - 4 In the Save As dialog box, enter a filename for the standalone harness model and click **OK**.
 - 5 The harness converts to a standalone model.

Converting removes the harness from the main model and breaks the relationship to the main model. If a model has only one harness, its `harnessInfo.xml` file is deleted. If a model has more than one harness and you delete one of them, the `harnessInfo.xml` file is updated.

- To export all harnesses in a model:
 - 1 Navigate to the top level of the model. Do not select any blocks.
 - 2 On the **Apps** tab, under **Model Verification, Validation, and Test**, click **Simulink Test**.
 - 3 In the **Harness** tab, click **Detach and Export**.
 - 4 In the Export Test Harness to Independent Model dialog box, click **OK**.
 - 5 In the Save As dialog box, enter a filename for the separate model and click **OK**.
 - 6 All test harnesses are exported and converted into standalone models.

Exporting removes the harnesses from the main model, deletes the `harnessInfo.xml` file, and breaks the relationships to the main model.

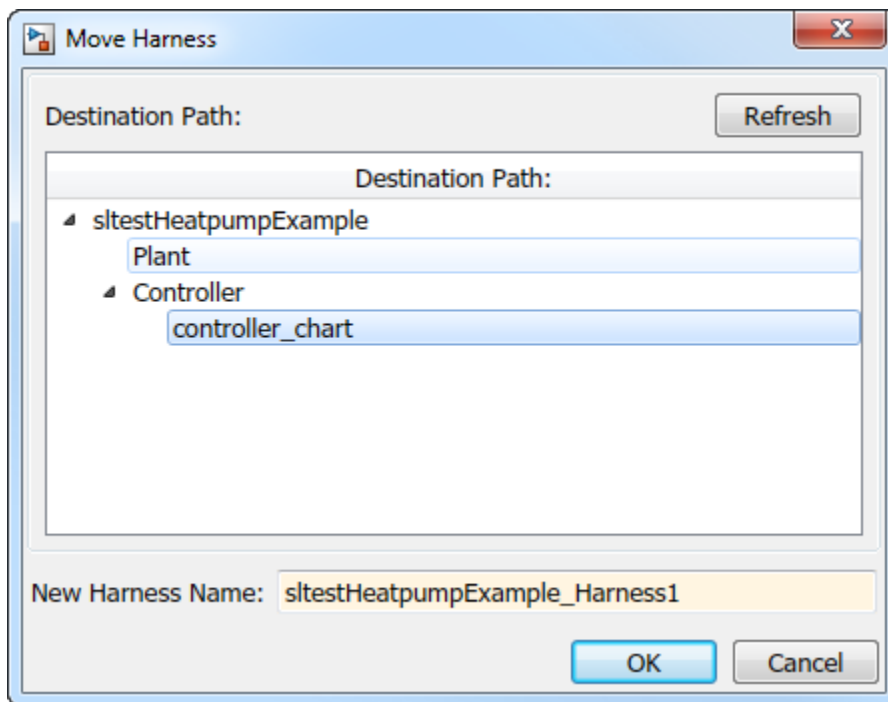
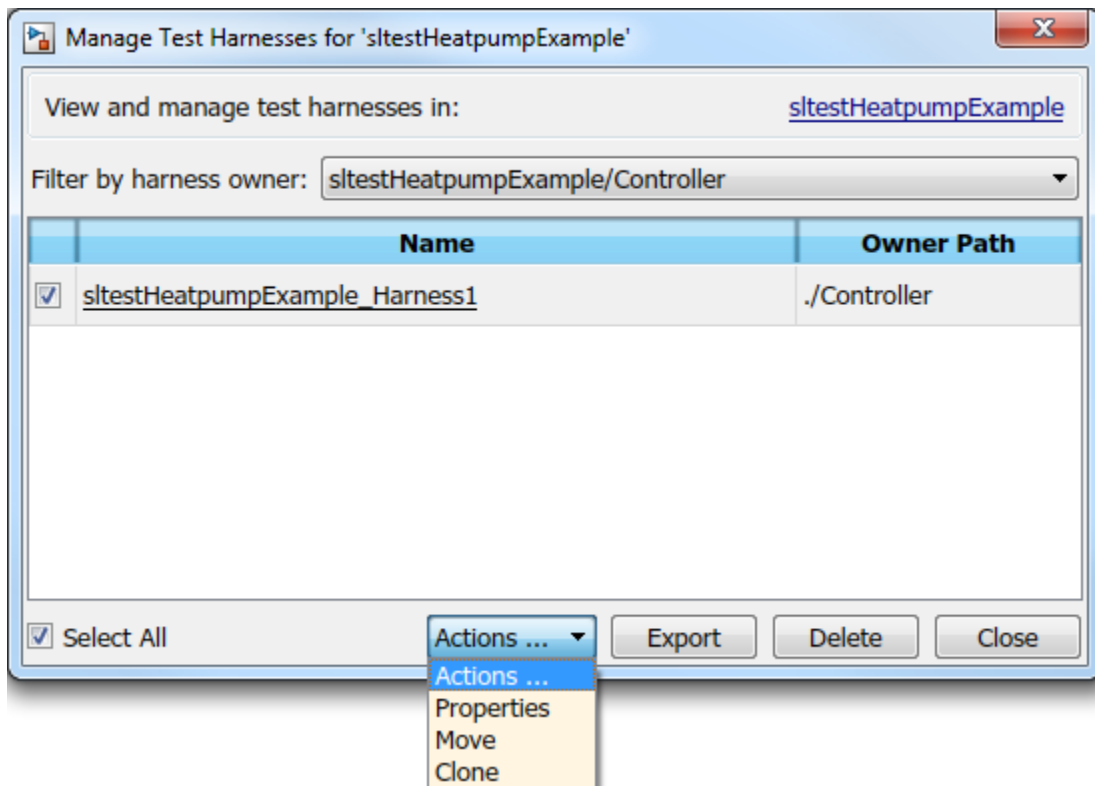
See `sltest.harness.export`.

Move and Clone Test Harnesses

Simulink Test gives you the ability to move or clone test harnesses from a source owner to a destination owner without having to compile the model. You can move or clone:

- Subsystem harnesses across subsystems. The destination subsystem could also be in a different model.
- Harnesses for library components across libraries.
- Subsystem Reference block harnesses to other Subsystem Reference block harnesses.
- Subsystem Reference block harnesses to and from Subsystem model harnesses.

To move or clone harnesses, right-click the Simulink canvas and select **Test Harness > Manage Test Harnesses**. The Manage Test Harness dialog box opens and lists the test harnesses associated with the subsystem or block specified in **Filter by harness owner**. Click **Actions** to access the Move and Clone options.



Select the destination path and name your test harness.

Test Harness Transfer When Converting Subsystems to Model References

When you convert a Subsystem or Subsystem Reference block to a Model block, the test harnesses are transferred to the model being referenced. Harnesses on the Subsystem block are converted to block diagram harnesses. Nested harnesses in subsystems are copied to identical blocks in the Model block. All transferred harnesses are internal harnesses in the model reference. Test harnesses might be renamed when they are transferred. You can see feedback about the harness transfer in the **Complete Conversion** pane of the Conversion Advisor or at the MATLAB command line.

These limitations apply to converting a Subsystem or Subsystem Reference block to a Model block:

- SIL and PIL harnesses are not transferred.
- Requirements in a test harness for a subsystem are not transferred. You must copy them manually.

Clone and Export a Test Harness to a Separate Model

This example demonstrates cloning an existing test harness and exporting the cloned harness to a separate model. This can be useful if you want to create a copy of a test harness as a separate model, but leave the test harness associated with the model component.

High-level Workflow

- 1 If you don't know the exact properties of the test harness you want to clone, get them using `sltest.harness.find`. You need the harness owner ID and the harness name.
- 2 Clone the test harness using `sltest.harness.clone`.
- 3 Export the test harness to a separate model using `sltest.harness.export`. Note that there is no association between the exported model and the original model. The exported model stands alone.

Open the Model

```
open_system("sf_car")
```


Clone the Test Harness

Clone the test harness using `sltest.harness.clone`, the `ownerFullPath` and the `name` fields of the harness properties structure.

```
newHarness = "ControllerHarness";
sltest.harness.clone(properties.ownerFullPath, ...
    properties.name,newHarness)
```

Export the Test Harness to a Separate Model

Export the test harness using `sltest.harness.export`. The exported model name is `ControllerTestModel`.

```
sltest.harness.export("sf_car/shift_logic",newHarness,...
    Name="ControllerTestModel")
```

Close the model without saving it.

```
close_system("sf_car",0)
```

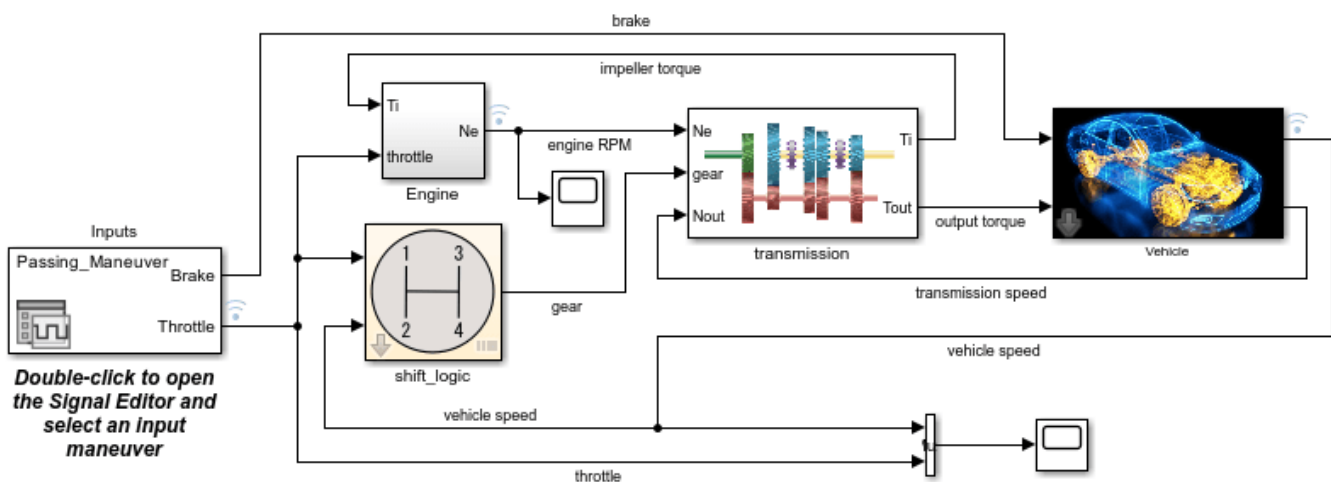
Delete Test Harnesses Programmatically

This example shows how to delete test harnesses programmatically. Deleting using the programmatic interface can be useful when your model has multiple test harnesses at different hierarchy levels. This example demonstrates how to create four test harnesses and then, delete them.

1. Open the model.

```
model = "sf_car";
open_system(model);
```

Model a Car with Automatic Transmission



Copyright 1997-2025 The MathWorks, Inc.

2. Display the name and owner of the original harness in the model.

```
testHarnesses = sltest.harness.find(model);
originalHarness = testHarnesses.name;
disp(table({testHarnesses.name}', ...
    {testHarnesses.ownerFullPath}', ...
    VariableNames=["Name", "Owner"])))
```

Name	Owner
{'TestDownshiftPointsHarness'}	{'sf_car/shift_logic'}

3. Create two harnesses for the transmission subsystem, and two harnesses for the transmission ratio subsystem.

```
subsystem1 = "sf_car/transmission";
subsystem2 = "sf_car/transmission/transmission_ratio";
```

```
sltest.harness.create(subsystem1);
sltest.harness.create(subsystem1);
sltest.harness.create(subsystem2);
sltest.harness.create(subsystem2);
```

4. Display the names and owners of the harnesses in the model.

```
testHarnesses = sltest.harness.find(model);
disp(table({testHarnesses.name}', ...
    {testHarnesses.ownerFullPath}', ...
    VariableNames=["Name", "Owner"])))
```

Name	Owner
{'TestDownshiftPointsHarness'}	{'sf_car/shift_logic'}
{'sf_car_Harness1'}	{'sf_car/transmission'}
{'sf_car_Harness2'}	{'sf_car/transmission'}
{'sf_car_Harness3'}	{'sf_car/transmission/transmission_ratio'}
{'sf_car_Harness4'}	{'sf_car/transmission/transmission_ratio'}

5. Delete the new harnesses.

```
for k = 1:length(testHarnesses)
    if ~strcmp(testHarnesses(k).name, originalHarness)
        sltest.harness.delete(testHarnesses(k).ownerFullPath, ...
            testHarnesses(k).name)
    end
end
```

6. Display the name and owner of the harness in the model.

```
testHarnesses = sltest.harness.find(model);
disp(table({testHarnesses.name}', ...
    {testHarnesses.ownerFullPath}', ...
    VariableNames=["Name", "Owner"])))
```

Name	Owner
------	-------


```
{'TestDownshiftPointsHarness'}    {'sf_car/shift_logic'}
```

7. Close the model without saving it.

```
close_system(model,0);
```

Export Test Harness to Previous Version

When you export a Simulink model that has one or more test harnesses to a previous MATLAB version, the impact on the harnesses depends on whether they are internal or external.

- For models with internal harnesses, the harnesses stay internal and are exported with the model.
- For models with external harnesses, the harnesses in the exported model are converted to internal harnesses. To change the internal harnesses back to external, see “Convert Between Internal and External Test Harnesses” on page 2-30.

You cannot export internal or external harnesses without also exporting their parent model to a previous version. To work around this restriction, export the harness to a standalone model, export the harness model to the previous version, and then import it into the main model. After importing the harness, check the harness settings. You might need to reset some properties, such as the synchronization mode and rebuild. See “Export Test Harnesses to Standalone Models” on page 2-33 and “Change Test Harness Properties” on page 2-15.

See Also

`sltest.harness.create` | `sltest.harness.clone` | `sltest.harness.delete` |
`sltest.harness.export` | `sltest.harness.find` | `sltest.harness.load` |
`sltest.harness.open` | `sltest.harness.move` | Model Reference Conversion Advisor

More About

- “Test Harness and Model Relationship” on page 2-2
- “Create or Import Test Harnesses and Select Properties” on page 2-13

Customize Test Harnesses

In this section...

“Callback Function Definition and Harness Information” on page 2-40

“Display Harness Information struct Contents” on page 2-42

“Share Data Between Callbacks” on page 2-42

“Customize a Test Harness to Create Mixed Source Types” on page 2-43

“Test Harness Callback Example” on page 2-44

You can customize a test harness by using one or more functions that run as callbacks after creating the test harness. You can use one function to run as a callback for rebuilding the test harness. In the function, write the commands to customize your test harness. For example, you can create functions to:

- Connect custom source or sink blocks.
- Add a plant subsystem for closed-loop testing.
- Change the configuration set.
- Enable signal logging.
- Change the simulation stop time.

To customize a test harness using callback functions:

- 1** Create the callback function.
- 2** In the function, use the Simulink programmatic interface to script the commands to customize the test harness. For more information, see the functions listed in “Programmatic Model Editing”.
- 3** Specify the function or functions as the post-create or a single function as a post-rebuild callback:
 - For a new test harness,
 - If you are using the UI, enter the function names in the **Post-create callback method**, separated by commas, or a single function name in the **Post-rebuild callback method** in the Create Test Harness dialog box.
 - If you are using `sltest.harness.create`, specify the function as the `PostCreateCallback` or `PostRebuildCallback` value. For the `PostCreateCallback`, you can specify more than one function.
 - For an existing test harness,
 - If you are using the UI, enter the function name in **Post-rebuild callback method** in the harness properties dialog box.
 - If you are using `sltest.harness.set`, specify the function as the `PostRebuildCallback` value.

Another way to customize test harnesses is by setting your own defaults for creating harnesses. For more information, see “Create or Import Test Harnesses and Select Properties” on page 2-13.

Callback Function Definition and Harness Information

The callback function declaration is

```
function myfun(x)
```

where `myfun` is the function name and `myfun` accepts input `x`. The `x` input is a struct of information about the test harness automatically created when the test harness uses the callback. You can choose the function and argument names.

For example, define a harness callback function `harnessCustomization.m`:

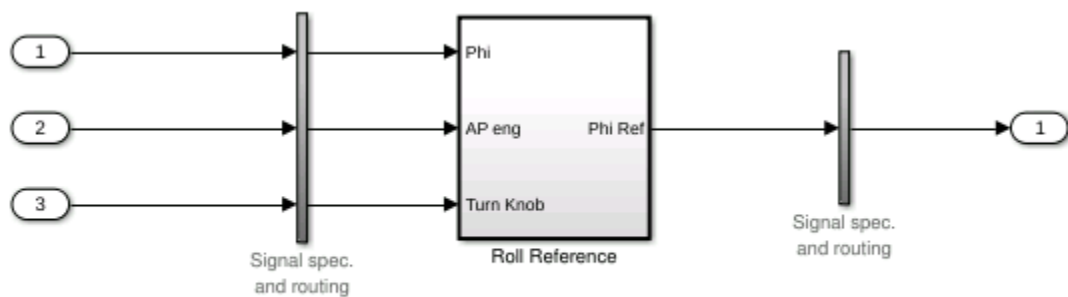
```
function harnessCustomization(harnessInfo)
```

```
% Script commands here to customize your test harness.
```

```
end
```

In this example, `harnessInfo` is the struct name and `harnessCustomization` is the function name. When the create or rebuild operation calls `harnessCustomization`, `harnessInfo` is populated with information about the test harness, including handles to the test harness model, main model, and blocks in the test harness.

For example, using `harnessCustomization` as a callback for the following test harness:



populates `harnessInfo` with handles to three sources, one sink, the main model, harness model, harness owner, component under test, and conversion subsystems:

```
harnessInfo =
```

```
struct with fields:
```

```

    MainModel: 2.0001
    HarnessModel: 1.1290e+03
        Owner: 17.0001
    HarnessCUT: 201.0110
    DataStoreMemory: []
    DataStoreRead: []
    DataStoreWrite: []
    Goto: []
    From: []
    GotoTag: []
    SimulinkFunctionCaller: []
    SimulinkFunctionStub: []
        Sources: [1.1530e+03 1.1540e+03 1.1550e+03]
        Sinks: 1.1630e+03
    AssessmentBlock: []

```

```
InputConversionSubsystem: 1.1360e+03
OutputConversionSubsystem: 1.1560e+03
CanvasArea: [215 140 770 260]
```

Use the struct fields to customize the test harness. For example:

- To add a Constant block named `ConstInput` to the test harness, get the name of the test harness model, then use the `add_block` function.

```
harnessName = get_param(harnessInfo.HarnessModel, 'Name');
block = add_block('simulink/Sources/Constant', ...
    [harnessName '/ConstInput']);
```

- To get the port handles for the component under test, get the `'PortHandles'` parameter for `harnessInfo.HarnessCUT`.

```
CUTPorts = get_param(harnessInfo.HarnessCUT, 'PortHandles');
```

- To get the simulation stop time for the test harness, get the `'StopTime'` parameter for `harnessInfo.HarnessModel`.

```
st = get_param(harnessInfo.HarnessModel, 'StopTime');
```

- To set a 16 second simulation stop time for the test harness, set the `'StopTime'` parameter for `harnessInfo.HarnessModel`.

```
set_param(harnessInfo.HarnessModel, 'StopTime', '16');
```

Display Harness Information struct Contents

To list the harness information for your test harness:

- 1 In the callback function, add the line

```
disp(harnessInfo)
```
- 2 Create or rebuild a test harness using the callback function.
- 3 When you create or rebuild the test harness, the harness information structure contents are displayed on the command line.

Share Data Between Callbacks

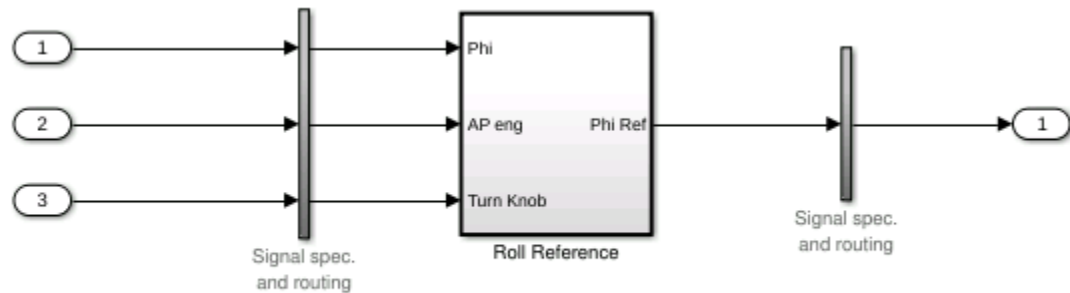
Callback scripts are evaluated in the MATLAB base workspace. To share data between callback scripts, use `assignin` and `evalin` to store and retrieve data from the base workspace. For example,

```
assignin('base', 'a', 2);
a = evalin('base', 'a');
```

For parallel execution, post-load and cleanup callbacks are evaluated on the parallel MATLAB worker, where the callbacks have their own base workspaces. Variables created in a test case pre-load callback and test file and test suite setup callbacks are also available in the parallel MATLAB worker base workspace. Prior to execution, the base workspace variables are transferred from the client MATLAB to the parallel MATLAB workers.

Customize a Test Harness to Create Mixed Source Types

This example harness callback function connects a Constant block to the third component input of this example test harness.



The function follows the procedure:

- 1 Get the harness model name.
- 2 Add a Constant block.
- 3 Get the port handles for the Constant block.
- 4 Get the port handles for the input conversion subsystem.
- 5 Get the handles for lines connected to the input conversion subsystem.
- 6 Delete the existing Inport block.
- 7 Delete the remaining line.
- 8 Connect a new line from the Constant block to input 3 of the input conversion subsystem.

```
function harnessCustomization(harnessInfo)

% Get harness model name:
harnessName = get_param(harnessInfo.HarnessModel, 'Name');

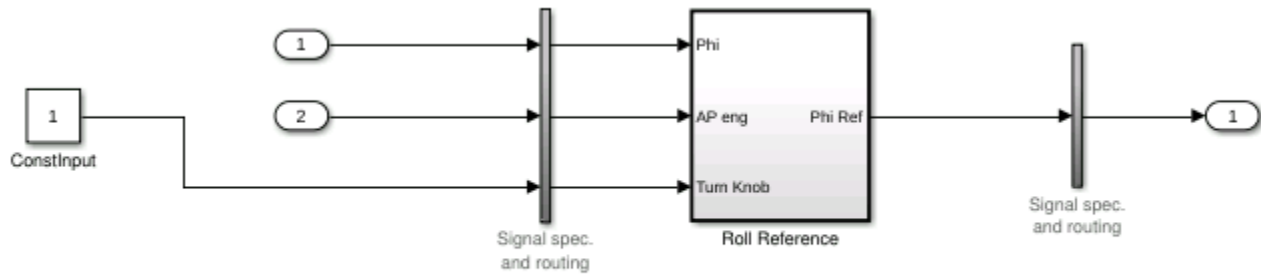
% Add Constant block:
constBlock = add_block('simulink/Sources/Constant', ...
    [harnessName '/ConstInput']);

% Get handles for relevant ports and lines:
constPorts = get_param(constBlock, 'PortHandles');
icsPorts = get_param(harnessInfo.InputConversionSubsystem, ...
    'PortHandles');
icsLineHandles = get_param(...
    (harnessInfo.InputConversionSubsystem, 'LineHandles'));

% Delete the existing Inport block and the adjacent line:
delete_block(harnessInfo.Sources(3));
delete_line(icsLineHandles.Inport(3));

% Connect the Constant block to the input
% conversion subsystem:
add_line(harnessInfo.HarnessModel, constPorts.Outport, ...
    icsPorts.Inport(3), 'autorouting', 'on');

end
```



Test Harness Callback Example

This example shows how to use a post-create callback to customize a test harness. The callback changes one harness source from an Inport block to Constant block and enables signal logging in the test harness.

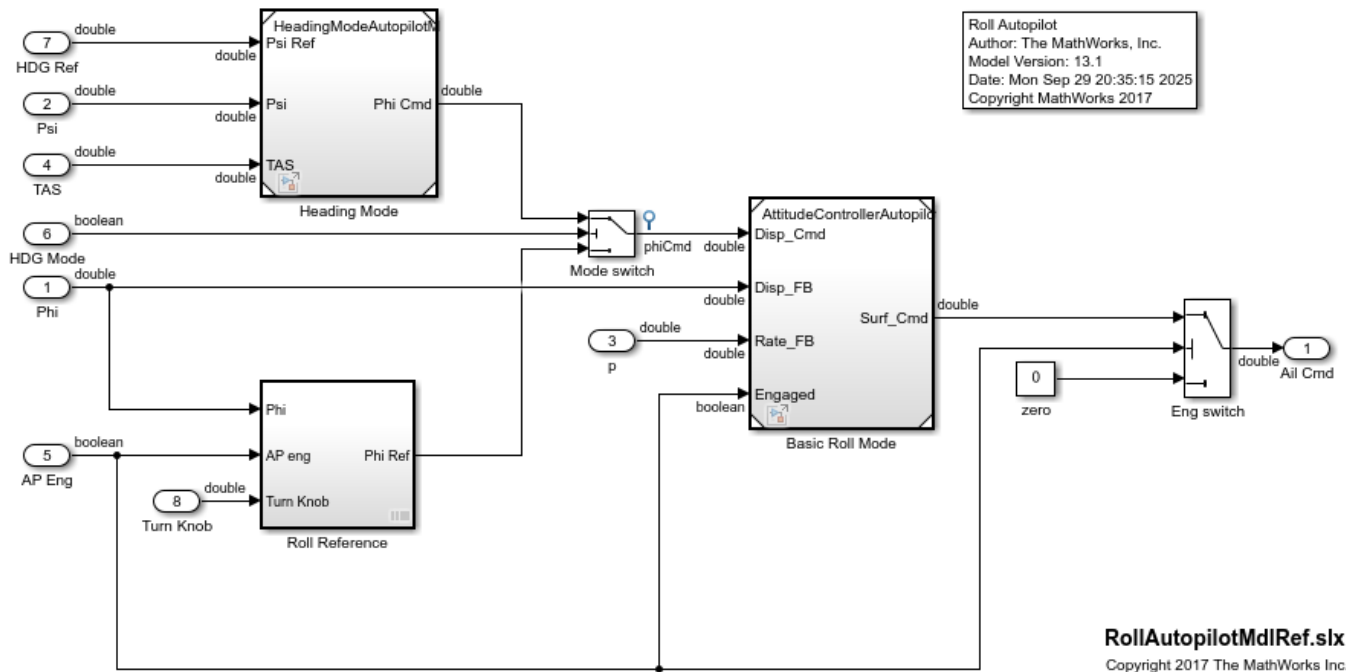
The Model

In this example, you create a test harness for the Roll Reference subsystem.

```
open_system('RollAutopilotMdlRef')
```

Requirements-based Testing for Controller Development

This model is used to show how to perform requirements-based testing using test harnesses, Test Sequence blocks, and the test manager.
To view the demo, enter `sltestRequirementsTestingAutopilotDemo` in MATLAB(R).



Get Path to the Harness Customization Function

```
cbFile = 'harnessSourceLogCustomization.m';
```

The Customization Function and Test Harness Information

The function `harnessSourceLogCustomization` changes the third source block, and enables signal logging on the component under test inputs and outputs. You can read the function by entering:

```
type(cbFile)
```

As an alternative to including output logging code in the callback, you can use **Log Output Signals** in the Create New Harness dialog box, or use `'LogOutputs', true` as an input to `sltest.harness.create`. These options log all component under test output signals in the test harness and return test results for those signals.

The function `harnessSourceLogCustomization` uses an argument. The argument is a struct listing test harness information. The information includes handles to blocks in the test harness, including:

- Component under test
- Input subsystems
- Sources and sinks
- The harness owner in the main model

For example, `harnessInfo.Sources` lists the handles to the test harness source blocks.

Create the Customized Test Harness

1. In the `RollAutopilotMdlRef` model, right-click the `Roll Reference` subsystem and select **Test Harness > Create for Roll Reference**.

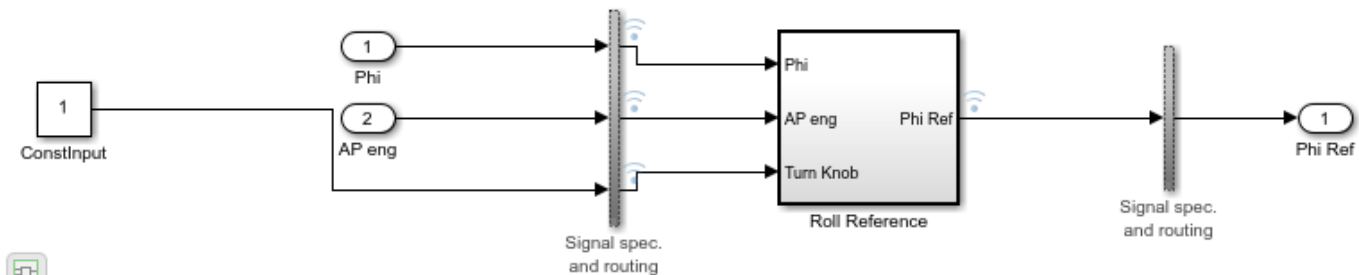
2. In the harness creation dialog box, for **Post-create callback method**, enter `harnessSourceLogCustomization`.

3. Click OK to create the test harness. The harness shows the signal logging and simulation stop time specified in the callback function.

You can also use the `sltest.harness.create` function to create the test harness, specifying the callback function with the 'PostCreateCallback' name-value pair.

```
sltest.harness.create('RollAutopilotMdlRef/Roll Reference',...
    'Name','LoggingHarness',...
    'PostCreateCallback','harnessSourceLogCustomization');

sltest.harness.open('RollAutopilotMdlRef/Roll Reference', ...
    'LoggingHarness');
```



```
close_system('RollAutopilotMdlRef',0);
```

See Also

`sltest.harness.create` | `sltest.harness.set`

Related Examples

- “Create or Import Test Harnesses and Select Properties” on page 2-13

Create Test Harnesses from Standalone Models

In this section...
"Test Harness Import Workflow" on page 2-47
"Component Compatibility for Test Harness Import" on page 2-48
"Import a Standalone Model as a Test Harness" on page 2-49

Standalone test models are often used to verify your main model. You can create Simulink Test test harnesses by importing your standalone test models. Importing standalone models enables synchronization and management features, allowing you to:

- Iterate on your design, using model and test harness synchronization
- Manage test harnesses, using the UI and programmatic interfaces
- Clarify ownership of a test harness by a model, subsystem, or library being tested

A common test model passes input signals to a copy of a subsystem or a Model block referencing your main model. Test models include models created by Simulink Coverage™ and Simulink Design Verifier™.

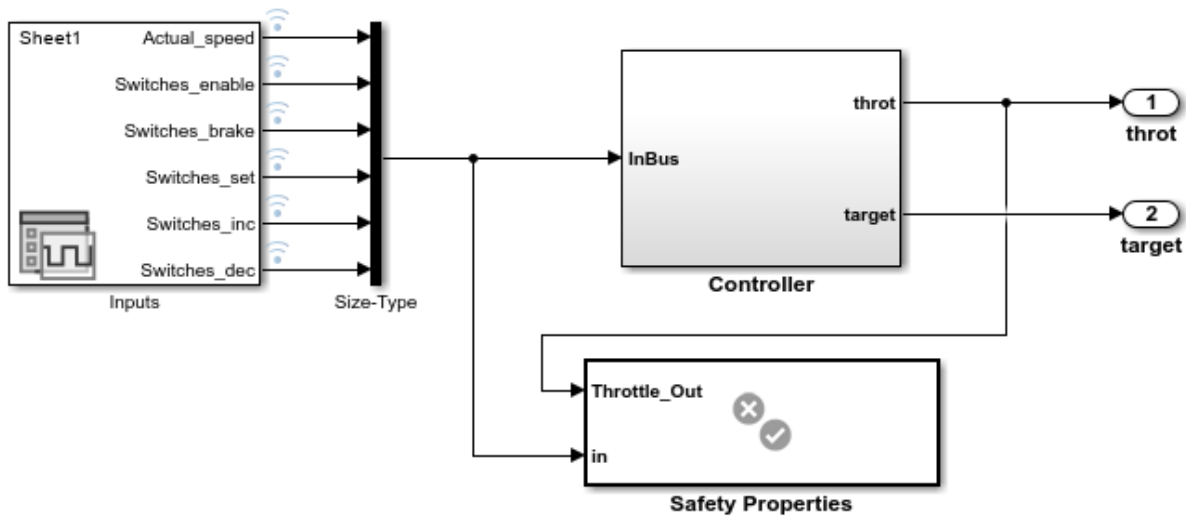
Test Harness Import Workflow

Before importing a standalone model as a test harness, determine:

- In the main model, the model or component to associate the test harness with.
- The path to the standalone model.
- The tested component in the standalone model.

For example, this standalone model tests the Controller subsystem. The model passes Inputs to Controller. Safety Properties verifies the Controller output.

Simulink Test Basic Cruise Control Verification



Copyright 2006-2022 The MathWorks, Inc.

Component Compatibility for Test Harness Import

When you import a model as a test harness, the component in the main model must be compatible with the component in the standalone model.

Component Compatibility for Test Harness Import

In the main model, if the component is:	In the standalone model, the tested component must be:
A user-defined function block (e.g. an S-Function block)	The same block type
The top-level model	A Model block or a subsystem
A subsystem	A subsystem, Model block, or a user-defined function block
A Subsystem Reference block	Subsystem model
A Model block	A Model block or a subsystem

You cannot create a test harness by importing:

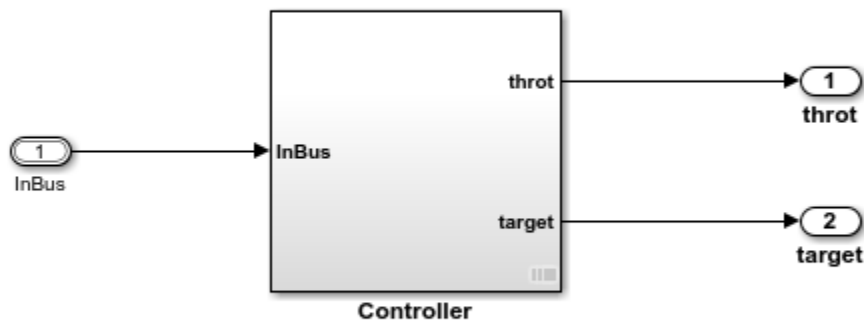
- Libraries
- Models that have existing test harnesses
- Models with unsaved changes. Save open models before importing

Import a Standalone Model as a Test Harness

This example shows how to import a standalone test model to create a test harness in Simulink Test.

The main model `sltestBasicCruiseControl` is a cruise control system, with root import and output blocks.

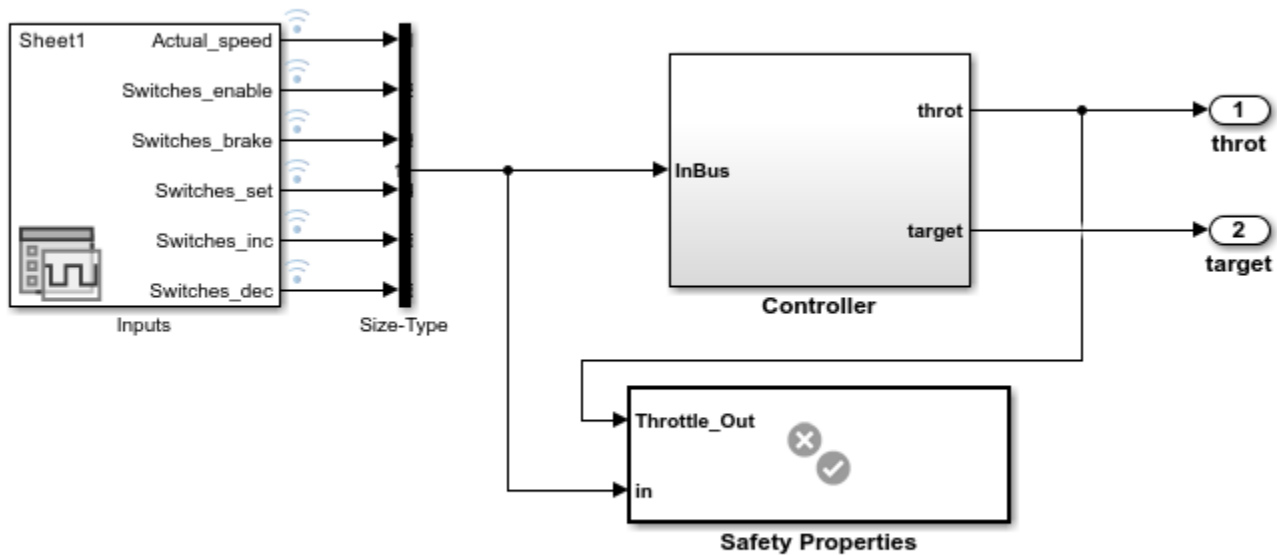
Simulink Test: Basic Cruise Control



Copyright 2006-2022 The MathWorks, Inc.

The test model contains a Signal Editor block driving a copy of the **Controller** subsystem, with a subsystem verifying that the throttle output goes to 0 if the brake is applied for three consecutive time steps.

Simulink Test Basic Cruise Control Verification

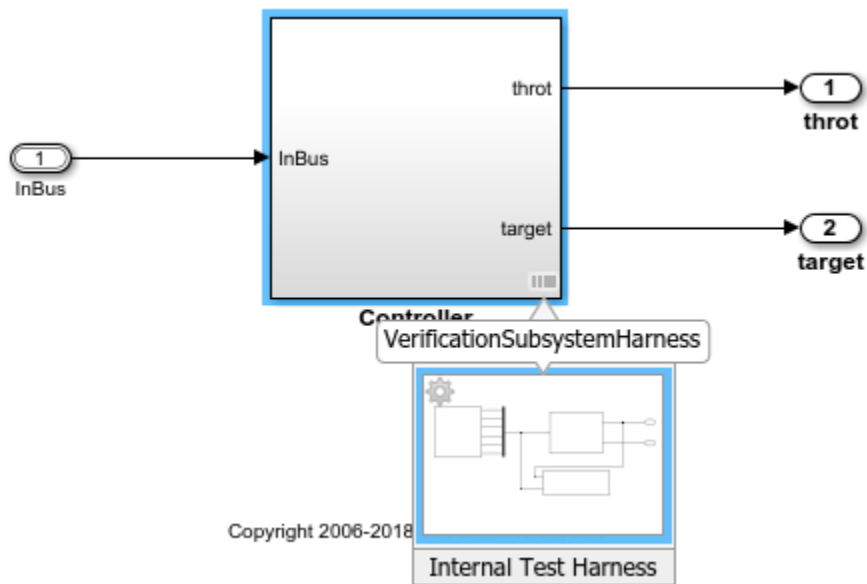


Copyright 2006-2022 The MathWorks, Inc.

Create a Test Harness from the Standalone Model

1. In the main model, right-click the Controller subsystem and select **Test Harness > Import for 'Controller'**.
2. Set the following harness properties:
 - **Name:** VerificationSubsystemHarness
 - **Simulink model to import:** Click **Browse** and select `sltestBasicCruiseControlHarnessModel` in the MATLAB® `examples/simulinktest` directory.
 - **Component under Test in imported model:** Controller
3. Click **OK**.

A test harness is created from the standalone model, owned by the Controller subsystem in the main model. Click the badge to preview the test harness.



See Also

`sltest.harness.import`

Related Examples

- “Test Harness and Model Relationship” on page 2-2
- “Synchronize Changes Between Test Harness and Model” on page 2-52

Synchronize Changes Between Test Harness and Model

In this section...

- “Set Synchronization for a New Test Harness” on page 2-52
- “Change Synchronization of an Existing Test Harness” on page 2-57
- “Synchronize Configuration Set and Model Workspace Data” on page 2-57
- “Check for Unsynchronized Component Differences” on page 2-57
- “Rebuild a Test Harness” on page 2-57
- “Push Changes from Test Harness to Model” on page 2-58
- “Check Component and Push Parameter to Main Model” on page 2-58

A test harness provides an isolated environment to test design changes. You can synchronize changes from the test harness to the main model, or from the main model to the test harness. Synchronization includes these model elements:

- Component under test (CUT) — CUT block and blocks, signals, and other entities in the CUT.
- Block parameters — Values of parameters associated with the block. To see the parameters of a block, right-click on the block and select **Block Parameters** from the context menu.
- Optionally, the active configuration set of the model or test harness. See “Manage Configuration Sets for a Model” for information on configuration sets

To synchronize the active configuration set and model workspace parameters between the test harness and main model, select **Update Configuration Parameters and Model Workspace data on rebuild** in the **Advanced Properties** tab of the Create Test Harness dialog box.

You do not need to synchronize base workspace data because it is available to both the test harness and main model. Subsystem model test harnesses always sync with their underlying models.

Set Synchronization for a New Test Harness

When you create a test harness, you specify when changes in the test harness are synchronized with the main model. Synchronization can occur automatically or manually. If you plan to try different component designs in the test harness, use manual synchronization to avoid overwriting the component in the main model.

For all synchronization types, you can simulate the main model even if a test harness is open. You can also create harnesses for model components other than the current component under test and its nested subsystems. You can have more than one harness owned by the same model open at the same time.

The settings for the **Synchronization Mode** parameter are:

- **Synchronize on harness open and close** — When you open one or more test harnesses, synchronization occurs from the model to the harness. If only one harness is open, when you close the harness, synchronization occurs from the harness to the model. When more than one harness is open and you close only one of them, synchronization to the model does not occur.
- **Synchronize on harness open** — When you open one or more test harnesses, synchronization occurs from the model to the harness.

- **Synchronize only during push and rebuild** — When only one test harness is open and you click **Push Changes** in the toolbar, synchronization occurs from the harness to the model. When you click **Rebuild Harness**, synchronization occurs from the model to the harness. When more than one harness is open, synchronization occurs only when you rebuild the model. Push synchronization is not supported.
- **Synchronize only during rebuild** — When one or more harnesses are open, synchronization occurs only when you click **Rebuild Harness**. Changes synchronize from the model to the test harness.

Note Synchronization copies subsystems from the main model to the subsystem test harness model and from the subsystem test harness model to the main model. Simulink block copy callbacks run both when the main model synchronizes to the subsystem test harness and when the subsystem test harness synchronizes to the main model. Synchronization for subsystems does not synchronize code or state mappings.

If you create a test harness in software-in-the-loop (SIL) or processor-in-the-loop (PIL) mode for a Model block, the block mode in the test harness is changed to SIL or PIL, respectively. This mode is not updated to the main model when you close the test harness.

If you use a SIL or PIL block in the test harness, consider setting the test harness to rebuild every time it opens. Regularly rebuilding the test harness keeps the generated code referenced by the SIL/PIL block as a reflection of the main model.

The table indicates whether you can change the model, harness, or the CUT in the model or test harness when one or more test harnesses are open for each synchronization type.

Synchronization Type	Supported Model Elements	Locking When One Harness Is Open	Locking When More Than One Harness Is Open
Synchronize on harness open and close	Available for: - Subsystems, including Stateflow charts and MATLAB Function blocks - Model blocks - S-function blocks - Subsystem Reference blocks — only the block parameters are synchronized Not available for: - Block diagrams - SIL/PIL harnesses - Subsystem model harnesses - System Composer models, components, or CUTs	These elements are unlocked: - Main models and main model harnesses - CUTs that are not subsystems in the main model and harness - Subsystem harnesses and CUTs in the harness These elements are locked: - Subsystem CUTs in the main model	These elements are unlocked: - Main models and main model harnesses - Harness CUTs - CUTs that are not subsystems and harnesses, library harnesses, and MATLAB functions - Subsystem CUTs in the main model and in the harness, but no synchronization occurs These elements are locked: - Main model CUTs

Synchronization Type	Supported Model Elements	Locking When One Harness Is Open	Locking When More Than One Harness Is Open
Synchronize on harness open	Available for: - Block diagrams - Subsystems, including Stateflow charts and MATLAB Function blocks - Model blocks - S-function blocks - Subsystem Reference blocks — only the block parameters are synchronized Not available for: - SIL or PIL harnesses - Subsystem model harnesses	These elements are unlocked: - Main models and main model harnesses - CUTs that are not subsystems in the main model and harness These elements are locked: - Subsystem CUTs in the main model and harness	These elements are unlocked: - Main models and main model harnesses - CUTs that are not subsystems in the main model and harness - System Composer main models and CUTs in the harness for inline components These elements are locked: - Subsystem CUTs in the main model and harness - System Composer Model reference component CUTs in the main model and harness

Synchronization Type	Supported Model Elements	Locking When One Harness Is Open	Locking When More Than One Harness Is Open
Synchronize only during push and rebuild	Available for: - Subsystems, including Stateflow charts and MATLAB Function blocks - Model blocks - S-function blocks - Subsystem models — only synchronize on push and rebuild. Not available for: - Block diagrams - SIL or PIL harnesses - Components in libraries - System Composer models, components, or CUTs	These elements are unlocked: Main model, harnesses, and all types of CUTs in the model and harnesses, including subsystems.	These elements are unlocked: - Main model, harnesses, and all types of CUTs in the harnesses, including Subsystem harnesses These elements are locked: - Main model CUTs.
Synchronize only during rebuild	Available for: - Block diagrams - SIL/PIL verification mode components Not available for: - Subsystems, including Stateflow charts and MATLAB Function blocks - S-function blocks - Components in libraries - System Composer models, components, or CUTs	These elements are unlocked: - Main models, harnesses, and all types of CUTs, except SIL/PIL verification mode components in the model These elements are locked: - SIL or PIL verification mode components in the harness are locked and masked. Note Synchronize only during rebuild is not available for Subsystems.	All elements are unlocked.

Change Synchronization of an Existing Test Harness

To change a test harness synchronization mode:

- 1 Close the test harness.
- 2 In the main model, click the harness badge on the block or the Simulink canvas.
- 3 In the test harness thumbnail preview, click the **Harness operations** icon and select **Properties**.
- 4 Change the **Synchronization Mode** in the properties dialog box.

If you use the command line, set the `SynchronizationMode` property with `sltest.harness.set`.

Synchronize Configuration Set and Model Workspace Data

To synchronize the active configuration set and workspace parameters between the test harness and main model, select **Update Configuration Parameters and Model Workspace data on rebuild** in the harness creation or harness properties dialog box.

Check for Unsynchronized Component Differences

If your test harness does not synchronize changes, you can check for unsynchronized component differences between the test harness and main model. Checking for unsynchronized differences can be useful if:

- You are making tentative design changes in the test harness and want to check that the main model component is not overwritten.
- You have made design changes to the main model and want to check which test harnesses must be rebuilt.

From the test harness window, select **Check Harness** to check for differences. If the component differs, you can push changes from the test harness to the main model, or rebuild the test harness from the main model. Also see the `sltest.harness.check` function.

Consider these conditions when checking for unsynchronized differences:

- `sltest.harness.check` only includes the block diagram, block parameters, and mask parameters in the comparison between the test harness and main model. Port options, compiled attributes, hidden parameters, and Model block data logging parameters are not included in the comparison.
- If the component contains a Simscape™ Solver Configuration block, the check result always shows that the component differs between the test harness and main model. The Solver Configuration block is affected by Simscape blocks outside the component, and therefore always differs between the test harness and main model.

Rebuild a Test Harness

Rebuild a test harness to reflect the latest state of the main model. In the test harness, select **Rebuild Harness**. In addition to updating the component under test and block parameters, this operation rebuilds harness conversion subsystems. If the test harness does not have conversion subsystems, rebuilding adds them.

Rebuilding can disconnect signal lines. For example, if signal names changed in the main model, signal lines in the test harness can be disconnected. If lines are disconnected, reconnect signal lines to the component under test or conversion subsystems. If you specified to use existing generated code for a SIL/PIL subsystem using `sltest.harness.create` or `sltest.harness.set`, the harness rebuild uses that code instead of regenerating it.

For more information, see “Create or Import Test Harnesses and Select Properties” on page 2-13 and `sltest.harness.rebuild`.

Push Changes from Test Harness to Model

After changing your system in the test harness, if only one harness is open, you can push changes to the main model. In the test harness, select **Push Changes**. This process overwrites the component in the main model. Pushing changes to the model is not supported when more than one harness is open.

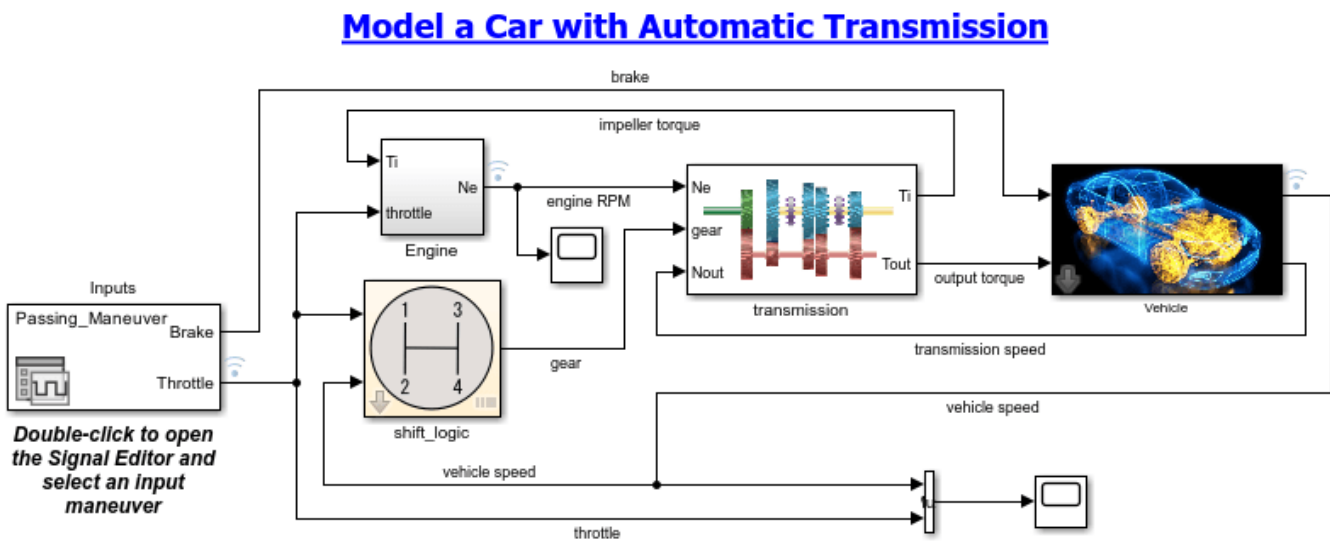
Check Component and Push Parameter to Main Model

This example shows a basic workflow of updating a parameter in a test harness, checking the synchronization between the test harness and main model, and pushing the parameter change from the test harness to the main model.

This example also includes programmatic steps.

Open the model `sf_car`. The model includes a transmission shift controller algorithm and simplified powertrain and vehicle dynamics.

```
model = "sf_car";  
open_system(model);
```



Copyright 1997-2025 The MathWorks, Inc.


```
logical
    0

details

details =

    struct with fields:

        overall: 0
        contents: 1
        reason: 'The contents of harnessed component and the contents of the component in the main
```

Update the Parameter to the Main Model

1. In the test harness, on the **Harness** tab, click **Push Changes**.
2. In the main model, double-click the shift_logic subsystem. The parameter value is updated.

To push the change using the command line, use:

```
sltest.harness.push(chart,harness)
```

3. Update the main model and test harness.

```
set_param(model,SimulationCommand="update");
set_param(harness,SimulationCommand="update");
```

4. Re-check the synchronization between the test harness and the main model by running the `sltest.harness.check` function a second time.

```
[comparison,details] = sltest.harness.check(chart,harness);
```

The results show that the component under test is the same between the test harness and the main model.

```
comparison
```

```
comparison =
```

```
logical
```

```
1
```

```
details
```

```
details =
```

```
struct with fields:
```

```
    overall: 1
    contents: 1
```

```
reason: 'The checksum of the harnessed component and the component in the main model are s
```

5. Close the model without resaving it.

```
close_system(model,0);
```

See Also

```
sltest.harness.check | sltest.harness.push | sltest.harness.rebuild
```

Related Examples

- “Test Harness and Model Relationship” on page 2-2
- “SIL Verification for a Subsystem” on page 5-2

Test Library Blocks

In this section...

“Library Testing Workflow” on page 2-62
 “Library and Linked Subsystem Test Harnesses” on page 2-62
 “Edit Library Block from a Test Harness” on page 2-63
 “Testing a Library and a Linked Block” on page 2-63
 “SIL Testing a Reusable Library Subsystem” on page 2-68

If your model includes instances of blocks from a library, you can test both the source block in the library, and individual block instances in other models. You can also test software-in-the-loop (SIL) code generated for a reusable library subsystem. First, create test harnesses for a library block to test your design. Once the library block meets your requirements, create test harnesses for linked blocks and test the subsystem instances.

Library Testing Workflow

This procedure outlines an example workflow for testing library subsystems and linked subsystems.

- 1 Create a test case and a test harness for the library subsystem.
- 2 Test the library subsystem. If it fails your requirements, revise the design and test again.
- 3 Lock the library when your tests pass.
- 4 In your model, create a linked subsystem and retain the library test harnesses.
- 5 Compare the output of the linked instance to that of the library block using an equivalence test case.
- 6 Create additional test cases and test harnesses for the linked instance.
- 7 Promote a test harness from the linked subsystem to the library if you want to include the test harness with future linked subsystems.

Library and Linked Subsystem Test Harnesses

A test harness for a library subsystem has specific properties:

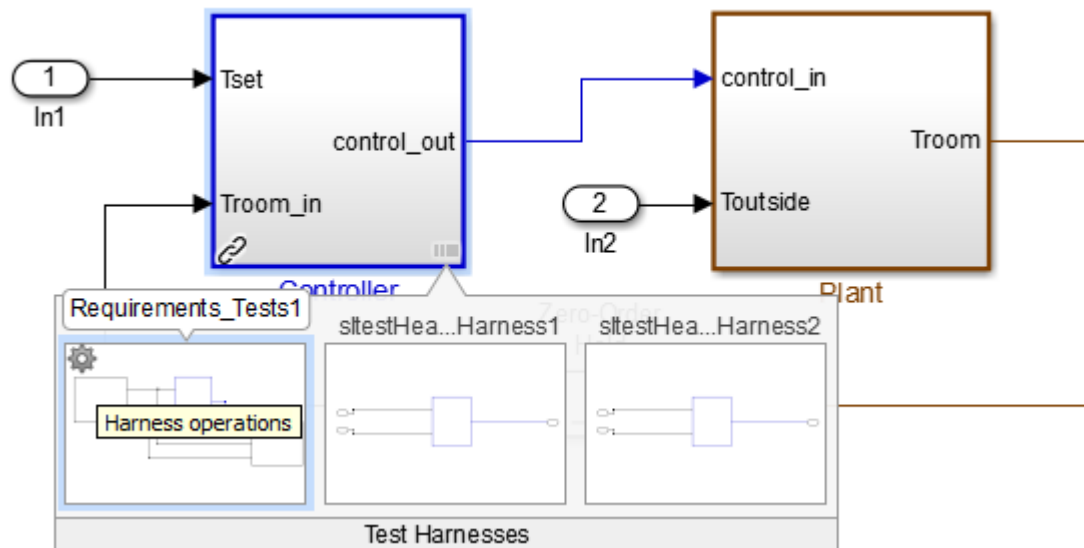
- Libraries do not compile, so a test harness for a library subsystem does not use compiled attributes such as data type or sample rate.
- A test harness for a library subsystem does not generate conversion subsystems for the block inputs and outputs.
- A library subsystem test harness does not use push or rebuild operations, because libraries do not use configuration parameters.

When you create a linked subsystem from a library subsystem, test harnesses copy to the linked instance. If you do not need the test harnesses, you can delete them. For instructions on deleting all test harnesses from a model, see “Manage Test Harnesses” on page 2-29.

When you create a test harness for a linked subsystem, the harness associates with the linked subsystem, not the library subsystem. You can move a test harness from a linked subsystem to the

library subsystem. For example, this linked subsystem **Controller** has three test harnesses. To move the **Requirements_Tests1** test harness to the library:

- 1 Click the harness badge on the linked subsystem.
- 2 Click the **Harness Operations**  icon.



- 3 Select **Move to Library**.
- 4 A dialog box informs you that moving the harness removes it from the linked subsystem.
- 5 After confirmation, the harness appears with the library subsystem.

Edit Library Block from a Test Harness

You can apply an iterative design and test workflow to libraries by testing a library block in a test harness and updating the component under test. Changes to the component under test synchronize to the library when you close the test harness.

If you have a library block whose design is complete, set your test harnesses to prevent changes to the component under test. You can set this property when you create the test harness or after harness creation. See “Create or Import Test Harnesses and Select Properties” on page 2-13.

Testing a Library and a Linked Block

Verify a reusable subsystem in a library and in a larger system.

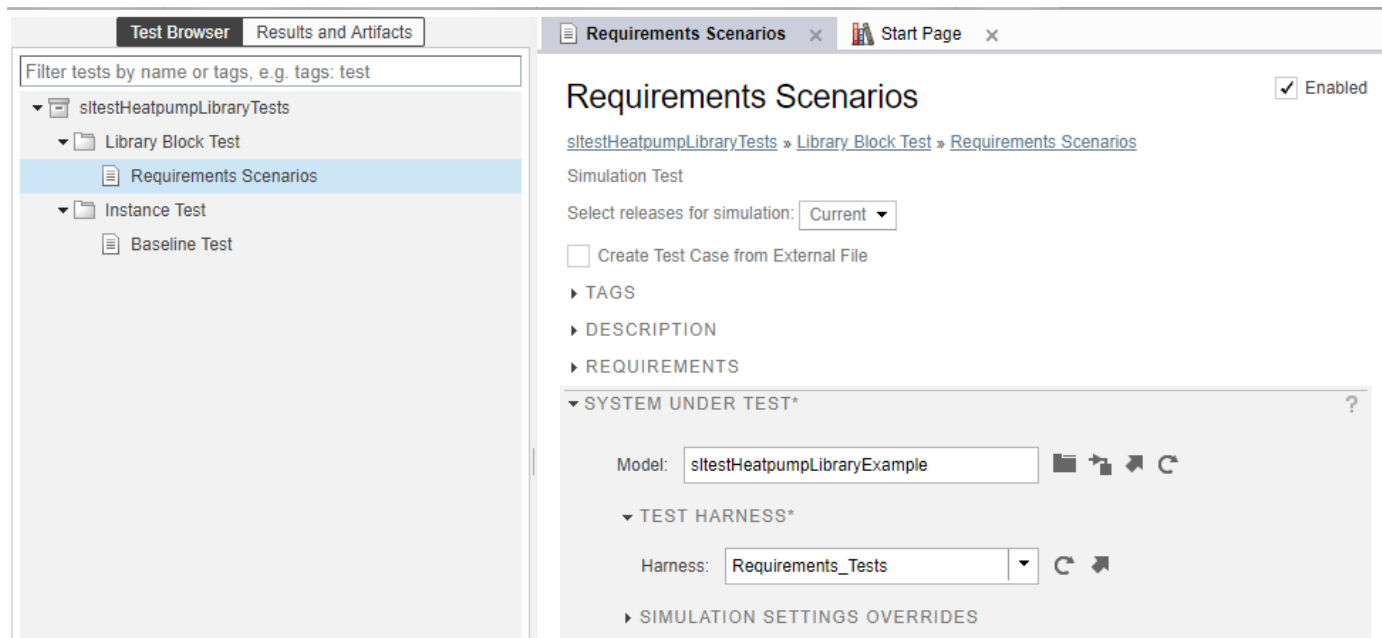
This example demonstrates a test case that confirms a library block meets a short set of requirements. After testing the library block, you execute a baseline test of a linked block and capture the baseline results. You then promote the baseline test harness to the library.

The library block controls a simple heat pump system by supplying on/off signals to a fan and compressor, and specifying the heat pump mode (heating or cooling).

Open the Test File

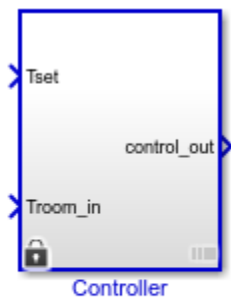
Enter the following to store paths and filenames for the example, and to open the test file. The test file contains a test case for the library block and for the block instance in a closed-loop model.

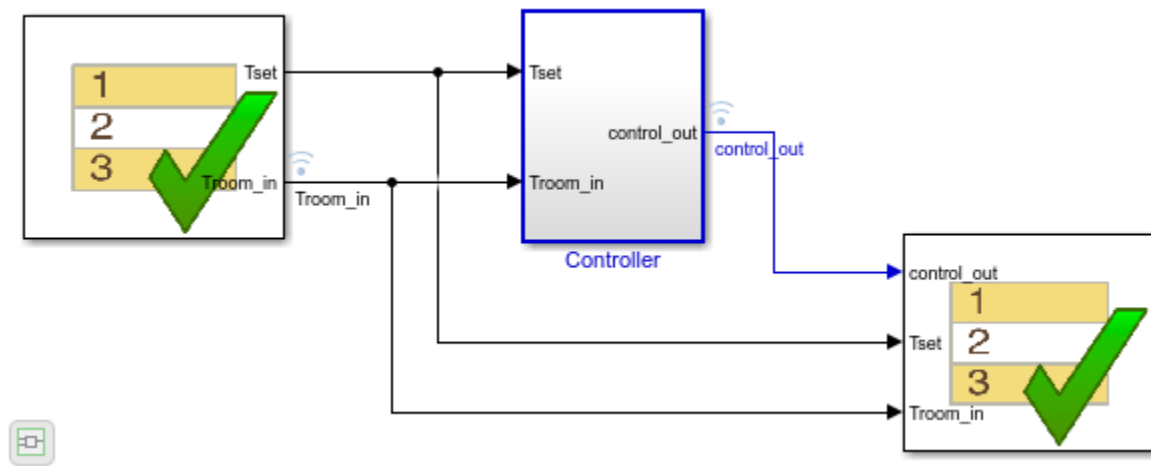
```
testFile = 'sltestHeatpumpLibraryTests.mldatx';
library = 'sltestHeatpumpLibraryExample';
system = 'sltestHeatpumpLibraryLinkExample';
sltest.testmanager.load(testFile);
sltest.testmanager.view;
```



Expand the **Library Block Test** test suite, and highlight the **Requirements Scenarios** test case in the test browser. Expand the **Test Harness** section of **System Under Test**, and click the arrow to open the test harness for the library block.

```
open_system(library);
sltest.harness.open([library '/Controller'], 'Requirements_Tests');
```





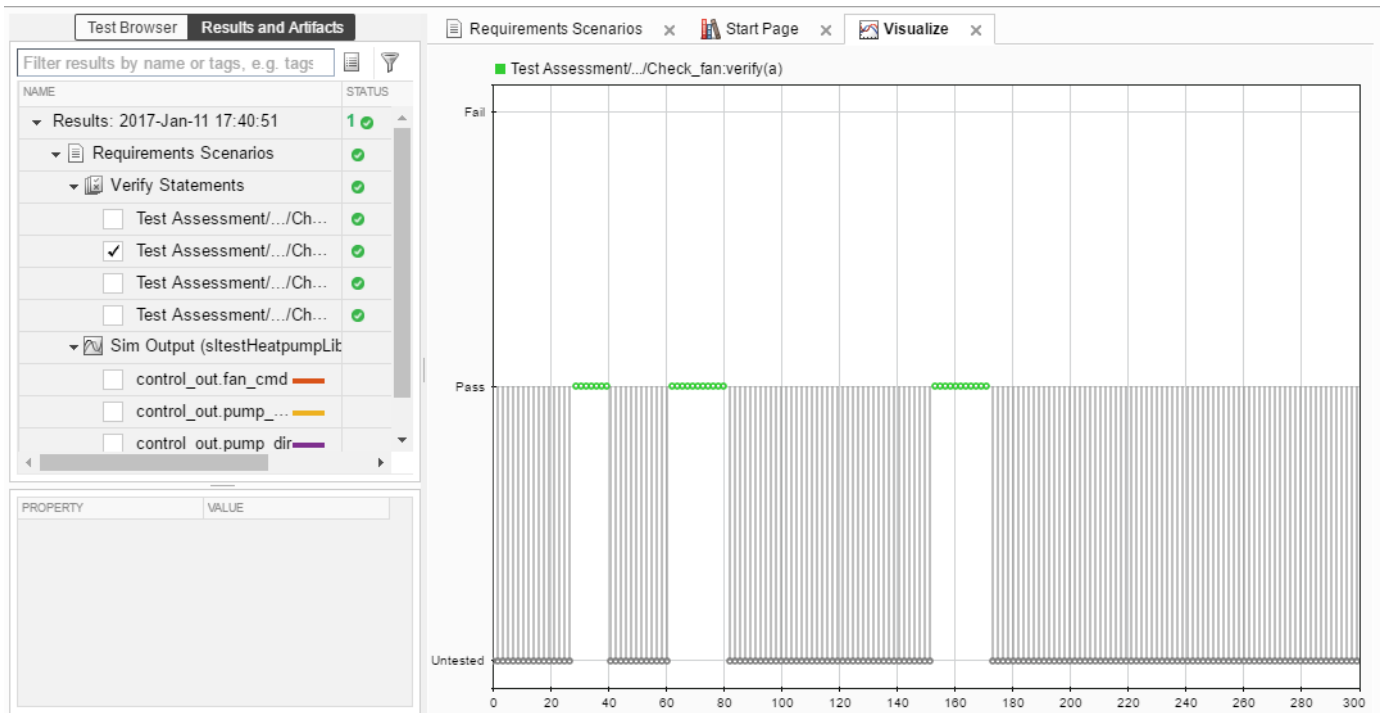
The Test Sequence block sets three scenarios for the controller:

- The controller at idle
- The controller activating the fan only
- The controller activating the heating and cooling system

The Test Assessment block in the test harness checks the signals for each scenario. Since the test inputs and assessments are contained in the test harness, and no baseline data is being captured, the test case is a simulation test.

Run the Requirements-Based Test

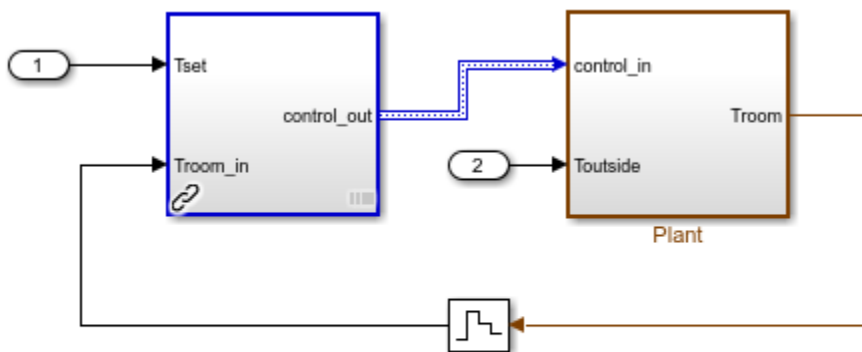
In the test manager, run the Requirements Scenarios test case. The `verify` statement results show that the `control_out` signals pass.



Open the Linked Block Model

In the test manager, expand **Instance Test**. Highlight the **Baseline Test** test case. In the **System Under Test**, click the arrow next to the **Model** field to open the model.

```
sltest.harness.close([library '/Controller'], 'Requirements_Tests');
open_system(system);
sim(system);
```



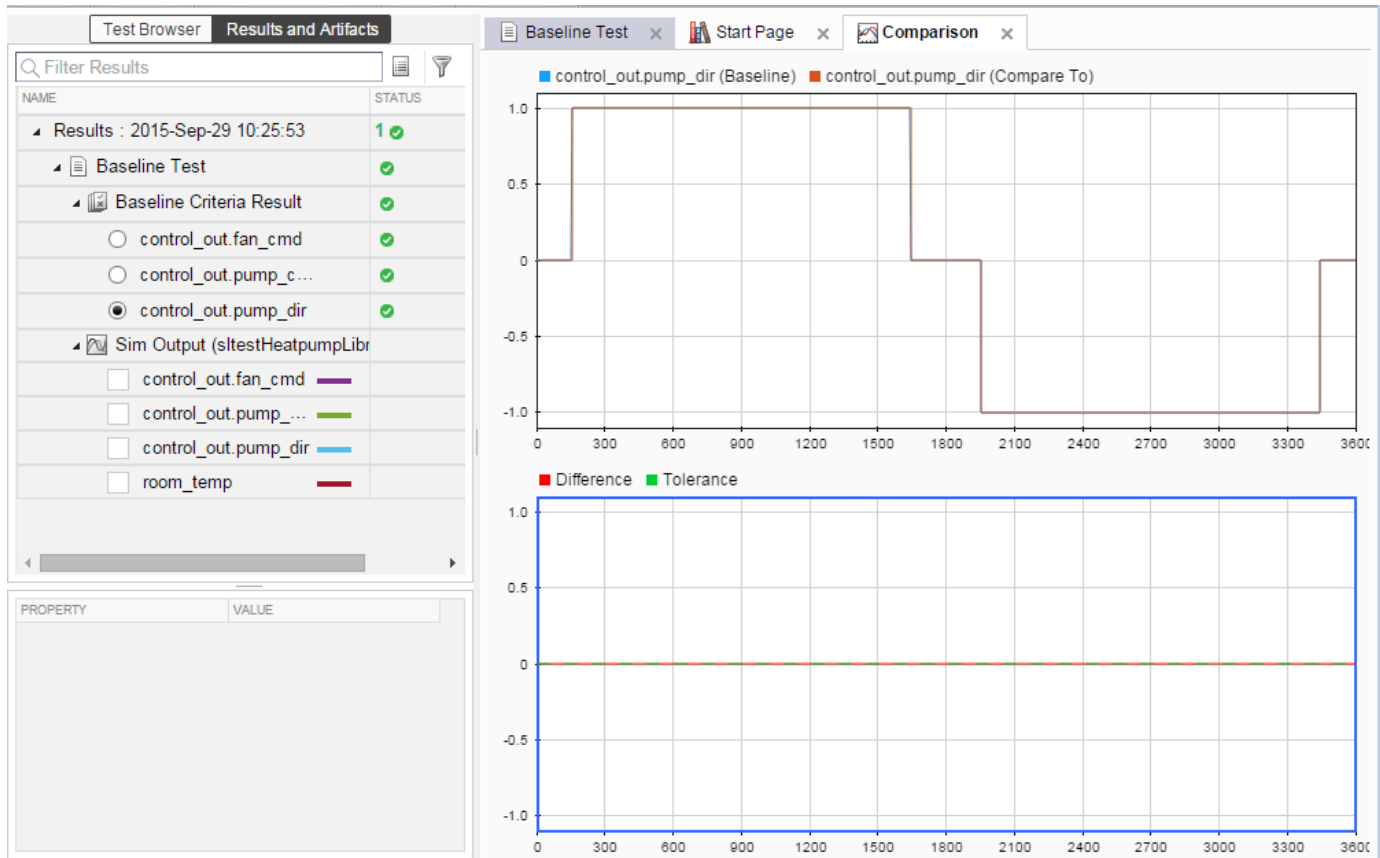
Copyright 1990-2014 The MathWorks, Inc.

The controller is a linked block to the library. It is associated with a test harness **Baseline Test** that compares simulation results of the instance against baseline data. In your workflow, successful baseline testing for an instances of a library block can show that the linked block simulates correctly

in the containing model. The test harness supplies a sine wave temperature and captures the controller output.

Run the Baseline Test and Observe Results

In the test manager, click **Run** to execute the test. The results show that the baseline test passes.



Move the Test Harness to the Library

If you develop a particularly useful test for a linked block, you can promote the test harness from a linked block to the source library block. The test harness then copies to all future instances of the library block.

Move the **Baseline_controller_tests** test harness to the library block:

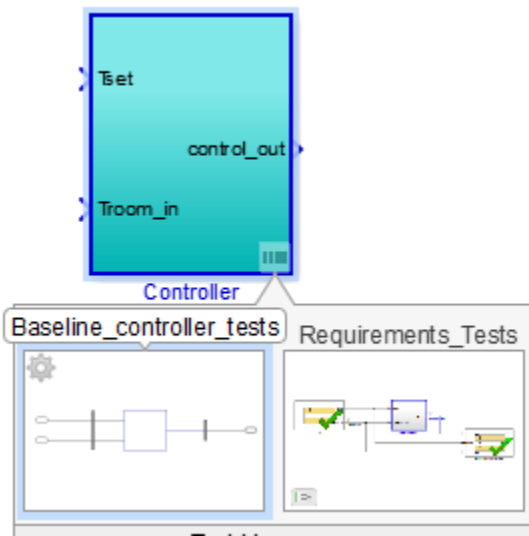
1. In the sltestHeatpumpLibraryLinkExample model, click the harness badge and hover over the **Baseline_controller_tests** test harness.

2. Click the harness operations icon



3. Select **Move to Library**. A dialog informs you that the operation deletes the test harness from the instance and adds it to the library. Click **Yes**.

4. The test harness moves to the Controller library block.



```
close_system(library,0);
close_system(system,0);
clear(library,system,testFile);
sltest.testmanager.clear;
sltest.testmanager.clearResults;
```

SIL Testing a Reusable Library Subsystem

This example shows how to unit test a reusable component in a library. It tests software-in-the-loop (SIL) code generated for a subsystem by using an equivalence test.

The reusable library subsystem must be at the top level of the library and must have function interfaces to lock down the subsystem interface. For information on reusable subsystem libraries, function interfaces, and workflow limitations, see “Library-Based Code Generation for Reusable Library Subsystems” (Embedded Coder).

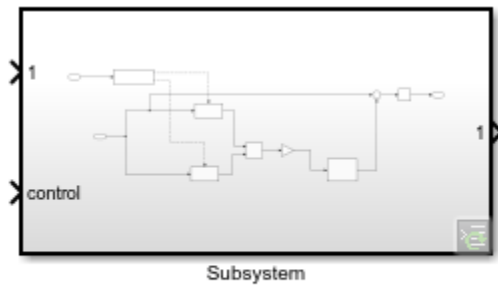
Set Up the Code Generation Environment

```
orig = Simulink.fileGenControl('get','CodeGenFolderStructure');
Simulink.fileGenControl('set','CodeGenFolderStructure',...
    Simulink.filegen.CodeGenFolderStructure.TargetEnvironmentSubfolder);
```

Open the Reusable Library

This library contains a subsystem block at the top level.

ReuseLibSubsysExample



Copyright 2019 The MathWorks, Inc.

Build the Library

Build the reusable library to generate code and create the function interfaces. After the code generation completes, you can view the function interfaces by clicking the lower right corner the library to open the Manage Function Interfaces dialog box.

```
slbuild('ReuseLibSubsysExample');
```

```
### Unable to find Simulink cache file "ReuseLibSubsysExample.slxc".
### Searching for referenced models in model 'Double'.
### Total of 1 models to build.
### Starting build procedure for: Double
### Generating code and artifacts to 'Target environment subfolder' folder structure
### Generating code into build folder: C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Double
### Invoking Target Language Compiler on Double.rtw
### Using System Target File: B:\runnable\matlab\rtw\c\ert\ert.tlc
### Loading TLC function libraries
.....
### Initial pass through model to cache user defined code
.
### Caching model source code
.....
### Writing source file Subsystem_gmjUKRe0.c
### Writing header file Double_types.h
.
### Writing header file Double.h
### Writing header file rtwtypes.h
### Writing header file Subsystem_gmjUKRe0.h
### Writing source file Double.c
.
### Writing header file Double_private.h
### Writing source file ert_main.c
### TLC code generation complete (took 7.909s).
### Saving binary information cache.
### Using toolchain: Microsoft Visual C++ 2022 v17.0 | nmake (64-bit Windows)
### Creating 'C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\_shared\rtwshared.mk' ...
### Using toolchain: Microsoft Visual C++ 2022 v17.0 | nmake (64-bit Windows)
### Creating 'C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Double\Double.mk' ...
### Successful completion of code generation for: Double
```

The following files will be copied from IntelWin64_shared to C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Double:

Subsystem_gmjUKRe0.c

```
Subsystem_gmjUKRe0.h
shared_file.dmr
```

```
Files copied from IntelWin64\_shared to C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Re
### Searching for referenced models in model 'Single'.
### Total of 1 models to build.
### Starting build procedure for: Single
### Generating code and artifacts to 'Target environment subfolder' folder structure
### Generating code into build folder: C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Sing
### Invoking Target Language Compiler on Single.rtw
### Using System Target File: B:\runnable\matlab\rtw\c\ert\ert.tlc
### Loading TLC function libraries
.....
### Initial pass through model to cache user defined code
.
### Caching model source code
.....
### Writing source file Subsystem_TZgYvBnF.c
### Writing header file Single_types.h
.
### Writing header file Single.h
### Writing header file Subsystem_TZgYvBnF.h
### Writing source file Single.c
### Writing header file Single_private.h
### Writing source file ert_main.c
.
### TLC code generation complete (took 4.089s).
### Saving binary information cache.
### Using toolchain: Microsoft Visual C++ 2022 v17.0 | nmake (64-bit Windows)
### Creating 'C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\_shared\rtwshared.mk' ...
### Using toolchain: Microsoft Visual C++ 2022 v17.0 | nmake (64-bit Windows)
### Creating 'C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Single\Single.mk' ...
### Successful completion of code generation for: Single
```

The following files will be copied from IntelWin64_shared to C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Single\

```
Subsystem_TZgYvBnF.c
Subsystem_TZgYvBnF.h
shared_file.dmr
```

Files copied from IntelWin64_shared to C:\TEMP\tpc773320b\simulinktest-ex98874249\IntelWin64\Re

Open the Test Manager

Click the Subsystem in the library model to select it. Then, open the Test Manager.

```
sltest.testmanager.view
```

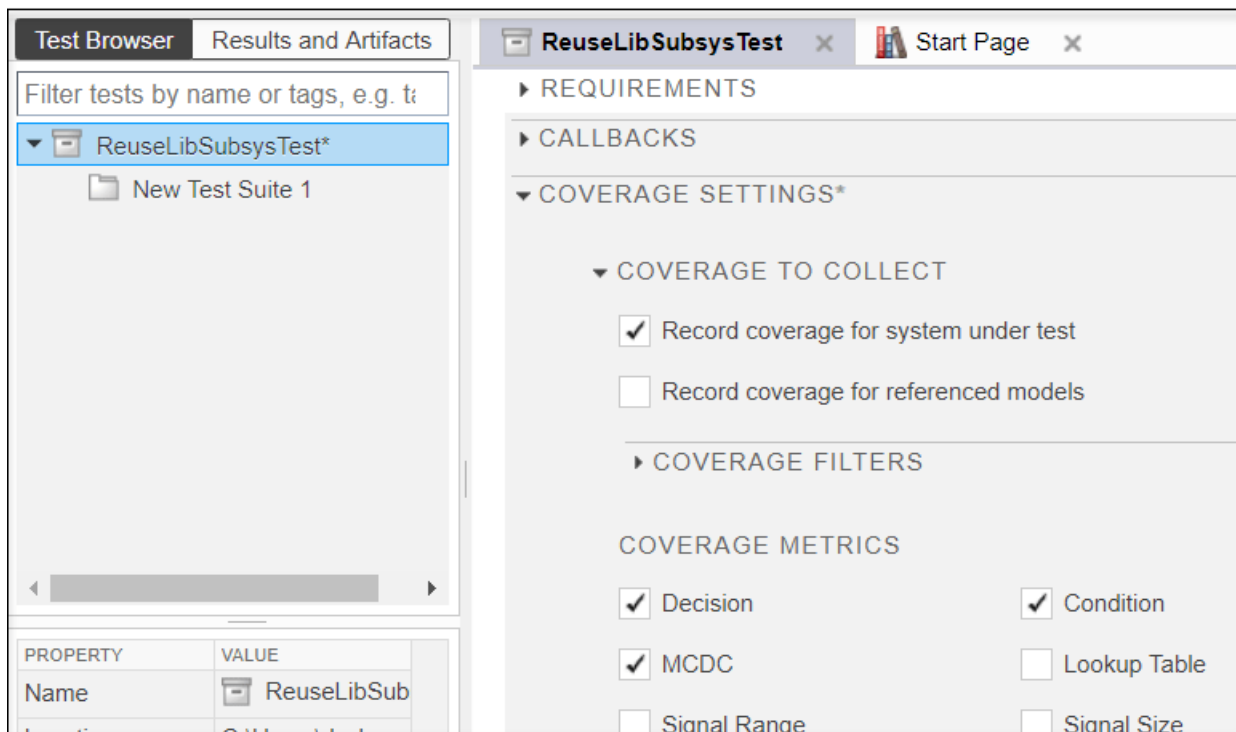
Open the Test File

- 1 In the Test Manager, click **Open** and select the `ReuseLibSubsysTest.mldatx` file.

Enable Coverage Collection

- 1 Select **ReuseLibSubsysTest** in the Test Browser pane.
- 2 Then, expand the Coverage Settings section in the main pane.
- 3 In Coverage to Collect, check **Record coverage for system under test** and **Record coverage for referenced models**.

- 4 In Coverage Metrics, check that **Decision**, **MCDC**, and **Condition** are selected.



Open the Create Test for Model Component wizard

- 1 Click **New > Test for Model Component** to open the Create Test for Model Component wizard.
- 2 On the **System** page of the wizard, click the second icon (Use current model) next to the **Top Model** field. Both the **Top Model** field and components hierarchy fill in.
- 3 Select **Subsystem** in the components hierarchy and click the plus sign. The Subsystem component is added to the **Selected components**.
- 4 Click the refresh icon next to **Select function interface** and select **Double** from the dropdown.
- 5 Click **Next**.

Top Model:

Search components

- ▼ ReuseLibSubsysExample
 - ▼ Subsystem
 - If Action Subsystem
 - If Action Subsystem1

Selected components

- ReuseLibSubsysExample/Subsystem

☐ Select all Subsystems

☐ Select all Model References

Use currently selected components

Select a function interface:

Select the Test Inputs

- 1 On the **Test Inputs** page, select **Use Design Verifier to generate test input scenarios**. Then click **Next**.

How do you want to set up the inputs?

- ☒ Use Design Verifier to generate test input scenarios

Create inputs using Simulink Design Verifier. [Design Verifier Settings](#)

☐ Simulate top model and use the recorded component inputs in the analysis

- ☐ Specify inputs in the created harness

Create a new test harness for component. Inputs should be added to the harness

Select How to Test the Component

- 1 On the **Verification Strategy** page, select **Perform back-to-back testing**.
- 2 Set **Simulation1** to Normal mode.
- 3 Set **Simulation2** to Software-in-the-Loop (SIL) mode.
- 4 Click **Next**.

How do you want to test the component?

- ☐ Use component under test output as baseline

Simulate the top model and record the outputs of the component to be used as baseline

- ☒ Perform back-to-back testing

Set up a test to compare the component under test outputs in different simulation modes

Select simulation modes:

Simulation1:

Simulation2:

☒ Set Model coverage objectives as Enhanced MCDC

- ☐ Define the verification logic in the created harness

No verification logic will be automatically added to the test

Specify the Input Source, Data File Format, and Test File

- 1 On the **Generated Test** page, select Inports as the test harness input source.
- 2 Select MAT as the file format.
- 3 Select **Add tests to currently selected test file**.

How do you want to save the test data?

Select test harness input source: Specify the file format

Inports MAT

Specify location to save test data:

sltest_ReuseLibSubsysExample 

Where do you want to save the generated test(s)?

☒ Add tests to the currently selected test file.

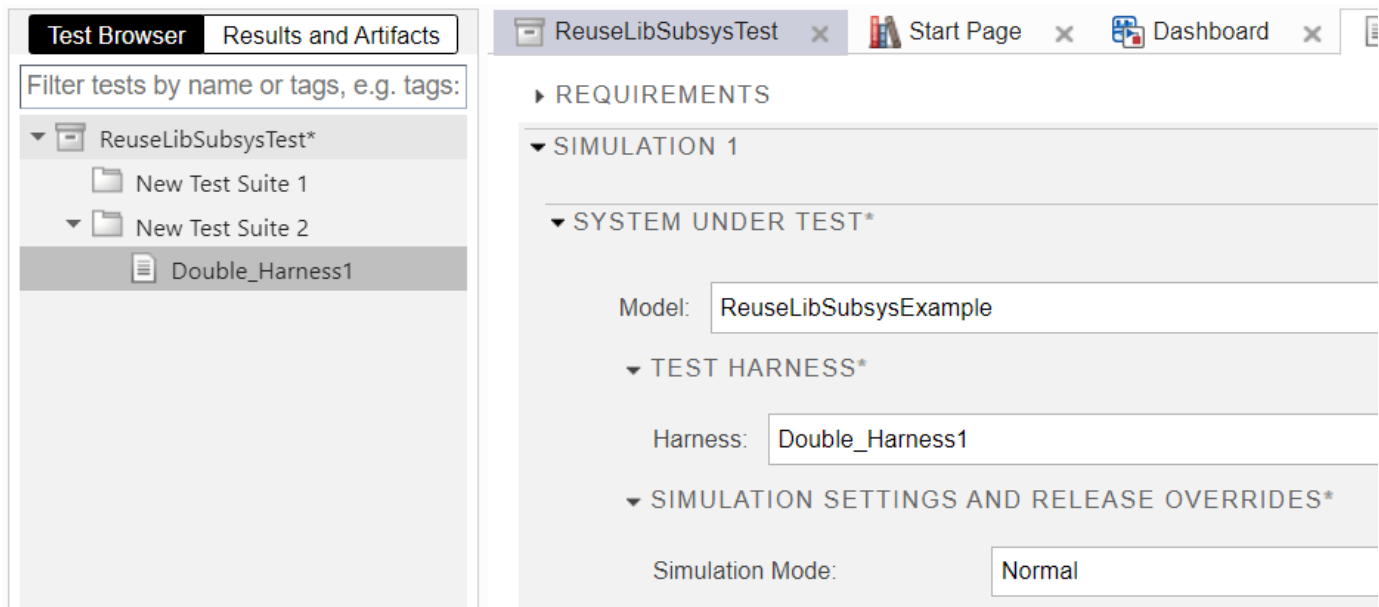
☐ Create a new test file containing the test(s).

Test File Location: *sltest_ReuseLibSubsysExample_tests* 

Generate the Test Case and Return to the Test Manager

Click **Done** to generate the Double_Harness1 test case for the Double function interface.

After test case generation completes, the Double_Harness1 test case appears in the **Test Browser** pane of the Test Manager. To check that the **Simulation Modes** are set correctly, expand Simulation 1 > System Under Test > Simulation Settings and Release Overrides. Expand Simulation 2 in the same way. The **Simulation Mode** for Simulation 1 is set to Normal and Simulation 2 is set to Software-in-the-Loop (SIL).



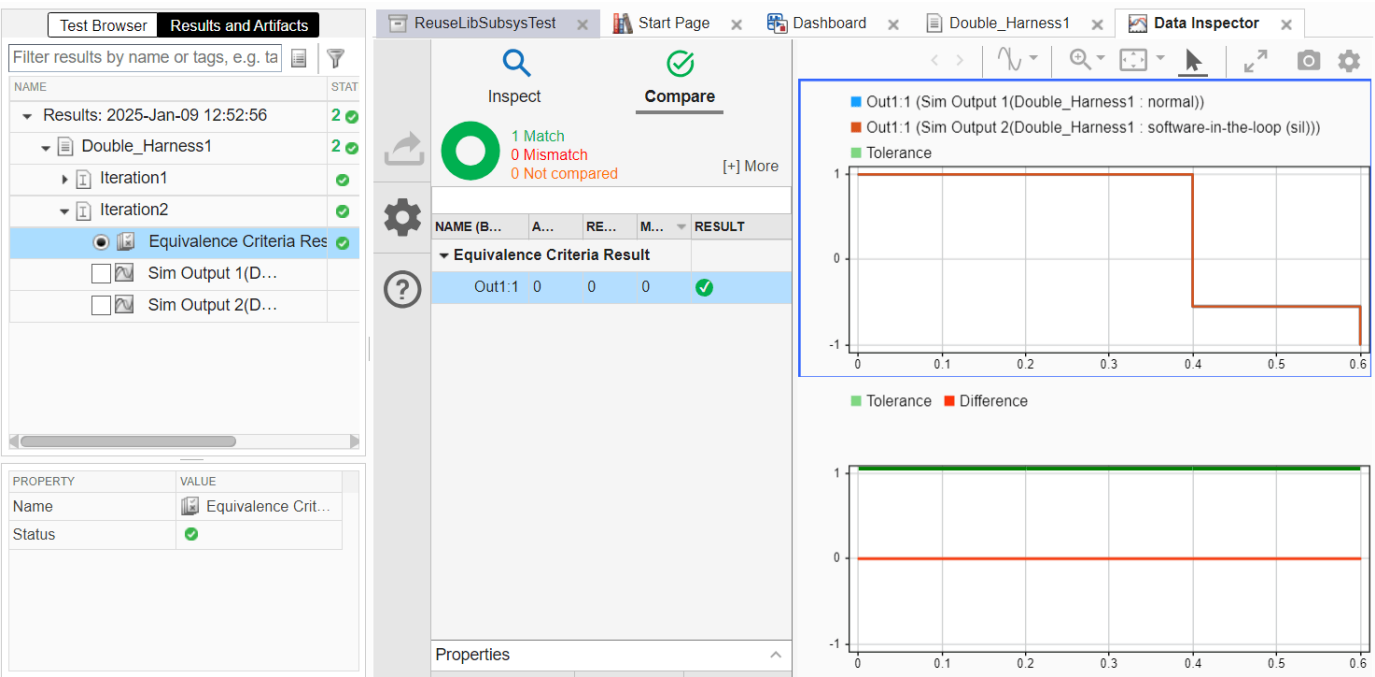
Run the Test

Click **Run** in the Test Manager toolbar.

View Equivalence Results

- 1 Expand the Results > Double_Harness1 > Iteration2.
- 2 Select **Equivalence Criteria Result**.
- 3 In the **Compare** pane, select the Out1:1 signal if it is not already selected.

Notice that the Difference plot shows no differences between the two signals, which indicates that the Normal and SIL simulations produce the same results. You can reuse the SIL code associated with the reusable library subsystem for other SIL tests.

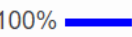
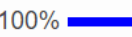
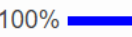
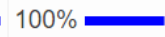


View Coverage Results

Select **Results**(in the ***Results and Artifacts** pane. The **Aggregated Coverage Results** section shows the combined model coverage. The two coverage results that show 100% are for the equivalence test run for the Double function interface. The other result shows 0% because the Single function interface was not tested.

▼ AGGREGATED COVERAGE RESULTS

Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.

ANALYZED MODEL	REPORT	SIM MODE	COM...	DECISION	EXECUTION	FUNCTION	+
 ReuseLibSubsysExample/Subsystem		Normal	4	100% 	100% 	--	
Subsystem_gmjUKRe0.c		SIL	6	100% 	100% 	100% 	
Subsystem_TZgYvBnF.c		SIL	6	0% 	0% 	0% 	

Clean up

```
Simulink.fileGenControl('set','CodeGenFolderStructure',orig);
```

See Also

More About

- “Library-Based Code Generation for Reusable Library Subsystems” (Embedded Coder)
- “Generate Reusable Code from Library Subsystems Shared Across Models” (Embedded Coder)

- “Create or Import Test Harnesses and Select Properties” on page 2-13
- “Import Test Cases for Equivalence Testing” on page 5-16
- “Generate Tests and Test Harnesses for a Model or Components” on page 6-20

Test Sequences and Assessments

- “Test Sequence Basics” on page 3-2
- “Use Stateflow Chart for Test Harness Inputs and Scheduling” on page 3-5
- “Assess Simulation and Compare Output Data” on page 3-11
- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Author a Test Sequence and Test Assessment” on page 3-22
- “Test Sequence Editor” on page 3-29
- “Transitions, Temporal Operators, and Messages in Test Sequence Blocks” on page 3-36
- “Generate Test Signals” on page 3-42
- “Using an External Function in a Test Sequence Block” on page 3-45
- “Programmatically Create a Test Sequence” on page 3-47
- “Programmatically Create and Run Test Sequence Scenarios” on page 3-51
- “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54
- “Scenario Parameter Section” on page 3-62
- “Define Test Steps and Assessments” on page 3-63
- “Debug a Test Sequence” on page 3-67
- “Test Downshift Points of a Transmission Controller” on page 3-70
- “Examine Model Verification Results by Using Simulation Data Inspector” on page 3-75
- “Fix Requirements-Based Testing Issues” on page 3-79
- “Assess Temporal Logic by Using Temporal Assessments” on page 3-85
- “Test Traffic Light Control by Using Logical and Temporal Assessments” on page 3-91
- “Logical and Temporal Assessment Syntax” on page 3-99

Test Sequence Basics

In this section...

“Test Sequence Hierarchy” on page 3-2

“Test Sequence Scenarios” on page 3-2

“Transition Types” on page 3-2

A test sequence consists of test steps arranged in a hierarchy. You can use a test sequence to define test inputs and to define how a test progresses in response to the simulation. A test step contains actions that execute at the beginning of the step. A test step can contain transitions that define when the step stops executing, and which test step executes next. Actions and transitions use MATLAB as the action language. You create test sequences by using the Test Sequence block and the Test Sequence Editor on page 3-29. See “Use Stateflow Chart for Test Harness Inputs and Scheduling” on page 3-5.

Test Sequence Hierarchy

Test sequences defined in Test Sequence blocks can have parent steps and substeps. Substeps can activate only if the parent step is active. A group of steps in the same hierarchy level shares a common transition type. When you create a test step, the step becomes a transition option for other steps in the same group.

Test Sequence Scenarios

In a Test Sequence block, you can define multiple test sequences, which are called test sequence scenarios. By using scenarios, you can define distinct test sequences without having multiple Test Sequence blocks in your test harness. You can run test sequence scenarios in these ways:

- Activate a single scenario from the Test Sequence Editor and run the model
- Activate a single scenario using API commands and run the model
- Control the active scenario with a workspace variable and run the model
- Use a custom test script to loop through scenarios when running the model
- Define iterations in the Test Manager to run more than one scenario in a single test case

For more information and examples of using test sequence scenarios, see “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54 and “Programmatically Create and Run Test Sequence Scenarios” on page 3-51.

Transition Types

Test sequences defined in Test Sequence blocks transition from one step to another in two ways:

- **Standard transition:** You can define a sequence of actions that react to simulation conditions using a standard transition sequence. Standard transition sequences start with the first step and progress according to transition conditions and next steps. When there are multiple transitions in a step, the transitions are executed in the order in which they are listed. For more information on transitions, see “Define Test Steps and Assessments” on page 3-63.

This test sequence sets the value of Boolean outputs RedButtonIn and GreenButtonIn, with transitions happening after each step has been active for 1 sec.

Step	Transition	Next Step
PressNeitherButton RedButtonIN = false; GreenButtonIN = false;	1. after(1,sec)	PressBothButtons ▼
PressBothButtons RedButtonIN = true; GreenButtonIN = true;	1. after(1,sec)	PressRedButton ▼
PressRedButton RedButtonIN = true; GreenButtonIN = false;	1. after(1,sec)	PressGreenButton ▼
PressGreenButton RedButtonIN = false; GreenButtonIN = true;	1. after(1,sec)	EndTest ▼
EndTest		

- When decomposition:** When decomposition sequences are analogous to switch statements in programming. In a When decomposition sequence, steps activate based on specific conditions occurring in your model. You do not use transitions between steps. At each time step, the block evaluates the condition expression for each step sequentially. When the block reaches a step whose condition evaluates to true, the step becomes active and the subsequent steps remain untested.

This When decomposition contains three verify statements. Each verify statement is active when the signal gear is equal to a different value. For more information, see “Verify Model Simulation by Using when Decomposition” on page 3-14.

Symbols	Step	Transition	Next Step
Input 1. speed 2. throttle 3. gear Output Local Constant Parameter Data Store Memory	Assessments Check1st when gear == 1 verify(speed < 45) Check2nd when gear == 2 verify(speed < 75) Check3rd when gear == 3 verify(speed < 105) Else		

See Also

Test Sequence

More About

- “Test Sequence Editor” on page 3-29
- “Define Test Steps and Assessments” on page 3-63
- “Author a Test Sequence and Test Assessment” on page 3-22
- “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54
- “Programmatically Create and Run Test Sequence Scenarios” on page 3-51
- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Use Stateflow Chart for Test Harness Inputs and Scheduling” on page 3-5

Use Stateflow Chart for Test Harness Inputs and Scheduling

In this section...

“Use a Stateflow Chart for Test Harness Scheduling” on page 3-5

“Use a Stateflow Chart as a Test Harness Source” on page 3-6

“Stateflow Chart as Test Harness Scheduler and Source” on page 3-6

Use a Stateflow Chart for Test Harness Scheduling

You can define test harness scheduling using a Test Sequence block, a MATLAB Function block, or a Stateflow chart. If you use a Stateflow chart as a scheduler, you can use Stateflow features that are not available with either the Test Sequence block or MATLAB Function block. You can define more complicated scheduling by using Stateflow variants, graphical functions, super transitions, and super steps. For example, with Stateflow variants, you can specify multiple test scenarios in a single test harness. If you do not need to test multiple test scenarios or use complicated sequence logic, use the Test Sequence block, which has simpler syntax for test scheduling.

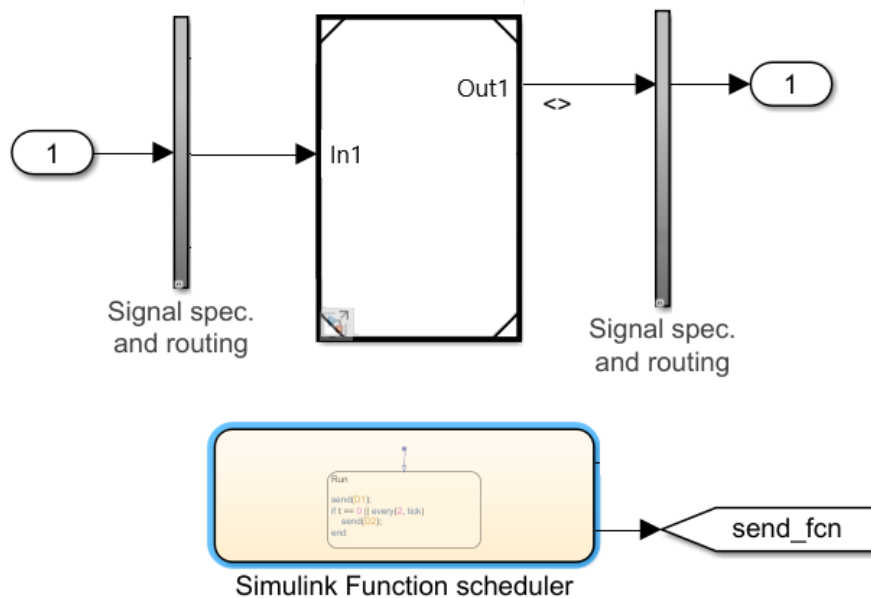
Note You must have a Stateflow license to use a chart for test harness inputs or scheduling.

To use a Stateflow chart as a test harness test scheduler, the model or subsystem under test must have at least one function call signal.

When setting up a test harness from a model, the steps for using a chart as the scheduler are:

- 1 In a model or subsystem, right-click and select **Test Harness > Create for Model** or **Create for <subsystem>**, respectively.
 - For a model, in the Create a Test Harness dialog box, set **Add scheduler for function-calls and rates** to Chart.
 - For a subsystem, in the Create a Test Harness dialog box, set **Generate function call signals** to Chart.

A chart named Simulink Function scheduler is added to the test harness.



- 2 Open the Stateflow chart and define the test sequence using Stateflow states, transitions and other objects. The Stateflow states serve the same purpose as the sequence steps in a Test Sequence block. The transitions define the criteria for moving from one state to another.

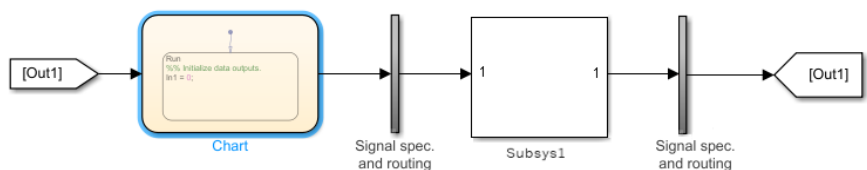
To programmatically specify a Stateflow chart as a scheduler, set the SchedulerBlock property of `sltest.harness.create` to `Chart`.

Use a Stateflow Chart as a Test Harness Source

When creating a test harness from a model, the steps for using a chart as the test harness source are:

- 1 In a model or subsystem, right-click and select **Test Harness > Create for Model** or **Create for <subsystem>**, respectively.
- 2 In the Create Test Harness dialog box, in the **Sources and Sinks** section, select **Chart** instead of **Inport**.

A chart is added to the test harness. For example,



- 3 Open the Stateflow chart and define the test harness sources using Stateflow logic.

To programmatically specify a Stateflow chart as a source, set the Source property of `sltest.harness.create` to `Chart`.

Stateflow Chart as Test Harness Scheduler and Source

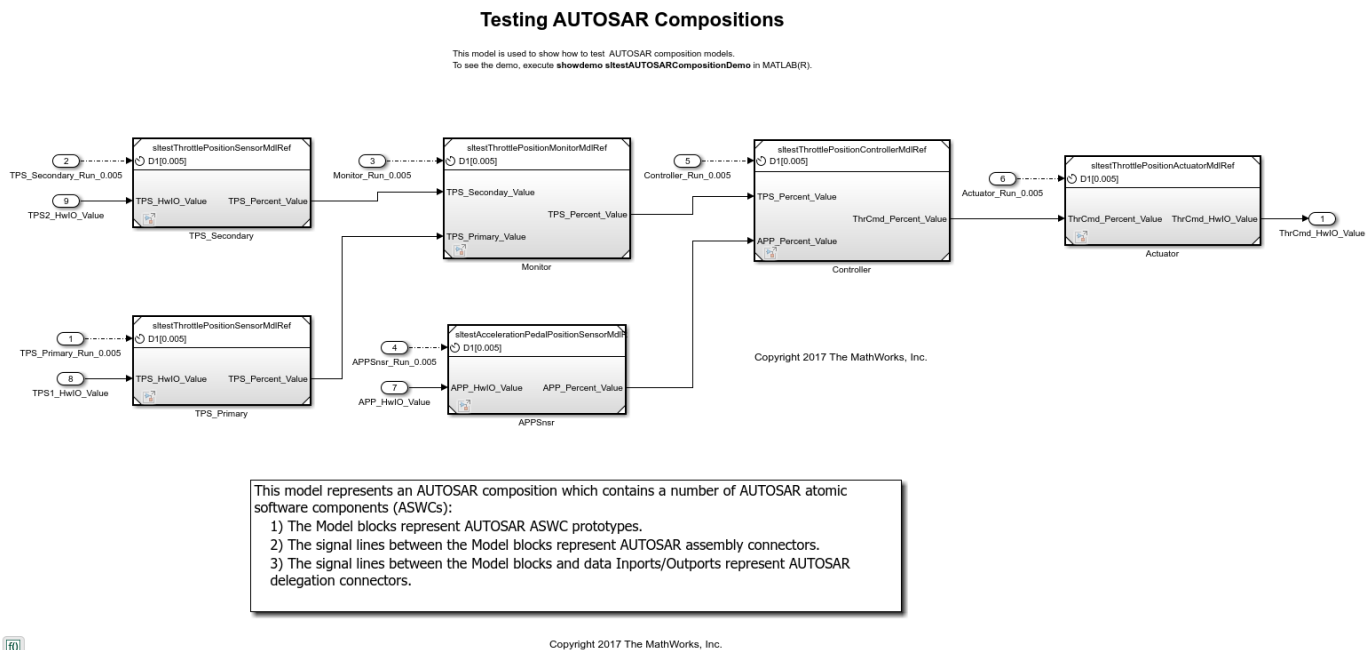
This example shows how to use a single Stateflow chart as both a test scheduler and source in a test harness. The test harness for the `sltest_autosar_chart.slx` model in this example has already been created.

`sltest_autosar_chart` is an AUTOSAR composition model of a throttle position controller for an automobile. AUTOSAR composition models contain a network of interconnected Model blocks, each of which represents an atomic AUTOSAR software component (ASWC). The Simulink inports and outports represent AUTOSAR ports. The signal lines represent AUTOSAR component connectors.

The inputs that capture the primary and secondary throttle position are modeled using an external time series input and are directly fed through the Chart without modification. This modeling style is useful when some stimulus inputs can be modeled and others are only available as externally captured data.

Navigate to a directory with write permissions before running this example.

Open the Model



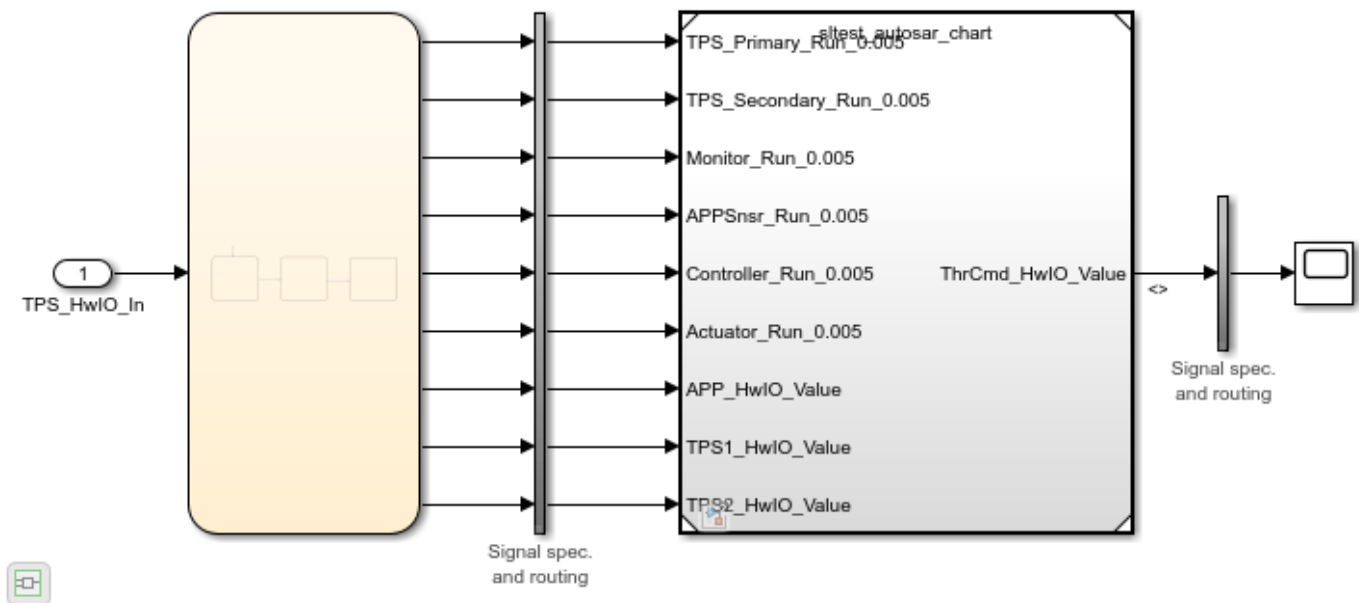
Open the Test Harness

The test harness has already been created for this example.

This image shows the portion of the Create Test Harness dialog where Chart was selected as both the source and scheduler. You do not need to recreate the test harness.

Basic Properties	Advanced Properties	Description
Name: sltest_autosar_chart_Harness1		
<i>Harnesses saved internally. More information</i>		
Sources and Sinks		
Chart	↔	Component under Test
☐ Create scalar inputs		
Add scheduler for function-calls and rates: Chart		

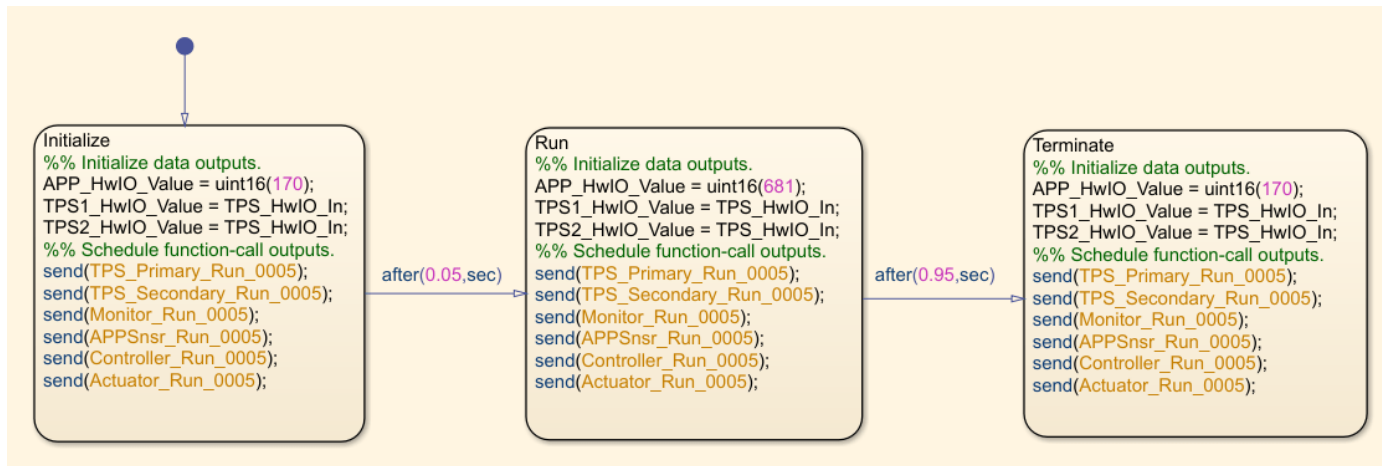
To open the harness, use the perspective control in the lower-right corner of the editor canvas and click **Internal Test Harness**.



The test harness opens.

Open the Stateflow Chart

Double-click the chart in the test harness to view the scheduling logic.



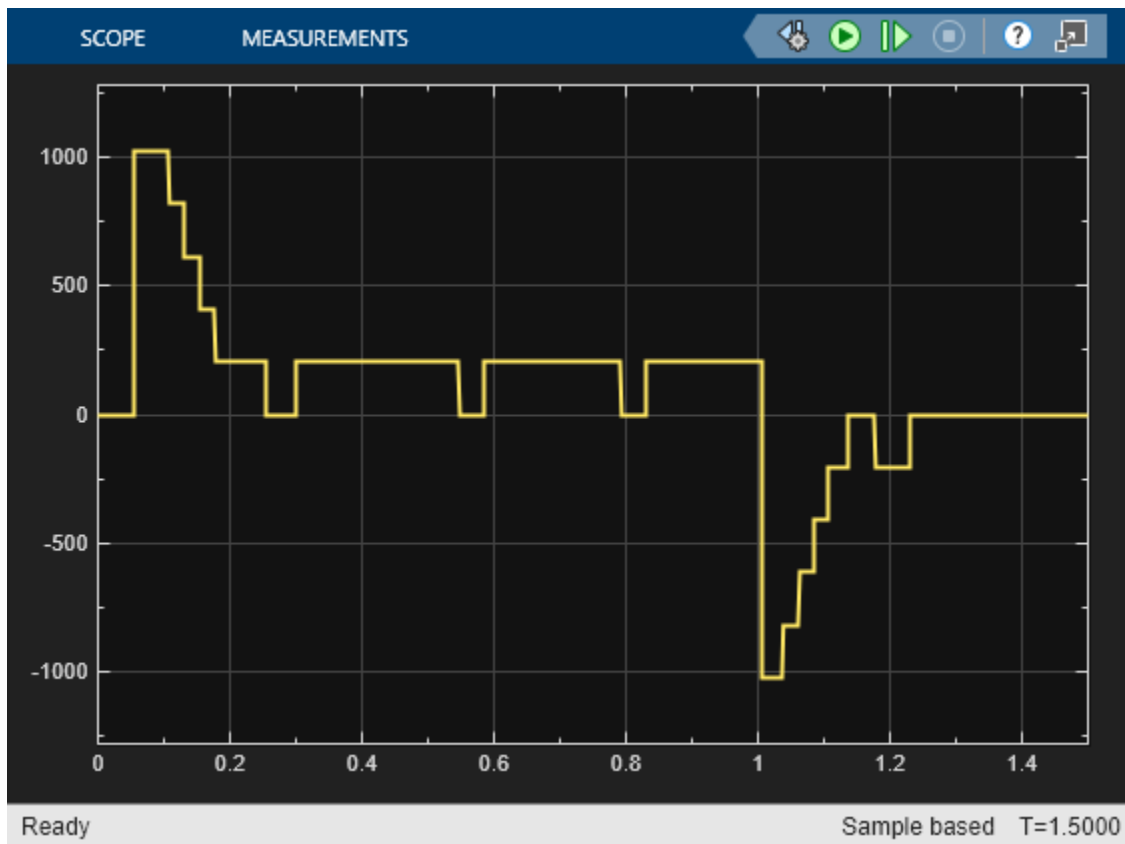
The component under test (the AUTOSAR model) requires the accelerator pedal position sensor input APP_HwIO_Value, which is modeled in the chart by three states.

The **Initialize** state sets the input to a nominal value (170) and the **Run** state models a steady acceleration command for 950 ms. The acceleration command is reset to the nominal value in the **Terminate** state.

The component under test uses the export-function modeling style. (See “Export-Function Models Overview”.) When the test harness was created, its Stateflow chart was configured to call each root-level Simulink Function block and send a trigger event to each function-call subsystem in the model. In this example, the code to send the trigger events is in each state after the stimulus waveforms have been generated.

Run the Model

Run the model from the test harness. To see the throttle command output, open the Scope in the test harness.



See Also

`sltest.harness.create` | Test Sequence | Function Caller

More About

- “Chart Programming” (Stateflow)
- “Transitions, Temporal Operators, and Messages in Test Sequence Blocks” on page 3-36
- “Create or Import Test Harnesses and Select Properties” on page 2-13
- “Test Sequence Basics” on page 3-2
- “Test Sequence Editor” on page 3-29

Assess Simulation and Compare Output Data

In this section...
“Compare Simulation Data to Baseline Data or Another Simulation” on page 3-11 “Post-Process Results With a Custom Script” on page 3-12 “Run-Time Assessments” on page 3-12 “Logical and Temporal Assessments” on page 3-13

Functional testing requires assessing simulation behavior and comparing simulation output to expected output. For example, you can:

- Analyze signal behavior in a time interval after an event.
- Compare two variables during simulation.
- Compare timeseries data to a baseline.
- Find peaks in timeseries data, and compare the peaks to a pattern.

This topic provides an overview to help you author assessments for your particular application. In the topic, you can find links to more detailed examples of each assessment.

You can include assessments in a test case, a model, or a test harness.

- In a test case, you can:
 - Compare simulation output to baseline data.
 - Compare the output of two simulations.
 - Post-process simulation output using a custom script.
 - Assess temporal properties using logical and temporal assessments. If you have one or more defined assessments and their associated symbols in a test case, you can use the API or the Test Manager to obtain a list and information about them, copy them to another test case, and remove them from a test case. For information on using the API, see `sltest.testmanager.Assessment`, `sltest.testmanager.AssessmentSymbol`, and `sltest.testmanager.TestCase`. For the Test Manager, see “Assess Temporal Logic by Using Temporal Assessments” on page 3-85.
- In a test harness or model, you can:
 - Verify logical conditions in run-time using a `verify` statement, which returns a `pass`, `fail`, or `untested` result for each time step.
 - Use `assert` statements to stop simulation on a failure.
 - Use blocks from the Model Verification or Simulink Design Verifier library.

Compare Simulation Data to Baseline Data or Another Simulation

Baseline criteria are tolerances for simulation data compared to baseline data. Equivalence criteria are tolerances for two sets of simulation data, each from a different simulation. You can set tolerances for numeric, enumerated, or logical data.

Set a numeric tolerance using absolute or relative tolerances. Set time tolerances using leading and lagging tolerances. For numeric data, you can specify absolute tolerance, relative tolerance, leading

tolerance, or lagging tolerance. For enumerated or logical data, you can specify leading or lagging tolerance. Results outside the tolerances fail. For more information, see “Set Signal Tolerances” on page 6-169.

Specify the baseline data and tolerances in the Test Manager **Baseline Criteria** or **Equivalence Criteria** section. Results appear in the **Results and Artifacts** pane. The comparison plot displays the data and differences.

This graphic shows an example of baseline criteria. The baseline criteria sets a relative tolerance for signals output torque and vehicle speed.

SIGNAL NAME ▲	ABS TOL	REL TOL	LD TOL	LG TOL
▼ ☐ BrakeThrottleBaseline3.mat	0	0.10%	0	0
☒ output torque	0	0.10%	0	0
☒ vehicle speed	0	0.10%	0	0

Post-Process Results With a Custom Script

You can analyze simulation data using specialized functions by using a custom criteria script. For example, you could find peaks in timeseries data using Curve Fitting Toolbox™ functions. A custom criteria script is MATLAB code that runs after the simulation. Custom criteria scripts use the MATLAB Unit Test framework.

Write a custom criteria script in the Test Manager **Custom Criteria** section of the test case. Custom criteria results appear in the **Results and Artifacts** pane. Results are shown for individual MATLAB Unit Test qualifications. For more information, see “Process Test Results with Custom Scripts” on page 6-197.

This simple test case custom criteria verifies that the value of slope is greater than 0.

```
% A simple custom criteria
test.verifyGreaterThan(slope,0,'slope must be greater than 0')
```

Run-Time Assessments

verify Statements

For run-time assessments, use `verify` statements in a Test Assessment or Test Sequence block. A `verify` statement evaluates a logical expression and returns a pass, fail, or untested result for each simulation time step. `verify` statements can include temporal and conditional syntax. A failure does not stop simulation.

For information about using `verify` statements in your model, see “Verify Model Simulation by Using when Decomposition” on page 3-14 and `verify`.

assert Statements

Similar to `verify` statements, `assert` statements evaluate logical expressions, but if the test is invalid, simulation stops. To make results easier to interpret, add an optional message. For more information, see `assert`.

Model Verification Blocks

Use blocks from the Simulink “Model Verification” library or the Simulink Design Verifier library to assess signals in your model or test harness. **pass**, **fail**, or **untested** results from each block appear in the Test Manager. For more information, see “Examine Model Verification Results by Using Simulation Data Inspector” on page 3-75.

Note All Model Verification library blocks, including the Assertion block, do not produce verification results when used in For Each subsystems. Use a Test Sequence block with **verify** statements instead.

Logical and Temporal Assessments

Logical and temporal assessments evaluate temporal properties such as model timing and event ordering over logged data. Use temporal assessments for additional system verification after the simulation is complete. Temporal assessments are associated with test cases in the Test Manager. Author temporal assessments by using the **Logical and Temporal Assessments Editor**. See “Assess Temporal Logic by Using Temporal Assessments” on page 3-85 for more information.

Temporal assessment evaluation results appear in the **Results and Artifacts** pane. Use the Expression Tree to investigate results in detail. If you have a Requirements Toolbox license, you can establish traceability between requirements and temporal assessments by creating requirement links. See “Link to Requirements” on page 1-2 for more information.

See Also

Related Examples

- “Compare Simulation to Baseline Data” on page 6-246
- “Back-to-Back Equivalence Testing” on page 6-36

Verify Model Simulation by Using when Decomposition

In a Test Assessment block, you can define the simulation condition that activates a step that contains a **verify** statement by using a when decomposition sequence. In a when decomposition sequence, steps activate based on the simulation conditions that you define. For more information about when decomposition sequences, see “Test Sequence Basics” on page 3-2.

verify statements evaluate logical expressions and return a **pass**, **fail**, or **untested** result for each time step and for the overall simulation. A **fail** result at any time step results in a **fail** result for the overall simulation. If no failing results occur, a **pass** at any time step results in a **pass** result for the overall simulation. Otherwise, the overall result is **untested**. You can then review the test results in the Test Manager or the Simulation Data Inspector if you run the simulation without a test case. You can choose to display only **pass** and **fail** statements by logging only tested **verify** statement results. For more information on authoring **verify** statements, see **verify**.

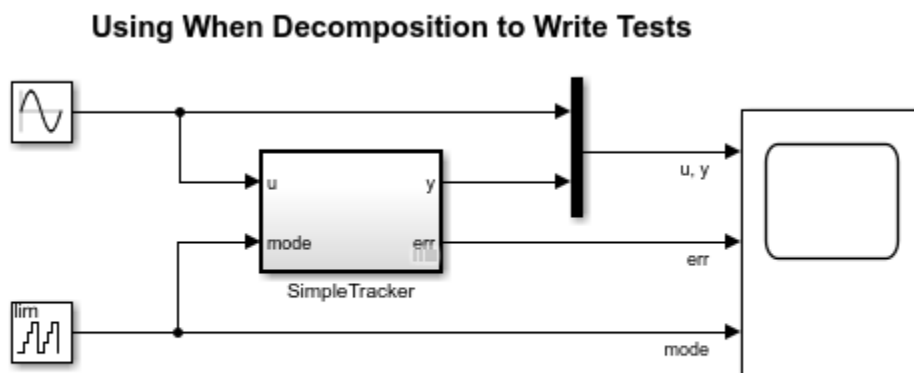
You can author **verify** statements in a when decomposition:

- If your model does not use a Test Sequence block as a source, or your test sequence steps do not correspond with the conditions to verify. In this situation, you can activate each **verify** statement using a signal condition.
- If your model uses a Test Sequence block as a source. In this situation, you can activate the **verify** statements in a Test Assessment block by using the active Test Sequence block step. See “Use Active Step Output as Input to Another Block” on page 3-35.

Activate **verify** Statements by Using Signal Conditions

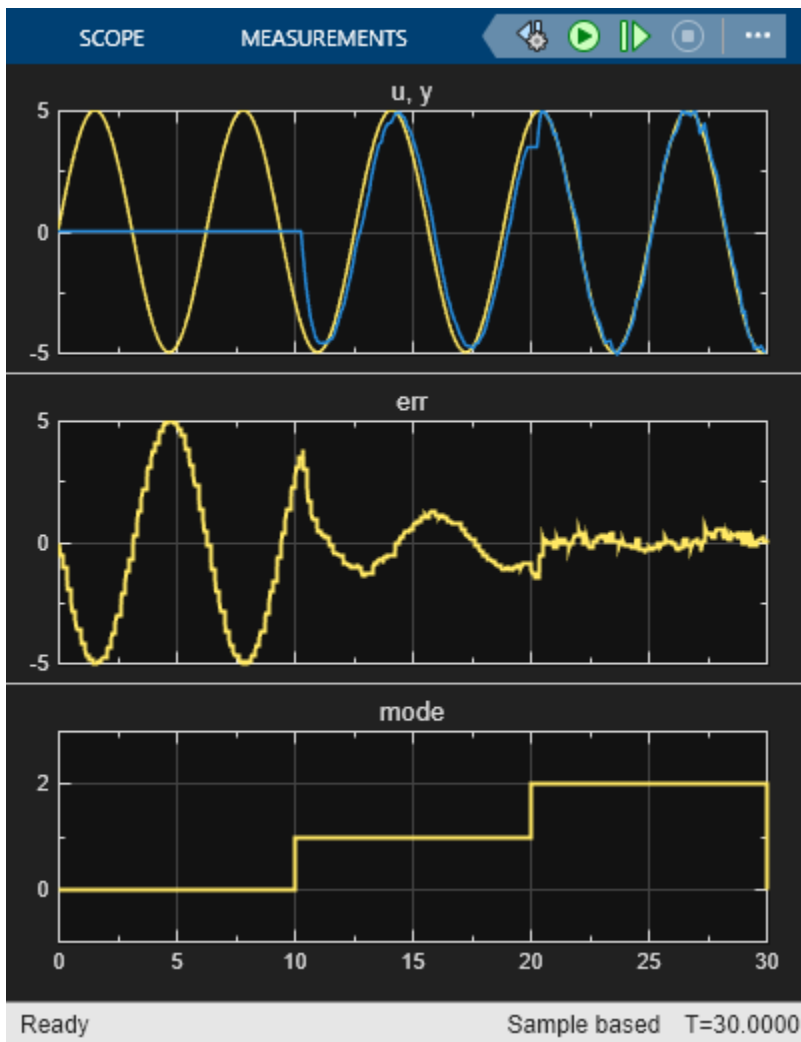
This example shows how to use **verify** statements in a when decomposition in a Test Assessment block to author assessments in a test harness.

This model implements a simple signal tracker that operates in three modes: off, slow, and quick.



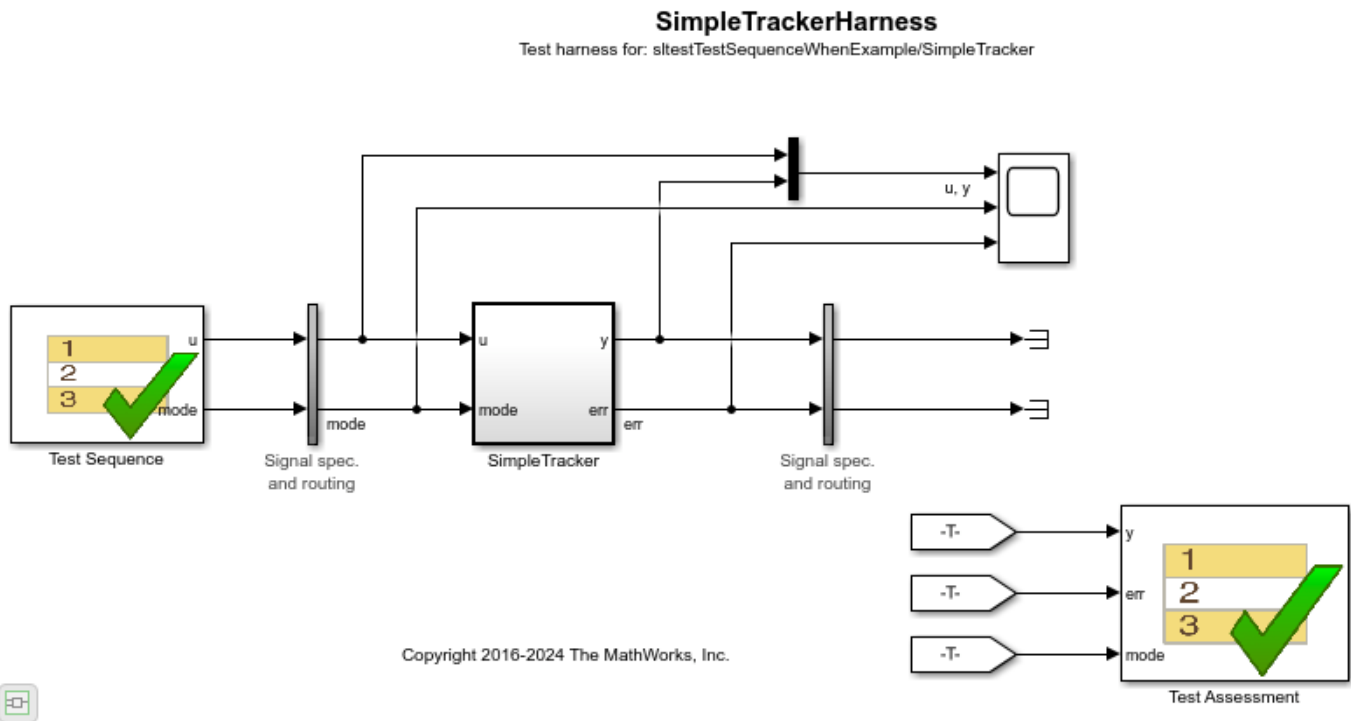
Copyright 2015-2024 The MathWorks, Inc.

To observe the output and error signals, simulate the model.



Open the Test Harness

The SimpleTracker subsystem contains a test harness that contains a Test Assessment block.



The Test Assessment block assesses the behavior of the SimpleTracker subsystem by using `verify` statements in a `when` decomposition.

Step

CheckError

OffMode when `mode == uint8(0)`

`verify(elapsed < 0.5 || y == 0, 'After 0.5 sec in Off, y must remain 0');`

SlowMode when `mode == uint8(1)`

`verify(elapsed < 0.5 || err < 2, 'After 0.5 sec in Slow, err must remain < 2');`

QuickMode

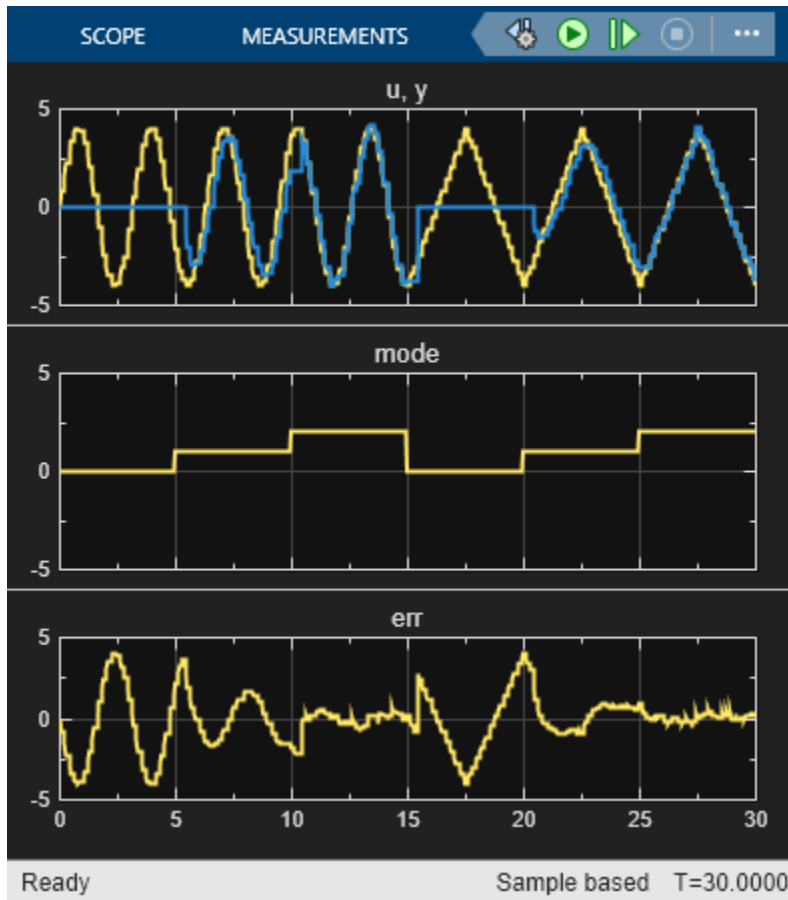
`verify(elapsed < 0.5 || err < 1, 'After 0.5 sec in Quick, err must remain < 1');`

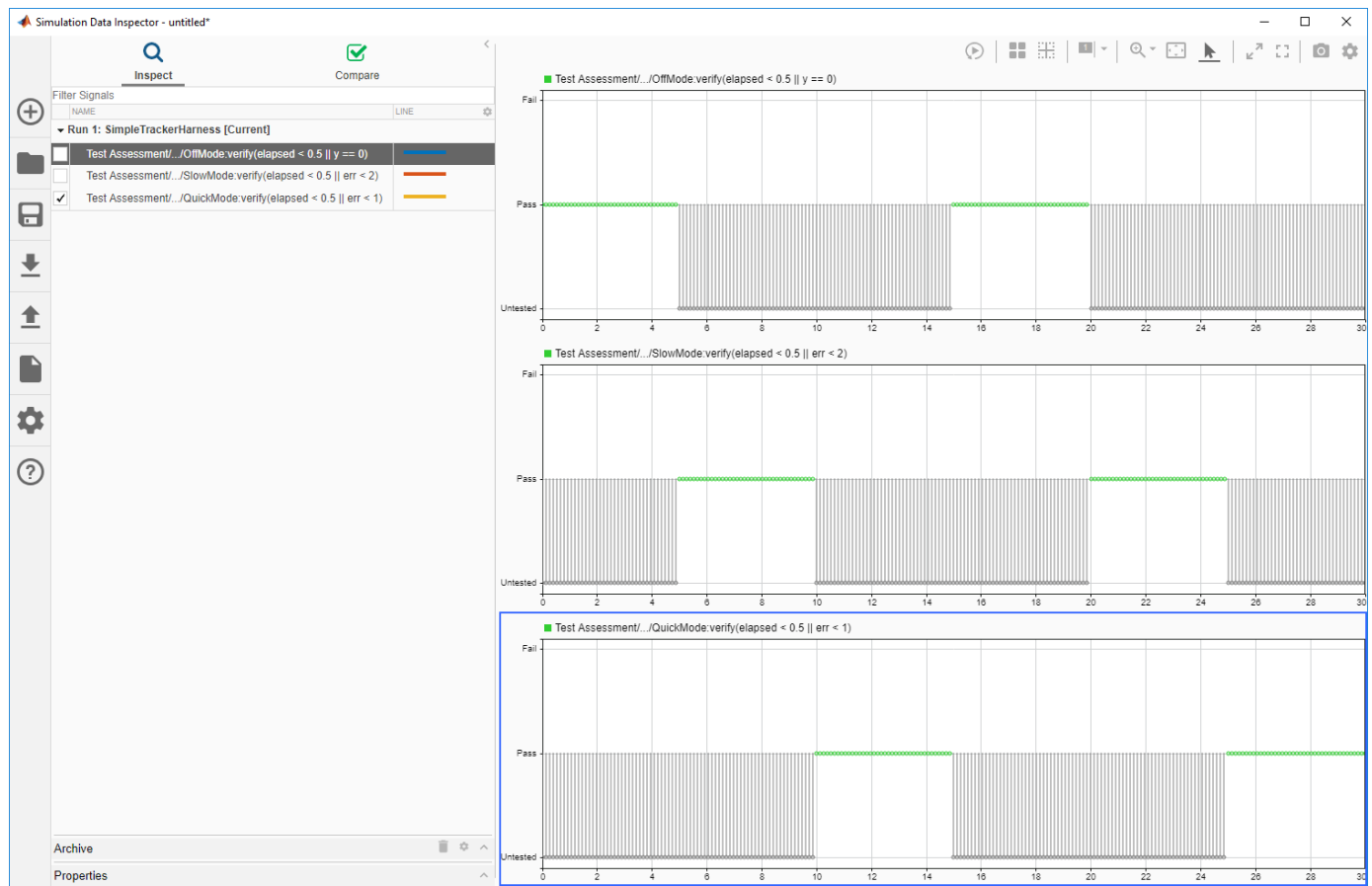
The CheckError step has a `when` decomposition with three substeps:

- OffMode is active when the value of mode is 0.
- SlowMode is active when the value of mode is 1.
- QuickMode is active for all other values of mode.

Run the Model Assessments

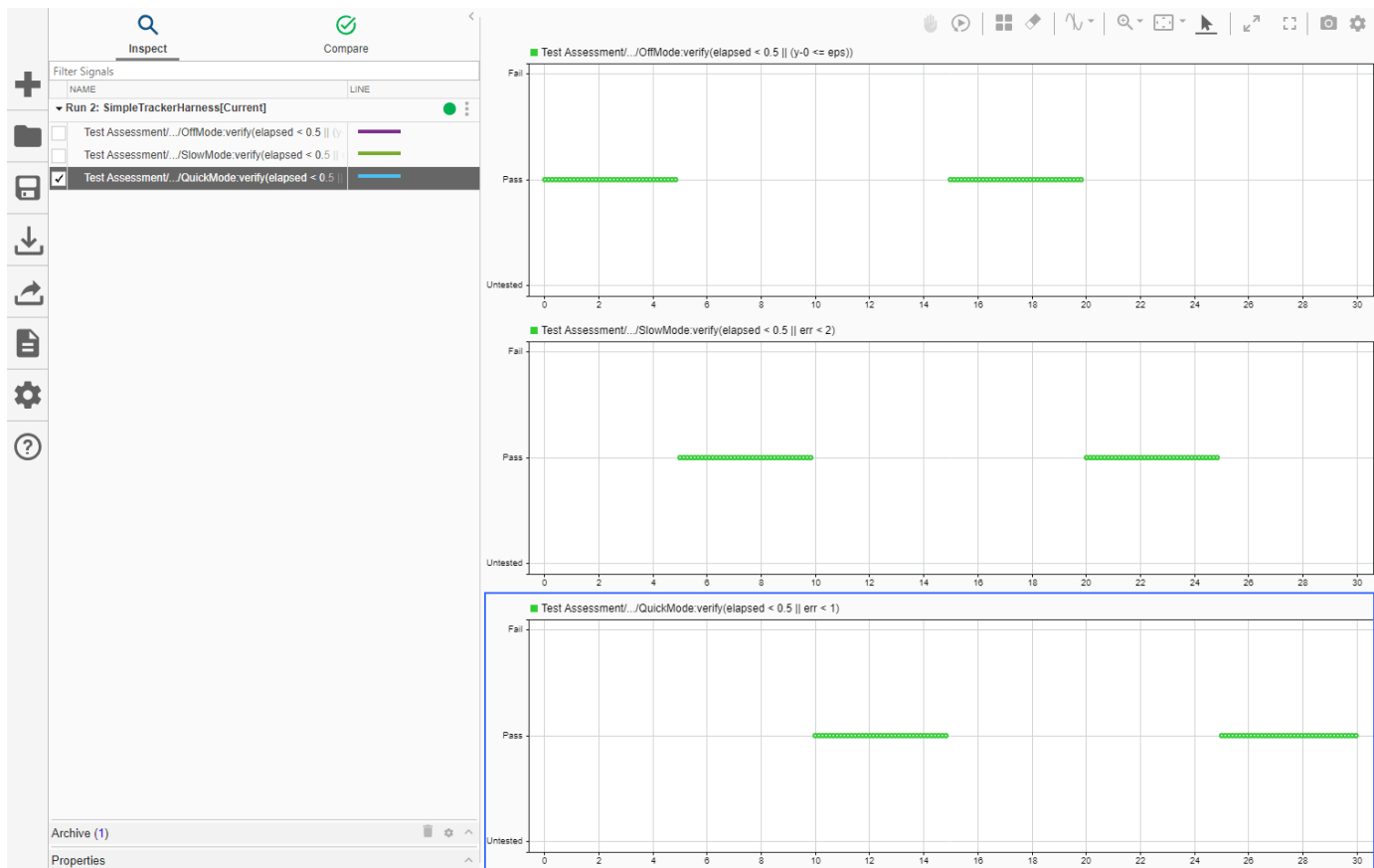
To run the assessments, simulate the test harness. Open the Simulation Data Inspector to inspect the results.





Log Only Tested verify Results

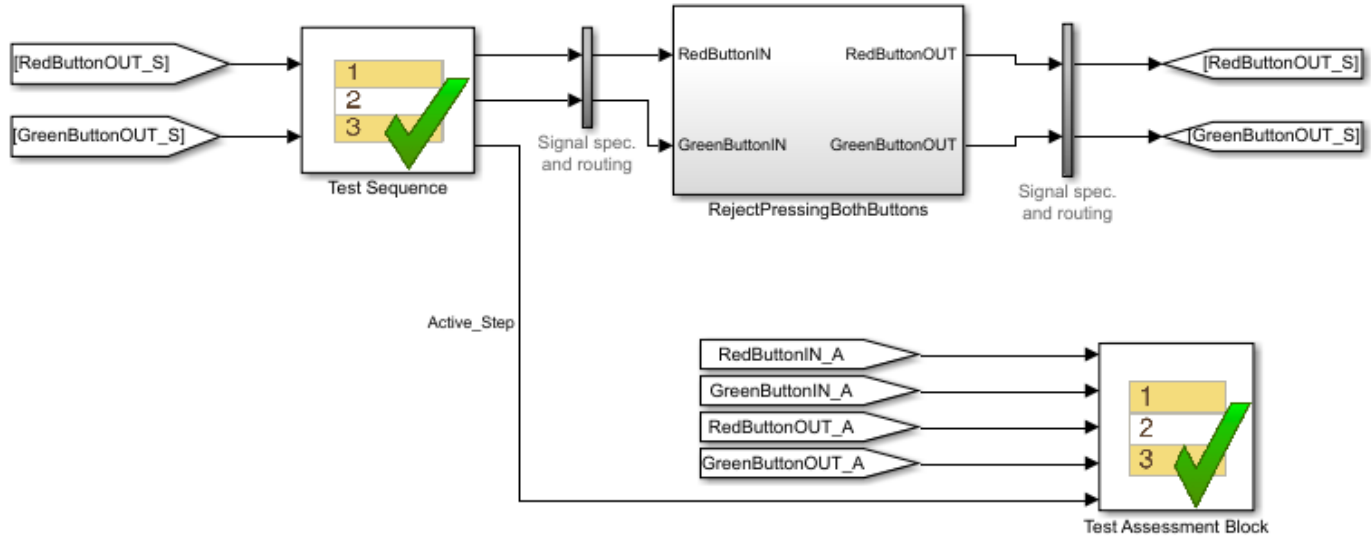
To log and display only pass and fail results, in the **Harness** tab, click **Suppress Untested Results**. Simulate the test harness and open the Simulation Data Inspector. The Simulation Data Inspector does not display the untested results.



Activate verify Statements by Using Test Sequence Steps

If your model uses a Test Sequence as a block source, you can connect the Test Sequence and Test Assessment blocks by using the active step signal from the Test Sequence block. You can create a when decomposition in the Test Assessment block and define `verify` statements to assess the model simulation.

For example, the `Harness1_TestSeqAndTestAssessment` harness contains a Test Sequence and Test Assessment block. The `Active_Step` signal connects the blocks.

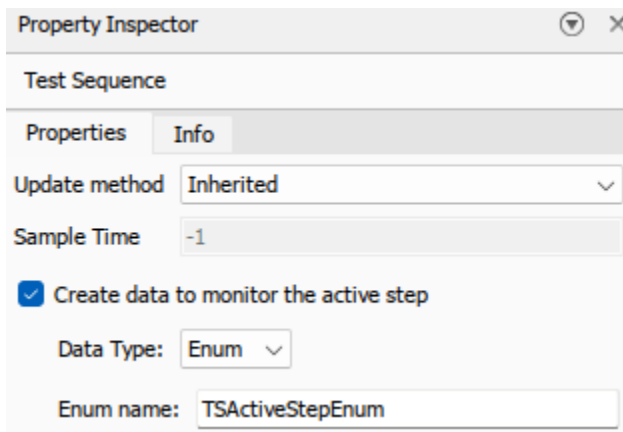


The Test Assessment block contains a when decomposition sequence with four substeps. Each substep contains a `verify` statement that evaluates two logical expressions. Each `verify` statement activates with a different Test Sequence block step.

The `Else` step in this example has no actions and handles the simulation conditions that do not match any of the preceding when conditions. This step cannot contain a when condition.

Symbols	Step	Transition	Next Step	Symbols	Step
Input 1. RedButtonOUT 2. GreenButtonOUT Output 1. RedButtonIN 2. GreenButtonIN 3. TSetActiveStepOUT Local Constant Parameter Data Store Memory	PressNeitherButton RedButtonIN = false; GreenButtonIN = false; PressBothButtons RedButtonIN = true; GreenButtonIN = true; PressRedButton RedButtonIN = true; GreenButtonIN = false; PressGreenButton RedButtonIN = false; GreenButtonIN = true; EndTest	1. after(1,sec) 1. after(1,sec) 1. after(1,sec) 1. after(1,sec)	PressBothButtons PressRedButton PressGreenButton EndTest	Input 1. RedButtonIN 2. GreenButtonIN 3. RedButtonOUT 4. GreenButtonOUT 5. TSetActiveStepIN Output Local Constant Parameter Data Store Memory	Assessments VerifyNeither when TSetActiveStepIN == TSetActiveStepEnum.PressNeitherButton verify(RedButtonOUT == false &&... GreenButtonOUT == false) VerifyBoth when TSetActiveStepIN == TSetActiveStepEnum.PressBothButtons verify(RedButtonOUT == false &&... GreenButtonOUT == false) VerifyRed when TSetActiveStepIN == TSetActiveStepEnum.PressRedButton verify(RedButtonOUT == true &&... GreenButtonOUT == false) VerifyGreen when TSetActiveStepIN == TSetActiveStepEnum.PressGreenButton verify(RedButtonOUT == false &&... GreenButtonOUT == true) Else

To activate the `verify` statements in a Test Assessment by using the active steps in a Test Sequence block, create active step data output for the Test Sequence block. Select the Test Sequence block, and in the Property Inspector, select **Create data to monitor the active step** and set the **Data Type** to Enum. Enter a name in **Enum Name**.



Then, create an input data for the Test Assessment block. Open the Test Assessment block, and, in the

Symbols pane, point to **Input**, click Add data , and name the input. Connect the Test Sequence block output to the Test Assessment block input.

Create a when decomposition in the Test Assessment block. By default, the Test Assessment block has a when decomposition sequence. To change between a standard sequence and a when decomposition sequence, right-click the parent step and select **When decomposition**. For each when decomposition step, define when the step is active by using the active step enumeration data.

```
VerifyBoth when TSActiveStepIN == ...
    TSActiveStepEnum.PressBothButtons
```

See Also

“Test Sequence Editor” on page 3-29 | Test Sequence | Test Assessment | `verify` |
`sltest.testmanager.Assessment` | `sltest.testmanager.AssessmentSymbol` |
`sltest.testmanager.TestCase`

Related Examples

- “Define Test Steps and Assessments” on page 3-63
- “Requirements-Based Testing for Model Development” on page 1-8

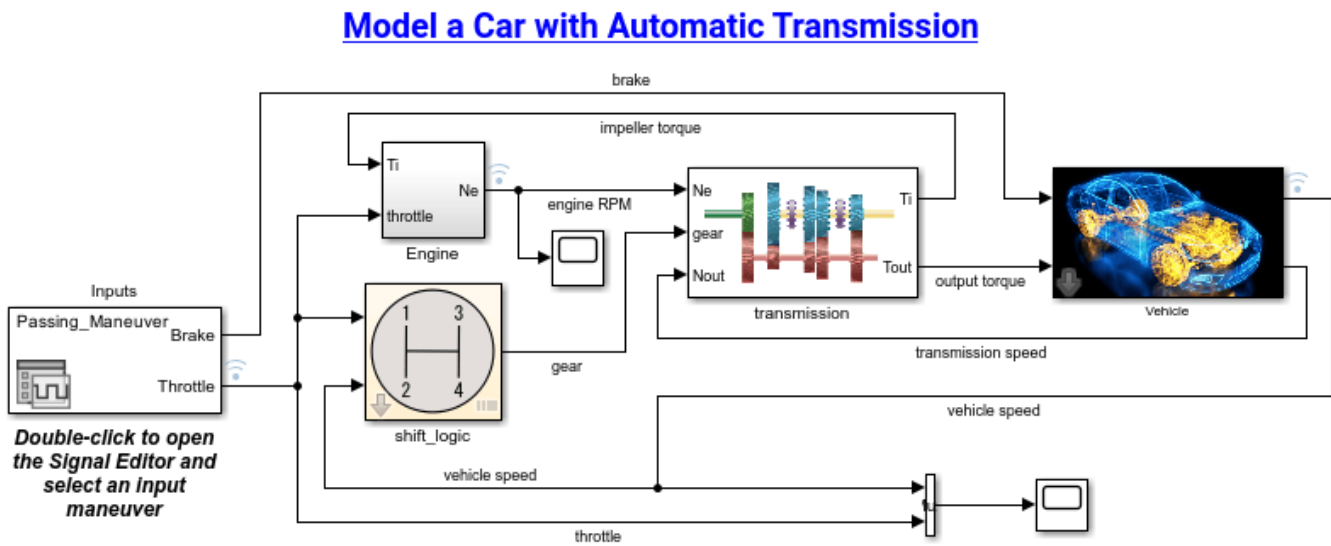
Author a Test Sequence and Test Assessment

This example shows how to use a Test Sequence block and Test Assessment block to create a test harness for a transmission shift logic controller. The Test Sequence block varies the speed input to the controller and the Test Assessment block verifies that the speed is appropriate for the gear position. For more information, see “Test Sequence Basics” on page 3-2.

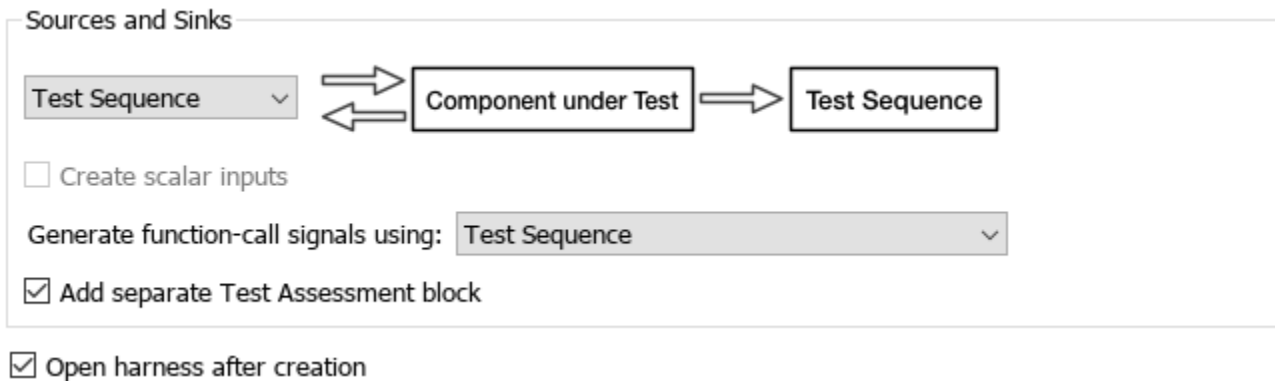
Create a Test Sequence

1. Open the model.

```
open_system("sf_car")
```

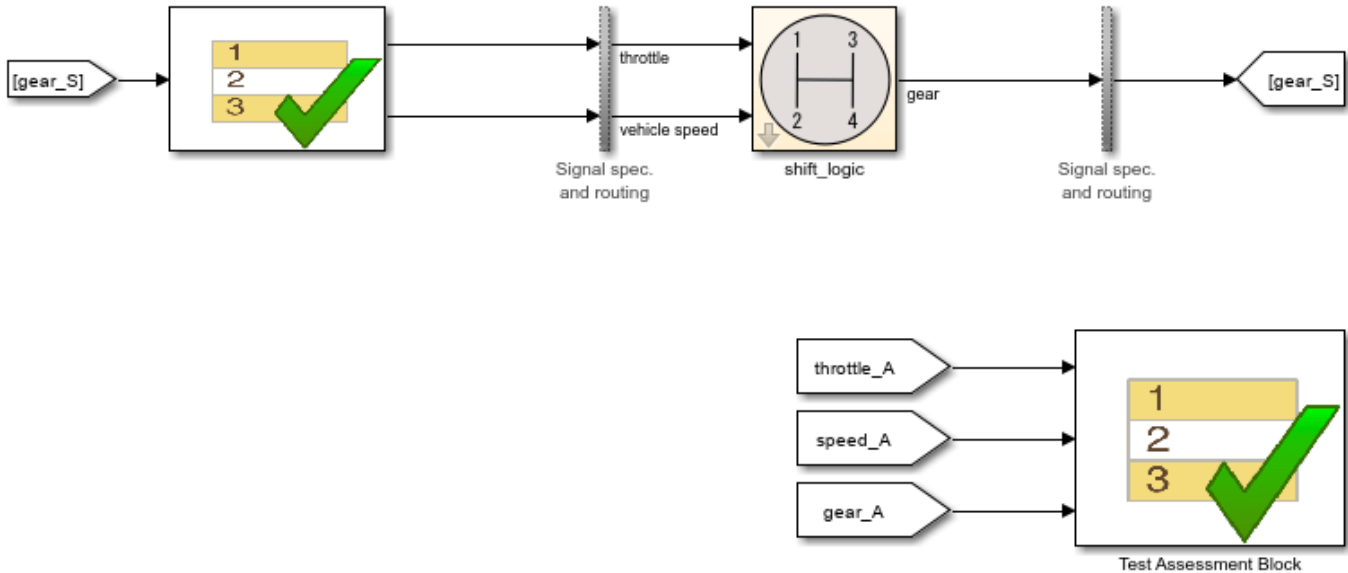


2. Right-click the `shift_logic` subsystem and select **Test Harness > Create for 'shift_logic'**.
3. In the Create Test Harness dialog box, under **Sources and Sinks**:
 - Set the source drop-down menu to Test Sequence.
 - Select **Add separate assessment block**.
 - Select **Open harness after creation**.



For more information, see “Create a Test Harness”.

4. Click **OK**. The test harness for the `shift_logic` subsystem opens.



5. Double-click the Test Sequence block. The “Test Sequence Editor” on page 3-29 opens.

Step	Transition	Next Step	Description
Run %% Initialize data outputs. speed = 0; throttle = 0;			

6. Change the name of the first step from `Run` to `Accelerate`, and add the step actions that specify the initial throttle setting and how the speed increases:

```
speed = 10*ramp(et);
throttle = 100;
```

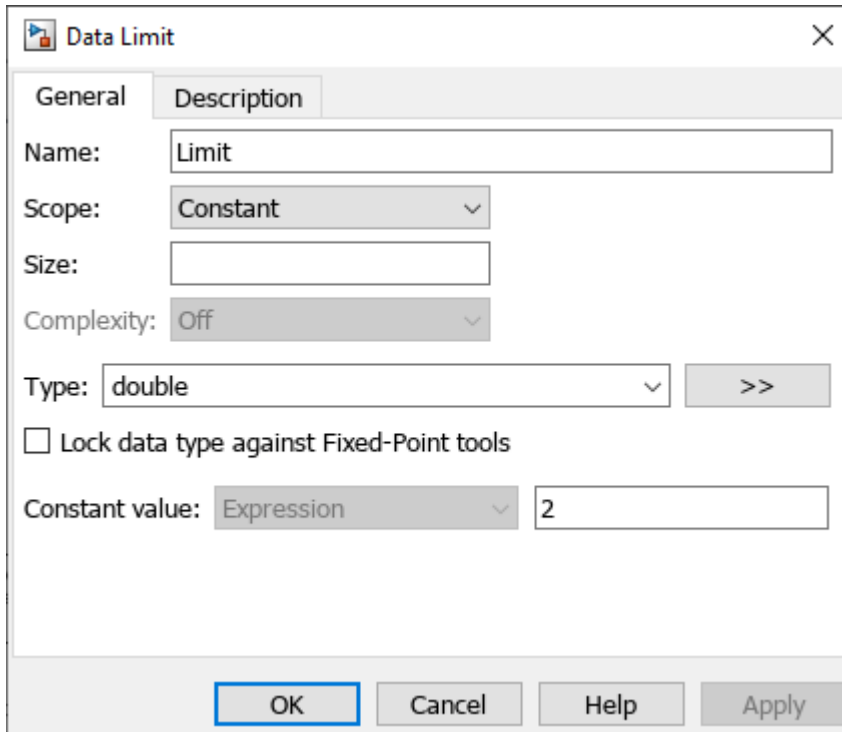
The `ramp` function returns the value of a ramp signal of slope 1 at time `et`, which is the elapsed time of the step. For this example, the speed at a particular time after the beginning of the step is 10 times the value of the ramp at that time. See “Test Sequence Editor” on page 3-29 for more information.

7. Right-click the `Accelerate` step and select **Add step after**. Change the name of this step from `step_1` to `Stop`, and add the step actions that stop the test when the throttle and speed are equal to 0:

```
throttle = 0;
speed = 0;
```

8. In the **Symbols** pane, point to **Constant** and click . Enter **Limit** for the constant name and click **Add data**.

9. Point to **Limit** and click the edit button . In the **Constant value** field, enter **2**. Click **OK**.



The **Data Limit** dialog box is shown with the **General** tab selected. The **Name** field contains "Limit". The **Scope** dropdown is set to "Constant". The **Size** field is empty. The **Complexity** dropdown is set to "Off". The **Type** dropdown is set to "double", and there is a ">>" button next to it. The **Lock data type against Fixed-Point tools** checkbox is unchecked. The **Constant value** section has a dropdown set to "Expression" and a text field containing "2". At the bottom are buttons for **OK**, **Cancel**, **Help**, and **Apply**.

10. Right-click the **Transition** column in the first step, select **Add transition**. Enter the transition condition for the **Accelerate** step. In this example, **Accelerate** transitions to **Stop** when the system is in fourth gear for at least two seconds. In the **Transition** column, enter:

```
duration(gear == 4) >= Limit
```

11. In the **Next Step** column, select **Stop**.

Symbols		Scenarios		Step	Transition	Next Step	Description
Input				Accelerate %% Initialize data outputs. speed = 10*ramp(et); throttle = 100; Stop throttle = 0; speed = 0;	1. duration(gear == 4) >= Limit	Stop ▼	
1.  gear							
Output							
1.  speed							
2.  throttle							
Local							
Constant							
Limit							

12. Save the changes and close the Test Sequence Editor.

Create a Test Assessment

Next, add verifications to the Test Assessment block.

1. Double-click the Test Assessment block to open the editor. The editor displays a default When decomposition sequence in the **Step** column. For more information, see “Activate verify Statements by Using Signal Conditions” on page 3-14.

Symbols	Scenarios	Step	Transition	Next Step	Description
Input 1. speed 2. throttle 3. gear Output Local Constant		Run step_1_1 when t < 5 step_1_2			

2. Change the name of the first step from Run to Assessments.

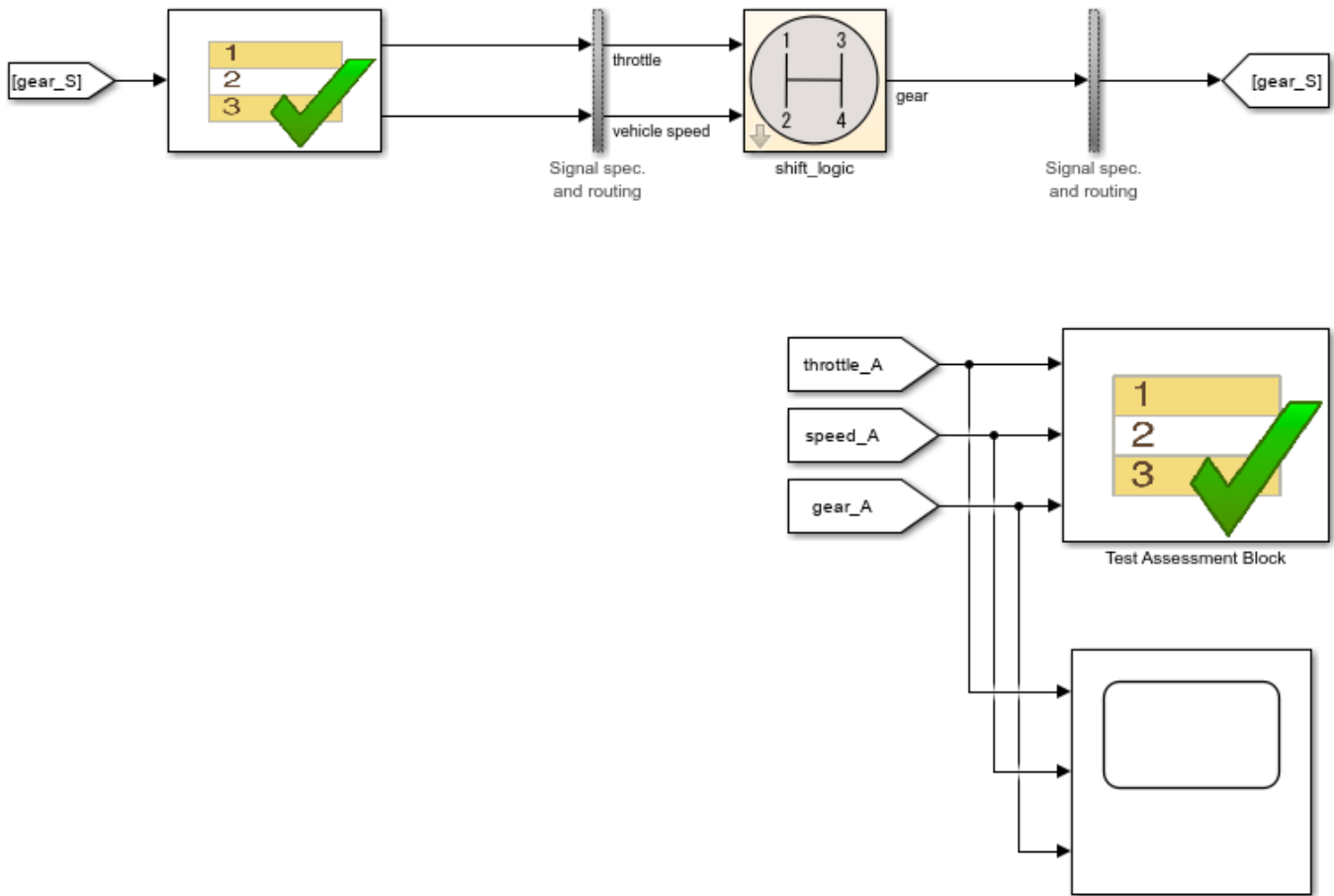
3. Change the names and add actions for four substeps:

- Rename step_1_1 to Check1st. Define the when condition as `gear == 1` and add a `verify` statement that evaluates whether the speed is less than 45.
- Rename step_1_2 to Check2nd. Define the when condition as `gear == 2` and add a `verify` statement that evaluates whether the speed is less than 75.
- Right-click the Check2nd step and click **Add step after**. Rename the step Check3rd. Define the when condition as `gear == 3` and add a `verify` statement that evaluates whether the speed is less than 105.
- Right click the Check3rd step and click **Add step after**. Rename the step else. This step handles simulation conditions outside of the preceding when conditions. In this step, you do not specify any actions.

Symbols	Scenarios	Step	Transition	Next Step	Description
Input 1. speed 2. throttle 3. gear Output Local Constant Parameter Data Store Memory		Assessments Check1st when gear == 1 verify(speed < 45) Check2nd when gear == 2 verify(speed < 75) Check3rd when gear == 3 verify(speed < 105) else			

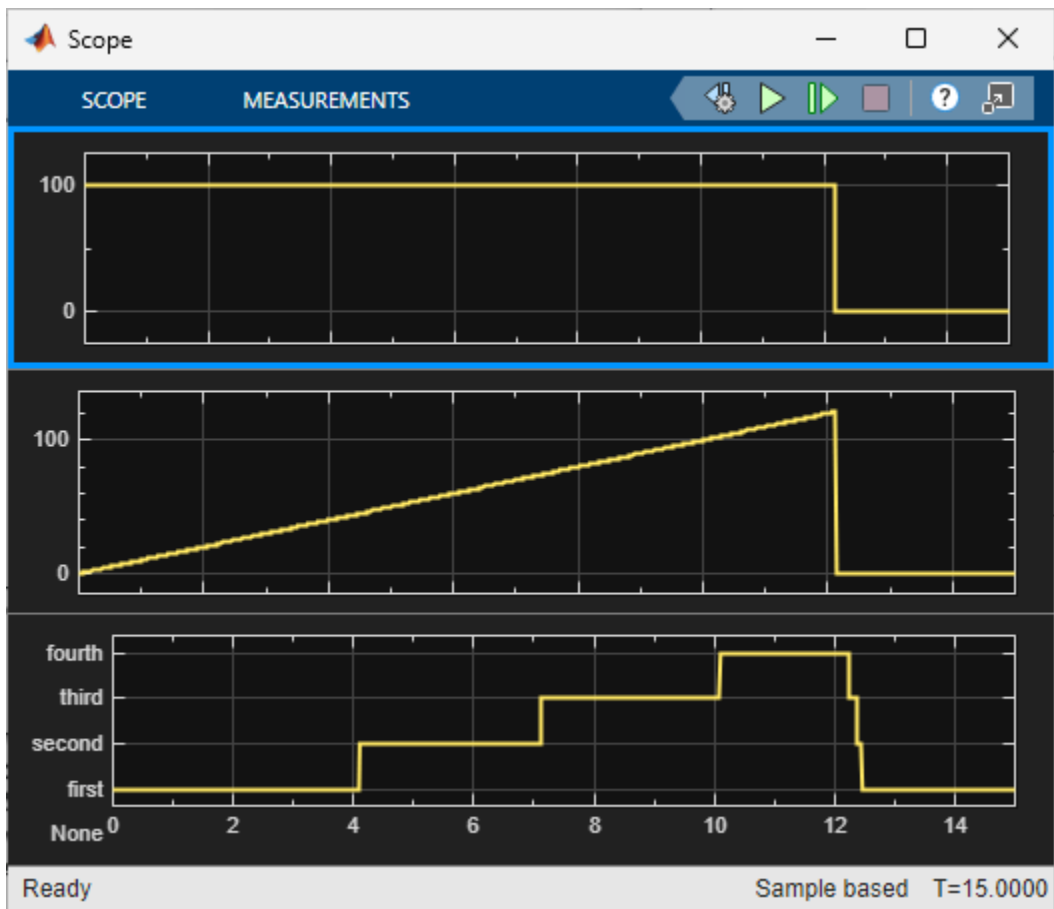
4. Save the changes and close the Test Sequence Editor.

5. In the test harness, add a Scope block and connect the speed, throttle, and gear signals to the scope.



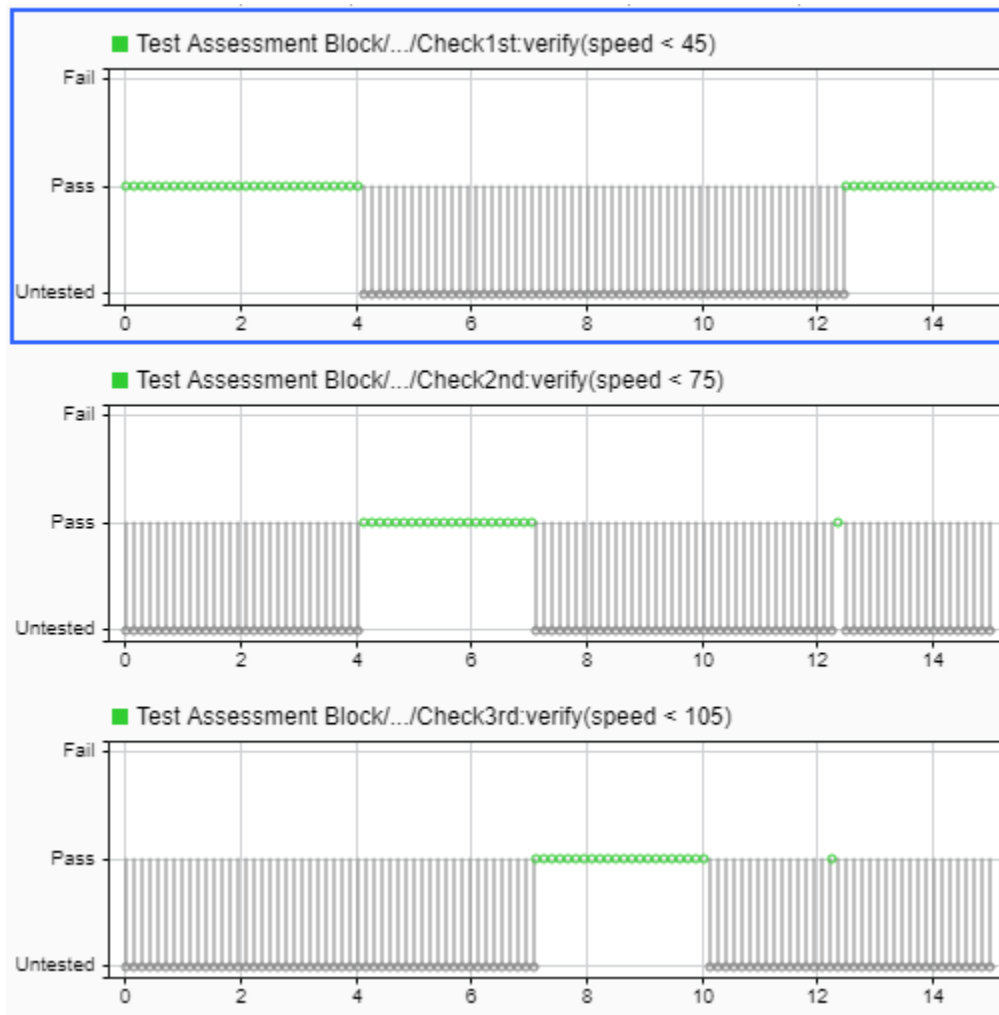
6. Open the Scope block. In **View > Layout**, select a three-column, one-row layout.

7. In the Simulink model, in the **Simulation** tab, set **Stop Time** to 15 and click **Run**. View the signal data in the Scope block. The speed plot shows how the speed increases to a maximum value of 105. The throttle plot shows that the throttle value is held at 100 throughout the test. The gear plot shows that the gear shifts to the next-higher gear at the appropriate speed. After the gear has been equal to 4 for 2 seconds, the speed and throttle go to zero and the gear decreases.



8. View the results of the `verify` statements in the Simulation Data Inspector.

- In the Simulink model, in the **Simulation** tab, click **Data Inspector**.
- Click the Visualizations and layouts button  and, under **Grid**, set **Rows** to 3 and **Columns** to 1.
- Select one plot, then select an assessment from the left pane. Repeat for each of the other plots and assessments.



When the model runs, the Test Assessment block verifies that the speed is in the correct range for each gear position. A pass indicates that the speed is in the correct range. Each `verify` statement is for a specific gear, so the speed is untested when the controller is in a different gear than the one being tested. There are two pass regions for the `Check1st` test assessment because the gear position goes back to first in the second step of the test sequence.

See Also

Test Sequence | Test Assessment

Related Examples

- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Test Sequence Basics” on page 3-2
- “Define Test Steps and Assessments” on page 3-63
- “Test Sequence Editor” on page 3-29

Test Sequence Editor

The Test Sequence Editor enables you to define and modify test sequences for Test Sequence and Test Assessment blocks. To open the Test Sequence Editor, double-click a Test Sequence or Test Assessment block.

Define Test Sequences

A test sequence consists of test steps arranged in a hierarchy. Test steps can contain transitions that define how a test progresses in response to the simulation. Test steps can also have a **When** decomposition that uses logic similar to an **if-elseif-else** statement. By default:

- New Test Sequence blocks contain two standard transition test steps.
- New Test Assessment blocks contain a **When** decomposition test step with two sub-steps.

For more information, see “Transition Types” on page 3-2.

To define a test sequence:

- 1 Add test steps, as described in “Manage Test Steps” on page 3-29.
- 2 In the **Step** cell, define outputs and assessments.
- 3 To add a transition from a test step:
 - a Point to the **Transition** cell and click **Add transition**.
 - b In the **Transition** cell, define the conditions for exiting the step.
 - c In the **Next Step** cell, select the next test step from the drop-down list.
- 4 To define a step with a **When** decomposition:
 - a Right-click a test step and select **When decomposition**. The step displays the icon .
 - b Add sub-steps, as described in “Manage Test Steps” on page 3-29.
 - c In the **Step** cell of each sub-step, enter the when operator, followed by a condition. Do not add a condition to the last sub-step.

Test Sequence Scenarios

To define multiple test sequences in a single Test Sequence block, use scenarios. In the left pane of the Test Sequence Editor, click **Scenarios**, then click **Use Scenarios**. The existing test steps and transitions are moved into a scenario tab named `Scenario_1`. Add more scenarios to define more test sequences. For more information on Test Sequence scenarios, see “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54.

Manage Test Steps

In the Test Sequence Editor, you can add and delete test steps to your test sequence. You can also reorder the test steps and change their position in the hierarchy.

Add and Delete Test Steps

To add a test step, right-click an existing step and select **Add step before** or **Add step after**.

To add a test step in a lower hierarchy level, right-click the parent step and select **Add sub-step**.

To delete a test step, right-click the step and select **Delete step**. If the test sequence contains only one test step, you cannot delete it. You can delete its contents by selecting **Erase last step content**.

Copy and Paste Test Steps

To copy a test step, right-click the area to the left of the step name and select **Copy step**. Alternatively, select the test step and use the shortcut **Ctrl+C**.

To cut a test step, right-click the area to the left of the step name and select **Cut step**. Alternatively, select the test step and use the shortcut **Ctrl+X**.

To paste a test step, right-click the area to the left of a step name and select **Paste step**, then:

- **Paste before step**
- **Paste after step**
- **Paste sub-step**

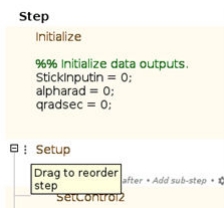
Alternatively, select the test step and use the shortcut **Ctrl+V**.

Reorder Test Steps and Transitions

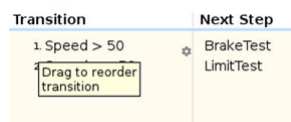
To reorder the test steps in a test sequence:

- 1 Point to a test step. The icon  appears to the left of the step name.
- 2 Click and drag the icon to reorder the test step.

You can reorder test steps within the same hierarchy level. When you move a test step, sub-steps move with the test step.



To reorder a transition within the same test step, click and drag the transition number to the new location. The corresponding next step moves with the transition.



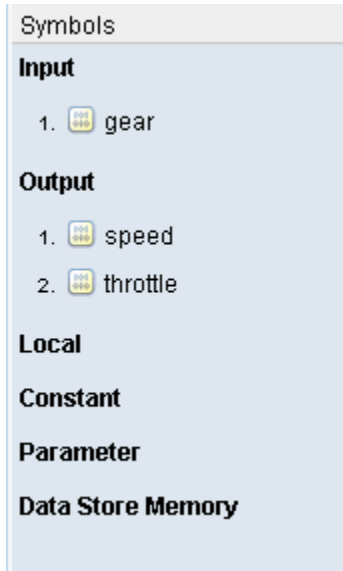
Change Test Step Hierarchy

To move a test step to a lower level in the hierarchy, right-click the step and select **Indent step**. You can only indent a test step when the preceding step is at the same hierarchy level. You cannot indent the first test step in a sequence or the first step in a hierarchy group.

To move a test step to a higher level in the hierarchy, right-click the step and select **Outdent step**. You can only move the last step in a hierarchy group to a higher level in the hierarchy.

Manage Input, Output, and Data Objects

In the **Symbols** sidebar of the Test Sequence Editor, you add, edit, or delete symbols in the Test Sequence block. You can access these symbols from test steps at any hierarchy level. To show or hide the **Symbols** sidebar, click the **Symbols Sidebar** button  on the Test Sequence Editor toolbar.



To add a data symbol, point to the node for a symbol type and click an add symbol button. Available options and additional setup steps depend on the symbol type.

Symbol Type	Description	Procedure for Adding Symbol
Input	Options for input entries include: - Data - Messages	- In the Symbols sidebar, point to the Input node and click either:  Add data  Add message - Enter the name of the input and press Enter.
Output	Options for output entries include: - Data - Messages - Function Calls - Triggers	- In the Symbols sidebar, point to the Output node and click:  Add data  Add message  Add function call  Add trigger - Enter the name of the output and press Enter.

Symbol Type	Description	Procedure for Adding Symbol
Local	Local data entries are available only inside the Test Sequence block in which they are defined.	1 In the Symbols sidebar, point to the Local node and click  Add data. 2 Enter the name of the local variable and press Enter. Initialize the local variable in the first test step.
Constant	Constants are read-only data entries available only inside the Test Sequence block in which they are defined.	1 In the Symbols sidebar, point to the Constant node and click  Add data. 2 Enter the name of the constant and press Enter. 3 Point to the name of the constant and click  Edit. 4 In the dialog box, in the Constant Value field, enter the value of the constant.
Parameter	Parameters are available inside and outside the Test Sequence block.	1 Using the Model Explorer, add a parameter in the workspace of the model that contains the Test Sequence block. 2 In the Symbols sidebar, point to the Parameter node and click  Add data. 3 Enter the name of the parameter and press Enter.
Data Store Memory	Data Store Memory entries are available inside and outside the Test Sequence block.	1 Using the Model Explorer, add a <code>Simulink.Signal</code> object in the workspace of the model that contains the Test Sequence block. Alternatively, add a Data Store Memory block to the model. To set or modify global parameters in the Test Sequence block, you can use <code>Simulink.Signal</code> objects as Data Store Memory. 2 In the Symbols sidebar, point to the Data Store Memory node and click  Add data. 3 Enter the name of the data store and press Enter.

To edit a data symbol, point to the name of the symbol and click  **Edit**.

To delete a data symbol, point to the name of the symbol and click  **Delete**.

Find and Replace

You can find and replace text in Test Sequence actions, transitions, and descriptions by using the **Find & Replace** tool in the Test Sequence Editor.

- 1 To open the **Find & Replace** tool, click the  icon in the toolbar.

- 2 In the **Find what** field, enter the text you want to locate.
- 3 In the **Replace with** field, enter the updated text.
- 4 To locate the text, click **Find Next** or **Find Previous**.
- 5 To replace the old text with the updated text, click **Replace**.

When running a search, the **Find & Replace** tool searches descriptions only if the description column is open.

Automatic Syntax Correction

The Test Sequence Editor changes the syntax automatically for:

- **Increment and decrement operations**, such as `a++` and `a--`. For example, `a++` is changed to `a=a+1`.
- **Assignment operations**, such as `a+=expr`, `a-=expr`, `a*=expr`, and `a/=expr`. For example, `a+=b` is changed to `a=a+b`.
- **Evaluation operations**, such as `a!=expr` and `!a`. For example, `a!=b` is changed to `a~=b`.
- **Explicit casts for literal constant assignments**. For example, if `y` is defined as type `single`, then `y=1` is changed to `y=single(1)`.

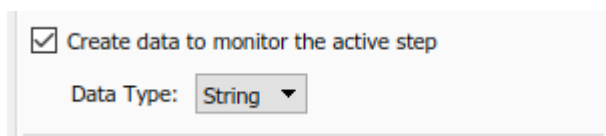
Output and View Active Step Data

When you run a test, the current step in the test sequence is the active step. When you enable creating active step data, a new output port is added to the Test Sequence block for the active step signal. You can analyze the active step data or use the output signal as an input to other blocks in your test harness. For example, a Test Assessment block can use the active step input as a trigger signal. You can also plot the active step data in the Simulation Data Inspector to see how the active step changes over time.

Enable Active Step Output

To create the active step data:

- 1 From the Test Sequence Editor, open the Model Explorer. Alternatively, open the Property Inspector from the test harness or model that contains the Test Sequence or Test Assessment block.
- 2 Enable **Create data to monitor the active step**.



- 3 Set **Data Type** to `String` or `Enum`. The default value is `String`.
 - `String` — Output the active step data as a string. Use this option if you use duplicate step names across scenarios or if you use duplicate substep names in different steps of the same scenario. When you select `String`, the output includes the step name and indicates which scenario is active. The step name string is `<scenario>.<step>.<substep>`. If the block does not have an active step at a time step, the output is an empty string. This situation might

occur if a step uses an enabled subsystem and that subsystem is not enabled during that time step.

- **Enum** — Output the active step as an enumerated value. Selecting Enum enables the **Enum name** parameter. The data is output in the format <Enum name>.<step name>. The scenario of that step is not included in the active step name. You cannot use Enum for the active step output if you have duplicate step or substep names in your test sequence.

☒ Create data to monitor the active step

Data Type: Enum ▼

Enum name: Test_Sequence_Active_Step_Enum

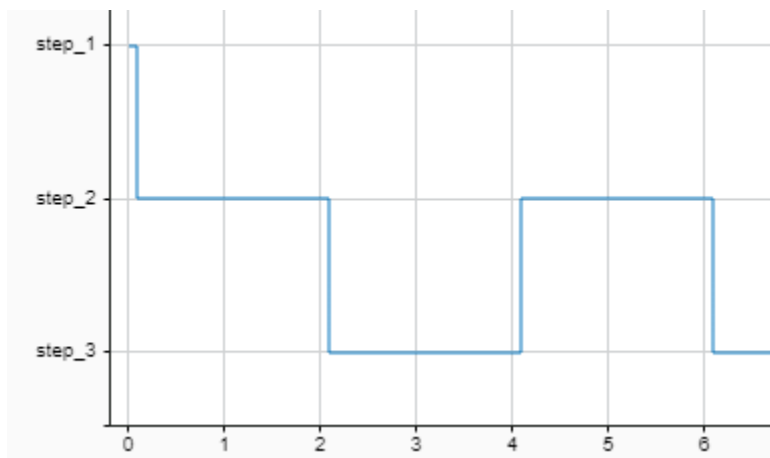
- 4 Click **Apply**.

View Active Step Data

You can view the active step output data in the Simulation Data Inspector, or if you run your test in the Test Manager, in the Test Manager **Results** pane. Before running the test, in the harness, right-click the active data output signal of the Test Sequence block and click **Log Selected Signal**. If you want to plot the active step output, you do not need to connect the active step output signal to any component.

After running a test, open the Simulation Data Inspector or the Test Manager **Results**. The format of the active step plot differs depending on whether the output is an enumerated or string type.

- **Enumerated type** — The x-axis is time and the y-axis is the step.



- **String type** — The x-axis is time. Steps appear as blocks of time during the period they are active. If there is not enough space to show the full string names, the Simulation Data Inspector truncates the beginning of the names so the step name displays.



If there are too many step blocks to display in the plot, only the steps that fit are shown.

You can add the active step data plot to a Test Results report in the same way you include other test results.

Use Active Step Output as Input to Another Block

To use the active step output signal as input to another block, such as a Test Assessment block, connect the signal to the input port of the block. You can then use the signal to trigger actions and assessments based on the active step.

See Also

Test Assessment | Test Sequence

Related Examples

- "Test Sequence Basics" on page 3-2
- "Author a Test Sequence and Test Assessment" on page 3-22
- "Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager" on page 3-54
- "Define Test Steps and Assessments" on page 3-63
- "Use Stateflow Chart for Test Harness Inputs and Scheduling" on page 3-5

Transitions, Temporal Operators, and Messages in Test Sequence Blocks

In this section...

“Transition Between Steps Using Temporal or Signal Conditions” on page 3-36

“Temporal Operators” on page 3-36

“Transition Operators” on page 3-37

“Use Messages in Test Sequences” on page 3-37

Transition Between Steps Using Temporal or Signal Conditions

The Test Sequence block uses MATLAB as the action language. You can transition between test steps by evaluating the component under test. You can use conditional logic, temporal operators, and event operators.

Consider a simple test sequence that outputs a sine wave at three frequencies. The Test Sequence block steps through several actions based on changes in the signal switch. See `hasChanged`.

Data Symbols	Step	Transition	Next Step
Input Switch	Initialize SignalOut = 0;	1. true	Sine ▼
Output SignalOut	Sine SignalOut = sin(et*2*pi/10);	1. hasChanged(Switch)	Sine8 ▼
Local	Sine8 SignalOut = sin(et*8*pi/10);	1. hasChangedFrom(Switch,1)	Sine16 ▼
Constant	Sine16 SignalOut = sin(et*16*pi/10);	1. hasChangedTo(Switch,13.344)	Stop ▼
Parameter	Stop SignalOut = 0;		
Data Store Memory			

Temporal Operators

To create an expression that evaluates the simulation time, use temporal operators. Variables in signal conditions must be inputs, parameters, or constants in the Test Sequence block. You can use these temporal operators:

Operator	Description
et	The elapsed time of the test step. You can specify the unit of time. If you do not specify a unit of time, the elapsed time is returned in seconds.

Operator	Description
<code>t</code>	The elapsed time of the simulation. You can specify the unit of time. If you do not specify a unit of time, the elapsed time is returned in seconds.
<code>after</code>	Returns <code>true</code> if the specified time has elapsed since the beginning of the current test step.
<code>before</code>	Returns <code>true</code> until the specified time has elapsed from the beginning of the current test step.
<code>duration</code>	Returns the elapsed time for which the specified condition has been <code>true</code> . The elapsed time resets when the test step is re-entered or when the specified condition is no longer <code>true</code> .

Transition Operators

To create expressions that evaluate signal events, use transition operators. To evaluate signal events, the signals must be inputs in the Test Sequence or Test Assessment block.

Operator	Description
<code>hasChanged</code>	Returns <code>true</code> if the specified signal changes in value since the beginning of the test step, and otherwise returns <code>false</code> .
<code>hasChangedFrom</code>	Returns <code>true</code> if the specified signal changes from the specified value to a different value, and otherwise returns <code>false</code> .
<code>hasChangedTo</code>	Returns <code>true</code> if the specified signal changes to the specified value, and otherwise returns <code>false</code> .

Use Messages in Test Sequences

Messages carry data between Test Sequence blocks and other blocks such as Stateflow® charts. Messages can be used to model asynchronous events. A message is queued until you evaluate it, which removes it from the queue. You can use messages and message data inside a test sequence. The message remains valid until you forward it, or the time step ends. For more information, see “Messages” (Stateflow) in the Stateflow® documentation.

Receive Messages and Access Message Data

If your Test Sequence block has a message input, you can use queued messages in test sequence actions or transitions. Use the `receive` command before accessing message data or forwarding a message.

To create a message input, hover over **Input** in the **Symbols** sidebar, click the add message icon, and enter the message name.

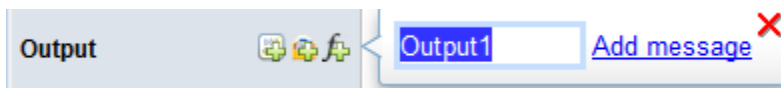


`receive(M)` determines whether a message is present in the input queue `M`, and removes the message from the queue. `receive(M)` returns `true` if a message is in the queue, and `false` if not. Once the message is received, you can access the message data using the dot notation, `M.data`, or forward the message. The message is valid until it is forwarded or the current time step ends.

The order of message removal depends on the queue type. Set the queue type using the message properties dialog box. In the **Symbols** sidebar, click the edit icon next to the message input, and select the **Queue type**.

Send Messages

To send a message, create a message output and use the `send` command. To create a message output, hover over **Output** in the **Symbols** sidebar, click the add message icon, and enter the message name.



You can assign data to the message using the dot notation `M.data`, where `M` is the message output of the Test Sequence block. `send(M)` sends the message.

Forward Messages

You can forward a message from an input message queue to an output port. To forward a message:

- 1 Receive the message from the input queue using `receive`.
- 2 Forward the message using the command `forward(M,M_out)` where `M` is the message input queue and `M_out` is the message output.

Compare Test Sequences Using Data and Messages

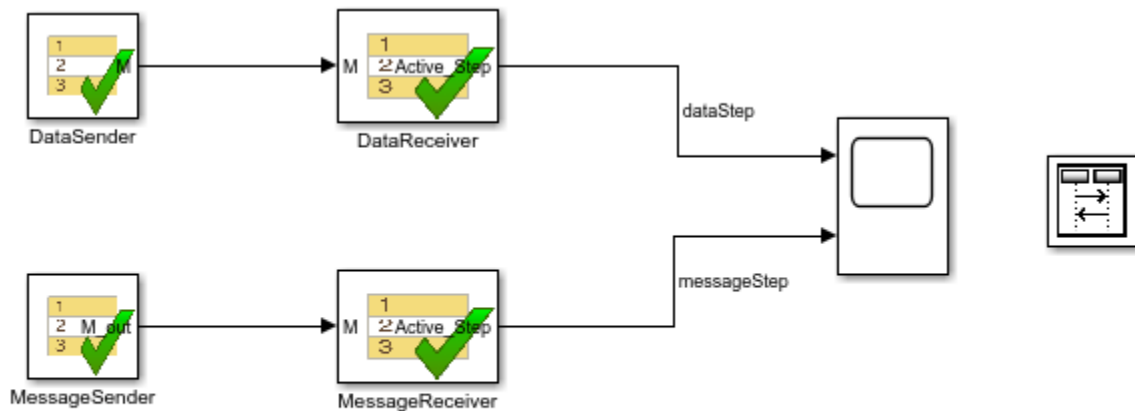
This example demonstrates message inputs and outputs, sending, and receiving a message. The model compares two pairs of test sequences. Each pair is comprised of a sending and receiving test sequence block. The first pair sends and receives data, and the second sends and receives a message.

Set the model name variable.

```
model = 'sltest_testsequence_data_vs_message';
```

Open the model.

```
open_system(model)
```



Copyright 2015-2020 The MathWorks, Inc.

Test Sequences Using Data

The DataSender block assigns a value to a data output M.

Step	Transition	Next Step	Description
step_1 M = 3.5;	1. true	step_2 ▼	Assigns a value to the data
step_2			

The DataReceiver block waits 3 seconds, then transitions to step S2. Step S2 transitions to step S3 using a condition comparing M to the expected value, and does the same for S3 to S4.

Step	Transition	Next Step	Description
S1	1. after(3,sec)	S2 ▼	Waits
S2	1. M == 3.5	S3 ▼	
S3	1. M == 3.5	S4 ▼	
S4			

Test Sequences Using Messages

The MessageSender block assigns a value to the message data of a message output M_out, then sends the message to the MessageReceiver block.

Step	Transition	Next Step	Description
step_1 M.data = 3.5;	1. <i>true</i>	step_2 ▼	Assigns a value to the message's data
step_2 send(M)	1. <i>true</i>	step_3 ▼	Sends the message
step_3			

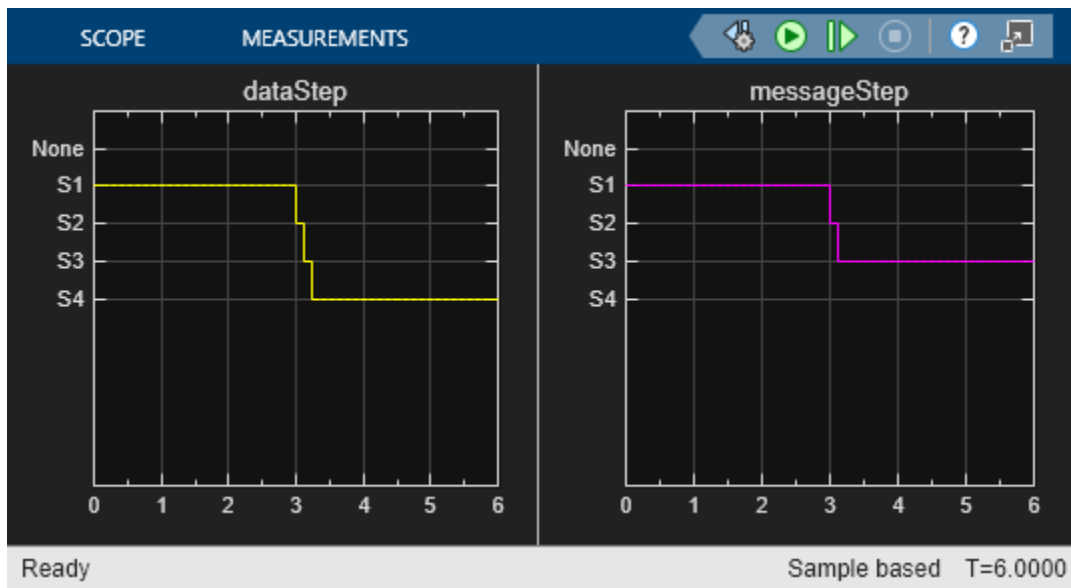
The MessageReceiver block waits 3 seconds, then transitions to step S2. Step S2's transition evaluates the queue M with `receive(M)`, removing the message from the queue. `receive(M)` returns `true` since the message is present. `M.data == 3.5` compares the message data to the expected value. The statement is true, and the sequence transitions to step S3.

Step	Transition	Next Step	Description
S1	1. <code>after(3,sec)</code>	S2 ▼	Waits.
S2	1. <code>receive(M) && M.data == 3.5</code>	S3 ▼	Transitions to S3 if a message is available in the queue and message data == 3.5.
S3	1. <code>receive(M)</code>	S4 ▼	Transitions to S4 if a message is available in the queue. (it is not, because it has been received).
S4			

When step S3's transition condition evaluates, no messages are present in the queue. Therefore, S3 does not transition to S4.

Run the test and observe the output comparing the different behaviors of the test sequence pairs.

```
open_system([model '/Scope'])
sim(model)
```

```
close_system(model,0)
clear(model)
```

See Also

Test Sequence | “Define Test Steps and Assessments” on page 3-63

Related Examples

- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Generate Test Signals” on page 3-42
- “Use Stateflow Chart for Test Harness Inputs and Scheduling” on page 3-5

Generate Test Signals

In this section...

“Signal Generation Functions” on page 3-42

“Sinusoidal and Random Number Functions in Test Sequences” on page 3-42

In the Test Sequence block, you can generate signals to use for testing. First, define an output data symbol using the **Data Symbols** pane, and then use that output name with a signal generation function in a test step. For information on adding symbols, see “Manage Input, Output, and Data Objects” on page 3-31. For an example that shows how to implement signal functions in a Test Sequence block, see “Sinusoidal and Random Number Functions in Test Sequences” on page 3-42

Signal Generation Functions

You can use these functions in the Test Sequence block to create test signals, random number values, and natural exponents. You can use temporal operators with some signal generation functions. For more information, see “Temporal Operators” on page 3-36.

Note Scaling, rounding, and other approximations of argument values can affect function outputs.

Function	Description
sin	Returns the sine of the input in radians.
cos	Returns the cosine of the input argument in radians.
square	Generates a square wave test signal.
sawtooth	Generates a sawtooth wave test signal.
triangle	Generates a triangle wave test signal.
ramp	Returns the value of the input.
heaviside	Returns 0 when the input is less than or equal to 0, and otherwise returns 1.
exp	Returns the natural exponential function, e^x .
rand	Returns uniformly distributed pseudorandom values.
randn	Returns normally distributed pseudorandom values.
latch	Saves the value of the input and returns the saved value. Resets the saved value when the step exits and reevaluates when the step is next active.

Sinusoidal and Random Number Functions in Test Sequences

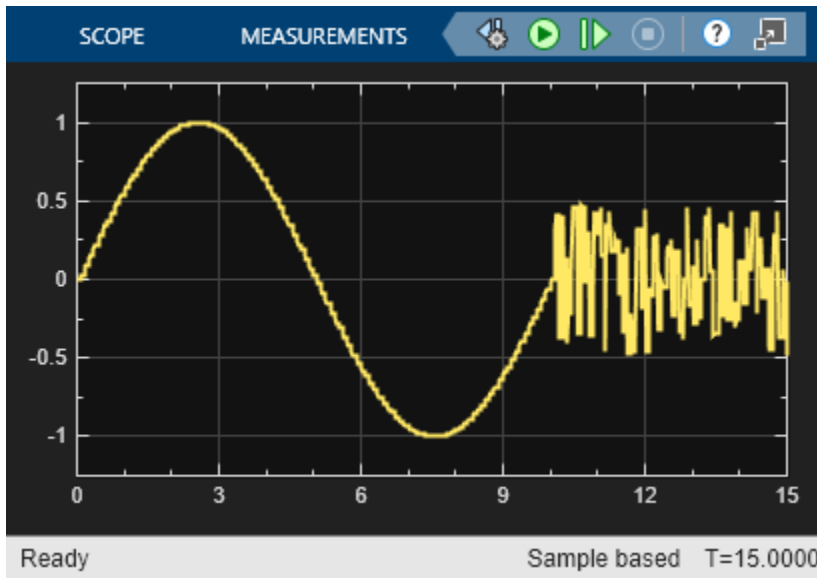
This example shows how to produce a sine and a random number test signal in a Test Sequence block.

If you recreate this test sequence, before running it, set the `nr` symbol size and type. Hover over the `nr` symbol and click its Edit icon to open the Data Inspector. Set the Size to 1 and the Type to double.

The step Sine outputs a sine wave with a period of 10 seconds, specified by the argument $et*2*\pi/10$. The step Random outputs a random number in the interval -0.5 to 0.5 .

Symbols	Step	Transition	Next Step
Input			
Output			
1.  sg			
Local			
nr			
Constant			
Parameter			
Data Store Memory			
	Initialize sg = 0;	1. true	Sine ▼
	Sine sg = sin(et*2*pi/10);	1. after(10,sec)	Stop ▼
	Stop sg = 0;	1. true	Random ▼
	Random coder.extrinsic('rand'); nr = rand; sg = nr - 0.5;	1. after(10,sec)	End ▼
	End sg = 0;		

The test sequence produces signal sg.



See Also

Test Sequence | “Define Test Steps and Assessments” on page 3-63

Related Examples

- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Transitions, Temporal Operators, and Messages in Test Sequence Blocks” on page 3-36
- “Using an External Function in a Test Sequence Block” on page 3-45

Using an External Function in a Test Sequence Block

This example shows how to call an externally-defined function from the Test Sequence block. The provided function, `Attenuate.m`, is defined in a script on the MATLAB® path and is called from the test sequence. The `Attenuate.m` function is:

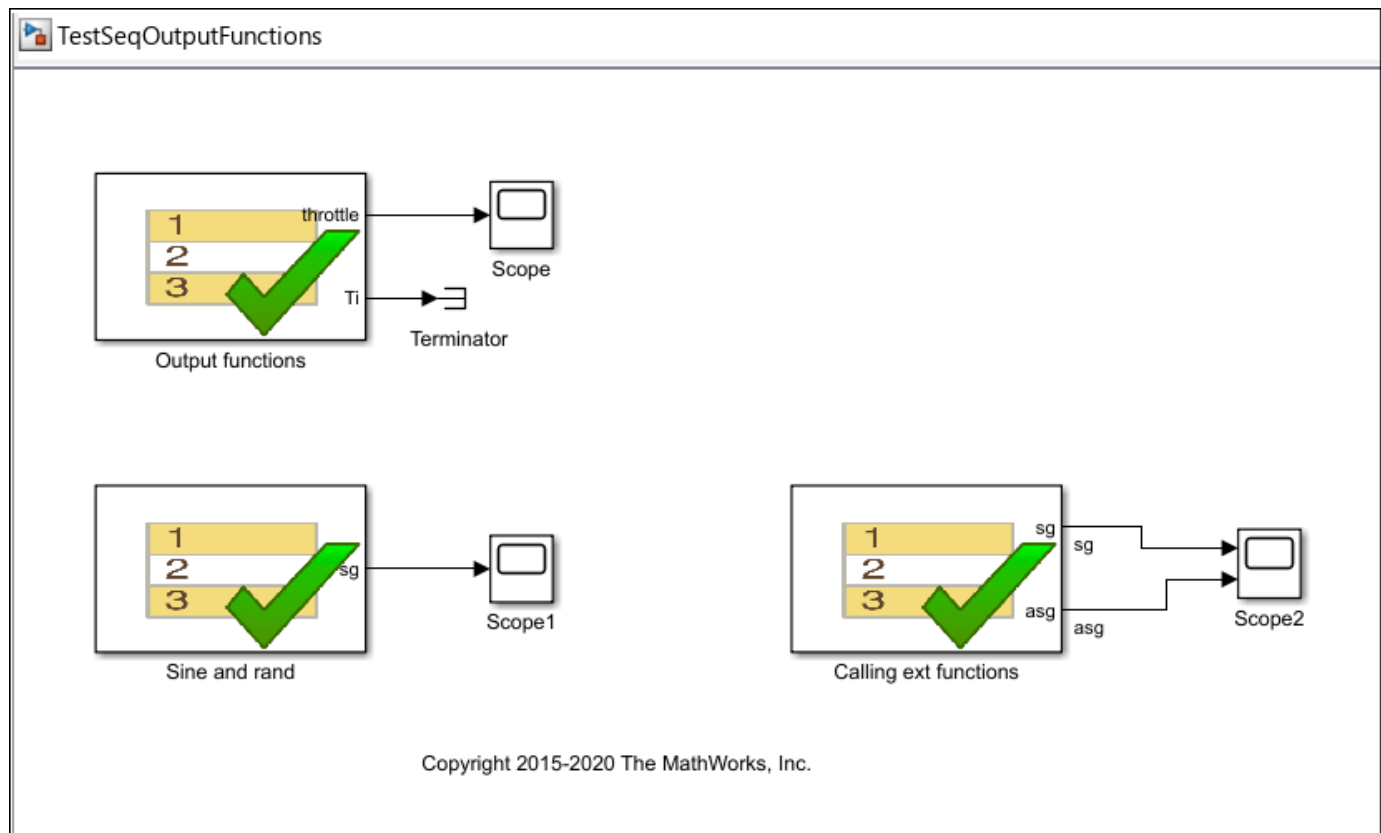
```
function[y] = Attenuate(x)

y = 0.65*x;

end
```

Open the TestSeqOutputFunctions model

```
model = 'TestSeqOutputFunctions';
open_system(model)
```



Open the Test Sequence block

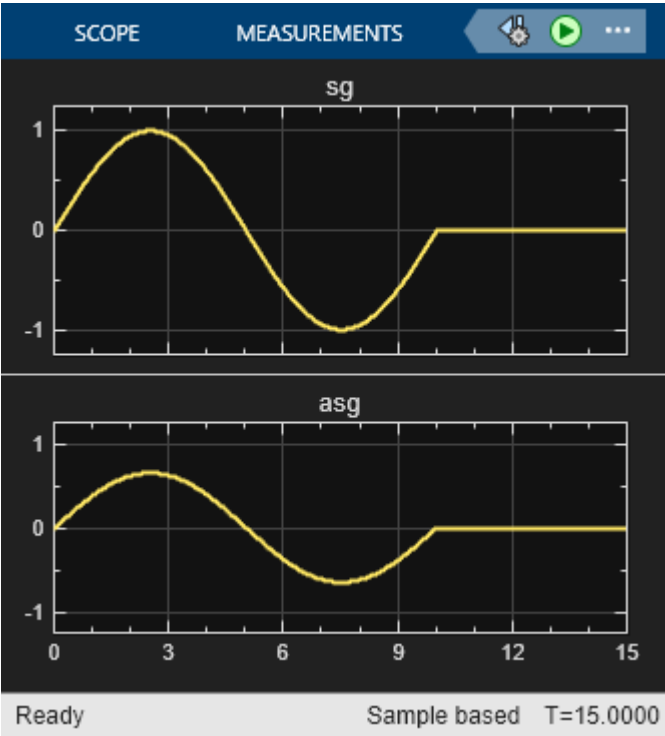
In the model, double-click the Test Sequence block, **Calling ext functions**. The **ReducedSine** step in the Test Sequence block uses the external function, `Attenuate`, to reduce the amplitude of the signal `sg`.

Symbols	Step	Transition	Next Step
Input			
Output			
1. sg	Initialize sg = 0;	1. true	ReducedSine ▼
2. asg	ReducedSine sg = sin(et*2*pi/10); asg = Attenuate(sg);	1. after(10,sec)	Stop ▼
Local			
Constant			
Parameter			
Data Store Memory			
	Stop sg = 0;		

Simulate the model

Simulate the model and view the output signal sg and attenuated signal asg in the Scope2 block.

```
sim(model)
open_system([model '/Scope2'])
```



See Also

More About

- “Test Sequence Basics” on page 3-2
- “Define Test Steps and Assessments” on page 3-63

Programmatically Create a Test Sequence

This example shows how to create a test harness and test sequence using the programmatic interface. You create a test harness that contains a Test Sequence block. You use the block to author a series of test steps that verify two functional attributes of a cruise control system.

Load the Model

```
model = 'sltestCruiseChart';
load_system(model)
```

Create a Test Harness Containing a Test Sequence Block

Create and load the test harness. Set the harness stop time to 15 seconds.

```
sltest.harness.create(model, 'Name', 'Harness1', ...
    'Source', 'Test Sequence');
sltest.harness.load(model, 'Harness1');
set_param('Harness1', 'StopTime', '15');
```

Author the Test Sequence

1. Add a local variable `endTest` and set the data type to `boolean`. You use `endTest` to transition between test steps. Variables in test sequences are called symbols.

```
sltest.testsequence.addSymbol('Harness1/Test Sequence', 'endTest', ...
    'Data', 'Local');
sltest.testsequence.editSymbol('Harness1/Test Sequence', 'endTest', ...
    'DataType', 'boolean');
```

2. Change the name of the Run step to `Initialize1`.

```
sltest.testsequence.editStep('Harness1/Test Sequence', 'Run', ...
    'Name', 'Initialize1');
```

3. Add a `BrakeTest` step, which checks that the cruise control disengages when the brake is applied. Add substeps to the `BrakeTest` step that define the test scenario actions and verifications.

```
sltest.testsequence.addStepAfter('Harness1/Test Sequence', ...
    'BrakeTest', 'Initialize1', 'Action', 'endTest = false;')

% Transition from |Initialize1| to |BrakeTest|.
sltest.testsequence.addTransition('Harness1/Test Sequence', ...
    'Initialize1', 'true', 'BrakeTest')

% Enable the cruise control and set the speed.
% |SetValuesActions| is the actions for BrakeTest.SetValues.
setValuesActions = sprintf('CruiseOnOff = true;\nSpeed = 50;');
sltest.testsequence.addStep('Harness1/Test Sequence', ...
    'BrakeTest.SetValues', 'Action', setValuesActions)

% Engage the cruise control.
setCCActions = sprintf('CoastSetSw = true;');
sltest.testsequence.addStepAfter('Harness1/Test Sequence', ...
    'BrakeTest.Engage', 'BrakeTest.SetValues', 'Action', setCCActions)
```

```
% Apply the brake.
brakeActions = sprintf('CoastSetSw = false;\nBrake = true;');
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'BrakeTest.Brake','BrakeTest.Engage','Action',brakeActions)

% Verify that the cruise control is off.
brakeVerifyActions = sprintf( ...
    'verify(engaged == false)\nendTest = true;');
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'BrakeTest.Verify','BrakeTest.Brake','Action', ...
    brakeVerifyActions)

% Add transitions between steps.
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'BrakeTest.SetValues','true','BrakeTest.Engage')
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'BrakeTest.Engage','after(2,sec)','BrakeTest.Brake')
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'BrakeTest.Brake','true','BrakeTest.Verify')
```

4. Add a second step Initialize2 to initialize component inputs and a transition from BrakeTest to Initialize2.

```
init2Actions = sprintf(['CruiseOnOff = false;\n'...
    'Brake = false;\n'...
    'Speed = 0;\n'...
    'CoastSetSw = false;\n'...
    'AccelResSw = false;']);
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'Initialize2','BrakeTest','Action',init2Actions)
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'BrakeTest','endTest == true','Initialize2')
```

5. Add a LimitTest step, which checks that the cruise control disengages when the vehicle speed exceeds the high limit. Add a transition from the Initialize2 step, and add substeps to define the actions and verification.

```
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'LimitTest','Initialize2')
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'Initialize2','true','LimitTest')

% Enable cruise control and set the speed.
setValuesActions2 = sprintf('CruiseOnOff = true;\nSpeed = 60;');
sltest.testsequence.addStep('Harness1/Test Sequence',...
    'LimitTest.SetValues','Action',setValuesActions2)

% Engage the cruise control.
setCCActions = sprintf('CoastSetSw = true;');
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'LimitTest.Engage','LimitTest.SetValues','Action',setCCActions)

% Ramp up the vehicle speed.
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'LimitTest.RampUp','LimitTest.Engage','Action', ...
    'Speed = Speed + ramp(5*et);')

% Verify that the cruise control is off.
```



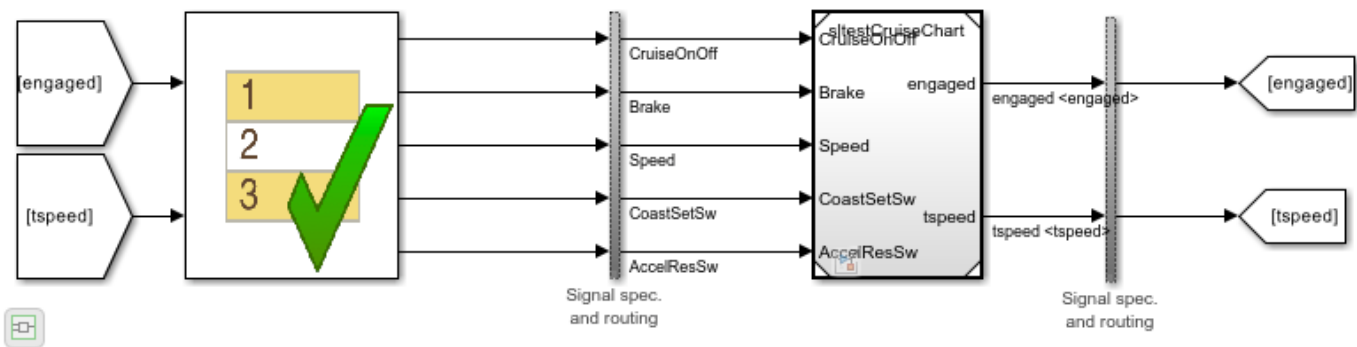
```

highLimVerifyActions = sprintf('verify(engaged == false)');
sltest.testsequence.addStepAfter('Harness1/Test Sequence',...
    'LimitTest.VerifyHigh','LimitTest.RampUp','Action', ...
    highLimVerifyActions)

% Add transitions between steps. The speed ramp transitions when the
% vehicle speed exceeds 90.
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'LimitTest.SetValues','true','LimitTest.Engage')
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'LimitTest.Engage','true','LimitTest.RampUp')
sltest.testsequence.addTransition('Harness1/Test Sequence',...
    'LimitTest.RampUp','Speed > 90','LimitTest.VerifyHigh')
    
```

Open the test harness.

```
sltest.harness.open(model, 'Harness1');
```



Double-click the Test Sequence block to open the editor and view the test sequence.

Step	Transition	Next Step	Description
Initialize1 %% Initialize data outputs. CruiseOnOff = false; Brake = false; Speed = single(0); CoastSetSw = false; AccelResSw = false;	1. true	BrakeT... ▼	
BrakeTest endTest = false;	1. endTest == true	Initialize2 ▼	
SetValues CruiseOnOff = true; Speed = 50;	1. true	Engage ▼	
Engage CoastSetSw = true;	1. after(2,sec)	Brake ▼	
Brake CoastSetSw = false; Brake = true;	1. true	Verify ▼	
Verify verify(engaged == false) endTest = true;			
Initialize2 CruiseOnOff = false; Brake = false; Speed = 0; CoastSetSw = false; AccelResSw = false;	1. true	LimitTest ▼	
LimitTest			

Close the Test Harness and Model

```
sltest.harness.close(model, 'Harness1');
close_system(model, 0);
```

See Also

```
sltest.harness.create | sltest.testsequence.addSymbol |
sltest.testsequence.editSymbol | sltest.testsequence.addStep |
sltest.testsequence.addStepAfter | sltest.testsequence.addTransition
```

Related Examples

- “Define Test Steps and Assessments” on page 3-63

Programmatically Create and Run Test Sequence Scenarios

This example shows how to create and define multiple test scenarios in a single Test Sequence block. Defining more than one test sequence in a block lets you reduce the number of separate Test Sequence blocks in your test harness.

This example uses the `HeatPumpScenario1` model, which already has a test harness that contains a Test Sequence block. In this example, you convert the block to use scenarios, add a new scenario to the block, edit a scenario step, and activate the new scenario so that it runs when the model simulates.

This example also shows how to use scenarios in iterations to run multiple iterations in a single test case.

Open the Model and Test Harness

Open the Controller subsystem of the `HeatPumpScenario1` model and its test harness, `ScenarioTest`.

```
open_system('HeatPumpScenario1')
sltest.harness.open('HeatPumpScenario1/Controller', 'ScenarioTest');
```

Set Up the Scenarios

Enable Scenarios

Set the Test Sequence block to use scenarios. The existing steps and transitions are moved into a scenario that, in this example, is named `FirstScenario`. Note that once you change a Test Sequence block to use scenarios, you cannot revert that block to non-scenario mode.

```
sltest.testsequence.useScenario('ScenarioTest/Test Sequence', ...
    'FirstScenario');
```

Add Another Scenario

Add a second scenario to the Test Sequence block. Name the scenario `NewScenario`.

```
sltest.testsequence.addScenario('ScenarioTest/Test Sequence', ...
    'NewScenario');
```

Edit the Scenario Contents

Edit the first step of the new scenario to change the values of the `Troom_in` and `Tset` variables. Preface the name of the step with the scenario name that contains the step. Similarly, when adding or changing transitions, you must also preface the transition with the scenario name.

```
action = sprintf('Troom_in = 75;\nTset = 75;\n');
sltest.testsequence.editStep('ScenarioTest/Test Sequence', ...
    'NewScenario.step_1', 'Action', action);
```

To view the scenario contents, use `sltest.testsequence.findStep(blockPath)`, which returns an array containing the step names for all scenarios. Then, use `sltest.testsequence.readStep(stepName)` or `sltest.testsequence.readTransition(stepName)` to see the contents of the specified step or transition, respectively. You can also view the scenario contents by double-clicking the Test Sequence block in the harness to open the block editor.

Specify Which Scenario to Run

Specify the new scenario to run during model simulation. This scenario is the *active* scenario, which is the only scenario that runs during the simulation. For an alternative way to activate and run a scenario, or to run scenarios using iterations, see below.

```
sltest.testsequence.activateScenario('ScenarioTest/Test Sequence',...  
    'NewScenario');
```

Run the Model

Run the Model Using the New Scenario

You can run only one active scenario a time, unless you use a loop at the command line or in a script, or run iterations (see below). Note that fast restart is supported when switching active scenarios and running the model.

```
sim('ScenarioTest')
```

In the test harness, view the Scope blocks to see the simulation results for the new scenario.

Run a Different Scenario

Activate the first scenario.

```
sltest.testsequence.activateScenario('ScenarioTest/Test Sequence',...  
    'FirstScenario');
```

Rerun the model.

```
sim('ScenarioTest')
```

In the test harness, view the Scope blocks to see the simulation results for the first scenario.

Use a Workspace Variable to Activate and Run a Scenario

In some cases, such as for looping through scenarios, you might want to use a workspace variable to control which scenario to activate, instead of using `activateScenario`. The steps for using a workspace variable are:

- 1 Set the scenario control source to the workspace by using
`sltest.testsequence.setScenarioControlSource('ScenarioTest/Test Sequence',sltest.testsequence.ScenarioControlSource.Workspace);`
- 2 Create a variable in the base workspace, model workspace, or data dictionary to specify the active scenario using its index value. For example, `Active_Scenario_Index = 1;`
- 3 Run the model, which uses the steps and transitions in the active scenario.

To run a different scenario, change the `Active_Scenario_Index` to the desired scenario, for example, `Active_Scenario_Index = 2`, and then rerun the model.

To change the name of the active scenario parameter from `Active_Scenario_Index` to, for example, `ScenarioIndex`, use `sltest.testsequence.editSymbol('ScenarioTest/Test Sequence',... 'Active_Scenario_Index','Name','ScenarioIndex');`

and then create the `ScenarioIndex` variable in the base workspace. Use `Scenario_Index = 2` to set the variable to run the scenario identified by index 2, and then run the model.

Run Scenarios Using Iterations

You can use iterations to run multiple scenarios in a single test case. The following steps use the same model, Test Sequence block, and scenarios defined above.

1. Set up the test file, test suite, and test case.

```
tf = sltest.testmanager.TestFile('Scenario Iterations Test');
ts = getTestSuites(tf);
tc = createTestCase(ts, 'simulation', 'Sim Iterations');
```

2. Set the model and test harness for the test case.

```
setProperty(tc, 'Model', 'HeatPumpScenario1');
setProperty(tc, 'HarnessOwner', 'HeatPumpScenario1/Controller', ...
    'HarnessName', 'ScenarioTest');
```

3. Get the names of the scenarios in the Test Sequence block.

```
tseq_block = 'ScenarioTest/Test Sequence';
scenarioNames = sltest.testsequence.getAllScenarios(tseq_block);
```

4. Set the Test Sequence block and default scenario for the test case.

```
setProperty(tc, 'TestSequenceBlock', tseq_block);
setProperty(tc, 'TestSequenceScenario', 'FirstScenario');
```

5. Use a loop to create the iterations, assign a scenario to each iteration, and add the iterations to the test case.

```
for i = 1:numel(scenarioNames)
    testItr = sltestiteration;
    setTestParam(testItr, 'TestSequenceScenario', scenarioNames{i});
    addIteration(tc, testItr);
end
```

6. Run the test case.

```
run(tc);
```

Close the Test Harness and Model

```
sltest.harness.close('HeatPumpScenario1/Controller', 'ScenarioTest');
close_system('HeatPumpScenario1', 0);
```

See Also

```
sltest.testsequence.useScenario | sltest.testsequence.setScenarioControlSource |
sltest.testsequence.getActiveScenario | sltest.testsequence.editScenario |
sltest.testsequence.deleteScenario | sltest.testsequence.addScenario |
sltest.testsequence.activateScenario
```

More About

- “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54

Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager

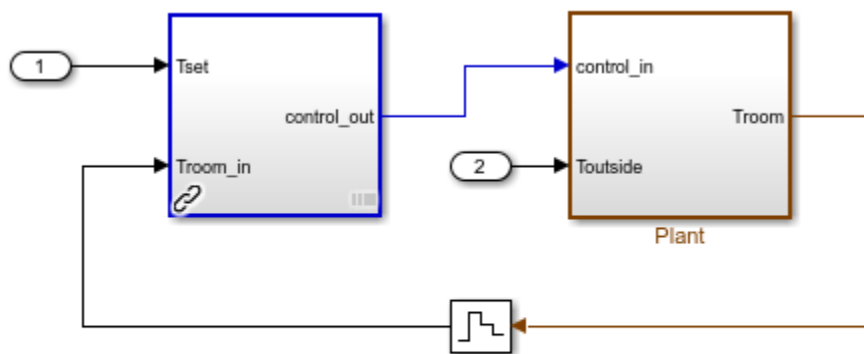
This example shows how to create and use scenarios in the Test Sequence Editor. Scenarios let you include multiple test sequences in a Test Sequence block. If your test harness includes more than one Test Sequence block, you can move each test sequence to a scenario in a single Test Sequence block.

The example also shows how to use the Test Manager to set up and use iterations to run multiple scenarios in a single test case.

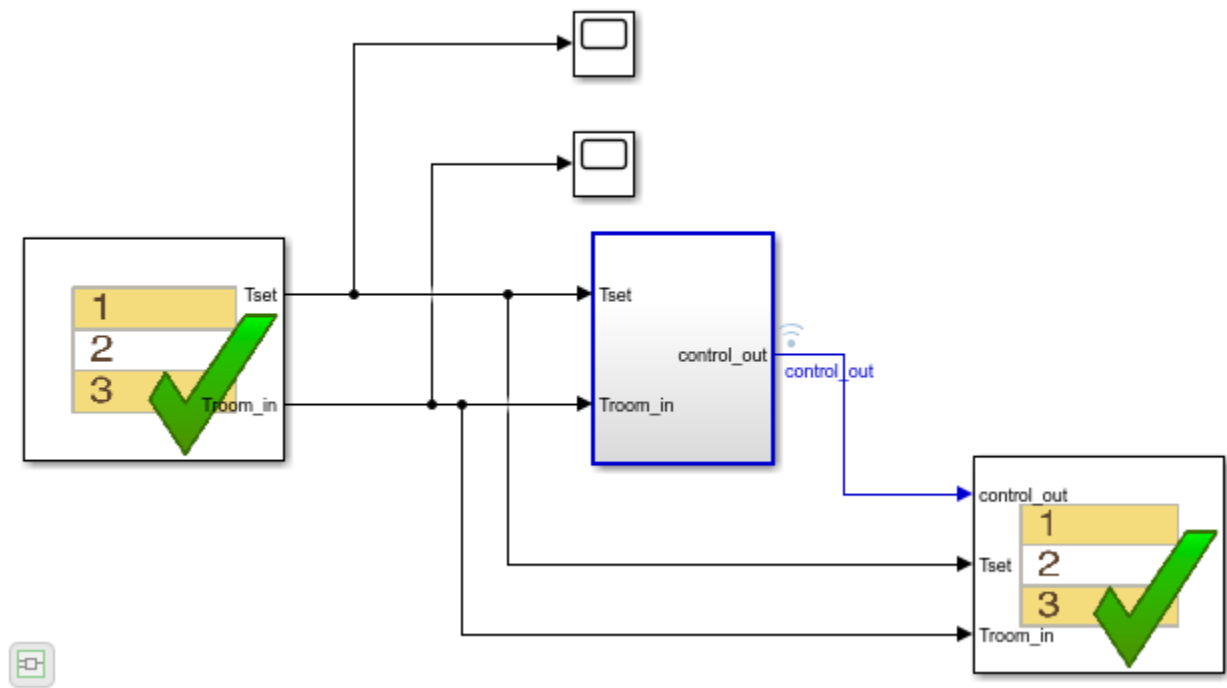
Open the Model and Test Harness

Open the HeatPumpScenario model, ScenarioTest harness, and Test Sequence Editor.

```
open_system('HeatPumpScenario')  
sltest.harness.open('HeatPumpScenario/Controller', 'ScenarioTest');  
open_system('ScenarioTest/Test Sequence')
```

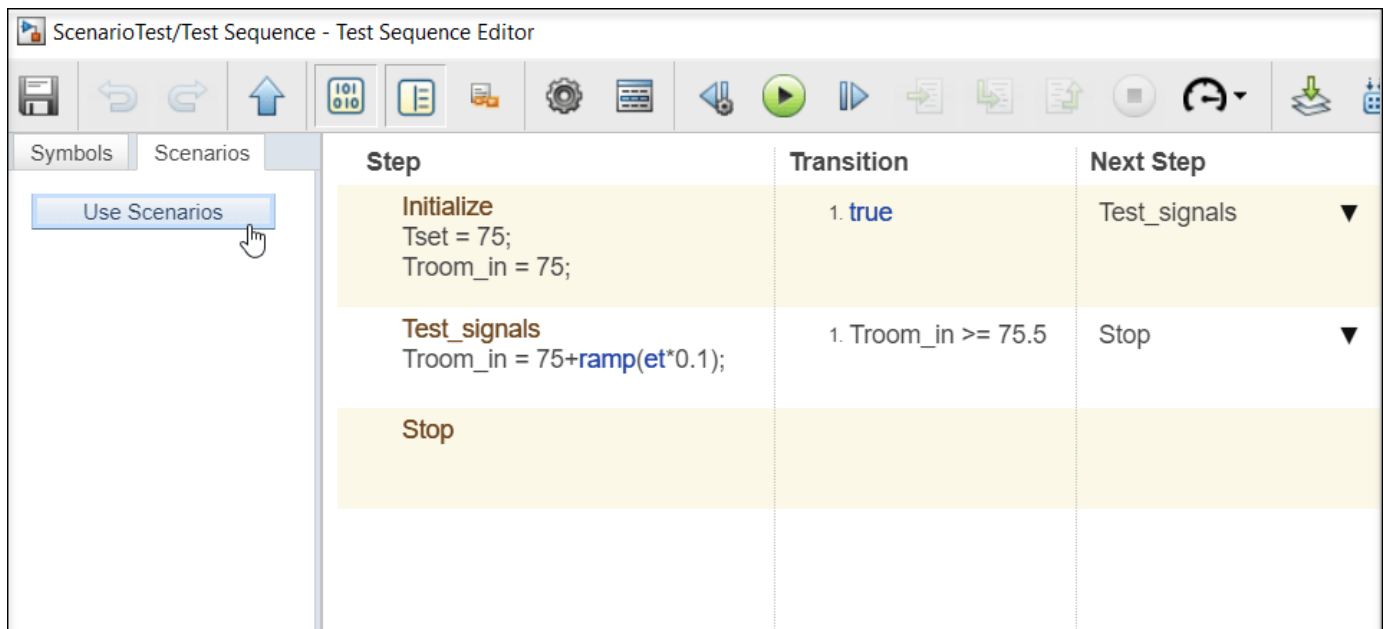


Copyright 1990-2014 The MathWorks, Inc.



Enable Scenarios

In the panel on the left side of the Test Sequence Editor, switch to the **Scenarios** tab and click **Use Scenarios**.



In the **Start Using Scenarios dialog box**, click **OK**, which confirms that when you switch to scenario mode you cannot revert the Test Sequence block to non-scenario mode. The existing steps and transitions are moved into a tab named **Scenario_1**.

Symbols

Scenarios

Scenario List 

1 ⚡ Scenario_1

Scenario Parameter

Active_Scenario_Index

☐ Control active scenario from workspace

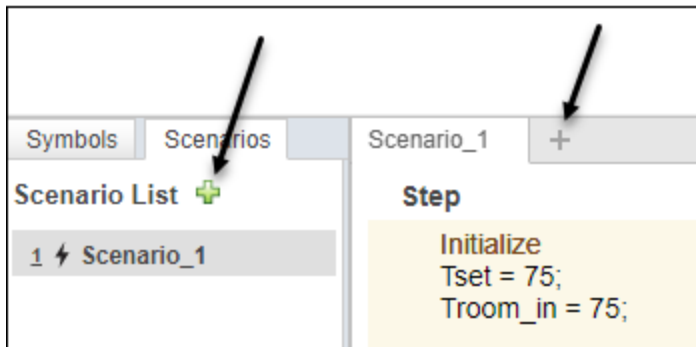
Scenario_1

+

Step	Transition	Next Step
Initialize Tset = 75; Troom_in = 75;	1. true	Test_signals
Test_signals Troom_in = 75+ramp(et*0.1);	1. Troom_in >= 75.5	Stop
Stop		

Add a New Scenario

To add a new scenario, in the **Scenarios** tab, click the plus sign next to **Scenario List**. Alternatively, click the plus sign next to the **Scenario_1** tab label. The new scenario is named **Scenario_2**.

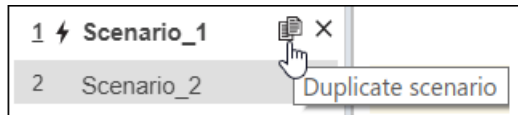
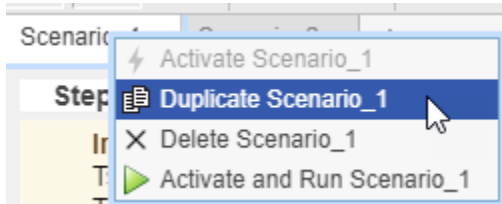


Symbols	Scenarios	Scenario_1	Scenario_2	+
Scenario List + 1 ⚡ Scenario_1 2 Scenario_2		Step step_1 step_2	Transition 1. true	Next Step step_2
Scenario Parameter Active_Scenario_Index ☐ Control active scenario from workspace				

Duplicate a Scenario

To write a new scenario from an existing one, you can duplicate the existing scenario.

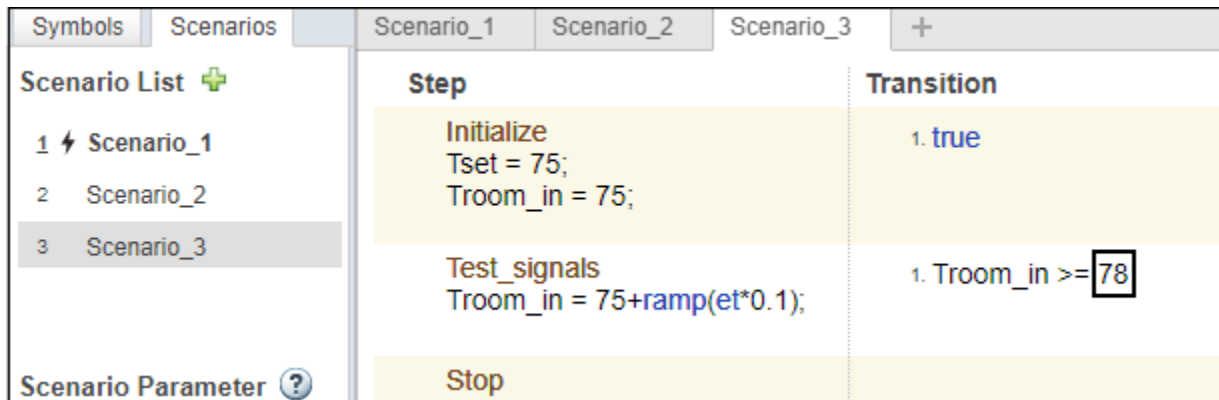
Right-click the **Scenario_1** tab label and select **Duplicate Scenario_1** from the context menu. Alternatively, go to the **Scenarios** side panel, point to **Scenario_1** in the **Scenario List** to display the **Duplicate scenario** button, and click it.



The **Scenario List** section updates and lists the new scenario, **Scenario_3**, which has the same content as **Scenario_1**.

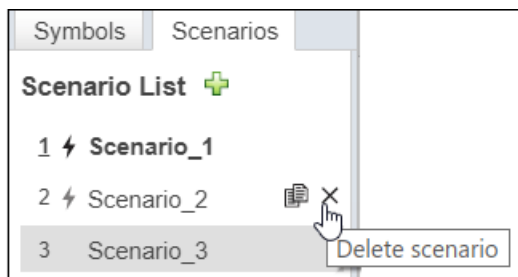
Edit the Steps and Transitions

Modify **Scenario_3** to change the transition of the **Test_signals** step to `Troom_in >= 78`.

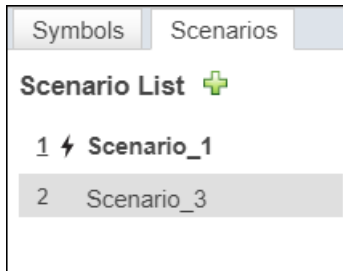


Delete a Scenario

Right-click the **Scenario_2** tab label and select **Delete Scenario_2** from the context menu. Alternatively, in the left pane, in the **Scenarios** tab, point to **Scenario_2** to display the **Delete Scenario** icon. Click the icon and then click **OK** in the dialog box to delete **Scenario_2**.



The name of **Scenario_3** does not change, but the scenario index shown to the left of the scenario name in the **Scenario List** changes to 2 because it is now the second scenario.

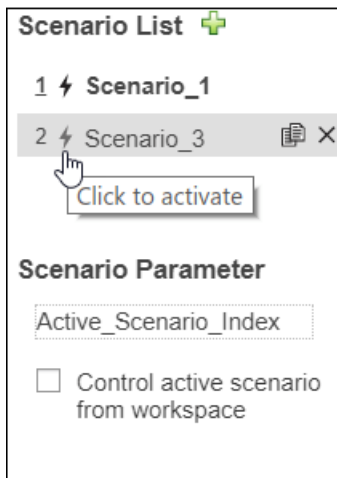


Activate Scenario

In the left pane, in the **Scenario** tab, a black lightning bolt icon and a bold Scenario name indicate that **Scenario_1** is the currently active scenario. When you run the model, only the active scenario runs.

To change the active scenario to **Scenario_3**, right-click the **Scenario_3** tab and select **Activate Scenario_3** from the context menu. Alternatively, in the left pane, in the **Scenarios** tab, point to **Scenario_3** to display the **Click to Activate** icon, which is a gray lightning bolt. Click the lightning bolt to make **Scenario_3** the active scenario.

You can also control the active scenario from the command line. See the **Programmatically Control the Active Scenario** section below.



Run the Active Scenario

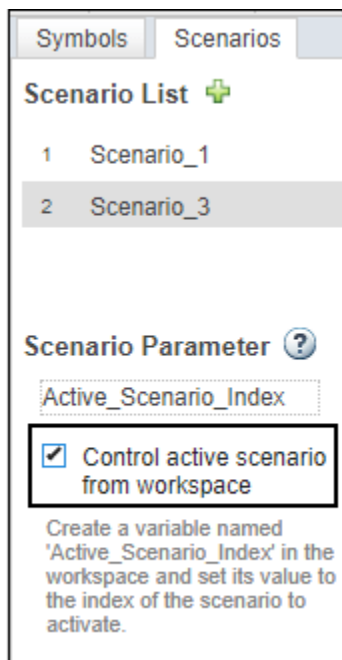
Right-click the **Scenario_3** tab label and select **Run Scenario_3** from the context menu. If you did not activate the scenario, the menu option is **Activate and Run Scenario_3**. Alternatively, if you run the model in the usual way, the active scenario runs. The Test Sequence Editor displays the active scenario during model simulation.

Programmatically Control the Active Scenario

You can alternatively use a variable in the base workspace, model workspace, or a data dictionary to programmatically control the active scenario. To activate a scenario, set the variable value to the index of the scenario.

1. In the Test Sequence Editor **Scenarios** tab, go to the **Scenario Parameter** section.
2. Enable **Control active scenario from workspace**. The previously active scenario deactivates. When you control the active scenario using a workspace variable, the Test Sequence Editor does not know which scenario is active until you click **Run**.

If you select **Control active scenario from workspace**, you cannot activate a scenario from the Test Sequence Editor using the right-click context menu or the **Scenario List**.



3. For this example, in the base workspace, create a variable named `Active_Scenario_Index` and set it to activate the first scenario, `Scenario_1`, by entering: `Active_Scenario_Index = Simulink.Parameter(1);`
4. Return to the Test Sequence Editor and click **Run**. `Scenario_1` runs.

Instead of using `Active_Scenario_Index` as the name of the variable, you can specify a different name.

1. For this example, in the Test Sequence Editor, in the **Scenario Parameter** section, click on `Active_Scenario_Index` and enter a new name, such as `ChangeScenario`.
2. In the base workspace, create a variable named `ChangeScenario`. Set it to the desired scenario index, such as 1, by entering: `ChangeScenario = Simulink.Parameter(1);`
3. Return to the Test Sequence Editor and click **Run**. `Scenario_1` runs.

Use Iterations to Run Multiple Scenarios in a Test Case

You can run multiple scenarios in a test case by using iterations. This section describes how to set up and run scenarios in iterations by using the Test Manager.

1. In the Test Manager, create a test file, test suite, and simulation test case.
2. In the **System Under Test** section, set the **Model** to HeatPumpScenario and the **Harness** to ScenarioTest.
3. In the **Inputs** section, click the Refresh icon next to the **Test Sequence Block** field to populate it with paths to Test Sequence blocks in the harness.
4. Set **Test Sequence Block** to the ScenarioTest/Test Sequence block, which has the scenarios to use in the iterations.
5. Click the Refresh icon next to the **Override with Scenario** field to populate it with the scenarios in the selected block.
6. Set **Override with Scenario** to Scenario_1, which set that scenario as the default for all iterations. This scenario overrides the active scenario in the Test Sequence block. In the **Iterations** section, you can change this default scenario to another scenario for each iteration.

▼ INPUTS*

☐ Include input data in test result

☐ Stop simulation at last time point

EXTERNAL INPUTS

NAME	FILE
No inputs available. Click Add to add new inputs.	

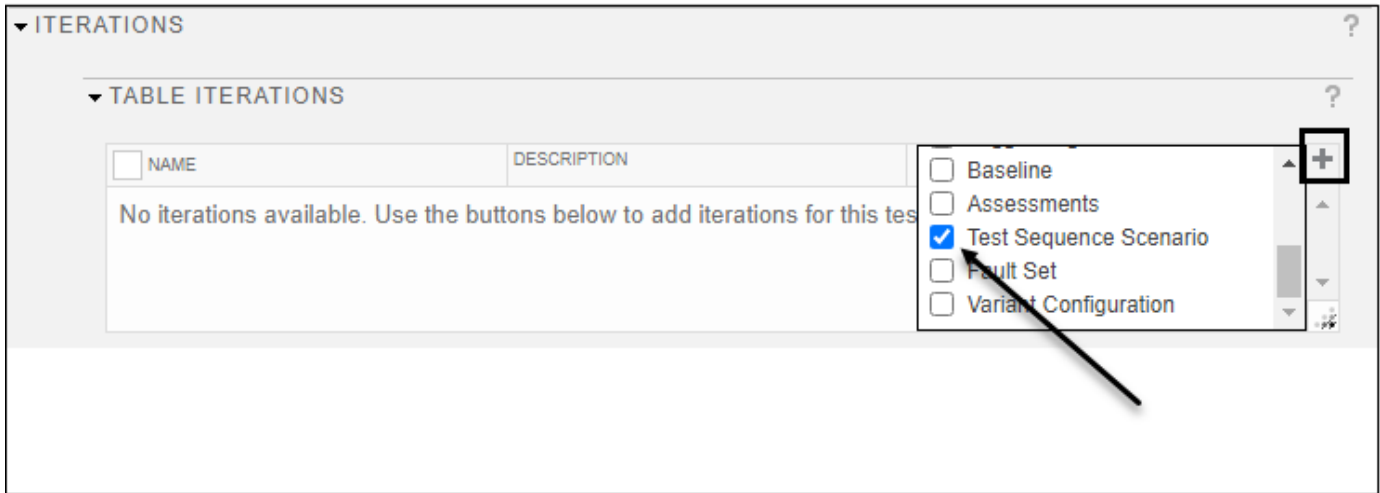
☐ Signal Editor Scenario [Model Settings]

Test Sequence Block: ScenarioTest/Test Sequence

Override with Scenario: Scenario_1

If you do not select a scenario, the active scenario in the Test Sequence block is used as the default.

7. In the **Iterations** section, expand **Table Iterations**, click the plus sign in the upper right of the table and select **Test Sequence Scenario** to add that column to the table.



8. Click **Add** at the bottom of the **Table Iterations** to add the individual iterations. The **Test Sequence Scenario** column for each iteration shows the default scenario. In this case, it shows [Default] Scenario_1.

Alternatively, click **Auto Generate** and select **Test Sequence Scenario** to generate an iteration for each scenario in the selected Test Sequence block. In the **Test Sequence Scenario** column, each iteration is assigned a separate iteration.

9. In the **Test Sequence Scenario** column of an iteration, click the scenario name to view a list of available scenarios. Select a different scenario from the default for one of the iterations.

10. Run the test.

See Also

sltest.harness.open

More About

- “Programmatically Create and Run Test Sequence Scenarios” on page 3-51
- “Test Sequence Editor” on page 3-29

Scenario Parameter Section

Use the options in the Scenario Parameter section to control the active scenario with a variable in the base workspace, model workspace, or a data dictionary.

- 1** Enable **Control active scenario from workspace**. The previously active scenario is deactivated and no active scenario is shown in the Scenario List. When you control the active scenario using a workspace variable, the Test Sequence Editor does not know which scenario is active until you click **Run**.
- 2** If desired, edit `Active_Scenario_Index` to change the variable name.
- 3** Create a variable with the same name in the base workspace, model workspace, or a data dictionary. For example, in the base workspace, create a variable named `Active_Scenario_Index`.
- 4** Set the value of the variable to the index of the scenario to activate. For example, in the base workspace, use `Active_Scenario_Index = Simulink.Parameter(2)` to activate the second scenario.
- 5** Click **Run** in the Test Sequence Editor to run the scenario.

Define Test Steps and Assessments

In this section...
"Assessment Statements" on page 3-63 "Temporal Operators" on page 3-63 "Transition Operators" on page 3-64 "Signal Generation Functions" on page 3-64 "Logical Operators" on page 3-65 "Relational Operators" on page 3-65

You can assess your model simulation by creating test step actions, transitions, and assessments in Test Sequence and Test Assessment blocks

Test Sequence and Test Assessment blocks use MATLAB as the action language. Steps and transitions can include strings, including string comparisons, as well as assessment statements, signal generation functions, and relational operators. For more information, see Test Assessment, Test Sequence, and "Test Sequence Editor" on page 3-29.

Assessment Statements

To verify simulation, stop simulation, and return verification results, use assessment statements.

Keyword	Description
verify	Evaluates a logical expression and returns a pass , fail , or untested result. You can use optional arguments to label results in the Test Manager and Diagnostic Viewer or return an error message if a verify statement fails.
assert	Evaluates a logical expression and stops simulation if the expression evaluates to false . You can use optional arguments to return an error message in the Test Manager or Diagnostic Viewer.

Temporal Operators

To create an expression that evaluates the simulation time, use temporal operators. Variables in signal conditions must be inputs, parameters, or constants in the Test Sequence block. You can use these temporal operators:

Operator	Description
et	The elapsed time of the test step. You can specify the unit of time. If you do not specify a unit of time, the elapsed time is returned in seconds.

Operator	Description
t	The elapsed time of the simulation. You can specify the unit of time. If you do not specify a unit of time, the elapsed time is returned in seconds.
after	Returns <code>true</code> if the specified time has elapsed since the beginning of the current test step.
before	Returns <code>true</code> until the specified time has elapsed from the beginning of the current test step.
duration	Returns the elapsed time for which the specified condition has been <code>true</code> . The elapsed time resets when the test step is re-entered or when the specified condition is no longer <code>true</code> .

Transition Operators

To create expressions that evaluate signal events, use transition operators. To evaluate signal events, the signals must be inputs in the Test Sequence or Test Assessment block.

Operator	Description
hasChanged	Returns <code>true</code> if the specified signal changes in value since the beginning of the test step, and otherwise returns <code>false</code> .
hasChangedFrom	Returns <code>true</code> if the specified signal changes from the specified value to a different value, and otherwise returns <code>false</code> .
hasChangedTo	Returns <code>true</code> if the specified signal changes to the specified value, and otherwise returns <code>false</code> .

Signal Generation Functions

You can use these functions in the Test Sequence block to create test signals, random number values, and natural exponents. You can use temporal operators with some signal generation functions. For more information, see “Temporal Operators” on page 3-36.

Note Scaling, rounding, and other approximations of argument values can affect function outputs.

Function	Description
sin	Returns the sine of the input in radians.
cos	Returns the cosine of the input argument in radians.
square	Generates a square wave test signal.
sawtooth	Generates a sawtooth wave test signal.
triangle	Generates a triangle wave test signal.

Function	Description
ramp	Returns the value of the input.
heaviside	Returns 0 when the input is less than or equal to 0, and otherwise returns 1.
exp	Returns the natural exponential function, e^x .
rand	Returns uniformly distributed pseudorandom values.
randn	Returns normally distributed pseudorandom values.
latch	Saves the value of the input and returns the saved value. Resets the saved value when the step exits and reevaluates when the step is next active.

Logical Operators

You can use logical connectives in actions, transitions, and assessments. In these examples, p and q represent Boolean signals or logical expressions.

Operation	Syntax	Description	Example
Negation	$\sim p$	not p	<code>verify($\sim p$)</code>
Conjunction	$p \ \&\& \ q$	p and q	<code>verify($p \ \&\& \ q$)</code>
Disjunction	$p \ \ q$	p or q	<code>verify($p \ \ q$)</code>
Implication	$\sim p \ \ q$	if p , q . Logically equivalent to implication $p \rightarrow q$.	<code>verify($\sim p \ \ q$)</code>
Biconditional	$(p \ \&\& \ q) \ \ (\sim p \ \&\& \ \sim q)$	p and q , or not p and not q . Logically equivalent to biconditional $p \leftrightarrow q$.	<code>verify($(p \ \&\& \ q) \ \ (\sim p \ \&\& \ \sim q)$)</code>

Relational Operators

You can use relational operators in actions, transitions, and assessments. In these examples, x and y represent numeric-type variables.

Using `==` or `~=` operators in a `verify` statement returns a warning when comparing floating-point data. Consider the precision limitations associated with floating-point numbers when implementing `verify` statements. See “Floating-Point Numbers”. If you use floating-point data, consider defining a tolerance for the assessment. For example, instead of `verify( $x == 5$ )`, verify x within a tolerance of 0.001:

```
verify(abs(x-5) < 0.001)
```

Operator and Syntax	Description	Example
$x > y$	Greater than	<code>verify($x > y$)</code>

Operator and Syntax	Description	Example
<code>x < y</code>	Less than	<code>verify(x < y)</code>
<code>x >= y</code>	Greater than or equal to	<code>verify(x >= y)</code>
<code>x <= y</code>	Less than or equal to	<code>verify(x <= y)</code>
<code>x == y</code>	Equal to	<code>verify(x == y)</code>
<code>x ~= y</code>	Not equal to	<code>verify(x ~= y)</code>

See Also

Related Examples

- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Transitions, Temporal Operators, and Messages in Test Sequence Blocks” on page 3-36
- “Generate Test Signals” on page 3-42
- “Programmatically Create a Test Sequence” on page 3-47

Debug a Test Sequence

This example shows how to debug a test sequence by using the Test Sequence Editor. Debugging involves setting breakpoints to stop simulation, observing data and test sequence progression, and manually stepping through test steps.

Open the model and the Test Sequence Editor block

```
TestSeqDebug
open_system('TestSeqDebug/Test Sequence')
```

View Test Step Execution During Simulation

By default, simulation animates the test sequence by highlighting active steps and transitions. Observing test step execution can help you debug, particularly when manually stepping through the test sequence. Adjust the animation speed using the **Change Animation Speed** button  in the toolbar.

Animation speed affects simulation speed. If you slow down animation speed for debugging, return the speed to **Fast** or **Lightning Fast** when you finish debugging to avoid slowing your simulation. If you do not need the test step highlights and want the fastest simulation, choose **None**.

Set Breakpoints to Enable Debugging

Enable debugging for a test sequence by adding one or more regular or conditional breakpoints. Regular breakpoints halt simulation every time the test step is evaluated. Therefore, breakpoints on some test steps, such as **When decomposition** parent steps, halt simulation repeatedly because the step is evaluated repeatedly. Conditional breakpoints halt simulation only when the specified condition is met. When simulation halts, you can view the data used in the test sequence to investigate the sequence simulation behavior.

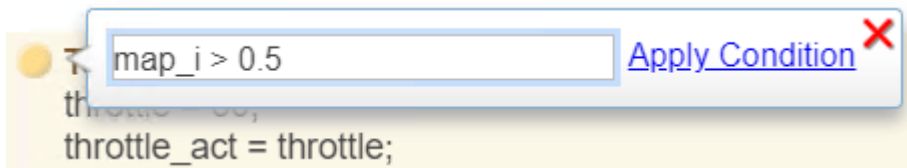
You can add regular and conditional breakpoints to test step actions and transitions.

- To add a regular breakpoint to a test step, right-click the step or action and select **Break while executing step**. For a transition, point to the transition, click the gear icon , and select **Break when transition taken**. A red icon indicates a regular breakpoint.

Step	Transition
 Stabilize_Engine throttle = 20; % 0 ==> sensor fai speed = 300; % 0 ==> sensor fai ego = ego_i; % 12 ==> sensor fa map = map_i; % 0 ==> sensor fai	1. after(10, sec)

Step	Transition
Stabilize_Engine throttle = 20; % 0 ==> sensor f speed = 300; % 0 ==> sensor f ego = ego_i; % 12 ==> sensor map = map_i; % 0 ==> sensor f	 1. after(10, sec)

- To add a conditional breakpoint, first add a regular breakpoint. Then, right-click the breakpoint icon and select **Set or Modify Condition**. In the text field of the dialog box, specify the condition to apply to the step or transition and click **Apply Condition**. To indicate that it is a conditional breakpoint, the icon color changes to yellow.

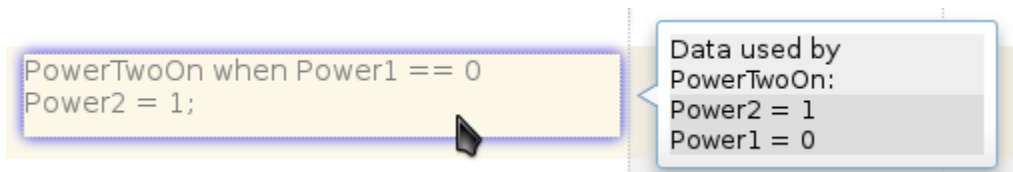


- To change a conditional breakpoint back to a regular breakpoint, right-click the breakpoint icon and select **Set or Modify Condition**. Delete the conditional text and click **Apply Condition**. The breakpoint icon color changes to red.
- You can remove a breakpoint using these methods:
 - Click the breakpoint icon.
 - Right-click the breakpoint icon and select **Clear Breakpoint**.
- For a step or action, right-click the step breakpoint icon and deselect **Break while executing step**. For a transition, point to the transition, click the gear, and deselect **Break when transition taken**.

After adding breakpoints, simulate the test sequence by clicking **Run**.

View Data Values During Simulation

If the simulation pauses (for example, at a breakpoint), you can view the status of data used in a test step by hovering over the test step. The data values at the current simulation time display next to the test sequence cell.



If you advance the simulation to another stop (for example, using the keyboard shortcuts), the data display does not update. Move off the test step and then hover over the step again to refresh the values.

Step Through Simulation

When simulation halts, you can step through the test sequence using these toolbar buttons.



Simulate until breakpoint - Simulation runs until the next breakpoint



Step forward through simulation time - Simulation advances one simulation step



Step forward through test step actions and transitions - Simulation advances by each step of a test sequence, with pauses at actions and transitions. Does not step into a function call.



Step in to a test step group or called function - Simulation advances into the substeps of a parent step and executes each action and transition. Steps into a function call.



Step out of a test step group or called function - Simulation advances through the remaining substeps of a parent step and then out to the parent step hierarchy level. Also finishes execution of a function call.

See Also

Test Sequence | “Test Sequence Editor” on page 3-29

Test Downshift Points of a Transmission Controller

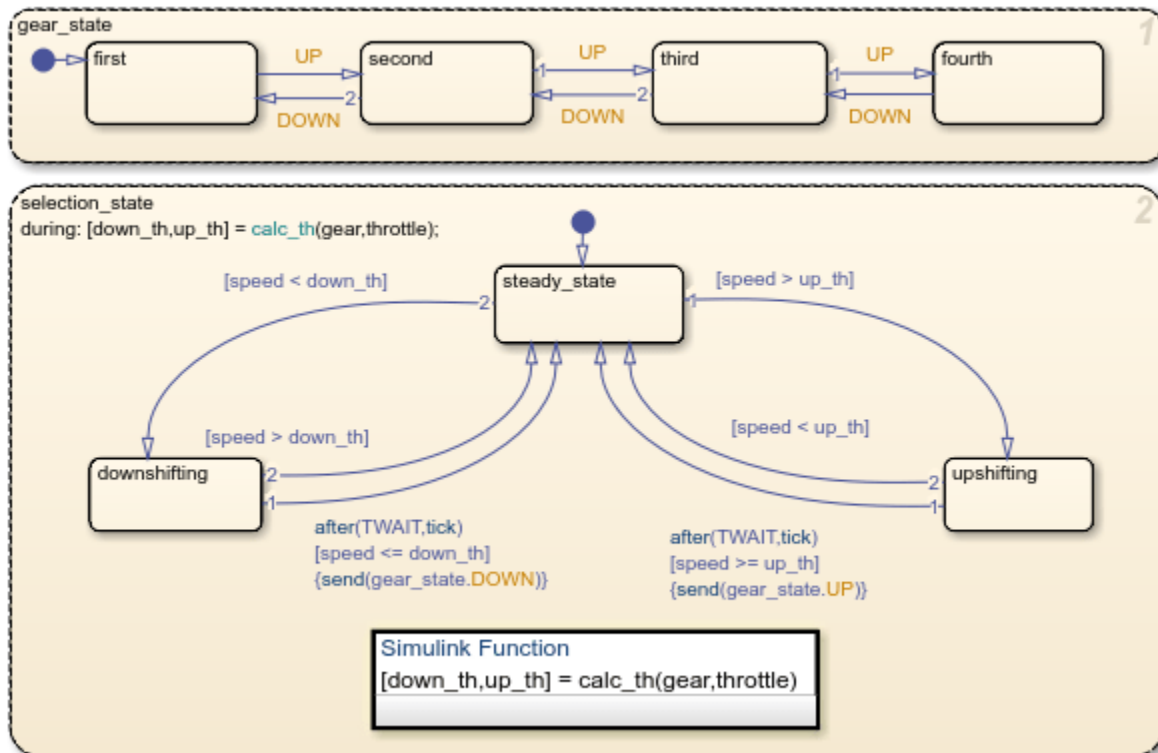
This example demonstrates how to test a transmission shift logic controller using test sequences and test assessments.

The Model and Controller

This example uses the `sf_car` model, which is a simplified drivetrain system arranged in a controller-plant configuration. The objective is to unit test the downshift behavior of the transmission controller.

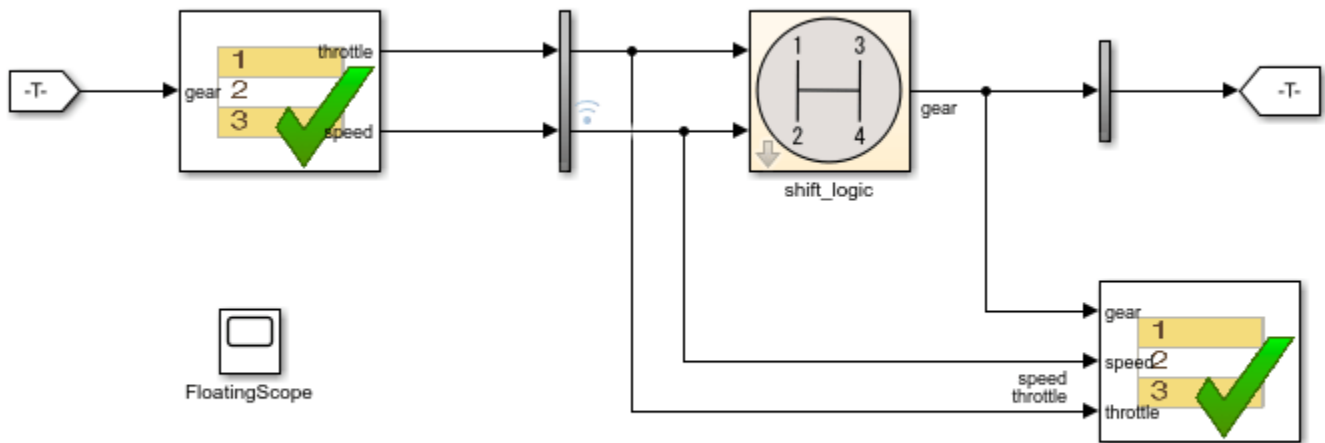
The Test

The controller should downshift between gear ratios in response to an increasing throttle application. The test inputs hold vehicle speed constant while ramping the throttle. The Test Assessment block includes requirements-based assessments of the controller performance.



Open the Test Harness

Click the badge on the subsystem `shift_logic` and open the test harness `TestDownshiftPointsHarness`. The test harness contains a Test Sequence block and a Test Assessment block connected to the controller subsystem.



Copyright 2014-2024 The MathWorks, Inc.

The Test Sequence

Double-click the Test Sequence block to open the Test Sequence Editor.

The test sequence ramps speed to 75 to initialize the controller in fourth gear. Throttle is then ramped at constant speed until a gear change. Subsequent initialization and downshifts execute. After the change to first gear, the test sequence stops.

Step	Transition	Next Step
initialize_4_3 throttle = 10; speed = 0+ramp(25*et);	1. speed == 75	down_4_3 ▼
down_4_3 throttle = 10+ramp(10*et); speed = 75;	1. hasChanged(gear)	initialize_3_2 ▼
initialize_3_2 throttle = 10; speed = 45;	1. after(4,sec)	down_3_2 ▼
down_3_2 throttle = 10+ramp(10*et); speed = 45;	1. hasChanged(gear)	initialize_2_1 ▼
initialize_2_1 throttle = 10; speed = 15;	1. after(4,sec)	down_2_1 ▼
down_2_1 throttle = 10+ramp(10*et); speed = 15;	1. hasChanged(gear)	stop ▼
stop throttle = 0; speed = 0;		

Test Assessments for the Controller

This example tests the following conditions:

- Speed value shall be greater than or equal to 0.
- Gear value shall be greater than 0.
- Throttle value shall be between 0 and 100.
- The shift controller shall keep the vehicle speed below specified maximums in each of the first three gears.

Open the Test Assessment block. The `assert` statements correspond to the first three conditions. If the controller violates an assertion, the simulation fails.

```
assert(speed >= 0, 'speed must be >= 0');
assert(throttle >= 0, 'throttle must be >= 0 and <= 100');
assert(throttle <= 100, 'throttle must be >= 0 and <= 100');
assert(gear > 0, 'gear must be > 0');
```


The last condition is checked by three `verify` statements corresponding to the maximum speeds in gears 3, 2, and 1:

- Vehicle speed shall not exceed 90 in gear 3.
- Vehicle speed shall not exceed 50 in gear 2.
- Vehicle speed shall not exceed 30 in gear 1.

A When decomposition sequence contains the **verify** statements. In the When decomposition sequence, signal conditions determine the active step. A step includes a condition preceded by the **when** operator. The last step **Else** covers undefined conditions and does not use a **when** statement. For more information on When decomposition, see "Transition Types" in "Test Sequence Basics" on page 3-2.

```
OverSpeed3 when gear==3
verify(speed <= 90, 'Engine overspeed in gear 3')
```

```
OverSpeed2 when gear==2
verify(speed <= 50, 'Engine overspeed in gear 2')
```

```
OverSpeed1 when gear==1
verify(speed <= 30, 'Engine overspeed in gear 1')
```

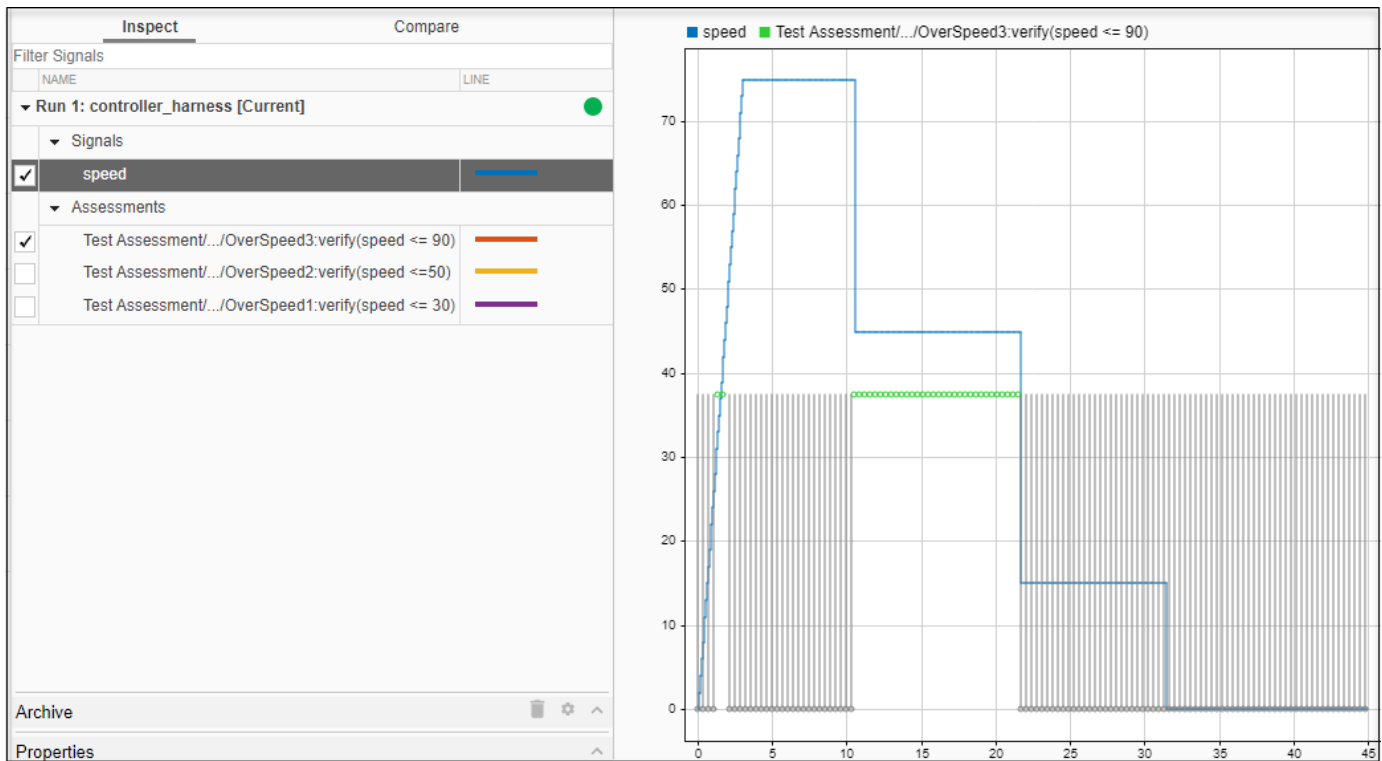
Step	Transition	Next Step
AssertConditions % These conditions ensure simulation validity. <pre>assert(speed >= 0, 'speed must be >= 0'); assert(throttle >= 0, 'throttle must be >= 0 and <= 100'); assert(throttle <= 100, 'throttle must be >= 0 and <= 100'); assert(gear > 0, 'gear must be > 0');</pre>		
OverSpeed3 when gear==3 % Verify speed within specified range for 3rd gear <pre>verify(speed <= 90,'Engine overspeed in gear 3')</pre>		
OverSpeed2 when gear==2 % Verify speed within specified range for 2nd gear <pre>verify(speed <=50,'Engine overspeed in gear 2')</pre>		
OverSpeed1 when gear==1 % Verify speed within specified range for 1st gear <pre>verify(speed <= 30,'Engine overspeed in gear 1')</pre>		
Else % Else step required for any conditions not corresponding to % the above three when conditions		

Testing the Controller

Simulating the test harness demonstrates the progressive throttle ramp at each test step and the corresponding downshifts. The controller passes all of the assessments in the Test Assessment block.

View the Results

Click the Simulation Data Inspector button in the test harness toolstrip to view the results. You can compare the speed signal to the `verify` statement outputs.



See Also

Test Assessment

Related Examples

- “Test Sequence Basics” on page 3-2
- “Test Sequence Editor” on page 3-29
- “Define Test Steps and Assessments” on page 3-63

Examine Model Verification Results by Using Simulation Data Inspector

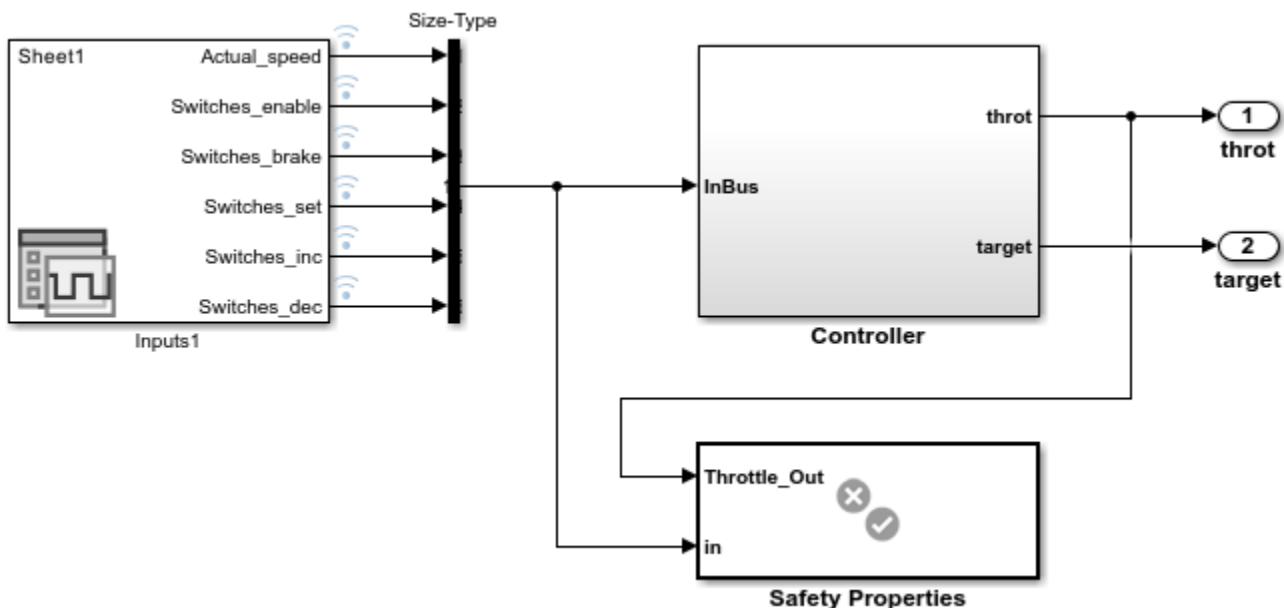
This example shows how to use the Simulation Data Inspector to view the output from a model verification block in a system under test. If you have Simulink® Test™, model verification blocks return Pass, Fail, or Untested results at each time step. By examining the results of a model verification block, you can:

- Determine the simulation time when a failure occurs.
- Compare the verification results with other relevant signals.
- Trace failures from the Simulation Data Inspector back to the model.

For more information, see “Model Verification Blocks” on page 3-13.

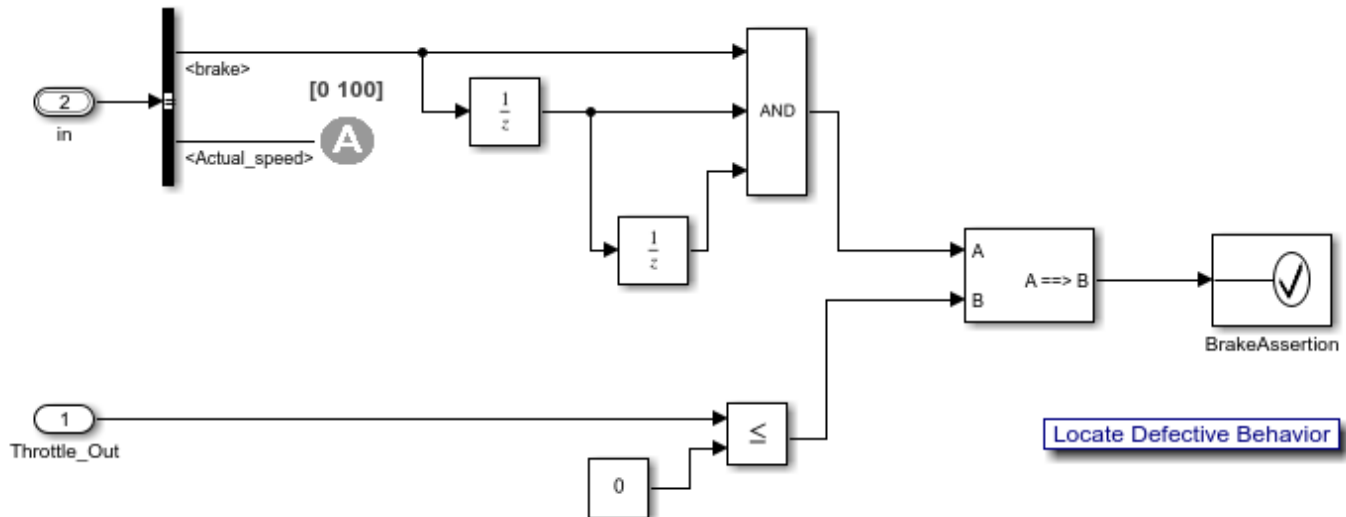
Verify Model Behavior With Assertion Block

In this example, the subsystem block **Controller** models the cruise control system in a car. This subsystem outputs the throttle value based on the difference between the actual and target speeds.



The verification subsystem **Safety Properties** uses an Assertion block to check that the system disengages when the brake is applied for three consecutive time steps.

Property: When the brake is applied for three consecutive steps, the throttle goes to zero.

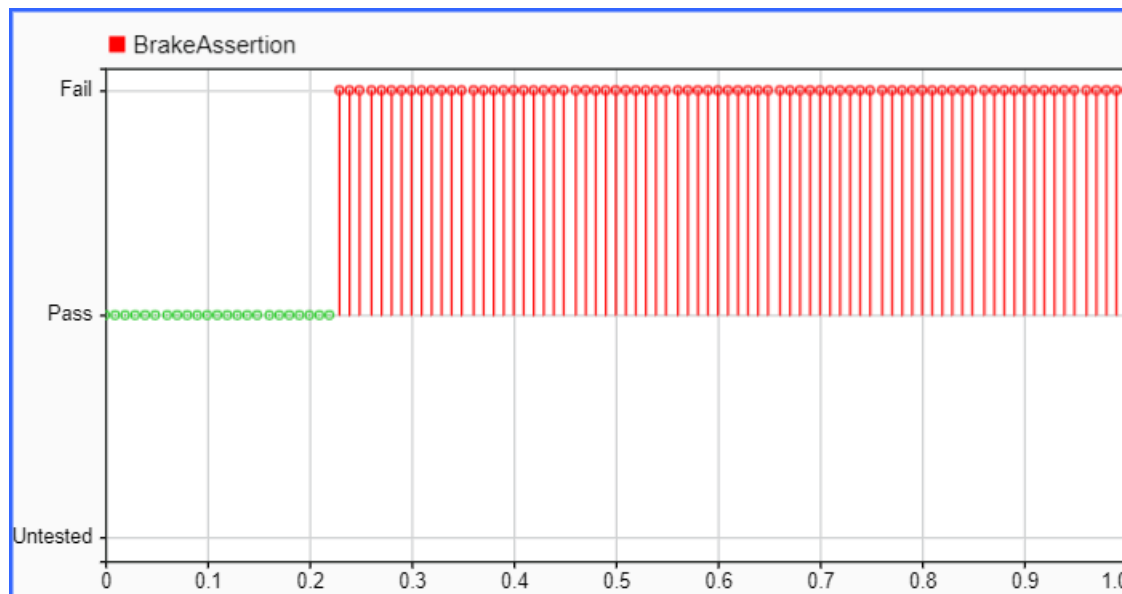


Determine Simulation Time of Failure

Simulate the model and view the output of the Assertion block in the Simulation Data Inspector.

- 1 In the **Simulation** tab, click **Run**.
- 2 In the **Simulation** tab, under **Review Results**, select **Data Inspector**.
- 3 In the Simulation Data Inspector navigation pane, select BrakeAssertion.

The results show that the assertion fails at 0.23 seconds.

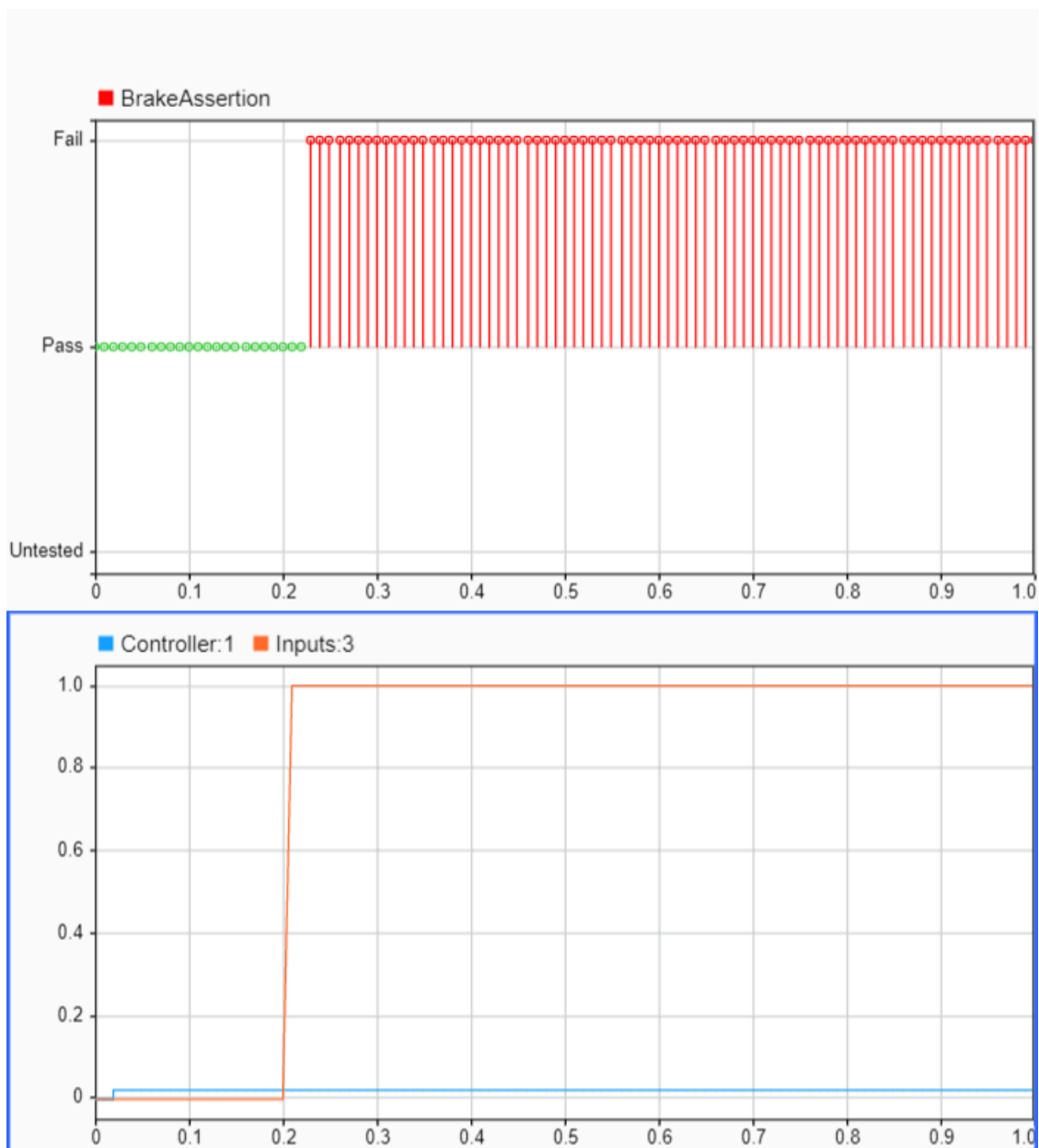


Compare Verification Results with Other Signals

Examine the cause of the failure by plotting the values of the brake and throttle signals.

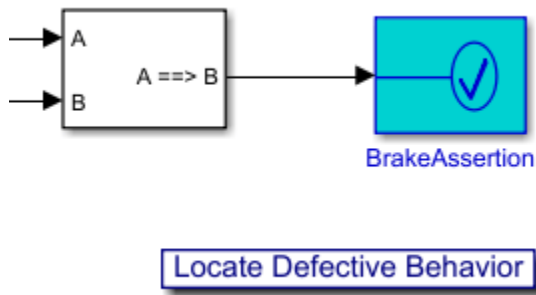
- 1 Right-click the `throt` signal and select **Log Selected Signals**.
- 2 Simulate the model.
- 3 Configure the Simulation Data Inspector with two subplots.
- 4 In the Simulation Data Inspector navigation pane, select the signals to plot. For the first subplot, select `BrakeAssertion`. For the second subplot, select `Controller:1` (throttle) and `Inputs:3` (brake).

The results show that pressing the brake at 0.2 seconds does not disengage the throttle.



Trace Failure Back to the Model

Find the block that produces a verification result by tracing the result from the Simulation Data Inspector back to the model. In the Simulation Data Inspector navigation pane, right-click `BrakeAssertion` and select **Highlight in Model**. The editor opens the verification subsystem and highlights the Assertion block.



See Also

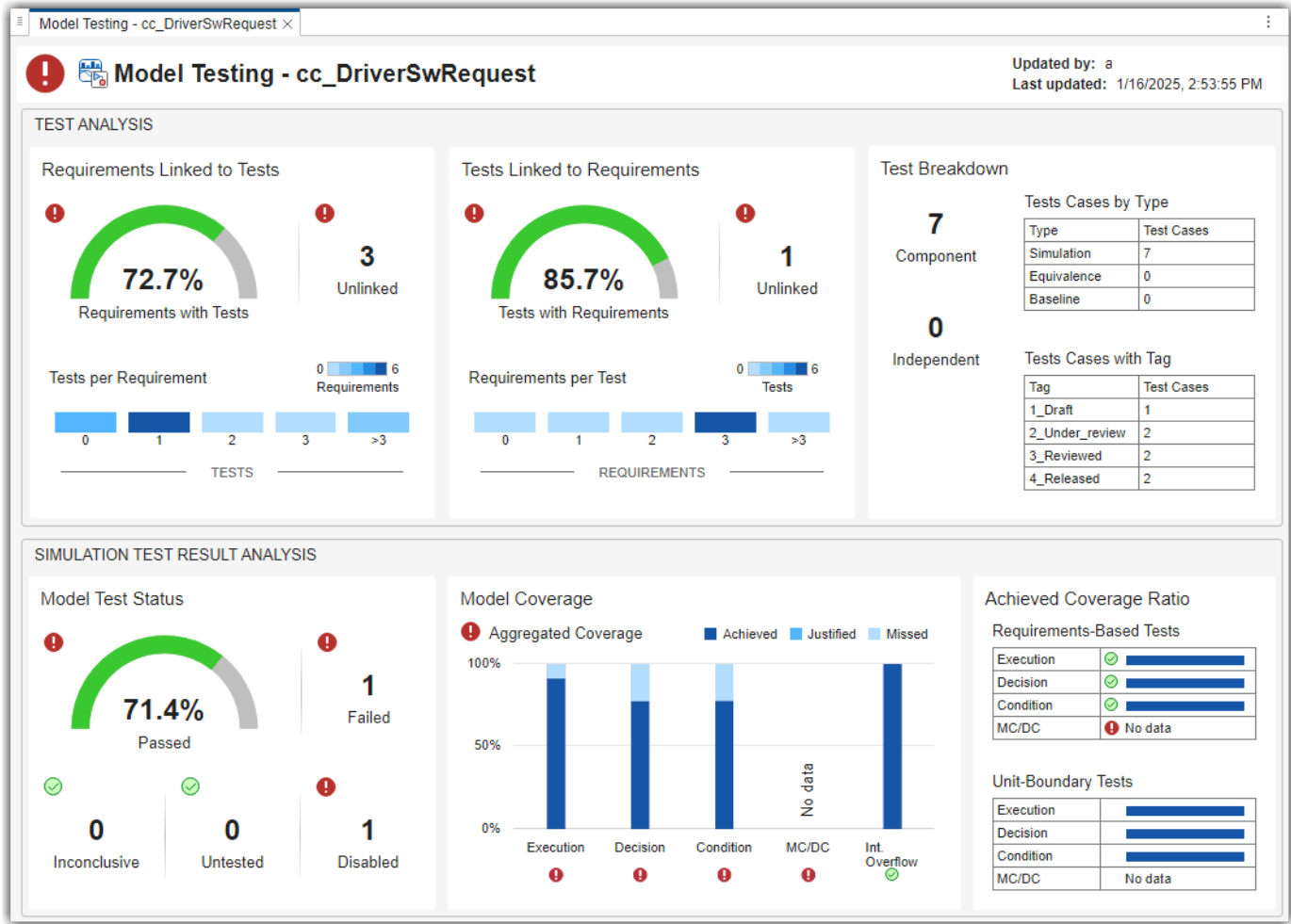
`sltest.getAssessments` | Assertion | Implies (Simulink Design Verifier) | Proof Assumption

More About

- “View Simulation Data in Simulation Data Inspector”

Fix Requirements-Based Testing Issues

This example shows how to address common traceability issues in model requirements and tests by using the Model Testing Dashboard. The dashboard analyzes the testing artifacts in a project and reports metric data on quality and completeness measurements such as traceability and coverage, which reflect guidelines in industry-recognized software development standards, such as ISO 26262 and DO-178C. The dashboard widgets summarize the data so that you can track your requirements-based testing progress and fix the gaps that the dashboard highlights. You can click the widgets to open tables with detailed information, where you can find and fix the testing artifacts that do not meet the corresponding standards.



Collect Metrics for Testing Artifacts in Project

The dashboard displays testing data for a model and the artifacts that the unit traces to within a project. For this example, open the project and collect metric data for the artifacts.

1. Open the project that contains models and testing artifacts. For this example, in the MATLAB® Command Window, enter:

```
openProject("cc_CruiseControl");
```

2. Open the Dashboard window. To open the Model Testing Dashboard: on the **Project** tab, click **Model Testing Dashboard**. Or, in the command line, enter:

```
modelTestingDashboard
```

3. The **Project** panel displays artifacts from the current project that are compatible with the currently selected dashboard. View the metric results for the unit `cc_DriverSwRequest`. In the **Project** panel, click the name of the unit, `cc_DriverSwRequest`. When you initially select `cc_DriverSwRequest`, the dashboard collects the metric results for uncollected metrics and populates the widgets with the data for the unit.

Link Requirement to Implementation in Model

The **Artifacts** panel shows artifacts such as requirements, tests, and test results that trace to the unit selected in the **Project** panel.

In the **Artifacts** panel, the **Trace Issues** folder shows artifacts that do not trace to unit models in the project. The **Trace Issues** folder contains subfolders for:

- **Unexpected Implementation Links** — Requirement links of **Type Implements** for a requirement of **Type Container** or **Type Informational**. The dashboard does not expect these links to be of **Type Implements** because container requirements and informational requirements do not contribute to the Implementation and Verification status of the requirement set that they are in. If a requirement is not meant to be implemented, you can change the link type. For example, you can change a requirement of **Type Informational** to have a link of **Type Related to**.
- **Unresolved and Unsupported Links** — Requirement links which are broken or not supported by the dashboard. For example, if a model block implements a requirement, but you delete the model block, the requirement link is now unresolved. The Model Testing Dashboard does not support traceability analysis for some artifacts and some links. If you expect a link to trace to a unit and it does not, see the troubleshooting solutions in “Resolve Missing Artifacts, Links, and Results” (Simulink Check).
- **Untraced Tests** — Tests that execute on models or subsystems that are not on the project path.
- **Untraced Results** — Results that the dashboard can no longer trace to a test. For example, if a test produces results, but you delete the test, the results can no longer be traced to the test.

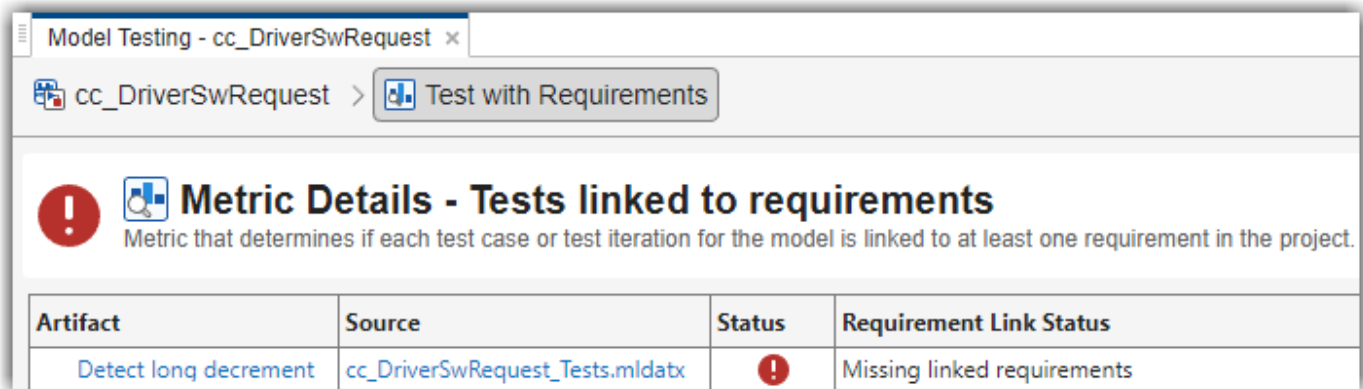
Address Testing Traceability Issues

The widgets in the **Test Analysis** section of the Model Testing Dashboard show data about the unit requirements, tests for the unit, and links between them. The widgets indicate if there are gaps in testing and traceability for the implemented requirements.

Link Requirements and Tests

For the unit `cc_DriverSwRequest`, the **Tests Linked to Requirements** section shows that some of the tests are missing links to requirements in the model.

To see detailed information about the missing links, in the **Tests Linked to Requirements** section, click the widget **Unlinked**. The dashboard opens the **Metric Details** for the widget with a table of metric values and hyperlinks to each related artifact. The table shows the tests that are implemented in the unit, but do not have links to requirements. The table is filtered to show only tests that are missing links to requirements.



Metric Details - Tests linked to requirements
Metric that determines if each test case or test iteration for the model is linked to at least one requirement in the project.

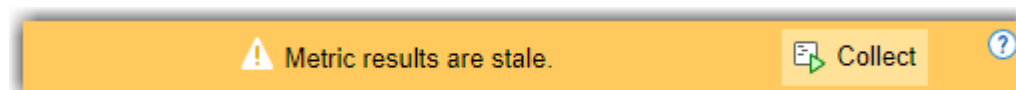
Artifact	Source	Status	Requirement Link Status
Detect long decrement	cc_DriverSwRequest_Tests.mldatx	!	Missing linked requirements

The test Detect long decrement is missing linked requirements.

- 1 In the **Artifact** column of the table, point to **Detect long decrement**. The tooltip shows that the test **Detect long decrement** is in the test suite **Unit test for DriverSwRequest**, in the test file **cc_DriverSwRequest_Tests**.
- 2 Click **Detect long decrement** to open the test in the Test Manager. For this example, the test needs to link to three requirements that already exist in the project. If there were not already requirements, you could add a requirement by using the Requirements Editor.
- 3 Open the software requirements in the Requirements Editor. In the **Artifacts** panel of the Dashboard window, expand the folder **Functional Requirements > Implemented** and double-click the requirement file **cc_SoftwareReqs.slreqx**.
- 4 View the software requirements in the container with the summary **Driver Switch Request Handling**. Expand **cc_SoftwareReqs > Driver Switch Request Handling**.
- 5 Select multiple software requirements. Hold down the **Ctrl** key as you click **Output request mode**, **Avoid repeating commands**, and **Long Increment/Decrement Switch recognition**. Keep these requirements selected in the Requirements Editor.
- 6 In the Test Manager, expand the **Requirements** section for the test Detect long decrement. Click the arrow next to the **Add** button and select **Link to Selected Requirement**. The traceability link indicates that the test Detect long decrement verifies the three requirements Output request mode, Avoid repeating commands, and Long Increment/Decrement Switch recognition.
- 7 The metric results in the dashboard reflect only the saved artifact files. To save the test suite **cc_DriverSwRequest_Tests.mldatx**, in the **Test Browser**, right-click **cc_DriverSwRequest_Tests** and click **Save**.

Refresh Metric Results in Dashboard

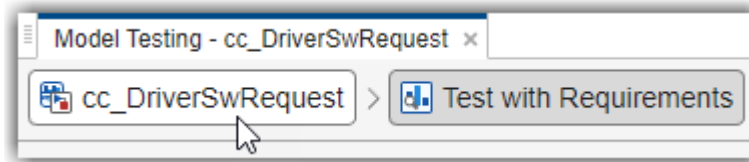
The dashboard detects that the metric results are now stale and shows a warning banner at the top of the dashboard.



! Metric results are stale. Collect ?

- 1 Click the **Collect** button on the warning banner to re-collect the metric data so that the **Metric Details** reflect the traceability link between the test and requirements.

- View the updated dashboard widgets by returning to the **Model Testing** results and collecting the metrics. At the top of the dashboard, there is a breadcrumb trail from the **Metric Details** back to the **Model Testing** results. Click the breadcrumb button for **cc_DriverSwRequest** to return to the **Model Testing** results for the unit. Then, update the impacted model testing metric results by clicking **Collect** in the warning banner.



The **Tests Linked to Requirements** section shows that there are no unlinked tests. The **Requirements Linked to Tests** section shows that there are 3 unlinked requirements. Typically, before running the tests, you investigate and address these testing traceability issues by adding tests and linking them to the requirements. For this example, leave the unlinked artifacts and continue to the next step of running the tests.

Test Model and Analyze Failures and Gaps

After you create and link unit tests that verify the requirements, run the tests to check that the functionality of the model meets the requirements. To see a summary of the test results and coverage measurements, use the widgets in the **Simulation Test Result Analysis** section of the dashboard. The widgets help show testing failures and gaps. Use the metric results to analyze the underlying artifacts and to address the issues.

Perform Unit Testing

Run the tests for the model by using the Test Manager. Save the test results in your project and review them in the Model Testing Dashboard.

- Open the unit tests for the model in the Test Manager. In the Model Testing Dashboard, in the **Artifacts** panel, expand the folder **Tests > Unit Tests** and double-click the test file **cc_DriverSwRequest_Tests.mldatx**.
- In the Test Manager, click **Run**.
- Select the results in the **Results and Artifacts** pane.
- Save the test results as a file in the project. On the **Tests** tab, in the **Results** section, click **Export**. Name the results file **Results1.mldatx** and save the file under the project root folder.

SUMMARY	
Name	Results: 2023-May-02 11:23:25
Outcome	5 ✔ 1 ✖ 1 ⚙
Start Time	05/02/2023 11:23:33
End Time	05/02/2023 11:24:25
Type	Result Set

The Model Testing Dashboard detects the results and automatically updates the **Artifacts** panel to include the new test results for the unit in the subfolder **Test Results > Model**.

The dashboard also detects that the metric results are now stale and shows a warning banner at the top of the dashboard.

The **Stale** icon  appears on the widgets in the **Simulation Test Result Analysis** section to indicate that they are showing stale data that does not include the changes.

Click the **Collect** button on the warning banner to re-collect the metric data and to update the stale widgets with data from the current artifacts.

Address Testing Failures and Gaps

For the unit `cc_DriverSwRequest`, the **Model Test Status** section of the dashboard indicates that one test failed and one test was disabled during the latest test run.

- 1 To view the disabled test, in the dashboard, click the **Disabled** widget. The table shows the disabled tests for the model.
- 2 Open the disabled test in the Test Manager. In the table, click the test artifact **Detect long decrement**.
- 3 Enable the test. In the **Test Browser**, right-click the test and click **Enabled**.
- 4 Re-run the test. In the **Test Browser**, right-click the test and click **Run** and save the test suite file.
- 5 View the updated number of disabled tests. In the dashboard, click the **Collect** button on the warning banner. Note that there are now zero disabled tests reported in the **Model Test Status** section of the dashboard.
- 6 View the failed test in the dashboard. Click the breadcrumb button for `cc_DriverSwRequest` to return to the **Model Testing** results and click the **Failed** widget.
- 7 Open the failed test in the Test Manager. In the table, click the test artifact **Detect set**.
- 8 Examine the test failure in the Test Manager. You can determine if you need to update the test or the model by using the test results and links to the model. For this example, instead of fixing the failure, use the breadcrumbs in the dashboard to return to the **Model Testing** results and continue on to examine test coverage.

Check if the tests that you ran fully exercised the model design by using the coverage metrics. For this example, the **Model Coverage** section of the dashboard indicates that some conditions in the model were not covered. Place your cursor over the **Decision** bar in the widget to see what percent of condition coverage was achieved.

- 1 View details about the decision coverage by clicking one of the **Decision** bars. For this example, click the **Decision** bar for **Achieved** coverage.
- 2 In the table, expand the model artifact. The table shows the test results for the model and the results files that contains them. For this example, click on the hyperlink to the source file **Results1.mldatx** to open the results file in the Test Manager.
- 3 To see detailed coverage results, use the Test Manager to open the model in the Coverage perspective. In the Test Manager, in the **Aggregated Coverage Results** section, in the **Analyzed Model** column, click `cc_DriverSwRequest`.
- 4 Coverage highlighting on the model shows the points that were not covered by the tests. For this example, do not fix the missing coverage. For a point that is not covered in your project, you can

add a test to cover it. You can find the requirement that is implemented by the model element or, if there is none, add a requirement for it. Then you can link the new test to the requirement. If the point should not be covered, you can justify the missing coverage by using a filter.

Once you have updated the unit tests to address failures and gaps in your project, run the tests and save the results. Then examine the results by collecting the metrics in the dashboard.

Iterative Requirements-Based Testing with Model Testing Dashboard

In a project with many artifacts and traceability connections, you can monitor the status of the design and testing artifacts whenever there is a change to a file in the project. After you change an artifact, use the dashboard to check if there are downstream testing impacts by updating the tracing data and metric results. Use the **Metric Details** tables to find and fix the affected artifacts. Track your progress by updating the dashboard widgets until they show that the model testing quality meets the standards for the project.

See Also

`modelTestingDashboard` (Simulink Check)

Assess Temporal Logic by Using Temporal Assessments

Hybrid systems with discrete and continuous time behavior can require complex timing-dependent signal logic. Simulink Test enables you to assess model timing and event ordering by authoring and including temporal assessments with test cases in the Test Manager.

To work with temporal assessments in the Test Manager:

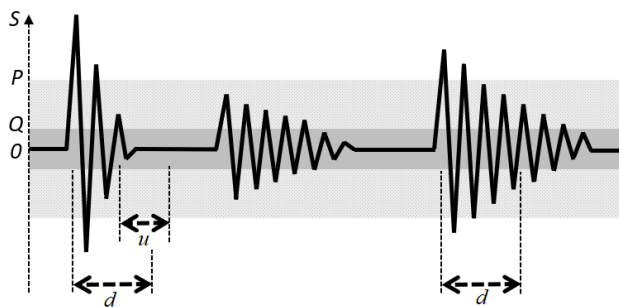
- 1 Select an assessment template.
- 2 Enter the assessment conditions.
 - Map symbols to model elements, such as signals, time series, or constants.
 - View the assessment summary.

You can copy, paste, and delete assessments by selecting the summary and right-clicking to display the context menu or by using keyboard shortcuts. When you copy and paste an assessment, it is added to the end of the assessments. If you delete an assessment, you cannot paste it even if you copied it before deleting it.

- 3 Run the test case.
- 4 Use the results to assess the system under test (SUT) against your requirements.

For example, consider a forced oscillation damping problem that has this requirement:

For a signal S , if the signal magnitude exceeds value P , then within d seconds, it must settle below value Q and stay below Q for u seconds.

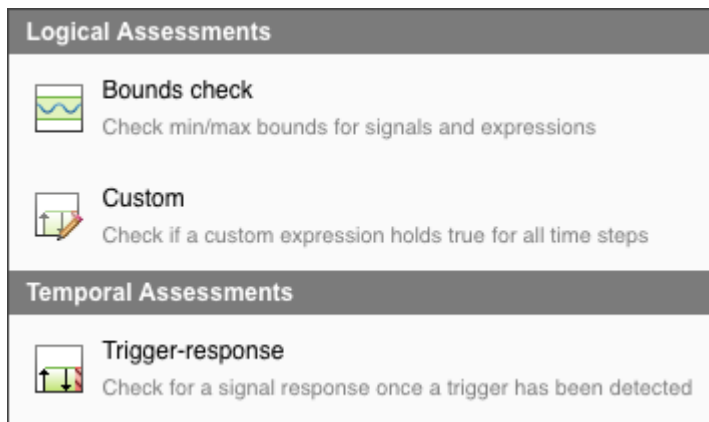


Create a Temporal Assessment

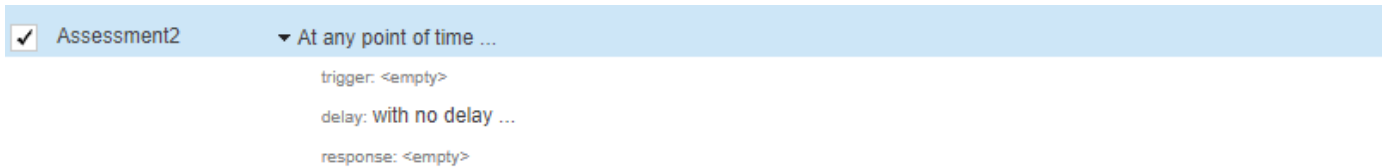
To create a temporal assessment:

- 1 Create or open a test case in the Test Manager.
- 2 Navigate to the **Logical and Temporal Assessments** Editor.
- 3 Click **Add Assessment**. These assessment templates are available:
 - **Logical Assessment Templates**
 - **Bounds Check** — Check maximum and minimum bounds for signals and expressions.
 - **Custom** — Check if a logical expression holds true for all time steps.
 - **Temporal Assessment Template**

- **Trigger-Response** — Check for a signal response when a trigger is detected.



For this example, select **Trigger-Response**.



The Trigger-Response template appears. To finish creating the assessment, you define temporal assessment conditions in the context of the SUT.

Define Temporal Assessment Conditions

A Trigger-response assessment requires a:

- Trigger parameter
- Response parameter
- Optional Delay parameter

Note Using `==` or `~=` operators in logical or temporal assessment statements might cause a test to issue a warning or fail when comparing floating-point data. If you use floating-point data, consider including a tolerance for the assessment. For example, instead of using `if(x == 5)` in a statement, check that `x` is within a tolerance of 0.001 by using `if(abs(x-5) < 0.001)`.

For the forced oscillation damping problem:

- 1 Select *whenever is true* as the trigger and enter `abs(S) > P` as the condition. The trigger condition is the condition pattern after which the response signal is evaluated. The response condition is triggered when the magnitude of signal `S` exceeds value `P`.
- 2 Select *must stay true for at least* as the response and enter `abs(S) < Q` and `u` as the condition and min-time respectively. The response condition describes the behavior of the SUT in response to the trigger condition. The response condition is that the magnitude of signal `S` must settle below value `Q` and stay below `Q` for at least `u` seconds.

- 3 Select *with a delay of at most* as the delay type and set *d* as the *max-time* parameter. The delay is an optional time interval that starts from a time reference parameter and continues to the point where the response condition is expected to be satisfied. The delay is at most *d* seconds.

All time units are seconds.

When you add a symbol as part of a temporal assessment parameter in the **Logical and Temporal Assessments** Editor, it is added to the list of symbols as an unresolved symbol. Resolve symbols by using the **Symbols** pane in the editor.

Resolve Assessment Parameter Symbols

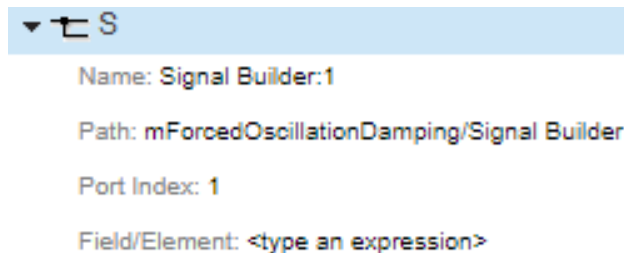
To resolve a symbol, right-click the symbol. Two options are available:

- 1 **Map to model element** - Use the mapping dialog box to map symbols to a signal, parameter, or block in the SUT.



Select a symbol to map from the drop-down list at the top of the mapping dialog box.

After you finish mapping symbols to model elements, the **Symbols** pane displays metadata that corresponds to the model element.



Signals that are mapped to a symbol used by an assessment in the editor are logged when you run the test case.

If you map a bus or an array to a symbol, use the **Field/Element** row in the **Symbols** pane to select a scalar signal from the bus or array. For example:

- To map a symbol to a bus signal containing a bus element *fieldA*, enter *.fieldA*.
- To map a symbol to the signal element that corresponds to index (5,5) in a signal array, enter (5,5).
- To combine both expressions, enter *.fieldA(5,5)*.

- 2 **Map to expression** - Assign a scalar constant value or variable to a symbol.

When you select **Map to expression**, you must enter an expression in the **Expression** field. The expression must be in MATLAB code and evaluate to a scalar literal or a *timeseries* object. If the expression is long or complicated, you can use the **Assessment Callback** section to write MATLAB code that retrieves the model, test, and simulation data and assigns the data to variables, then assign the variable to the symbol expression. See “Define Variables in the Assessment Callback Section” on page 3-102 for more information on defining variables.

To retrieve data stored in workspace variables, use the `evalin` function. For example, to assign the workspace variable `var` to a symbol, enter `evalin("base","var")` in the symbol expression directly, or enter `v = evalin("base","var")` in the **Assessment Callback** section and enter `v` in the symbol expression.

Because the `t` symbol is automatically bound to the simulation time, you do not need to map it to an expression. `t` is not visible in the **Symbols** pane.

Tip Entering `sig = sltest_simout.logout.get('mySignal')` in the **Assessment Callback** section and using **Map to expression** to map a symbol to the `sig` variable is equivalent to using **Map to model element** to map a symbol to the `mySignal` signal.

If you map a symbol to a discrete data signal that is linearly interpolated, the interpolation is automatically changed to zero-order hold during the assessment evaluation. Additionally, an information icon (i) appears next to the symbol name in the **Symbols** pane. Point to the icon and a tooltip appears which indicates that the linear interpolation was overridden.

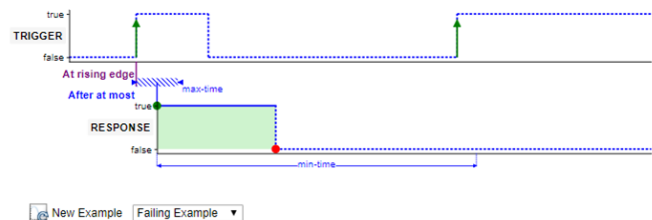
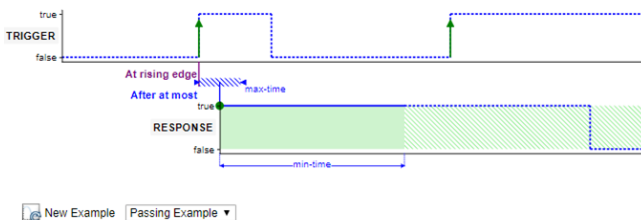
Review the Temporal Assessment Summary

After you enter the assessment parameters, click the arrow to the left of the assessment description to view the assessment summary.

► At any point of time, whenever $\text{abs}(S) > P$ is true then, with a delay of at most t seconds, $\text{abs}(S) < Q$ must stay true for at least u seconds

The **Visual Representation** pane provides a graphical illustration of a passing case for the assessment.

View passing and failing cases for the assessment by clicking the Explore Pattern  icon. Select the type of case you want to view from the drop-down list and click  to view different passing and failing cases.



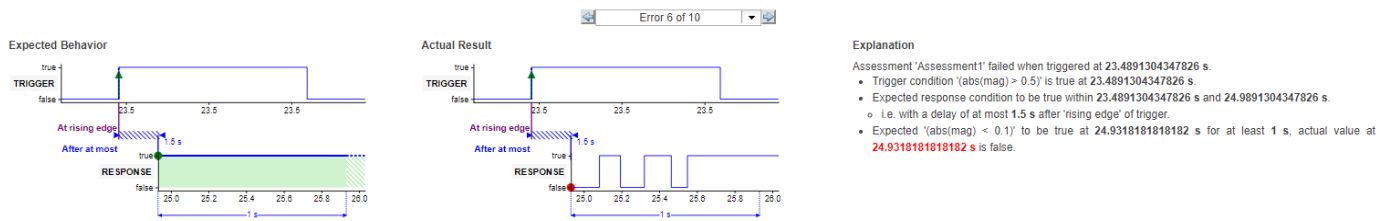
Evaluate the SUT

Run the test case to evaluate the SUT. Temporal assessments are evaluated after simulation by using logged signal data. Use the test case results to review the SUT against your requirements.

You can run test cases that contain logical or temporal assessments in multiple releases. For more information, see “Assess Temporal Logic in Multiple Releases” on page 6-106.

View Assessment Results

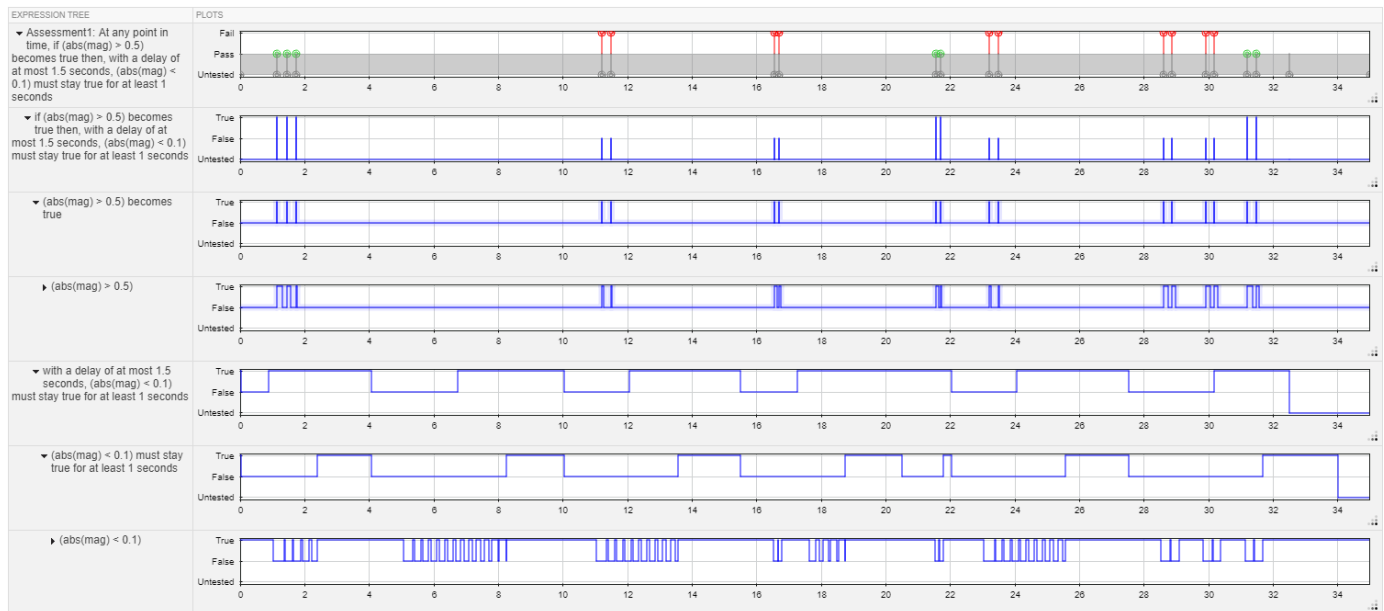
View the results of the assessment evaluation from the **Results and Artifacts** pane of the Test Manager. Select the test case and click the assessment in the **Results** tree to open a new **Assessment Result** tab. Simulink Test evaluates the assessment and displays the expected behavior and the actual result of the assessment execution with a description of the assessment failures at different time steps.



Note The assessment result figures cannot be exported to a Simulink Test report.

Investigate the SUT behavior using the and buttons and the textual descriptions at points of failure.

For a more detailed investigation, expand the Expression Tree to view results for every individual element of the assessment.



Use the zoom, pan, and data cursor functionalities to analyze assessment evaluation results in the Expression Tree.

Link Temporal Assessments to Requirements

If you have a Requirements Toolbox license, you can establish traceability between temporal assessments and requirements by linking assessments to requirements. To create links to requirements, select the assessment in the **Logical and Temporal Assessments** Editor and click the **Requirements** column to open the **Requirement Editor** dialog box. See “Link to Requirements” on page 1-2 for more information.

See Also

`sltest.testmanager.Assessment` | `sltest.testmanager.AssessmentSymbol` |
`sltest.testmanager.TestCase`

More About

- “Logical and Temporal Assessment Syntax” on page 3-99
- “Assess Temporal Logic in Multiple Releases” on page 6-106

Test Traffic Light Control by Using Logical and Temporal Assessments

This example shows how to use logical and temporal assessments to test the signal logic for a two-light traffic intersection. It also shows how to minimize untested results.

The model used in this example represents a controller in a two-light traffic intersection. The changes between the traffic light states depend on the traffic lights and the timing delay parameters defined in a Stateflow® chart. For more information about the Stateflow logic used in the model, see “Model an Intersection of One-Way Streets” (Stateflow).

Open and Run the Model

Open and simulate the model.

```
model = 'sltestTrafficLight';
open_system(model)
sim(model)
```

Plot the states of the two lights in the Simulation Data Inspector.

```
runData = Simulink.sdi.Run.getLatest;
LightState1 = getSignalsByName(runData, 'Light1');
LightState2 = getSignalsByName(runData, 'Light2');
Simulink.sdi.setSubPlotLayout(2,1);
plotOnSubPlot(LightState1,1,1,true);
plotOnSubPlot(LightState2,1,1,false);

plotOnSubPlot(LightState2,2,1,true);
plotOnSubPlot(LightState1,2,1,false);

Simulink.sdi.view
```

The traffic lights are approximately on opposite schedules. When one light is in the Green state, the other light is in the Red state, and vice versa. Additionally, the lights must pass through the Yellow state when transitioning from Green to Red.

To explore the Stateflow logic, consider:

- Adding a breakpoint in one of the two atomic subcharts to step through the logic. For more information on debugging state charts, see “Set Breakpoints to Debug Charts” (Stateflow).
- Inspecting the logic using the Sequence Viewer.
- Visualizing different signals using the Simulation Data Inspector.

Requirements and Enumeration Types of the Model

To test the logic of the traffic controller, the Stateflow chart outputs an enumerated type that corresponds to the Red, Yellow, or Green state of the light. By default, Stateflow automatically generates the enumerated type definition. To create a custom enumeration definition, see “Define State Activity Enumeration Type” (Stateflow).

Simulating the model creates the built-in enumeration definition. To confirm that the enumerated type definition `LightModeType` exists, use `which LightModeType`.

Design Requirements

The test file in this example tests the traffic light model against several requirements::

- The number of cars waiting at a light is always greater than or equal to zero.
- At any point in time, at least one of the lights is red.
- Every time the light becomes yellow, it stays yellow for a fixed amount of time within a tolerance value of 0.5 seconds before changing to red.
- Every time the light becomes green, it stays green between a minimum and maximum time within a tolerance of 0.5 seconds before changing to yellow.

The test file tests the requirements by using a bounds check assessment, a custom assessment, and two trigger-response assessments. For more information about linking to requirements, see “Link Temporal Assessments to Requirements” on page 3-90.

Run the Logical and Temporal Assessments

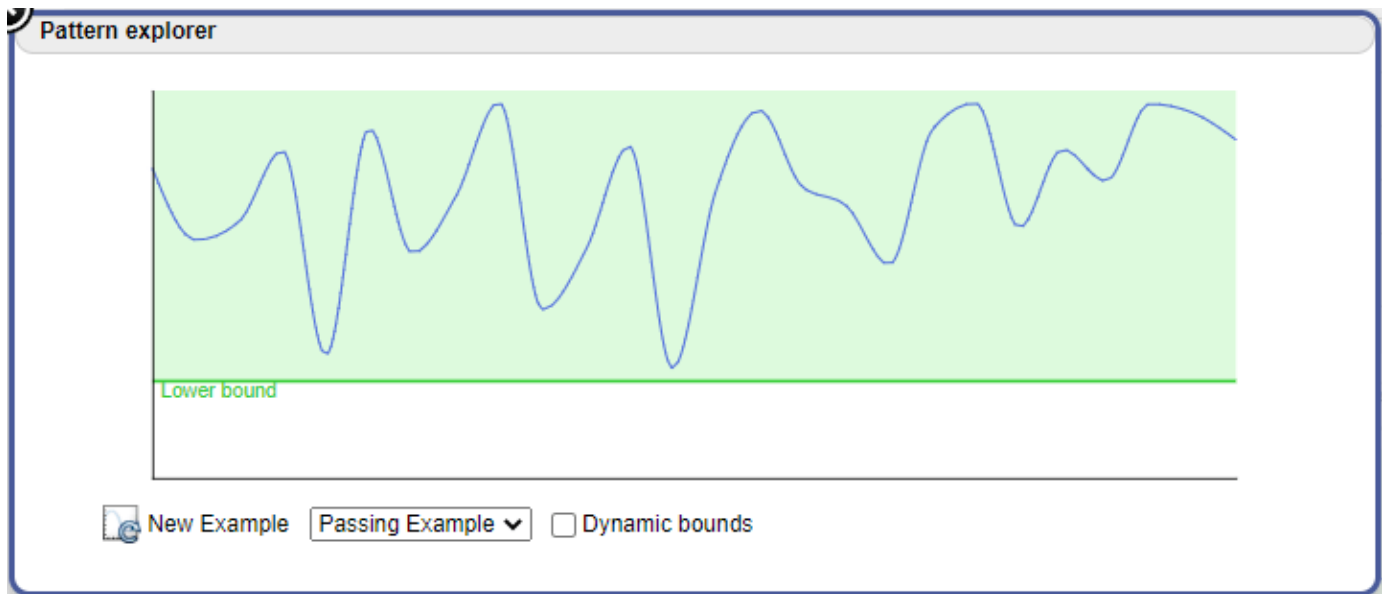
Load the test file and open the test assessments in the Test Manager.

```
sltest.testmanager.load('test_traffic.mldatx');  
sltest.testmanager.view;
```

Run the Bounds Check Test

Check whether the number of cars waiting at the light is always greater than or equal to zero by using a bounds check logical assessment. The symbol `NumCars` is mapped to the output of the `Car_Monitor1` subsystem. The `Car_Monitor1` subsystem outputs an `int32` type, so the lower-bound expression is cast as `int32(0)`. For more information about data type requirements, see “Data Types in Assessment Conditions” on page 3-102.

- 1 In the Test Manager, click **New Test Case 1** in the **Test Browser** pane.
- 2 Expand the **Logical and Temporal Assessments** section of the Test Manager
- 3 Select **Waiting Cars** in the table and verify that its **Assessment** logic is correct.
- 4 In the upper right of the **Visual Representation** pane, click the Explore Pattern icon to open the Pattern Explorer.
- 5 View the Passing and Failing Examples. This image shows a passing example:



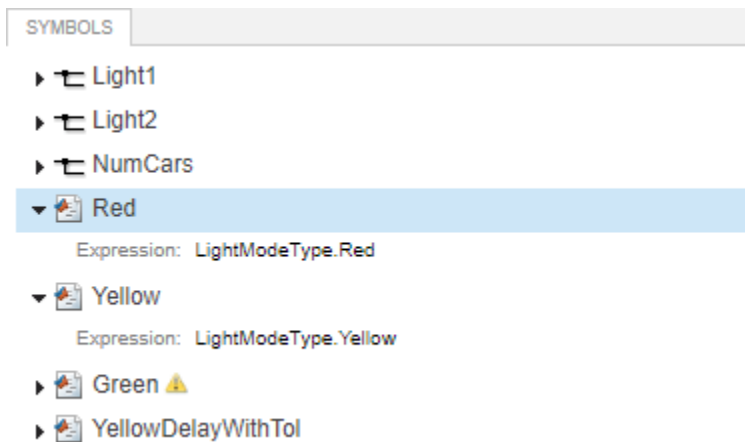
6. With the **Waiting cars** assessment selected, run the test case.
7. To view the results, expand the Results in the **Results and Artifacts** panel.
8. Select **New Test Case 1 > Logical and Temporal Assessments > Waiting cars**.

Run the Custom Logical Assessment

Compare the states of both lights at all time steps by using a custom logical assessment.

For safety reasons, at no point in time should both lights be green. In addition, other configurations are undesirable, such as one green light and one yellow light or two yellow lights. The assessment checks that one of the lights is always red.

In the **Logical and Temporal Assessments** section, this assessment uses the symbols, Red and Yellow, which each correspond to their respective color in the enumerated type definition. For instance, the Expression field for the Red symbol references the Red enumeration member of the LightModeType enumeration - LightModeType.Red. The Green symbol appears as an unused symbol because it is not used until you implement the Green to Yellow assessment. See Create a Trigger-Response Assessment to Evaluate Green to Yellow Transitions on page 3-96.



Note that the **Visual Representation** preview is blank because it is only available for bounds check and trigger-response assessments. Also, the custom-expression field of the custom check must follow the syntax rules described in “Logical and Temporal Assessment Conditions” on page 3-101.

- 1 Return to the **Test Browser** pane and expand the **Logical and Temporal Assessments** section.
- 2 Select **Both lights safety check**.
- 3 Rerun the test case.
- 4 To view the results, expand the Results in the **Results and Artifacts** pane and select **New Test Case 1 > Logical Temporal Assessments > Both lights safety check**.

Run the Trigger Response Logical Assessment

Use a trigger response logical assessment to assess the logic when **Light1** transitions from the **Yellow** state to the **Red** state. The assessment triggers when **Light1** enters the **Yellow** state. As shown in the Stateflow chart, for the **after(YELLOWDELAY,sec)** transitions in each atomic subchart, the state switches from **Yellow** to **Red** after a fixed delay of **YELLOWDELAY** seconds. To meet the requirements, the **YELLOWDELAY** value is adjusted by a tolerance value, **tol** for the assessment in the assessment callback.

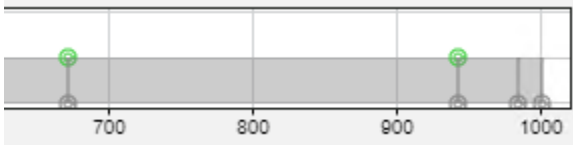


- 1 Under **Logical and Temporal Assessments**, enable **Light1 YellowToRed Transition** assessment.
- 2 Verify its **Assessment** logic summary.
- 3 Expand the summary and the trigger section and set **time-reference** to rising edge of trigger.
- 4 Run the test case.
- 5 In the **Results and Artifacts** pane, click **New Test Case 1 > Logical and Temporal Assessments > Light1 YellowToRed Transition**.
- 6 Observe that the assessment fails.

The Expected Behavior and Actual Result graphs show the assessment failures and the **Explanation** section describes the failures. The failures occur at the four points when the assessment triggers. In **Error 1 of 4**, the trigger condition becomes true at $t = 132.1$ and the **Explanation** section explains that the test expected the response condition to be true at 132.1 seconds. This result contradicts the requirement that the light stay yellow for a fixed amount of time before changing to red. The assessment fails because the evaluation of the response is at the rising edge of the trigger. The `Light1 == Red` response should not be evaluated until the trigger is false.

In the **Test Browser** pane, adjust the **time-reference** to falling edge of trigger. This setting ensures that the `Light1 == Red` response evaluates only when `Light1` is no longer yellow.

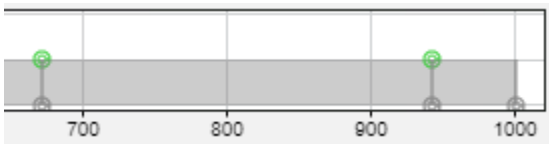
Rerun the test case. Now the assessment passes at the four points of the simulation where `Light1` turns yellow. Additionally, there is a point marked at $t = 984.5$, which corresponds to the point after which the assessment can no longer be evaluated. The logic specifies that the trigger condition must stay true for most `YellowDelayWithTol` seconds, or 15.5 seconds. After $t = 984.5$, there is not enough time left in the simulation to accurately assess the logic.



You might notice similar behavior for other assessments based on the timing parameters supplied to the trigger.

You can reevaluate the assessments to minimize untested results.

- 1 Click **New Test Case 1** in the current results.
- 2 At the bottom of the **Property-Value** pane in the lower left, enable **Extended Assessment Result**. The assessments are reevaluated and the results are updated.
- 3 The plot now shows that there are no untested results.



Create a Trigger-Response Assessment to Evaluate Green to Yellow Transitions

This assessment evaluates the `Light1` transitions from Green to Yellow during the first half of simulation. The assessment triggers when the light changes from Red to Green. The transition from Green to Yellow occurs within a fixed window of time based on the `greenLightRequested` parameter. To set up the parameters and use the built-in symbol `t` to restrict the assessment to the first half of the simulation, create an assessment callback.

Set Up the Parameters

- 1 In the Test Manager, expand the **Logical and Temporal Assessments** section.
- 2 Click **Add Assessment** and choose **Trigger-response**.
- 3 Double click the **Name** field. Rename the assessment `Light1 GreenToYellow`.
- 4 Set the trigger condition. Click the drop-down next to the **trigger** field and select **becomes true and stays true for between**. This logic is required because the light must stay green before switching to yellow. After you select the trigger type, the **condition**, **min-time (sec)**, **max-time (sec)**, and **time-reference** fields become visible.
- 5 For **condition**, enter `t < 500 & Light1 == Green`. The assessment uses the built-in symbol `t` to trigger a check when `Light1` becomes Green within the first 500 seconds of the simulation.
- 6 For **min-time (sec)**, enter `GreenMin`.
- 7 For **max-time (sec)**, enter `GreenMax`. Note that `GreenMin` and `GreenMax` are not yet defined and appear as Unresolved symbols in the **Symbols** pane.
- 8 Set the **time-reference** to **falling edge of trigger**, which ensures that the response when `Light1` becomes yellow is evaluated only when `Light1` is no longer green.

- 9 Leave the **delay** as with no delay.
- 10 Set the **response** to must be true. This option evaluates a single instance of time and captures whether the transition to Yellow occurs. After selecting the response type, the **condition** field becomes visible.
- 11 For **condition**, enter `Light1 == Yellow`.
- 12 Resolve the `GreenMin` and `GreenMax` symbols, by adding this assessment callback code to the existing code in the **Assessment Callback**. The callback extracts the Stateflow parameters that correspond to the Green transition, then adjusts them by the tolerance value specified in the requirements to prepare them for the min-time and max-time trigger fields.

```
greenMin = maskObj.getParameter('MINGREENDELAY');
greenMin = str2double(greenMin.Value);
greenMinAdj = greenMin - tol;
```

```
greenMax = maskObj.getParameter('GREENDELAY');
greenMax = str2double(greenMax.Value);
greenMaxAdj = greenMax + tol;
```

13. In **Symbols**, right-click the symbol name `GreenMin` and choose **Map to expression**.

14. In the **Expression** field, enter the variable name `greenMinAdj`.

15. Resolve the symbol `GreenMax` by repeating steps 13 and 14 by using `greenMaxAdj` for the expression.

View the Light1 GreenToYellow Logic

Collapse the assessment to read a summary of its logic:

The screenshot shows the Stateflow assessment interface. On the left, the assessment 'Light1 GreenToYellow' is selected, showing its logic: 'At any point of time, if `t<500 & Light1 == Green` becomes true and stays true for between `GreenMin` seconds and `GreenMax` seconds then, starting from falling edge of trigger, with no delay, `Light1 == Yellow` must be true'. The response is set to 'None'. On the right, the 'SYMBOLS' pane lists the symbols used in the assessment: `Light1`, `Light2`, `NumCars`, `Red`, `Yellow`, `Green`, `YellowDelayWithTol`, `GreenMin`, and `GreenMax`.

You can use the **Visual Representation** to preview the logic of the assessment. After `Light1` turns green, it must stay green for a period of time within the minimum and maximum times. When the `Light1` trigger is false and the light is no longer green, the `Light1` trigger is true and the light changes to yellow.

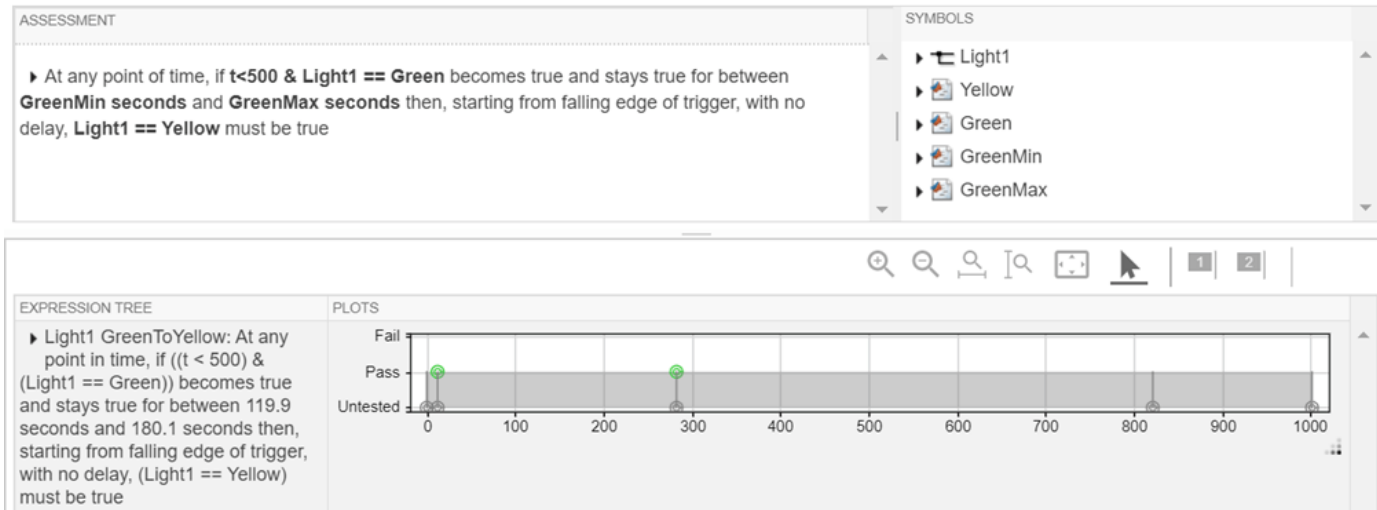
Run the Assessment and View the Results

Run the assessment.

In the **Results and Artifacts** pane, expand **New Test Case 1 > Logical and Temporal Assessments**. Select **Light1 GreenToYellow** and observe that the assessment triggers twice in the

first half of the simulation. The trigger times align with the initial Simulation Data Inspector results on page 3-91 when **Light1** enters the Green state at $t = 12.1$ and $t = 282.2$.

✓ **Light1 GreenToYellow**



See Also

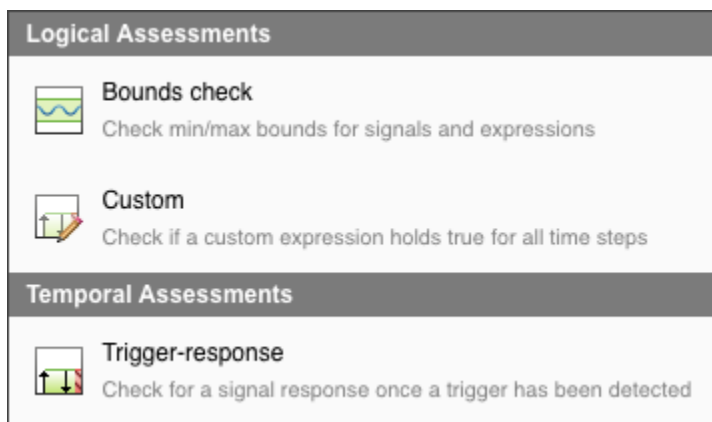
Related Examples

- “Logical and Temporal Assessment Syntax” on page 3-99
- “Define Test Steps and Assessments” on page 3-63
- “Reference Values by Name by Using Enumerated Data” (Stateflow)

Logical and Temporal Assessment Syntax

Simulink Test provides three logical and temporal assessment templates:

- **Logical Assessment Templates**
 - **Bounds Check** — Check maximum and minimum bounds for signals and expressions.
 - **Custom** — Check if a logical expression holds true for all time steps.
- **Temporal Assessment Template**
 - **Trigger-Response** — Check for a signal response when a trigger is detected.



Bounds Check Assessments

Create bounds check assessments to check if the signals and expressions you test satisfy the boundary condition patterns you specify for them. Boundary condition pattern templates let you test if signals and expressions in terms of boundary values that you specify are:

- **Always less than** (or equal to)
- **Always greater than** (or equal to)
- **Always inside**
- **Always outside**

Trigger-Response Assessments

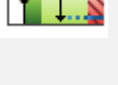
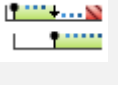
Create trigger-response assessments to verify a signal response when a trigger is detected. A trigger-response assessment requires:

- Trigger parameter
- Response parameter
- Optional Delay parameter

The trigger condition is the condition pattern based on which the response signal is evaluated. There are five trigger condition patterns available:

Trigger Condition Pattern		Behavior	Available Time References
	Whenever is true	Check the response signal continuously whenever the triggering condition is true.	N/A
	Becomes true	Check the response signal every time the triggering condition becomes true.	Rising edge
	Becomes true and stays true for at least	Check the response signal every time the triggering condition becomes true and stays true for at least the interval specified by the min-time parameter (in s). You also specify an additional time reference parameter at which to evaluate the response signal.	Rising edge of trigger or end of min-time
	Becomes true and stays true for at most	Check the response signal every time the triggering condition becomes true and stays true for at most the interval specified by the max-time parameter (in s). You also specify an additional time reference parameter at which to evaluate the response signal.	Rising or falling edge of trigger or end of max-time
	Becomes true and stays true for between	Check the response signal every time the triggering condition becomes true and stays true between the interval specified by the min-time and max-time parameters. You also specify an additional time reference parameter at which to evaluate the response signal.	Rising or falling edge of the trigger or end of min-time or max-time

To complete authoring a trigger-response assessment, you specify the response condition pattern and the response condition. There are five response condition patterns available:

Response Condition Pattern		Behavior
	Must be true	The response condition pattern must be true starting from the time reference parameter to the delay (if it is defined).
	Must stay true for at least	The response condition pattern must stay true for at least the duration specified by the min-time parameter.
	Must stay true for at most	The response condition pattern must stay true for at most the duration specified by the max-time parameter.
	Must stay true for between	The response condition pattern must stay true for at least the duration specified by the min-time parameter and at most the duration specified by the max-time parameter.
	Must stay true until	The response condition must stay true until the until-condition parameter becomes true, and the until-condition parameter must become true within the duration specified by the max-time parameter.

The delay is an optional time interval starting from the time reference parameter to the point where the response condition is expected to be satisfied. You can set the delay to a maximum value or specify a time range in seconds.

Custom Assessments

The custom assessments template allows you to specify logical MATLAB expressions that do not fit in previous templates. Assessments are meant to evaluate signal properties, so all symbols defined in a custom template must be mapped to signal data (model element or timeseries or a constant scalar value).

Logical and Temporal Assessment Conditions

You can enter MATLAB expressions that include these operators as the assessment conditions:

- **Logical operators:** &, |, and ~
- **Relational operators:** <, <=, ==, ~=, >=, and >

Note Using == or ~= operators in logical or temporal assessment statements might cause a test to issue a warning or fail when comparing floating-point data. If you use floating-point data,

consider including a tolerance for the assessment. For example, instead of using `if(x == 5)` in a statement, check that `x` is within a tolerance of 0.001 by using `if(abs(x-5) < 0.001)`.

- **Arithmetic operators:** `+`, `-`, and `*` (multiplication by scalar constants only)
- **Cast operators:**
 - Floating-point number: `single` and `double`
 - Unsigned integer: `uint8`, `uint16`, and `uint32`
 - Signed integer: `int8`, `int16`, and `int32`
 - Logical: `logical`

The functional forms of the logical, relational, and arithmetic operators are not supported. In addition to operators, you can use the `abs` function to construct assessment conditions. You can also use the `t` symbol to construct assessment conditions, which is automatically bound to simulation time. Use of the `t` symbol as a `min-time` or `max-time` parameter in assessment conditions is not supported. Event-based signals are not supported in logical or temporal assessments.

Data Types in Assessment Conditions

Logical and temporal assessment conditions support the built-in data types listed on “Data Types Supported by Simulink”, with the exception of `string`. You can also use `Simulink.defineIntEnumType`. Fixed-point data types are not supported in assessments.

All operands in an assessment condition must be of the same data type. You can use cast operators to change the data type of an operand or change an operand to a symbol and map the symbol to an expression. Read about mapping symbols to expression on “Resolve Assessment Parameter Symbols” on page 3-87. Read about defining variables for use in an expression on “Define Variables in the Assessment Callback Section” on page 3-102. When you map a symbol to an expression, the expression must be the same data type as other operands in the assessment condition.

When mapping a symbol to a bus signal or a multidimensional signal, you must map the symbol to only one element from the bus or multidimensional signal. Read about mapping to model elements on “Resolve Assessment Parameter Symbols” on page 3-87. The data type of the selected element from the bus or multidimensional signal should always be a supported type, and must be the same data type as other operands in the assessment condition.

Define Variables in the Assessment Callback Section

The **Assessment Callback** section allows you to define variables that you can use in logical and temporal assessment conditions and expressions. You can define variables the same way you do in the MATLAB workspace. This callback also has access to predefined variables that contain data from your test, model, and simulation, such as a signal from a Simulink block. You can define a variable as a function of this data. These objects are available:

Object Name	Description
<code>TestResult</code>	The test case result (<code>sltest.testmanager.TestCaseResult</code>) or test iteration result (<code>sltest.testmanager.TestIterationResult</code>) created from the simulation.

Object Name	Description
<code>sltest_simout</code>	An array of simulation outputs (<code>Simulink.SimulationOutput</code>).
<code>sltest_testCase</code>	Current test case object (<code>sltest.testmanager.TestCase</code>).
<code>sltest_bdroot</code>	Cell array of models simulated by the test case. Can be a harness model.
<code>sltest_sut</code>	Cell array of systems under test. For a harness, this array contains the component under test.
<code>sltest_isharness</code>	Cell array that returns true if <code>sltest_bdroot</code> is a harness model.
<code>sltest_iterationName</code>	Name of current test iteration.
<code>sltest_testIteration</code>	Current test iteration object (<code>sltest.testmanager.TestIteration</code>).

After defining the variables in the callback, you can map the symbols to variables for use in assessment conditions and expressions. Read about mapping a symbol to an expression on “Resolve Assessment Parameter Symbols” on page 3-87 for information on how to map symbols to variables.

The variables created in the **Assessment Callback** section can only be used in conditions and expressions in the **Logical and Temporal Assessments** pane. These variables cannot be used in other areas of the Test Manager. The **Assessment Callback** is saved as part of the test file.

See Also

More About

- “Assess Temporal Logic by Using Temporal Assessments” on page 3-85
- “Assess Temporal Logic in Multiple Releases” on page 6-106

Observers

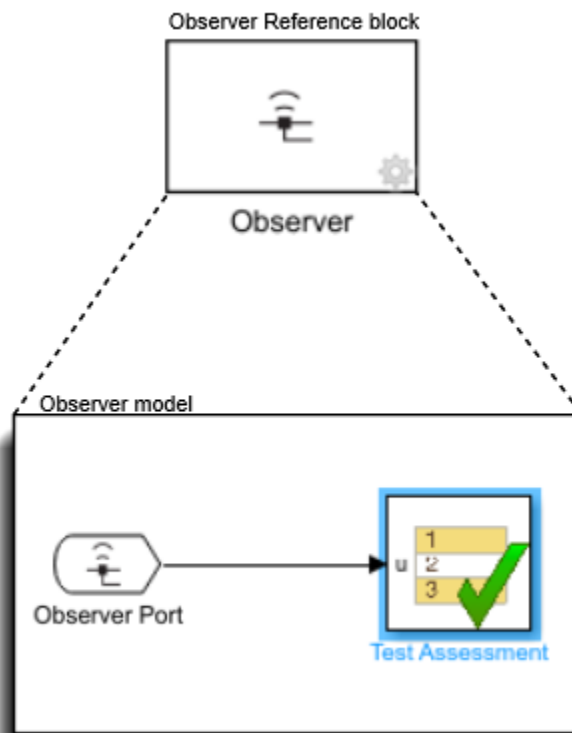
- “Access Model Data Wirelessly by Using Observers” on page 4-2
- “Observe Messages” on page 4-13
- “Observe Conditional Subsystem Signals” on page 4-17
- “Observe Internal Variables of an FMU” on page 4-21

Access Model Data Wirelessly by Using Observers

In this section...

“Observer Reference Block” on page 4-3
“Connect Signals or Other Model Data Using an Observer Port Block” on page 4-4
“Trace Observed Items to Model Signals and Objects” on page 4-6
“Simulate a System Model with an Observer Reference Block” on page 4-6
“Create Observers for Heat Pump Temperature Model” on page 4-7
“Convert Verification Subsystem to an Observer Reference” on page 4-9
“Observer Considerations and Limitations” on page 4-11

Observers allow you to monitor the dynamic response of your system model while preserving the system model design and system result integrity. Observers use two types of blocks, Observer Reference blocks and Observer Port blocks. The Observer Reference block wirelessly links a system model to an Observer model, which contains verification logic. Inside an Observer model, you use Observer Port blocks to access data from the system model to drive the verification logic.



The types of Simulink signals and model data you can observe are:

- Continuous-time and Discrete-time signals
- Zero-order hold signals

- Scalar signals
- Wide signals
- Nonvirtual bus signals
- Messages
- Conditional subsystem signals
- Stateflow local data parameters, except locals, parameters, signals, and other data defined in a Simulink subsystem inside a Stateflow state.
- Stateflow state self activity, except if that activity is in a Simulink subsystem inside a Stateflow state.

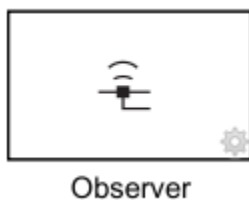
Observer Reference Block

Observer Reference blocks wirelessly link a system model to an Observer model. Observer Reference blocks can only be at the top level of a system model and do not have input or output ports. You map your Simulink signals or other model data to the Observer Port blocks that are contained within the Observer model. Once you map the Observer Port blocks to a signal or data, you can connect the ports to the verification subsystem within the Observer model. Running your system model also runs the linked Observer model.

Wireless access allows you to use observers to monitor your system model without causing changes to the system. Observers allow you to create a clear differentiation between your system design and verification subsystems.

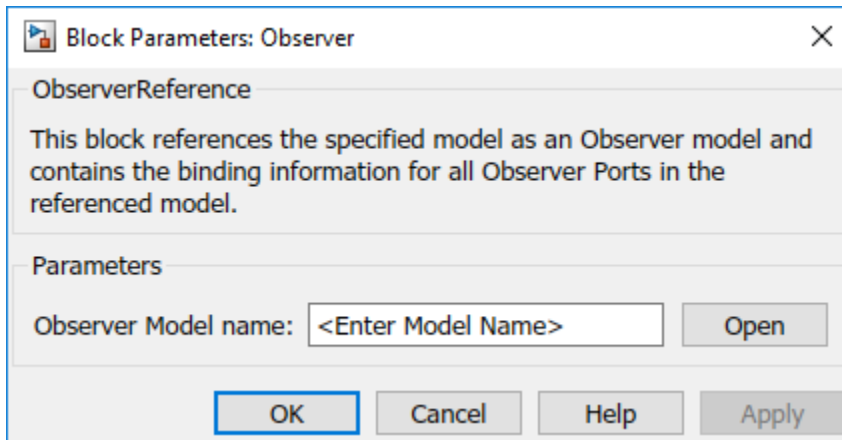
Add an Observer Reference Block

The Observer Reference block references a separate verification model that you use to verify your system model. To add an Observer Reference block to your system model, first, in the Simulink toolstrip, open Apps and click Simulink Test in the Model Verification, Validation, and Test section. Click **Add Observer Reference** in the **Tests** tab. Alternately, right-click the top level of your Simulink canvas and select **Observers > Add Observer Reference here** from the context menu. An Observer Reference block is added to your system model, and an Observer model is created and opened. You must save the Observer model in a writable folder on the MATLAB path.



Connect an Existing Observer Model

To connect an Observer Reference block to an Observer model that you have already created, first save your Observer model in a writable folder on the MATLAB path. Next, right-click on the Observer Reference block and select **Block Parameters (ObserverReference)**.



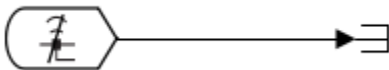
Enter the name of the Observer model that you want to connect to your system and select **Apply**. When you double-click your Observer Reference block, your Observer model opens in a new window.

Create an Observer Model from Signals or Other Model Data

To create an Observer model that is mapped to a signal line or observable data in your model, select one or more signals or the data that you want to observe. Then, click **Add Observer Reference** in the **Tests** tab. Alternately, right-click on the signal or data and select **Observers**, the item type to observe, and **New Observer**. Simulink creates an Observer model and adds an Observer Reference block to your system model.

Connect Signals or Other Model Data Using an Observer Port Block

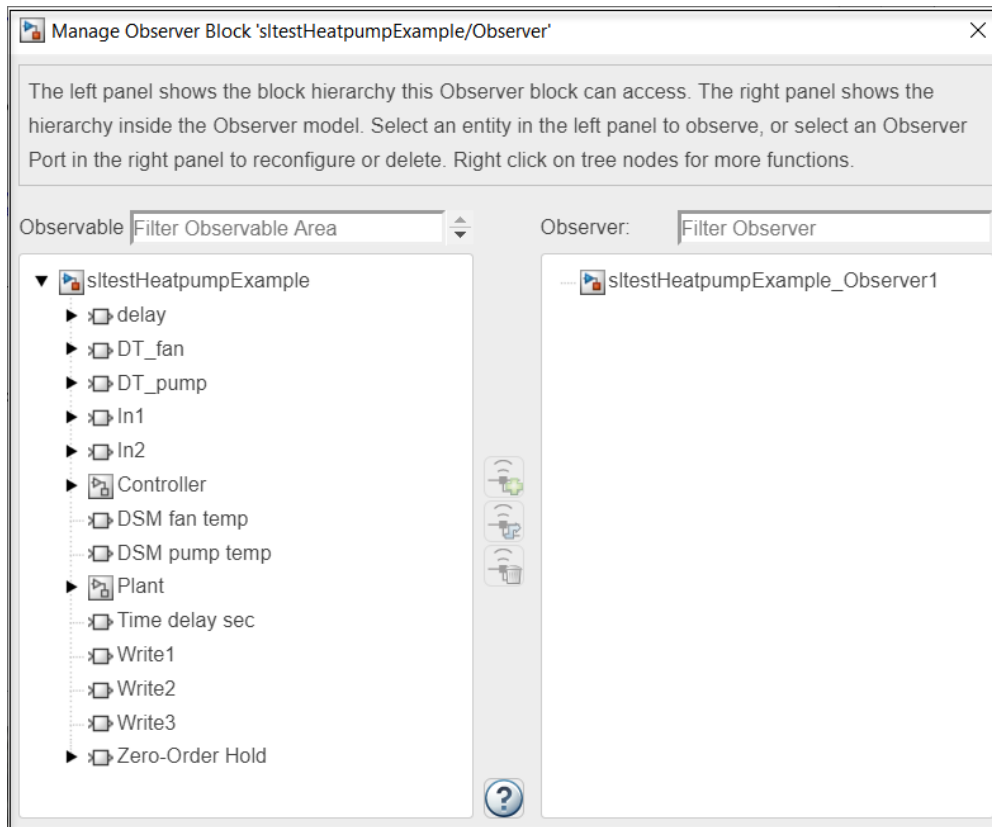
Each Observer model contains one or more Observer Port blocks. After mapping an Observer Port block to a model object or signal within a system model, the Observer Port block outputs the same output as its mapped object or signal. A new Observer Port block shows a line through the signal symbol, signifying that the block is not mapped to any signal or object.



Access the Manage Observer Dialog Box

To map an Observer Port block to a signal or object in your system model, open the Manage Observer dialog box using one of these methods:

- In the **Tests** tab, click **Manage Observer**.
- Click the gear in the lower-right corner of the Observer Reference block.
- Right-click the Observer Reference block and select **Observers > Manage Observer**.
- In the Observer model, double-click an Observer Port block.

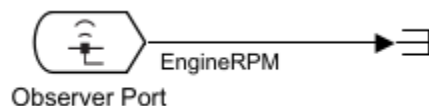


Using the Manage Observer dialog box you can:

- Filter and select signals and objects for observation
- Add, remove, or configure Observer Port blocks
- Trace signals and objects between observer ports and models

On the left side of the Manage Observer dialog box is the Observable Area panel. The Observable Area panel displays the block hierarchy and observable outputs of your model. Observed signals or objects appear bold in the hierarchy.

The right side of the Manage Observer dialog box shows the Observer panel. The Observer panel displays the block hierarchy, including Observer ports in the Observer Reference block. An Observer Port block that is mapped to a signal or object appears bold and displays the signal to which it is attached. Once the Observer Port is mapped to a signal or object, its block icon updates to show that the Observer Port is attached to a signal or object.



To view the full path of an observed object, point to an Observer Port block.

If you change the name of an observed signal or object in your system model, the Observer Reference block updates the name of the output signal from the Observer Port block. If a signal is not named and does not have a label, the output of the Observer Port block is set to an empty string.

Map an Observer Port Block to a Signal or Object

To map a signal or object to an Observer Port block, open the Manage Observer dialog box. In the Observable Area panel, select the signal or object that you want to observe. To map the signal or object to a new Observer Port block, double-click the selected item or click the Add New Observer

Port icon . To map the signal or object to an existing Observer Port block, select the Observer

Port in the Observer panel and click the Reconfigure Observer Port icon . In the Observer model, you can then connect the output from the Observer Port to a verification subsystem to test your results.

Trace Observed Items to Model Signals and Objects

You can trace observed items and their observer ports within the Manage Observer dialog box. You can also trace items between the Manage Observer dialog box and the system model, and between the system model and the Observer model.

To trace an observed item to its observer port within the Manage Observer dialog box, use one of these methods:

- Double-click on the ObserverPort item in the Observer panel. The observed item is highlighted in the Observable Area panel.
- Right-click on the ObserverPort item in the Observer panel and select **Show in left panel**. The observed item is highlighted in the Observable Area panel.

To trace an observed item or observer port between the Manage Observer dialog box and the system model, use one of these methods:

- Right-click on the ObserverPort item in the Observer panel or in the Observable Area panel and select **Show in model**. The observed item is highlighted in the model.
- Right-click on the observed signal or object in the system model and select **Go to associated Observer Ports**. The associated Observer Ports are highlighted in the Observer model.

To trace an observer port and observed item between the system model and the Observer model, use one of these methods:

- Right-click on the Observer Port in the Observer model and select **Observers > Go to observed <item type>**. The observed signal or object is highlighted in the system model.
- Right-click on the observed signal or object in the system model and select **Observers > Go to associated Observer Ports**. The associated Observer Ports are highlighted in the Observer model.

Simulate a System Model with an Observer Reference Block

The Observer model is used to monitor signals in your system model and check that your system model is running within specified parameters. With or without an Observer Reference block, your system model simulation results are the same. The Observer Reference block does not affect the compilation of your system model.

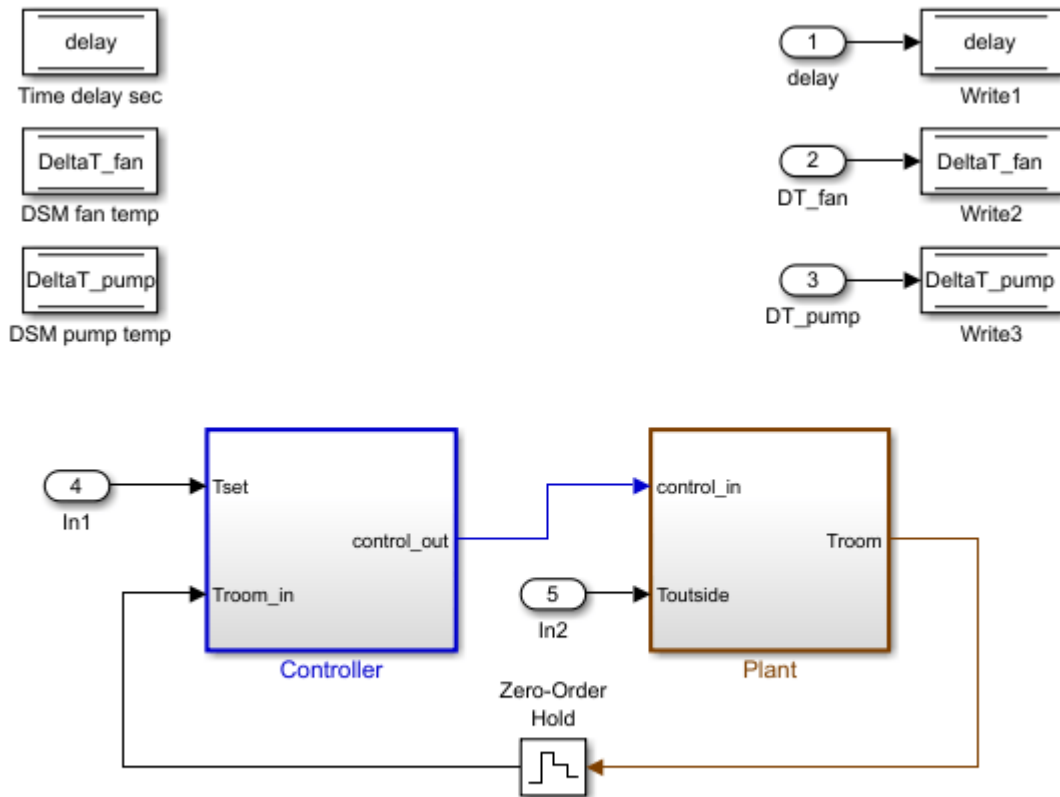
Note Both the system model and Observer model must run in normal simulation mode. Both models can run at fixed-step or variable-step rate, or one model can run at fixed rate and the other at variable rate. The two models can also use the same or different solvers. See “Choose a Solver”.

Create Observers for Heat Pump Temperature Model

This example shows how to use Observer Reference and Observer Port blocks to wirelessly observe signals. In this system, the plant is modeled using Simulink, and the controller is modeled using Stateflow. The goal of the example is to monitor both the temperature of the heat pump and when the pump is cooling or heating the room. The direction in which the fan is blowing indicates cooling or heating. The data name is `pump_dir`, and it is connected to port 3 in the Stateflow chart.

Open the Model

`sltestHeatpumpExample`



Copyright 1990-2023 The MathWorks, Inc.

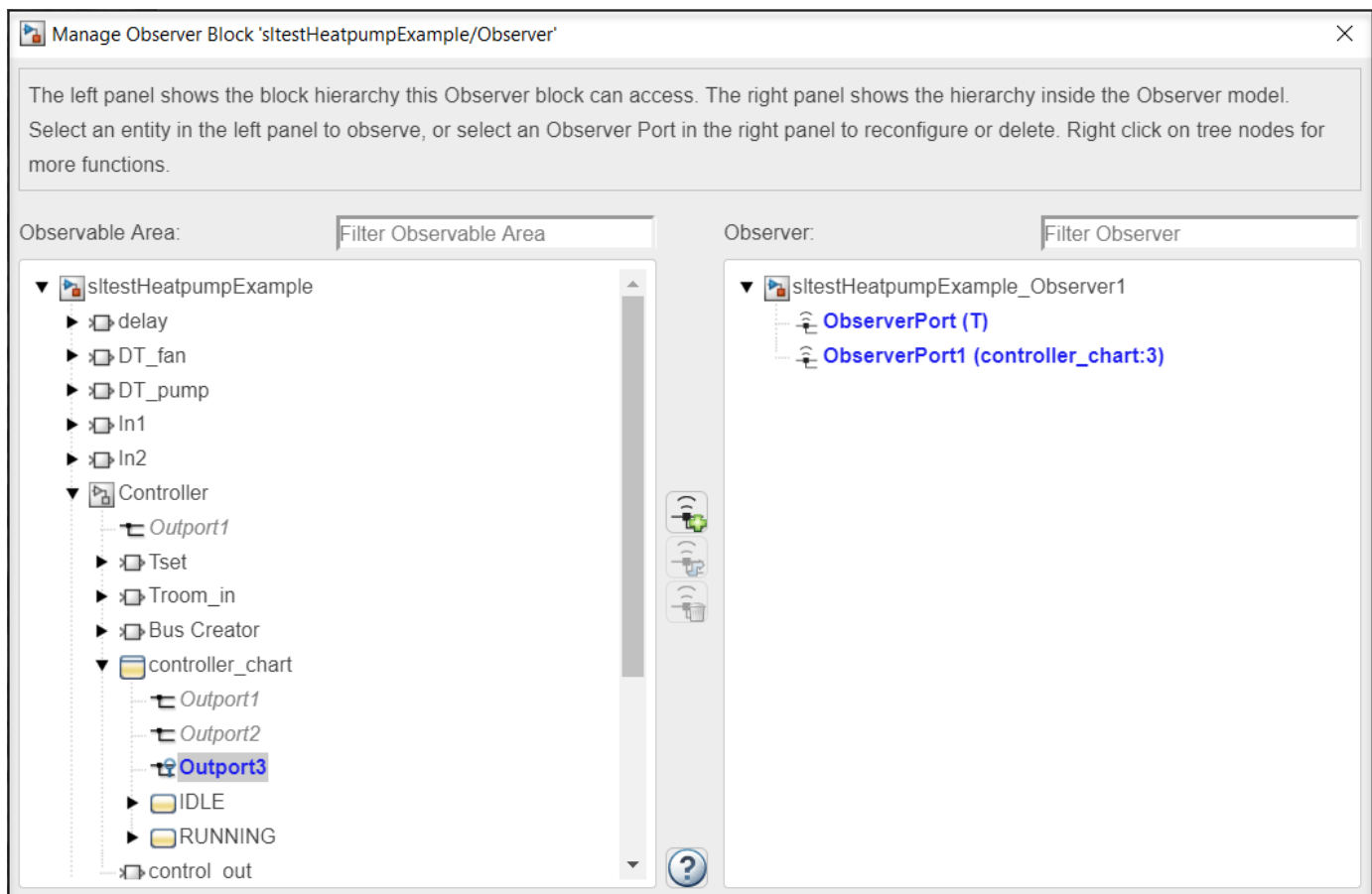
Create the Observer Model and Observer Reference Block

1. In the **Apps** tab of the model, click **Simulink Test** in the Model Verification, Validation, and Test section. The **Tests** tab opens

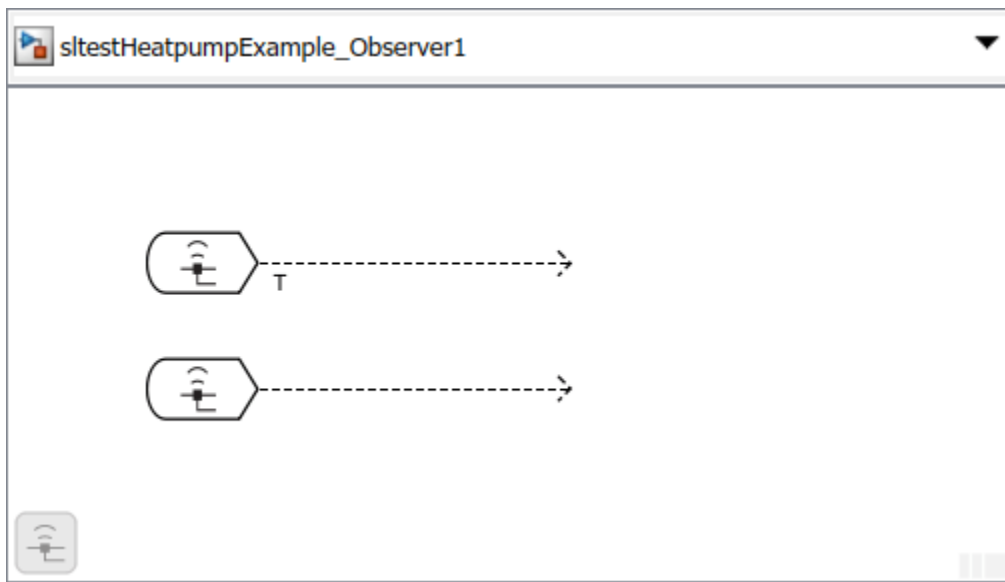
2. In the **Tests** tab, click **Add Observer Reference**. Simulink adds an Observer Reference block to your system model and creates an Observer model called `sltestHeatpumpExample_Observer1`.

Create and Map Observer Port Blocks

1. In the main model, open the Plant subsystem and right-click the signal `T`. Select **Observers > Observe selected signals > sltestHeatpumpExample/Observer (sltestHeatpumpExample_Observer1)**. The Observer model adds an Observer Port block that is mapped to signal `T`.
2. Save the new Observer model.
3. In the **Tests** tab of the Observer model, click **Add Observer Port** to add another Observer Port block. Double-click the new `ObserverPort1` block to open the Manage Observer dialog box. In the Observer panel, the second Observer Port, `ObserverPort1`, is listed below the first port.
4. To map `ObserverPort1` to the Simulink data `pump_dir`, click `ObserverPort1`. In the **Observable Area** panel, expand `Controller` and `controller_chart`, and select `Outport3`. Click the Reconfigure icon between the two panels. The `ObserverPort1` name updates to `ObserverPort1 (controller_chart:3)`.



5. The Observer Port blocks are in the Observer model and are now mapped and ready to be connected to Scope blocks or a Verification Subsystem (Simulink Design Verifier). If you want to implement a verification subsystem, see the Temperature Condition States section of “Refine, Test, and Debug a Subsystem” on page 2-22 for information about conditions to test.

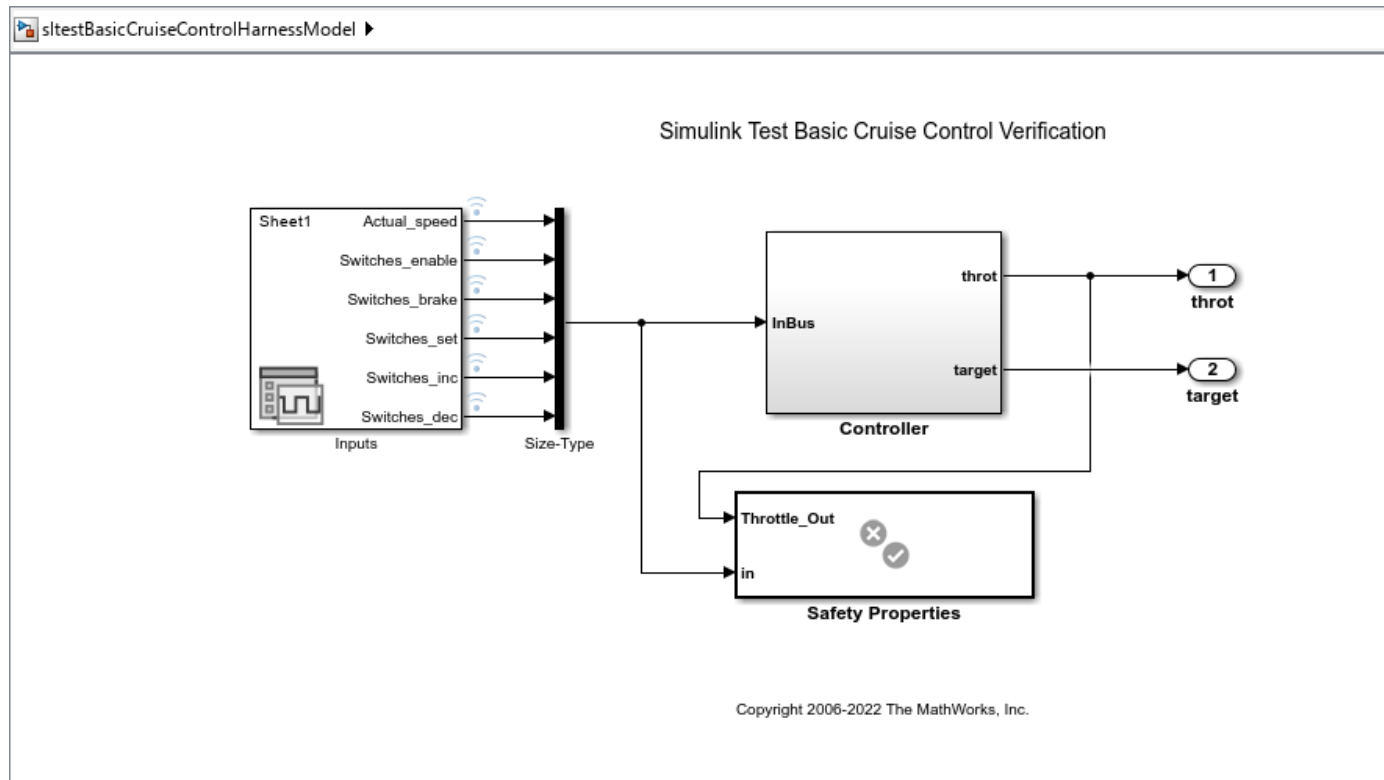


Copyright 2022 The MathWorks, Inc.

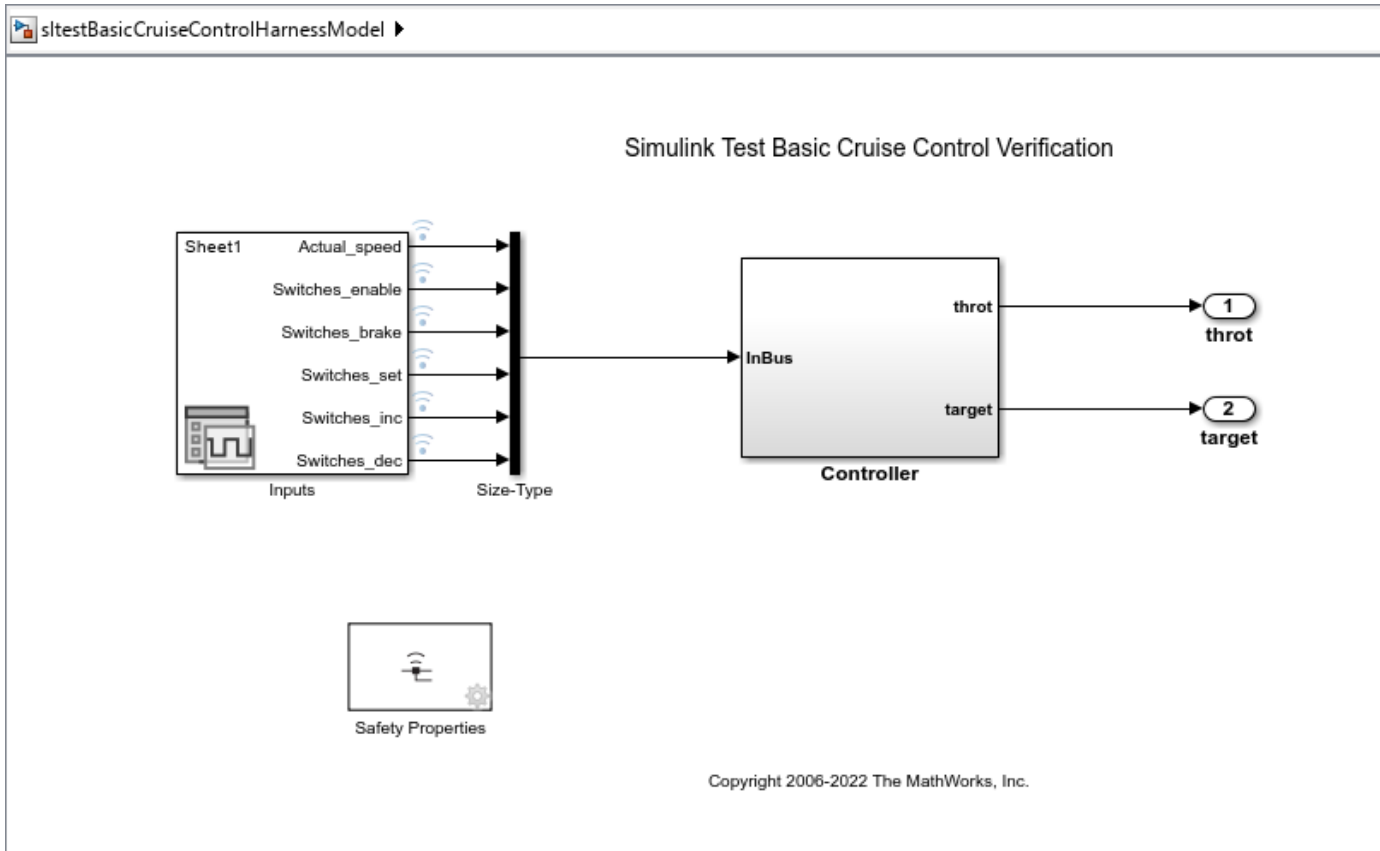
Convert Verification Subsystem to an Observer Reference

Converting a Verification Subsystem to an Observer Reference block is a way to declutter a system model. Select the subsystem to convert and, in the **Tests** tab, click **Send to Observer**. Alternately, right-click the verification subsystem and select **Observers > Move selected block to Observer > New Observer**. This operation cannot be undone.

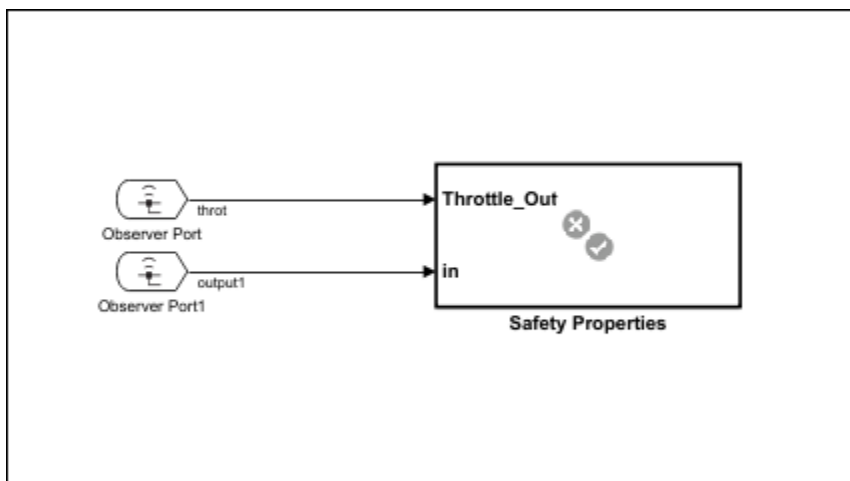
This model contains the Verification Subsystem, Safety Properties.



By converting the Safety Properties Verification Subsystem to an Observer Reference block, you remove the signals that link the verification subsystem to the system model while preserving the ability to test the integrity of the system.



The two signals, `throt` and `output1`, are automatically mapped to two Observer Port blocks in the Observer model, `sltestBasicCruiseControlHarnessModel_Observer1`.



Observer Considerations and Limitations

Model Simulation and Compilation

An Observer model does not simulate if:

- The Observer model contains root-level Inport or Outport blocks.
- The Observer model is a library or subsystem reference model.

Standalone compiling of observer models is not supported.

Observer Reference and Observer Port Blocks

Observer Reference blocks are ignored during simulation if:

- You use any simulation mode other than normal mode (for example, accelerator, SIL/PIL).
- You are generating code.
- The Observer Reference block is in a model reference hierarchy. Observer Reference blocks are supported only at the root of the top model.
- The Observer Reference block is in an Observer model. Recursion of Observer models is not supported.

Observer Port blocks do not support signals from root Inport blocks in referenced models that connect to blocks without direct-feedthrough inputs. To observe the signal, connect the signal to a block with direct-feedthrough inputs. For more information on blocks with and without direct-feedthrough inputs, see “Direct-Feedthrough Input Impact on Execution Order”.

Data Export and Output

- Logging signals or data store memory and saving final operating points are supported for Observers. All other data export options, such as time, state, output, final state, and save to file, are not supported.
- To Workspace and Dashboard blocks in Observers are not supported and do not produce output.

Mismatched Settings Between the Observer and Design Model

When these settings in the Observer model differ from the settings in the design model, the design model settings are used and the Observer model settings are ignored.

- Data import or export settings
- Coverage settings
- Solver stop time

See Also

Observer Port | Observer Reference | `sltest.observer.observeElement`

More About

- “Observe Messages” on page 4-13
- “Observe Conditional Subsystem Signals” on page 4-17

Observe Messages

Simulink uses messages to communicate between model components. When your model includes one or more message signals, you can create an Observer model to observe the message data. When you create the Observer model, it creates metadata associated with the message. You can test the message semantics and verify the message properties by using both the message data and metadata.

Message Bus Elements

When you create an Observer for a message signal, the Observer model automatically includes an Observer Port block and a Bus Selector block. The bus has two elements:

- **OrigPayload** — The message data being observed.
- **Metadata** with these elements:
 - `sltestEventMetadata.Message.id` — ID of message being observed, returned as an `int32` integer.
 - `sltestEventMetadata.Message.order` — Order of message action in the simulation, returned as an `int32` integer.
 - `sltestEventMetadata.Message.eventType` — The event type, returned as `slTestEventType` object. Valid values are `MessageArrival`, `MessageDeparture`, `MessageDrop`, and `Invalid`.
 - `sltestEventMetadata.Message.time` — Simulation time when the message was sent, received, or dropped, returned as a `double`.

Add a Message Observer

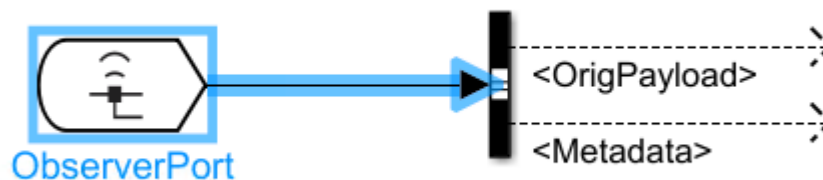
To add an observer for messages:

- 1 In the model that has one or more blocks that output messages, right-click the output message signal to observe.

Note Messages from blocks with asynchronous sample times are supported only for SimEvents® blocks that generate their own sample times.

- 2 Select **Observers > Observe selected signals > New Observer** to create an Observer model for the message signal. An Observer Reference block is added to the main model and an Observer model is created.

The new Observer model contains an Observer Port block and Bus Selector block. The Bus Selector block has two outputs, **OrigPayload** and **Metadata**. You can use the default settings in the Observer Port block to observe the outputs from the bus.



- 3 Connect the outputs from the bus to a block, such as a Test Assessment block or a Stateflow chart, to specify the logic to analyze or verify the message data or metadata.

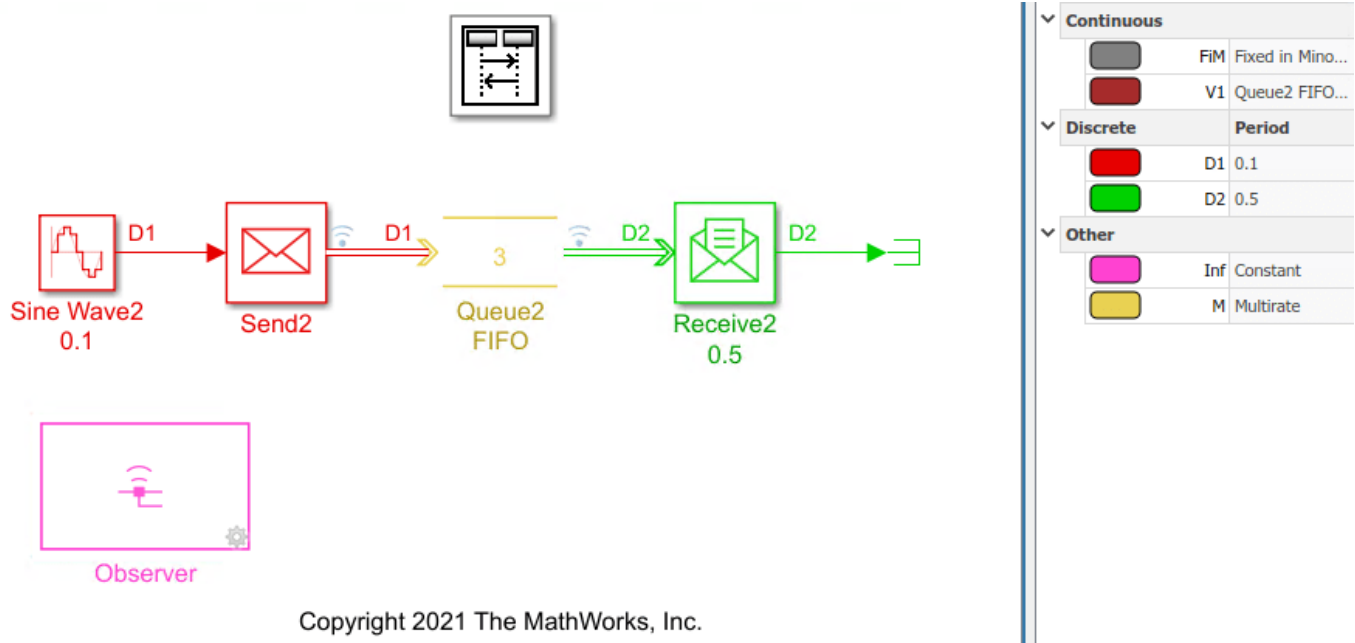
Observe a Message Signal

Open a model that has a block with message output. For this example, open the **Overflow** model, which contains an Observer model.

```
open_system('Overflow')
```

In this model, the **Sine Wave2** block sends data every 0.1 seconds, but the **Receive2** block receives data only every 0.5 seconds. Because the **Queue2 FIFO** block holds a maximum of three messages, an overflow occurs and some messages are dropped from the queue.

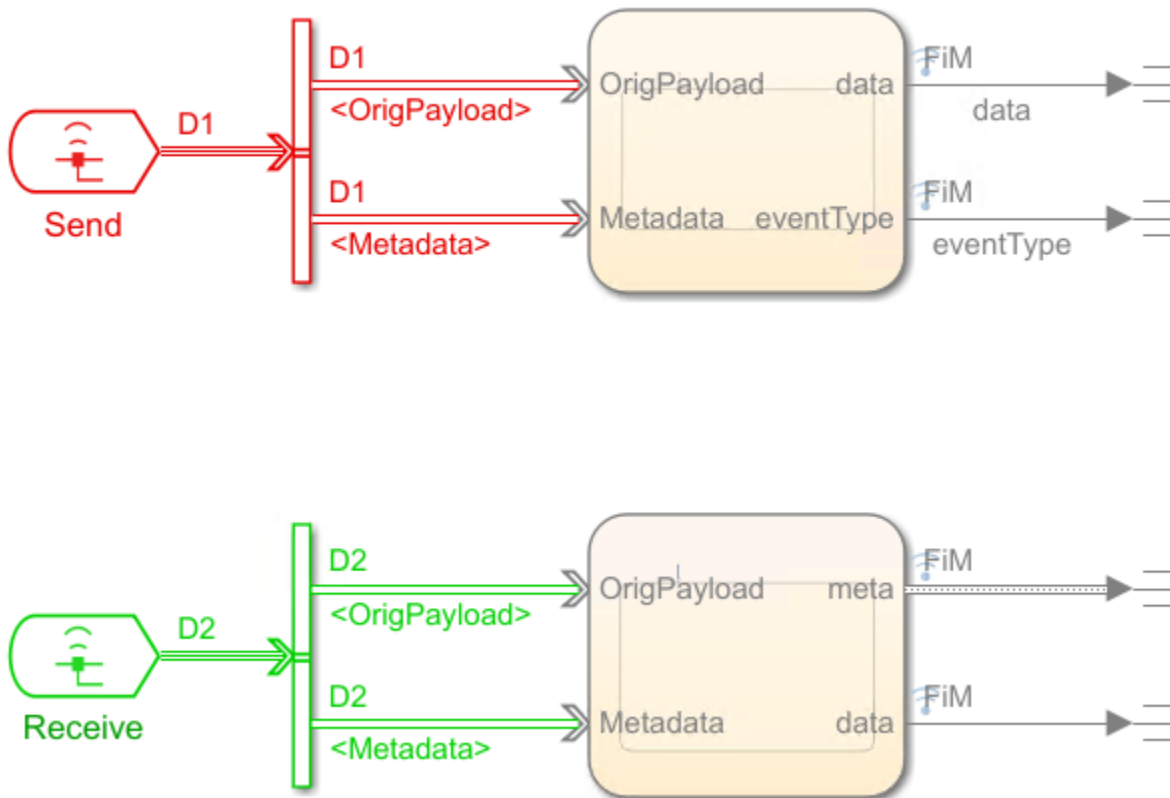
The Observer model was set up by selecting the message signals, right-clicking on one of them, and selecting **Observers > Observe selected signals > New Observer**. The Observer block, which is an Observer Reference block, was automatically added to the main model.



Open the Observer model by double-clicking the Observer block in the main model, or by using this command:

```
open_system('Overflow_Observer_1')
```

The Observer model contains two Observer Port blocks, **Send** and **Receive**, which correspond to the signals selected in the main model. Each of these blocks is connected to a Bus Selector block with two outputs. The **OrigPayload** output is the message data being observed and the **Metadata** output contains the message ID, order of the message in the simulation, the message event type, and the simulation time of the message.

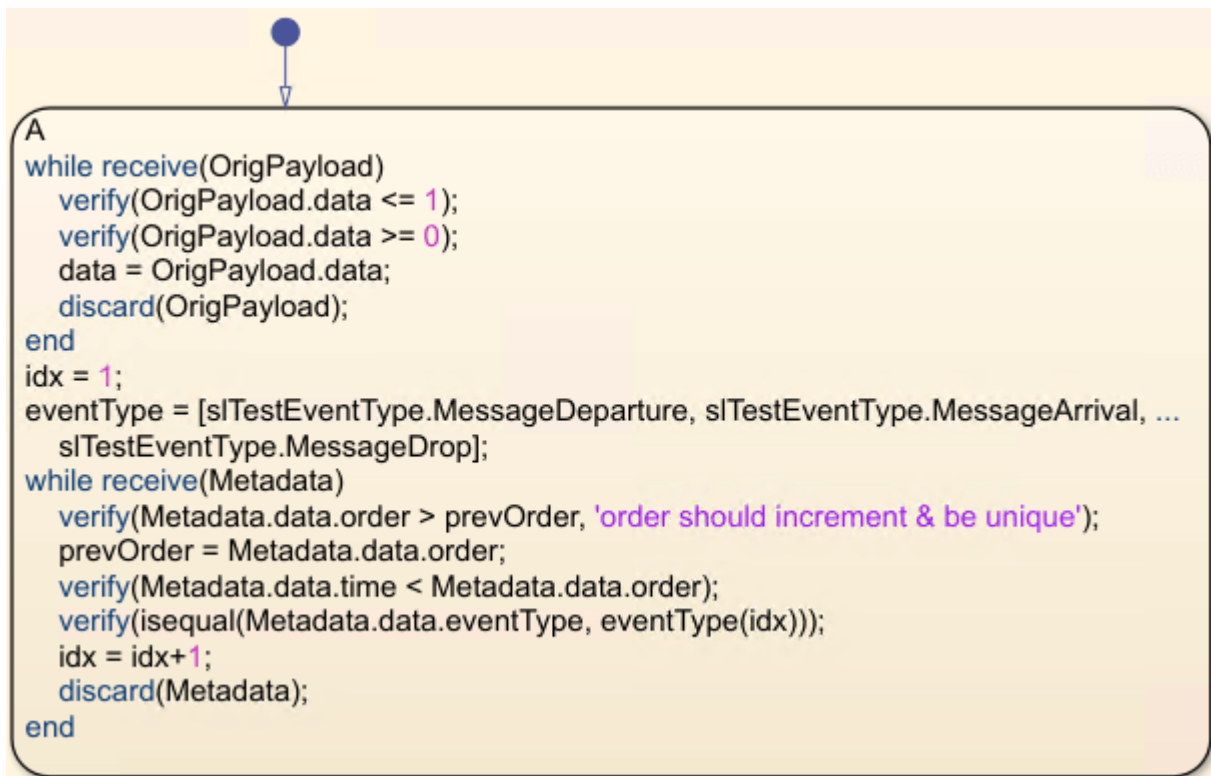


The bus signal outputs connect to two Stateflow charts, which analyze the message data.

The chart for the messages from the **Receive** Observer port block contains logic that verifies that the message data and metadata are not corrupted due to messages being dropped because of Queue block overflows.

The chart logic verifies that:

- Sine wave data is in the expected range, which is between 0 and 1
- The order of the messages and simulation time increments and that each order value is unique
- Message event type is a valid type



See Also

Observer Port | Observer Reference

More About

- “Messages”
- “Simulink Messages Overview”
- “Access Model Data Wirelessly by Using Observers” on page 4-2

Observe Conditional Subsystem Signals

Conditional subsystems are controlled by an external signal that enables or triggers the subsystem. By observing a conditional subsystem, you can check that the subsystem runs only when the controlling signal activates it, and check other functionality in the conditional subsystem. The blocks and subsystems for which you can add observers are:

- Enabled Subsystem
- Triggered Subsystem
- Message-Triggered Subsystem
- Enabled and Triggered Subsystem
- Triggered Stateflow Chart
- If Action Subsystem
- Switch Case Action Subsystem
- Function-Call Subsystem
- Simulink Function block
- Reset and reinitialize subsystems
- Aperiodic partition subsystems

For more information on conditional subsystems, see “Conditionally Executed Subsystems Overview”.

When you add an observer to a signal in a conditional subsystem, Simulink adds an Observer Reference block at the top level of the main model and creates an Observer model, which contains an Observer Port block in an aperiodic partition. This partition controls the scheduling of the conditional data. The testing logic in the partition also runs conditionally. Because the partition is created automatically when you create the observer, you do not need to change the partition or scheduling settings. You can change the name of the partition, but the name must be unique within the model.

Add an Observer for a Conditional Subsystem

The steps to add an observer to a conditional subsystem are:

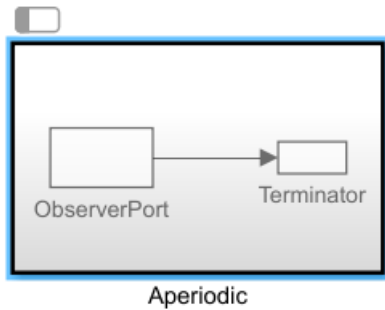
- 1 In the conditional subsystem, right-click the signal to observe.
- 2 Select **Observers > Observe selected signals > New Observer** to create an observer for the signal.

Alternatively, you can select the signal to observe, pause on the ellipsis to open the action bar,

then click **Observe in New Observer**



Simulink adds an Observer Reference block to the main model and creates a new Observer model. The new Observer model contains an aperiodic partition that contains an Observer Port block and a Terminator block. For information about partitions, see the "Partitioning a Model" section of “Create Partitions”.



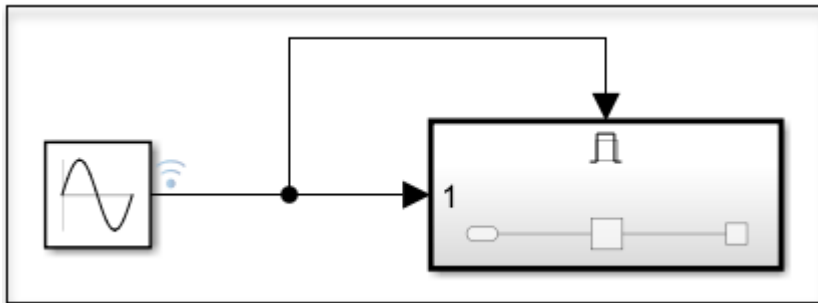
- 3 Replace the Terminator block with a verification block, such as a Test Assessment or Verification Subsystem block and connect the verification block to the Observer Port output. Add logic to verification block to analyze or verify the conditional subsystem signal.
- 4 Run the model from the main model, not from the Observer model.

Observe a Signal in a Conditional Subsystem

This example shows how to add an observer for a signal in a conditional subsystem. The `ObserveCondSubsys` model used in this example contains a sine wave control signal as the input to an Enabled Subsystem block. The Enabled Subsystem block is active only when the sine input is positive.

1. Open the `ObserveCondSubsys` model.

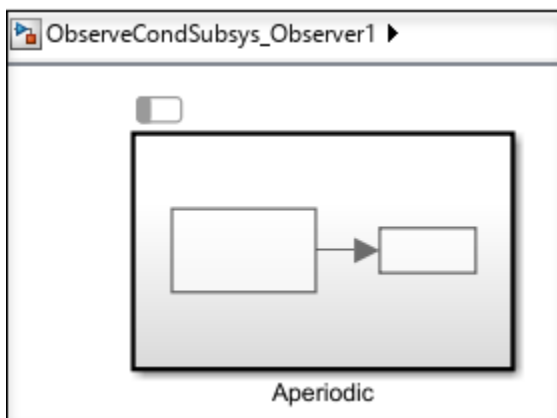
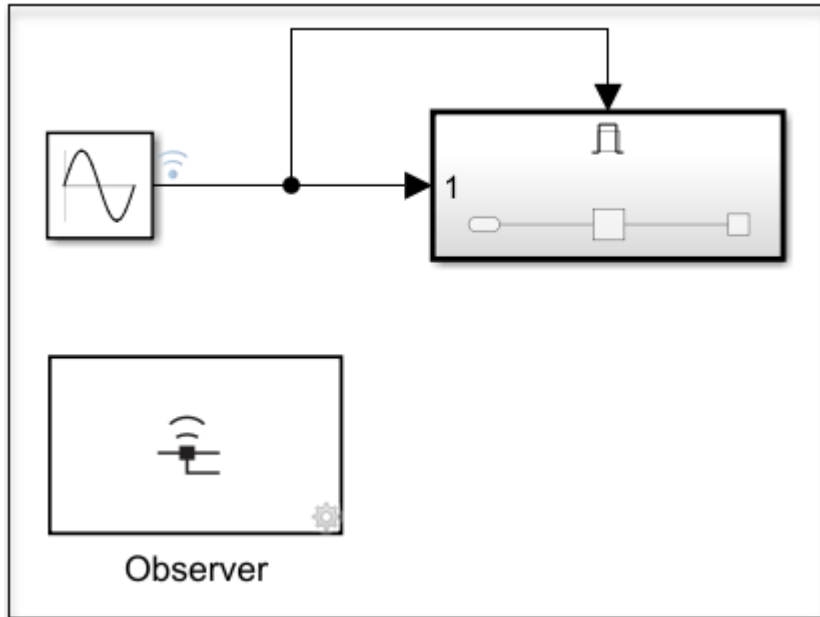
```
open_system('ObserveCondSubsys')
```



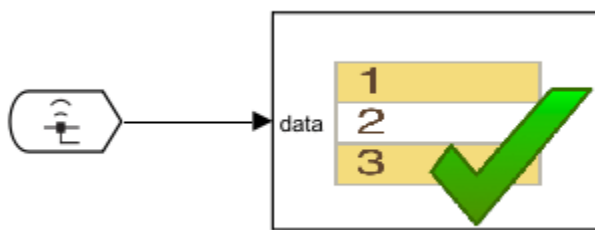
2. Open the Enabled Subsystem block.
3. Select the Integrator block output signal. Pause on the ellipses to open the action bar, then click **Observe in New Observer**.



Simulink adds an Observer Reference block to the top level of the model and opens a new Observer model. This model has an aperiodic partition that contains an Observer Port block with its output connected to a Terminator block.

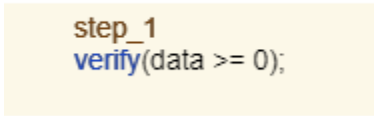


4. Open the Aperiodic partition by double-clicking it.
5. Replace the Terminator block with a Test Assessment block.



6. Open the Test Assessment block.
7. Delete **step_1_1** and **step_1_2**.
8. Right-click **step_1** and clear the **When decomposition** check box.
9. In **step_1**, add `verify(data >= 0);`

Step



```
step_1
verify(data >= 0);
```

10. Close the Test Assessment block.
11. Go to the main model and click **Run**.
12. After the model runs, at the MATLAB command line, use these commands to view the result.

```
run = Simulink.sdi.Run.getLatest;
dataset = Simulink.sdi.exportRun(run.Id);
assessmentSignalIndices = find(arrayfun(@(idx)...
    isequal(class(dataset{idx}), 'sltest.Assessment'),...
    1:dataset.numElements));
result = arrayfun(@(idx) dataset{idx}.Result, assessmentSignalIndices)
```

The `verify` statement produces a passing result, which indicates that the Enabled Subsystem block and its associated observer are active only when the sine input is positive.

Copyright 2022 The MathWorks, Inc.

Limitations

- Only one Observer Port can be in an aperiodic partition.
- Aperiodic partition names must be unique within a model.
- You cannot edit the scheduling trigger in the aperiodic partition.
- You cannot add events to the aperiodic partitions of the Observer model.
- You cannot drag and drop or reorder items in the Schedule Editor or by using `simulink.schedule.OrderedSchedule`.
- For Each, For, and While Subsystem blocks do not support observers.

See Also

Observer Port | Observer Reference | `sltest.observer.observeElement`

More About

- “Access Model Data Wirelessly by Using Observers” on page 4-2
- “Conditionally Executed Subsystems Overview”
- “Events in Schedule Editor”

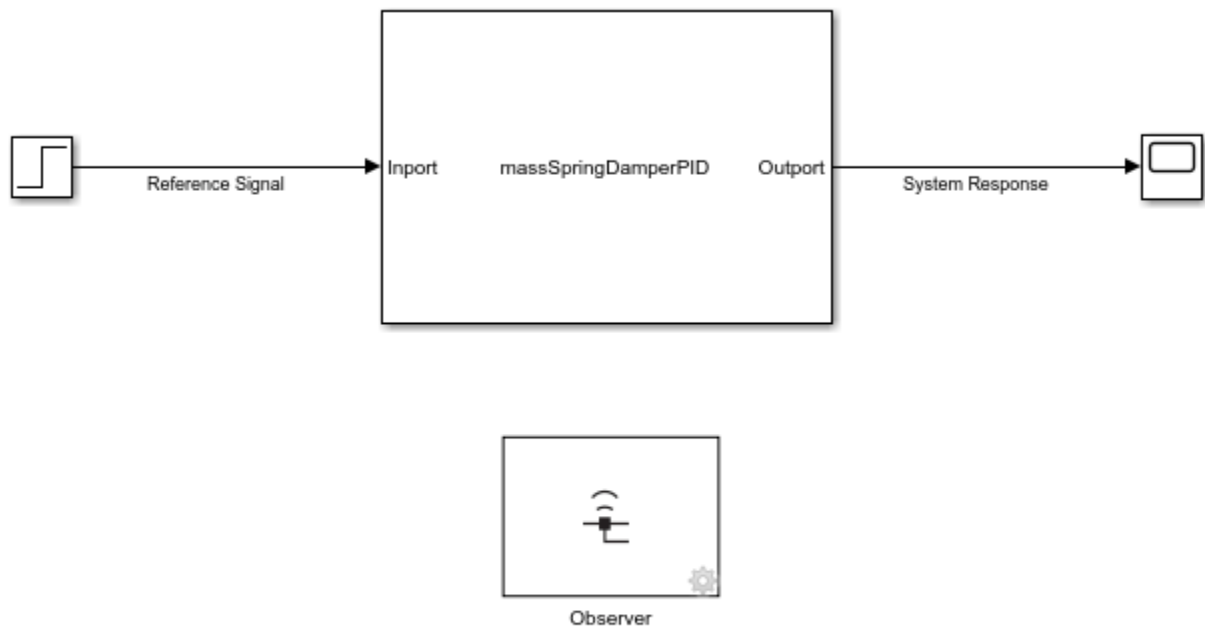
Observe Internal Variables of an FMU

Observers enable wireless access to internal variables of a Functional Mockup Unit (FMU) in Simulink. Use the Observer Reference and Observer Port blocks to access data and signals, configured as an internal variable, inside an FMU. You can observe scalar and bus signals configured as internal variables of the FMU.

Observe Internal Variables to Tune PID Controller Inside an FMU

In this system, the PID controller calculates the force that needs to be exerted on the mass to control its position. The Observer blocks allow you to tune the PID controller by providing wireless access to the control input and tracking errors that are configured as internal variables.

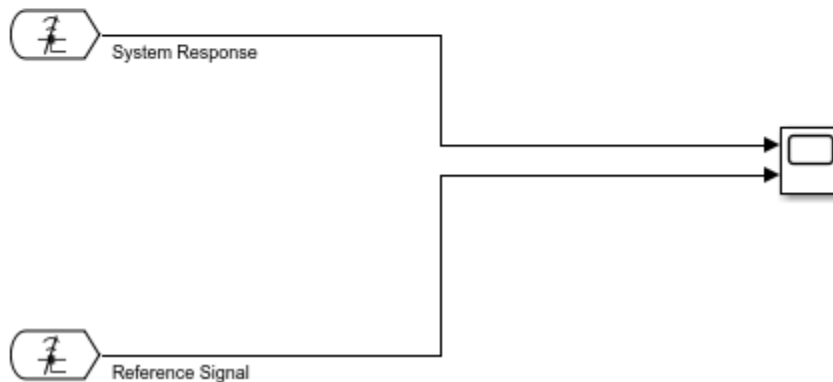
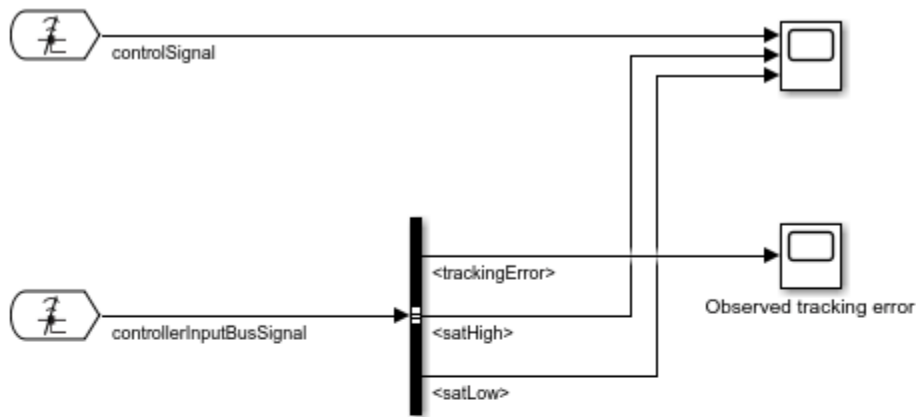
Import FMU With Internal Variables Into Simulink and Add Observer



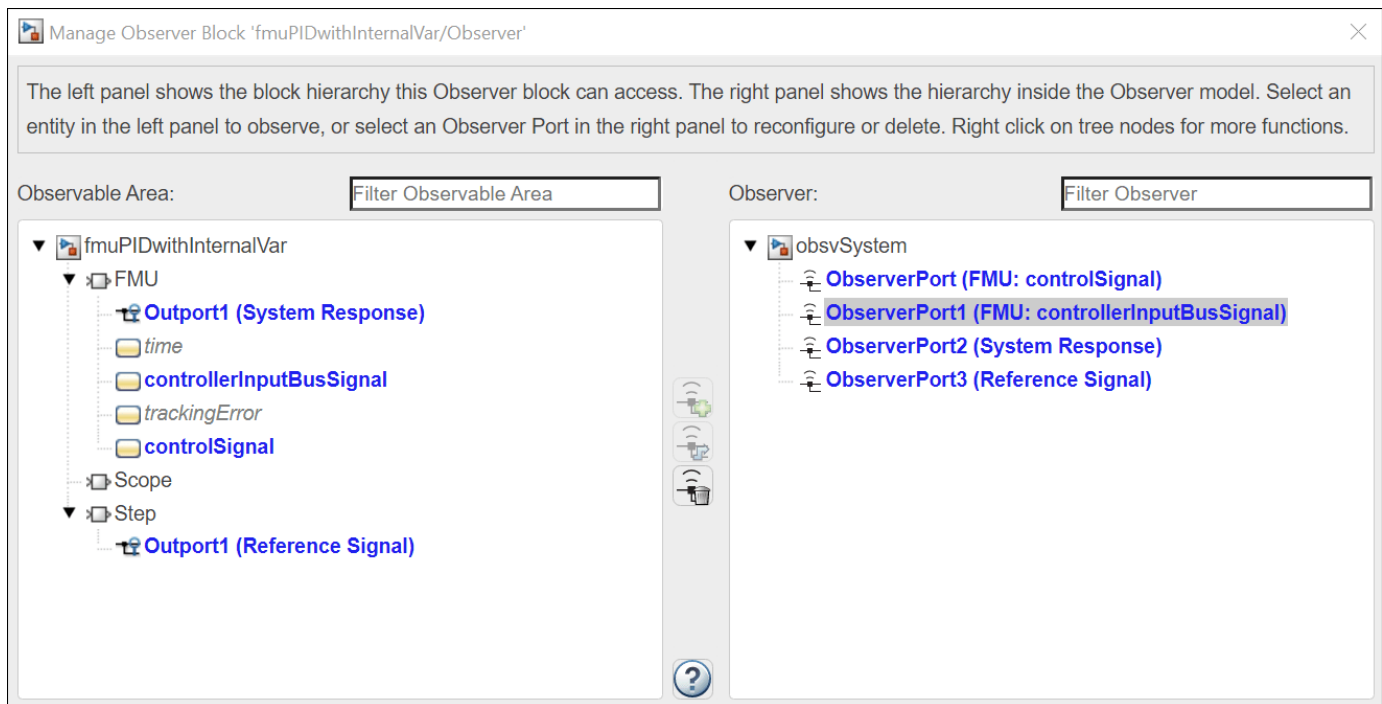
Copyright 2023 The MathWorks, Inc.

Use the FMU Import block to import the FMU into Simulink. This example uses an FMU generated using Simulink. However, the observer block is compatible with FMUs generated using external tools. Input to the FMU is the reference signal that specifies the desired position of the mass. Output from the FMU is the actual position of the mass. The PID coefficients and input saturations are tunable parameters of the FMU. The system inside the FMU consists of a cascaded PID and a mass-spring-damper system. The reference signal specifies the desired position of the mass. The PID controller provides the force as input to the mass-spring-damper system to control the position of the mass. Tracking error, control inputs and controller saturations are configured as internal variables of the system.

Add an Observer block to the main model that contains the FMU. The Observer block references an Observer model that contains the observer ports that access signals in the main model. Create the Observer model and specify its name in the **Observer Model name** of the Observer Block parameter dialog box. You can open the observer model by double-clicking the Observer block.



Add Observer Port blocks in the Observer model. Double-click on the observer ports to open the Manage Observer Block dialog box. The internal variables of the FMU are listed under the FMU block.

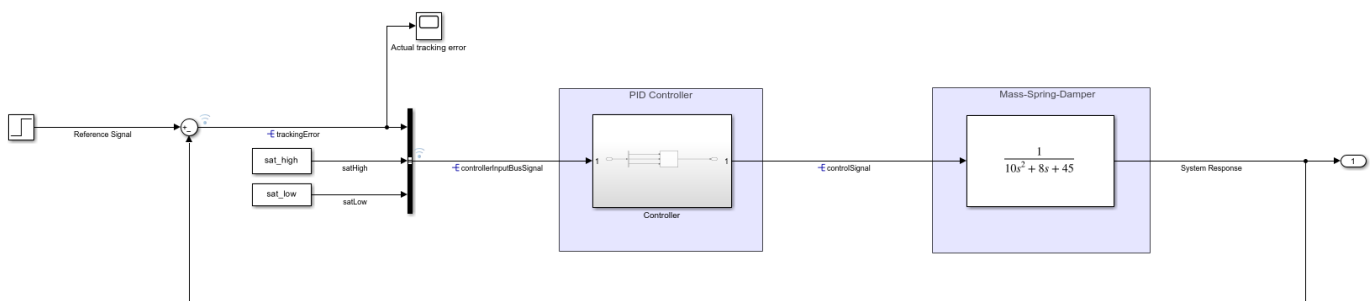


Connect the observer ports to internal variables of the FMU. For more information on connecting observer ports to observable entities, see “Access Model Data Wirelessly by Using Observers” on page 4-2.

The outputs of the observer ports can be used to wirelessly access the internal variables of the FMU along with other signals in the main model.

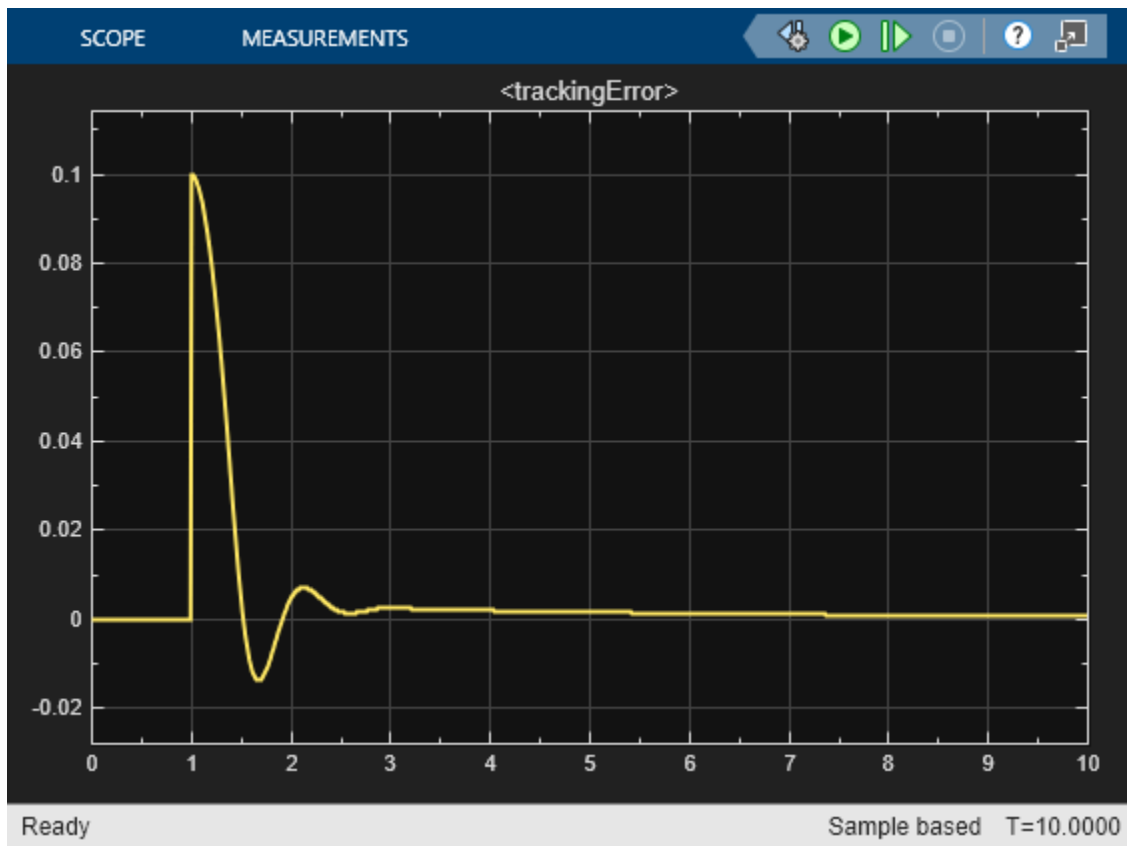
Compare Observer Output and Actual Signals

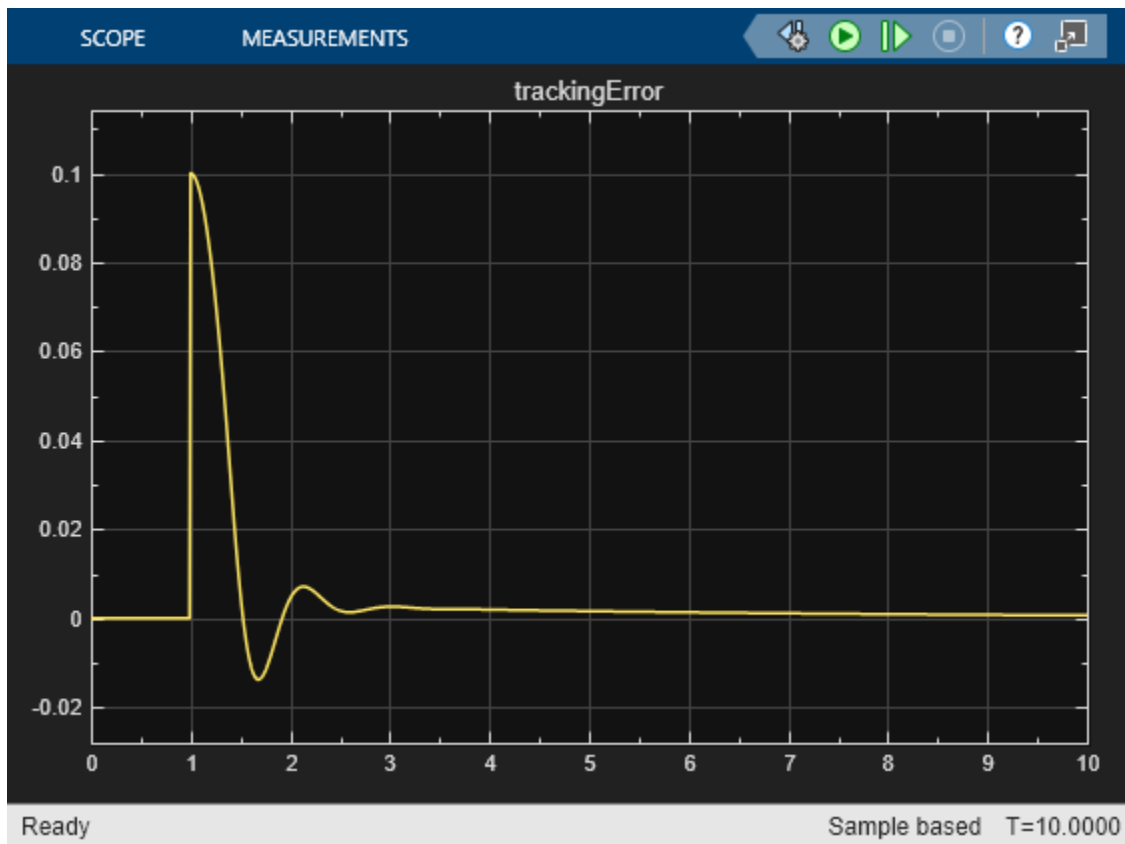
Open the original model that was used to create the FMU. Compare the tracking error and control inputs with their corresponding observed internal variables for identical controller parameters and reference signals.



Copyright 2023 The MathWorks, Inc.

Open the scope blocks to check if the tracking errors are identical for the actual and observed signals.





You can now use the observed signals to tune the PID controller inside the FMU in the main model.

See Also

Observer Port | Observer Reference | `sltest.observer.observeElement`

More About

- "Access Model Data Wirelessly by Using Observers" on page 4-2

Test Harness Software- and Processor-in-the-Loop

- “SIL Verification for a Subsystem” on page 5-2
- “Use SIL/PIL to Verify Generated Code from an Earlier Release” on page 5-6
- “Code Generation Verification Workflow with Simulink Test” on page 5-10
- “Import Test Cases for Equivalence Testing” on page 5-16
- “Test Integrated Code” on page 5-24

SIL Verification for a Subsystem

In this section...

- “Create a SIL Verification Harness for a Controller” on page 5-2
- “Configure and Simulate a SIL Verification Harness” on page 5-4
- “Compare the SIL Block and Model Controller Outputs” on page 5-4

This example shows subsystem verification by ensuring the output of software-in-the-loop (SIL) code matches that of the model subsystem. You generate a SIL verification harness, collect simulation results, and compare the results using the simulation data inspector. You can apply a similar process for processor-in-the-loop (PIL) verification.

With SIL simulation, you can verify the behavior of production source code on your host computer. With PIL simulation, you can verify the compiled object code that you intend to deploy in production. You can run the PIL object code on real target hardware or on an instruction set simulator.

If you have an Embedded Coder license, you can create a test harness in SIL or PIL mode for model verification. You can compare the SIL or PIL block results with the model results and collect metrics, including execution time and model code coverage. You cannot collect coverage on the SIL or PIL blocks. Using the test harness to perform SIL and PIL verification, you can:

- Manage the harness with your model. Generating the test harness generates the SIL block. The test harness is associated with the component under verification. You can save the test harness with the main model.
- Use built-in tools for these test-design-test workflows:
 - Checking the SIL or PIL block equivalence
 - Updating the SIL or PIL block to the latest model design
- View and compare logged data and signals using the Test Manager and Simulation Data Inspector.

When you create an equivalence test that compares normal and SIL or PIL simulation modes, a separate test harness is used to test each mode. However, if you are equivalence testing an atomic subsystem or Model block, a single test harness can be used for both the normal and SIL or PIL simulations. For information about when a single harness is used for atomic subsystem equivalence tests, see “Generate Tests and Test Harnesses for a Model or Components” on page 6-20.

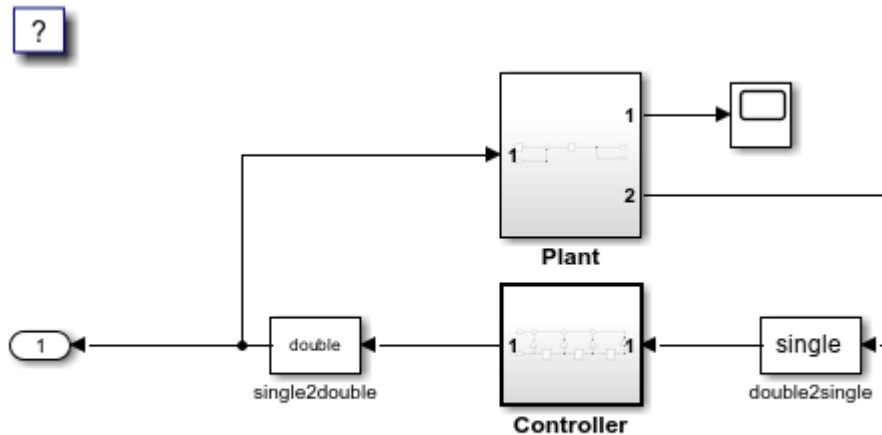
This example models a closed-loop controller-plant system. The controller regulates the plant output.

Create a SIL Verification Harness for a Controller

Create a SIL verification harness using data that you log from a closed-loop controller-plant system. The controller subsystem regulates the plant output. You need an Embedded Coder license for this example. Another way to create a SIL harness is with the Create Test for Model Component Wizard (see “Generate Tests and Test Harnesses for a Model or Components” on page 6-20 and “Back-to-Back Equivalence Testing” on page 6-36).

- 1 Open the example model by entering this command in the MATLAB Command Window.

```
openExample('ecoder/SILPILVerificationExample', ...
    supportingFile='SILBlock.slx')
```



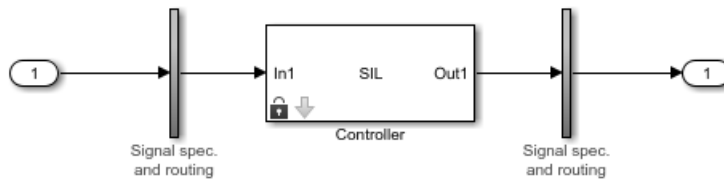
Copyright 2004-2020 The MathWorks, Inc.

- 2 Save a copy of the model using the name `controller_model` in a new folder, in a writable location on the MATLAB path.
- 3 Enable signal logging for the model. At the command prompt, enter


```
set_param(bdroot,SignalLogging="on",SignalLoggingName=...
"SIL_signals",SignalLoggingSaveFormat="Dataset");
```
- 4 Right-click the signal into Controller port In1, and select **Properties**. In the **Signal Properties** dialog box, for the **Signal name**, enter `controller_model_input`. Select **Log signal data** and click **OK**.
- 5 Right-click the signal out of Controller port Out1, and select **Properties**. In the **Signal Properties** dialog box, for the **Signal name**, enter `controller_model_output`. Select **Log signal data** and click **OK**.
- 6 Simulate the model.
- 7 Get the logged signals from the simulation output into the workspace. At the command prompt, enter


```
out_data = out.get("SIL_signals");
control_in1 = out_data.get("controller_model_input");
control_out1 = out_data.get("controller_model_output");
```
- 8 Create the software-in-the-loop test harness. Right-click the Controller subsystem and select **Test Harness > Create for 'Controller'**.
- 9 Set the harness properties:
 - **Name:** `SIL_harness`
 - **Sources and Sinks:** Inport and Outport
 - Select **Open harness after creation**
 - **Advanced Properties - Verification Mode:** Software-in-the-loop (SIL)

Click **OK**. The resulting test harness has a SIL block.



Configure and Simulate a SIL Verification Harness

Configure and simulate a SIL verification harness for a controller subsystem.

- 1 Configure the test harness to import the logged controller input values. From the top level of the test harness, in the model **Configuration Parameters** dialog box, in the **Data Import/Export** pane, select **Input**. Enter `control_in1.Values` as the input and click **OK**.
- 2 Enable signal logging for the test harness. At the command prompt, enter

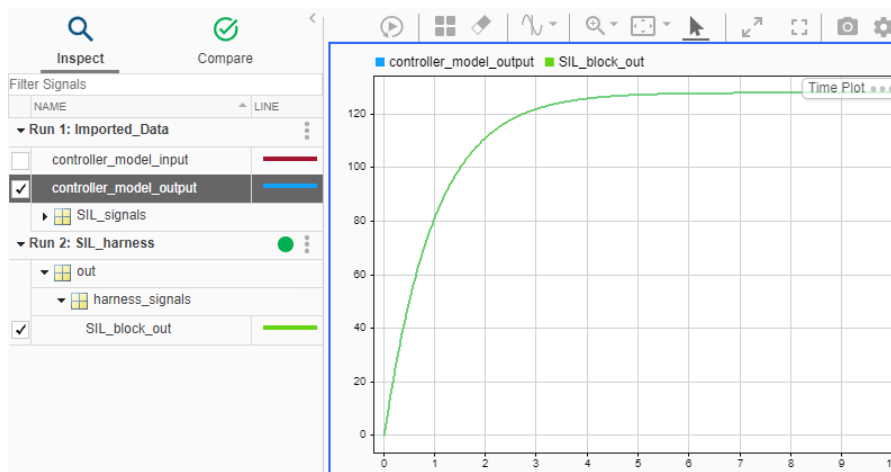
```
set_param("SIL_harness",SignalLogging="on",SignalLoggingName=...  
"harness_signals",SignalLoggingSaveFormat="Dataset");
```
- 3 Right-click the output signal of the SIL block and select **Properties**. In the **Signal Properties** dialog box, for the **Signal name**, enter `SIL_block_out`. Select **Log signal data** and click **OK**.
- 4 Simulate the harness.

Compare the SIL Block and Model Controller Outputs

Compare the outputs for a verification harness and a controller subsystem.

- 1 In the test harness model, in the **Simulation** tab, in the **Review Results** section, click **Data Inspector**  to open the Simulation Data Inspector.
- 2 In the Simulation Data Inspector, click **Import**. In the **Import** dialog box.
 - Set **Import from** to: Base workspace.
 - Set **Import to** to: New Run.
 - Under **Name**, select all of the check boxes to import data from all sources.
- 3 Click **Import**.
- 4 Select the `SIL_block_out` and `controller_model_out` signals in the **Runs** pane of the data inspector window.

The chart displays the two signals, which overlap. This result suggests equivalence for the SIL code. You can plot signal differences using the **Compare** tab in SDI, and perform more detailed analyses for verification. For more information, see “Compare Simulation Data”.



- 5 Close the test harness window. You return to the main model. The badge  on the Controller block indicates that the SIL harness is associated with the subsystem.

See Also

More About

- “Back-to-Back Equivalence Testing” on page 6-36
- “Generate Tests and Test Harnesses for a Model or Components” on page 6-20
- “Configure and Run SIL Simulation” (Embedded Coder)
- “Use SIL/PIL to Verify Generated Code from an Earlier Release” on page 5-6
- “Generate Subsystem Code as Separate Function and Files” (Simulink Coder)

Use SIL/PIL to Verify Generated Code from an Earlier Release

If you generate code for an atomic subsystem and verify it using SIL/PIL simulation in an earlier release, then you can verify that the model continues to work as expected in the current release by reusing that code. Reusing the generated code saves test harness generation time because you do not have to regenerate the code.

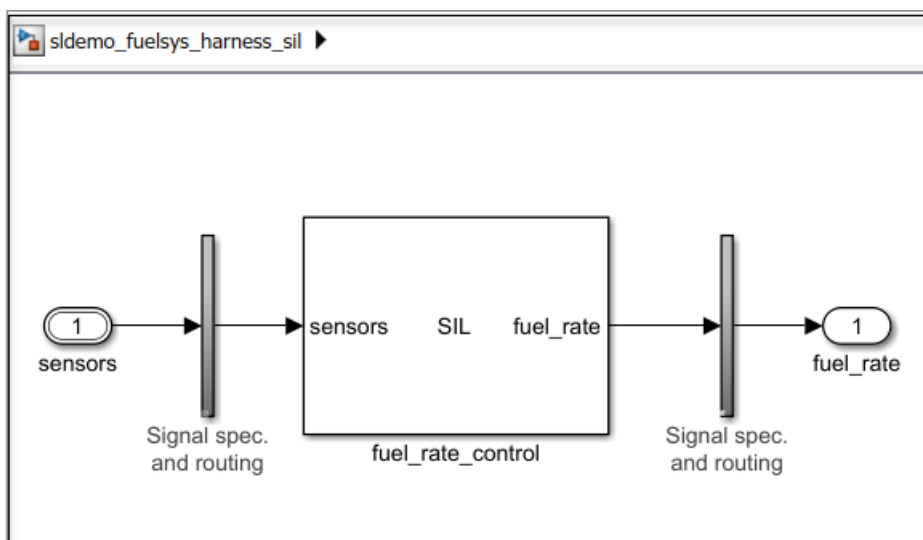
Note

- You cannot reuse generated code for test harnesses for whole models or Model blocks.
- You can reuse generated code from only the previous eight releases. Trying to reuse generated code from earlier releases produces an error.
- You must have an Embedded Coder license to reuse generated code from an earlier release.

Create Test Harness That Reuses Generated Code

To reuse generated code from an earlier release, create a test harness for SIL or PIL verification:

- 1 Right-click the atomic subsystem in your model for which you generated code in the earlier release and select **Test Harness > Create for <subsystem>**.
- 2 In the Create Test Harness dialog box, specify a **Name** for the harness.
- 3 In the **Advanced Properties** tab, set the harness as a SIL or PIL harness that verifies existing generated code:
 - a Change the **Verification Mode** to Software-in-the-Loop (SIL) or Processor-in-the-Loop (PIL).
 - b Select **Use existing generated code to create SIL/PIL block**.
 - c In the **Build folder** box, enter the full path to the folder that contains the generated code from the earlier release. For example, if you configured the system target file to be `ert.tlc`, the folder name ends in `"_ert_rtw"`.
- 4 Click **OK** to create the test harness.



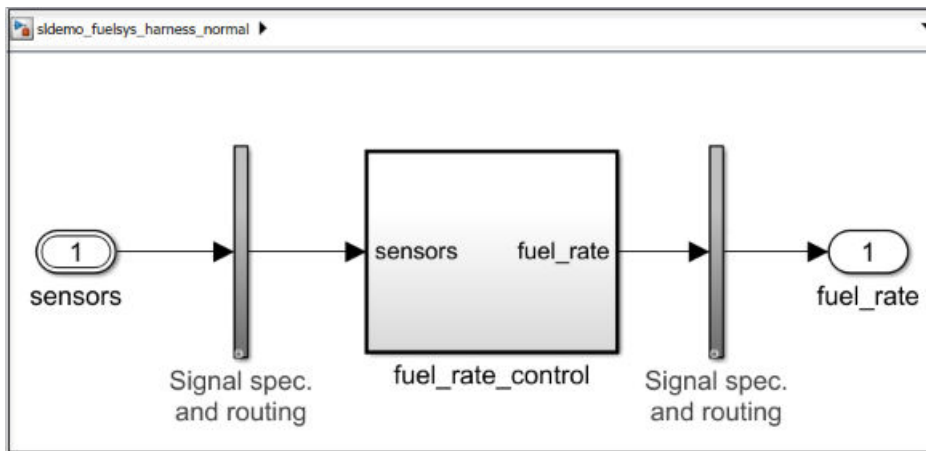
- 5 Select the signal exiting the subsystem in the test harness. Point to the ellipsis to open the action bar and select **Log Selected Signal**.
- 6 Close the harness. You do not need to explicitly save the harness.

To use previously generated code verified using a SIL/PIL subsystem programmatically, use the `ExistingBuildFolder` property of `sltest.harness.create` or `sltest.harness.set` to specify the location of the generated code.

Create Test Harness for Normal Simulation

To test the behavior of the subsystem in the current release, create a test harness for normal simulation:

- 1 Right-click the atomic subsystem in your model for which you generated code in the earlier release and select **Test Harness > Create for <subsystem>**.
- 2 In the Create Test Harness dialog box, specify a **Name** for the harness.
- 3 Click **OK** to create the test harness.



- 4 Select the signal exiting the subsystem in the test harness. Point to the ellipsis to open the action bar and select **Log Selected Signal**.
- 5 Close the harness.

Create Equivalence Test Case

To verify that the model in the current release continues to work as expected, create an equivalence test that compares the normal simulation in the current release and the SIL/PIL mode simulation from the earlier release:

- 1 Open the Test Manager.
`sltest.testmanager.view`
- 2 Click **New > Test File**. Name the test file `SIL_reuse`.
- 3 Delete New Test Case 1.
- 4 Highlight New Test Suite 1 and click **New > Test Case > Equivalence Test**.
- 5 Change the name of New Test Case 1 to `SIL equivalence test case`.

- 6 In the **Simulation 1** section, under **System Under Test**:
 - a Specify the **Model**.
 - b Under **Test Harness > Harness**, specify the test harness for normal simulation.
 - c Under **Simulation Settings and Release Overrides**, specify the **Start Time** and **Stop Time**. Leave **Release** as **Current**.
- 7 In the **Simulation 2** section, under **System Under Test**:
 - a Specify the **Model**.
 - b Under **Test Harness > Harness**, specify the test harness that uses the generated code.
 - c Under **Simulation Settings and Release Overrides**, specify the **Start Time** and **Stop Time**. Leave **Release** as **Current**.
- 8 For both simulations, in the **Inputs** sections, specify the inputs that you used to verify the system in the previous release. For example, if you saved the inputs in a MAT file:
 - a Under **External Inputs**, click **Add**.
 - b In the Add Input dialog box, specify the name of the file.
 - c Click **Map Inputs** and then click **Add**.

Add Input

INPUT FILE SPECIFICATION

File:

☐ Add iterations to run this input

INPUT MAPPING

Mapping Mode:

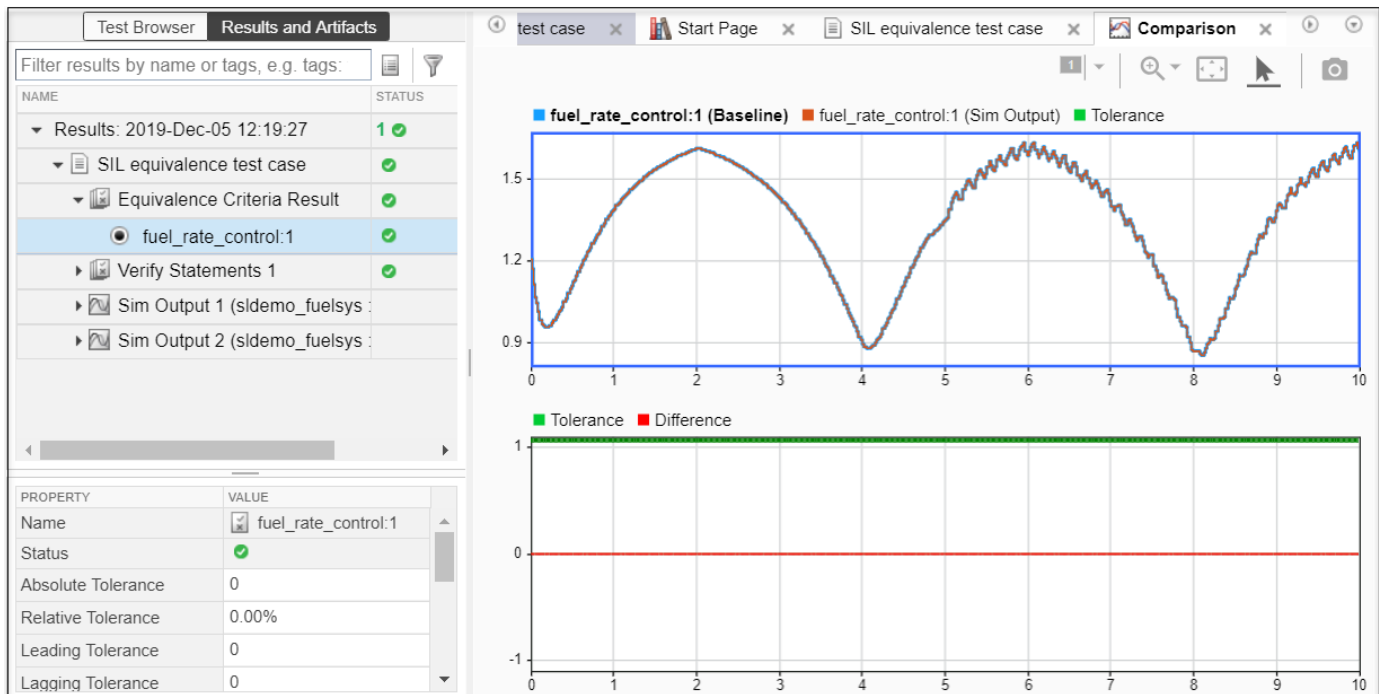
☒ Compile the system under test

MAPPING STATUS

ADVANCED

Run Test and Analyze Results

To verify that the results match the results produced by the same code in the earlier release, run the equivalence test by clicking **Run**. To view the output and analyze the test results, in the **Results and Artifacts** pane, expand Equivalence Criteria Result.



The upper plot shows the output of both test harnesses. The lower plot shows the difference between these outputs. If the difference is zero, the two results plots match exactly so the code verified in the earlier release and the code generated in the current release produce the same results.

See Also

`sltest.harness.create` | `sltest.harness.set` | `crossReleaseImport`

More About

- “Create or Import Test Harnesses and Select Properties” on page 2-13
- “SIL Verification for a Subsystem” on page 5-2

Code Generation Verification Workflow with Simulink Test

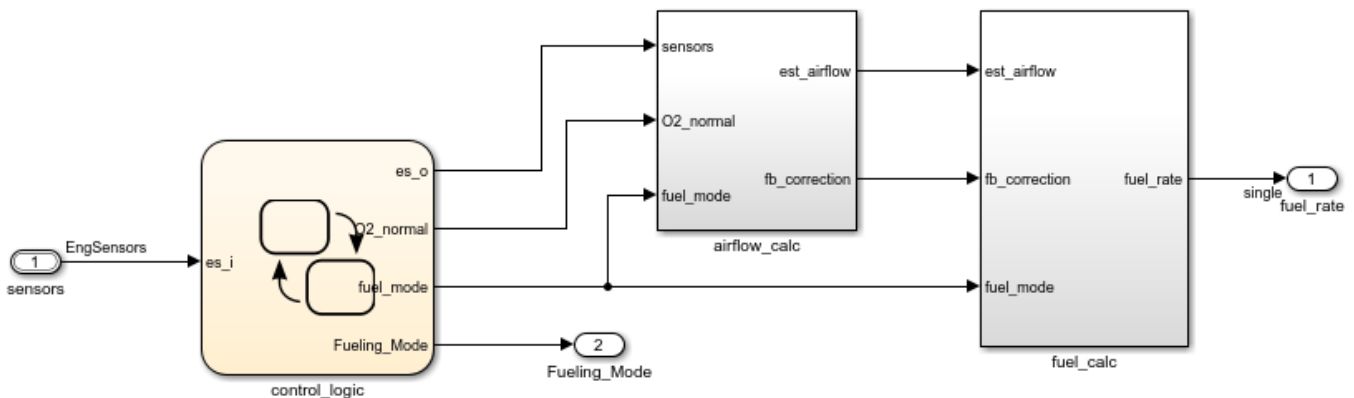
This example shows how to perform code generation verification (CGV) for a model using test harnesses, Test Sequence blocks, and the Test Manager.

Open the Fuel-Rate Controller Model

```
mdl = 'sltestFuelRateControlExample';
open_system(mdl);
```

Code Generation Verification Workflow with Simulink Test

This model is used to show how to perform code generation verification (CGV) using test harnesses, Test Sequence blocks and the test manager. To see the demo, execute `showdemo sltestFuelRateControlCGVDemo` in MATLAB(R).



Copyright 2015 The MathWorks, Inc.

Description of the Model

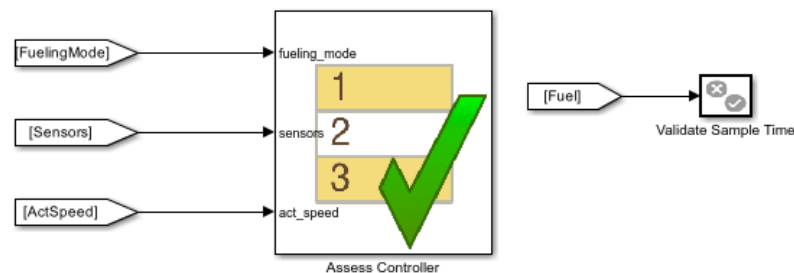
This example uses a model of a fuel-rate controller for a gasoline engine. The controller uses four sensors from the system to determine the proper fuel rate. The four sensors used from the system are throttle angle, speed, EGO and manifold absolute pressure [MAP].

The model uses three subsystems to calculate the fuel rate using the sensor inputs: `control_logic`, `airflow_calc`, and `fuel_calc`. The core control logic is implemented in the Stateflow® chart named `control_logic`. The control logic handles single sensor failures and engine overspeed protection. If a single sensor fails, operation continues but the air/fuel mixture is richer to allow smoother running at the cost of higher emissions. If more than one sensor fails, the engine shuts down as a safety measure, since the air/fuel ratio cannot be controlled reliably.

The model estimates the airflow rate and multiplies the estimate by the reciprocal of the desired ratio to give the fuel rate.

Open the Test Harness

```
sltest.harness.open mdl, 'fuel_rate_control_cgv');
```



Model the Plant

Notice that the plant has been modeled in the test harness instead of the main model. The main model is free of extraneous clutter so that code can be easily built for an ECU with minimal changes to the model.

The Test Sequence block named **Sequence Sensor Failures** models various sensor failure and engine overspeed scenarios. It accepts feedback from the plant and drives the controller with sensor data. This modeling pattern allows the Test Sequence block to control the feedback signals received by the Controller block and function as a canvas for authoring test cases. Open the Test Sequence block to see the modeled test scenarios.

Test Scenarios

For the first 10 seconds of simulation, the test is in stabilization mode, where the closed loop inputs from the plant are passed through to the controller. The throttle and speed inputs are set to nominal values that are within the normal operating envelope of the controller. The `Stabilize_Engine` step models this state. The Test then steps through the following modes:

- 1 `Test_Overspeed`: The throttle is ramped from 30 to 700
- 2 `Reset_To_Normal_Speed`: The throttle is ramped down to 400
- 3 `Test_EGO_Fault`: Simulate failure for 3 sec, then return to normal state
- 4 `Test_Throttle_Fault`: Simulate failure for 3 sec, then return to normal state
- 5 `Test_Speed_Fault`: Simulate failure for 3 sec, then return to normal state
- 6 `Test_Map_Fault`: Simulate failure for 3 sec, then return to normal state
- 7 `Test_Multi_Fault`: Simulate MAP and EGO failure for 3 sec
- 8 `Reset_MAP`: Normalize MAP sensor, and simulate just EGO failure for 3 sec
- 9 `Reset_To_Normal`: Terminate the test

Test Assessments

The Test Sequence block `Assess_Controller` verifies the controller output for the various test cases modeled by the `Sequence_Sensor_Failures` block. The following assessments are modeled:

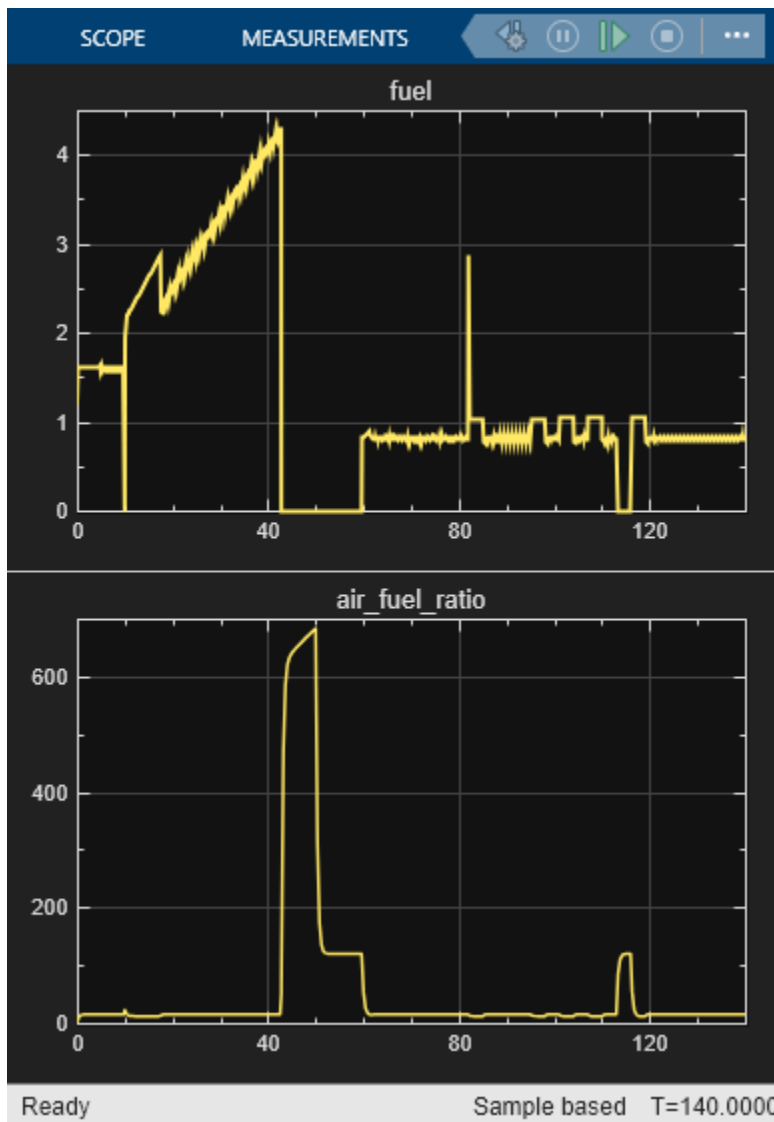
- 1 Assert that the fueling mode is in `Warmup` mode for the first 4.8 seconds
- 2 Assert that fueling mode switches to `Overspeed` mode when the actual speed exceeds 628
- 3 Assert that the fueling mode is not in `Single_Failure` mode when multiple sensors have failed.

```
open_system('fuel_rate_control_cgv/Assess_Controller');
```

Run a Simulation

Simulate the test harness and observe the `fuel_rate` and `air-fuel` ratio signals in the scope.

```
sim('fuel_rate_control_cgv')
```



Notice that no assertions trigger during the simulation, which indicates that all assessments modeled in Assess Controller pass.

Configure an Equivalence Test in the Test Manager

As part of code generation verification (CGV) for the controller system, it is important to assert that the functional behavior of the controller is the same during normal and software-in-the-loop (SIL) simulation modes. Use the Test Manager to perform this verification.

Open the Test Manager and test file.

```
sltest.testmanager.view
sltest.testmanager.TestFile('sltestFuelRateControlComparisonTestSuite.mldatx');
```

Model the Test Case

The equivalence test has been configured in the Test Manager so that the controller is simulated in normal and SIL modes and the numerical results are compared between these two runs. Explore the structure of the test case by clicking on different nodes of the test hierarchy in the **Test Browser**.

Run the Test Case

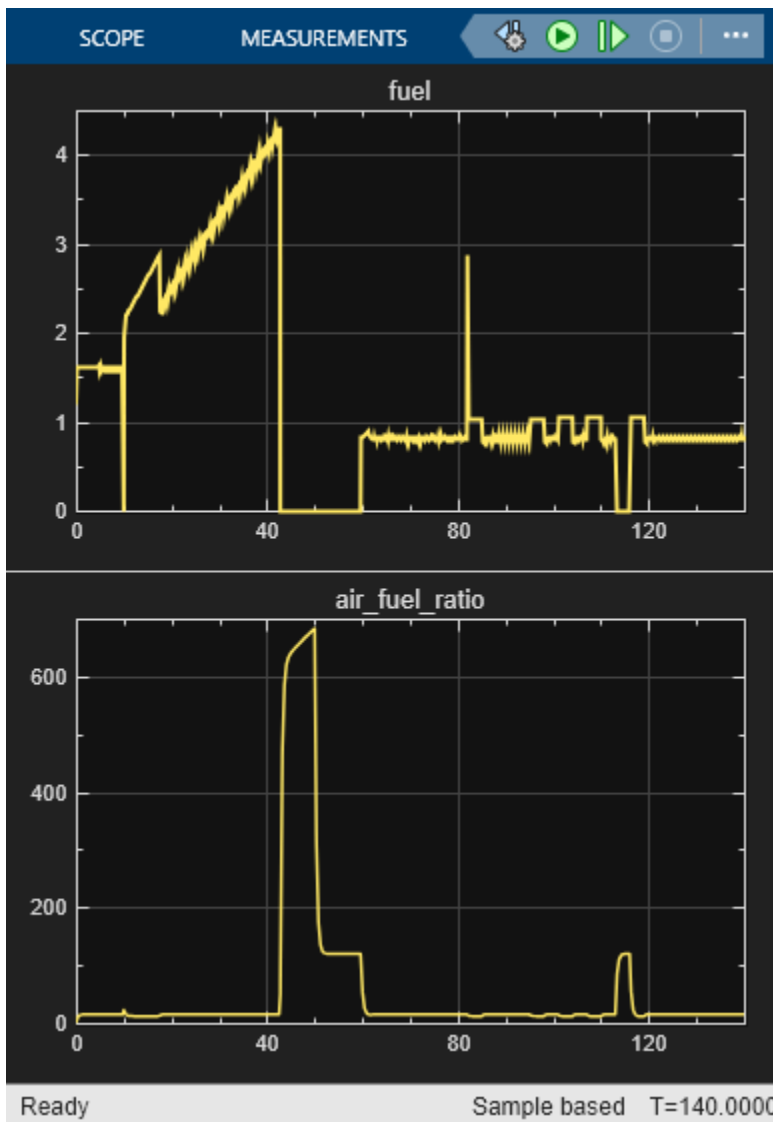
Run the test in the Test Manager. The pass/fail result is available in the **Results and Artifacts** pane of the Test Manager.

```
cgvresult = sltest.testmanager.run
```

```
cgvresult =
```

```
    ResultSet with properties:
```

```
        Outcome: Passed
        NumPassed: 1
        NumFailed: 0
        NumDisabled: 0
        NumIncomplete: 0
        NumTotal: 1
        NumTestCaseResults: 0
        NumTestSuiteResults: 0
        NumTestFileResults: 1
        StartTime: 15-Aug-2025 04:29:50
        StopTime: 15-Aug-2025 04:53:55
        Duration: 1445.7 sec
        Description: []
        Name: 'Results: 2025-Aug-15 04:29:49'
        CoverageResults: []
        UserData: []
```

Create the Report

Generate a report to view the result of the equivalence test.

```
sltest.testmanager.report(cgvresult,'cgvresult.zip','IncludeTestResults',int32(0));
```

Clear and Close the Test Manager and Close the Model

```
sltest.testmanager.clear
sltest.testmanager.clearResults
sltest.testmanager.close
close_system mdl,0;
clear mdl;
```

See Also

`sltest.testmanager.report`

Import Test Cases for Equivalence Testing

You can use the SIL/PIL Manager app in Embedded Coder to export test cases to the Test Manager. By using the app to export software-in-the-loop (SIL) or processor-in-the-loop (PIL) test cases, you do not have to write complicated test scripts for back-to-back testing.

Note You need both Simulink Test and Embedded Coder to use this feature.

Using **Export to Test Manager** in the SIL/PIL Manager app in **Automated Verification** mode exports a test case with two simulations, each in a different simulation mode. For back-to-back testing, you usually use Normal mode and SIL mode or Normal mode and PIL mode. When you export from the app, the Test Manager opens with the new equivalence test case in the Test Browser pane. If you export to a new test file, the Test Browser opens with a new test file and a new test suite for the test case. The test case includes a panel for each simulation (**SIMULATION 1** and **SIMULATION 2**). See SIL/PIL Manager (Embedded Coder) and “SIL/PIL Manager Verification Workflow” (Embedded Coder) for information on how to use the app to export a test case.

Settings for Test Case Simulations

The **System Under Test** in the SIL/PIL Manager app determines the settings for the test case simulations in the Test Manager. These settings for each type of system under test are described for exporting a test case that includes a SIL mode simulation. For a test that includes a PIL mode simulation, the settings are the same for each type of system under test.

- “Top-Level Model” on page 5-16
- “Model Block in SIL/PIL Mode” on page 5-17
- “Model Block or Reusable Library Subsystem in a Test Harness” on page 5-18

Top-Level Model

When the system under test is a **Top model**, the exported test case tests the entire model. The Test Harness field in Test Manager is blank.

Before exporting the test case, these settings are in the SIL/PIL Manager app.

System Under Test	Top model
Simulation Mode	Normal
SIL/PIL Mode	Software-in-the-Loop (SIL)

After exporting the test case, these settings are in the Test Manager for **SIMULATION 1**.

Property	Setting	Location in Test Manager
Model	Top model	SIMULATION 1 > SYSTEM UNDER TEST
Simulation mode	Normal	SIMULATION 1 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

Property	Setting	Location in Test Manager
Override model blocks in SIL/PIL to normal mode	Selected To run the simulation in Normal mode, model blocks set to SIL/PIL mode are overridden.	SIMULATION 1 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

After exporting the test case, these settings are in the Test Manager for **SIMULATION 2**.

Property	Setting	Location in Test Manager
Model	Top model	SIMULATION 2 > SYSTEM UNDER TEST
Simulation mode	Software-in-the-Loop (SIL)	SIMULATION 2 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES
Override model blocks in SIL/PIL to normal mode	Not selected The model blocks set to SIL or PIL mode run in SIL or PIL mode, respectively.	SIMULATION 2 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

Model Block in SIL/PIL Mode

When the system under test is **Model blocks in SIL/PIL mode**, the exported test case is a Model block in SIL or PIL simulation mode. The Test Harness field in Test Manager is blank.

Before exporting the test case, these settings are in the SIL/PIL Manager app.

System Under Test	Model blocks in SIL/PIL mode
Top Model Mode	Normal

After exporting the test case, these settings are in the Test Manager for **SIMULATION 1**.

Property	Setting	Location in Test Manager
Model	Top model	SIMULATION 1 > SYSTEM UNDER TEST
Simulation mode	Normal	SIMULATION 1 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES
Override model blocks in SIL/PIL to normal mode	Selected To run the simulation in Normal mode, model blocks set to SIL/PIL mode are overridden.	SIMULATION 1 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

After exporting the test case, these settings are in the Test Manager for **SIMULATION 2**.

Property	Setting	Location in Test Manager
Model	Top model	SIMULATION 2 > SYSTEM UNDER TEST
Simulation mode	Normal The system under test runs in SIL or PIL mode as set in the SIL/PIL Manager app. Other blocks run in Normal mode.	SIMULATION 2 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES
Override model blocks in SIL/PIL to normal mode	Not selected The model blocks set to SIL or PIL mode run in SIL or PIL mode, respectively.	SIMULATION 2 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

Model Block or Reusable Library Subsystem in a Test Harness

When the system under test is a Model block or a reusable library subsystem in a test harness, the exported test case is that block or subsystem in SIL or PIL simulation mode. Use the SIL/PIL Manager app from within the test harness.

Before exporting the test case, these settings are in the SIL/PIL Manager app.

System Under Test	Name of Model block or reusable library subsystem in the test harness. This field is not editable because you cannot change an entire harness to SIL/PIL mode.
Simulation Mode	Normal
SIL/PIL Mode	Software-in-the-Loop (SIL)

After exporting the test case, these settings are in the Test Manager for **SIMULATION 1**.

Property	Setting	Location in Test Manager
Model	Model block name	SIMULATION 1 > SYSTEM UNDER TEST
Harness	Harness name	SIMULATION 1 > SYSTEM UNDER TEST > TEST HARNESS
Simulation mode	Normal	SIMULATION 1 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES
Override model blocks in SIL/PIL to normal mode	Selected To run the simulation in Normal mode, Model blocks set to SIL/PIL mode are overridden.	SIMULATION 1 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

After exporting the test case, these settings are in the Test Manager for **SIMULATION 2**.

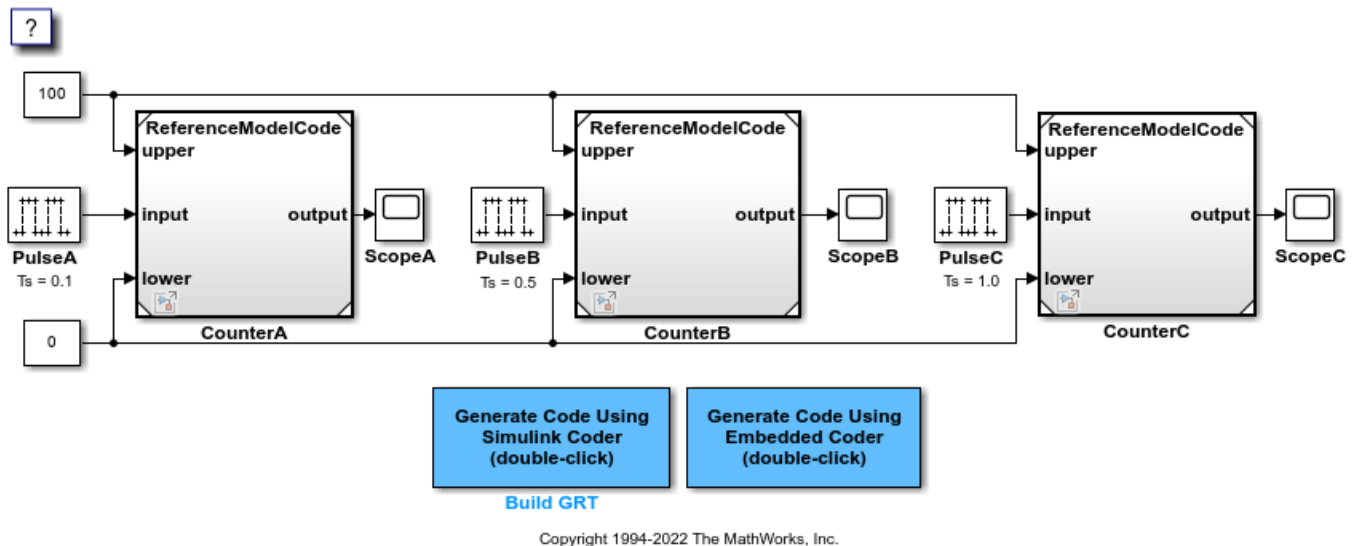
Property	Setting	Location in Test Manager
Model	Model block name	SIMULATION 2 > SYSTEM UNDER TEST
Harness	Harness name	SIMULATION 2 > SYSTEM UNDER TEST > TEST HARNESS
Simulation mode	Software-in-the-Loop (SIL)	SIMULATION 2 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES
Override model blocks in SIL/PIL to normal mode	Not selected The Model blocks set to SIL or PIL mode run in SIL or PIL mode, respectively.	SIMULATION 2 > SYSTEM UNDER TEST > SIMULATION SETTING OVERRIDES

Back-to-Back Testing a Model Using the SIL/PIL Manager App

This example shows how to perform back-to-back testing with a test case exported from the Embedded Coder SIL/PIL Manager app. The test case compares a model simulated in Normal mode and in Software-in-the-Loop (SIL) mode.

- 1 Open the TopModelCode model. At the command line, enter:

```
openExample('simulinkcoder/FilePackagingModelsCodeAndDataExample', ...
'supportingFile', 'TopModelCode');
```



Note Steps 2 through 4 apply specifically to this TopModelCode model. These steps might not be needed for other models.

- 2 For this model, click **Signal Table** in the Simulation tab. Select **Test Point** and **Log data** for the CounterA, CounterB, and CounterC signals.

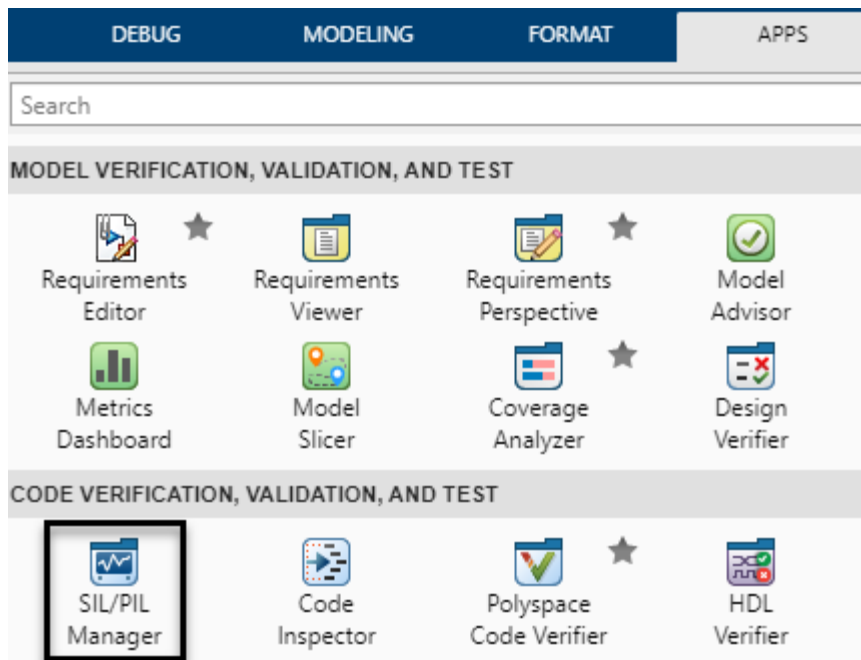
Model Data Editor			
Inports/Outputs Signals Data Stores States Parameters			
Instrumentation			
Source	Name	Test Point	Log Data
CounterA		☒	☒
CounterB		☒	☒
CounterC		☒	☒
Pulse (Ts=0.1)		☐	☐

- Right-click in the model and select **Model Configuration Parameters**. In the Configuration Parameters dialog box,

- In **Data Import/Export**, set the **Format** to Dataset.
- In **Code Generation > Interface**, select **signals** in the **Generate C API for** section.

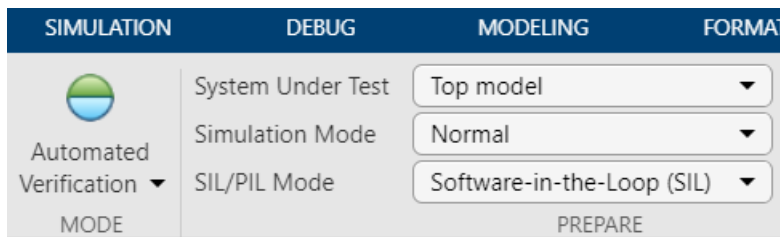
Click **OK**.

- Right-click on the Model blocks and select **Open as Top Model**. In the Configuration Parameters dialog box, set the same items as in Step 3.
- Expand the Apps tab in the model window and click **SIL/PIL Manager** under Code Verification, Validation, and Test.

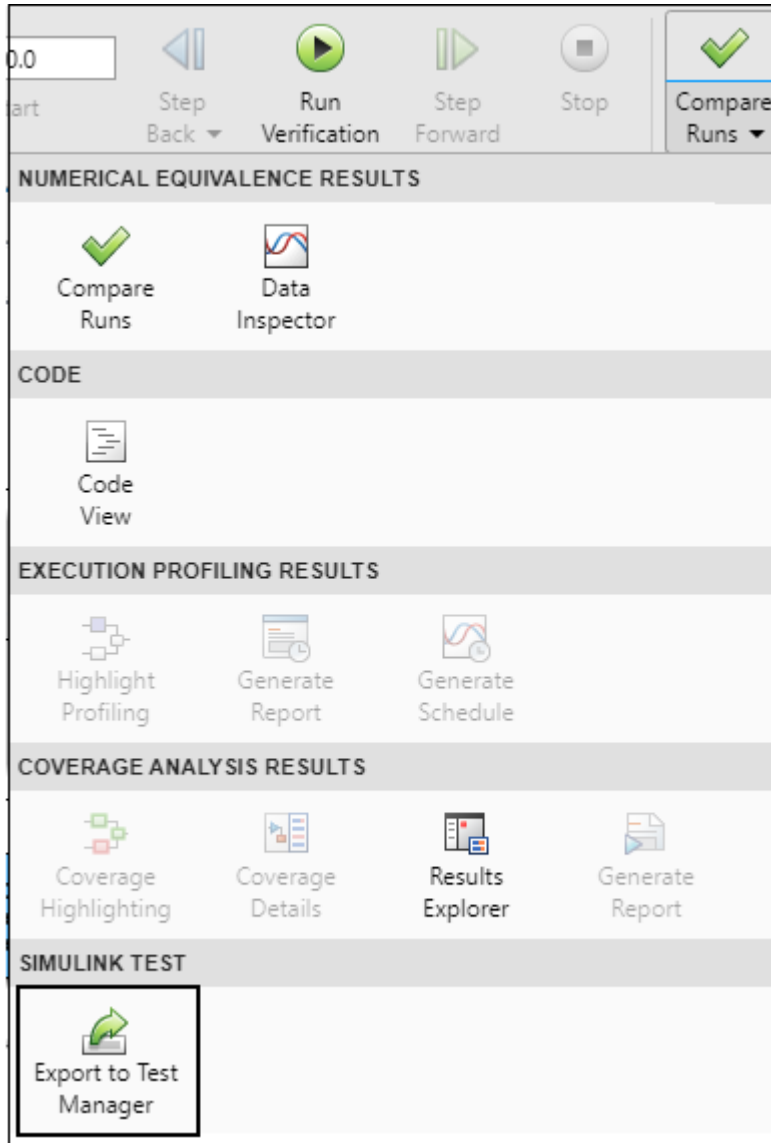


- In the SIL/PIL Manager toolstrip, if they are not already selected, select

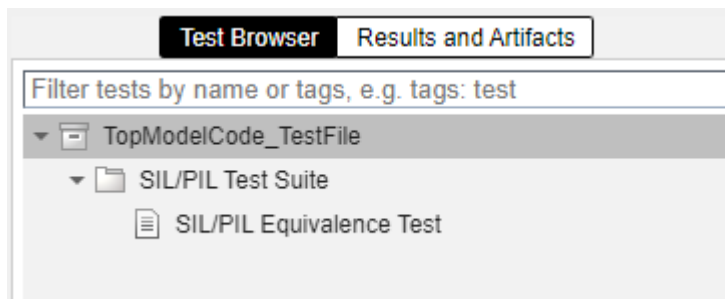
- Automated Verification**
- System Under Test** — Top Model
- Simulation Mode** — Normal
- SIL/PIL Mode** — Software-in-the-Loop (SIL)



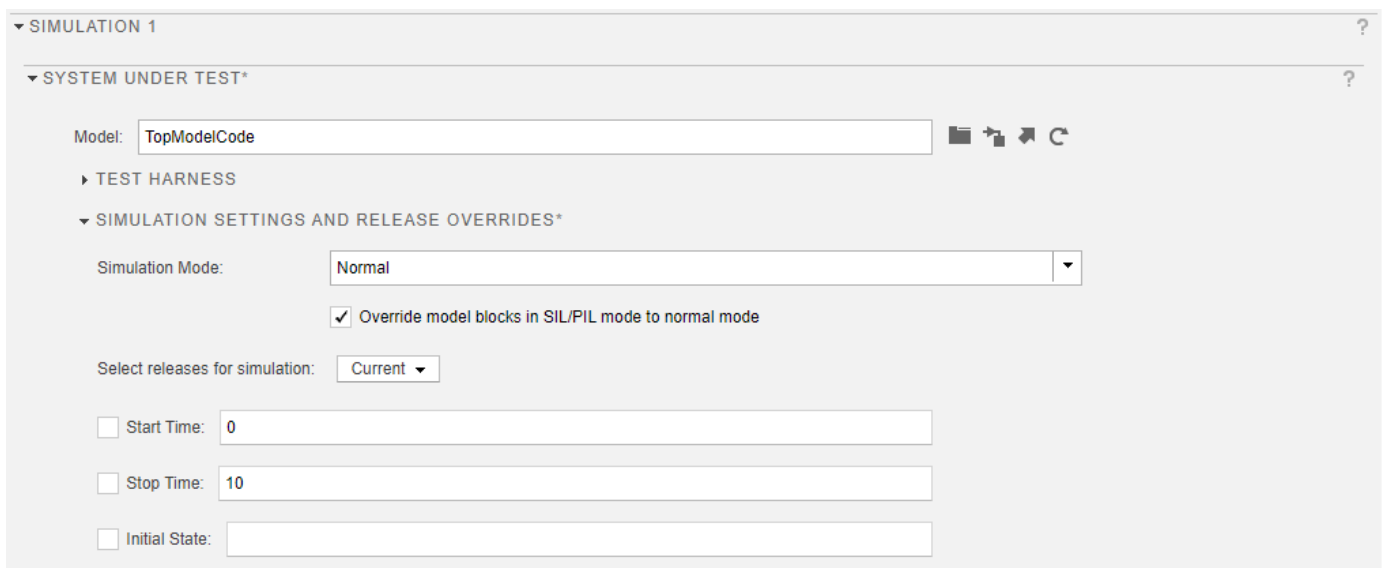
- 7 To export the test case, expand **Compare Runs** and click **Export to Test Manager**.



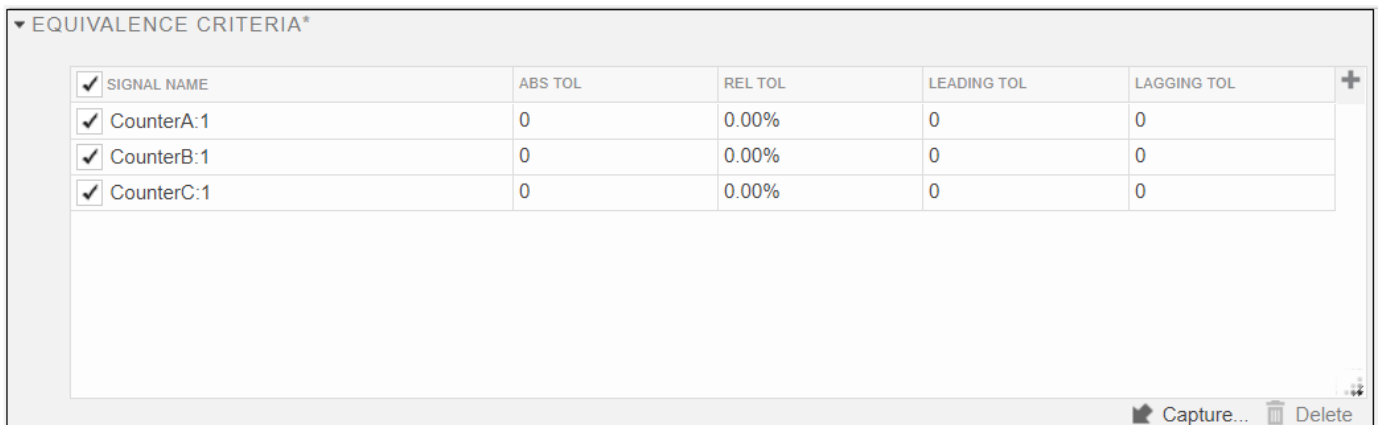
- 8 In the Export SIL/PIL Test Cases dialog box, use the default values and click OK. The Test Manager opens.
- 9 In the Test Manager, to see the imported test case and settings, expand TopModelCode_TestFile and SIL/PIL Test Suite in the Test Browser.



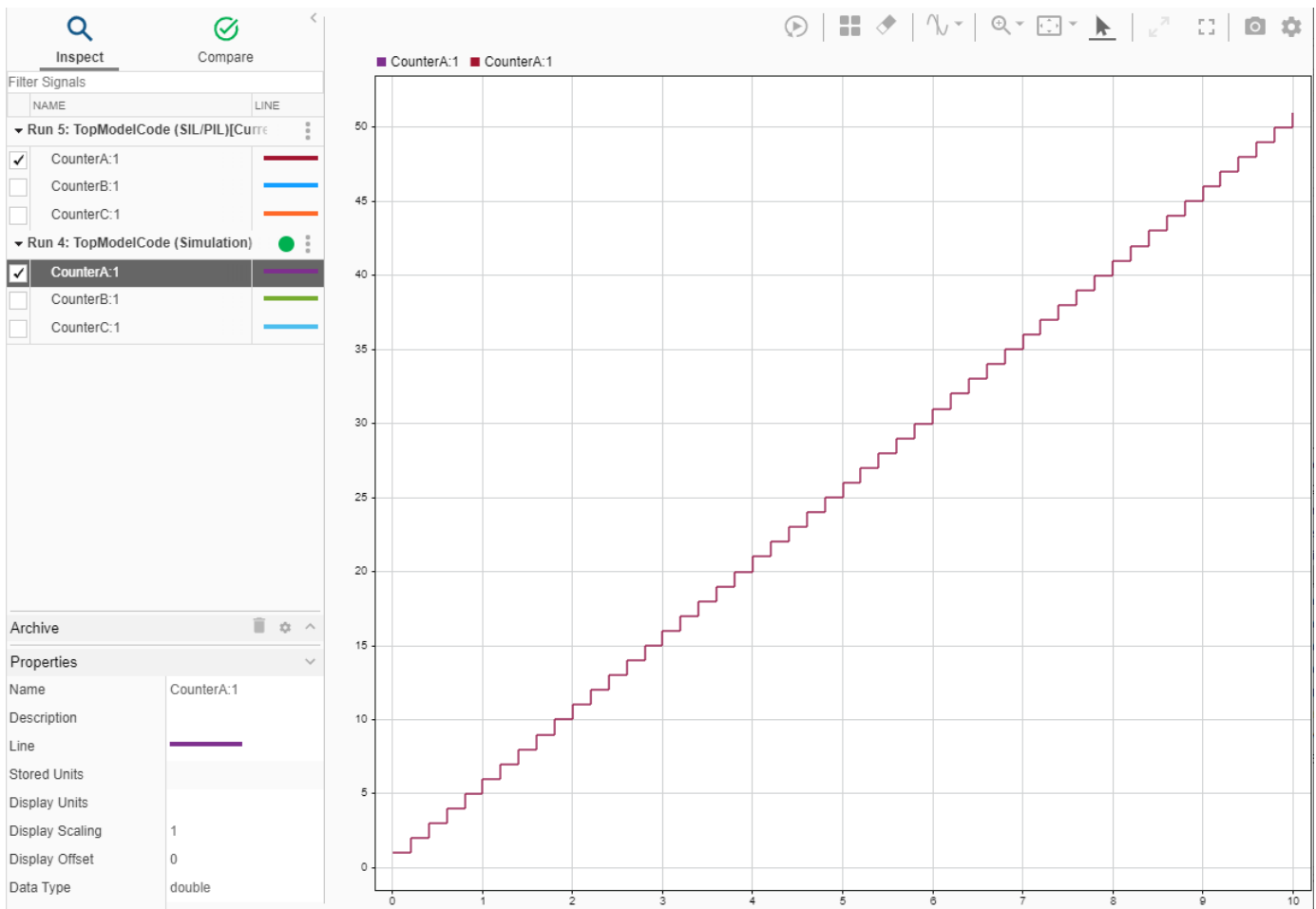
- 10 Select the SIL/PIL Equivalence test case. To see the settings for the simulation modes, expand the **SIMULATION 1** and **SIMULATION 2** sections. The expanded **SIMULATION 1** section is



- 11 Open the **Equivalence Criteria** section and click **Capture**. The model simulates and the section lists the signals to compare in the test case.



- 12 Click **Run** to run the test case.
- 13 In the Test Manager, the Results and Artifacts panel shows the pass/fail results. A Code Generation Report opens in a separate window.
- 14 Select one or more signals to plot the results.



The plot shows that the outputs from the two simulations are the same.

See Also

More About

- [SIL/PIL Manager \(Embedded Coder\)](#)
- [“SIL/PIL Manager Verification Workflow” \(Embedded Coder\)](#)
- [“Code Verification Through Software-in-the-Loop and Processor-in-the-Loop Execution” \(Embedded Coder\)](#)

Test Integrated Code

In this section...
"Test Integrated C Code" on page 5-24
"Testing Code in an S-Function Block" on page 5-24

Test Integrated C Code

If you have a model that integrates C code with a C Caller block, you can test the C code with the Test Manager and a test harness.

The C Caller block uses configuration parameters to define the custom code. If you change the configuration parameters, synchronize the parameters between the test harness and the model. For more information, see "Synchronize Changes Between Test Harness and Model" on page 2-52 and "Create or Import Test Harnesses and Select Properties" on page 2-13.

- If you change the test harness configuration parameters, you can push the configuration set to the main model. Click **Push Changes**, or use `sltest.harness.push`.
- If you change the main model configuration parameters in the main model, and you want to update the test harness parameters, the test harness must copy the configuration parameters on rebuild. You can set this property in two ways:
 - When you create the test harness, select **Update Configuration Parameters and Model Workspace data on rebuild**. You can also select **Rebuild Harness on Open**, which rebuilds every time the harness opens.
 - For existing test harnesses, in the harness preview, select one or more of **Rebuild Harness > Rebuild on Open**, **Rebuild Harness without Compiling Model**, and **Update Harness Configuration Settings and Model Workspace**. The **Update Harness Configuration Settings and Model Workspace** option updates the settings every time a rebuild occurs.

Testing Code in an S-Function Block

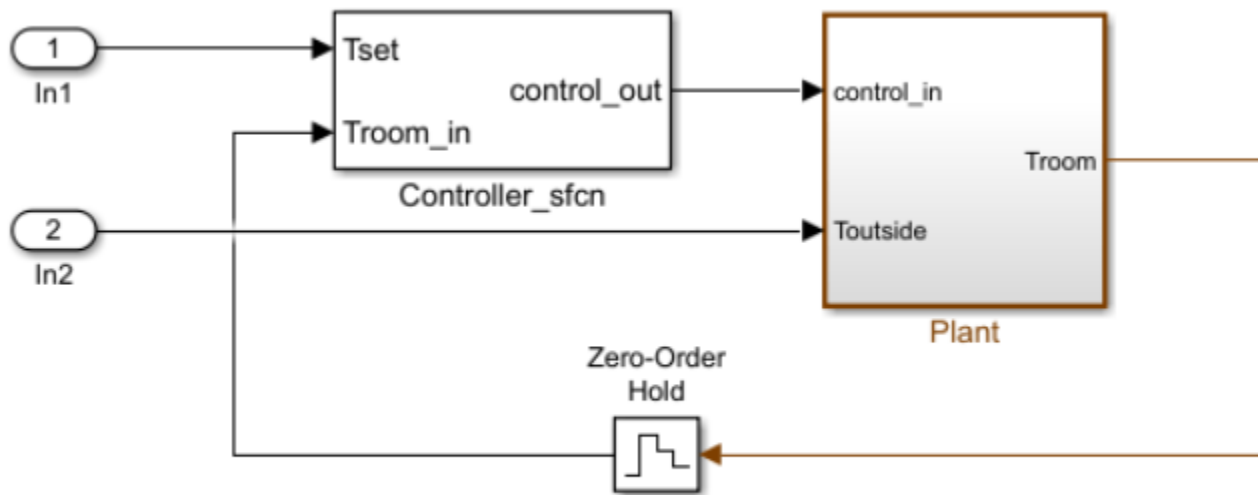
This example shows how to use a test harness to test code in an S-Function block. The main model for this example is a controller-plant model of an air conditioning/heat pump unit.

Note that this example works only on a 64-bit Windows platform.

S-Functions are computer language descriptions of Simulink blocks written in MATLAB, C, C++ or Fortran. You can test code wrapped in S-Functions by using test harnesses. Testing code in S-Functions can be helpful for regression testing of legacy code and for testing your code in a system context.

Open the Model

```
sltestHeatpumpSfunExample
```



Copyright 1990-2018 The MathWorks, Inc.

In the example model:

- The controller is an S-Function that accepts room temperature and specified temperature inputs.
- The controller output is a bus with signals that control the fan, heat pump, and the direction of the heat pump (heat or cool).
- The plant accepts the control bus. The heat pump and the fan signals are Boolean, and the heat pump direction is specified by +1 for cooling and -1 for heating.

Temperature Condition States

The test covers four temperature conditions. Each condition corresponds to one operating state with fan, pump, and pump direction signal outputs.

Temperature Condition	System State	Fan Command	Pump Command	Pump Direction
$ T_{room_in} - T_{set} < \Delta T_{fan}$	idle	0	0	0
$\Delta T_{fan} \leq T_{room_in} - T_{set} < \Delta T_{pump}$	fan only	1	0	0
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} < T_{room_in}$	cooling	1	1	-1
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} > T_{room_in}$	heating	1	1	1

Create Test Case

1. On the **Apps** tab, under **Model Verification, Validation, and Test**, click **Simulink Test**. Then, on the **Tests** tab, click **Simulink Test Manager**.
2. From the Test Manager toolstrip, click **New > Test File** to create a test file. Name and save the test file.

3. In the test case, under System Under Test, click the  button to load the current model into the test case.

Create Test Harness

1. In the model, right-click the Controller_sfcn subsystem and select **Test Harness > Create for 'Controller_sfcn'**.
2. Set the harness properties. In the **Basic Properties** tab, set the test harness properties. Set **Name** to test_harness_1 and Set **Sources** and **Sinks** to **None** and **Scope**.
3. Click **OK** to create the test harness.
4. In the Test Manager highlight **New Test Case 1**. In **System Under Test**, expand **Test Harness**. Refresh the test harness list and select test_harness_1 for the **Harness**.

Add Inputs and Set Simulation Parameters

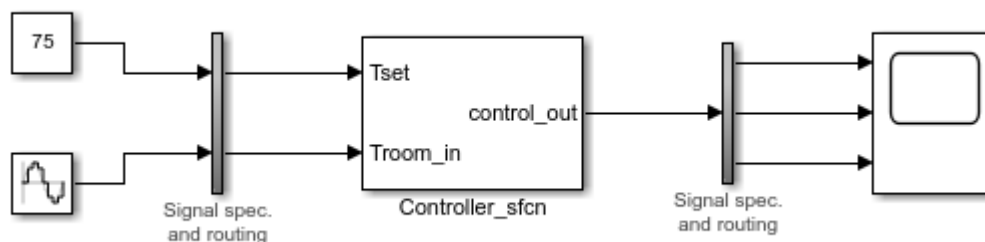
Create inputs in the test harness, with a constant Tset and a time-varying Troom_in.

1. In the test harness model, connect a Constant block to the Tset input and set the value to 75.
2. Add a Sine Wave block to the harness model to simulate a temperature signal. Connect the Sine Wave block to the conversion subsystem input Troom_in.
3. Double-click the Sine Wave block and set the parameters:

Parameter	Value
Amplitude	15
Bias	75
Frequency	$2\pi/3600$
Phase (rad)	0
Sample time	1

Select **Interpret vector parameters as 1-D**. Then, click **OK**.

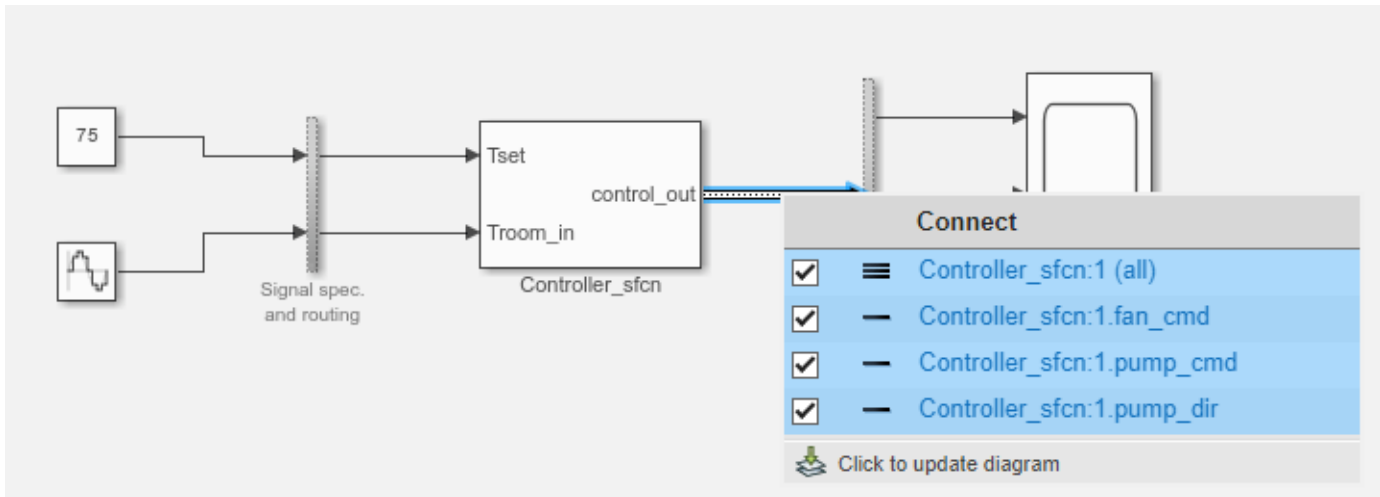
The resulting harness is:



4. In the Simulation toolstrip, set the **Stop Time** to 3600.

Obtain Baseline Data

1. In the Test Manager test case, in **Simulation Outputs**, click **Add**. Highlight the output bus from the controller S-Function.



2. In the Connect popup, select **Controller_sfcn:1(all)** and click **Click to update diagram**. Then, return to the Test Manager and click **Done** in the dialog box.

3. Under **Baseline Criteria**, click **Capture** to record a baseline data set from simulating the test harness. In the Capture Baseline dialog box, enter **sfcn_baseline** as the MAT format **File** name and click **Capture**. The baseline signals appear in the table.

▼ SIMULATION OUTPUTS* ?

LOGGED SIGNALS

NAME	BLOCK PATH	PORT INDEX
▼ ☒ Signal Set 1		
☒ Controller_sfcn:1	test_harness_1/Controller_sfcn	1

☐ Plot signals on the specified plots after simulation + Add ▼ Delete

OTHER OUTPUTS

☐ Override model settings

☐ States ☐ Output

☐ Final states ☒ Data stores ☒ Signal logging

► CONFIGURATION SETTINGS OVERRIDES ?

▼ BASELINE CRITERIA* ?

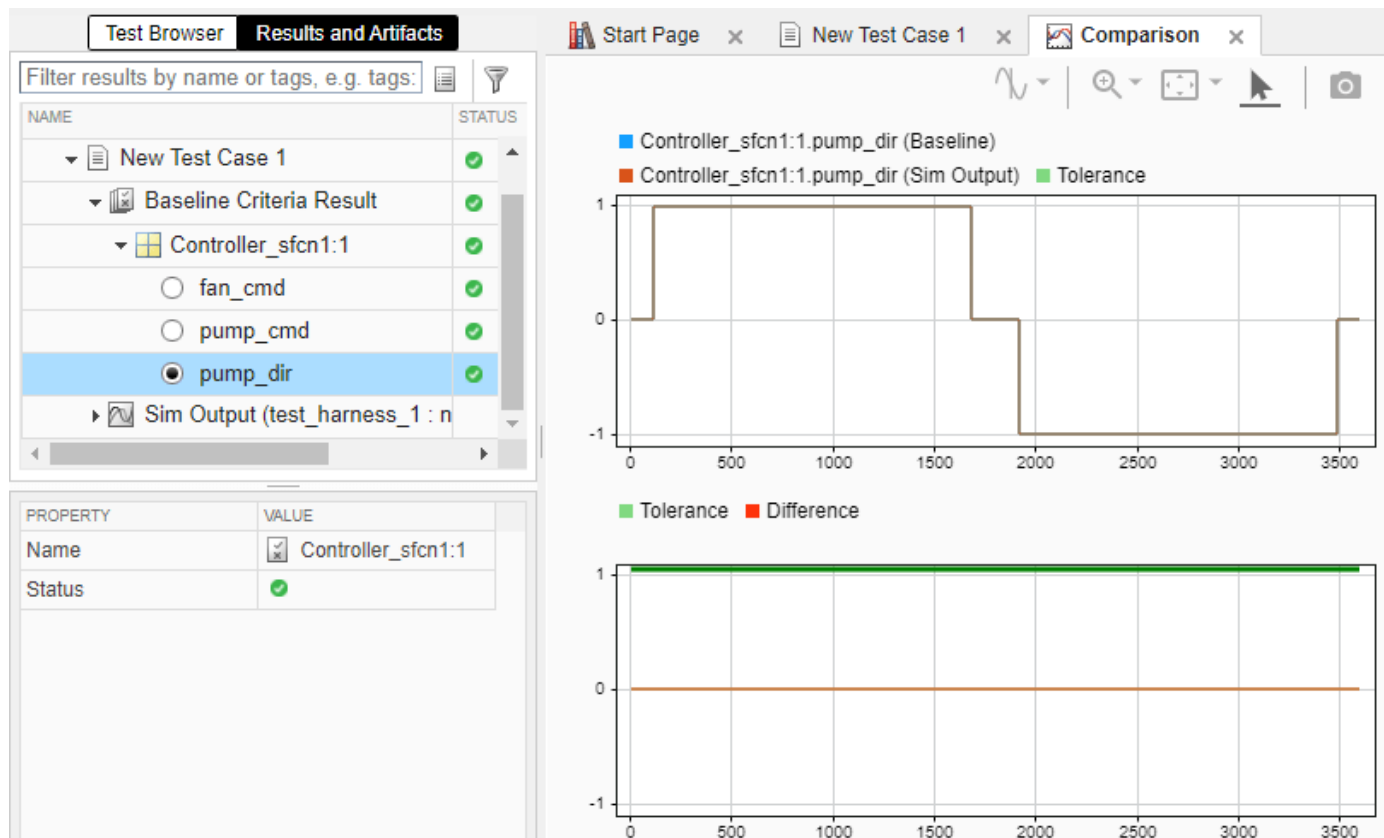
☐ Include baseline data in test result

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL	
▼ ☒ sfcn_baseline.mat	0	0.00%	0	0	+
☒ fan_cmd	0	0.00%	0	0	
☒ pump_cmd	0	0.00%	0	0	
☒ pump_dir	0	0.00%	0	0	

+ Add... Capture... Edit... Refresh Visualize Delete

Run Test Case and View Results

1. Run the test case. The test results appear in the **Results and Artifacts** pane.
2. Expand the results to view the baseline criteria result. The baseline test passes because the simulation output is identical to the baseline data.



See Also

sltest.CodeImporter

Related Examples

- “Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard” on page 10-11
- “Import Custom Code for Unit Testing Using API Commands” on page 10-5

Test Manager Test Cases

- “Manage Test File Dependencies” on page 6-3
- “Compare and Merge Test Files” on page 6-6
- “Create Multiple Baseline Tests in Batch Mode” on page 6-12
- “Test a Simulation for Run-Time Errors” on page 6-14
- “Automatically Create a Set of Test Cases” on page 6-17
- “Generate Tests and Test Harnesses for a Model or Components” on page 6-20
- “Override Model Parameters in a Test Case” on page 6-27
- “Override the Simulation Mode During Test Execution” on page 6-32
- “Back-to-Back Equivalence Testing” on page 6-36
- “Back-to-Back (MIL/SIL) Equivalence Testing an Atomic Subsystem” on page 6-50
- “Testing AUTOSAR Compositions” on page 6-59
- “Automate Testing for Highway Lane Following” on page 6-64
- “Synchronize Tests” on page 6-75
- “Use External File Data in Test Cases” on page 6-76
- “Create External Data Files to Use in Test Cases” on page 6-84
- “Capture Simulation Data in a Test Case” on page 6-93
- “Run Tests in Multiple Releases of MATLAB” on page 6-102
- “Examine Test Failures and Modify Baselines” on page 6-110
- “Create and Run Test Cases with Scripts” on page 6-115
- “Test Models Using MATLAB-Based Simulink Tests” on page 6-119
- “Using MATLAB-Based Simulink Tests in the Test Manager” on page 6-124
- “Collect Coverage Using MATLAB-Based Simulink Tests” on page 6-128
- “Create and Run a RoadRunner Test Case” on page 6-133
- “Test Iterations” on page 6-139
- “Capture Baseline Data from Iterations” on page 6-148
- “Collect Coverage in Tests” on page 6-151
- “Test Coverage for Requirements-Based Testing” on page 6-158
- “Increase Test Coverage for a Model” on page 6-163
- “Run Tests Using Parallel Execution” on page 6-167
- “Set Signal Tolerances” on page 6-169
- “Specify Test Properties in the Test Manager” on page 6-173
- “Preferences” on page 6-191
- “Increase Coverage by Generating Test Inputs” on page 6-192
- “Process Test Results with Custom Scripts” on page 6-197
- “Assess the Damping Ratio of a Flutter Suppression System” on page 6-203

- “Create, Store, and Open MATLAB Figures” on page 6-206
- “Test Models Using MATLAB Unit Test” on page 6-209
- “Output Results for Continuous Integration Systems” on page 6-212
- “Parametric Sweep for a Simscape Thermal Model” on page 6-216
- “Projector Controller Testing Using verify and Real-Time Tests” on page 6-222
- “Test Execution Order” on page 6-227
- “Filter Test Execution, Results, and Coverage” on page 6-231
- “Write and Run Tests that Inject Faults by Using the Simulink Test Manager” on page 6-232
- “Baseline Testing” on page 6-240
- “Author a Baseline Test Case” on page 6-242
- “Compare Simulation to Baseline Data” on page 6-246
- “Create a Test Results Report” on page 6-249

Manage Test File Dependencies

In this section...

“Package a Test File Using Projects” on page 6-3

“Find Test File Dependencies and Impact” on page 6-4

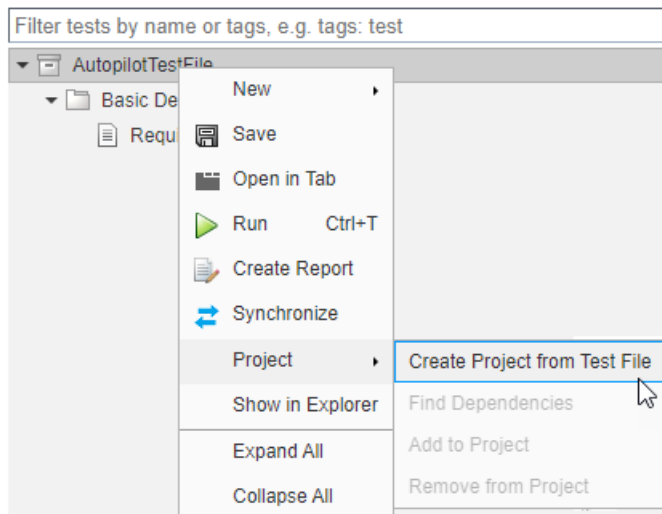
“Share a Test File with Dependencies” on page 6-5

You can help track and manage your test file dependencies by creating a project for your test file and the files it depends on. Examples of test file dependencies include requirements, data files, callbacks, test harnesses, and custom criteria scripts. Packaging test file dependencies in a project also helps you share tests with other users.

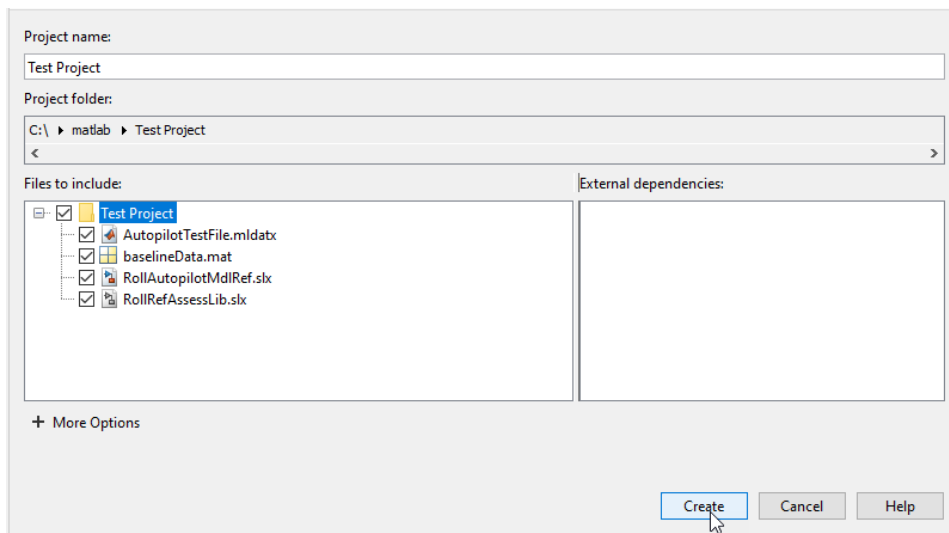
Package a Test File Using Projects

- 1 In the **Test Browser**, right-click the test file.
- 2 Select **Project > Create Project from Test File**.

Project opens and identifies the file dependencies of the test file. In this example, the test file contains a baseline test case that uses a baseline data file.



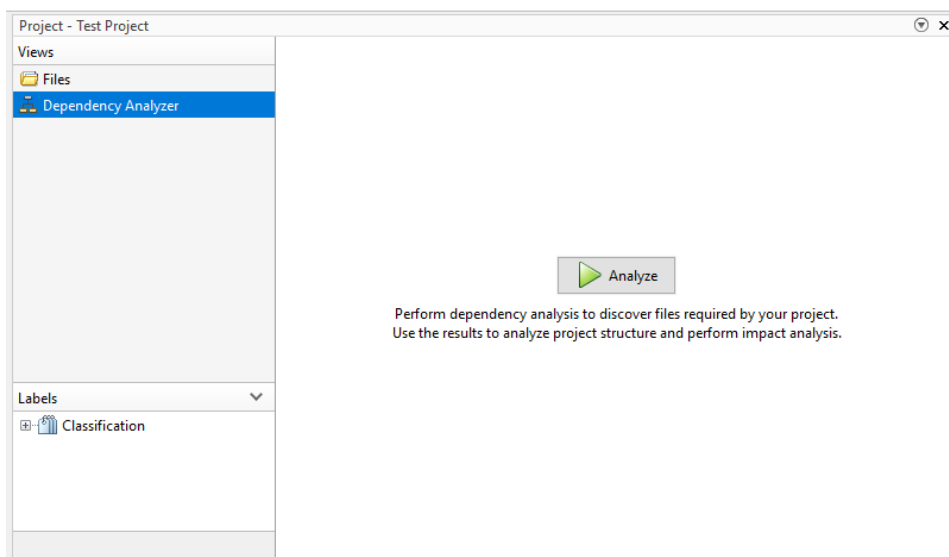
- 3 Specify project name, and verify the list of selected file dependencies.
- 4 Click **Create**.



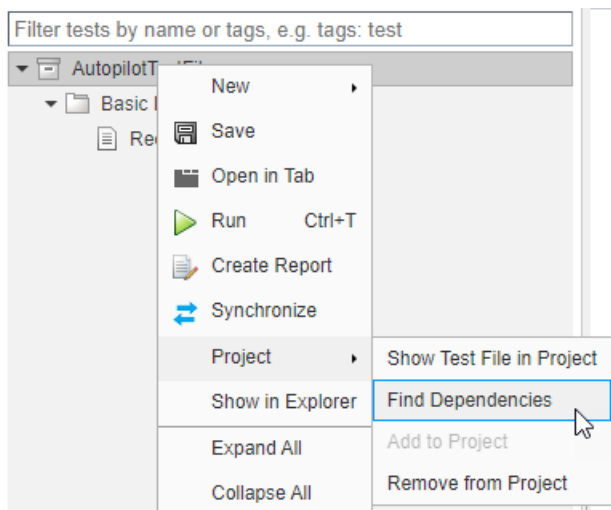
Find Test File Dependencies and Impact

You can find test file dependencies from the project or from the Test Manager. Your test file must be saved in a project.

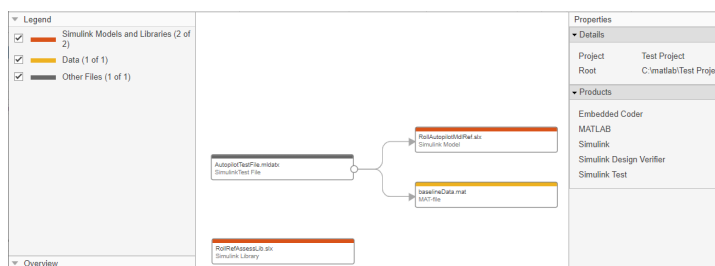
- 1 From the project, click **Analyze**



or from the Test Manager, right-click the test file. Select **Project > Find Dependencies**.



Dependencies are color coded in the file dependency graph.



If you want to change a model or requirement, you can determine the potential impact of the change on your tests.

- 1 In the dependency graph, select the item that could impact your tests.
- 2 In the Dependency Analyzer toolstrip, in the **Impact Analysis** section, click **Impacted**.

If you want to run a test file again, double-click the test file in the graph to open the Test Manager. In the Test Manager, click **Run**.

Share a Test File with Dependencies

You can easily share test files that are already saved in a project. If you send the project folder, it contains the file dependencies for the test file.

See Also

Related Examples

- “What Are MATLAB Projects in Simulink?”

Compare and Merge Test Files

This example shows how to compare and merge two MLDATX test files by using the Comparison Tool. If you have two tests of the same type, such as two baseline tests or two equivalence tests, you can merge the test cases into a target file. You cannot merge test cases of different test types, but you can add them to the target file as separate test cases.

Compare Test Files

Open the Comparison Tool.

```
visdiff("TestFile_1.mldatx","TestFile_2.mldatx")
```

Left : TestFile_1.mldatx		Right : TestFile_2.mldatx	
▼ TestFile_1 Test File Options ▼ New Test Suite 1 Baseline Test Case ▶ Equivalence Test Case		▼ TestFile_2 Test File Options ▼ New Test Suite 1 Baseline Test Case ▶ Simulation Test Case	
Parameter	Value	Parameter	Value
Tags		Tags	#customer_A_tests
Description	Test file has 2 different types of ...	Description	

The tool highlights the differences between the two files using these colors:

- Purple -- The content differs between the two files.
- Blue -- The content exists only in the file in the right pane.
- Yellow -- The content exists only in the file in the left pane.
- No highlight -- The content in both files is the same.

To generate a comparison report, select **Publish > Publish to HTML**, **Publish > Publish to Word**, or **Publish > Publish to PDF**. This image shows a sample of the report.

File Comparison Report

C:\MyTestFiles\TestFile_1.mldatx vs. C:\MyTestFiles\TestFile_2.mldatx
dschwart
23-Jun-2023

Files

	Left File	Right File
File name	TestFile_1	TestFile_2
File path	C:\MyTestFiles	C:\MyTestFiles
Last modified	23-Jun-2023 15:29:34	23-Jun-2023 15:30:38

Environment

MATLAB 23.2 (R2023b)

Comparison Results

TestFile_1	TestFile_2
Tags : Description : Test file has 2 different types of test cases.	Tags : #customer_A_tests Description :

Merge Test Files

1. If the Comparison Tool is not open, enter
`visdiff("TestFile_1.mldatx","TestFile_2.mldatx").`
2. Click **Merge Mode**. The Target File pane opens below the two file panes. The Comparison Tool populates the Target file with the content from the right pane file. The columns on the right pane contain one, two, or no radio buttons for each node or parameter in the target file. For nodes that can be merged, the radio buttons for the right pane file are selected by default.

The Target pane uses these colors:

- Blue -- The content from the right pane is selected.
- Yellow -- The content from the left pane is selected.
- No highlight -- The content in both files is the same.

The screenshot shows a 'Target' window with a tree view on the left and a table on the right.

Tree View:

- TestFile_2
 - Test File Options
 - New Test Suite 1
 - Baseline Test Case
 - Simulation Test Case (selected)
 - Equivalence Test Case

Table:

Name	Value	Y	Y
Tags	#customer_A_tests	☐	☒
Description		☐	☒

When you enter merge mode, the radio buttons indicate the actions you can take for the nodes you want to merge:

- No radio buttons — For nodes, you cannot merge when the content is the same in both files. For a parameter, you cannot merge if the parameter is read only.
- One radio button — The node is the same in the target file and in the file associated with the column of the radio button. This situation occurs when the corresponding node in the other file is not valid, so you cannot merge the content.
- Two radio buttons and one is selected — The node in the target file is the same as in the file associated with the column of the selected radio button. Depending on the type of content, selecting the other radio button either adds the node content or replaces it in the target file.

3. Select the radio button in the column for the file from which you want to merge, add, or delete the content. Nodes that have no radio buttons might have nodes at a lower hierarchy level that can be merged. For those nodes, select the upper-level node to see its hierarchy.

The validity of merged nodes and parameters is not evaluated until test runtime.

Use the Description radio buttons to select which description to use for the new test file. For this example, select the left radio button to use the description from the left file.

Name	Value	Y	Y
Tags	#customer_A_tests	☐	☒
Description	Test file has 2 different types of test cases.	☒	☐

Use the Equivalence Test Case radio buttons to select which test case to add to the new test file. For this example, select the left radio button to add the Equivalence Test Case from the left file.

Target

- TestFile_2
 - Test File Options
 - New Test Suite 1
 - Baseline Test Case
 - Simulation Test Case** (Selected with right radio button)
 - Equivalence Test Case

Name	Value	Y	Y
Test case type	Equivalence Test	☒	☐
Enabled	true	☒	☐
Tags		☒	☐
Description		☒	☐

The Simulation Test Case exists only in the right file and the target file. To delete the Simulation Test Case from the target file, select the left radio button.

Target

- TestFile_2
 - Test File Options
 - New Test Suite 1
 - Baseline Test Case
 - Simulation Test Case (Selected with left radio button)
 - Equivalence Test Case

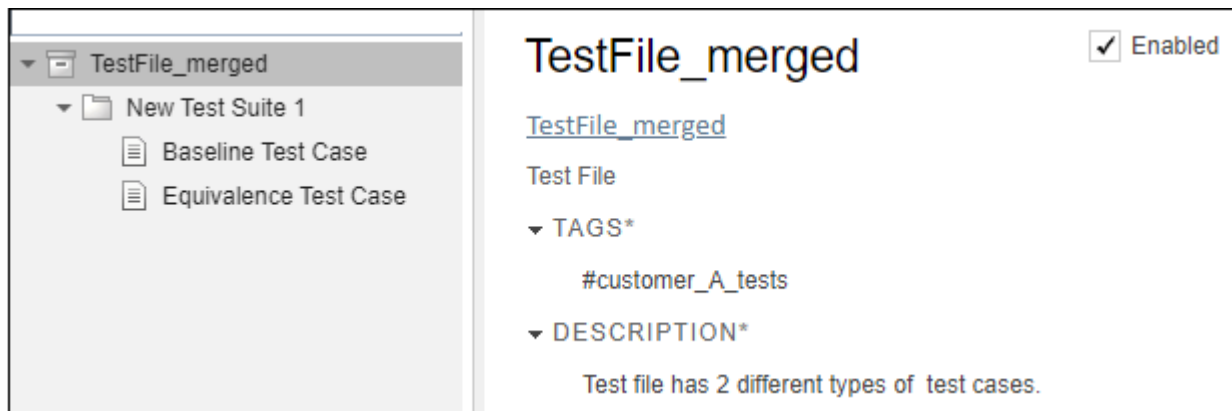
Name	Value	Y	Y

4. Click **Save Result** to save the merged target file. Specify `TestFile_merged.mldatx` as the file name. After you save the file, merge mode closes and you return to comparison mode. The saved target file becomes the new right pane file.

5. To see the merges you saved, open the Test Manager.

```
sltest.testmanager.view
```

Select **Open > Open (.mldatx)** from the Open pulldown and open the `TestFile_merged.mldatx` file.



The **Tags** contains the text from the right-pane file. The **Description** contains the text from the left-pane file. The file includes the equivalence test case and does not include the simulation test case.

File Merging Limitations

You cannot merge:

- Test cases of different test types.
- Other incompatible test files, such as files with different metadata, even if the file content is the same.
- Files or content from files attached to test files, such as input files, baseline and equivalence criteria, and coverage filters.
- Parameters that are automatically created when the target file is created, such as the file name, last modified by, last modified date, and release information.
- Linked requirements.
- Text fields. You can select a text field from the left or the right pane, but cannot combine them.

Additionally, merging has these limitations:

- Adding or deleting a node adds or deletes, respectively, all of its child nodes.
- If you add a test case that has the same name as a test case in the target file, a number is appended to the name of the added test case.
- Test case callbacks are shown as name-argument pairs.
- Interdependent settings, such as the system under test and its test harness, update together. For example, if you replace the target system under test so it uses the one from the other file, the test harness also changes to use the harness associated with the new system under test.
- Although you can change the **Coverage to Collect** settings at the test suite and test case levels, these changes are not saved in the target file unless they are the same as the settings at the test file level.
- When saving a merged file, you must explicitly include the `.mldatx` extension in the file name.
- When using the mlAutomerger Git plugin to automerger your test files, add `*.mldatx binary merge=mlAutomerger` to your `.gitattributes` file. See “Automatically Merge Models Locally and in CI Pipeline” for information about using mlAutomerger.

See Also

visdiff

Related Examples

- [“Compare Files and Folders and Merge Files”](#)
- [“Automatically Merge Models Locally and in CI Pipeline”](#)

Create Multiple Baseline Tests in Batch Mode

This example shows how to create baseline test cases and test harnesses for multiple components in batch mode using `sltest.testmanager.createTestForComponent`. The example also includes report generation.

Load the Model

```
load_system("sltestCruiseControl")
```

Customize the Test Harness Names

To improve traceability of testing artifacts, such as test case and test harness names, customize the default names of the harnesses to be created so the names use the component name instead of the owning model name.

```
sltest.harness.setHarnessCreateDefaults...  
    ("Name", "$Component$_Harness");
```

Create the Test Cases and Test Harnesses in Batch Mode

Use the `sltest.testmanager.createTestForComponent` API to create multiple test cases and test harnesses at the same time. Specify to create a test file and the test filename, the top model, components to test, and the test type.

```
[tc,status] = ...  
    sltest.testmanager.createTestForComponent(...  
        "CreateTestFile",true,...  
        "TestFile","batchBaselines.mldatx",...  
        "TopModel","sltestCruiseControl",...  
        "Component",[...  
            "sltestCruiseControl/TargetSpeedThrottle",...  
            "sltestCruiseControlMode/disableCaseDetection",...  
            "sltestCruiseControl/DriverSwRequest",...  
            "sltestCruiseControl"],...  
        "CreateExcelFile",true);
```

Test Execution

The `createTestForComponent` API generates the test file and test harnesses in the current working directory. Save the harnesses attached to the model, and save the test file.

```
tf = sltest.testmanager.TestFile("batchBaselines.mldatx");  
tf.saveToFile;
```

Run the tests and export and save the test results. Open the test manager to view the created test cases and results plots.

```
resultSet = tf.run;  
sltest.testmanager.exportResults(resultSet,...  
    "batchBaselinesResults.mldatx");  
sltestmgr;
```

Generate a Results Report

```
sltest.testmanager.report(...  
    resultSet,"myTestResultsReport.zip",...
```

```
"IncludeCoverageResult",true,...  
"IncludeSimulationSignalPlots",true,...  
"IncludeComparisonSignalPlots",true,...  
"IncludeTestResults",0,...  
"IncludeSimulationMetadata",true);
```

Clean Up

```
bdclose({"sltestCruiseControl",...  
        "sltestDriverSwRequest",...  
        "sltestCruiseControlMode"})  
sltest.testmanager.clear  
sltest.testmanager.clearResults  
sltest.testmanager.close
```

See Also

`sltest.testmanager.createTestForComponent`

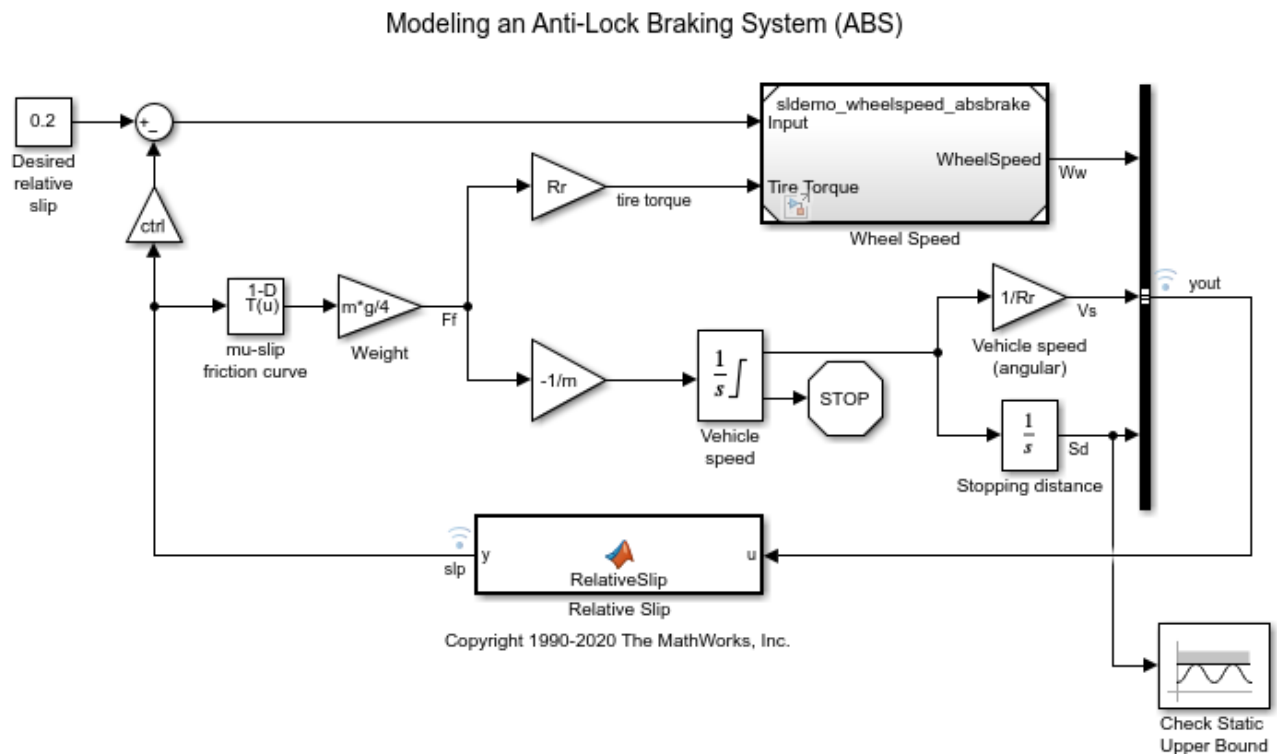
Test a Simulation for Run-Time Errors

In this example, use a simulation test case with the `sldemo_absbrake` model to test for simulation run-time errors.

Configure the Model

Configure the model to check if the stopping distance exceeds an upper bound.

- 1 To open the model, type `openExample('sldemo_absbrake')`.
- 2 Add the Check Static Upper Bound block from the Model Verification library to the model.
- 3 Connect the Check Static Upper Bound block to the `Sd` signal.



- 4 In the Check Static Upper Bound block dialog box, and set **Upper bound** to 725.

Create the Test Case

- 1 To open the Test Manager, on the **Apps** tab, under Model Verification, Validation, and Test, click **Simulink Test**. Then, on the **Tests** tab, click **Simulink Test Manager**.
- 2 To create a test file, click **New > Test File**. Name and save the test file.

The new test file consists of a test suite that contains one baseline test case. They appear in the **Test Browser** pane.

- 3 Select **New > Test Case > Simulation Test**.

- 4 Right-click the new simulation test case in the **Test Browser** pane, and select **Rename**. Rename the test case to **Upper Bound Test**.
- 5 In the test case, under **System Under Test**, click the **Use current model** button  to assign the `sldemo_absbrake` model to the test case.
- 6 Under **Parameter Overrides**, click **Add** to add a parameter set.
- 7 In the dialog box, click the **Refresh** button  to update the model parameter list.
- 8 Select the check box next to the workspace variable `m`. Click **OK**.
- 9 Double-click the **Override Value** and enter 55.

PARAMETER OVERRIDES		
PARAMETER SET / WORKSPACE VARIABLE	OVERRIDE VALUE	SOURCE
Parameter Set 1 m	55	base workspace

This value overrides the parameter value in the model when the simulation runs.

Note To restore the default value of a parameter, clear the value in the **Override Value** column and press **Enter**.

Run the Test Case

- 1 In the **Test Browser** pane, select the **Upper Bound Test** case.
- 2 In the Test Manager toolstrip, click **Run**. The test results appear in the **Results and Artifacts** pane.

View the Error

Click on the **Upper Bound Test** to view the run-time error.

The screenshot displays the 'Results and Artifacts' tab in the Test Manager. The left pane shows a tree view of test results for 'Results: 2021-Jan-15 16:08:15'. The 'Upper Bound Test' is selected, showing a status of 1 failure. Below it, 'Verify Statements' is also marked as failed. Under 'Verify Statements', 'Check Static Upper Bound' is failed, while 'Sim Output (sldemo_absbrake)' is passed. The 'Sim Output' section lists several variables: 'yout.Ww', 'yout.Vs', 'yout.Sd', and 'slp', each with a corresponding colored bar indicating their status.

The right pane shows the 'SUMMARY' for the 'Upper Bound Test'. It includes a table with the following details:

Name	Upper Bound Test
Outcome	1
Start Time	01/15/2021 16:09:00
End Time	01/15/2021 16:10:41
Type	Simulation Test
Test File Location	C:\Users\idschwart\demo_br_ex.mldatx
Test Case Definition	
Rerun Test Case	
Tags	
Cause of Failure	Errors running test case
Simulation Metadata	

Below the summary, the 'ERRORS' section lists the following messages:

- An error occurred ('Simulink:blocks:AssertionAssert') when calling 'sim':
- Assertion detected in 'sldemo_absbrake/Check Static Upper Bound' at time 12.1928

See Also

More About

- “Run Tests in Multiple Releases of MATLAB” on page 6-102

Automatically Create a Set of Test Cases

In this section...

“Creating Test Cases from Model Elements” on page 6-17

“Generating Test Cases from a Model” on page 6-17

Creating Test Cases from Model Elements

You can automatically create a set of test cases and iterations that correspond to blocks and test harnesses in your model. You specify whether the test cases are baseline, equivalence, or simulation test cases. To automatically create test cases, your model must contain either or both of the following:

- One Signal Editor block at the top level of the model. If the block has only one scenario, a test case is created. If the block has more than one scenario, an iteration is created for each scenario.
- Test harnesses. If a test harness contains one (and only one) Signal Editor block at the top level, a test case is created for the scenario in the block. If the block has more than one scenario, an iteration is created for each scenario.

To automatically create test cases or iterations for your model:

- 1 In the Test Manager, select **New > Test File from Model**.
- 2 In the dialog box, select the model that you want to generate test cases from. The model must be on the MATLAB path.
- 3 Select the test case type, and click **Create**.

Generating Test Cases from a Model

Generate test cases based on model hierarchy.

This example shows how to generate test cases based on the components in your model. This example uses the model `sf_car`, which has been pre-configured with the following:

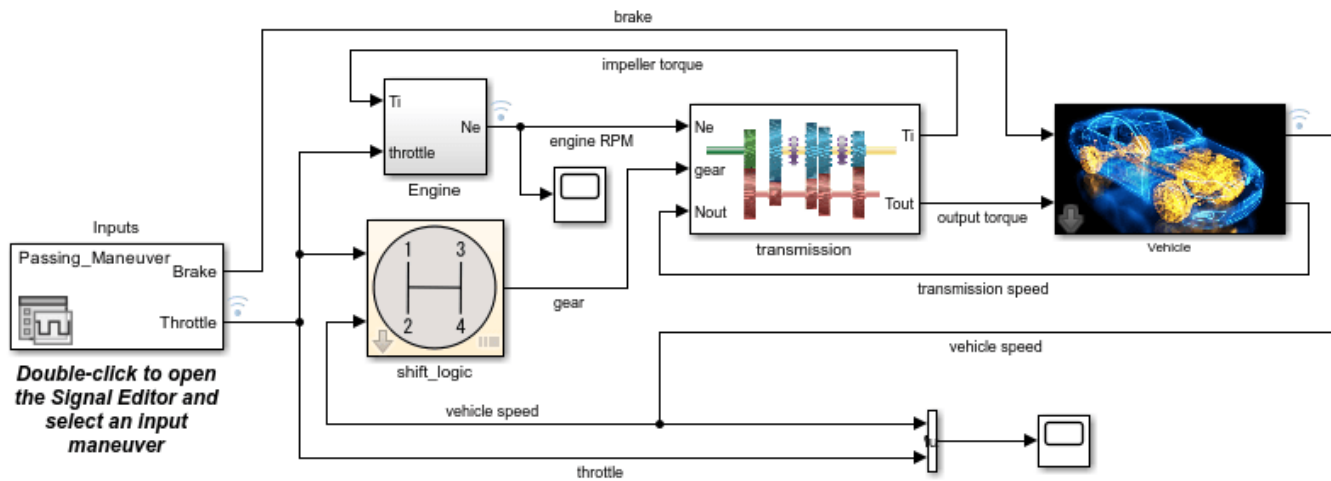
- Signal Editor block at the top level of the model
- Test harnesses at the top level of the model
- Signal Editor block at the top level of the test harness

Open the Model and Test Manager

Execute the following code to open the model configured with different components such as Signal Editor scenarios and test harnesses.

```
model = "sf_car";
open_system(model);
```

Model a Car with Automatic Transmission



Copyright 1997-2025 The MathWorks, Inc.

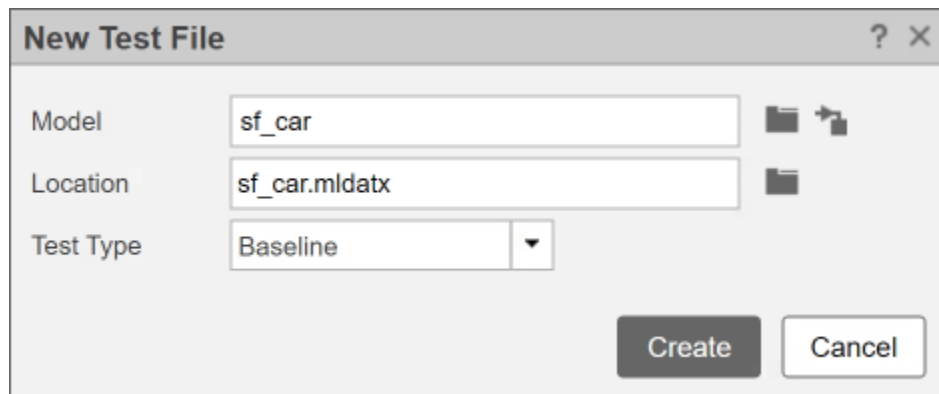
Open the test manager.

```
sltestmgr
```

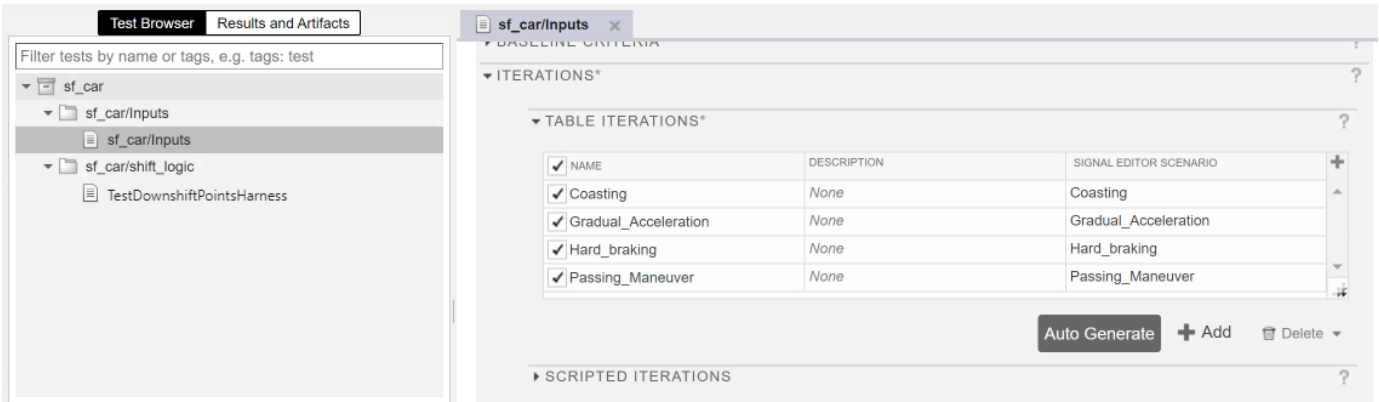
Generate Test Cases From the Model

In the test manager, click **New > Test File from Model**.

- 1 In the **New Test File** dialog box, click the **Use current model** button to specify `sf_car` as the **Model**.
- 2 Specify the **Location** of the test file.
- 3 Select the **Baseline** from the **Test Type** dropdown. All test cases generated will be of the test type specified here.
- 4 Click **Create**.



The `sf_car/Inputs` test case uses table iterations.



Before you run the test, you must specify the baseline criteria for each generated test case.

Clean Up

```
close_system(model, 0);
clear model;
sltest.testmanager.clear
sltest.testmanager.clearResults
sltest.testmanager.close
```

See Also

More About

- “Synchronize Tests” on page 6-75
- “Specify Test Properties in the Test Manager” on page 6-173
- “Test a Simulation for Run-Time Errors” on page 6-14
- “Back-to-Back Equivalence Testing” on page 6-36
- “Import Test Cases for Equivalence Testing” on page 5-16
- “Generate Tests and Test Harnesses for a Model or Components” on page 6-20

Generate Tests and Test Harnesses for a Model or Components

In the Test Manager, the Create Test for Component wizard creates a test harness and test case for a model or component in the model. You can also use the wizard to create test harnesses and test cases for multiple components in the model. Components for which you can create test harnesses include subsystems, Stateflow charts, and Model blocks. For a full list of components supported by test harnesses, see “Test Harness and Model Relationship” on page 2-2. If you select a model component that is not compatible with the wizard, that component does not appear in the wizard.

In the wizard, you specify:

- The model to test.
- The component or components to test, if you are not testing a whole model.
- The test inputs.
- The type of test to run on the component.
- Whether to save test data in a MAT file or Excel®. For more information on using Excel files in the Test Manager, see “Input, Baseline, and Parameter Override Test Case Data Formats in Excel” on page 6-89.

For an example that uses the wizard, see “Back-to-Back Equivalence Testing” on page 6-36.

Open the Create Test for Component Wizard

Before you run the wizard, check your model.

- If you are testing the code for an atomic subsystem by using an equivalence test that uses software-in-the-loop (SIL) or processor-in-the-loop (PIL) mode, verify that the subsystem already has generated code.
- If you are testing the code generated for a reusable library subsystem, before opening the Create Test for Component wizard, verify that the subsystem has defined function interfaces and that the library already has generated code. For information on reusable library subsystems, function interfaces, and generated code, see “Library-Based Code Generation for Reusable Library Subsystems” (Embedded Coder). You must have an Embedded Coder license to verify generated code.
- If you are testing one or more components, select the components in the model before opening the wizard. When the wizard opens, click **Use currently selected components** to add the selected components and fill in the **Top Model** field.

To open the Create Test for Component wizard, in the Test Manager select **New > Test for Model Component**.

Select Model or Component to Test

If you have not selected any components in the model, on the first page of the wizard, click the **Use current model** button  to fill in the **Top Model** field.

Then, if you are testing:

- A whole model, do not select add components to the **Selected components** pane.

To test the whole model without creating a test harness, clear **Create Test Harness for component**. If you are testing a specific component or components in the model, the wizard creates the test harnesses automatically, and the **Create Test Harness for component** option does not display.

- One or more components and you did not select the components in the model before opening the wizard, select the component or press **Ctrl** and click the components to test. Then click the plus button  to add the components to the **Selected components** pane. To remove one or more components, press **Ctrl** and click to select them in the **Selected components** pane, and click the remove button .

- Component in a Model block, you do not need to specify the Model block as the top model. Use the name of the model that contains the Model block as the **Top Model**.
- Reusable library subsystem that has a function interface, a **Function Interface Settings** option displays. Select the function interface for which you want to create a test. The **Function Interface Settings** option is not displayed if you select multiple components.



Function Interface Settings:

Select a function interface: 

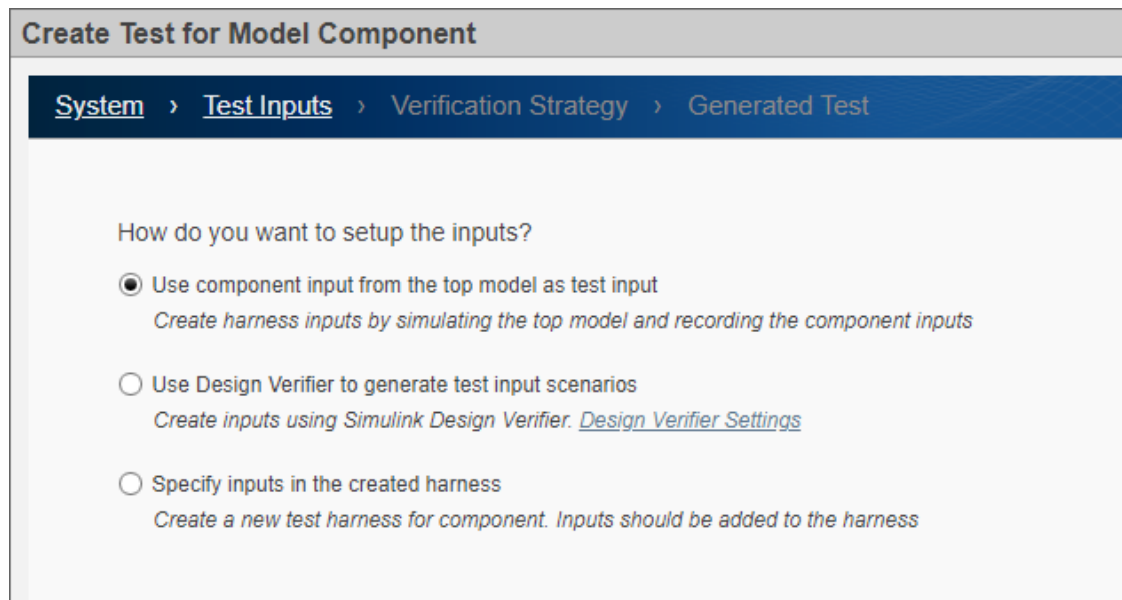
Reusable libraries contain components and subsystems that can be shared with multiple models. You can share the code generated by the subsystems if those subsystems are at the top level of the reusable library and if they have function interfaces. Function interfaces specify the subsystem input and output block parameter settings.

Note For an export-function model, the test harness creates a Test Sequence block automatically.

If you have selected one or more components in the model, on the first page of the wizard, click **Use currently selected components** to fill in the **Top Model** field and add the selected components automatically.

Click **Next** to go to the next page of the wizard.

Set Up Test Inputs



Create Test for Model Component

[System](#) > [Test Inputs](#) > [Verification Strategy](#) > [Generated Test](#)

How do you want to setup the inputs?

- ☒ Use component input from the top model as test input
Create harness inputs by simulating the top model and recording the component inputs
- ☐ Use Design Verifier to generate test input scenarios
Create inputs using Simulink Design Verifier. [Design Verifier Settings](#)
- ☐ Specify inputs in the created harness
Create a new test harness for component. Inputs should be added to the harness

On the **Test Inputs** page, select how to obtain the test inputs.

- **Use component input from the top model as test input** — Simulate the model and record the inputs to the component. Then, use those inputs as the inputs to the created test harness. Use this option for debugging.

Note If you are testing a subsystem that has function calls, you cannot obtain inputs by simulating the model because function calls cannot be logged. Use either of the other two options to obtain the inputs.

- **Use Design Verifier to generate test input scenarios** — Create test harness inputs to meet test coverage requirements using Simulink Design Verifier. This option appears only if Simulink Design Verifier is installed.

Simulate top model and use the recorded component inputs in the analysis — When coverage in test cases generated from Design Verifier is lower than expected, select this option to include top model simulation for the Design Verifier analysis.

- **Specify inputs in the created harness** — After the wizard creates the harness, open the harness in the Test Manager and manually specify the harness inputs. This option does not appear if you chose not to create a test harness.

Test Method

The screenshot shows the 'Create Test for Model Component' wizard with the 'Verification Strategy' tab selected. The breadcrumb trail at the top is 'System > Test Inputs > Verification Strategy > Generated Test'. The main question is 'How do you want to test the component?'. There are three radio button options:

- ☐ Use component under test output as baseline. Below it, the text reads: 'Simulate the top model and record the outputs of the component to be used as baseline'.
- ☒ Perform back-to-back testing. Below it, the text reads: 'Set up a test to compare the component under test outputs in different simulation modes'. Under this option, there is a section 'Select simulation modes:' with two dropdown menus. 'Simulation1:' is set to 'Normal' and 'Simulation2:' is set to 'Software-in-the-Loop (SIL)'.
- ☐ Define the verification logic in the created harness. Below it, the text reads: 'No verification logic will be automatically added to the test'.

On the **Verification Strategy** page, select how to test the component.

- **Use component under test output as baseline** — Simulate the model and record the outputs from the components, which are used as the baseline.
- **Perform back-to-back testing** — Compare the results of running the component in two different simulation modes. For each simulation, select the mode from the drop-down menu. To conduct SIL testing on an atomic subsystem or a reusable library subsystem, the subsystem or library that contains the subsystem must already have generated code.

If you selected **Use Design Verifier to generate test input scenarios** on the **Test Inputs** tab, and you select **Simulation2** to Software-in-the-Loop (SIL) or Processor-in-the-Loop (PIL), the wizard displays the **Set Model Coverage Objective as Enhanced MCDC** option. Enhanced MCDC extends MCDC coverage by generating test cases that avoid masking effects from downstream blocks. See “Enhanced MCDC Coverage in Simulink Design Verifier” (Simulink

Design Verifier) and “Create Back-to-Back Tests Using Enhanced MCDC” (Simulink Design Verifier).

- **Define the verification logic in the created harness** — After the wizard creates the harness, open the harness. Manually specify the verification logic using a Test Sequence or Test Assessments block in the generated harness. Alternatively, use logical and temporal assessments or custom criteria in the generated test case. This option does not appear if you are testing a top-level model and choose not to create a test harness.

Save Test Data

Create Test for Model Component

[System](#) > [Test Inputs](#) > [Verification Strategy](#) > [Generated Test](#)

How do you want to save the test data?

Select test harness input source: Specify the file format:

Specify location to save test data:

Where do you want to save the generated test(s)?

Test File Location:

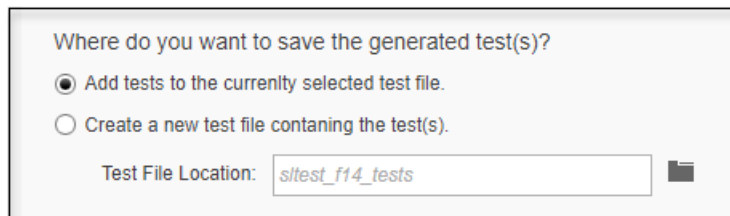
On the **Generated Test** page, select the format in which to save the test data and specify the filename for the generated tests.

- **Select test harness input source** — Select how the inputs generated by the Design Verifier are applied to the test harness. This option appears only if you select **Use Design Verifier to generate test input scenarios** on the Test Inputs tab.
 - **Inports** — Create a test harness with Inport blocks as the source.
 - **Signal Editor** — Create a test harness with the Signal Editor as the source that contains the input scenarios generated by the Design Verifier.
- **Specify the file format** — Specify the type of file in which to save data. This option appears only if you select Inports as the input source.
 - **Excel** — Saves the test inputs, outputs, and parameters to one sheet in an Excel spreadsheet file. For tests with multiple iterations, each iteration is in a separate sheet. For more information on using Excel files in the Test Manager, see “Input, Baseline, and Parameter Override Test Case Data Formats in Excel” on page 6-89.

- **MAT** — Saves inputs and outputs in separate MAT files. For tests that use Simulink Design Verifier, the wizard saves the inputs and parameters in one file and the outputs in a baseline file.
- **Specify location to save test data** — Specify the full path of the file. Alternatively, you can use the default filename and location, which saves `sltest_<model name>` in the current working folder. This option is available only for Excel format files. MAT files are saved to the default location specified in the model configuration settings.
- **Test File Location** — Specify the full path where you want to save the generated test files. Alternatively, you can use the default filename, which is `sltest_<model name>_tests`. The file is saved in the current working folder. This field appears only if you did not have a test file open in the Test Manager before you opened the wizard.

If you had a test file open in the Test Manager before you opened the wizard, these options are displayed instead of **Test File Location**:

- **Add tests to the currently selected test file** — The generated tests are added to the test file that was selected in the **Test Browser** panel of the Test Manager when you opened the wizard.
- **Create a new test file containing the test(s)** — A new test file is created for the tests. It appears in the **Test Browser** panel of the Test Manager.



Where do you want to save the generated test(s)?

☒ Add tests to the currently selected test file.

☐ Create a new test file containing the test(s).

Test File Location: 

Generate the Test Harness and Test Case

Click **Done** to generate a test harness and test case. A test harness is not created if you are testing a whole model and deselected **Create a Test Harness** on the first tab of the wizard.

The Test Manager then opens with the test case in the **Test Browser** pane and, if a test harness was created, the test harness name in the **Harness** field of the **System Under Test** section. The test case is named `<model name>_Harness<#>`.

Note If the model has an existing external harness, the wizard creates an additional external test harness for the component under test. If no harness exists or if an internal harness exists, the wizard creates an internal test harness.

If you are testing the code for an atomic subsystem or model block using an equivalence test and, on the **Verification Strategy** tab, you set **Simulation2** to **Software-in-the-Loop (SIL)** or **Processor-in-the-Loop (PIL)**, the wizard creates only one test harness for both the normal and SIL or PIL simulation modes. For other equivalence tests, the wizard creates two harnesses, one for each simulation mode. For the following types of subsystems and model configurations, the wizard creates two test harnesses, even if the subsystem is atomic.

- Virtual subsystems
- Function-call, For Each, If Action, S-Function, Initialize Function, Terminate Function, and Reset Function subsystems

- Stateflow charts
 - Subsystems where the `ERTFilePackagingFormat` property is set to `Compact` if the subsystem code has the `PreserveStaticInFcnDecls` set to on.
 - Subsystems that generate inline code, such as subsystems where the `RTWSystemCode` property is not either `Nonreusable function` or `Reusable function`).
 - Subsystems that contain referenced models, S-Function, Data Store Read, or Data Store Write blocks
 - Subsystems that have virtual buses at their interface
 - Subsystems that include LDRA or BullsEye code coverage
 - Subsystems that include signal logging
-

See Also

`sltest.testmanager.createTestForComponent`

More About

- “Back-to-Back Equivalence Testing” on page 6-36
- “Baseline Testing” on page 6-240
- “Test a Simulation for Run-Time Errors” on page 6-14
- “Automatically Create a Set of Test Cases” on page 6-17

Override Model Parameters in a Test Case

This example shows how to compare simulation to baseline data using a parameter override.

This example shows how to override a parameter defined in a model workspace using the Test Manager, and view its effect on model output compared to a baseline. Information about using external files to override parameters is also included.

Open the Test File

Open the Test Manager.

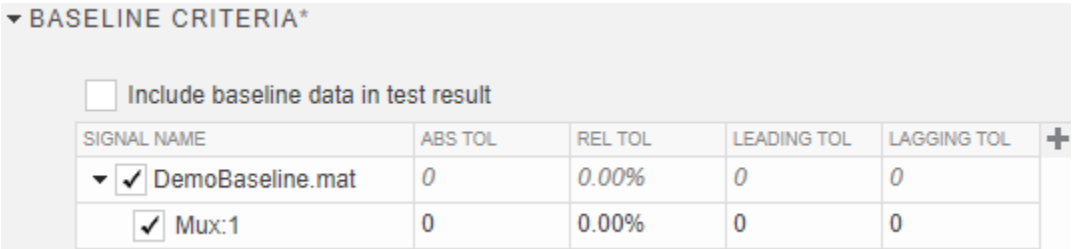
```
sltest.testmanager.view
```

Open the test file.

```
tf = sltest.testmanager.load('sltestParameterOverridesTest.mldatx');
```

Overriding a Model Parameter by Manually Setting Its Value

- 1. In the **Test Browser** pane, select the Test **Override** test case.
- 2. Scroll down to the **Baseline Criteria** section and click **Capture**.
- 3. Specify the **File** to which to save the baseline data. Then, click **Capture**.



- 4. Expand the **Parameter Overrides** section in the test case and click **Add**.
- 5. In the dialog box, if no parameters are displayed, click the Refresh button to display available parameters. Select **a**, which is the gain in the model.
- 6. Click **OK**.

WORKSPACE VARIABLE	CURRENT VALUE	SOURCE	MODEL ELEMENT
☒ a	1	model workspace	sltestParameterOverridesExample/Mu

- 7. The test case displays **a** in the overrides table. Double-click the **Override Value** and enter **1.1**.

Note that parameter overrides are evaluated when the test runs.

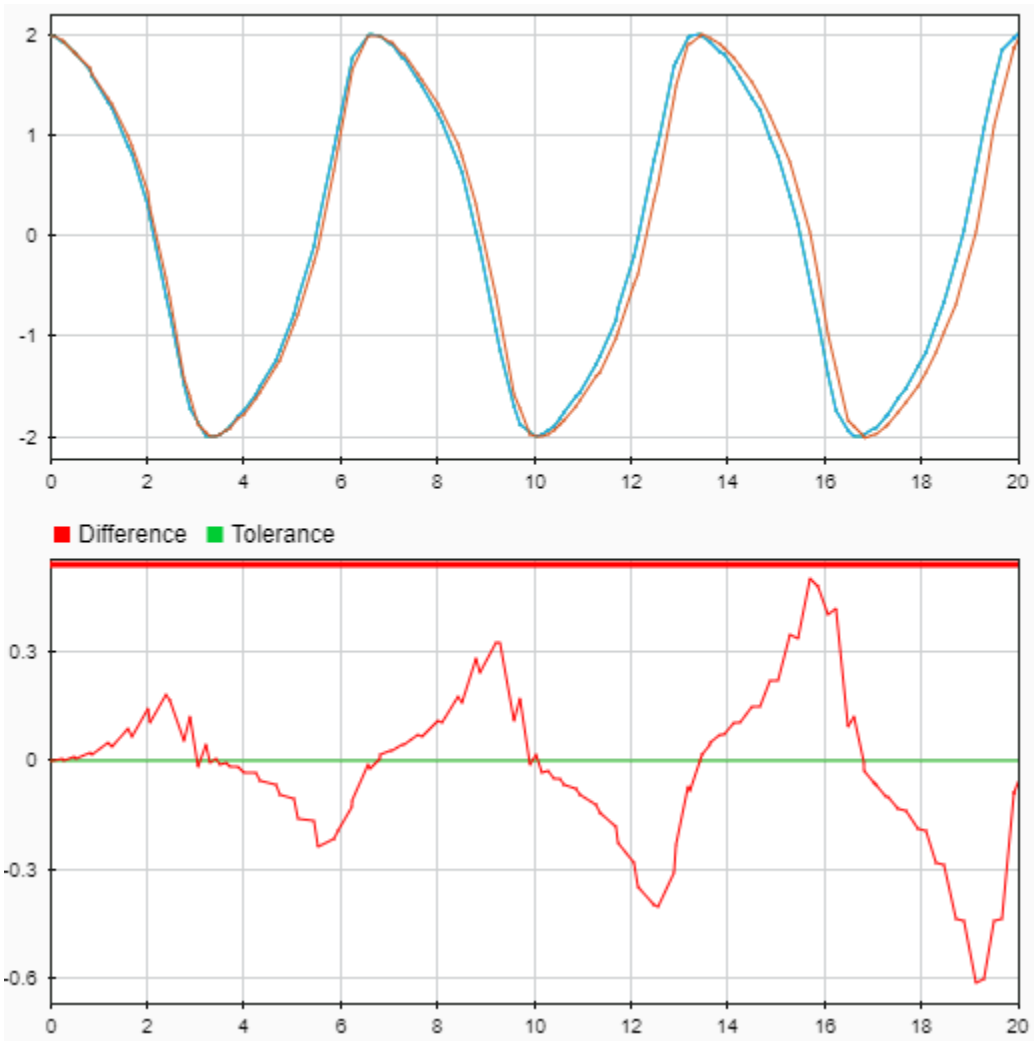
PARAMETER SET / WORKSPACE VARIABLE	OVERRIDE VALUE	SOURCE	MODEL ELEMENT
▼ ☒ Parameter Set 1			
☒ a	1.1	model workspace	sltestParameterOverridesExample/Mu

Run the Test and View Results

Select the test file in the **Test Browser** pane and click **Run**. In the **Results and Artifacts** pane, expand the results to see the **Baseline Criteria Result** and **Sim Output**.

▲ Test Override	✖
▲ Baseline Criteria Result	✖
○ Mux:1[1]	✖
○ Mux:1[2]	✖
▲ Sim Output (sltestParameterOve	
☐ Mux:1[1]	—
☐ Mux:1[2]	—

Select Mux: 1(1) under **Baseline Criteria Result** to see how overriding the parameter affects the mux signal when compared to the captured baseline. The comparison output shows a maximum difference of approximately 0.6.



Overriding a Model Parameter using a MAT-File

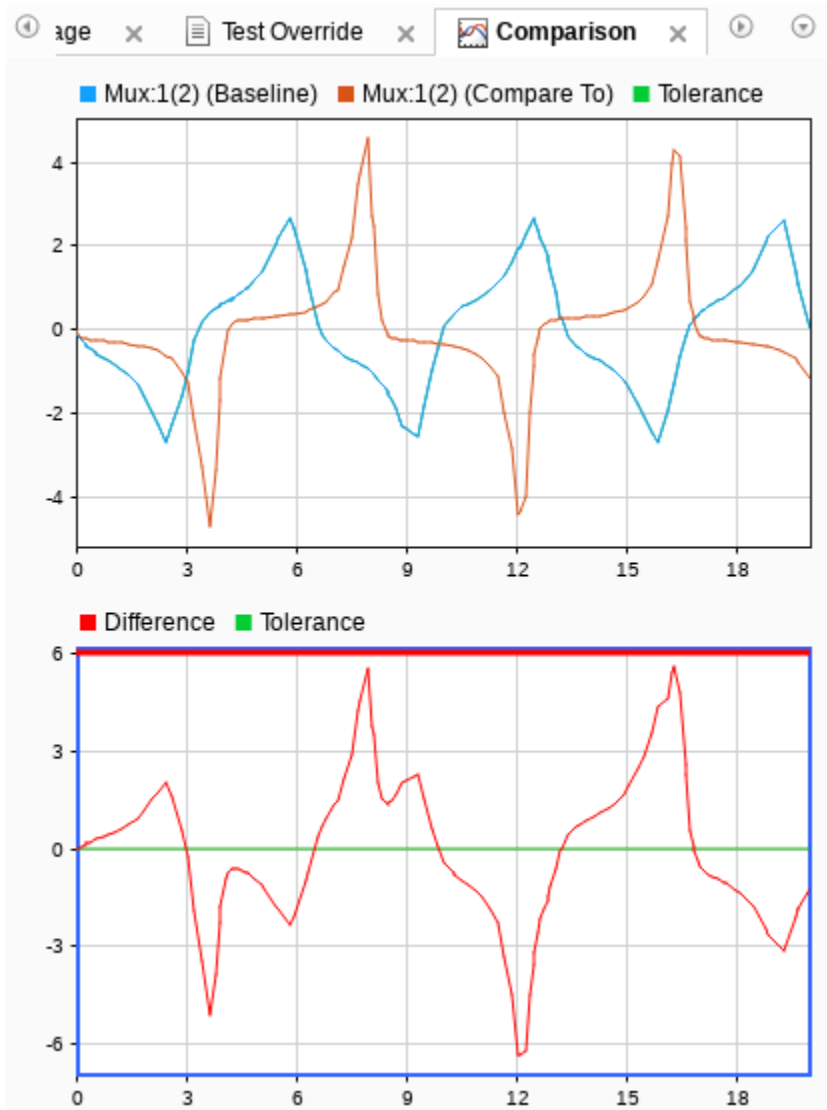
- 1. Return to the **Test Browser** panel, select the **Test Override** test case, and expand the **Parameter Overrides** section.
- 2. Click the **Add** arrow and select **Add File** from drop down.

Select the `sltestOverrideParameter.mat` file. This file contains data that can be used by the test case to override the parameter.

PARAMETER SET / WORKSPACE VARIABLE	OVERRIDE VALUE	SOURCE	MODEL ELEMENT
▼ ☐ Parameter Set 1			
☒ a	1	model workspace	sltestParameterOverridesExample/Mu
▼ ☒ sltestOverrideParameter.mat			
☒ a	2.718281728		

If you want to access the override value in the MATLAB® base workspace, select the row with the override value from the file, right-click, and select **Export**. The override value is exported to the workspace as a variable named **a**.

Run the test again. In the **Results and Artifacts** pane, expand the new results to see the **Baseline Criteria Result**. Select **Mux: 1(2)**, which shows the effect of increasing the value of the mux signal gain (a).



You can also use parameter override data from MATLAB script (.m) and Excel files.

```
sltest.testmanager.clearResults;  
close(tf);  
sltest.testmanager.close
```

See Also

More About

- “Use External File Data in Test Cases” on page 6-76
- “Create External Data Files to Use in Test Cases” on page 6-84

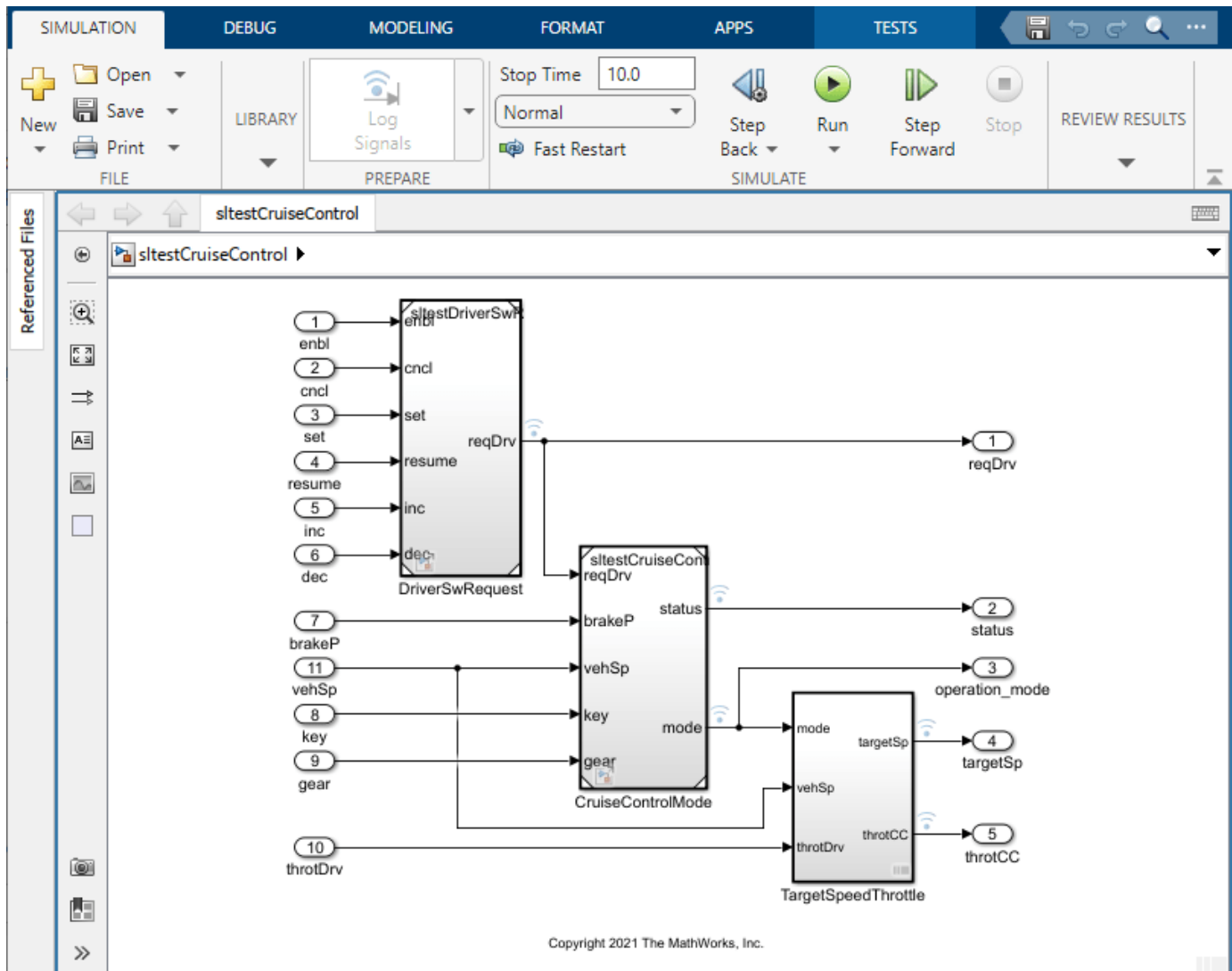
Override the Simulation Mode During Test Execution

This example shows how to override the simulation mode when you run a test. You can run a test in a simulation mode without modifying the simulation mode specified in the test case or the model settings.

Open the Model

```
open_system("sltestCruiseControl.slx")
```

In this model, the simulation mode is set to normal.

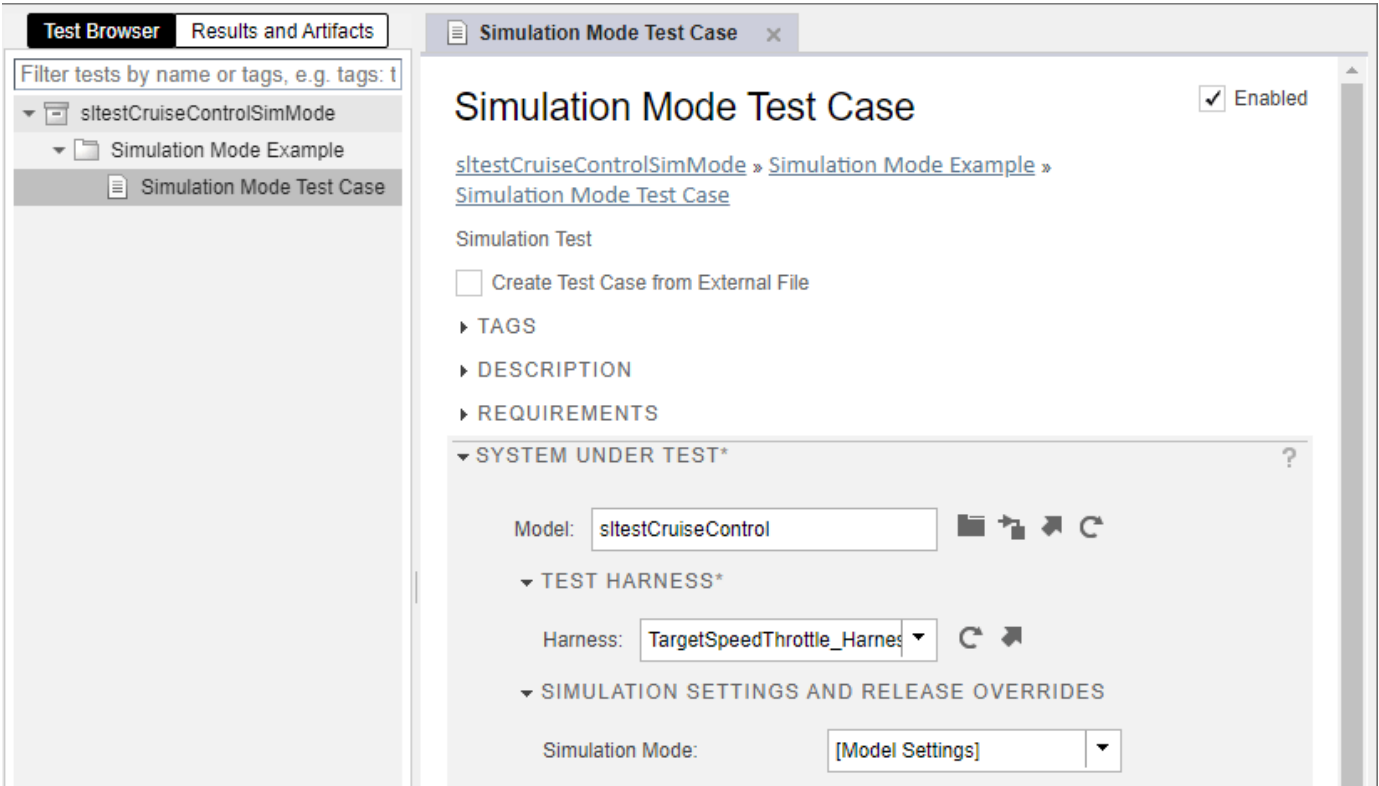


Open the Test File and Run the Test Case

Open the Simulink Test Manager and load the test file.

```
sltest.testmanager.view
sltest.testmanager.load("sltestCruiseControlSimMode.mldatx");
```


In the **Test Browser** pane, select **Simulation Mode Test Case**. In the test case properties, expand the **System Under Test** section, then expand the **Simulation Settings and Release Overrides** section. This test case uses the **Simulation Mode** specified in the model settings.



On the toolbar, click **Run**. In the **Results and Artifacts** pane, expand the results and select **Simulation Mode Test Case**. Confirm that the test uses normal simulation mode.

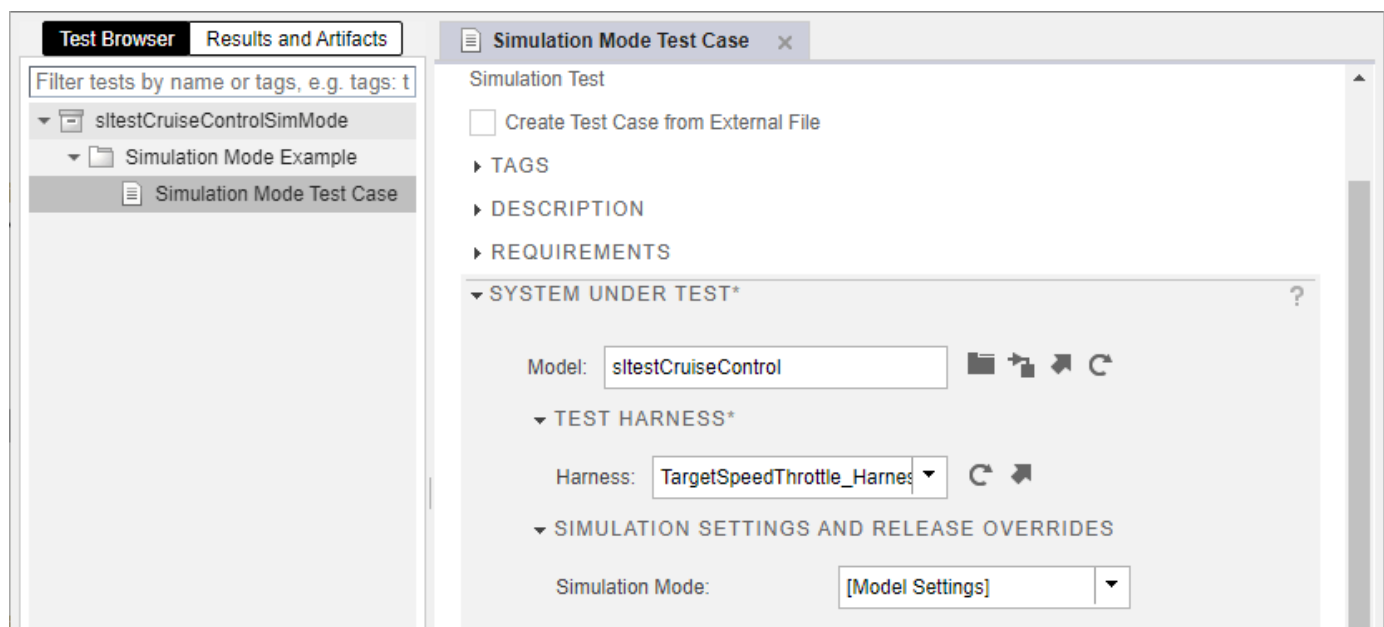
PROPERTY	VALUE
Name	Simulation Mode Test Case
Status	1
Start Time	07/21/2023 13:10:56
End Time	07/21/2023 13:10:57
Type	Simulation Test
Test File Location	C:\Users\
Model	sltestCruiseControl
Simulation Mode	normal
Harness Name	TargetSpeedThrottle_Harness

Run the Test Case in a Different Simulation Mode

In the **Test Browser** pane, select **Simulation Mode Test Case**. Click the **Run** drop-down, and select **Run Selected in > Accelerator**. In the **Results and Artifacts** pane, expand the second set of results and select **Simulation Mode Test Case**. The simulation runs in accelerator mode.

PROPERTY	VALUE
Name	Simulation Mode Test Case
Status	1
Start Time	07/21/2023 13:29:49
End Time	07/21/2023 13:29:51
Type	Simulation Test
Test File Location	C:\Users\w\
Model	sltestCruiseControl
Simulation Mode	accelerator
Harness Name	TargetSpeedThrottle_Harness

In the **Test Browser** pane, select **Simulation Mode Test Case**. Under **System Under Test**, in the **Simulation Settings and Release Overrides** section, the test file is not modified and does not display unsaved changes.



Clean Up

Clear and close the Test Manager. Then, close the model without saving.

```
sltest.testmanager.clearResults
sltest.testmanager.clear
```

```
sltest.testmanager.close  
close_system("sltestCruiseControl.slx",0)
```

See Also

More About

- “Specify Test Properties in the Test Manager” on page 6-173
- “Back-to-Back Equivalence Testing” on page 6-36

Back-to-Back Equivalence Testing

This example shows how to create and run a back-to-back test on a whole model to compare the results of normal mode simulation and software-in-the-loop (SIL) simulation. Back-to-back testing, or equivalence testing, tests whether the generated code produces the same results as the model simulation. This type of testing can also expose coverage gaps and increase confidence that the generated code is working as expected. Implementing and testing functional requirements, validating the code generated from the model, and achieving 100% model and code coverage are key parts of compliance for software and safety standards and certifications, such as ISO 26262 and IEC 61508.

This example shows only model and SIL testing. SIL simulation compiles and runs the generated code on your development computer. Processor-in-the-loop (PIL) simulation cross-compiles source code on your development computer, and then downloads and runs the object code on a target processor or an equivalent instruction set simulator. For full compliance, you need to also do PIL testing. See “Configure and Run PIL Simulation” (Embedded Coder) and “SIL and PIL Limitations” (Embedded Coder) for more information.

The test results of normal mode simulation and the generated code might differ due to numerical differences, such as differences in tolerances when comparing floating-point values. Time delays or compiler issues might also cause differences between the code and simulation. Back-to-back test results that do not match can expose one or more existing issues, which you can fix so the model and generated code comply with the applicable industry standards and certifications.

The `sf_boilerController` model in this example contains a Stateflow chart that models a temperature controller for a boiler. The model is derived from the “Model Bang-Bang Temperature Control System” (Stateflow) example.

Open the Model and Test Manager

Open the model. Then open the Test Manager and clear the previously loaded test files and results.

```
Model = "sf_boilerController";
open_system(Model)
```

```
sltest.testmanager.view
```

```
sltest.testmanager.clear
sltest.testmanager.clearResults
```

Open the Create Test for Model Component Wizard

In the Test Manager, select **New > Test for Model Component** to open the Create Test for Model Component wizard.

Specify the Model and Components to Test

1. In the System tab, click the Use current model button  to add the model to the **Top Model** field.
2. In the left pane, select the top-level model, `sf_boilerController` to select the whole model.
3. Click the Add button  to add the model to the **Selected components** pane.
4. Click **Next**.

System > Test Inputs > Verification Strategy > Generated Test

Top Model:

Search components

▼  sf_boilerController

 Bang-Bang Controller





 sf_boilerController

☐ Select all Subsystems 

☐ Select all Model References 

☒ Create Test Harness for component

 Use currently selected components

Specify the Test Inputs

In the Test Inputs tab, use the default setting, **Use component input from the top model as test input**. Then click **Next**.

System › **Test Inputs** › Verification Strategy › Generated Test

How do you want to set up the inputs?

- ☒ Use component input from the top model as test input
Create harness inputs by simulating the top model and recording the component inputs
- ☐ Use Design Verifier to generate test input scenarios
Create inputs using Simulink Design Verifier. [Design Verifier Settings](#)
- ☐ Specify inputs in the created harness
Create a new test harness for component. Inputs should be added to the harness

Specify How to Test the Component

In the Verification Strategy tab, specify to create a back-to-back test.

1. Select **Perform back-to-back testing**.
2. Ensure that **Simulation1** is set to Normal and **Simulation2** is set to Software-in-the-Loop (SIL). Then click **Next**.

System › Test Inputs › Verification Strategy › Generated Test

How do you want to test the component?

☐ Use component under test output as baseline

Simulate the top model and record the outputs of the component to be used as baseline

☒ Perform back-to-back testing

Set up a test to compare the component under test outputs in different simulation modes

Select simulation modes:

Simulation1: Normal ▼

Simulation2: Software-in-the-Loop (SIL) ▼

☐ Define the verification logic in the created harness

No verification logic will be automatically added to the test

Specify the Input, File Format, and File Locations

In the Generated Test tab, specify the input source, the file format, and where to save the test data and the generated tests.

1. Ensure that **Select test harness input source** is Imports.
2. Change **Specify the file format** to Excel. Saving the inputs to Excel lets you view and manually edit the file contents outside of Simulink.
3. Use the default values for the locations to save the test data and the generated tests.
4. Click **Done**.

The Create Test for Model Component wizard sets up signal logging for the outputs from the chart, simulates the model, captures the signals, and creates a test harness and test case. It also creates the Excel input file, `sltest_sf_boilerController.xlsx`, and the test file, `sltest_sf_boilerController_tests.mldatx`.

System › Test Inputs › Verification Strategy › Generated Test

How do you want to save the test data?

Select test harness input source: Specify the file format:

Specify location to save test data:



Where do you want to save the generated test(s)?

Test File Location: 

Check the Simulation Modes

1. In the **Test Browser** pane of the Test Manager, select the test case the wizard created, `sf_boilerController_Harness1`. The wizard automatically sets the test case name to the *model name_associated test harness name*.
2. In the right pane, expand the **Simulation 1** section and then expand **Simulation Settings and Release Overrides**. Check that the **Simulation Mode** parameter is `Normal`.

sltest_sf_boilerController_tests*

New Test Suite 1

sf_boilerController_Harness1

SIMULATION 1

SYSTEM UNDER TEST*

Model: sf_boilerController

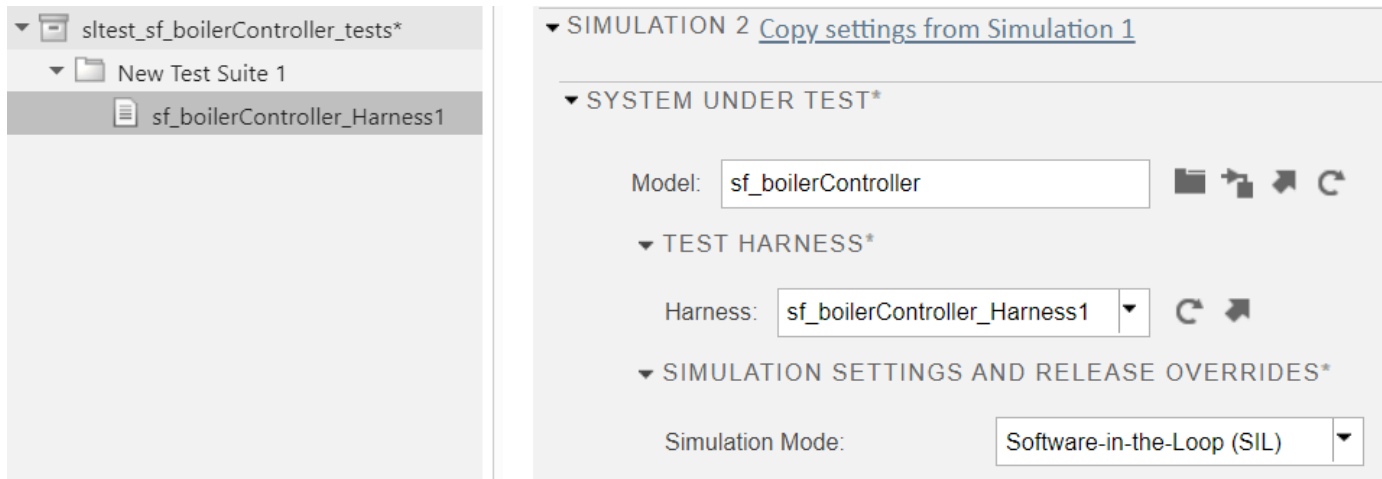
TEST HARNESS*

Harness: sf_boilerController_Harness1

SIMULATION SETTINGS AND RELEASE OVERRIDES*

Simulation Mode: Normal

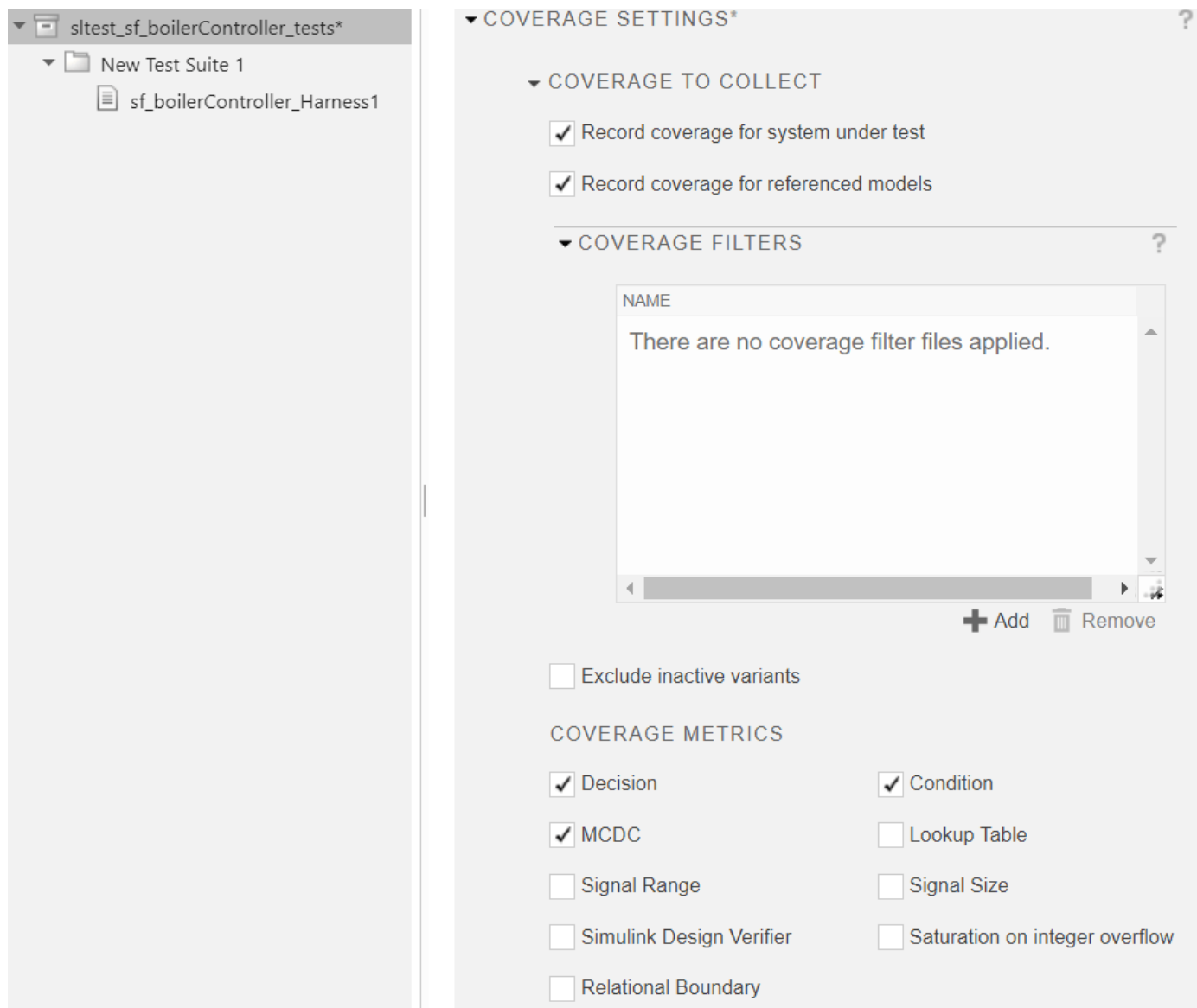
3. Expand the **Simulation 2** section and its **Simulation Settings and Release Overrides**. Check that its **Simulation Mode** parameter is Software-in-the-Loop (SIL).



Set the Coverage to Collect

1. In the **Test Browser** pane, select the test file, `sltest_sf_boilerController_tests`.
2. In the right pane, expand **Coverage Settings**. You can set up coverage only at the test file level. The coverage settings propagate to the test cases in the file.
3. Select **Record coverage for system under test** and **Record coverage for referenced models** to enable coverage collection.
4. Under **Coverage Metrics**, select **Decision**, **Condition**, and **MCDC**. Most certification standards, including ISO 26262, require these metrics.
 - Decision coverage analyzes decision points in the model. A decision is a place in the model where one or more input signals decide the output signal of a block. Decision coverage determines whether all decisions are tested.
 - Condition coverage analyzes blocks that output the logical combination of their inputs and Stateflow transitions. For this example, a test case achieves full coverage when it causes each condition on a Stateflow transition to be true at least once and false at least once during the simulation.
 - MCDC (Modified Condition/Decision Coverage) extends decision and condition coverage. It analyzes whether each condition within a decision independently affects the decision outcome, and is evaluated to all possible outcomes at least once. For this example, MCDC analyzes the Stateflow transitions.

For more information about coverage and coverage metrics, see “Model Coverage” (Simulink Coverage) and “Types of Model Coverage” (Simulink Coverage), respectively.



Run the Test Case and View the Results

1. Select the test case, `sf_boilerController_Harness1`, and click the **Run** button in the toolbar. When the test runs, the left pane displays the **Results and Artifacts** pane.
2. When the test finishes, click the Results in the **Results and Artifacts** pane to update the right pane.

The **Aggregated Coverage Results** section shows the code coverage for SIL mode simulation and coverage for functional elements in the model for normal mode simulation. Coverage is less than 100% for both the normal and SIL simulation modes, which indicates that not all of the code and model pathways were exercised during the test. Function coverage metrics do not apply to Normal simulation mode.

ANALYZED MODEL	REPORT	SIM MODE	COM...	DECISION	CONDITION	MCDC	EXECUTION	FUNCTION	FUNCTION ...
 sf_boilerController		ModelRefSIL	21	39% 	0% 	--	39% 	60% 	20% 
 sf_boilerController		Normal	14	32% 	25% 	0% 	--	--	--

Increase the Coverage

You can increase the coverage by using Simulink Design Verifier. After you run a test and collect coverage, Simulink Design Verifier can generate new test cases with different input to test the untested parts of the model.

1. In the **Sim Mode** column of the **Aggregated Coverage Results** table, click **Normal**.
2. Below the table, click **Add Tests for Missing Coverage**.
3. In the Add Tests for Missing Coverage dialog box, set **Test Case** to <Create a new test case>.
4. Set **Test Type** to Equivalence Test.

Add Tests for Missing Coverage

Generate tests for missing coverage for sf_boilerController

Harness:

Source:

Test Case:

Test Type:

Test File:

5. Leave the other fields as their default values. Click **OK**. Simulink Design Verifier creates a new test suite, test case, and test iterations. The **Test Browser** shows the new test suite and test case, New Test Case 1.

6. Select New Test Case 1 and expand the **Iterations** section of New Test Case 1 to see the iterations.

Set the Simulation Modes, Run the New Test Case, and View the Updated Coverage

1. Expand **Simulation 1** and **Simulation Settings and Release Overrides**. Change the **Simulation Mode** to Normal.
2. Expand **Simulation 2** and **Simulation Settings and Release Overrides**. Change the **Simulation Mode** to Software-in-the-Loop (SIL).

3. Click the **Run** button in the toolbar.

4. When the test finishes running, click the latest **Results** in the **Results and Artifacts** panel to update the right side pane.

5. View the **Aggregated Coverage Results**, which are 100% for normal mode, but the **Decision** metric result for SIL mode is only 94%. The missing **Decision** coverage for this model is caused by code that evaluates the false condition when values are outside the specified range. The code generator adds integer overflow protection to reduce the long amount of time the test case would need to evaluate all false values.

ANALYZED MODEL	REPORT	SIM MODE	COM...	DECISION	CONDITION	MCDC	EXECUTION	FUNCTION	FUNCTION ...
sf_boilerController		ModelRef...	21	94%	100%	--	100%	100%	100%
sf_boilerController		Normal	14	100%	100%	100%	--	--	--

Justify and Filter the Missing Coverage

You can provide a justification rationale for code lines or model elements that you want to mark as unsatisfied coverage. Reasons for justifying coverage results that might produce less than 100% coverage include code generation optimization, code replacement libraries, error conditions, and defensive code.

1. Click the Create aggregated coverage report arrow in the **Report** column for the SIL simulation to create a SIL Coverage Summary report.

ANALYZED MODEL	REPORT	SIM MODE	COM...	DECISION	CONDITION	MCDC
sf_boilerController		ModelRefSIL	21	94%	100%	--
sf_boilerController						100%

Create aggregated coverage report

2. In the SIL Coverage Summary Report for `sf_boilerController`, go to section **5.1 Decision localDW->temporalCounter_i1 < 31U (line 90)**. This is one of two lines of code the contributes to the lower **Decision** coverage result. The specified range has a limited number of integer values so coverage for the **true** result is 100%, but the **false** result has an unlimited number of possible values which would require a very long time to test, so the **false** result cannot be considered to have 100% coverage.

3. Click the **Justify or Exclude** link or the Add justification rule icon next to the highlighted **false** result to open the Filter Explorer.

5.1. Decision [localDW->temporalCounter i1 < 31U](#) (line 90)

Justify or Exclude

Function: [sf boilerController](#)

Model Objects: [SFunction](#), [Bang-Bang Controller](#), [SF: Bang-Bang Controller](#)

Uncovered Links: 

Metric	Coverage
Decision	50% (1/2) decision outcomes

Decisions analyzed

localDW->temporalCounter i1 < 31U	50%
false	0/390 
true	390/390 T1

4. In the **Filter Rules** section of the Filter Explorer, change the **Mode** for the displayed code line to **Justified**. Add "Integer overflow protection in generated code can only be exercised by unrealistically long test case" as the **Rationale** for the justification. For information about coverage filtering, see "Coverage Filtering" (Simulink Coverage) and "Coverage Filter Rules and Files" (Simulink Coverage).

Filter Explorer

Applied filters (1)

- Untitled

Filter Editor

Name:

Filename: (not saved)

[Save as](#)

Description:

Filter Rules

Model Code

Name	Type	Mode	Rationale
localDW->temporalCounter...	by C/C++ decision expression	Justified	Integer overflow protection i...

5. Click **Apply** and in the Save Filter File dialog box, enter boilerController_covfilter as the name of the coverage filter (.cvf) file and click **Save**.

6. In the SIL Coverage Summary Report for sf_boilerController, notice that section **5.1** is now **Justified**, its **Coverage** is 100%, and the **false** line of the table is highlighted in light blue.

5.1. Decision [localDW->temporalCounter_i1 < 31U](#) (line 90)

Justified J1.

Function: [sf_boilerController](#)

Model Objects: [SFunction](#), [Bang-Bang Controller](#), [SF: Bang-Bang Controller](#)

Metric

Decision

Coverage

100% ((1+1)/2) decision outcomes

Decisions analyzed

localDW->temporalCounter_i1 < 31U	50%
false	0/390
true	390/390 T1

7. Go to section **5.2 Decision [localDW->temporalCounter_i2 < 63U](#) (line 94)**. in the report and repeat steps 3 and 4 to justify the missing coverage for that **false** decision. Use the same rationale that you used for section **5.1**.

8. Click **Apply** to update the boilerController_covfilter.cvf coverage filter file and the SIL Coverage Report. The Summary section of the report shows 100% coverage with the light blue portions of the coverage percent bars indicating justified coverage. In the Test Manager, the **Aggregated Coverage Results** show the updated coverage and the coverage filter file is listed in the Coverage Filters section.

ANALYZED MODEL	REPORT	SIM MODE	COM...	DECISION	CONDITION	MCDC	EXECUTION	FUNCTION	FUNCTION CALL	
sf_boilerController		ModelRefSIL	21	100%	100%	--	100%	100%	100%	
sf_boilerController		Normal	14	100%	100%	100%	--	--	--	

☐ Scope coverage results to linked requirements
☐ Exclude inactive variants

Add Tests for Missing Coverage
 Export

▼ COVERAGE FILTERS

NAME

boilerController_covfilter.cvf

9. At the bottom of the **Coverage Filters** pane, click **Update Test File** to append the coverage filter to the test file.

View the Simulation Output Results for Equivalence

Expand the **Results > New Test Case 1 > Iteration1** and select **Equivalence Criteria Result**. Then, in the **Compare** tab of the embedded Simulation Data Inspector, select the **Output:1** signal.

Filter results by name or tags, e.g. tags: test

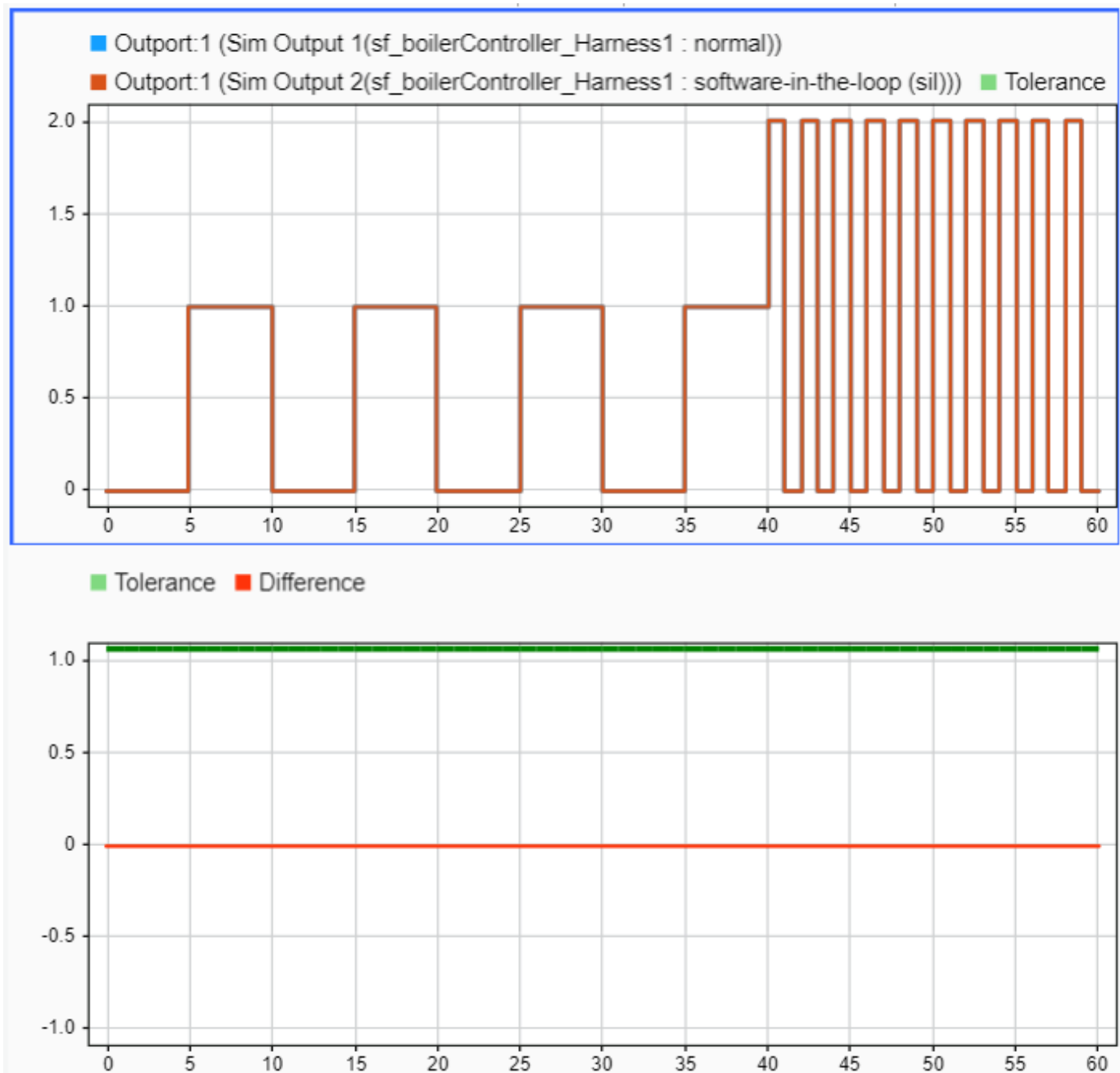
NAME	STATUS
▶ Results: 2024-Dec-09 13:59:31	1
▼ Results: 2024-Dec-09 14:04:24	6
▼ New Test Case 1	6
▼ Iteration1	
Equivalence Criteria Result	
☐ Sim Output 1(sf_b...	
☐ Sim Output 2(sf_b...	

Inspect **Compare**

2 Match
 0 Mismatch
 0 Not compared [+] More

NAME (BASE)	A..	R...	MAX...	RESULT
Output:1	0	0	0	
Output1:1	0	0	0	

The upper plot shows both the normal mode and SIL mode outputs, which overlap. The lower plot shows that the difference between the output signals is zero, so the normal mode and SIL mode simulations match.



Clean up

Use these commands to close the model and close and clear the Test Manager:

```
close_system(Model,0)
sltest.testmanager.clear
sltest.testmanager.clearResults
```



```
sltest.testmanager.close
```

See Also

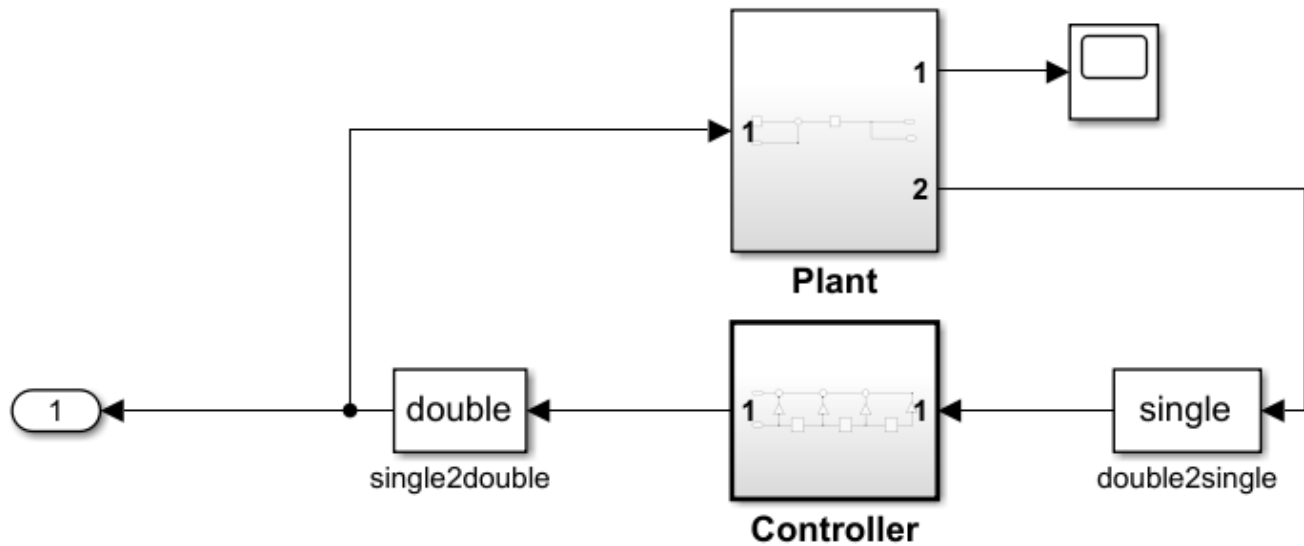
`sltest.testmanager.TestFile` | `sltest.testmanager.TestCase` | Simulink Test Manager

Related Examples

- “Generate Tests and Test Harnesses for a Model or Components” on page 6-20
- “Import Test Cases for Equivalence Testing” on page 5-16
- “Back-to-Back (MIL/SIL) Equivalence Testing an Atomic Subsystem” on page 6-50
- “Configure and Run SIL Simulation” (Embedded Coder)
- “SIL/PIL Manager Verification Workflow” (Embedded Coder)
- “Unit Test Subsystem Code with SIL/PIL Manager” (Embedded Coder)

Back-to-Back (MIL/SIL) Equivalence Testing an Atomic Subsystem

This example shows how to create and run a back-to-back test, which is also known as an equivalence test, for an atomic subsystem.



Generate Code for the Model

Open the model, configure the system on which you are running Simulink, and generate code for the whole model.

```
model = 'sltestMILSILEquivalence';
open_system(model);
% Configure for code generation
if ismac
    lProdHWDeviceType = 'Intel->x86-64 (Mac OS X)';
elseif isunix
    lProdHWDeviceType = 'Intel->x86-64 (Linux 64)';
else
    lProdHWDeviceType = 'Intel->x86-64 (Windows64)';
end

set_param(model, 'ProdHWDeviceType', lProdHWDeviceType);
slbuild(model);

### Searching for referenced models in model 'sltestMILSILEquivalence'.
### Total of 1 models to build.
### Starting build procedure for: sltestMILSILEquivalence
### Successful completion of build procedure for: sltestMILSILEquivalence

Build Summary

Top model targets:
```

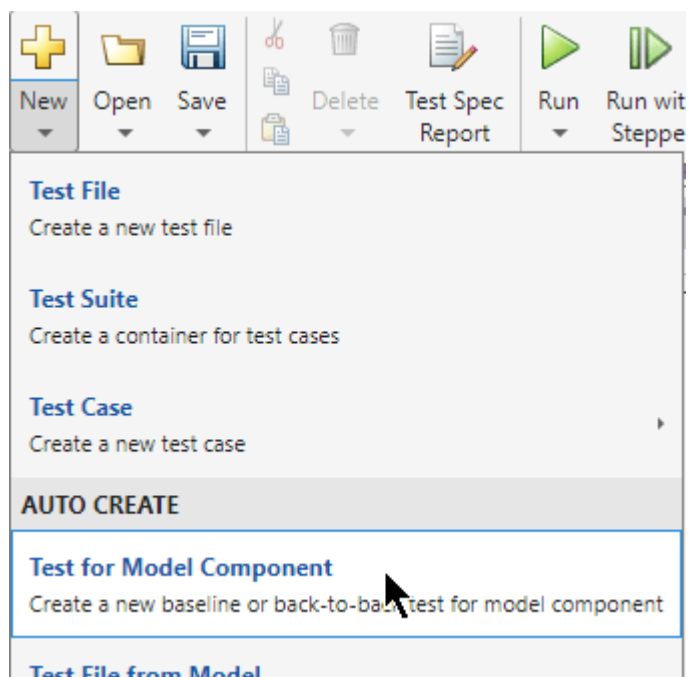
Model	Build Reason	Status
sltestMILSILEquivalence	Information cache folder or artifacts were missing.	Code generated and
1 of 1 models built (0 models already up to date)		
Build duration: 0h 1m 13.557s		

Create a Back-to-Back Test Using the Test Manager and Test For Model Component Wizard

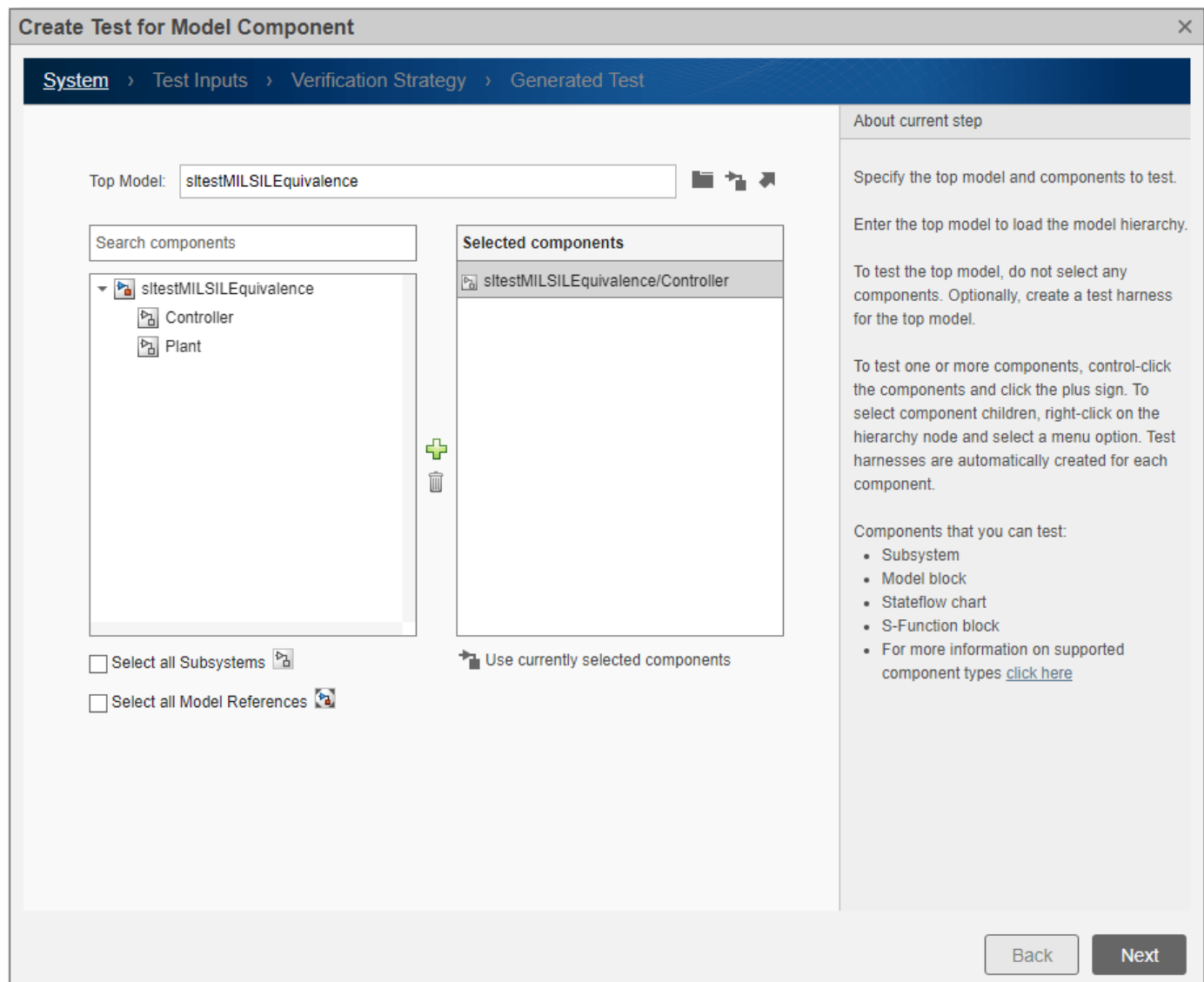
1. Open the Test Manager.

```
sltest.testmanager.view;
```

2. Click **New > Test For Model Component**.



3. Click the Use current model icon  to add the sltestMILSILEquivalence model to the **Top Model** field. Select Controller and click the plus sign. Then click **Next**.



4. Select **Use component input from the top model as test input**. Then click **Next**.

Create Test for Model Component

System > Test Inputs > Verification Strategy > Generated Test

How do you want to set up the inputs?

☒ Use component input from the top model as test input
Create harness inputs by simulating the top model and recording the component inputs

☐ Use Design Verifier to generate test input scenarios
Create inputs using Simulink Design Verifier. [Design Verifier Settings](#)

☐ Specify inputs in the created harness
Create a new test harness for component. Inputs should be added to the harness

About current step

Select how to generate input vectors to test the component.

About selected option

Simulates the top model, records the component inputs, creates a test harness for the component and uses the recorded inputs as harness inputs.

Typically a debugging workflow.

Back

Next

5. Select **Perform back-to-back testing**. Check that **Simulation1** is set to Normal and **Simulation2** is set to Software-in-the-Loop (SIL). Then click **Next**.

Create Test for Model Component

System > Test Inputs > Verification Strategy > Generated Test

How do you want to test the component?

☐ Use component under test output as baseline
Simulate the top model and record the outputs of the component to be used as baseline

☒ Perform back-to-back testing
Set up a test to compare the component under test outputs in different simulation modes

Select simulation modes:

Simulation1: Normal ▼

Simulation2: Software-in-the-Loop (SIL) ▼

☒ Set Model coverage objectives as Enhanced MCDC

☐ Define the verification logic in the created harness
No verification logic will be automatically added to the test

About current step

Select how to verify the test outputs.

You can perform baseline testing, back to back (equivalency) testing, or manually set up the verification logic in the auto created test harness.

About selected option

Auto creates the required test harnesses for the component under test.

Sets up a test case containing two simulations to compare the harness outputs in the selected simulation modes.

You can set 'Enhanced MCDC' as coverage objectives for Design Verifier test generation. [Click here](#) for more details.

Back Next

6. Leave the default values for saving the test data and generated test:

- **Select test harness input source:** Inports,
- **Specify the file format:** MAT
- **Specify location to save test data:** `sltest_sltestMILSILEquivalence`
- **Test File Location:** `sltest_sltestMILSILEquivalence_tests`

Then, click **Done**. The Create Tests for Model Component generates the back-to-back (equivalence) test and then returns to the Test Manager.

Create Test for Model Component

System > Test Inputs > Verification Strategy > Generated Test

How do you want to save the test data?

Select test harness input source:

Inports

Specify the file format

MAT

Specify location to save test data:

sltest_sltestMILSILEquivalence

Where do you want to save the generated test(s)?

Test File Location:

sltest_sltestMILSILEquivalence_tests

About current step

Specify the file format and file location to save the test artifacts (inputs, expected outputs).

For MAT file format, inputs and outputs are saved in separate files at the specified location.

For Excel spreadsheet, the inputs and outputs are saved to the same sheet.

Add the generated test(s) to a file loaded in Test Manager or create a new test file at the location specified in the Test File field.

Back

Done

7. In the Test Manager **Test Browser** pane, select the test file, `sltest_sltestMILSILEquivalence_tests`. Enable coverage collection by expanding the **Coverage Settings** section. Under **Coverage to Collect**, select **Record coverage for system under test**. Under **Coverage Metrics**, select **Decision** and **MCDC**. Coverage assesses the completeness of a test.

▼ COVERAGE SETTINGS*

▼ COVERAGE TO COLLECT

☒ Record coverage for system under test

☐ Record coverage for referenced models

▼ COVERAGE FILTERS

NAME

There are no coverage filter files applied.

COVERAGE METRICS

☒ Decision

☐ Condition

☒ MCDC

☐ Lookup Table

8. Select the `sltestMILSILEquivalence_Harness1` test case. Expand the **Simulation 1** and **Simulation 2** sections and their **System Under Test** and **Simulation Settings and Release Overrides** subsections to see the settings.

▼ SIMULATION 1

▼ SYSTEM UNDER TEST*

Model:

▼ TEST HARNESS*

Harness:

▼ SIMULATION SETTINGS AND RELEASE OVERRIDES*

Simulation Mode:

▼ SIMULATION 2 [Copy settings from Simulation 1](#)

▼ SYSTEM UNDER TEST*

Model:

▼ TEST HARNESS*

Harness:

▼ SIMULATION SETTINGS AND RELEASE OVERRIDES*

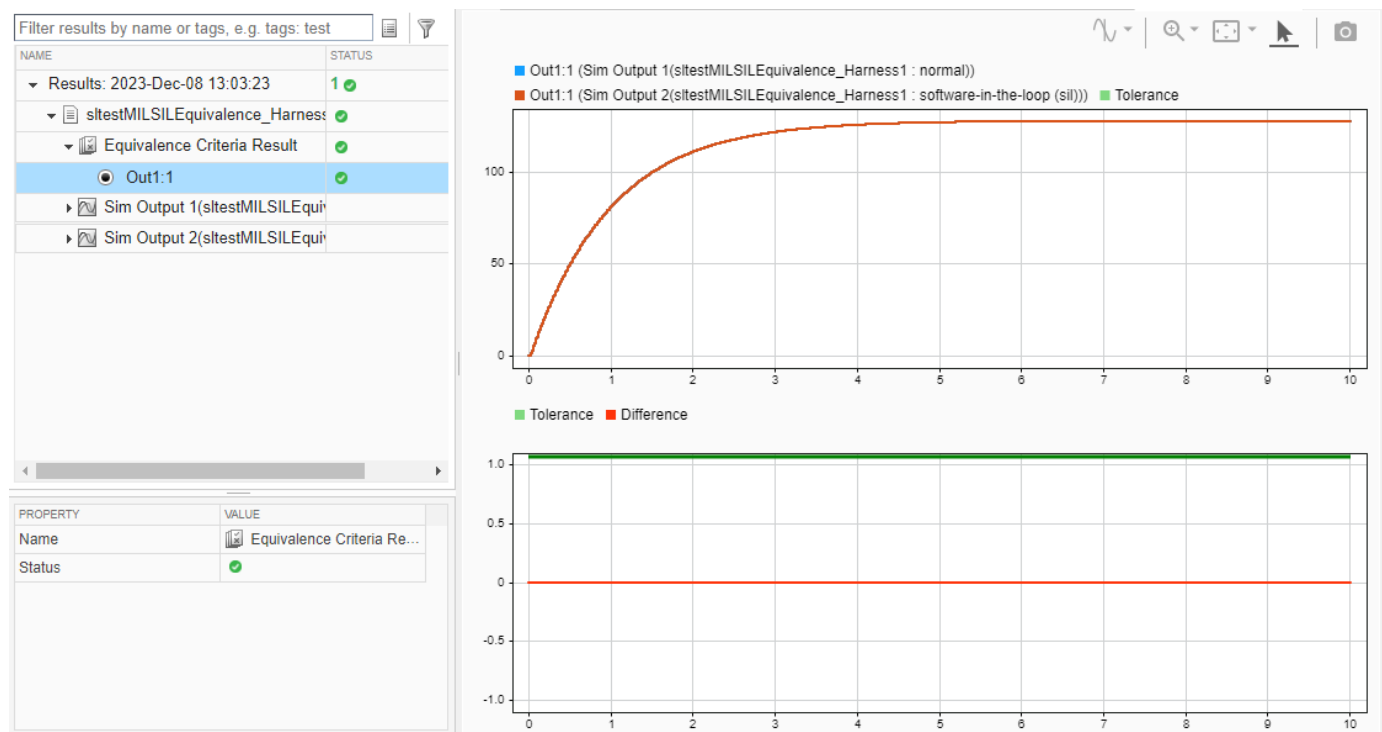
Simulation Mode:

Run the Test and View the Results

1. Click **Run** in the Test Manager toolbar.
2. When the test run completes, in the **Results and Artifacts** pane, select **Results** and view the **Aggregated Coverage Results** section. Both the Normal and SIL simulations have 100% coverage, which indicates that testing is complete for the selected model coverage metrics.

▼ AGGREGATED COVERAGE RESULTS					
Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.					
ANALYZED MODEL	REPORT	SIM MODE	COM...	EXECUTION	FUNCTION
 sltestMILSILEquivalence/Controller		Normal	1	100%	--
 sltestMILSILEquivalence/Controller		SIL	14	100%	100%

3. Expand the **Results** hierarchy and select **Out1:1** under **Equivalence Criteria Result**. The difference between the Normal and SIL simulations is zero, which shows that both simulations produce the same results.



Close the Model and Clear and Close the Test Manager

```
close_system(model,0)
sltest.testmanager.clear
sltest.testmanager.clearResults
sltest.testmanager.close
```

See Also

More About

- “Back-to-Back Equivalence Testing” on page 6-36

Testing AUTOSAR Compositions

This example shows how to run back-to-back tests on an AUTOSAR composition model.

This example demonstrates test harness features and back-to-back testing workflows for an AUTOSAR composition model. Switch to a directory with write permissions.

The example uses a model of a throttle position controller for an automobile. It is based closely on a shipping AUTOSAR Blockset example. For details, see “Import AUTOSAR Composition to Simulink” (AUTOSAR Blockset).

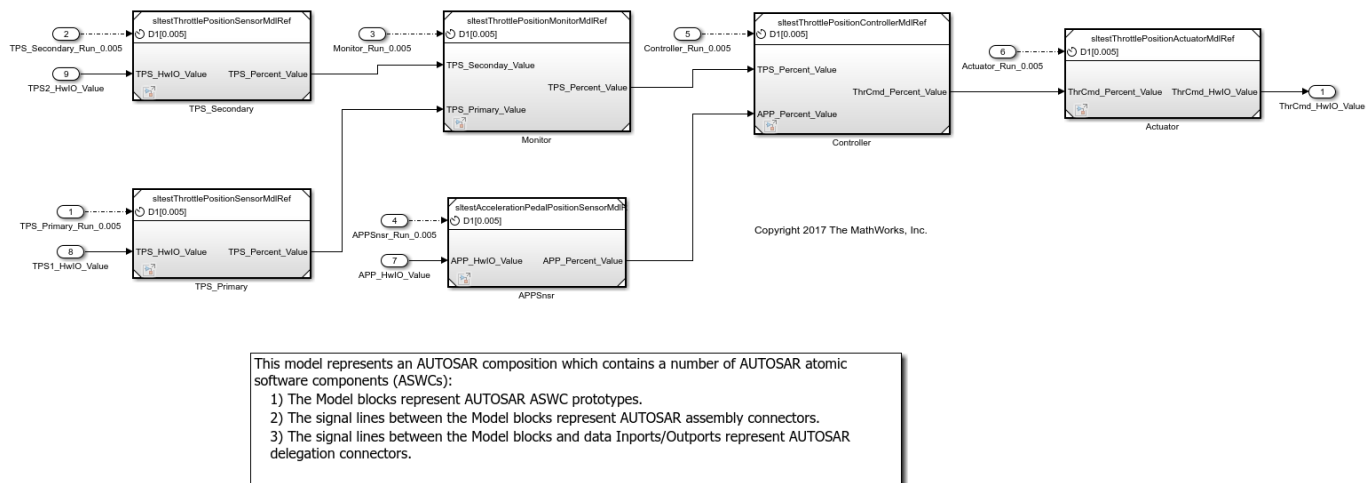
Open the AUTOSAR Composition Model

AUTOSAR composition models contain a network of interconnected Model blocks, each of which represents an atomic AUTOSAR software component (ASWC). The throttle position controller composition was created by an AUTOSAR authoring tool (AAT) and imported into Simulink using an ARXML file that describes the composition.

The composition model contains six component models, one for each atomic software component in the composition. Simulink inports and outports represent AUTOSAR ports and signal lines represent AUTOSAR component connectors.

```
mdl = 'sltestThrottlePositionControlCompositionExample.slx';
open_system(mdl);
```

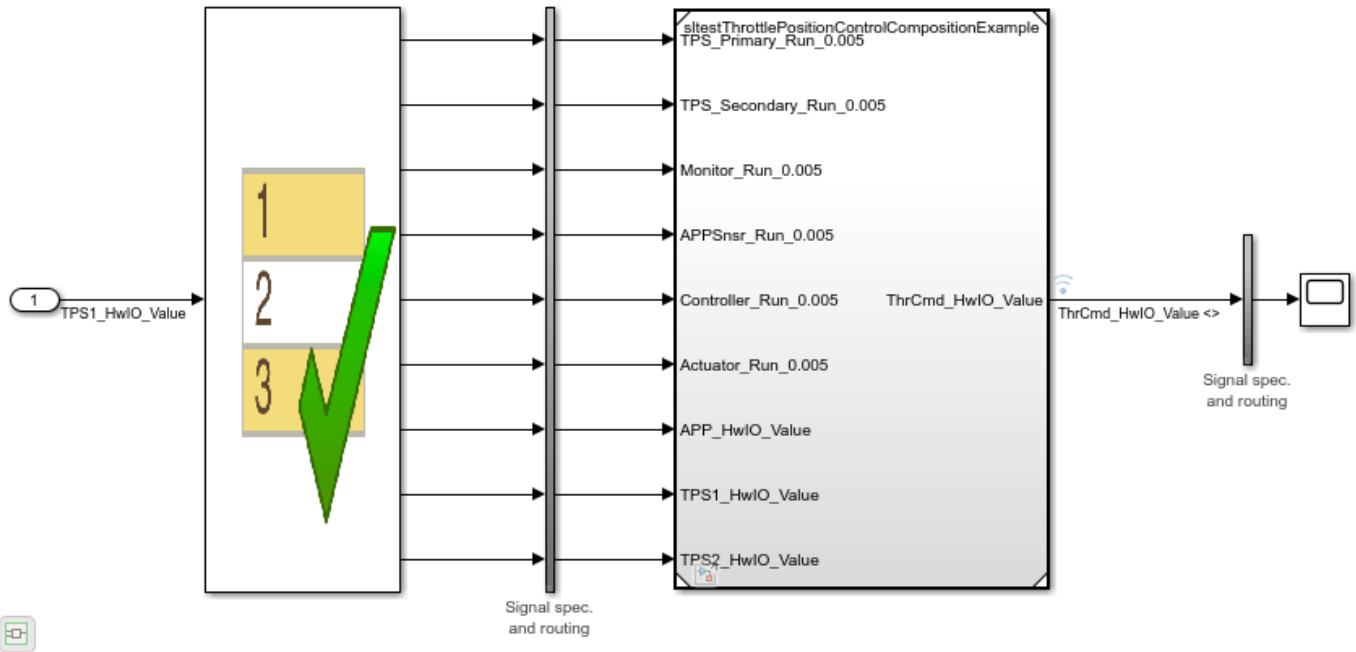
Testing AUTOSAR Compositions



Open Test Harness

A test harness for the model has been generated and can be opened using the perspective control in the lower right corner of the editor canvas. Alternately, use:

```
sltest.harness.open('sltestThrottlePositionControlCompositionExample',...
    'BasicSchedulerTest');
```



A Test Sequence block is used as the source. The component under test requires the accelerator pedal position sensor input APP_HwIO_Value, which is modeled in the Test Sequence block using a simple three step sequence:

BasicSchedulerTest/Test Sequence - Test Sequence Editor				
Symbols	Step	Transition	Next Step	Description
Input 1. TPS_HwIO_In Output 1. APP_HwIO_Vali 2. TPS1_HwIO_Value 3. TPS2_HwIO_Ve 4. fx TPS_Primary_F 5. fx TPS_Secondary_Rur 6. fx Monitor_Run_ 7. fx APPSnsr_Run_0005 8. fx Controller_Run_000 9. fx Actuator_Run_0005 Local Constant Parameter Data Store Memory	Initialize <pre>%% Initialize data outputs. APP_HwIO_Value = uint16(170); TPS1_HwIO_Value = TPS_HwIO_In; TPS2_HwIO_Value = TPS_HwIO_In; %% Schedule function-call outputs. send(TPS_Primary_Run_0005) send(TPS_Secondary_Run_0005) send(Monitor_Run_0005) send(APPSnsr_Run_0005) send(Controller_Run_0005) send(Actuator_Run_0005)</pre>	1. after(0.05,sec)	Run ▼	Initialize the acceleration pedal position command to a nominal value and hold for 50 ms.
	Run <pre>%% Initialize data outputs. APP_HwIO_Value = uint16(681); TPS1_HwIO_Value = TPS_HwIO_In; TPS2_HwIO_Value = TPS_HwIO_In; %% Schedule function-call outputs. send(TPS_Primary_Run_0005) send(TPS_Secondary_Run_0005) send(Monitor_Run_0005) send(APPSnsr_Run_0005) send(Controller_Run_0005) send(Actuator_Run_0005)</pre>	1. after(0.95, sec)	Terminate ▼	Start the test by applying a steady acceleration command for 950 ms.
	Terminate <pre>%% Initialize data outputs. APP_HwIO_Value = uint16(170); TPS1_HwIO_Value = TPS_HwIO_In; TPS2_HwIO_Value = TPS_HwIO_In; %% Schedule function-call outputs. send(TPS_Primary_Run_0005) send(TPS_Secondary_Run_0005) send(Monitor_Run_0005) send(APPSnsr_Run_0005) send(Controller_Run_0005) send(Actuator_Run_0005)</pre>			Reset the acceleration command to the nominal value and terminate the test.

The **Initialize** step sets the input to a nominal value and the **Run** step models a steady acceleration command for 950 ms. The acceleration command is reset to the nominal value in the **Terminate** step. The component under test requires two additional inputs that capture the primary and secondary throttle position sensor readouts. These inputs are modeled using an external time series input and are directly fed through the Test Sequence block without modification. This modeling style is useful when some stimulus inputs can be modeled and others are only available as externally captured data.

Test Harnesses for Export Functions

The component under test is the AUTOSAR composition model, which uses the export-function modeling style. When you create a test harness for an export-function model, the harness will contain a Test Sequence block configured to call each root-level Simulink Function block and send a trigger event to each function-call subsystem in the model. The generated Test Sequence block can be used as a convenient starting point for modeling a scheduler.

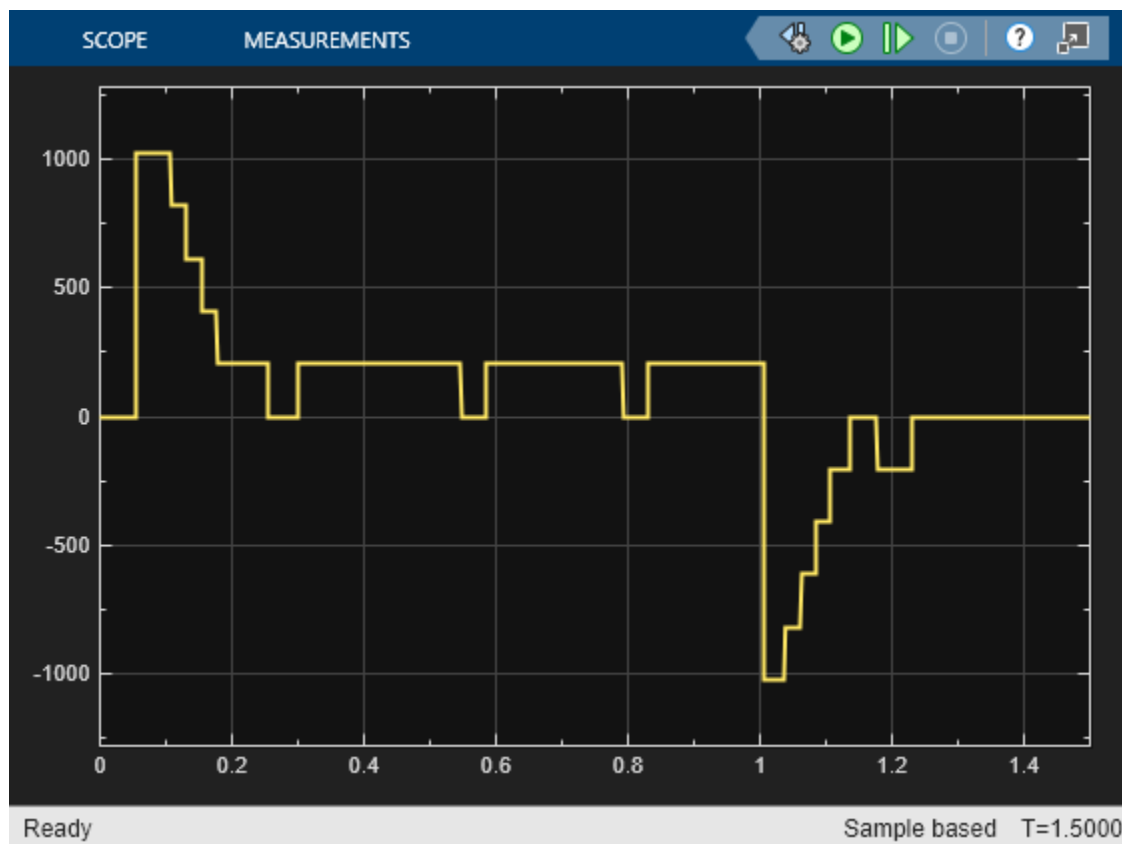
In this example, since the input signal data is also being generated by a Test Sequence source block, the code to send the trigger events has been consolidated into a single Test Sequence block and

embedded in each step after the stimulus waveforms have been generated. The call order of the trigger events are computed using compiled information from the composition model.

```
1 send(TPS_Primary_Run_0005)
2 send(TPS_Secondary_Run_0005)
3 send(Monitor_Run_0005)
4 send(APPSnsr_Run_0005)
5 send(Controller_Run_0005)
6 send(Actuator_Run_0005)
```

Simulate the model to see the throttle command output from the component under test.

```
sim('BasicSchedulerTest');
open_system('BasicSchedulerTest/Scope');
```



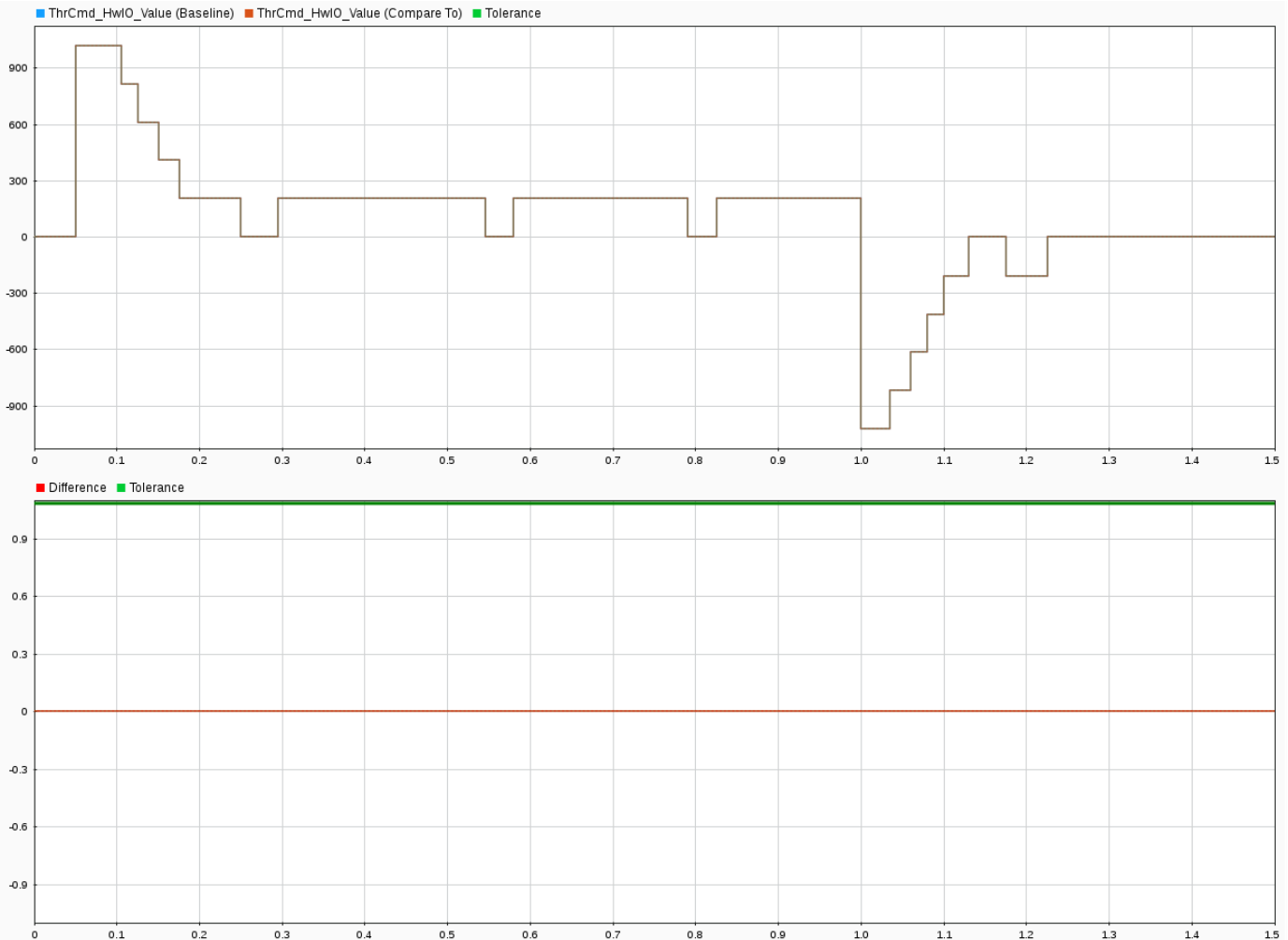
Back-to-back Testing

The test manager can be used to lock down simulation behavior and verify equivalence in software-in-the-loop (SIL) mode. Open the test file and run the equivalence test.

```
close_system mdl,0);
file_mldatx = 'sltestThrottlePositionControlTests.mldatx';
open(file_mldatx);
sltest.testmanager.run;
```

The test case verifies the open-loop behavior of the Throttle Position Controller ASWC within the composition model. The first part of the equivalence test case runs the test harness containing the composition in normal simulation mode. The second part of the test uses the Post-Load callback to switch the Throttle Position Controller ASWC to software-in-the-loop (SIL) mode with Top model code interface. The results of both simulations show that the behavior is equivalent.

Note that since SIL mode is set in a callback, the results do not show the second simulation as actually having been run in SIL mode.



Cleanup

```
clear sltestThrottlePositionControlData HBridgeCmd_LkupTbl ...
    SensorSelection SetpointPercent_LkupTbl TPSPPrimaryPercent_LkupTbl...
    TPSSSecondaryPercent_LkupTbl TPSPercent_LkupTbl tout logsout mdl file_mldatx;
sltest.testmanager.clear;
sltest.testmanager.clearResults;
sltest.testmanager.close;
```

Automate Testing for Highway Lane Following

This example shows how to assess the functionality of a lane-following application by defining scenarios based on requirements, automating testing of components and the generated code for those components. The components include lane-detection, sensor fusion, decision logic, and controls. This example builds on the “Highway Lane Following” (Automated Driving Toolbox) example.

Introduction

A highway lane-following system steers a vehicle to travel within a marked lane. It also maintains a set velocity or safe distance from a preceding vehicle in the same lane. The system typically includes lane detection, sensor fusion, decision logic, and controls components. System-level simulation is a common technique for assessing functionality of the integrated components. Simulations are configured to test scenarios based on system requirements. Automatically running these simulations enables regression testing to verify system-level functionality.

The “Highway Lane Following” (Automated Driving Toolbox) example showed how to simulate a system-level model for lane-following. This example shows how to automate testing that model against multiple scenarios using Simulink Test™. The scenarios are based on system-level requirements. In this example, you will:

- 1 **Review requirements:** The requirements describe system-level test conditions. Simulation test scenarios are created to represent these conditions.
- 2 **Review the test bench model:** Review the system-level lane-following test bench model that contains metric assessments. These metric assessments integrate the test bench model with Simulink Test for the automated testing.
- 3 **Disable runtime visualizations:** Runtime visualizations are disabled to reduce execution time for the automated testing.
- 4 **Automate testing:** A test manager is configured to simulate each test scenario, assess success criteria, and report results. The results are explored dynamically in the test manager and exported to a PDF for external reviewers.
- 5 **Automate testing with generated code:** The lane detection, sensor fusion, decision logic, and controls components are configured to generate C++ code. The automated testing is run on the generated code to verify expected behavior.
- 6 **Automate testing in parallel:** Overall execution time for running the tests is reduced using parallel computing on a multi-core computer.

Testing the system-level model requires a photorealistic simulation environment. In this example, you enable system-level simulation through integration with the Unreal Engine from Epic Games®. This example requires a Windows® 64-bit platform.

```
if ~ispc
    error(['This example is supported only on Microsoft', char(174), ' Windows 64-bit platform.'])
end
```

To ensure reproducibility of the simulation results, set the random seed.

```
rng(0);
```


Review Requirements

Requirements Toolbox™ lets you author, analyze, and manage requirements within Simulink. This example contains ten test scenarios, with high-level testing requirements defined for each scenario. Open the requirement set.

To explore the test bench model, load the highway lane following project.

```
openProject("HighwayLaneFollowing");
```

```
open('HighwayLaneFollowingTestRequirements.slsreq')
```

Alternatively, you can also open the file from the **Requirements** tab of the Requirements Manager app in Simulink.

The screenshot shows the Requirements Editor interface. On the left, a tree view lists ten requirements under the 'HighwayLaneFollowingTestRequirements' set. Requirement 3, 'scenario_LFACC_03_Curve_StopnGo', is selected. The main panel displays the details for Requirement 3, including its properties (Type: Functional, Index: 3, Custom ID: 3, Summary: scenario_LFACC_03_Curve_StopnGo) and a table of test descriptions, target vehicles, and requirements on the ego vehicle.

Test Description	Target vehicles	Requirements on Ego vehicle
Stop and go test in curved road 	Lead vehicle: Initial velocity: 13.6m/s Headway: 50m Event: Lead vehicle travels with initial velocity for 4s, then slows down to 8m/s and after 10s increase its velocity to 13m/s in ego lane. Slow moving vehicle - 1: Set velocity: 8m/s Headway: 1100m Lane: Adjacent to ego lane Slow moving vehicle - 2:	Initial velocity: 14m/s Lateral deviation < 0.45m Time gap > 0.8s Expected behavior: Ego vehicle should first decelerate when lead vehicle slows down and then accelerate when lead vehicle increase its velocity such that it maintains safety time gap from lead vehicle while following the lanes.

Below the table, there is a 'Keywords' field, a 'Revision information' section, and a 'Links' section showing a link to 'scenario_LFACC_03_Curve_StopnGo'.

Each row in this file specifies the requirements in textual and graphical formats for testing the lane-following system for a test scenario. The scenarios with the **scenario_LF** prefix enable you to test lane-detection and lane-following algorithms without obstruction by other vehicles. The scenarios with the **scenario_LFACC** prefix enable you to test lane-detection, lane-following, and ACC behavior with other vehicles on the road.

- 1 **scenario_LF_01_Straight_RightLane** — Straight road scenario with ego vehicle in right lane.

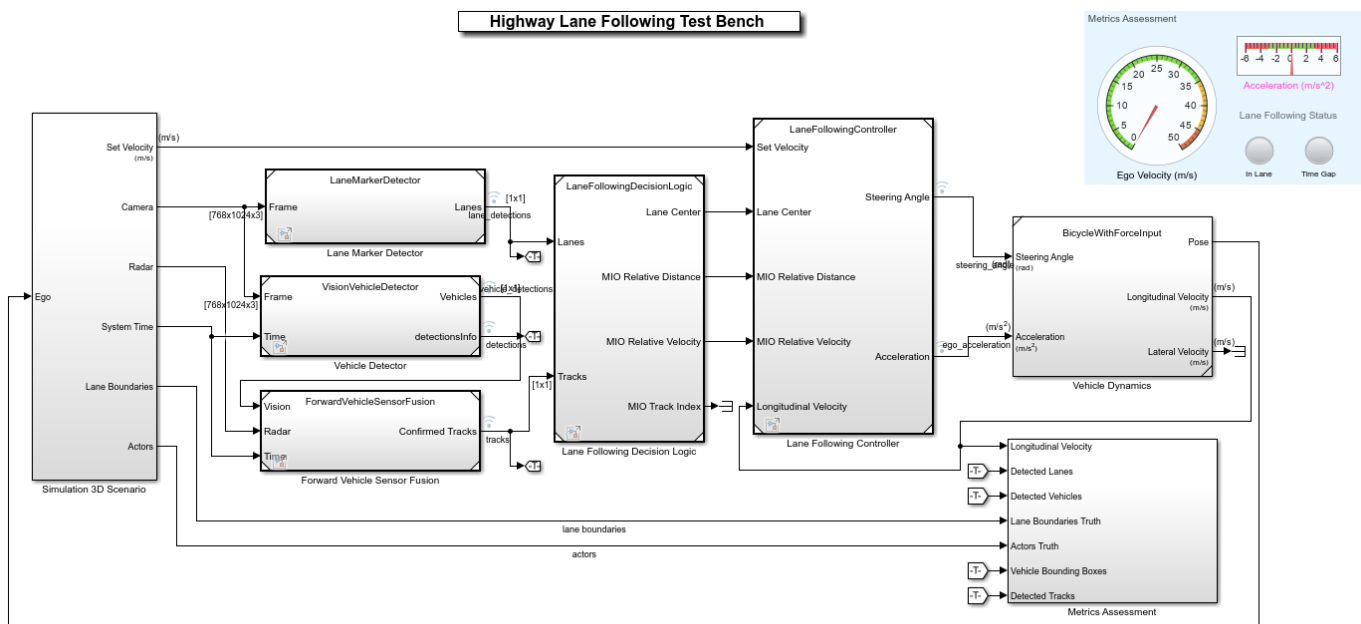
- 2 scenario_LF_02_Straight_LeftLane — Straight road scenario with ego vehicle in left lane.
- 3 scenario_LF_03_Curve_LeftLane — Curved road scenario with ego vehicle in left lane.
- 4 scenario_LF_04_Curve_RightLane — Curved road scenario with ego vehicle in right lane.
- 5 scenario_LFACC_01_Curve_DecelTarget — Curved road scenario with a decelerating lead vehicle in ego lane.
- 6 scenario_LFACC_02_Curve_AutoRetarget — Curved road scenario with changing lead vehicles in ego lane. This scenario tests the ability of the ego vehicle to retarget to a new lead vehicle while driving along a curve.
- 7 scenario_LFACC_03_Curve_StopnGo — Curved road scenario with a lead vehicle slowing down in ego lane.
- 8 scenario_LFACC_04_Curve_CutInOut — Curved road scenario with a fast moving car in the adjacent lane cuts into the ego lane and cuts out from ego lane.
- 9 scenario_LFACC_05_Curve_CutInOut_TooClose — Curved road scenario with a fast moving car in the adjacent lane cuts into the ego lane and cuts out from ego lane aggressively.
- 10 scenario_LFACC_06_Straight_StopandGoLeadCar — Straight road scenario with a lead vehicle that breaks down in ego lane.

These requirements are implemented as test scenarios with the same names as the scenarios used in the HighwayLaneFollowingTestBench model.

Review Test Bench Model

This example reuses the HighwayLaneFollowingTestBench model from the “Highway Lane Following” (Automated Driving Toolbox) example. Open the test bench model.

```
open_system("HighwayLaneFollowingTestBench");
```



Copyright 2019-2023 The MathWorks, Inc.

This test bench model has **Simulation 3D Scenario**, **Lane Marker Detector**, **Vehicle Detector**, **Forward Vehicle Sensor Fusion**, **Lane Following Decision Logic** and **Lane Following Controller** and **Vehicle Dynamics** components.

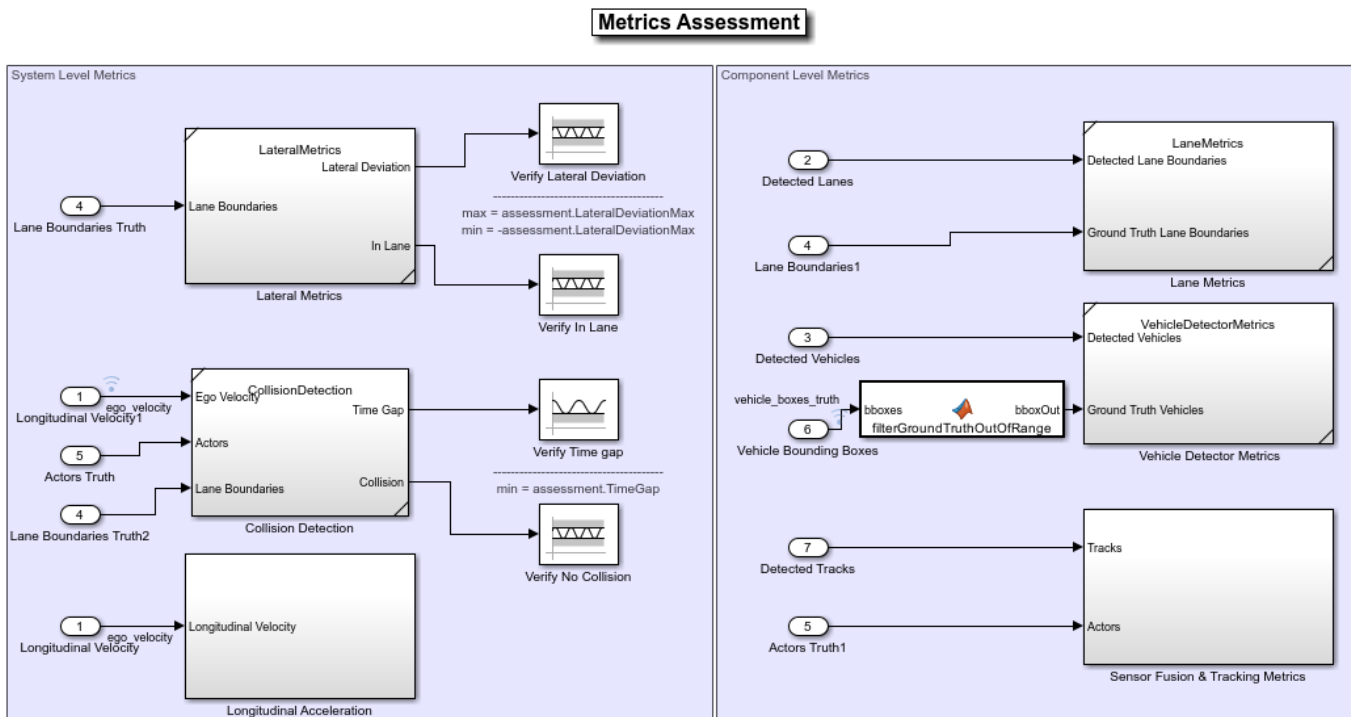
This test bench model is configured using the `helperSLHighwayLaneFollowingSetup` script. This setup script takes `scenarioName` as input. `scenarioName` can be any one of the previously described test scenarios. To run the setup script, use code:

```
scenarioName = "scenario_LFACC_03_Curve_StopnGo";
helperSLHighwayLaneFollowingSetup("scenarioFcnName", scenarioName);
```

You can now simulate the model and visualize the results. For more details on the analysis of the simulation results and the design of individual components in the test bench model, see the “Highway Lane Following” (Automated Driving Toolbox) example.

In this example, the focus is more on automating the simulation runs for this test bench model using Simulink Test for the different test scenarios. The **Metrics Assessment** subsystem enables integration of system-level metric evaluations with Simulink Test. Open the **Metrics Assessment** subsystem.

```
open_system("HighwayLaneFollowingTestBench/Metrics Assessment");
```



Using this example, you can evaluate the system-level behavior using four system-level metrics. Additionally, you can also compute component-level metrics to analyze individual components and their impact on the overall system performance.

System-Level Metrics

- **Verify Lateral Deviation** — This block verifies that the lateral deviation from the center line of the lane is within prescribed thresholds for the corresponding scenario. Define the thresholds when you author the test scenario.
- **Verify In Lane** — This block verifies that the ego vehicle is following one of the lanes on the road throughout the simulation.
- **Verify Time gap** — This block verifies that the time gap between the ego vehicle and the lead vehicle is more than 0.8 seconds. The time gap between the two vehicles is defined as the ratio of the calculated headway distance to the ego vehicle velocity.
- **Verify No Collision** — This block verifies that the ego vehicle does not collide with the lead vehicle at any point during the simulation.

Component-Level Metrics

- **Lane Metrics** — This block verifies that distances between the detected lane boundaries and the ground truth data are within the thresholds specified in a test scenario.
- **Vehicle Detector Metrics** — This blocks computes and logs true positives, false negatives, and false positives for the detections.
- **Sensor Fusion & Tracking Metrics** — This subsystem computes generalized optimal subpattern assignment (GOSPA) metric, localization error, missed target error, and false track error. For more information on these metrics, see “Forward Vehicle Sensor Fusion” (Automated Driving Toolbox) example.

Disable Runtime Visualizations

The system-level test bench model visualizes intermediate outputs during the simulation for the analysis of different components in the model. These visualizations are not required when the tests are automated. You can reduce execution time for the automated testing by disabling them.

Disable runtime visualizations for the **Lane Marker Detector** subsystem.

```
load_system('LaneMarkerDetector');  
blk = 'LaneMarkerDetector/Lane Marker Detector';  
set_param(blk, 'EnableDisplays', 'off');
```

Disable runtime visualizations for the **Vehicle Detector** subsystem.

```
load_system('VisionVehicleDetector');  
blk = 'VisionVehicleDetector/Pack Detections/Pack Vehicle Detections';  
set_param(blk, 'EnableDisplay', 'off');
```

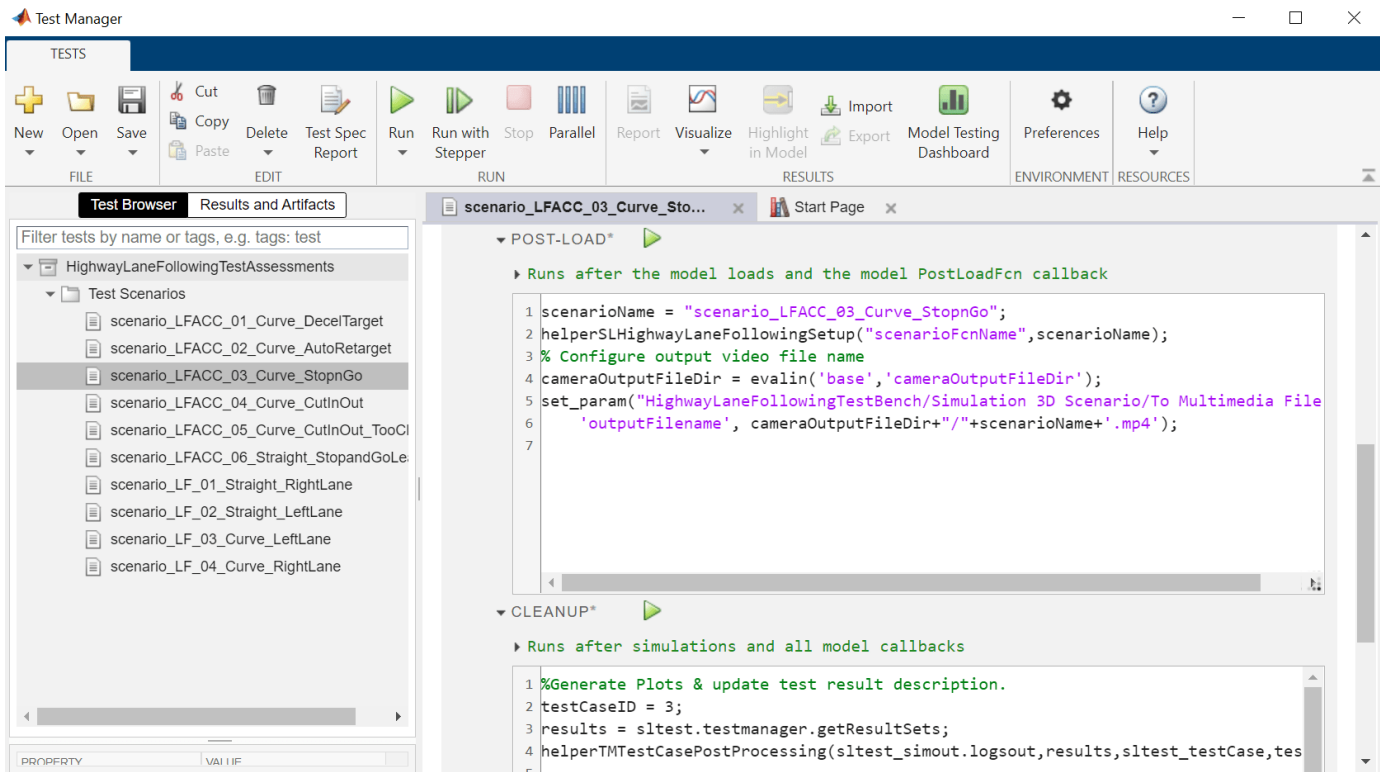
Configure the Simulation 3D Scene Configuration (Automated Driving Toolbox) block to run the Unreal Engine in headless mode, where the 3D simulation window is disabled.

```
blk = ['HighwayLaneFollowingTestBench/Simulation 3D Scenario/', ...  
      'Simulation 3D Scene Configuration'];  
set_param(blk, 'EnableWindow', 'off');
```

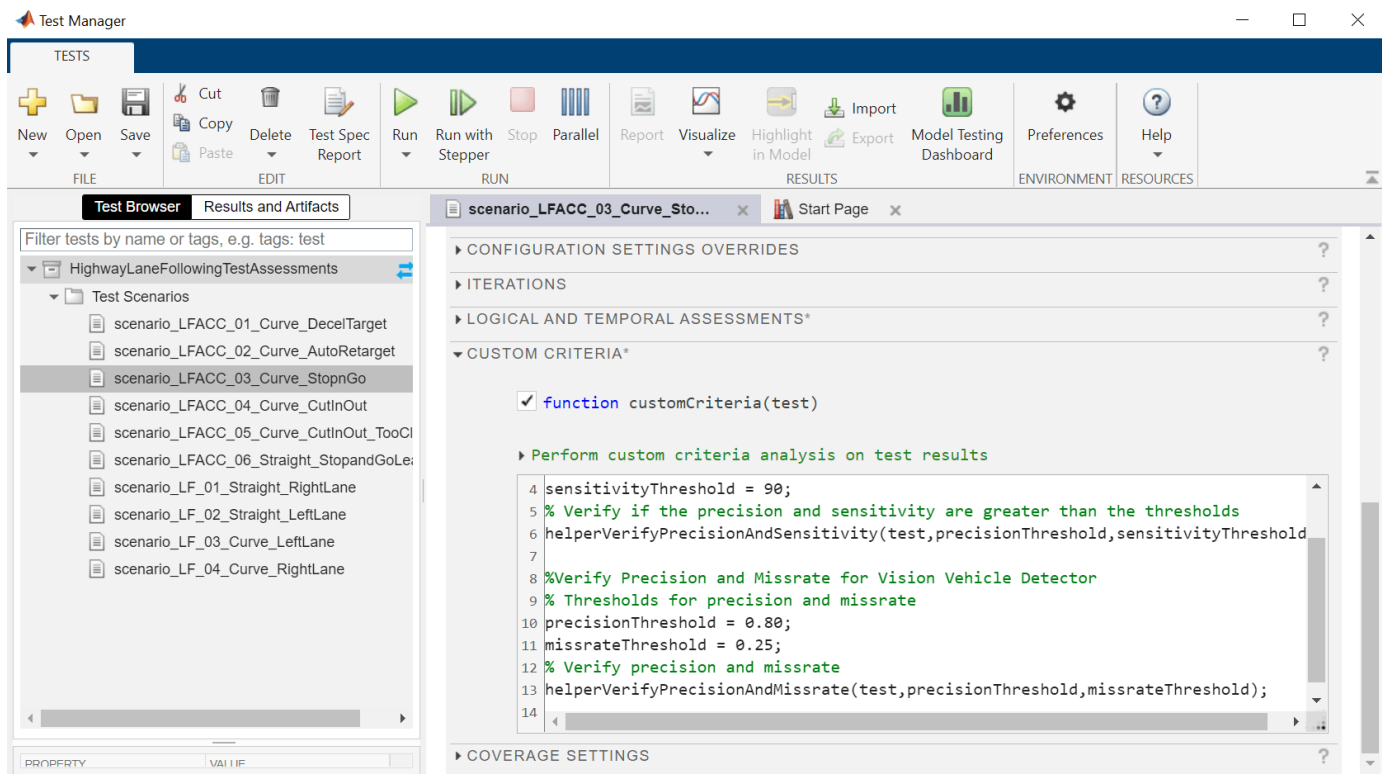
Automate Testing

The Test Manager is configured to automate the testing of the lane-following application. Open the HighwayLaneFollowingTestAssessments.mldatx test file in the Test Manager.

```
sltestmgr;  
testFile = sltest.testmanager.load('HighwayLaneFollowingTestAssessments.mldatx');
```



Observe the populated test cases that were authored previously in this file. Each test case is linked to the corresponding requirement in the Requirements Editor for traceability. Each test case uses the POST-LOAD callback to run the setup script with appropriate inputs and to configure the output video filename. After the simulation of the test case, it invokes `helperTMTTestCasePostProcessing` from the CLEAN-UP callback to assess performances of the overall system and individual components by generating the plots explained in the “Highway Lane Following” (Automated Driving Toolbox) example.



After simulation of the test case, Simulink Test also invokes these functions from the CUSTOM CRITERIA callback to get additional metrics for lane marker detector and vehicle detector components:

- `helperVerifyPrecisionAndSensitivity` — Verifies that the precision and sensitivity metrics of the lane marker detector component are within the predefined threshold limit.
- `helperVerifyPrecisionAndMissrate` — Verifies that the precision and miss rate metrics of the vehicle detector component are within the predefined threshold limit.

Run and explore results for a single test scenario:

To reduce command-window output, turn off the MPC update messages.

```
mpcverbosity('off');
```

To test the system-level model with the `scenario_LFACC_03_Curve_StopnGo` test scenario from Simulink Test, use this code:

```
testSuite = getTestSuiteByName(testFile, 'Test Scenarios');
testCase = getTestCaseByName(testSuite, 'scenario_LFACC_03_Curve_StopnGo');
resultObj = run(testCase);
```

To generate a report after the simulation, use this code:

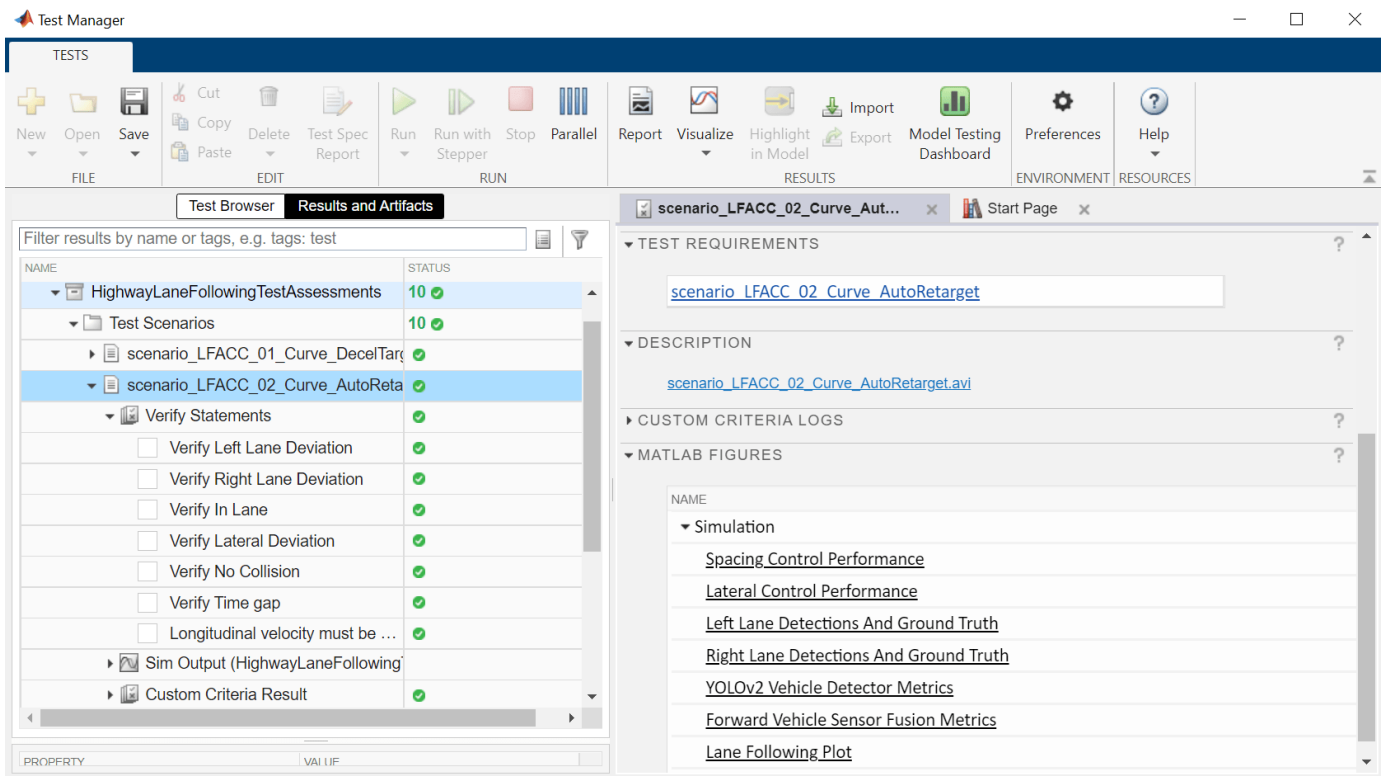
```
sltest.testmanager.report(resultObj, 'Report.pdf', ...,
    'Title', 'Highway Lane Following', ...,
    'IncludeMATLABFigures', true, ...,
    'IncludeErrorMessage', true, ...,
    'IncludeTestResults', 0, 'LaunchReport', true);
```

Examine the report `Report.pdf`. Observe that the **Test environment** section shows the platform on which the test is run and the MATLAB® version used for testing. The **Summary** section shows the outcome of the test and duration of the simulation in seconds. The **Results** section shows pass/fail results based on the assessment criteria. This section also shows the plots logged from the `helperGenerateFilesForLaneFollowingReport` function.

Run and explore results for all test scenarios:

You can simulate the system for all the tests by using `sltest.testmanager.run`. Alternatively, you can simulate the system by clicking **Play** in the Test Manager app.

After completion of the test simulations, the results for all the tests can be viewed in the **Results and Artifacts** tab of the Test Manager. For each test case, the Check Static Range blocks in the model are associated with the Test Manager to visualize overall pass/fail results.



You can find the generated report in current working directory. This report contains a detailed summary of pass/fail statuses and plots for each test case.

Summary

Name	Outcome	Duration (Seconds)
 HighwayLaneFollowingTestAssessments	10 	2149.277
 Test Scenarios	10 	2149.278
 scenario LFACC 01 Curve DecelTarget		448.803
 scenario LFACC 02 Curve AutoRetarget		218.906
 scenario LFACC 03 Curve StopnGo		268.746
 scenario LFACC 04 Curve CutInOut		200.885
 scenario LFACC 05 Curve CutInOut TooClose		237.335
 scenario LFACC 06 Straight StopandGoLeadCar		128.585
 scenario LF 01 Straight RightLane		157.953
 scenario LF 02 Straight LeftLane		145.923
 scenario LF 03 Curve LeftLane		177.465
 scenario LF 04 Curve RightLane		164.469

Verify test status in Requirements Editor:

Open the Requirements Editor and select **Display**. Then, select **Verification Status** to see a verification status summary for each requirement. Green and red bars indicate the pass/fail status of simulation results for each test.

Requirements Editor

REQUIREMENTS

FILE REQUIREMENTS LINKS VIEW EDIT ANALYSIS SHARE

New Requirement Set Open Import Save Add Requirement Delete Promote Requirement (disabled because sort is enabled) Demote Requirement (disabled because sort is enabled) Add Link Delete Clear Issue Show Requirements Show Links Columns Refresh Search Traceability Matrix Traceability Diagram Export

Index	Summary	Verified
1	scenario_LFACC_01_Curve_DecelTarget	
2	scenario_LFACC_02_Curve_AutoRetarget	
3	scenario_LFACC_03_Curve_StopnGo	
4	scenario_LFACC_04_Curve_CutInOut	
5	scenario_LFACC_06_Straight_StopandGoLead...	
6	scenario_LFACC_05_Curve_CutInOut_TooClose	
7	scenario_LF_01_Straight_RightLane	
8	scenario_LF_02_Straight_LeftLane	
9	scenario_LF_03_Curve_LeftLane	
10	scenario_LF_04_Curve_RightLane	

Requirement: 3

Details

▼ Properties

Type: Functional

Index: 3

Custom ID: 3

Summary: scenario_LFACC_03_Curve_StopnGo

Description Rationale

Test Description	Target vehicles	Requirements on Ego vehicle
Stop and go test in curved road 	Lead vehicle: Initial velocity: 13.6m/s Headway: 50m Event: Lead vehicle travels with initial velocity for 4s, then slows down to 8m/s and after 10s increase its velocity to 13m/s in ego lane. Slow moving vehicle - 1: Set velocity: 8m/s Headway: 1100m Lane: Adjacent to ego lane Slow moving vehicle - 2:	Initial velocity: 14m/s Lateral deviation < 0.45m Time gap > 0.8s Expected behavior: Ego vehicle should first decelerate when lead vehicle slows down and then accelerate when lead vehicle increase its velocity such that it maintains safety time gap from lead vehicle while following the lanes.

Keywords:

Revision information:

▼ Links

Verified by:

scenario_LFACC_03_Curve_StopnGo

Automate Testing with Generated Code

The HighwayLaneFollowingTestBench model enables integrated testing of **Lane Marker Detector**, **Vehicle Detector**, **Forward Vehicle Sensor Fusion**, **Lane Following Decision Logic**, and **Lane Following Controller** components. It is often helpful to perform regression testing of these components through software-in-the-loop (SIL) verification. If you have Embedded Coder™ Simulink Coder™ license, then you can generate code for these components. This workflow lets you verify that the generated code produces expected results that match the system-level requirements throughout simulation.

Set **Lane Marker Detector** to run in Software-in-the-loop mode.

```
model = 'HighwayLaneFollowingTestBench/Lane Marker Detector';
set_param(model, 'SimulationMode', 'Software-in-the-loop');
```

Set **Vehicle Detector** to run in Software-in-the-loop mode.

```
model = 'HighwayLaneFollowingTestBench/Vehicle Detector';
set_param(model, 'SimulationMode', 'Software-in-the-loop');
```

Set **Forward Vehicle Sensor Fusion** to run in Software-in-the-loop mode.

```
model = 'HighwayLaneFollowingTestBench/Forward Vehicle Sensor Fusion';  
set_param(model, 'SimulationMode', 'Software-in-the-loop');
```

Set **Lane Following Decision Logic** to run in Software-in-the-loop mode.

```
model = 'HighwayLaneFollowingTestBench/Lane Following Decision Logic';  
set_param(model, 'SimulationMode', 'Software-in-the-loop');
```

Set **Lane Following Controller** to run in Software-in-the-loop mode.

```
model = 'HighwayLaneFollowingTestBench/Lane Following Controller';  
set_param(model, 'SimulationMode', 'Software-in-the-loop');
```

Now, run `sltest.testmanager.run` to simulate the system for all the test scenarios. After the completion of tests, review the plots and results in the generated report.

Enable the MPC update messages again.

```
mpcverbosity('on');
```

Automate Testing in Parallel

If you have a Parallel Computing Toolbox™ license, then you can configure Test Manager to execute tests in parallel using a parallel pool. To run tests in parallel, save the models after disabling the runtime visualizations using `save_system('LaneMarkerDetector')`, `save_system('VisionVehicleDetector')` and `save_system('HighwayLaneFollowingTestBench')`. Test Manager uses the default Parallel Computing Toolbox cluster and executes tests only on the local machine. Running tests in parallel can speed up execution and decrease the amount of time it takes to get test results. For more information on how to configure tests in parallel from the Test Manager, see “Run Tests Using Parallel Execution” on page 6-167.

See Also

Simulink Test Manager

Related Examples

- “Manage Requirements In the Requirements Editor and Property Inspector” (Requirements Toolbox)

Synchronize Tests

If you change the system under test, you can synchronize the test cases to reflect the model changes. Also, if you remove model components, you can disable or delete test cases in the Test Manager when you synchronize.

Synchronizing your test file automatically creates a new test case for:

- Each new scenario in the Signal Editor block at the top level of your model and the top level of each test harness. The model must have only one Signal Editor block at those levels to create a test case.
- Each new test harness in the model.

To synchronize your test file:

- 1 In the Test Manager **Test Browser** pane, hover over the test filename that you want to update.
- 2 Click the synchronization button  next to the test filename.
- 3 Follow the prompts to specify:
 - The type of test file to create for the new components
 - Whether to disable or delete out-of-date components

Disabled tests appear in the list in italic.

See Also

More About

- “Automatically Create a Set of Test Cases” on page 6-17

Use External File Data in Test Cases

In this section...

“Data Mapping” on page 6-76
 “Create a Test Case from an Excel Spreadsheet” on page 6-77
 “Import an Excel Spreadsheet into an Existing Test Case” on page 6-78
 “Add Multiple Microsoft Excel Spreadsheets as Input to a Test Case” on page 6-78
 “Include Microsoft Excel Test Data in Test Results” on page 6-79
 “Importing Test Inputs from Microsoft Excel” on page 6-79
 “Add a MAT-File or MATLAB Script (.m) File as External Input” on page 6-82
 “Using External Files Programmatically” on page 6-83

Test cases can use data, such as inputs, parameter overrides, expected outcomes, and baseline data, defined in MAT files, MATLAB script (.m) files, or Microsoft® Excel files. For information on how to create and format these files, see “Create External Data Files to Use in Test Cases” on page 6-84, and for Excel, see also “Microsoft Excel Import, Export, and Logging Format”.

In the Test Manager, you can

- Import baseline data and inputs from Excel and MAT files
- Import parameter values from Excel files, MAT files, and MATLAB scripts
- Export parameters to MAT files and MATLAB script (.m) files
- Add multiple external input files to a test case
- Assign one input file to each iteration

Data Mapping

Mapping Modes

To use external data, you map the data to your system under test (SUT) using these mapping modes:

- The names of the inport block the signal data corresponds to
- The full block pathname, in the form `system/block`
- The name of the signal associated with the inport block
- The port number, that is, sequential port numbers of the inport blocks, starting at 1

For more information about how Simulink handles inport mapping, see “Map Root Inport Signal Data”.

Mapping Status

When you map external inputs to model elements, the mapping creates these possible results. These results appear under **Inputs** in the Test Manager interface in the **Status** column:

- Mapped — The mapping succeeded and no further action is needed.

- **Failed** — The mapping failed. Click the **Failed** link for more information.
- **Warning** — The mapping occurred with warnings. Click the **Warning** link to see whether you need to address them
- **Stale** — This status can occur when you update your external inputs in Test Manager. A stale state occurs if you did not map the new inputs. To address this status, click the **Status** link, which opens the Add Input dialog box. Click **Map Inputs** to map the new input data and then click **Add**.

Create a Test Case from an Excel Spreadsheet

This example shows how to import an Excel spreadsheet to define a test case.

1. Open the Simulink Test Manager.

```
sltestmgr
```

2. Select **New > Test from Spreadsheet**. This opens the **Create Test from Spreadsheet** Wizard.
3. Select **Use existing test data from a spreadsheet**. Then, click the folder icon and from the working folder, select `coordinate_text.xlsx` as the spreadsheet. Click **Next**.
4. Enter `coordinate_transform_test` as the model. Click **Next**.
5. On the Attributes page, check that the attribute categories in the spreadsheet are displayed.
6. Click **Validate** to map each input to the model by block name. When the attributes are validated, the icons change to green. If necessary, go back to the Data or System tab to change the spreadsheet and/or system under test, click **Refresh**, and validate again. Click **Next**.
7. Specify a location to save the test file and click **Done**.

Simulink Test creates a new test case and imports the spreadsheet.

New Test Case 1

[mytestfile](#) » [New Test Suite 1](#) » [New Test Case 1](#)

Baseline Test

☒ Create Test Case from External File

File:



► TAGS

To see the data used from the spreadsheet, expand these sections:

- Parameter Overrides
- Inputs
- Baseline Criteria
- Table Iterations section under Iterations.

The fields defined in the spreadsheet are locked to the spreadsheet, and cannot be edited in the Test Manager. To see how the data is formatted or to edit the fields, open the `coordinate_text.xlsx` file outside of MATLAB.

If you cannot see all the data in a table column in a section, click **+** in the upper right corner to hide other columns and resize the desired column.

For multi-dimensional signals, each dimension is represented in a separate column in the spreadsheet. By default, only the dimensions with non-zero values are included. If all dimensions have zero value, then only the last dimension is included in the spreadsheet.

Import an Excel Spreadsheet into an Existing Test Case

If you have a test case and want to add test data to it from an Excel spreadsheet, you must associate the test case with the spreadsheet:

- 1 Open the test case.
- 2 Check the **Create Test Case from External File** option. You can use this option only for Excel spreadsheets.
- 3 Click the Browse files button  and select the spreadsheet that contains the test data.

The input, parameter, and comparison signal data in the spreadsheet override the data in the test case. The fields defined in the spreadsheet are locked to the spreadsheet.

To edit the spreadsheet, do one of the following:

- In the **Inputs** section, select the Excel external input file and click **Edit** to open the Excel spreadsheet.
- Edit and save the spreadsheet outside of MATLAB. In the Test Manager, next to the **File** field, click **Refresh**.

Add Multiple Microsoft Excel Spreadsheets as Input to a Test Case

You can import multiple Microsoft Excel spreadsheets at once and specify a range of data. Selecting sheets and specifying ranges is useful when each sheet contains a different data set or the same file contains input data and expected outputs.

- 1 In the test case, expand the **Inputs** section and click **Add**.
- 2 Browse to your Microsoft Excel file and click **Add**.
- 3 Select each sheet that contains input data. You can specify a range of data.
- 4 If you want to use each sheet to create an input set in the table, select **Create scenarios from each sheet**.
- 5 Under **Input Mapping**, select a mapping mode.
- 6 Click **Map Inputs**. The **Mapping Status** table shows the port and signal mapping.

For more information about troubleshooting the mapping, see “Understand Mapping Results”.

- 7 Click **Add**.

Include Microsoft Excel Test Data in Test Results

- 1 In the test case, expand the **Inputs** section and click **Include input data in test result**.
- 2 Under the **External Inputs** table, click **Add**.
- 3 In the Add Input dialog box, specify the Excel filename and the mapping mode, which specifies how to map the Excel data to root-level Inport blocks in the model.
- 4 Click **Map Inputs**. The **Mapping Status** table shows the port and signal mapping.
- 5 Click **Add**.

Importing Test Inputs from Microsoft Excel

Test a model using inputs stored in Microsoft® Excel®.

This example shows how to create a test case in the Test Manager and map inputs to the test case from a Microsoft® Excel® file. Input mapping supports Microsoft Excel spreadsheets only for Microsoft Windows®.

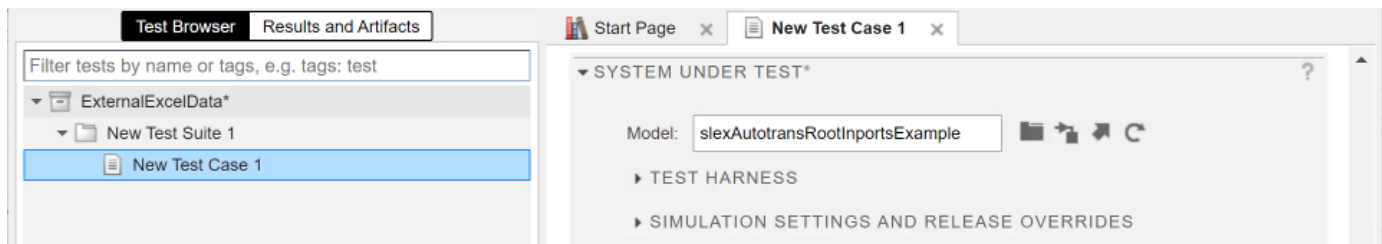
Create a Test File

1. Open the Test Manager.

```
sltest.testmanager.view
```

2. In the test manager toolbar, select **New > Test File**. Save the file to a writable directory. The test manager creates a test file with an empty baseline test case.

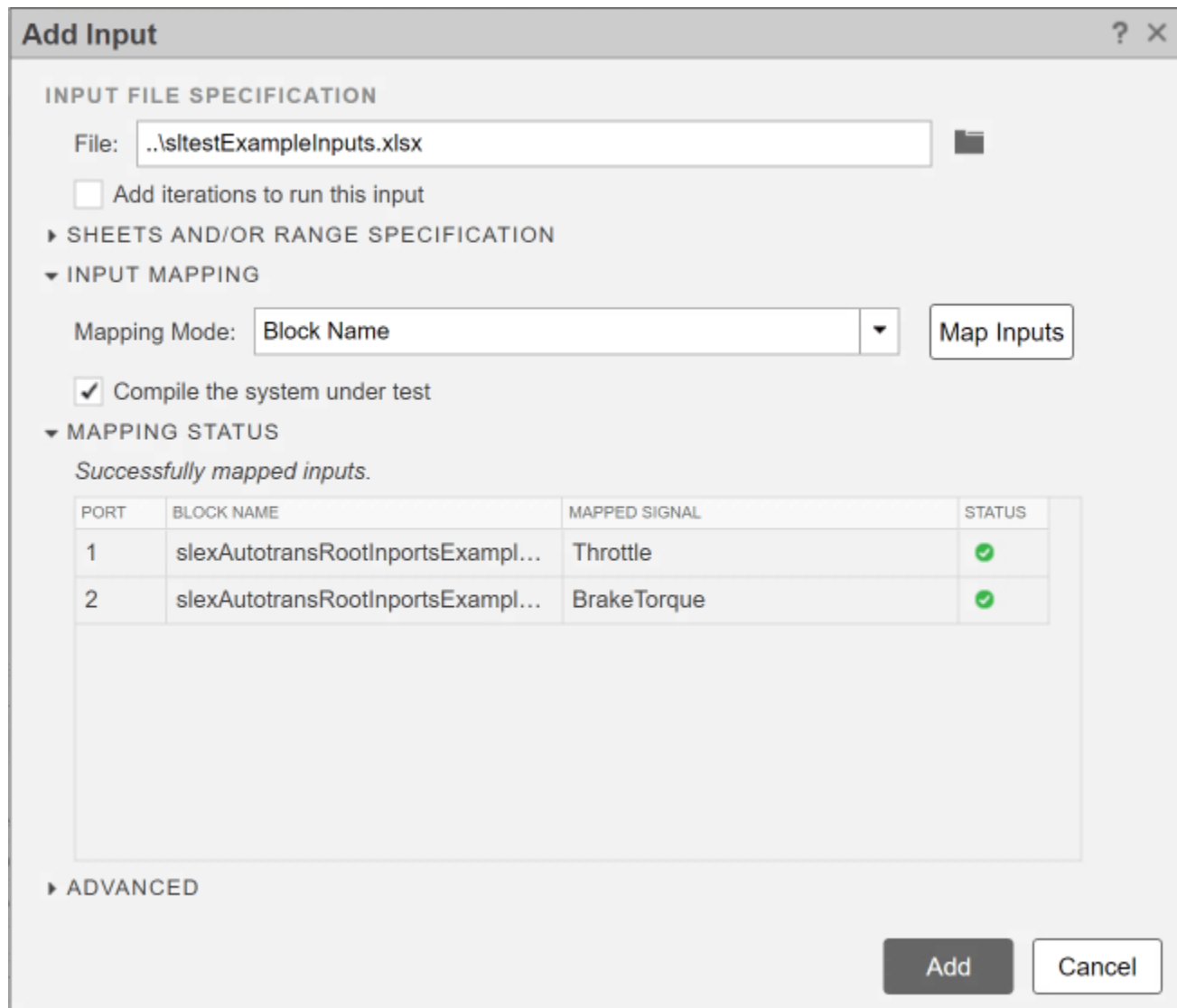
3. In the test browser, select the test case. In the test editor, under the **System Under Test** section, enter `sllexAutotransRootInportsExample`.



Configure the External Inputs

1. Expand the **Inputs** section of the test case.
2. To include the input data in the test results, click **Include input data in test result**.
3. Under the **External Inputs** table, click **Add**.
4. In the **Add Input** dialog box, for **File**, select the `sltestExampleInputs.xlsx` from the current directory. This file contains two tabs, named **Acceleration** and **Braking**. Each tab represents a complete set of inputs for a single simulation.
5. In the **Add Input** dialog box,

- Select the **Acceleration** sheet from the sheets table.
- Select **Mapping Mode** : Block Name.
- Click **Map Inputs**.
- Click **Add**.



Add Input ? X

INPUT FILE SPECIFICATION

File: 

☐ Add iterations to run this input

▶ **SHEETS AND/OR RANGE SPECIFICATION**

▼ **INPUT MAPPING**

Mapping Mode: ▼

☒ Compile the system under test

▼ **MAPPING STATUS**

Successfully mapped inputs.

PORT	BLOCK NAME	MAPPED SIGNAL	STATUS
1	slexAutotransRootInportsExempl...	Throttle	✓
2	slexAutotransRootInportsExempl...	BrakeTorque	✓

▶ **ADVANCED**

The **Mapping Mode** controls the method used to map data from the Microsoft Excel sheet to root-level Inport blocks in the model.

The test case shows the inputs mapped.

▼ INPUTS*

☒ Include input data in test result

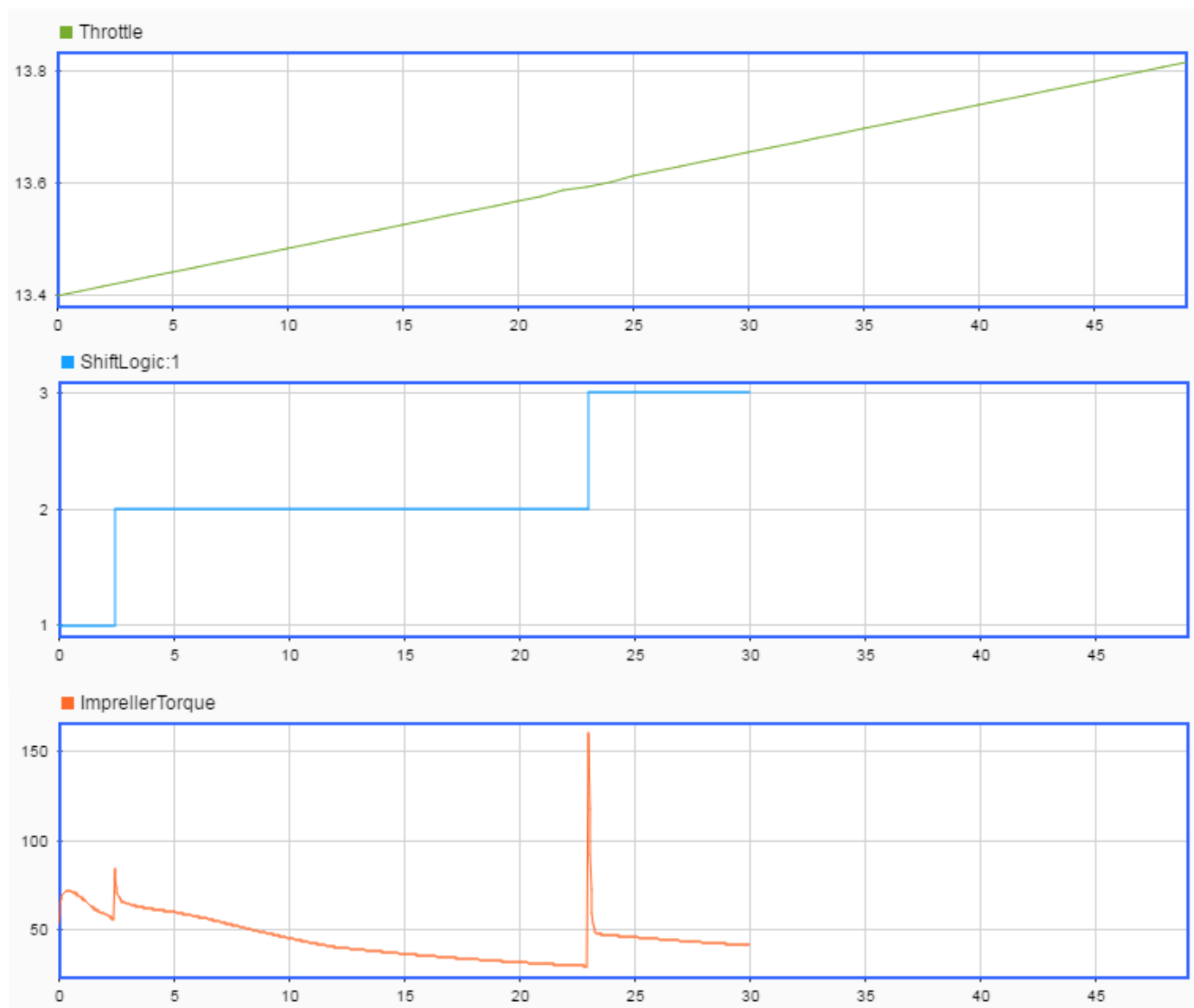
☐ Stop simulation at last time point

EXTERNAL INPUTS

NAME	FILE	SHEET	STATUS
☒ Acceleration	M:	Acceleration	Mapped

Run the Test

1. In the toolbar, click **Run**.
2. In the **Results and Artifacts** pane, you can plot signals from the external inputs or the simulation output.



Add a MAT-File or MATLAB Script (.m) File as External Input

- 1 In the test case, expand the **Inputs** section and click **Add**.
- 2 Browse to the MAT-file or MATLAB script (.m file) and click **Add**.
- 3 Under **Input Mapping**, choose a mapping mode.
- 4 Click **Map Inputs**. The **Mapping Status** table shows the port and signal mapping.

For information about troubleshooting the mapping status, see “Understand Mapping Results”.

- 5 Click **Add**.

You can also use an external MAT-file or MATLAB script (.m) file for baseline data and parameter overrides.

To create a MAT-file, see the `matfile` and `save` functions. Use a text editor to create a MATLAB script (.m) file.

Using External Files Programmatically

You can use MATLAB commands to specify the external file to use in a test case. See the `IsTestDataReferenced` and `TestDataPath` properties described in the `setProperty` method of `sltest.testmanager.TestCase`. For example,

```
tc = createTestCase(myTestSuite,'myTestCase');  
setProperty(tc,'Model','myTestModel');  
tc.setProperty('IsTestDataReferenced',true);  
tc.setProperty('TestDataPath','C:/myTestFiles/myData.xlsx');
```

See Also

`sltest.testmanager.TestInput` | `sltest.io.SimulinkTestSpreadsheet`

More About

- “Map Signal Data to Root Input Ports”
- “Map Root Inport Signal Data”
- “Create External Data Files to Use in Test Cases” on page 6-84
- “Override Model Parameters in a Test Case” on page 6-27

Create External Data Files to Use in Test Cases

You can create Microsoft Excel files, MAT files and MATLAB scripts for test case data, such as inputs, baseline data, and parameter overrides. The files use specific formats and can contain multiple types of data.

Note You can create files for input data only for tests that run in the current release. To select the current release in the Test Manager, in the **System Under Test** section of a test case, use **Select releases for simulation**.

- “Generate an Excel File” on page 6-84
- “Input, Baseline, and Parameter Override Test Case Data Formats in Excel” on page 6-89
- “Create a MAT File for Test Case Inputs” on page 6-91
- “MATLAB Script Format for Test Case Data” on page 6-92

Generate an Excel File

This example shows how to generate an Excel® file for a system under test (SUT). The generated spreadsheet extracts data from the model or test harness. If needed, you can edit the spreadsheet to add other data. Then, you can import the data from the file into Simulink® Test™ as a test case.

To extract data from the model, use the Create Test from Spreadsheet wizard. The wizard parses the SUT for test attributes and generates a template spreadsheet and a test case with these types of data:

- Inputs — Inputs are characterized by root input ports
- Parameters — Named parameters in the model
- Comparison signals — Logged signals and output ports

The wizard allows you to filter and edit the attributes needed for testing. The resulting spreadsheet has separate column sets for inputs, parameters, and comparison signals. If your test requires multiple iterations, the wizard creates a separate sheet in the same file for each iteration. You can expand the spreadsheet to add time-based signal data, tolerances, and parameter overrides.

Open the Model and the Test Manager

```
open_system("coordinate_transform_test")
sltest.testmanager.view
```

Open and Use the Create Test from Spreadsheet Wizard

In the Test Manager, select **New > Test from Spreadsheet..**

Data Page

Select **Create a test template file for specifying data** and click **Next**.

Create Test from Spreadsheet

[Data](#) › [System](#) › [Attributes](#) › [Scenarios](#) › [Generated Test](#)

Do you have existing test specification data?

☐ Use existing test data from a spreadsheet



☒ Create a test template file for specifying data

System Page

Leave `coordinate_transform_test` as the **Model** and **None** as the **Harness**. Click **Next**.

Create Test from Spreadsheet

[Data](#) › [System](#) › [Attributes](#) › [Scenarios](#) › [Generated Test](#)

What is your System Under Test (SUT)?

Model:



Harness:



Attributes Page

Select the test attribute categories to include in the spreadsheet. For this example, leave **Inputs** and **Parameters** and **Yes, include all attributes in the spreadsheet** selected. Click **Next**.

Create Test from Spreadsheet

[Data](#) › [System](#) › [Attributes](#) › [Scenarios](#) › [Generated Test](#)

What test attribute categories do you want to include in your test?

☒ Inputs
Include root level Inport blocks from the system under test.

☒ Parameters
Include named parameters from the system under test.

☐ Comparison signals
Include logged signals from the system under test.

 Refresh

Do you want to include all attributes in the selected categories?

☒ Yes, include all attributes in the spreadsheet

☐ No, I want to filter and edit the attributes.

Note that if you change the SUT during the selection process, click **Refresh** to synchronize the attribute lists with the SUT.

Also, note that if you do not want to use all the attributes in a category as is, select **No, I want to filter and edit the attributes**. Then when you click **Next**, the **Inputs** and **Parameters** tabs are displayed. Select the inputs to use and clear the parameters that are not needed. A **Comparison** tab is also displayed if you selected **Comparisons** in addition to the **Inputs** and **Parameters** on the **Attributes** page. In the Comparison tab, select the signals to use for the baseline comparisons and if desired, set the tolerances. To specify different tolerances for each signal, open and edit the spreadsheet after the wizard generates the spreadsheet

Scenarios page

On the **Scenarios** page, enter 1 as the number of test scenarios and leave the base name for the sheets in the spreadsheet as Scenario. Click **Next**.

Create Test from Spreadsheet

[Data](#) › [System](#) › [Attributes](#) › [Scenarios](#) › [Generated Test](#)

Test Scenario Templates

Specify the number of test scenarios:

Specify a base name for scenarios:

Note that if you enter a number greater than 1, the wizard generates each scenario as a separate sheet in the spreadsheet. The Test Manager uses each sheet as a separate iteration.

Generated Test page

On the **Generated Test** page, specify `coordinate_test.mldatx` as the test file name and the `coordinate_test.xlsx` as the Excel spreadsheet name. Click **Done** to create the test and generate the test and Excel files in the working directory.

Create Test from Spreadsheet

Data › System › Attributes › Scenarios › Generated Test

Where do you want to save the test?

Specify a location to save the test file:

coordinate_test.mldatx 

Specify a location to save the spreadsheet:

coordinate_test.xlsx 

The wizard creates the two files:

- Excel spreadsheet — The spreadsheet includes columns for inputs, parameters, and comparison signals. Inputs and comparisons have different time bases. The wizard generates an identical sheet for each test scenario. Complete the spreadsheet with specific values, such as parameter overrides, to uniquely define each scenario. To view the Excel file, open `coordinate_test.xlsx`. For more information on the file format, see “Microsoft Excel Import, Export, and Logging Format”.

time	Magnitude	Angle	Parameter:	Value:	time	point.pixels_x	point.pixels_y
						AbsTol: 0	AbsTol: 0
						BlockPath: coordinate_transform_test/Bus Creator	BlockPath: coordinate_transform_test/Bus Creator
						PortIndex: 1	PortIndex: 1
	Source: Input					Source: Output	
0	0	0	Xscale	1	0	0	0
10	0	0	Yscale	1	0.2	0	0
					0.4	0	0
					0.6	0	0
					0.8	0	0

- Test file — The test case imports the Excel spreadsheet and creates the test case. The resulting test case in the Test Manager has the **Create Test Case from External File** option selected. The **Parameter Overrides** and **Inputs** sections include data from the file.

New Test Case 1

[coordinate_test_signals](#) » [New Test Suite 1](#) » [New Test Case 1](#)

Baseline Test

Select releases for simulation: Current ▼

☒ Create Test Case from External File

File:  

▶ TAGS

▼ PARAMETER OVERRIDES* 

PARAMETER SET / WORKSPACE VARIABLE	SHEETS	OVERRIDE VALUE	SC
▼ ☐ coordinate_test_signals.xlsx	Scenario1		
☒ Xscale		1	
☒ Yscale		1	
▼ ☐ coordinate_test_signals.xls...	Scenario2		
☒ Xscale		1	
☒ Yscale		1	

The fields in the spreadsheet are locked in the Test Manager. To change the locked fields, edit the spreadsheet outside of the Test Manager and MATLAB. If you change one or more parameters and selected **Comparison** on the **Attributes** page, capture the baseline again by clicking **Capture** in the **Baseline Criteria** section.

Input, Baseline, and Parameter Override Test Case Data Formats in Excel

As an alternative or in addition to using the wizard, you can specify signal data and time data associated with the signal in a Microsoft Excel file to use as input to your test case or as baseline criteria. To support a range of models and configurations, you can specify signal data of most data types. You can indicate whether signals are scalar, multidimensional, or complex. You can optionally specify the data type, block path and port index, units, interpolation type, and function-call execution times. You can also specify values for parameter overrides. If your Excel file has more than one sheet, the data on each sheet is used in the Test Manager as a separate iteration.

For general information on how to format data in Excel files, see “Microsoft Excel Import, Export, and Logging Format”. For information on how to use Excel and other external files, see “Use External File Data in Test Cases” on page 6-76.

Excel File Inputs Format

To create an Excel file and view the format for model input data, click **Create** in the **Inputs** section of a test case in the Test Manager. Then, set **Excel** as the **File format**. The input data includes a time column and a column of data values for each model input. See the example spreadsheet in “Multiple Runs”.

You can add a table iteration to the test case and assign the input file to it by clicking **Create** in the **Inputs** section of the Test Manager. Then, in the Create Input Data File dialog box, click **Add iterations to run this input**. After you create the input file, continue to specify the iteration. For more information on iterations, see “Test Iterations” on page 6-139.

To use an existing Excel file for the model input data, click **Add**.

Excel File Baseline Data Format

To view the Excel format for the baseline data, first capture the baseline from your model. Click **Capture** in the **Baseline Criteria** section of a test case in the Test Manager. Then, select **Excel** as the **File format**. The Excel file includes the time and associated value for each signal in the model that is marked for logging. Information about the logged signal includes the signal name, absolute tolerance, path of the block from which the signal originates, the port index of the block, and the source type.

time	EngineRPM	Gear	ImpellerTorque	OutputTorque
	AbsTol: 0	AbsTol: 0	AbsTol: 0	AbsTol: 0
	BlockPath: sltestExcelExample/Engine	BlockPath: sltestE	BlockPath: sltestExcelExe	BlockPath: sltestExcelExan
	PortIndex: 1	PortIndex: 1	PortIndex: 1	PortIndex: 2
	Source: Output			
		Interp: zoh		
0	1000	1	52.91933848	282.6515006
0.04	843.499908	1	37.70034938	198.2234252
0.08	712.907973	1	26.96490285	139.5499868
0.12	601.1621027	1	19.19944609	97.72824423
0.16	600	1	19.13843127	96.57115783
0.2	600	1	19.15131864	95.80637046
0.24	600	1	19.16756607	95.10509703

This sample baseline spreadsheet includes uniquely named signals and signal block paths. Causes of signals not being well defined are:

- When a signal does not have a name in the model, a name is automatically assigned to it in the spreadsheet.
- When you manually create a spreadsheet and do not include the signal name, a signal name with the format **block_name: port_index** is assigned to the signal.
- When the signal name included in a spreadsheet does not uniquely identify the signal, add the block path for that signal. For example, signals with the same names at different levels in the model hierarchy are not uniquely identified if you only specify the name. If you do not include the block path, the first signal with that name is used.

You can include both baseline (**Source:Output**) data and input (**Source:Input**) data in the same Excel file. See the example spreadsheet in “Multiple Runs”.

To use a manually created Excel file as expected outputs, click **Add** in the **Baseline Criteria** section of the test case in the Test Manager and specify the file name.

To edit external files from the **Baseline Criteria** section of the Test Manager, select the file and click **Edit**. Excel files open in Excel by default, but you can also open and edit an Excel file outside of MATLAB.

Excel File Parameter Overrides Format

The parameters in a spreadsheet are in two columns labeled **Parameter**, which is the parameter name, and **Value**, which is the parameter value. See the example spreadsheet in “Multiple Runs”. To override a parameter, change its value in the spreadsheet before you add the file to the test case.

For parameter types such as `Simulink.Parameter`, `Simulink.DataDictionary`, or `Simulink.LookupTable` objects, you can specify only the values to override. The values you do not specify use the values set in the model. The Excel format for overriding these parameters is:

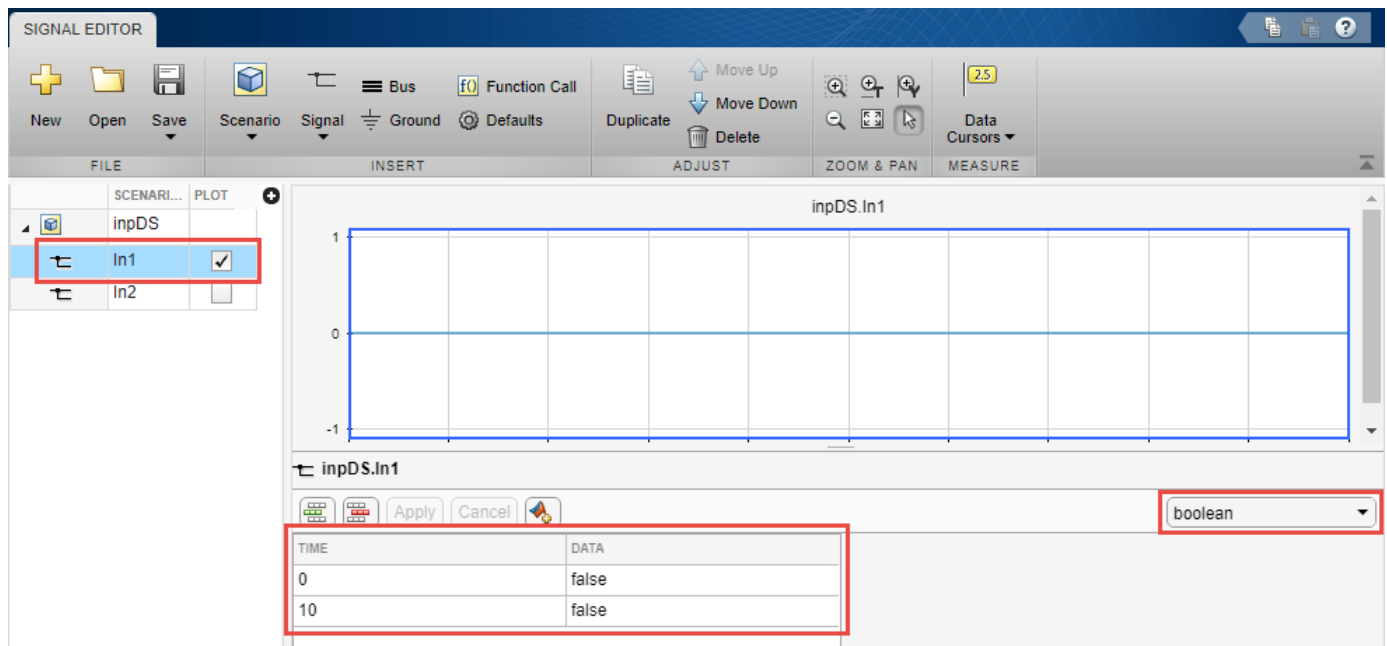
Parameter	Value
Speed	Simulink.Parameter; Speed.Value = 5; Speed.Complexity = 'real';

To use parameter overrides from an Excel file, click **Add > Add File** in the **Parameter Overrides** section of a test case in the Test Manager.

Create a MAT File for Test Case Inputs

You cannot manually create a MAT file to use in a test case. Follow these steps to generate a MAT file.

- 1 In the test case, under **System Under Test**, specify the model whose input data you want to create a MAT file for.
- 2 In the **Inputs** section of the test case, click **Create**.
- 3 In the dialog box, set **File Format** to MAT. Specify the location for the MAT file and click **Create**. To edit an existing file, select the file and click **Edit**. The Signal Editor opens.
- 4 In the left pane of the Signal Editor, expand the data node. Then select the signal whose data you want to add.
- 5 Specify the signal data. In the bottom pane, select the data type from the list, and enter the time and signal data for the signal.



- 6 To update your signal data, click **Apply**.
- 7 After adding the signal data, click **Save**.

MATLAB Script Format for Test Case Data

You can use MATLAB scripts to specify test case data. The files include MATLAB language commands formatted in the same way as code you write at the command line. To edit a script file, open the file in a text editor. For example, a MATLAB script file that sets two parameter values contains this code:

```
a_min = 1.3;
a_max = 22.4;
```

To add these parameters from the file to your test case, use **Add > Add File** in the Test Manager **Parameter Overrides** section.

See Also

`sltest.testmanager.BaselineCriteria` | `sltest.testmanager.TestInput`

More About

- “Microsoft Excel Import, Export, and Logging Format”
- “Use External File Data in Test Cases” on page 6-76
- “Create and Edit Signal Data”
- “Simulation Settings and Release Overrides” on page 6-177
- “Baseline Criteria” on page 6-184

Capture Simulation Data in a Test Case

Capture signal data in your test results by adding signals to the **Simulation Outputs** section of the test case. Each output is called a logged signal. Signals listed in **Simulation Outputs** appear in the test results along with signals that are already selected for logging in Simulink. You can also log a portion of a signal by using conditional or duration triggers to start and stop logging.

You can use logged signals for data comparison in baseline criteria, equivalence criteria, custom criteria, and for data visualization in the Simulation Data Inspector. Logged signals enable you to further test your Simulink model without changing the model. In addition to signals from the top model, you can also log signals from subsystems and model references. You can select signals associated with local and global data store memory, and from data store memory that uses a `Simulink.Signal` object.

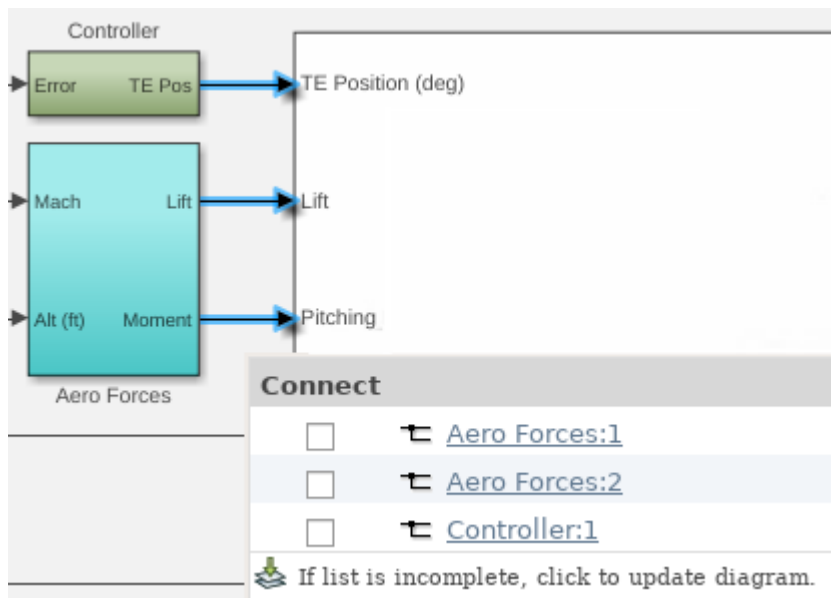
Add Logged Signals When Creating a Test Harness

When you right-click a model or model component and click **Test Harness > Create for <Model or Model Element>**, the Create Test Harness dialog box opens. To log all output signals of the component under test, select **Log Output Signals**, which is on the **Basic Properties** tab of the Create New Harness dialog for Subsystem block diagram and library harnesses. For all other types of harnesses, the option is on the **Advanced Properties** tab. Signals that are not compatible with logging do not have a logging badge attached to them. The compatibility is determined at compile time. After the harness is created, you can turn off logging for a signal by opening the harness, right-clicking the signal, and selecting **Stop Logging Selected Signals**. If a signal does not have a name, it is assigned one for test execution using the format `<component under test name>:<output port number>`. Unnamed propagated signals use the name of the signal from which they propagate. To log all test harness signals programmatically, include `'LogOutputs', true` as an input to `sltest.harness.create`.

Add Logged Signals in the Test Manager

To add signals:

- 1 Open the model `sltestFlutterSuppressionSystemExample`.
`openExample('sltestFlutterSuppressionSystemExample')`
- 2 Use `sltest.testmanager.view` to open the Test Manager.
- 3 Click **Open** and open the `sltestFlutterCriteriaTest.mldatx` test file.
- 4 In the Test Manager, under **Simulation Outputs**, click **Add**.
- 5 In the system under test, highlight blocks or signals that you want to log. To select multiple items, click and drag a selection box over multiple items.
- 6 A dialog box appears. Select signals in the dialog box.



- 7 Continue adding signals to the test case. Each time you select a signal, the dialog box also shows previously logged signals. You can remove a signal from logging by clearing the selection.
- 8 When you finish adding signals, return to the Test Manager and click **Done**.
- 9 The signals appear in the **Logged Signals** table in the test case.

LOGGED SIGNALS			
NAME	SOURCE	PORT INDEX	PLOT INDEX
▼ ☒ Signal Set 1			
☒ Aero Forces:1	sltestFlutterSuppression...	1	
☒ Aero Forces:2	sltestFlutterSuppression...	2	
☒ Controller:1	sltestFlutterSuppression...	1	
☐ Plot signals on the specified plots after simulation			
+ Add 🗑️ Delete			

To add a signal set, click the **Add** arrow and select **Signal Set**.

To specify a specific plot for a signal, enter a number in the **Plot Index** column. By default, the signals appear on one plot.

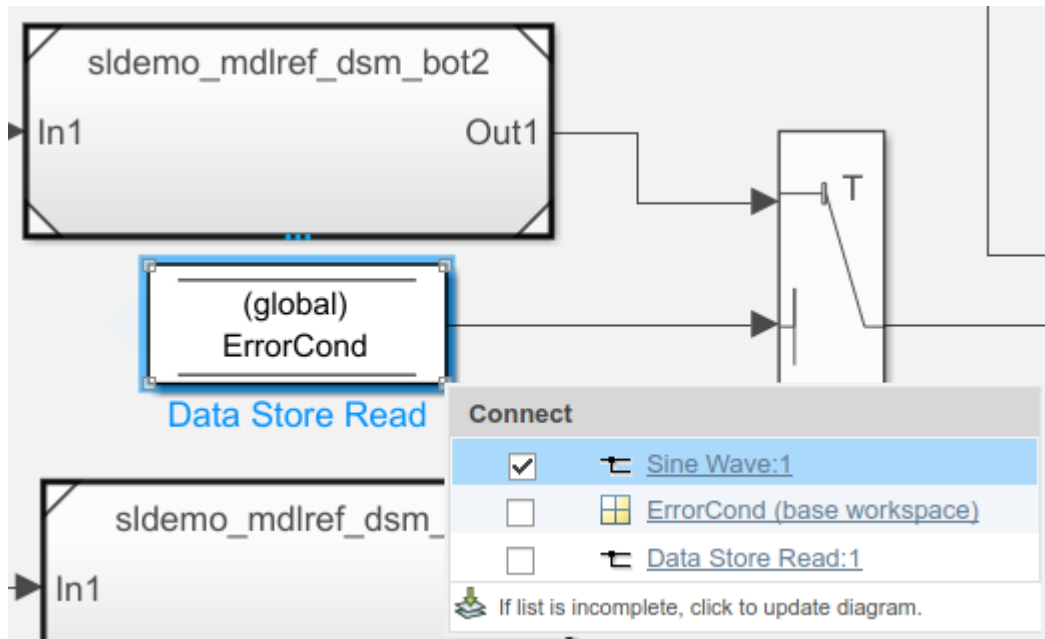
You can specify to display the plot immediately after running the test by selecting the **Plot signals on the specified plots after simulation** check box.

After you run the test, the logged signals appear in the test case results under **Sim Output**. Select each signal to display on the plot. If you specify a plot index, the signal appears in the plot number you specified.

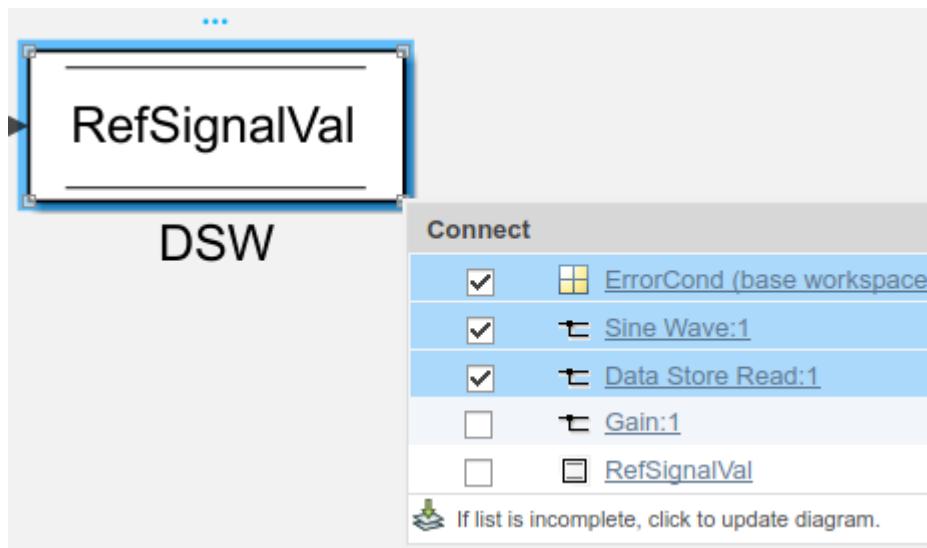
Capture Data from Local and Global Data Stores

Perform similar steps to add simulation output associated with data store memory:

- 1 Use `openExample('sldemo_mdref_dsm')` to open the model, which contains local and global data store memory.
- 2 Use `sltest.testmanager.view` to open the Test Manager.
- 3 Click **New > Test File from Model** to create a new test file.
- 4 Go to the top model, select the Sine Wave block signal and click **Log Selected Signal**. Mark the Data Store Read (ErrorCond) block signal for logging, too.
- 5 Select the Data Store Read block in the top model. In the **Modeling** tab, click **Update Model**. The model displays the signal storage class for the block, (`global`).
- 6 In the Test Manager, select New Test Case 1 and expand the **Simulation Outputs** section. Click **Add** in the lower right of **Logged Signals**.
- 7 In the model, select the Sine Wave block and in the Connect dialog box, select Sine Wave:1. Then, select the Data Store Read block and in the dialog box, select Data Store Read: 1 and ErrorCond (base workspace).



- 8 In the model, double-click `sldemo_mdref_dsm_bot` to open it. Then, open the subsystem `PositiveSS`. Select the Data Store Write block. The dialog box displays the input signal from the Gain block and the data store memory, `RefSignalVal`.



- 9 Select the `RefSignalVal` data store memory for logging.
- 10 Finish selecting signals by clicking **Done** in the Test Manager window. In the Test manager, the signals appear under **Logged Signals**. The **Source** column displays the full path information for each signal.

LOGGED SIGNALS			
NAME	SOURCE	PORT INDEX	
▼ ☒ Signal Set 1			
☒ Data Store Read:1	sldemo_mdref_dsm/Data Store Read	1	
☒ ErrorCond	base workspace		
☒ RefSignalVal	sldemo_mdref_dsm/A/DSM		
☒ Sine Wave:1	sldemo_mdref_dsm/Sine Wave	1	
☐ Plot signals on the specified plots after simulation			

Log Leaf Signals of a Bus

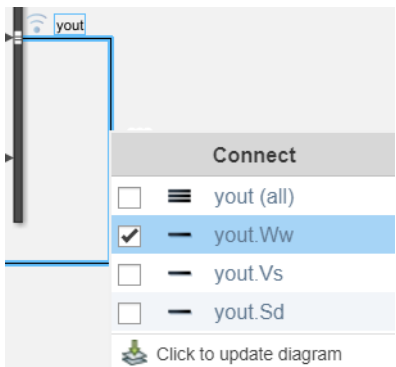
In addition to logging an entire Bus block, you can select one or more individual signals within a bus and add them to the **Logged Signals** section of Test Manager. For large buses, adding only the needed signals might reduce the amount of time it takes a test case to run.

- 1 Open the model using `openExample('sldemo_absbrake')`.
- 2 Use `sltest.testmanager.view` to open the Test Manager.
- 3 Click **New > Test File** to create a new test file. Name and save the file.

Under **System Under Test**, click the **Use current model** icon.

- 4 Under **Simulation Outputs**, click **Add**.
- 5 In the system under test, select the signal exiting the Bus, `yout`.

- 6 A dialog box appears. Select the desired bus or bus leaf signals in the dialog box. Three horizontal lines indicate the whole bus. A single horizontal line next to a signal name indicates that it is a bus leaf signal. If the leaf signals do not appear in the dialog, click **Click to update diagram**.



- 7 Continue adding signals to the test case, such as the non-bus Sd and tire torque signals
8 When you finish adding signals, return to the Test Manager and click **Done**.
9 The signals appear in the **Logged Signals** table in the test case.

LOGGED SIGNALS			
NAME	SOURCE	PORT INDEX	PLOT INDEX
▼ ✓ Signal Set 1			
✓ Sd	sldemo_absbrake/Stop...	1	
✓ tire torque	sldemo_absbrake/Rr	1	
✓ yout.Ww	sldemo_absbrake/Bus ...	1	
Plot signals on the specified plots after simulation			
+ Add ▼ Delete			

To add a signal set, click the **Add** arrow and select **Signal Set**.

To specify a specific plot for a signal, enter a number in the **Plot Index** column. By default, the signals appear on one plot.

You can specify to display the plot immediately after running the test by selecting the **Plot signals on the specified plots after simulation** check box.

Use Triggers to Start and Stop Signal Logging

By default, output signals are logged for the entire simulation. To specify when signal logging starts and stops, use output triggers. An output trigger can be a condition expression or a duration. When a condition expression evaluates to true, logging starts or stops. The start duration is the time in seconds when logging starts after simulation starts. The stop duration is when logging stops after signal logging starts. To specify the triggers and the symbols to use in the conditional triggers, In the Test Manager, use the **Output Triggers** subsection under **Simulation Outputs** for a test case. To obtain the trigger results programmatically, use `sltest.testmanager.OutputTrigger` to specify triggers, and `sltest.testmanager.OutputTriggerResult` to obtain the trigger results. You must, however, use the Test Manager to define and map symbols that are used in conditional triggers.

Add Triggers to a Test Case

This example shows how to add triggers to a test case by using the Test Manager. The triggers start logging output data when the Ww output signal is greater than 40 and stop logging output data after 4 seconds.

1. Open the model.

sldemo_absbrake

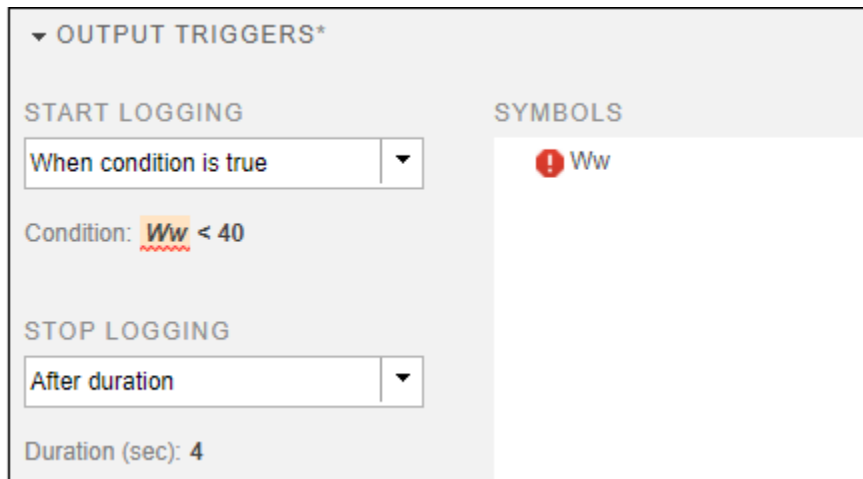
2. Open the test file in the Test Manager. This file contains a predefined test case, Trigger Test Case.

```
sltest.testmanager.load('AddTriggersToTest.mldatx');  
sltest.testmanager.view;
```

3. Select **Trigger Test Case** and expand the **Simulation Outputs** section.

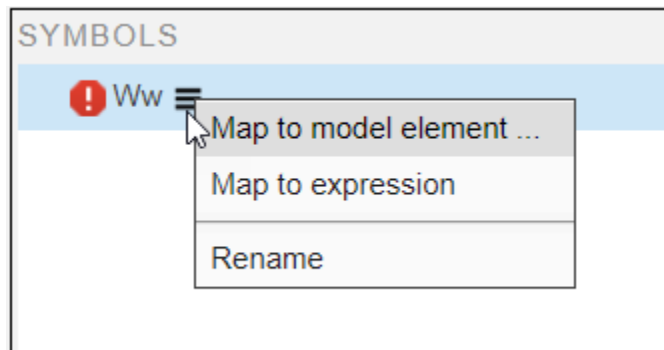
4. Under **Output Triggers**, set **Start Logging** to When condition is true. Then, click the edit icon next to **Condition** and enter $Ww < 40$.

5. Set **Stop Logging** to After Duration. Click next to **Duration (sec)** and enter 4. Leave **Shift time to zero** selected. This option shifts the logged signal so the triggered output shows as starting at time 0.

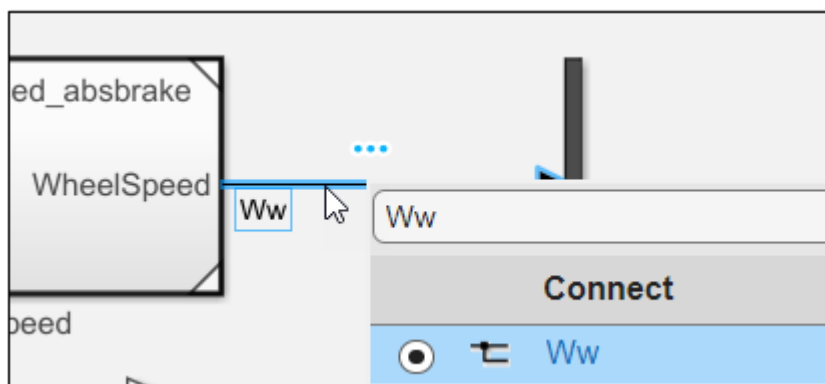


Ww appears in **Symbols** with a red icon, which indicates it is an unresolved symbol.

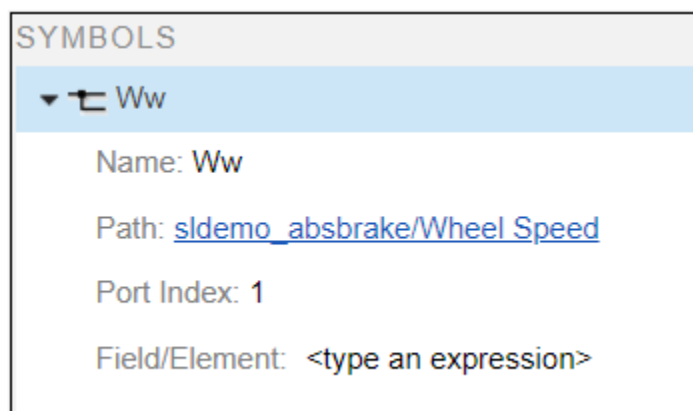
6. Point to Ww, click on the menu icon and select **Map to model element**.



In the model, select the Ww signal. In the Connect dialog box, select Ww.

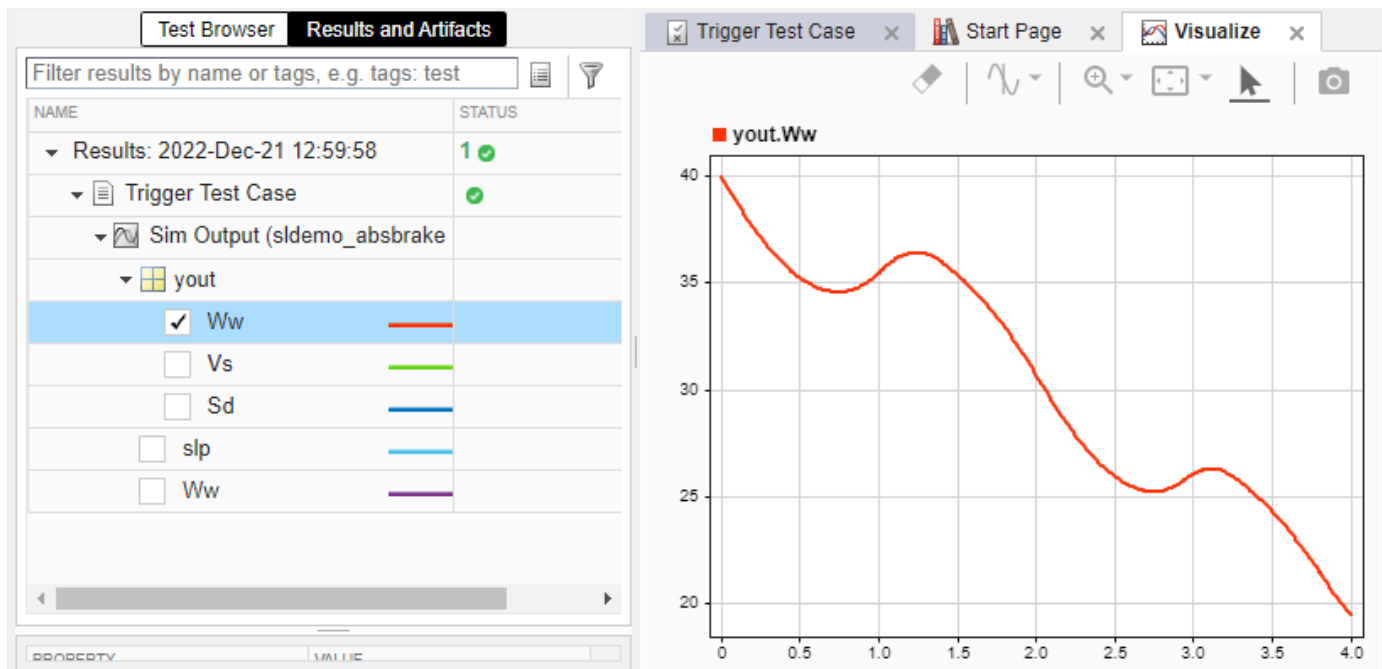


7. Return to the Test Manager and click **Done**. The Ww symbol updates and displays its details.



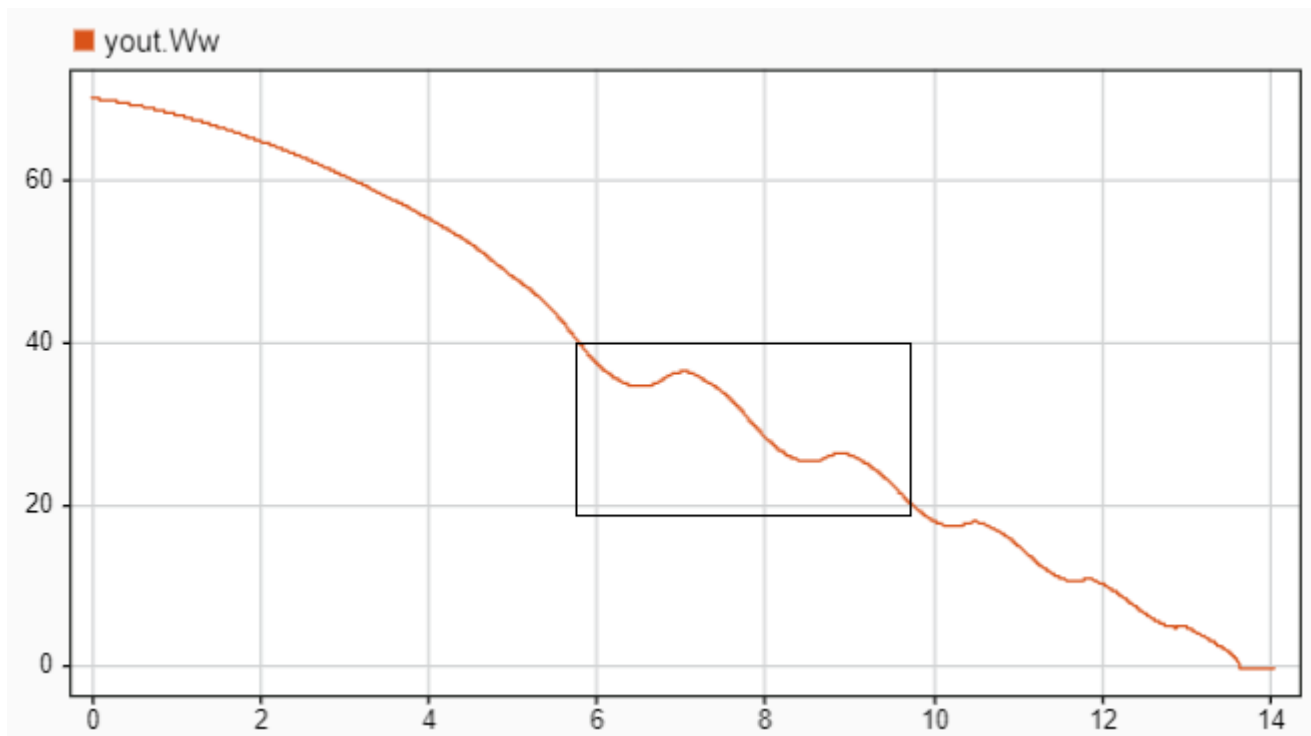
8. Run the test.

9. In the **Results and Artifacts** pane, display the triggered Ww output.

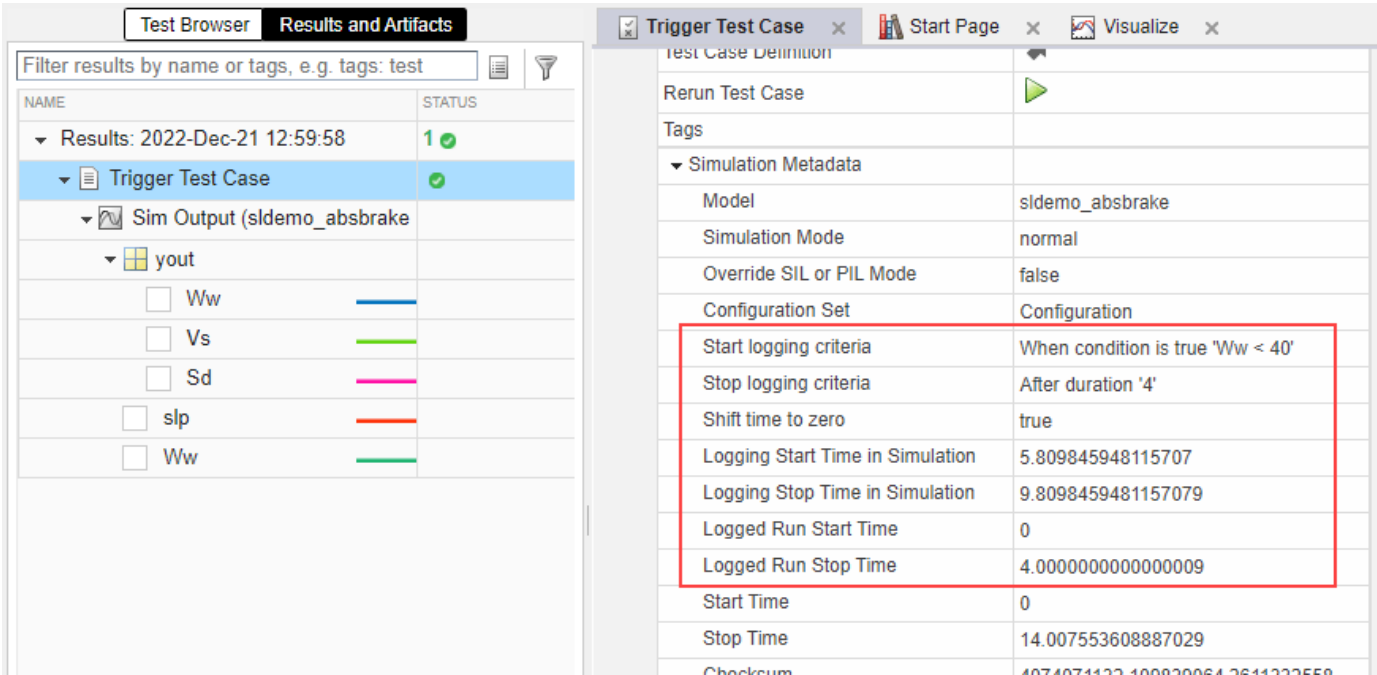


Compare the triggered output plot to the plot below, which shows the output when the entire simulation is logged. The triggered portion is highlighted. The triggered plot above is shifted to start at time 0. Logging starts when the signal equals 40 and stops 4 seconds later.

To create the plot of the entire simulation, set **Start Logging** to *On simulation start* and **Stop Logging** to *When simulation stops*.



11. To see a summary of the simulation, including the trigger information, select **Trigger Test Case** and expand **Simulation Metadata**.



12. Clean up and close the Test Manager and close the model.

```
sltest.testmanager.clear
sltest.testmanager.clearResults
sltest.testmanager.close
```

Trigger Limitations

These limitations apply to test cases that have conditional or duration triggers enabled:

- You cannot collect coverage.
- If a test case has iterations, the start and stop triggers defined for the test case apply to all iterations.

See Also

```
sltest.testmanager.TestCase | sltest.testmanager.LoggedSignal |
sltest.testmanager.LoggedSignalSet | Simulink.Signal |
sltest.testmanager.OutputTrigger | sltest.testmanager.OutputTriggerResult |
sltest.testmanager.TriggerMode | Simulink Test Manager
```

More About

- “Assess Simulation and Compare Output Data” on page 3-11
- “Compare Simulation to Baseline Data” on page 6-246

Run Tests in Multiple Releases of MATLAB

If you have more than one release of MATLAB installed, you can run tests in multiple releases. Starting with R2011b, you can also run tests in releases that do not have Simulink Test. Running tests in multiple releases enables you to use test functionality from later releases while running the tests in your preferred release of Simulink. If you have code that was generated in a previous release, you can verify that the model in the current release continues to work as expected. Also, in the current release you can create a test harness using the previously generated code, rather than having to regenerate it. Comparing test results across multiple releases can help you to better understand Simulink changes before upgrading to a new version of MATLAB and Simulink.

Code that was generated in a previous release to verify that the model in the current release continues to work as expected. In the current release you can create a test harness using the previously generated code, rather than having to regenerate it.

Although you can run test cases on models in previous releases, the release you run the test in must support the features of the test. For example, if your test involves test harnesses or test sequences, the release must support those features for the test to run.

Before you can create tests that use additional releases, add the releases to your list of available releases using Test Manager preferences. See “Add Releases Using Test Manager Preferences” on page 6-103.

Considerations for Testing in Multiple Releases

Testing Models in Previous or Later Releases

Your model or test harness must be compatible with the MATLAB version running your test.

- If you have a model created in a newer version of MATLAB, to test the model in a previous version of MATLAB, export the model to a previous version and simulate the exported model with the previous MATLAB version. For more information, see the information on exporting a model in “Save Models”.
- To test a model in a more recent version of MATLAB, consider using the Upgrade Advisor to upgrade your model for the more recent release. For more information, see “Upgrade Models Using Upgrade Advisor”.

Test Case Compatibility with Previous Releases

When collecting coverage in multiple-release tests, you can run test cases up to three years (six releases) prior to the current release. Tests that contain logical or temporal assessments are supported in R2016b and later releases.

Multiple-release testing for test cases with fault settings is supported in R2024b and later. When you select releases for simulation for a test case with fault settings, you can select R2023b or later.

Test Case Limitations with Multiple Release Testing

Certain features are not supported for multiple-release testing:

- Parallel test execution
- MATLAB Unit Test framework test cases

- Real-time tests
- Models with observers
- Input data defined in an external Excel document
- Custom figures from test case callbacks
- Variant configurations

Add Releases Using Test Manager Preferences

Before you can create tests for multiple releases, use Test Manager preferences to include the MATLAB release you want to test in. You can also delete a release that you added to the available releases list. However, you cannot delete the release from which you are running Test Manager.

- 1 In the Test Manager, click **Preferences**.
- 2 In the Preferences dialog box, click **Release**. The **Release** pane lists the release you are running Test Manager from.
- 3 In the **Release** pane, click **Add/Remove releases** to open the Release Manager.
- 4 In the Release Manager, click **Add**.
- 5 Browse to the location of the MATLAB release you want to add and click **OK**.
- 6 To change the release name that will appear in the Test Manager, edit the **Name** field.
- 7 Close the Release Manager. The Preferences dialog box shows the selected releases. Deselect releases you do not want to make available for running tests.

Run Baseline Tests in Multiple Releases

When you run a baseline test with the Test Manager set up for multiple releases, you can:

- Create the baseline in the release you want to see the results in, for example, to try different parameters and apply tolerances.
- Create the baseline in one release and run it in another release. Using this approach you can, for example, know whether a newer release produces the same simulation outputs as an earlier release.

Create the baseline.

- 1 Make sure that the release has been added to your Test Manager preferences.
- 2 Create a test file, if necessary, and add a baseline test case to it.
- 3 Select the test case.
- 4 Under **System Under Test**, enter the name of the model you want to test.
- 5 Set up the rest of the test.
- 6 Capture the baseline. Under **Baseline Criteria**, click **Capture**. Specify the format and file in which to save the baseline and select the release in which to capture the baseline. Then, click **Capture** to simulate the model.

For more information about capturing baselines, see “Capture Baseline Criteria” on page 6-185.

After you create the baseline, run the test in the selected releases. Each release you selected generates a set of results.

- 1 In the test case, expand **Simulation Setting and Release Overrides** and, in the **Select releases for simulation** drop-down menu, select the releases you want to use to compare against your baseline.
- 2 Specify the test options.
- 3 From the toolbar, click **Run**.

For each release that you select when you run the test case, the pass-fail results appear in the **Results and Artifacts** pane. For results from a release other than the one you are running Test Manager from, the release number appears in the name.

Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: test	
NAME	STATUS
▼ Results: 2020-Jun-12 16:33:38	2 ✓
▼ R2020a:Baseline Test Case	✓
▶ Baseline Criteria Result	✓
▶ Sim Output (vdp : normal)	
▼ Baseline Test Case	✓
▶ Baseline Criteria Result	✓
▶ Sim Output (vdp : normal)	

Run Equivalence Tests in Multiple Releases

When you run an equivalence test, you compare two simulations. Each simulation runs in a single release, which can be the same or different. Examples of equivalence tests include comparing models run in different model simulation modes, such as normal and software-in-the-Loop (SIL), or comparing different tolerance settings.

- 1 Make sure that the releases have been added to your Test Manager preferences.
- 2 Create a test file, if necessary, and add an equivalence test case to it.
- 3 Select the test case.
- 4 Under **Simulation 1, System Under Test**, enter the name of the model you want to test.
- 5 Expand **Simulation Setting and Release Overrides** and, in the **Select releases for simulation** drop-down menu, select the release for Simulation 1 of the equivalence test. For an equivalence test, only one release can be selected for each simulation.
- 6 Set up the rest of the test.
- 7 Repeat steps 4 through 6 for **Simulation 2**.
- 8 In the toolbar, click **Run**.

The test runs each simulation in the release you selected and compares the results for equivalence. For each release that you selected when you ran the test case, the pass-fail results appear in the **Results and Artifacts** pane. For results from a release other than the one you are running Test Manager from, the release number appears in the name.

Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: test	
NAME	STATUS
▼ Results: 2020-Jun-21 14:51:59	1 ✓
▼ SIL/PIL Equivalence test	✓
▼ Equivalence Criteria Result	✓
○ CounterA:1	✓
○ CounterB:1	✓
○ CounterC:1	✓
▼ Sim Output 1 (rtwdemo_mdrefl)	
✓ CounterA:1	
CounterB:1	
CounterC:1	
▼ R2020a: Sim Output 2 (rtwden	
✓ CounterA:1	
CounterB:1	
CounterC:1	

Run Simulation Tests in Multiple Releases

Running a simulation test simulates the model in each release you select using the criteria you specify in the test case.

- 1 Make sure that the releases have been added to your Test Manager preferences.
- 2 Create a test file, if necessary, and add a simulation test case template to it.
- 3 Select the test case.
- 4 Under **System Under Test**, enter the model you want to test.
- 5 Expand **Simulation Setting and Release Overrides** and, in the **Select releases for simulation** drop-down menu, select the release options for the simulation.
- 6 Under **Simulation Outputs**, select the signals to log.
- 7 In the toolstrip, click **Run**.

The test runs, simulating for each release you selected. For each release, the pass-fail results appear in the **Results and Artifacts** pane. For results from a release other than the one you are running Test Manager from, the release number appears in the name.

Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: test	
NAME	STATUS
▼ Results: 2020-Jun-10 12:39:00	1 ✓
▶ R2020a: Test Case 1	✓
▶ R2019b: Test Case 1	✓
Test Case 1	✓

Assess Temporal Logic in Multiple Releases

You can run tests that contain logical and temporal assessments in multiple releases to test signal logic for models created in an earlier release. You can also compare assessment results across releases when you run the tests in multiple releases. For more information, see “Assess Temporal Logic by Using Temporal Assessments” on page 3-85.

You can run these test case types with logical and temporal assessments:

- Baseline tests
- Equivalence tests
- Simulation tests

Run Tests with Logical and Temporal Assessments

To run tests logic with logical and temporal assessments in multiple releases:

- 1 Start MATLAB R2021b or later.
- 2 Open the Test Manager. For more information, see “Open the Simulink Test Manager”.
- 3 In the Test Manager, add the releases to your Test Manager preferences. For more information, see “Add Releases Using Test Manager Preferences” on page 6-103.
- 4 Create a new test file with a baseline, equivalence, or simulation test case, or open an existing one. For more information, see:
 - “Baseline Testing” on page 6-240
 - “Back-to-Back Equivalence Testing” on page 6-36
 - “Test a Simulation for Run-Time Errors” on page 6-14
- 5 In the Test Manager, specify your test case properties, including the system under test and other properties that you want to apply. For more information, see “Specify Test Properties in the Test Manager” on page 6-173.
- 6 Add a logical or temporal assessment to your test case. For more information, see “Assess Temporal Logic by Using Temporal Assessments” on page 3-85 and “Logical and Temporal Assessment Syntax” on page 3-99.
- 7 Select the releases to run the test in. In the Test Manager, select your test case. In **System Under Test**, under **Simulation Settings and Release Overrides**, next to **Select releases for simulation**, select the releases to run the test case in from the list.

If you are using a baseline or simulation test case, you can run the test in multiple releases in a single run by selecting multiple releases from the list. If you are using an equivalence test case, you can select one release under **Simulation 1** and another release under **Simulation 2**. For more information, see:

- “Run Baseline Tests in Multiple Releases” on page 6-103
 - “Run Equivalence Tests in Multiple Releases” on page 6-104
 - “Run Simulation Tests in Multiple Releases” on page 6-105
- 8 Run the test. In the Test Manager, click **Run**.

Evaluate Assessment Results

The **Results and Artifacts** pane displays the test results for each release you selected. The test release appears in the name of each test result from a release other than the version you ran Test Manager from.

Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: tes	
NAME	STATUS
▼ Results: 2021-Jul-07 08:21:18	2 ✓
▼ R2020a: TC_Simulation	✓
▶ Sim Output (sltestBaselineBasi	
▼ Logical and Temporal Assessm	✓
○ Assessment1	✓
▼ TC_Simulation	✓
▶ Sim Output (sltestBaselineBasi	
▼ Logical and Temporal Assessm	✓
● Assessment1	✓

You can evaluate the assessment results independently from other pass-fail criteria. For example, while a baseline test case might fail due to a failing baseline criteria, a logical or temporal assessment in the test case might pass.

You can also examine detailed assessment signal behavior. For more information, see “View Assessment Results” on page 3-89.

Collect Coverage in Multiple-Release Tests

To add coverage collection for multiple releases, you must have a Simulink Coverage license. Set up your test as described in “Run Baseline Tests in Multiple Releases” on page 6-103, “Run Equivalence Tests in Multiple Releases” on page 6-104, or “Run Simulation Tests in Multiple Releases” on page 6-105. You can use external test harnesses to increase coverage for multiple-release tests. Before you capture the baseline or run the equivalence or simulation test, enable coverage collection.

- 1 Click the test file that contains your test case. To collect coverage for test suites or test cases, you must enable coverage at the test file level.
- 2 In the **Coverage Settings** section, select **Record coverage for system under test**, **Record coverage for referenced models**, or both.
- 3 Select the types of coverage to collect under **Coverage Metrics** to collect.

After you run the test, the **Results and Artifacts** pane shows the pass-fail results for each release in the test suite.

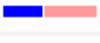
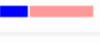
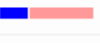
Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: tes	
NAME	STATUS
▼ Results: 2020-Nov-24 13:58:48	2 ✓
▼ MRT_test	2 ✓
▼ MRT Test Suite	2 ✓
R2020b: MultiRelease test case	✓
R2020a: MultiRelease test case	✓

To view the coverage results for a release, select its test case and expand the **Coverage Results** section. The table lists the model, release, and the coverage percentages for the metrics you selected.

▼ COVERAGE RESULTS							
ANALYZED MODEL	RELEASE	REPORT	COMPLEXI...	DECISION	CONDITION	EXECUTION	
RollAutopilotRevised	R2020b		9	43%	30%	85%	
Add Tests for Missing Coverage Export							

To view aggregated coverage results for the releases in your test, select the test suite that contains the releases and expand the **Aggregated Coverage Results** section.

▼ AGGREGATED COVERAGE RESULTS ?

ANALYZED MODEL	RELEASE	REPORT	COMPLEXI...	DECISION	CONDITION	EXECUTION	+
 RollAutopilotRevised	R2020a		9	43% 	30% 	85% 	+
 RollAutopilotRevised	R2020b		9	43% 	30% 	85% 	+

+ Add Tests for Missing Coverage Export

To use the current release to add tests for missing coverage to an older release, click the row and click **Add Tests for Missing Coverage**. You can also use coverage filters, generate reports, merge results, import and export results, and scope coverage to linked requirements. For more information, see “Collect Coverage in Tests” on page 6-151 and “Increase Test Coverage for a Model” on page 6-163.

See Also

`sltest.testmanager.getpref` | `sltest.testmanager.setpref`

More About

- “Simulation Settings and Release Overrides” on page 6-177
- “Baseline Testing” on page 6-240
- “Back-to-Back Equivalence Testing” on page 6-36
- “Test a Simulation for Run-Time Errors” on page 6-14
- “Assess Temporal Logic by Using Temporal Assessments” on page 3-85
- “Collect Coverage in Tests” on page 6-151

Examine Test Failures and Modify Baselines

After you run a baseline test in the Test Manager, you can update the baseline. For example:

- If you changed your model, you can use the new simulation output as the baseline. You can examine the failures that occurred because of the differences and update the baseline with part or all of the new output. See “Examine Test Failure Signals and Update Baseline Test” on page 6-110.
- If your test plan changed and you expect different outputs, you can manually edit the time points. See “Manually Update Signal Data in a Baseline” on page 6-112.

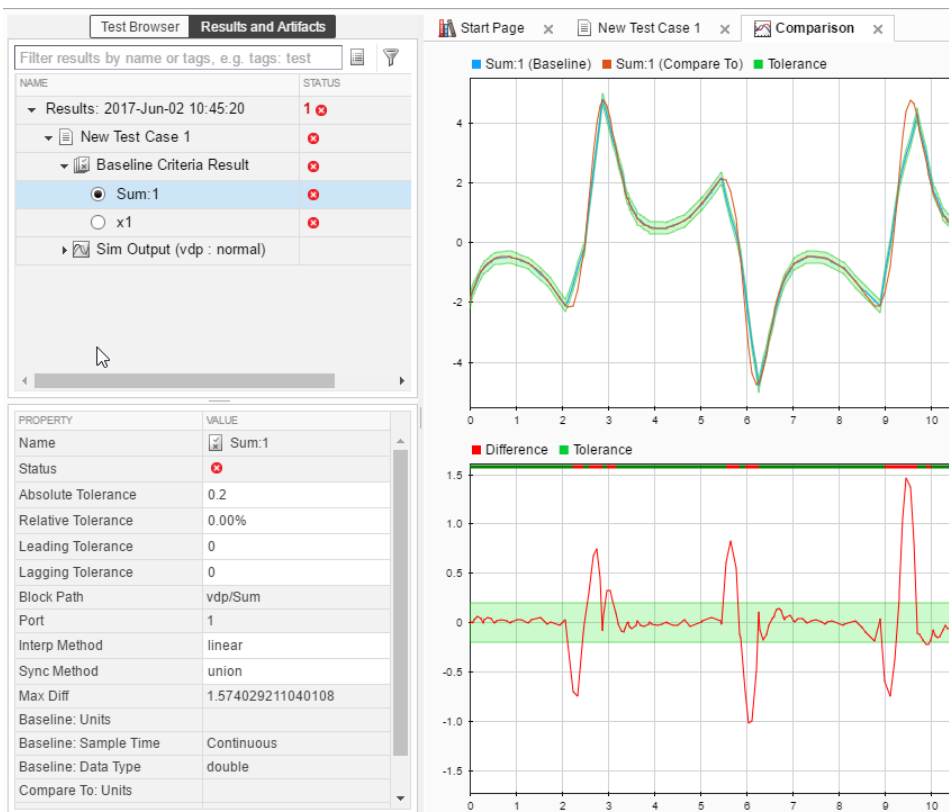
Examine Test Failure Signals and Update Baseline Test

Suppose that you run a test against a baseline and the result does not match the baseline, causing test failure. It is possible that the newer simulation better represents your desired test results or that some of the points of failure are your preferred results. You can examine the signal and failures in the data inspector view in Test Manager and decide whether you want to update the baseline or sections of the baseline.

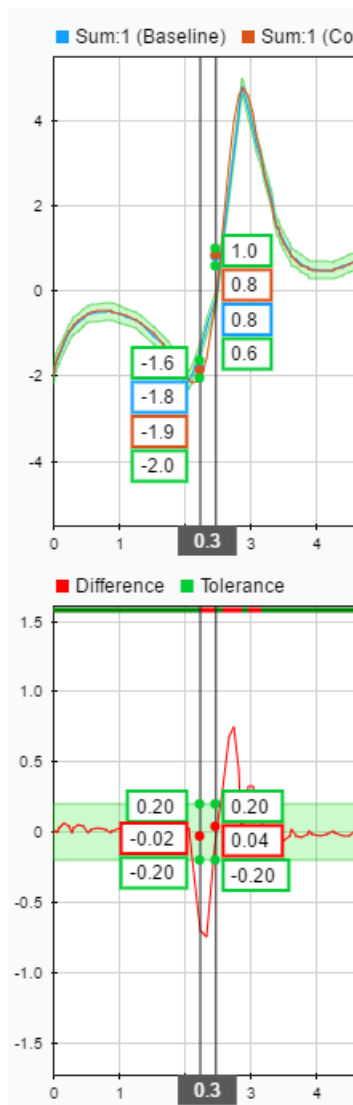
Suppose that your model uses a new solver. When you run the test case, the results do not match, causing the test to fail.

- 1 Open the test file that contains the baseline test case you want to run.
- 2 Select the test case and run it.
- 3 If the test fails, in the **Results and Artifacts** pane, expand the Baseline Criteria. Select a signal that failed that you want to examine.

When you select the signal, the data inspector view opens. The top graph is the baseline simulation signal overly. The bottom is the difference between those signals and the tolerance. You can adjust tolerances in the pane in the lower-left corner of the Test Manager. This example shows an absolute tolerance of .2.



- 4 To examine each failure, in the toolbar, click **Next Failure** or **Previous Failure**. Each contiguous set of failed signal comparison points makes up one region. Data cursors show the bounds of each region.



- 5 You can update the baseline data to use newer simulation results clicking **Update Baseline**



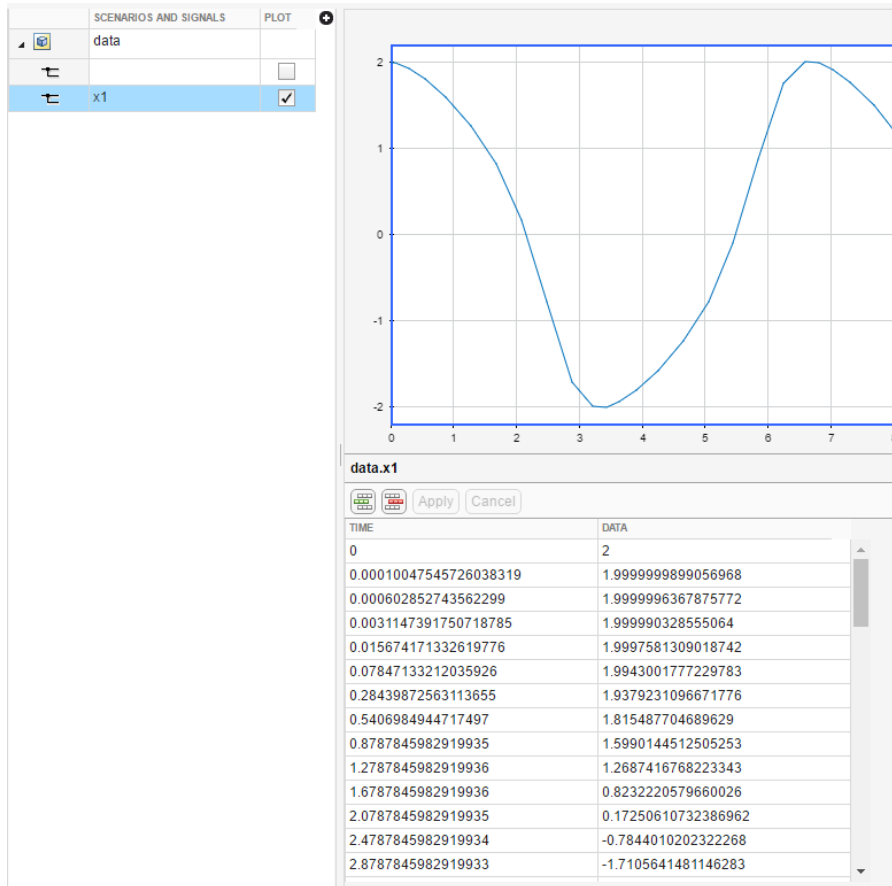
- To update the entire signal, select **Replace One Signal in Baseline File** from the dropdown.
- To update only the data in the failure region, select **Replace Signal Segment in Baseline File** from the dropdown.
- To replace all signal data in the baseline with the new data, select **Use All Sim Output Signals as Baseline** from the dropdown.

Manually Update Signal Data in a Baseline

If your model changes such that you expect a different simulation output, you can update all or part of the baseline signal data. If the baseline is a MAT file, you can edit the data in the Signal Editor. If the baseline is a Microsoft Excel file, you can edit the data in Excel.

To update signal data in a MAT file baseline:

- 1 Open the test file that contains the baseline you want to edit.
- 2 Select the test case.
- 3 Under **Baseline Criteria**, select the baseline whose signal data you want to edit. Click **Edit**.
- 4 The Signal Editor opens. In the **Scenarios and Signals** pane, expand the data node.
- 5 Select the check box next to the signal whose data you want to edit.



Tip To see the time and data for points, display a data cursor and drag it along the signal.

- 6 Edit the signal data in the table, and then click **Apply**.
- 7 To update the baseline with the new expected output data, click **Save**.

To update baseline signal data in an Excel file:

- 1 Open the Excel file that contains the baseline you want to edit.
- 2 Go to the sheet that contains the baseline. The sheet name corresponds to the baseline source name in the Test Manager **Baseline Criteria** section.
- 3 Edit the signal data in the sheet, and then save the Excel file.

See Also

More About

- “Create and Edit Signal Data”
- “Inspect Simulation Data”
- “Compare Simulation to Baseline Data” on page 6-246

Create and Run Test Cases with Scripts

In this section...

“Create and Run a Baseline Test Case” on page 6-115
 “Create and Run an Equivalence Test Case” on page 6-116
 “Run a Test Case and Collect Coverage” on page 6-117
 “Create and Run Test Case Iterations” on page 6-117

Create and Run a Baseline Test Case

This example shows how to use `sltest.testmanager` functions, classes, and methods to automate tests and generate reports. You can create a test case, edit the test case criteria, run the test case, export simulation output, and generate results reports programmatically. The example compares the simulation output of the model to a baseline.

```
% Open the model for this example
openExample('sldemo_absbrake');

% Create the test file, test suite, and test case structure
tf = sltest.testmanager.TestFile('API Test File');
ts = createTestSuite(tf,'API Test Suite');
tc = createTestCase(ts,'baseline','Baseline API Test Case');

% Remove the default test suite
tsDel = getTestSuiteByName(tf,'New Test Suite 1');
remove(tsDel);

% Assign the system under test to the test case
setProperty(tc,'Model','sldemo_absbrake');

% Capture the baseline criteria
baseline = captureBaselineCriteria(tc,'baseline_API.mat',true);

% Test a new model parameter by overriding it in the test case
% parameter set
ps = addParameterSet(tc,'Name','API Parameter Set');
po = addParameterOverride(ps,'m',55);

% Set the baseline criteria tolerance for one signal
sc = getSignalCriteria(baseline);
sc(1).AbsTol = 9;

% Run the test case and return an object with results data
ResultsObj = run(tc);

% Get the test case result and the Sim Output run dataset
tcr = getTestCaseResults(ResultsObj);
runDataset = getOutputRuns(tcr);

% Open the Test Manager so you can view the simulation
% output and comparison data
sltest.testmanager.view;

% Generate a report from the results data
```

```
filePath = 'test_report.pdf';
sltest.testmanager.report(ResultsObj,filePath,...
    'Author','Test Engineer',...
    'IncludeSimulationSignalPlots',true,...
    'IncludeComparisonSignalPlots',true);

% Export the Sim Output run dataset
dataset = export(runDataset);
```

The test case fails because only one of the signal comparisons between the simulation output and the baseline criteria is within tolerance. The results report is a PDF and opens when it is completed. For more report generation settings, see the `sltest.testmanager.report` function reference page.

Create and Run an Equivalence Test Case

This example compares signal data between two simulations to test for equivalence.

```
% Open the model for this example
openExample('sldemo_absbrake');

% Create the test file, test suite, and test case structure
tf = sltest.testmanager.TestFile('API Test File 1');
ts = createTestSuite(tf,'API Test Suite');
tc = createTestCase(ts,'equivalence','Equivalence Test Case');

% Remove the default test suite
tsDel = getTestSuiteByName(tf,'New Test Suite 1');
remove(tsDel);

% Assign the system under test to the test case
% for Simulation 1 and Simulation 2
setProperty(tc,'Model','sldemo_absbrake','SimulationIndex',1);
setProperty(tc,'Model','sldemo_absbrake','SimulationIndex',2);

% Add a parameter override to Simulation 1 and 2
ps1 = addParameterSet(tc,'Name','Parameter Set 1','SimulationIndex',1);
po1 = addParameterOverride(ps1,'Rr',1.20);

ps2 = addParameterSet(tc,'Name','Parameter Set 2','SimulationIndex',2);
po2 = addParameterOverride(ps2,'Rr',1.24);

% Capture equivalence criteria
eq = captureEquivalenceCriteria(tc);

% Set the equivalence criteria tolerance for one signal
sc = getSignalCriteria(eq);
sc(1).AbsTol = 2.2;

% Run the test case and return an object with results data
ResultsObj = run(tc);

% Open the Test Manager so you can view the simulation
% output and comparison data
sltest.testmanager.view;
```

In the Equivalence Criteria Result section of the Test Manager results, the `yout.Ww` signal passes because of the tolerance value. The other signal comparisons do not pass, and the overall test case fails.

Run a Test Case and Collect Coverage

This example shows how to use a simulation test case to collect coverage results. To collect coverage, you need a Simulink Coverage license.

```
% Open the model for this example
openExample('sldemo_autotrans');

% Create the test file, test suite, and test case structure
tf = sltest.testmanager.TestFile('API Test File');
ts = createTestSuite(tf, 'API Test Suite');
tc = createTestCase(ts, 'simulation', 'Coverage Test Case');

% Remove the default test suite
tsDel = getTestSuiteByName(tf, 'New Test Suite 1');
remove(tsDel);

% Assign the system under test to the test case
setProperty(tc, 'Model', 'sldemo_autotrans');

% Turn on coverage settings at test-file level
cov = getCoverageSettings(tf);
cov.RecordCoverage = true;

% Enable MCDC and signal range coverage metrics
cov.MetricSettings = 'mr';

% Run the test case and return an object with results data
rs = run(tf);

% Get the coverage results
cr = getCoverageResults(rs);

% Open the Test Manager to view results
sltest.testmanager.view;
```

In the **Results and Artifacts** pane of the Test Manager, click on Results. You can view the aggregated coverage results.

Create and Run Test Case Iterations

This example shows how to create test iterations. You can create table iterations programmatically that appear in the **Iterations** section of a test case. The example creates a simulation test case and assigns a Signal Editor scenario for each iteration.

```
% Open the model for this example
openExample('sldemo_autotrans');

% Create test file, test suite, and test case structure
tf = sltest.testmanager.TestFile('Iterations Test File');
ts = getTestSuites(tf);
tc = createTestCase(ts, 'simulation', 'Simulation Iterations');
```

```
% Specify model as system under test
setProperty(tc, 'Model', 'sldemo_autotrans');

% Set up table iteration
% Create iteration object
testItr1 = sltestiteration;
% Set iteration settings
setTestParam(testItr1, 'SignalEditorScenario', 'Passing Maneuver');
% Add the iteration to test case
addIteration(tc, testItr1);

% Set up another table iteration
% Create iteration object
testItr2 = sltestiteration;
% Set iteration settings
setTestParam(testItr2, 'SignalEditorScenario', 'Coasting');
% Add the iteration to test case
addIteration(tc, testItr2);

% Run test case that contains iterations
results = run(tc);

% Get iteration results
tcResults = getTestCaseResults(results);
iterResults = getIterationResults(tcResults);
```

See Also

sltest.testmanager.TestFile | sltest.testmanager.TestSuite |
sltest.testmanager.TestCase | sltest.testmanager.TestCaseResult |
sltestiteration | sltest.testmanager.TestIteration |
sltest.testmanager.TestIterationResult | sltest.testmanager.ParameterSet |
sltest.testmanager.BaselineCriteria | sltest.testmanager.EquivalenceCriteria |
sltest.testmanager.run | sltest.testmanager.report

More About

- “Import Test Cases for Equivalence Testing” on page 5-16
- “Run Tests for Variant Models Using Variant Configurations”
- “Use Scenarios and Insert Signals in Signal Editor”

Test Models Using MATLAB-Based Simulink Tests

A MATLAB-based Simulink test is defined in a MATLAB code (.m) file that you create in MATLAB, and then open, run, and view results in the Test Manager. The test file is a class definition file that inherits from `sltest.TestCase`. The inheritance enables you to open the test file in the Test Manager. When you open a MATLAB test file in the Test Manager, it appears and behaves the same as a test created in the Test Manager, although with some limited functionality (see “Limitations of MATLAB- Based Tests” on page 6-123). In addition to using a MATLAB test in the Test Manager, you can use the MATLAB test at the command line like any other unit test file.

Because these test files are text (.m) files, you can edit, compare to and merge with other .m test files, and link from the file to requirements. In contrast, test files created in the Test Manager or by using the Simulink Test API are saved as binary MLDATX files.

Classes and Methods

TestCase Class and Methods

The `sltest.TestCase` class and its methods work specifically with MATLAB tests. You can use these methods in test files and at the command line, except for `sltest.TestCase.forInteractiveUse`, which can only be used at the command line. In addition to these methods, you can use the `matlab.unittest.TestCase` methods with MATLAB tests.

<code>sltest.TestCase</code>	Class from which to inherit
<code>loadSystem</code>	Loads model
<code>simulate</code>	Simulate model
<code>assumeSignalsMatch</code>	Assume two sets of data are equivalent
<code>assertSignalsMatch</code>	Assert two sets of data are equivalent
<code>fatalAssertSignalsMatch</code>	Fatal assert two sets of data are equivalent
<code>verifySignalsMatch</code>	Verify two sets of data are equivalent
<code>sltest.TestCase.forInteractiveUse</code>	Create test case for use at command line
<code>createTemporaryFolder</code>	Create temporary folder that is deleted when test case goes out of scope
<code>createSimulationInput</code>	Creates <code>Simulink.SimulationInput</code> or <code>sltest.harness.SimulationInput</code> object.

Test Harness Class

`sltest.harness.SimulationInput` creates an object that you can use to specify changes applied to a test harness during simulation. In addition to using this class for MATLAB-based Simulink tests, you can use it in other MATLAB code.

Test Runner Methods

These methods of `matlab.unittest.TestRunner` apply specifically to MATLAB-based Simulink tests.

- `addModelCoverage` — enables model coverage collection using the test runner. Instead of using this method, if you open your MATLAB-based Simulink test file in the Test Manager, you can enable coverage collection there.

- `addSimulinkTestResults` — pushes test results to the Simulink Test Manager.

Plugin Classes

These `sltest.plugins` classes enable functionality for MATLAB-based tests. In addition to these methods, you can use other `sltest.plugins` classes with these tests. The plugins can be attached to a `matlab.unittest.TestRunner` to enable functionality while running an `sltest.TestCase` test.

<code>sltest.plugins.MATLABTestCaseIntegrationPlugin</code>	Enable integrating MATLAB test simulation and test results with the Test Manager
<code>sltest.plugins.ToTestManagerLog</code>	Enable writing text output to Test Manager results Logged Signals pane of the Test Manager
<code>sltest.plugins.ModelCoveragePlugin</code>	Enable collecting model coverage

Creating a Baseline MATLAB-Based Simulink Test

To create a baseline MATLAB test:

- 1 Create a MATLAB code (.m) file that defines the test cases. You can launch the MATLAB Editor from the command line, or from the Test Manager by using **New > Test Case > MATLAB-Based Simulink Test**.

See “Class-Based Unit Tests”. The only difference for MATLAB tests is that the class must inherit from `sltest.TestCase`, instead of from `matlab.unittest.TestCase`.

This sample MATLAB test file includes one test, which is defined in the `testOne` function. When you run the test in the Test Manager, the test loads the model named `sltestMATLABBasedTestExample`. It then sets the value of the `gain2_var` variable, and simulates the model. Finally, the test compares model simulation output to the baseline data MAT file.

```
classdef myTest < sltest.TestCase
    methods (Test)
        function testOne(testCase)
            testCase.loadSystem...
                ('sltestMATLABBasedTestExample');
            evalin('base','gain2_var = 2.01;');
            simOut = testCase.simulate...
                ('sltestMATLABBasedTestExample');
            testCase.verifySignalsMatch(simOut,'baselineOne.mat',...
                'AbsTol',0.015);
        end
    end
end
```

- 2 If you are using a test harness, replace the `simOut` line in the above MATLAB test file with

```
simOut = testCase.simulate('sltestMATLABBasedTestExample',...
    'WithHarness','sltestMATLABBasedTestExample_harness1');
```

or, to specify the stop the simulation time, replace `simOut` with

```
in = testCase.createSimulationInput('sltestMATLABBasedTestExample',...
    'WithHarness','sltestMATLABBasedTestExample_harness1');
```



```
in = in.setModelParameter("StopTime","10")
simOut = testCase.simulate(in);
```

- 3 If a baseline data MAT file does not already exist or if you need to update it, at the MATLAB command line, use:

```
runtests(<test>,'GenerateBaselines',true)
```

For the sample file, <test> is 'myTest/testOne'.

When you generate baselines, the test begins running. It pauses to open a Simulation Data Inspector report, and you are prompted at the MATLAB command line to review the baseline data. When you approve the data, it saves the baseline data to a new MAT file or updates the existing MAT file. Then, the test continues to run, but fails because the new or updated baseline data is not included in the current run. Rerun the test using the `runtests` command to use the new or updated baseline and produce a passing result. You can also rerun the test by using the yellow rerun hyperlink at the command line.

- 4 Optionally, if you have a Simulink Coverage license, you can include coverage collection in your test by using the `sltest.plugins.ModelCoveragePlugin`. See “Collect Coverage Using MATLAB-Based Simulink Tests” on page 6-128 for an example and information on coverage collection.

Alternatively, you can collect coverage by using the Test Manager. After you open the test file in the Test Manager, select the test file, select a **Coverage Settings** option, and select the coverage metrics. The test suites and test cases in the file inherit the coverage settings from the test file. If you set the coverage collection and metrics in the Test Manager, you do not need to import or add the coverage plugins to the runner.

If you set the coverage metrics in the Test Manager and then close the Test Manager, the coverage settings are not saved to the MATLAB-based Simulink test code (.m) file.

- 5 Optionally, if you have a Requirements Toolbox license, you can add requirements. Open the Test Manager and update the test file.
 - a Click **Open > Open MATLAB-Based Simulink Test (.m)** and select the test file. The test file loads and its test hierarchy displays in the **Test Browser** pane. If you select the test file, the **Requirements** and **Test File Content** panes appear in the Test Manager.
 - b Add requirements by expanding the **Requirements** section by clicking **Add** to open the Outgoing Links Editor. See “Link to Requirements” on page 1-2 for information on adding requirements.
- 6 To update the MATLAB code (.m) test file from the Test Manager, click the **Open test in the MATLAB Editor** link.
- 7 After you edit the code file and save your changes, or after adding coverage or requirements, return to the Test Manager and click the synchronization button  next to the test file in the **Test Browser** pane.
- 8 Run the test, view the results, and create a test results report.
 - a Click **Run** to run the test.
 - b To view the results, expand the rows in the **Results and Artifacts** pane.
 - c To view coverage results, in the **Results and Artifacts** pane, select the **Results** item and expand the **Aggregated Coverage** section. See “Collect Coverage in Tests” on page 6-151 for information.

- d Optionally, create a test results report. See “Create, Set Options, and Use Templates for Test Result Reports” on page 7-17.

Alternatively, instead of adding coverage (Step 4) and running the test (Step 8) in the Test Manager, you can use these commands at the MATLAB command line to add coverage, run the test, and push the results to the Test Manager. To collect coverage, you must add the `MATLABTestCaseIntegrationPlugin` and the `ModelCoveragePlugin` to the `testrunner`. The `addModelCoverage` function adds the `ModelCoveragePlugin`.

```
suite = testsuite('myTests');
runner = testrunner('textoutput');
runner.addModelCoverage(...
    "CollectMetrics",["MCDC","Condition"]);
runner.addPlugin(sltest.plugins.MATLABTestCaseIntegrationPlugin);
runner.run(suite);
```

Linking to Requirements from a MATLAB-Based Simulink Test File

Note You must have Requirements Toolbox to include requirements links.

To add links to requirements from a file being edited in the MATLAB Editor, see “Link MATLAB Code to Requirements” (Requirements Toolbox). For MATLAB test files, you add links using the same process. However, the text you select in the MATLAB code (.m) file determines the type of link and the test to which it is added. If you select:

- Class definition line (e.g., `classdef myClass < sltest.TestCase`) — Adds a **Verified By** link for the whole test file
- Text inside a test function — Adds a **Verified By** link for that function
- Text across multiple test functions — Adds a **Verified By** link for the first function in the selection
- Any other text selected — Adds a **Related To** link for the selection

After you add requirements links, you can view the verification status in the Requirements Editor by clicking **Display > Verification Status**. To update the status of a **Verified By** requirement, right-click on the requirement and select **Run Tests**. See “Review Requirements Verification Status” (Requirements Toolbox).

Links that you create in the MATLAB code (.m) file appear in the **Requirements** section of the Test Manager. Linking to requirements from in the Test Manager works the same as described in “Link to Requirements” on page 1-2.

When you have the Requirements Editor open and you click on an incoming link that is for a MATLAB test, if you have a Simulink Test license, the Test Manager opens and goes to the associated test. If a license is not available, the MATLAB Editor opens and goes to the line of code associated with that requirement.

For a parameterized test, Requirements Toolbox does not support linking to individual parameterized versions of the test. In your .m file, if you create a link from parameterized test to a requirement, the link is associated with all versions of that test. In the Test Manager, if you create a link from a version of a parameterized test to a requirement, the link is associated with all versions of that test.

Limitations of MATLAB- Based Tests

MATLAB-based Simulink tests do not support:

- Test types other than baseline tests.
- Running tests in parallel.
- Running tests in multiple releases.
- Test tags and descriptions.
- Callbacks. (However, while callbacks are not supported in the Test Manager for MATLAB-based tests, you can use `TestClassSetup` and `TestMethodSetup`, or fixtures in your `.m` file to achieve similar functionality. See “Write Setup and Teardown Code Using Classes”.)
- Logical and temporal assessments.
- Enabling coverage collection or changing coverage metrics at the test suite or test file level in the Test Manager.
- Saving coverage settings to the MATLAB-based Simulink test MATLAB code (`.m`) file from the Test Manager.

See Also

`addSimulinkTestResults` | `addModelCoverage` | `sltest.plugins.ModelCoveragePlugin` | `sltest.plugins.MATLABTestCaseIntegrationPlugin` | `sltest.TestCase` | `matlab.unittest.TestCase` | `matlab.unittest.TestRunner`

Related Examples

- “Using MATLAB-Based Simulink Tests in the Test Manager” on page 6-124
- “Collect Coverage Using MATLAB-Based Simulink Tests” on page 6-128
- “Test Models Using MATLAB Unit Test” on page 6-209
- “Link to Requirements” on page 1-2
- “Link MATLAB Code to Requirements” (Requirements Toolbox)
- “Link Test Cases to Requirements” (Requirements Toolbox)

Using MATLAB-Based Simulink Tests in the Test Manager

This example shows how to create a MATLAB®-based Simulink® test, generate a baseline, and load, run, and view test results in the Test Manager. When you load a MATLAB-based Simulink test case code .m file into the Test Manager, the test case appears and behaves the same as a test case created directly in the Test Manager. The only difference is that for a MATLAB-based Simulink test, you can select or change coverage collection and metrics only at the test file level, not at either the test suite or test case level.

This example, which uses the internal test harness `sltestMATLABBasedTestExample_harness` verifies the `sltestMATLABBasedTestExample` model against a generated baseline.

Baseline Test Class Definition File

The class definition file, `BaselineTest.m`, has already been created and is provided with this example.

The test case file, `BaselineTest.m`, is derived from `sltest.TestCase`, which in turn is derived from `matlab.unittest.TestCase`. All of the `matlab.unittest.TestCase` methods are also available as a part of `sltest.TestCase`.

Baseline Test File Contents

The class definition file, `BaselineTest.m`, contains:

```
classdef BaselineTest < sltest.TestCase

    methods (Test)
        function testOne(testCase)
            testCase.loadSystem('sltestMATLABBasedTestExample');
            evalin('base','gain2_var = 2.01;');
            simOut = testCase.simulate('sltestMATLABBasedTestExample',...
                'WithHarness','sltestMATLABBasedTestExample_harness');
            testCase.verifySignalsMatch(simOut,'baseline1.mat','AbsTol',0.015);
        end
    end
end
```

The file includes:

- Inheritance from `sltest.TestCase`.
- A test function named `testOne`, which is in a `methods` block that has the `Test` attribute.

The `testOne` function:

- Uses the `testCase.loadSystem` method to load the `sltestMATLABBasedTestExample` model.
- Changes the value of the `gain2_var` in the model to 2.01.
- Uses the `testCase.simulate` method to simulate the model with the harness.
- Uses the `testCase.verifySignalsMatch` method to compare the output of `simulate`, `simOut`, to the baseline data MAT-file named `baseline1.mat`. It also sets an absolute tolerance. If you remove the tolerance setting from the file before running the test, the test fails because the value of `gain2_var` was changed from its original value in the model.

Baseline Data File

The baseline data file, `baseline1.mat`, has already been generated and is provided with this example. The baseline data file was created using this process:

1. Use `runtests('BaselineTest/testOne','GenerateBaselines',true)`.
2. After the baseline test runs, a Simulation Data Inspection report shows the output from the signals. View the Actual Results in the report and approve the baseline data. The data is saved in a MAT-file, which for this example is named `baseline1.mat`.

Open the MATLAB-based Simulink Test in the Test Manager

Open the Test Manager.

```
sltest.testmanager.view
```

In the Test Manager, click **Open** and select **Open MATLAB-based Simulink Test (.m)**.

In the Open File dialog box, select `BaselineTest.m`.

The Test Manager populates the **Test Browser** with **testOne** from the `BaselineTest.m` file.



Set Up Coverage Collection

If you want to collect coverage for the test, select the `BaselineTest` file. In the Coverage Settings section, select the Coverage to Collect and the Coverage Metrics. If you want to collect coverage for the test, select the test file, `BaselineTest`, in the **Test Browser** pane. In the Coverage Settings section, select a **Coverage to Collect** option and **Coverage Metrics** options.

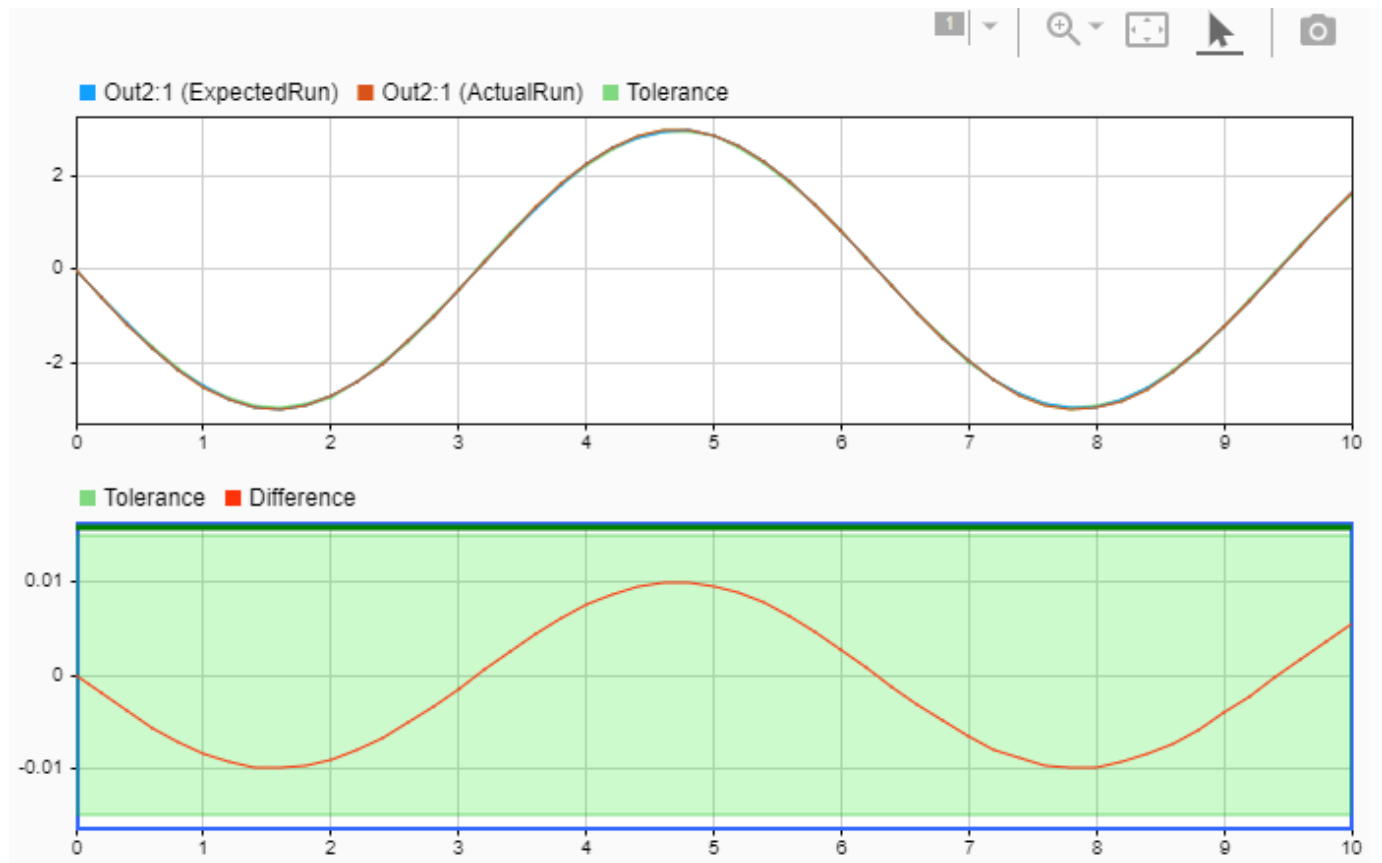
Run and Visualize the Results in the Test Manager

Click **Run** to execute the test.

After the test completes, expand all rows in the **Results and Artifacts** pane. Notice that `testOne` passes.

Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: test	
NAME	STATUS
▼ Results: 2020-Jun-11 09:47:14	2 ✓
▼ BaselineTest	2 ✓
▼ testOne	✓
▼ Simulation Output Comparison Result	✓
○ Out1:1	✓
○ Out2:1	✓
▼ Sim Output (sltestMATLABBasedTestE	
□ Out1:1	
□ Out2:1	

To view the data comparison, select **Out2:1** under **testOne > Simulation Output Comparison Results**. The simulation and baseline signals match within the specified tolerance.



If you collected coverage, select the `BaselineTest` test file and view the **Aggregated Coverage Results** section.

Clear and close the Test Manager

```
sltest.testmanager.clear  
sltest.testmanager.clearResults  
sltest.testmanager.close
```

See Also

`sltest.TestCase` | `loadSystem` | `simulate` | `verifySignalsMatch` | `runtests`

Related Examples

- “Test Models Using MATLAB-Based Simulink Tests” on page 6-119

Collect Coverage Using MATLAB-Based Simulink Tests

This example shows how to use a MATLAB®-based Simulink® test to collect coverage on a model with a test harness, and use the MATLAB Test Framework to populate the results in the Test Manager. MATLAB-based Simulink tests are MATLAB code (.m) file test case class definitions that inherit from `sltest.TestCase`.

MATLAB-Based Simulink Test File

The MATLAB-based Simulink test file, `TestHarnessWithModelCoverage.m`, has been created and is provided with this example. The test file contains two test functions. Each one has a harness model to drive input data to test the subsystem `TestHarnessWithModelCoverage/Subsystem1` and compare with corresponding baseline. This test uses a `Simulink.SimulationOutput` object when simulating the model.

```
classdef TestHarnessWithModelCoverage < sltest.TestCase

    methods (Test)
        function testOne(testCase)
            in = testCase.createSimulationInput('simpleSwitchWithSubsystemIn',...
                'WithHarness','simpleSwitchWithSubsystemIn_Harness1');
            simOut = testCase.simulate(in);
            testCase.verifySignalsMatch(simOut,'baselineOne.mat');
        end
        function testTwo(testCase)
            in = testCase.createSimulationInput('simpleSwitchWithSubsystemIn',...
                'WithHarness','simpleSwitchWithSubsystemIn_Harness2');
            simOut = testCase.simulate(in);
            testCase.verifySignalsMatch(simOut,'baselineTwo.mat');
        end
    end
end

end
```

Create a TestRunner and Test Suite

Create a `TestRunner` to run the `sltest_ratelim` model.

```
import matlab.unittest.TestRunner;
runner = TestRunner.withTextOutput;
```

Create a `TestSuite` to use with the `TestRunner`.

```
suite = testsuite('TestHarnessWithModelCoverage');
```

Configure the Test Runner

Use plugin methods to configure the `TestRunner` to add test results from an `sltest.TestCase` to the Test Manager. Add the `TestRunnerPlugin` to the `TestRunner`.

```
import sltest.plugins.MATLABTestCaseIntegrationPlugin;
runner.addPlugin(MATLABTestCaseIntegrationPlugin);
```

The `DiagnosticsOutputPlugin` and the `ToTestManagerLog` stream the diagnostics from an `sltest.TestCase` run to the logs of `TestCaseResults` in the Test Manager. The diagnostics

include passing diagnostics for tests that pass. Add the `DiagnosticsOutputPlugin` and `ToTestManagerLog` to the `TestRunner`.

```
import sltest.plugins.ToTestManagerLog;
import matlab.unittest.plugins.DiagnosticsOutputPlugin;
streamOutput = ToTestManagerLog();
diagnosticsOutputPlugin = DiagnosticsOutputPlugin...
    (streamOutput, 'IncludingPassingDiagnostics', true);
runner.addPlugin(diagnosticsOutputPlugin);
```

Configure Coverage Collection for a Simulink Model

Models in an `sltest.TestCase` that are simulated using the `simulate` method can collect coverage. Use the `ModelCoveragePlugin` to configure coverage metrics collection. This example collects MCDC coverage. Add the `ModelCoveragePlugin` to the `TestRunner`.

```
import sltest.plugins.coverage.CoverageMetrics;
import sltest.plugins.ModelCoveragePlugin;
mcdcMetrics = CoverageMetrics('MDCD', true);
runner.addPlugin(ModelCoveragePlugin('Collecting', mcdcMetrics));
```

Alternatively, you can turn on coverage collection and set coverage metrics in the Test Manager. If you use this alternative, you do not need to import or add the coverage plugins to the runner.

Collect and Add Coverage and Test Results to the Test Manager

Now that the `TestRunner` is fully configured, use the `run` function to collect coverage and add the coverage and test results to the Test Manager.

```
run(runner, suite);
```

```
Setting up ResultTestFixture
Done setting up ResultTestFixture
```

```
Running TestHarnessWithModelCoverage
..
Done TestHarnessWithModelCoverage
```

```
Coverage Report for simpleSwitchWithSubsystemIn/Subsystem1
C:\TEMP\tpb9dde44b_1cf5_4426_885a_4eb72bce9d69.html
Tearing down ResultTestFixture
Done tearing down ResultTestFixture
```

`run` also generates a report that includes cumulative coverage for the test suite that was run. Use the [Coverage Report for sltest_ratelim](#) link to view the report.

Open the Test Manager

```
sltest.testmanager.view
```

Select the **Results and Artifacts** pane and expand the **Results** and **BaselineTestWithCoverage** rows.

Test Browser

Results and Artifacts

NAME	STATUS
▼ Results: 2020-Dec-16 20:56:32	2
▼ TestHarnessWithModelCoverage	2
testOne	
testTwo	

Select the `testOne` row.

Test Browser

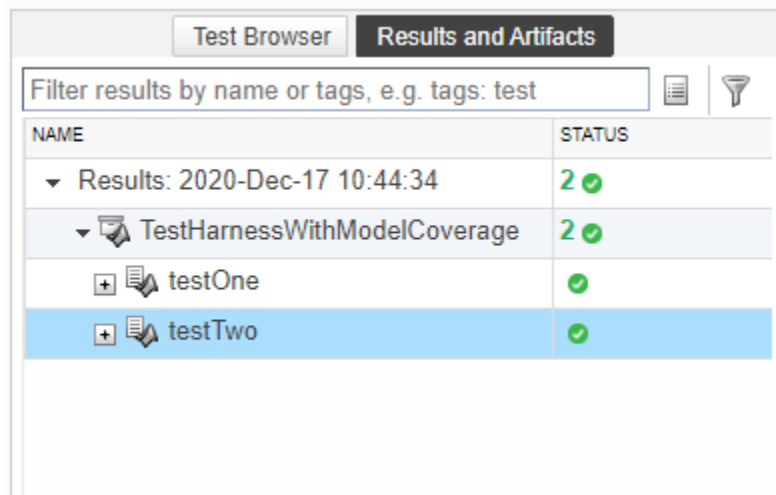
Results and Artifacts

NAME	STATUS
▼ Results: 2020-Dec-17 10:44:34	2 ✓
▼  TestHarnessWithModelCoverage	2 ✓
  testOne	✓
  testTwo	✓

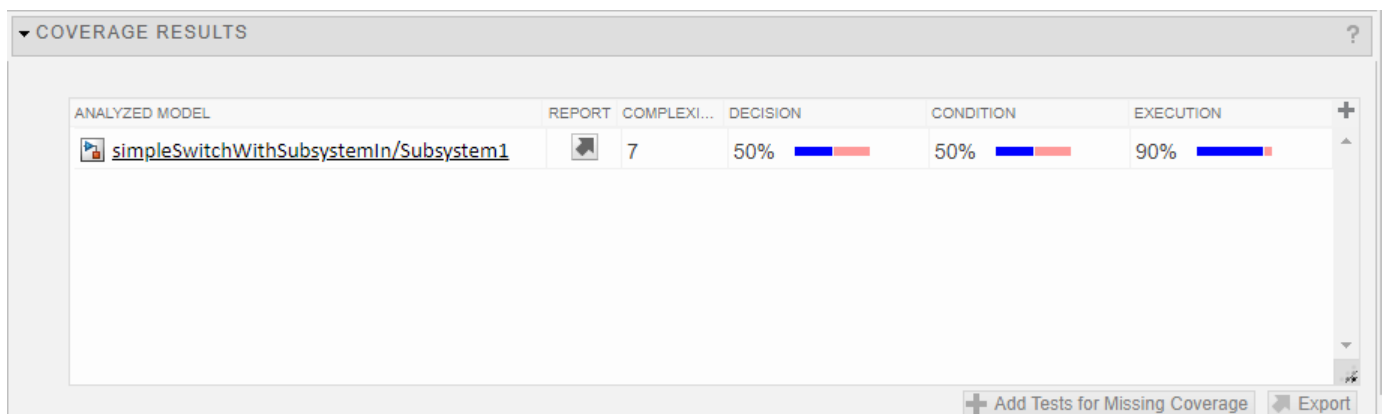
The **Coverage Results** section shows the coverage collected for `sltest_ratelim` from `test0ne`.

COVERAGE RESULTS						
ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	EXECUTION	
 <u>simpleSwitchWithSubsystemIn/Subsystem1</u>		7	50%	50%	90%	

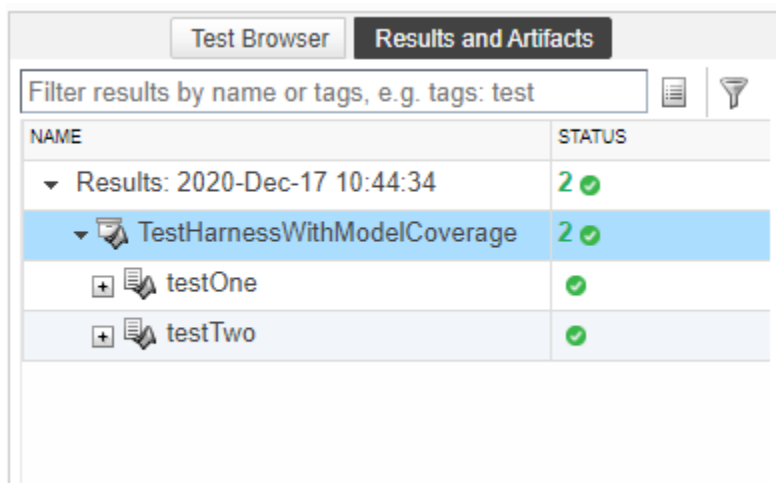
Select the testTwo row.



The **Coverage Results** section shows the coverage collected for `sltest_ratelim` from `testTwo`.



Select the `BaselineTestWithCoverage` row.



The **Aggregated Coverage Results** section shows the aggregation of the coverage collected for `sltest_ratelim` from `testOne` and `testTwo`. The aggregated results show full coverage for the specified coverage metrics.

▼ AGGREGATED COVERAGE RESULTS ?					
ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	EXECUTION
 simpleSwitchWithSubsystemIn/Subsystem1		7	100%	100%	100%

+ Add Tests for Missing Coverage  Export

See Also

`sltest.TestCase` | `matlab.unittest.TestRunner` |
`sltest.plugins.ModelCoveragePlugin` | `sltest.plugins.coverage.CoverageMetrics` |
`sltest.plugins.MATLABTestCaseIntegrationPlugin` |
`sltest.plugins.ToTestManagerLog` |
`matlab.unittest.plugins.DiagnosticsOutputPlugin` | `verifySignalsMatch` |
`Simulink.SimulationOutput`

Create and Run a RoadRunner Test Case

You can use the Test Manager or Simulink Test API commands to create and run RoadRunner test cases. RoadRunner automates different types of agents on ego vehicles in road scenario simulations.

Note To run a RoadRunner test case, you must have a RoadRunner and a RoadRunner Scenario license.

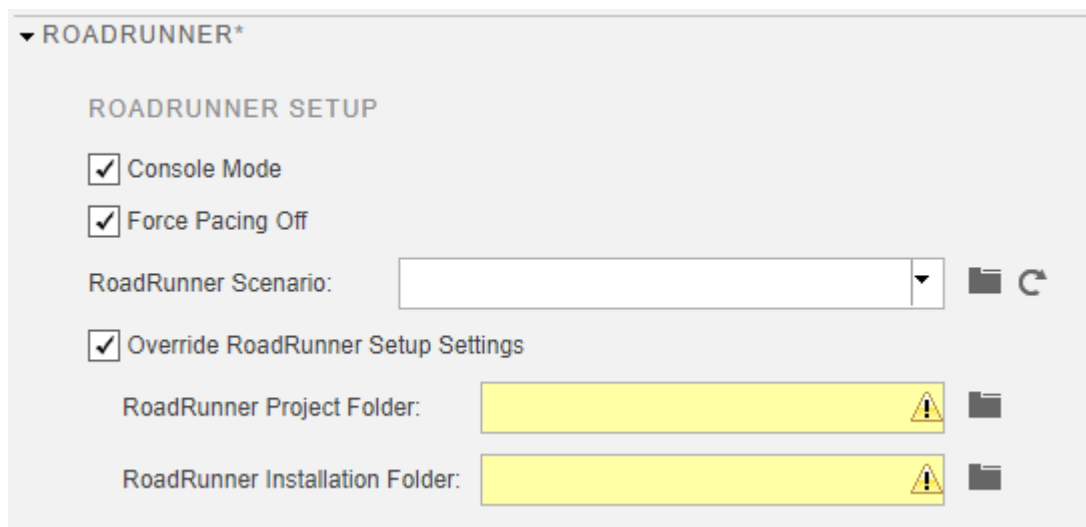
Set Up RoadRunner and Simulink Model

Before you can run a RoadRunner test case, set up your RoadRunner project and scenario using the RoadRunner application and verify that the scenario runs as expected. See “Simulate RoadRunner Scenarios with Actors Modeled in Simulink” (Automated Driving Toolbox), “RoadRunner Project and Scene System” (RoadRunner), and “Create Simple RoadRunner Scene” (RoadRunner).

You must also develop the Simulink model that you use as a behavior in RoadRunner. You also use this model as the system under test in Simulink Test.

Use the Test Manager to Run a RoadRunner Test

- 1 Open the Test Manager by using `sltest.testmanager.view`.
- 2 Create a test file by clicking **New > Test File**.
- 3 Create a RoadRunner test case template by clicking **New > Test Case > RoadRunner Test**. The **RoadRunner** section of the test case opens.



- 4 Set the simulation options and specify the folder and file locations.
 - **Console Mode** — When you select this option, the RoadRunner user interface is not displayed when the test runs.
 - **Force Pacing Off** — When you select this option, the RoadRunner simulation runs at its normal speed. When this option is cleared, you can tune the pace in the RoadRunner application to control the speed of the simulation.

- **RoadRunner Scenario** — Full path to RoadRunner scenario, which is usually in the project folder. To populate the drop-down menu with the scenarios in the project folder, click the Refresh button . Then, select a RoadRunner scenario.

Note You can test only one RoadRunner scenario for each test case. To test multiple scenarios, use separate test cases or test iterations.

- **Override RoadRunner Setup Settings** — When you select this option, you override the preferred path settings and the section displays the **RoadRunner Project Folder** and **RoadRunner Installation Folder** fields. When you clear this option, the test case uses the preferred path settings. For more information on how to specify preferred path settings, see `roadrunnerSetup`. For information on specifying path settings when testing in parallel, see “Executing RoadRunner Tests in Parallel” on page 6-137.
 - **RoadRunner Project Folder** — Full path to the RoadRunner project folder. When you make changes to this field, you must refresh the list of scenarios before you make a selection from the **RoadRunner Scenario** drop-down menu.
 - **RoadRunner Installation Folder** — Full path to the RoadRunner executable folder.
- 5 In the **System Under Test** section, enter the name of the Simulink model that you attached to the ego vehicle in the RoadRunner scenario. The folder for the model must be on the MATLAB path.
 - 6 In the **Iterations** section, you can choose whether to stop the test case execution when a test iteration failure occurs by selecting **Enable stop on failure for test iterations**. Use the **Scripted Iterations** section to set model parameters or RoadRunner scenario variables.

This example shows two scenario variables defined in RoadRunner.

Variables			
	Name	Value	
1	Ego_Car_Initial_Speed	15	
2	Yellow_Car_Initial_Speed	30	

This image shows how to set the values of the variables in two iterations and add the iterations to the test case.

▼ SCRIPTED ITERATIONS*

▶ [Help on creating test iterations:](#)

```

1 testItr1 = sltestiteration;
2 setRoadRunnerScenarioVariable(testItr1,"Ego_Car_Initial_Speed","13");
3 addIteration(sltest_testCase,testItr1,"Iteration 1");
4
5 testItr2 = sltestiteration;
6 setRoadRunnerScenarioVariable(testItr2,"Yellow_Car_Initial_Speed","20");
7 addIteration(sltest_testCase,testItr2,"Iteration 2");

```

Iteration Templates*Generate an iteration script using templates***Show Iterations***Show the list of iterations that will execute*☐ Enable stop on failure for test iterations

You can also define custom variables in the **Scripted Iterations** section and then use them in a callback. For example, this image shows how to define a custom variable to use in two iterations, then use a callback to print the variable.

▼ SCRIPTED ITERATIONS*

▶ [Help on creating test iterations:](#)

```

1 testItr1 = sltestiteration;
2 setRoadRunnerCustomVariable(testItr1,"tag","PRINT_ME_ITERATION_1");
3 addIteration(sltest_testCase,testItr1,"Iteration 1");
4
5 testItr2 = sltestiteration;
6 setRoadRunnerCustomVariable(testItr2,"tag","PRINT_ME_ITERATION_2");
7 addIteration(sltest_testCase,testItr2,"Iteration 2");

```

▼ POST-LOAD* 

▼ Runs after the model loads and the model PostLoadFcn callback

The following variables have been pre-defined for you:

- sltest_bdroot : Model simulated by the test case. Can be a harness model
- sltest_sut : The system under test. For a harness, it is the component under test
- sltest_isharness: Returns true if sltest_bdroot is a harness model
- sltest_iterationName: Name of current test iteration
- sltest_testIteration: Current test iteration object
- sltest_testCase: Current test case object
- sltest_roadRunnerApp: RoadRunner Application object
- sltest_roadRunnerSim: RoadRunner Simulation object

```

1 tag = sltest_testIteration.getRoadRunnerCustomVariableValueByName('tag');
2 disp(tag); % this prints out the value of the custom variable previously authored

```

- 7 In the **Callbacks** section, in addition to other variables, you can use these two RoadRunner-specific variables in RoadRunner test cases:
 - sltest_roadRunnerApp — RoadRunner application object
 - sltest_roadRunnerSim — RoadRunner simulation object
- 8 Use other test case sections as needed. Other test case sections work the same for RoadRunner tests as they do for other types of tests. See “Specify Test Properties in the Test Manager” on page 6-173.
- 9 Click **Run** to run the RoadRunner test. A RoadRunner session starts and runs the scenario simulation and model. A RoadRunner application session opens and then closes when each iteration is complete. If you selected **Console Mode**, the RoadRunner interface does not display.

The test uses the simulation mode set in the Simulink model. You cannot override the simulation mode for a RoadRunner test case in the Test Manager. To change the simulation mode, change the mode in the Simulink model.

- 10 When the test finishes running, the **Simulation Metadata** table in the **Results Summary** section includes RoadRunner information.

▼ Simulation Metadata	
RoadRunner Project Folder	/home/bchen/Desktop/RoadRunner/Project
RoadRunner Installation Folder	/usr/local/RoadRunner_R2023a/bin/glnxa64
RoadRunner Scenario	/home/bchen/Desktop/RoadRunner/Project/Scenarios/slbehavior.rrscenario
RoadRunner Console Mode	true
RoadRunner Pacing Off	true
Model	testCar1
Simulation Mode	normal

- 11 Optionally, generate reports of the RoadRunner test specifications and test results.

Use Simulink Test API Commands to Create and Run a RoadRunner Test

For general information on using Simulink Test API commands to create a test script, see “Create and Run Test Cases with Scripts” on page 6-115. To create a RoadRunner test using the API:

- 1 Create a test file by using `sltest.testmanager.TestFile`.
- 2 Create a new test suite by using `sltest.testmanager.TestSuite`.
- 3 Create a RoadRunner test case by using `createTestCase` with the `runOnPlatform="RoadRunner"` input argument.
- 4 Specify the RoadRunner options by using these `setProperty` method name-value arguments:
 - `RoadRunnerRunInConsoleMode`
 - `RoadRunnerTurnOffPacing`
 - `RoadRunnerProjectFolder`
 - `OverrideRoadRunnerDefaultSettings`
 - `RoadRunnerInstallationFolder`
 - `RoadRunnerScenarioPath`
 - `StopOnFailure`
- 5 Assign the Simulink model as the system under test by using the `Model` name-value argument of `setProperty`.
- 6 If needed, set up test iterations by using `sltest.testmanager.TestIteration`. You can switch the RoadRunner scenario by using `setRoadRunnerScenario`. You can also define custom variables and set RoadRunner scenario variables in iterations and then use the variables in callbacks. Set the values for RoadRunner scenario variables or custom variables by using `setRoadRunnerScenarioVariable` and `setRoadRunnerCustomVariable`.
- 7 Define callbacks for the test case, if needed, by using `PreLoadCallback`, `PostLoadCallback`, or `CleanupCallback` name-value arguments of the `setProperty` method.
- 8 Run the test case.
- 9 Generate a results report, if needed, by using `sltest.testmanager.TestResultReport`.

RoadRunner Test Case Limitations

- Testing multiple Simulink agents is not supported.
- Testing MATLAB agents is not supported.
- You cannot convert between a regular test case and a RoadRunner test case.
- Multiple release testing and using the Model Slicer are not supported.
- Fast restart is not supported in the **Iterations** section of the Test Manager.

Executing RoadRunner Tests in Parallel

When you run a RoadRunner test in parallel:

- Stop on failure for test iterations is not supported.
- Testing on a remote cluster is not supported. For more information, see `parallel.Cluster` (Parallel Computing Toolbox).

- Preferred path settings only propagate to new workers. To propagate path setting changes to active workers, select **Override RoadRunner Setup Settings** and specify the **RoadRunner Project Folder** and **RoadRunner Installation Folder** fields. Otherwise, shut down the active workers before updating the default path settings and running the test.

For more information on testing in parallel, see “Run Tests Using Parallel Execution” on page 6-167.

See Also

setRoadRunnerScenario | setRoadRunnerCustomVariable |
setRoadRunnerScenarioVariable | getRoadRunnerCustomVariableValueByName |
setProperty

Related Examples

- “RoadRunner Project and Scene System” (RoadRunner)

Test Iterations

In this section...
"Create Table Iterations" on page 6-139
"Create Scripted Iterations" on page 6-143
"Sweep Through a Set of Parameters" on page 6-145

You can run the same test case with different data or configuration sets by using test case iterations. Iterations can use different:

- Parameters
- External inputs
- Configuration sets
- Variant configurations
- Signal Editor scenarios
- Test Sequence scenarios
- Baseline data
- Simulation modes
- Fault sets
- Sequence diagrams

Set up iterations in the **Iterations** section of a test case. You can create iterations by using the table in the **Iterations** section of the Test Manager or by using a script.

To use Test Sequence scenarios in iterations, in the **Inputs** section, set **Test Sequence Block** to the block that contains the scenarios. Then, to use a scenario as the default for each iteration, select the scenario from **Override with Scenario**. If you don't select a scenario, the Test Manager uses the active scenario in the Test Sequence block. Use the **Test Sequence Scenario** column in the table to change the scenario for an iteration. For more information, see "Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager" on page 3-54.

To use different simulation modes, such as normal and software-in-the-loop (SIL) simulation modes, for a baseline or simulation test, first, set up the test case. Then, in the **Iterations** table, click **Auto Generate**. In the Auto Generate Iterations dialog box, select **Simulation Modes** and one or more other options. For each option, the number of iterations created doubles, one for the mode of the model and one for SIL mode.

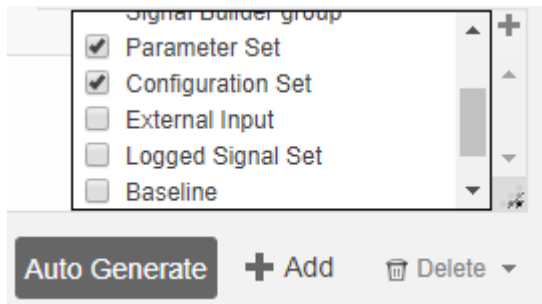
If the test collects coverage using Simulink Coverage, the same coverage settings apply to all iterations in the test case.

Whether you use table or scripted iterations, you can see the iterations in the test case by clicking the **Show Iterations** button.

Create Table Iterations

Use the **Table Iterations** section to add iterations based on items in your model or test case. To create iterations with the table, first make the appropriate columns visible:

- 1 Expand the **Iterations > Table Iterations** section.
- 2 In the table, add or remove columns by clicking the Add column  button and selecting items in the list. For example, to select display parameter and configuration sets, select the **Parameter Set** and **Configuration Set** items.



Add Iterations Manually

- 1 To manually add iterations, click **Add**. The table displays a new iteration row.
- 2 Enter an iteration name in the **Name** field and select items for the iteration. For example, this test case has four iterations. Each iteration uses a different combination of external input and baseline data.

▼ TABLE ITERATIONS*			
☒ NAME	EXTERNAL INPUT	BASELINE	
☒ Passing	BrakeThrottle_InputData.xlsx	BrakeThrottleBaseline1.mat	
☒ GradualAccel	BrakeThrottle_InputData.xlsx (1)	BrakeThrottleBaseline2.mat	
☒ HardBrake	BrakeThrottle_InputData.xlsx (2)	BrakeThrottleBaseline3.mat	
☒ Coast	BrakeThrottle_InputData.xlsx (3)	BrakeThrottleBaseline4.mat	

Generate Table Iterations

You can also automatically generate iterations from data in your test case and model.

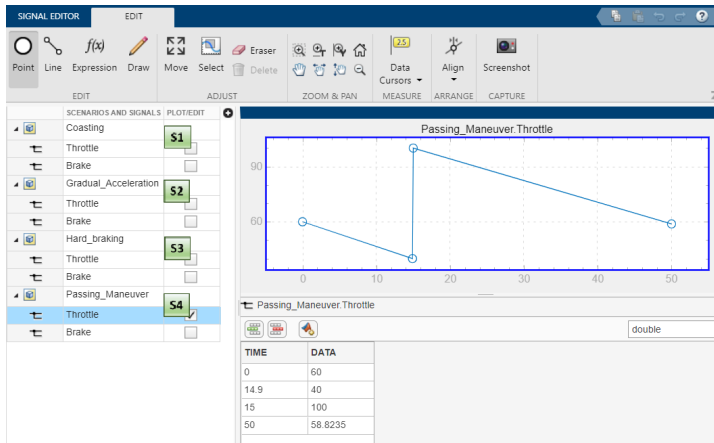
- 1 Click the **Auto Generate** button.
- 2 Select items for which to generate iterations.

For Test Sequence scenarios, the Test Manager generates an iteration for each scenario in the block you select in the **Test Sequence Block** in the **Inputs** section.

If you select multiple items, the Test Manager creates iterations in sequential pairings. For example:

- The model `sldemo_autotrans` has a Signal Editor block with four signal scenarios, named `Coasting`, `Gradual_Acceleration`, `Hard_braking`, and `Passing_Maneuver`, each of which has Throttle and Brake signals. To open this model, type `openExample("sldemo_autotrans")` at the command line. To view the Signal Editor, double-click the `ManeuversGUI` block to open the Block Parameters dialog box. Then, click the Open Signal Editor button.

- The test case has three parameter sets, labeled P1, P2, and P3.
- Automatically generating iterations from Signal Editor scenarios and parameter sets results in three iterations. The iterations are limited by the three parameter sets. Each iteration contains one Signal Editor scenario and one parameter set. The Signal Editor scenario and parameter set are matched in the order that they appear in the Signal Editor block and parameter set section.



PARAMETER OVERRIDES			
PARAMETER SET / WORKSPACE VARIABLE	OVERRIDE VALUE	SOURCE	MODEL ELEMENT
☒ Parameter Set 1	P1		
☐ Parameter Set 2	P2		
☐ Parameter Set 3	P3		

ITERATIONS				
TABLE ITERATIONS				
NAME	DESCRIPTION	REQUIREMENTS	SIGNAL BUILDER GROUP	PARAMETER SET
☒ iteration1	None	I1	Passing Maneuver	Parameter Set 1
☒ iteration2	None	I2	Gradual Acceleration	Parameter Set 2
☒ iteration3	None	I3	Hard braking	Parameter Set 3

3 Specify an optional naming rule for the iterations. In the **Iteration naming rule** box, enter the rule using:

- The name of each setting you want to use in the name, with spaces removed
- An underscore or space to separate each setting

For example, if you want to include the name of the parameter set, configuration set, and baseline filename, enter `ParameterSet_ConfigurationSet_Baseline`.

Section Option	Purpose
Signal Editor Scenario	Applies to the Inputs section of a simulation, baseline, or equivalence test case, for the specified Signal Editor Scenario . The Test Manager uses each Signal Editor scenario to generate an iteration.
Parameter Set	Applies to the Parameter Overrides section of a simulation, baseline, or equivalence test case. The Test Manager uses each parameter override set to generate an iteration.

Section Option	Purpose
External Input	Applies to the Inputs section of a simulation, baseline, or equivalence test case, for the specified External Inputs sets. The Test Manager uses each external input set to generate an iteration.
Configuration Set	Applies to the Configuration Settings Overrides section of a simulation, baseline, or equivalence test case. Each iteration uses the specified configuration setting.
Variant Configuration	Applies to the Parameter Overrides section of a simulation, baseline, or equivalence test case. Each iteration uses the specified variant configuration.
Logged Signal Set	Applies to the Logging section of a simulation, baseline, or equivalence test case. The Test Manager uses each logged signal set to generate an iteration.
Baseline	Applies only to the Baseline Criteria section of a baseline test case. The Test Manager uses each baseline criteria set to generate an iteration.
Test Sequence scenario	Applies to the Inputs section of a simulation, baseline, or equivalence test case, for the specified Test Sequence Block . The Test Manager uses each Test Sequence scenario to generate an iteration.
Fault Set	Applies to the Fault Settings section of a simulation or baseline test case. The Test Manager uses each fault set to generate an iteration.
Sequence Diagrams	Applies to the Sequence Diagram Assessment section of a simulation or baseline test case. The Test Manager uses each sequence diagram to generate an iteration.
Simulation Modes	Applies to the Iterations table section of a simulation or baseline test case. The Test Manager creates an iteration for the current simulation mode of the model and a SIL iteration for each other test setting selected in the Auto Generate Iterations dialog box.
Simulation 1 or 2	Applies only to equivalence test case types. At the top of the Auto Generate Iterations dialog box, select Simulation 1 or Simulation 2 . The simulation you select in the Auto Generate Iterations dialog box corresponds to the two simulation sections within the equivalence test case.

Create Scripted Iterations

You can run a custom set of iterations by using a script in the **Scripted Iterations** section. For example, you can define parameter sets or customize iteration order. The Test Manager generates the iterations at run time when a test executes and run before loading the model.

▼ SCRIPTED ITERATIONS*

► Help on creating test iterations:

```

1
2 %% Iterate over all External Inputs.
3
4 % Determine the number of possible iterations
5 numSteps = length(sltest_externalInputs);
6
7 % Create each iteration
8 for k = 1 : numSteps
9     % Set up a new iteration object
10    testItr = sltestiteration();
11
12    % Set iteration settings
13    setTestParam(testItr, 'ExternalInput', sltest_externalInputs{k});
14
15    % Register the iteration to run in this test case
16    addIteration(sltest_testCase, testItr); % You can pass in an optional iteration name
17 end
18

```

Iteration Templates

Generate an iteration script using templates

Iteration Script Components

An iteration script must contain certain components. A script must contain these elements:

- 1 An iteration object created using `sltestiteration`
- 2 An iteration setting set using `setTestParam`
- 3 The iteration registration added using `addIteration`

For example, this script creates an iteration that runs one signal scenario from a Signal Editor block.

```

%% Iterate Using a Signal Editor Scenario

% Set up a new iteration object
testItr = sltestiteration;

% Set iteration setting using Signal Editor scenario
setTestParam(testItr, 'SignalEditorScenario', ...
    sltest_signalEditorScenarios{1});

% Add the iteration to run in this test case
% The predefined sltest_testCase variable is used here
addIteration(sltest_testCase, testItr);

```

For more information about the test iteration class, see `sltest.testmanager.TestIteration`. You can iterate over multiple items, such as Signal Editor or Test Sequence scenarios. You can iterate

over all Signal Editor or Test Sequence scenarios in the block by putting the basic iteration script in a loop. For example:

```
%% Iterate Over All Signal Editor Scenarios

% Determine the number of possible iterations
numSteps = length(sltest_signalEditorScenarios);

% Create each iteration
for k = 1 : numSteps
    % Set up a new iteration object
    testItr = sltestiteration;

    % Set iteration settings
    setTestParam(testItr, 'SignalEditorScenario', ...
        sltest_signalEditorScenarios{k});

    % Add the iteration to run in this test case
    % You can pass in an optional iteration name
    addIteration(sltest_testCase, testItr);
end
```

Predefined Variables

You can use predefined variables to write iterations scripts. To see the list of predefined variables in the Test Manager, expand the **Help on creating test iterations** section. You write the iterations script in the script box in the **Scripted Iterations** section. The script box is a functional workspace, which means the MATLAB base workspace cannot access information from the script box. If you define variables in the script box, then other workspaces cannot use the variable.

The predefined variables are:

- `sltest_bdroot` — Model simulated by the test case, defined as a string
- `sltest_sut` — The System Under Test, defined as a string
- `sltest_isharness` — true if `sltest_bdroot` is a harness model, defined as a logical
- `sltest_externalInputs` — Name of external inputs, defined as a cell array of strings
- `sltest_parameterSets` — Name of parameter override sets, defined as a cell array of strings
- `sltest_configSets` — Name of configuration settings, defined as a cell array of strings
- `sltest_signalEditorScenarios` — Name of Signal Editor scenarios, defined as a 2-D cell array of character vectors.
- `sltest_loggedSignalSets` — Name of logged signal sets, defined as a 2-D cell array of character vectors.
- `sltest_testSequenceScenarios` — Name of test sequence scenarios, defined as a 2-D cell array of character vectors.
- `sltest_faultSets` — Name of fault sets, defined as a 2-D cell array of character vectors. You must have Simulink Fault Analyzer™ to use this predefined variable.
- `sltest_sequenceDiagrams` — Name of sequence diagrams, defined as a cell array of character vectors.
- `sltest_baselines` — Name of baseline criteria, defined as a cell array of character vectors, for baseline tests only.
- `sltest_tableIterations` — Iteration objects created in the iterations table, defined as a cell array of `sltest.testmanager.TestIteration` objects
- `sltest_testCase` — Current test case object, defined as an `sltest.testmanager.TestCase` object

Scripted Iteration Templates

If you are using a baseline test case, you can quickly generate iterations for your test case by using templates for Signal Editor scenarios, parameter sets, external inputs, configuration sets, and baseline sets. Scripted iteration templates follow lockstep ordering and pairing of test settings. For more information about lockstep ordering, see “Create Table Iterations” on page 6-139.

For example, if you want to run all Signal Editor scenarios in a scripted iteration:

- 1 Click **Iteration Templates**.
- 2 Select the test case settings you want to iterate through. Click **OK**.

The Test Manager generates the script and adds it to the script box below any existing scripts.

- 3 To generate a table that previews the iterations that execute when you run the test case, click **Show Iterations**.

Sweep Through a Set of Parameters

This example shows how to use scripted iterations to test a model by sweeping through a set of parameters. You can use this iteration script to try different values for the model workspace parameter `Iei` and model parameter `UpperSaturationLimit` in the model `sf_car`.

1. Open the model.

```
sf_car
```

2. Open the Test Manager.

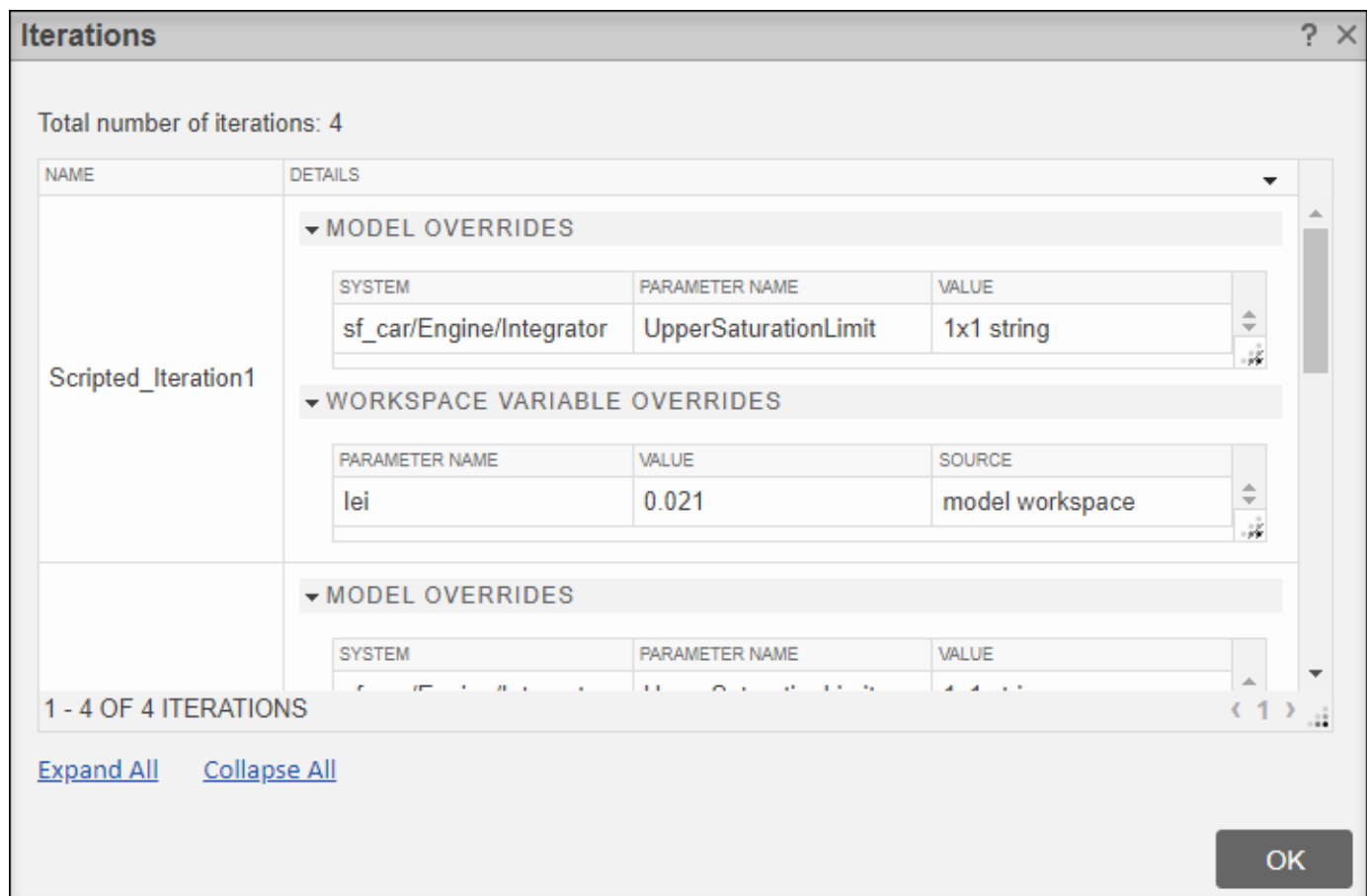
```
sltest.testmanager.view
```

3. Use **New > Test File** to create a new test file, which contains a default test suite and test case.
4. Select **New Test Case 1** and enter `sf_car` in the **Model** field of the **System Under Test** section.
5. Add this script in the **Iterations > Scripted Iterations** section.

```
% Iterate over Iei parameter
% Set up the parameter values to sweep over
IeiValues = [0.021,0.022,0.022,0.023];
UprSatValues = [2000,3000,4000,5000];
numSteps = length(IeiValues);

% Create each iteration
for k = 1 : numSteps
    testItr = sltestiteration;
    setVariable(testItr,'Name','Iei','Source','model workspace', ...
        'Value',IeiValues(k));
    testItr.setModelParam('sf_car/Engine/Integrator', ...
        'UpperSaturationLimit',string(UprSatValues(k)));
    addIteration(sltest_testCase,testItr);
end
```

6. After you add the script, click **Show Iterations**. You can see the iterations that the script created.



7. Click **Run** to run the test and generate a result for each iteration.

Test Browser Results and Artifacts	
Filter results by name or tags, e.g. tags: test	
NAME	STATUS
▼ Results: 2023-Nov-06 14:30:29	4 ✓
▼ New Test Case 1	4 ✓
Scripted_Iteration1	✓
Scripted_Iteration2	✓
Scripted_Iteration3	✓
Scripted_Iteration4	✓

8. Clear and close the Test Manager

```
sltest.testmanager.clear
sltest.testmanager.clearResults
sltest.testmanager.close
```

See Also

`sltest.testmanager.TestIteration | setModelParam`

Related Examples

- “Capture Baseline Data from Iterations” on page 6-148
- “Create and Run Test Cases with Scripts” on page 6-115
- “Programmatically Create and Run Test Sequence Scenarios” on page 3-51
- “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54

Capture Baseline Data from Iterations

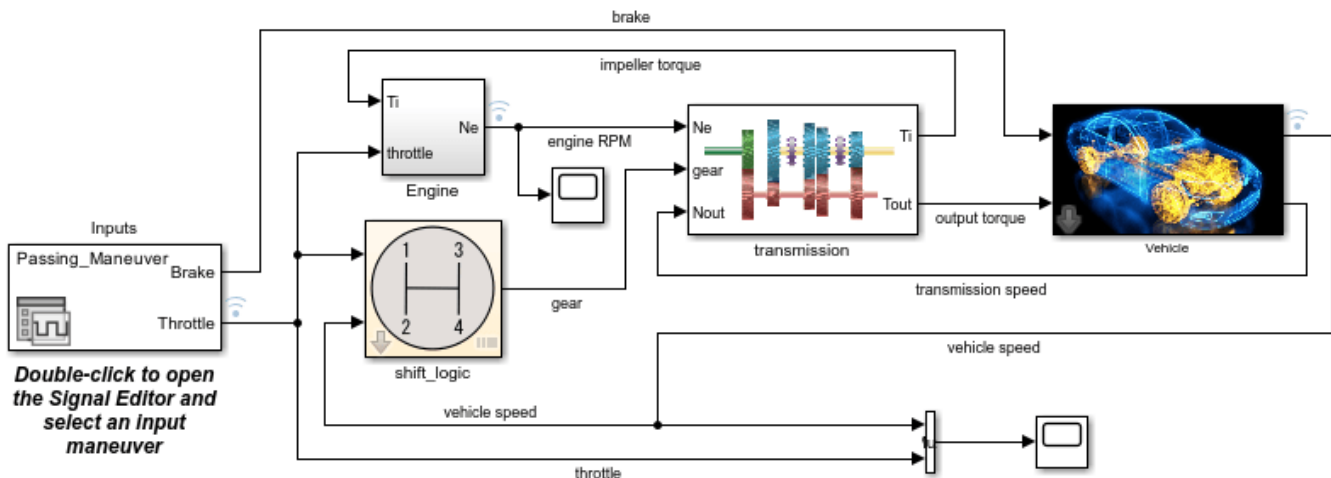
This example shows how to create a baseline test by capturing data from a test case with table iterations. The model `sf_car` contains a Signal Editor that you can use to create iterations.

1. Open the model and clear `OpenAtSimulationStart` property for scopes.

```
model = "sf_car";
open_system(model);
```

```
scopeConfig1 = get_param(model+"/Scope1","ScopeConfiguration");
scopeConfig1.OpenAtSimulationStart = false;
scopeConfig2 = get_param(model+"/Scope2","ScopeConfiguration");
scopeConfig2.OpenAtSimulationStart = false;
```

Model a Car with Automatic Transmission



2. Open the Test Manager.

```
sltestmgr
```

3. In the Test Manager, click **Test File from Model** from the **New** dropdown.

4. Specify the test file.

- 1 Enter `sf_car` as the **Model**.
- 2 Enter a test filename or full path in **Location**.
- 3 Select **Baseline** as the **Test Type**.

5. Select the test case by expanding the test file and selecting the `sf_car/Inputs` test case.

6. Select the signals for the baseline data:

- 1 In the **Simulation Outputs** section, click **Add**.
- 2 In the model canvas, select the output `torque` signal and in the Connect dialog, check the box for that signal.

- 3 Repeat the previous step for the `vehicle_speed` signal.
 - 4 In the Test Manager message dialog box, click **Done**. The signals appear in the **Logged Signals** table.
7. To view the iterations for the test case, expand the **Iterations** and **Table Iterations** sections. The iterations for the selected test case automatically appear. The iterations correspond to the four Signal Editor scenarios.
8. Capture baseline data for the iterations.
- 1 In the **Baseline Criteria** section, click the arrow next to **Capture**.
 - 2 Select MAT as the **File format**.
 - 3 Specify the location to save the baseline data files in the **File** field.
 - 4 Select **Capture Baselines for Iterations**.
 - 5 Click **Capture**.

The model simulates for all Signal Editor scenarios. The baseline data for `output_torque` and `vehicle_speed` are captured in four MAT files. Also, each baseline data set is added to its corresponding iterations in the table. Note that iteration settings take precedence over test case settings. In this example, since the baseline data sets are associated with iterations, all of the baseline files are used when the iterations run, regardless of the selections in the **Baseline Criteria** section of the test case.

▼ BASELINE CRITERIA* ?

☐ Include baseline data in test result

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL	+
▶ ☐ Coasting.mat	0	0.00%	0	0	
▶ ☐ Gradual_Acceleration.mat	0	0.00%	0	0	
▶ ☐ Hard_braking.mat	0	0.00%	0	0	
▶ ☐ Passing_Maneuver.mat	0	0.00%	0	0	

+ Add...
Capture...
Edit...
Refresh
Visualize
Delete

▼ ITERATIONS* ?

▼ TABLE ITERATIONS* ?

✓ NAME	DESCRIPTION	SIGNAL EDITOR SCENA...	BASELINE	+
✓ Coasting	None	Coasting	Coasting.mat	▲
✓ Gradual_Acceleration	None	Gradual_Acceleration	Gradual_Acceleration.mat	
✓ Hard_braking	None	Hard_braking	Hard_braking.mat	
✓ Passing_Maneuver	None	Passing_Maneuver	Passing_Maneuver.mat	▼

Auto Generate
+ Add
Delete ▼

```
sltest.testmanager.close
close_system(model,0)
```

See Also

Related Examples

- “Test Iterations” on page 6-139
- “Baseline Testing” on page 6-240

Collect Coverage in Tests

In this section...

“Set Up Coverage Collection Using the Test Manager” on page 6-151

“View Coverage Results in the Test Manager” on page 6-153

“Add Tests for Missing Coverage” on page 6-155

“Coverage Filtering Using the Test Manager” on page 6-156

Coverage refers to determining the testing completeness of models and generated code by analyzing how much of the model has been exercised. To collect coverage using the Simulink Test Test Manager or `sltest.testmanager.CoverageSettings`, you must have Simulink Coverage installed. Although you can set up and run test cases using only Simulink Coverage, Simulink Test provides additional test creation and test management features. For tests with coverage collection turned on, the Test Manager includes the coverage of each metric you choose to collect in the results. If you have Requirements Toolbox installed, you can also use the Test Manager to verify that coverage results are traced to specific requirements.

Note Coverage is supported for Model Reference blocks, atomic Subsystem blocks, and top-level models configured for Software-in-the-Loop (SIL) or Processor-in-the-Loop (PIL). Coverage is not supported for SIL or PIL models in subsystems.

For information on considerations when collecting coverage in a test harness, see Test Harness Considerations in “Test Harness and Model Relationship” on page 2-2.

Set Up Coverage Collection Using the Test Manager

In the Test Manager, you can enable coverage and select the coverage metrics at the test file level. Test suites and test cases inherit the coverage settings from the test file. You can turn off coverage collection for individual test suites and test cases. However, you cannot turn off coverage at the test suite or test case level for MATLAB-based Simulink tests. For information about MATLAB-based Simulink tests, see “Test Models Using MATLAB-Based Simulink Tests” on page 6-119.

Note The Test Manager uses its coverage settings and not the coverage settings from the model Configuration Parameters.

To set up the Test Manager to include coverage collection:

- 1 Create a test file and set up a test case for your model.
- 2 Select the test file and expand the **Coverage Settings** section. Under **Coverage to Collect**, select the coverage:
 - **Record coverage for system under test** — Collects coverage for the model or, when included, the component specified in the “System Under Test” on page 6-176 section for each test case. If you are using a test harness, the system under test is the component for which the harness is created. The test harness is not the system under test.
 - For a block diagram, the system under test is the whole block diagram.

- For a Model block, the system under test is the referenced model.
- For a subsystem, the system under test is that subsystem.
- **Record coverage for referenced models** — Collects coverage for models that are referenced from within the specified system under test. If the test harness references another model, the coverage results are included for that model, too.

The selected coverage settings propagate from the test file to the test suites and test cases in the test file.

The screenshot shows the 'COVERAGE SETTINGS*' dialog box. It is divided into several sections:

- COVERAGE TO COLLECT:** Contains two checkboxes: 'Record coverage for system under test' (checked) and 'Record coverage for referenced models' (unchecked).
- COVERAGE FILTERS:** Features a list box with the text 'There are no coverage filter files applied.' Below the list box are '+ Add' and 'Remove' buttons.
- COVERAGE METRICS:** A grid of checkboxes for selecting metrics to collect:

☒ Decision	☒ Condition
☒ MCDC	☐ Lookup Table
☐ Signal Range	☐ Signal Size
☐ Simulink Design Verifier	☐ Saturation on integer overflow
☐ Relational Boundary	

- Optionally, to add or remove existing coverage filter files, click **Add** or **Remove**, respectively, in the Coverage Filters section and select the filter file. More than one filter file can be applied at the same time.
- Select the coverage metrics to collect. For information on metrics, see “Types of Model Coverage” (Simulink Coverage) and “Model Objects That Receive Coverage” (Simulink Coverage).

- 5 Run the test. Coverage is collected for the test suites and test cases in the test file.

To turn off coverage collection for a test suite or test case, select the test suite or test case and then, deselect the **Coverage to Collect** option. To turn off one or more types of coverage collection, deselect them from the **Coverage Metrics** section. For MATLAB-Based Simulink tests, you can change the coverage collection and coverage metrics only at the test file level.

View Coverage Results in the Test Manager

View Aggregated Coverage Results and Metrics

After you collect coverage, use the **Results and Artifacts** pane in the Test Manager to view the results. Coverage results aggregated for all test cases are reported in results sets. Filtered coverage results are shown only at the Results level, and not at the test file, test suite, or test case level. When analyzing filtered coverage results, viewing them at the Results level provides overall coverage information. For example, if coverage is less than 100% for a specific test case, the missing coverage might be included in a different test case.

Select a **Results** item in the pane and expand the **Aggregated Coverage Results** section. The coverage percentage is shown for each metric and the colors summarize the coverage results.

- Dark blue — Satisfied coverage
- Red — Unsatisfied coverage
- Light blue — Justified coverage

The screenshot shows the Test Manager interface with the 'Results and Artifacts' pane selected. The left sidebar shows a list of results sets, with 'Results: 2020-Jul-16 10:42:39' selected. The main pane displays the 'SUMMARY' and 'AGGREGATED COVERAGE RESULTS' for this test suite.

SUMMARY

Name	Results: 2020-Jul-16 10:42:39
Outcome	1 ✓
Start Time	07/16/2020 10:42:46
End Time	07/16/2020 10:42:50
Type	Result Set

AGGREGATED COVERAGE RESULTS

Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.

ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	MCDC	EXECUTION
slvndemo_covfilt	29	48%	60%	50%	74%	

At the bottom, there is a checkbox for 'Scope coverage results to linked requirements' and buttons for 'Add Tests for Missing Coverage' and 'Export'.

To aggregate results from different test files into a single result set, select the separate results in the **Results and Artifacts** list. Then, from the context menu, select **Merge Coverage Results**. A results set that contains the combined coverage results appears in the list.

Scoping Coverage for Requirements-Based Tests

For requirements-based design and testing, such as for compliance to DO-178B, enable **Scope coverage results to linked requirements** to check that your model design is executing the

requirements and that the tests are verifying those requirements. Both Simulink Coverage and Requirements Toolbox licenses are required. This option is available only if the results set contains more than one simulation, such as multiple test cases or iterations.

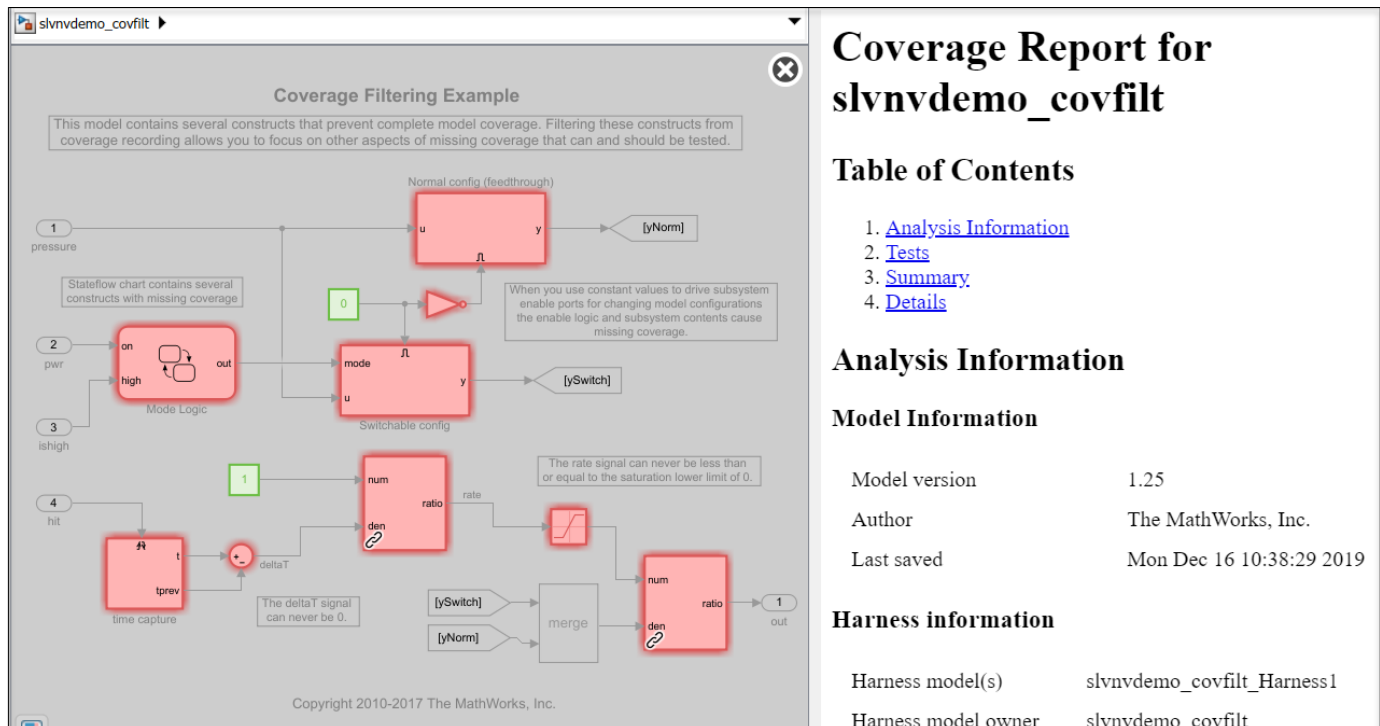
When the **Scope coverage results to linked requirements** check box is selected, coverage results include only tests that are directly linked to requirements and are explicitly tested. Requirements linked to steps in a Test Sequence block are not included in the coverage results. The aggregated results update automatically without having to resimulate the model. If you have tests that touch a model component but are not directly linked to a requirement, your aggregated coverage results percentages might decrease when you enable scoping. To obtain 100% coverage to your requirements, you might need to update your tests, add requirements links, or justify or exclude some items from coverage.

Trace Coverage Results to the Model

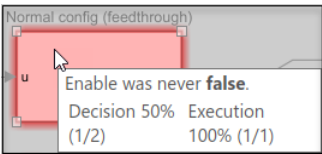
To navigate from the test coverage results in the Test Manager to the model, click the model name in the Aggregated Coverage Results table.

ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDI
 <u>slvndemo_covfilt</u>		29	48% 	60%

The model opens, and its Coverage Report opens in the Coverage Details pane of the model window. In this sample model, the model elements are red because they have less than 100% coverage.



Point to a model element to see a summary of its metrics and block execution.



Click a model element to scroll to its detailed coverage results information in the **Coverage Details** pane.

slvndemo_covfilt

7. SubSystem block "[Switchable config](#)"

[Justify or Exclude](#)

Parent: [/slvndemo_covfilt](#)

Uncovered Links:

Metric	Coverage (this object)	Coverage (inc. descendants)
Cyclomatic Complexity	2	5
Decision	50% (1/2) decision outcomes	17% (1/6) decision outcomes
Execution	NA	20% (1/5) objective outcomes

Decisions analyzed

enable logical value	50%
false	101/101
true	0/101

Create a Coverage Report

To create a report of the coverage for a model, click the arrow in the **Report** column of the **Aggregated Coverage Results** table.

ANALYZED MODEL	REPORT	COMPLEXI...	DECISION
slvndemo_covfilt		29	36%

Add Tests for Missing Coverage

If you have a Simulink Design Verifier license, you can generate additional test cases to increase the coverage in your models.

In the Test Manager,

- 1 In the Test Manager, select the **Results and Artifacts** pane.
- 2 Select the **Results** item for which you want to collect more coverage.
- 3 In the right pane, in the **Aggregated Coverage Results** section, select the line with incomplete coverage in the table.

- 4 At the bottom of the **Aggregated Coverage Results** section, click **Add Tests for Missing Coverage**.
- 5 In the Add Test for Missing Coverage dialog box,
 - **Harness** — Select whether to use the existing harness or create a new harness
 - **Source** — Select the source of the inputs to the harness. If you use an existing harness, the **Source** field is read only.
 - **Test Case** — Select whether to use the existing test case or create a new test case. If you create a new harness, the only option is to use a new test case.
 - **Test Type** — Select the type of test to use for the new test case. This field is displayed if you select to create either a new harness or a new test case.
 - **Test File** — Select whether to use the existing test file or create a new test file. This field is displayed if you select to create either a new harness or a new test case.
 - **Location** — If you select to create a new test file, specify the path and name of the test file.
- 6 Click **OK** to generate test cases that add the missing coverage.
- 7 If you created a new test case or new harness, in the **Test Browser** pane, drag and drop the test case into the test suite that contains the original test case.
- 8 Rerun the test suite.

For a complete example of how to increase test coverage in the Test Manager, see “Increase Test Coverage for a Model” on page 6-163.

Alternatively, you can create and use tests to increase coverage programmatically by using `sltest.testmanager.addTestsForMissingCoverage` and `sltest.testmanager.TestOptions`.

Coverage Filtering Using the Test Manager

Coverage filter rules specify one or more model objects or lines of generated code to exclude from coverage collection or for which you want to justify the coverage results. A set of coverage filter rules is contained in a filter file, which can be applied to the model or code being tested. You can apply more than one filter file to a test and also, reuse filter files for different models. When you apply a new or updated filter, the aggregated coverage results, which are shown for a result set, update automatically. You do not have to resimulate your model. For more information, see “Coverage Filtering” (Simulink Coverage).

To view the filtered coverage results, select a result set (that is, a **Results** item) in the **Results and Artifacts** pane.

From the Test Manager, you can:

- Add or remove an existing coverage filter file — In the **Test Browser** pane, select the test file and expand the **Coverage Settings** section. Click **Add** or **Remove** at the bottom of the **Coverage Filters** or **Applied Coverage Filters** table and select the coverage filter file to add or remove, respectively. More than one coverage filter file can be applied to the coverage results.
- Edit or create a filter file, define a filter rule, and justify or exclude coverage — From a **Coverage Report** or the **Coverage Details** pane of a model, open the Simulink Coverage Filter Editor by clicking on a justify icon  or a **Justify** or **Exclude** link. When the Filter Editor is open, the Test Manager is locked. When you close the Filter Editor, the Test Manager is enabled and the

results and applied filters list are updated with your changes. For information on using the Filter Editor, see “Creating and Using Coverage Filters” (Simulink Coverage) and “Create, Edit, and View Coverage Filter Rules” (Simulink Coverage).

- Append currently applied coverage filters to the test file — Click **Update Test File**.
- View coverage results — Select a **Results** item in the **Results and Artifacts** pane and expand the **Aggregated Coverage Results** section.

For more information on coverage filters, rules, and files, see the Coverage Filtering topics in “Analyze Coverage and View Results” (Simulink Coverage).

See Also

`sltest.testmanager.CoverageSettings` |
`sltest.testmanager.addTestsForMissingCoverage` | `sltest.testmanager.TestOptions`

More About

- “Analyze Coverage and View Results” (Simulink Coverage)
- “Model Objects That Receive Coverage” (Simulink Coverage)
- “Perform Functional Testing and Analyze Test Coverage” on page 11-11
- “Test Coverage for Requirements-Based Testing” on page 6-158
- “Assess Coverage Results from Requirements-Based Tests” (Simulink Coverage)
- “Trace Coverage Results to Requirements” (Simulink Coverage)
- “Verify Requirements by Using Tests” (Requirements Toolbox)
- “Add Tests for Missing Coverage” on page 7-40
- “Collect Coverage Using MATLAB-Based Simulink Tests” on page 6-128

Test Coverage for Requirements-Based Testing

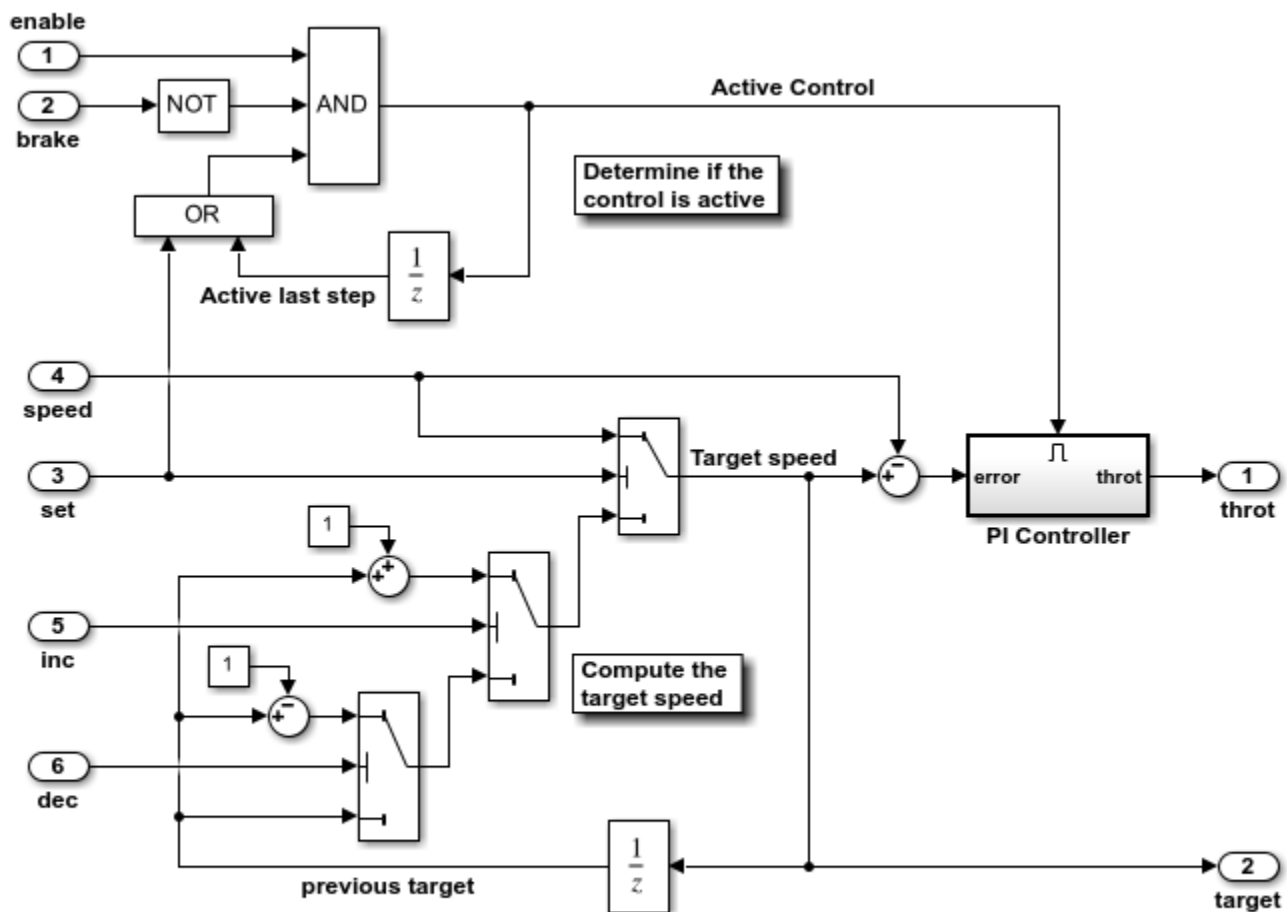
This example shows how to collect test coverage for a model that implements requirements. Coverage refers to determining the testing completeness by analyzing how much of the model logic is exercised. For requirements-based testing, coverage results can be scoped to linked requirements. With this scoping you can assess if each model element is covered by the intended test case.

The example shows how scoping coverage results to linked requirements can reveal both inadequate requirement linking and testing gaps. It also shows how to increase the coverage.

The model in this example is `cruiseControlRBTCovExample`, which represents a cruise control system. This model implements and is linked to requirements. A test file has already been created for this example.

Open the Cruise Control Model

`cruiseControlRBTCovExample`



Copyright 2006-2024 The MathWorks, Inc.

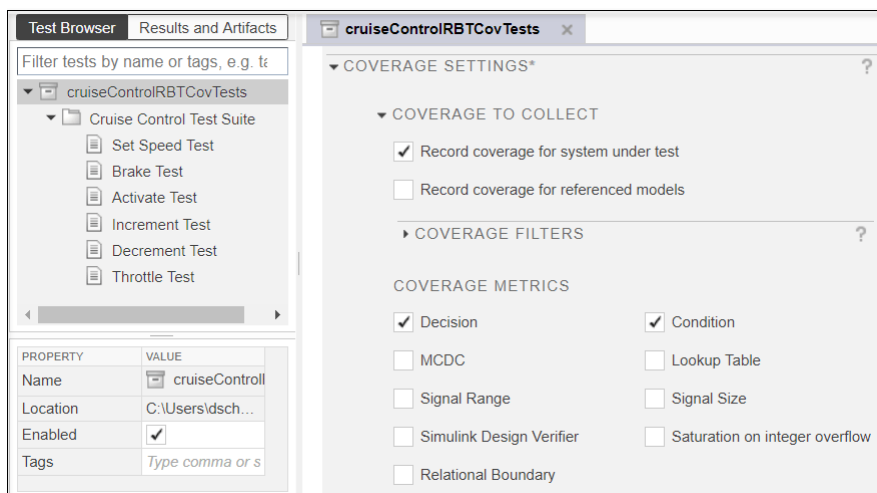
View the Linked Requirements

The requirements for this cruise control system have been captured in the Requirements Editor. To view the requirements, use `slreq.open('cruiseControlRBTcovReqs.slreqx')`.

Open the Test Manager and Test File

Use `sltestmgr` to open the Test Manager.

Click **Open** and select `cruiseControlRBTcovTests.mldatx`. The tests have been written to verify that the model behavior meets the specified requirements. They have also been set up to record Decision and Condition coverage. Expand Coverage Settings to see the selected metrics.



Each test case verifies and is linked to a requirement. For example, the Throttle Test verifies the THROTTLE requirement. This requirement specifies that the throttle is applied smoothly if the speed differs from the target. The test verifies this behavior using a logical assessment, which checks that the throttle rate of change is between -1 and 1 radians per second, as defined in the requirement description.

Run the Test and View Coverage Results

Run the test.

Click on Results in the Results and Artifacts pane when the test finishes running. Note that the tests pass and that 100% aggregated coverage is reported.

AGGREGATED COVERAGE RESULTS					
Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.					
ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	EXECUTION
cruiseControlRBTcovExample		8	100%	100%	100%

Turn on Scoping the Test Results to Linked Requirements

Click the top-level Results in the Results and Artifacts pane. Then, in the Aggregated Coverage Results pane, click the **Scope coverage results to linked requirements** check box. Scoping the results means that each test only contributes coverage for the corresponding model elements that implement the requirement verified by that test. Scoping checks that model elements are covered by the intended test cases. The coverage results, which update automatically, now show aggregated coverage for Decision and Execution at 92% and 76%, respectively.

▼ AGGREGATED COVERAGE RESULTS ?

Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.

ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	EXECUTION
 cruiseControlRBTcovExample		8	92%	100%	76%

☒ Scope coverage results to linked requirements

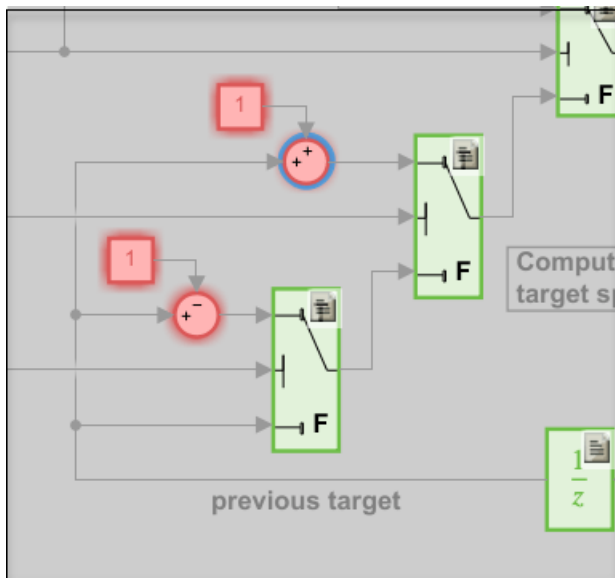
+ Add Tests for Missing Coverage
  Export

View the Coverage Results in the Model

Click on the model name in the Analyzed Model column to highlight the coverage results in the model and display the Coverage Report details.

In the model, if the Requirements table is not shown below the model, open it by clicking the Perspectives views in the lower right corner of the model canvas and then, clicking Requirements.

Open the Controller subsystem. Blocks that do not have 100% coverage appear in red. Two sets of Constant and Sum blocks are not linked to requirements and were never executed.



Link Blocks to Requirements

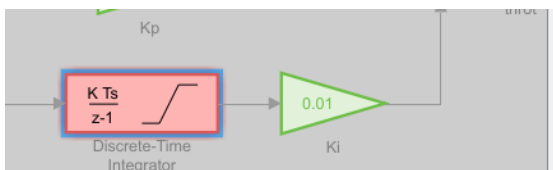
In this case, the missing coverage indicates insufficient requirements linking. These Constant and Sum blocks are necessary for implementing the INCREMENT and DECREMENT requirements and should be linked to the appropriate requirements.

In the table in the Requirements Browser, expand `cruiseControlRbtCovReqs`. Right-click on the upper Constant block and select **Requirements > Link to Selection in Requirements Browser**. Then, click on the INCREMENT requirement in the Requirements table. Repeat this for the upper Sum block.

For the lower Constant and Sum blocks, repeat the linking steps, but link to the DECREMENT requirement.

Increase Coverage from a Specific Test

Open the PI Controller and click on the Discrete-Time Integrator block. The Coverage Details show that the `true` decision for the upper limit was executed by the Increment Test (T4), rather than the Throttle Test (T6). Since the block is part of the implementation of the THROTTLE requirement, it should have been tested by the Throttle Test, which verifies the THROTTLE requirement. The Increment Test does not verify this requirement and does not contribute coverage for this block when the Scope model coverage to linked requirements setting is enabled.



true	0/801 T4
------	-------------

To resolve the missing coverage for this block, the Throttle Test needs to be updated to exercise the Discrete-Time Integrator block more.

In the Test Browser pane of the Test Manager, select Throttle Test. Under Inputs, select `td_throttle_updated.mat` as the External Inputs file. This updated input throttle data file has

some additional seconds of test data, which increase the target speed more aggressively while maintaining the actual speed.

Select `cruiseControlRBTcovTests` in the Test Browser pane and rerun the test. Click the **Scope coverage results to linked requirements** check box. The coverage results show 100% coverage, which indicates that the tests adequately execute the model.

Revised Test Reveals an Issue in the Design

The revised Throttle Test now fails verification. The failure occurs because the throttle increases too aggressively and is outside the required boundaries specified in the test. This indicates an issue with the model design. The PI Controller block implementation needs to be updated to apply the throttle within the required limits, including when the target and actual speeds differ significantly.

Conclusion

In summary, scoping coverage results to linked requirements can help reveal gaps in testing. Scoping accomplishes this by assessing that each model element is exercised by the test that verifies the corresponding requirement.

See Also

Related Examples

- “Collect Coverage in Tests” on page 6-151
- “Increase Test Coverage for a Model” on page 6-163

Increase Test Coverage for a Model

This example shows how to increase test coverage for a model by generating test inputs.

If your tests achieve incomplete model coverage, you can increase coverage by generating test inputs using Simulink® Design Verifier™.

This example shows how to increase test coverage beyond an initial test case. You measure initial coverage of a test case. Then, you generate new test cases, add them to the test suite, run the tests, and review the aggregate coverage.

Workflow

This example tests a component of an autopilot system using a test harness. Time series data from the base workspace is mapped to root inputs in the test harness. The test file is configured to collect coverage.

The example workflow is:

- 1 Measure model coverage of the initial test case.
- 2 Generate additional tests to achieve greater coverage.
- 3 Add the new test cases to the test file.
- 4 Run all test cases and review aggregate coverage.

Paths and Example Files

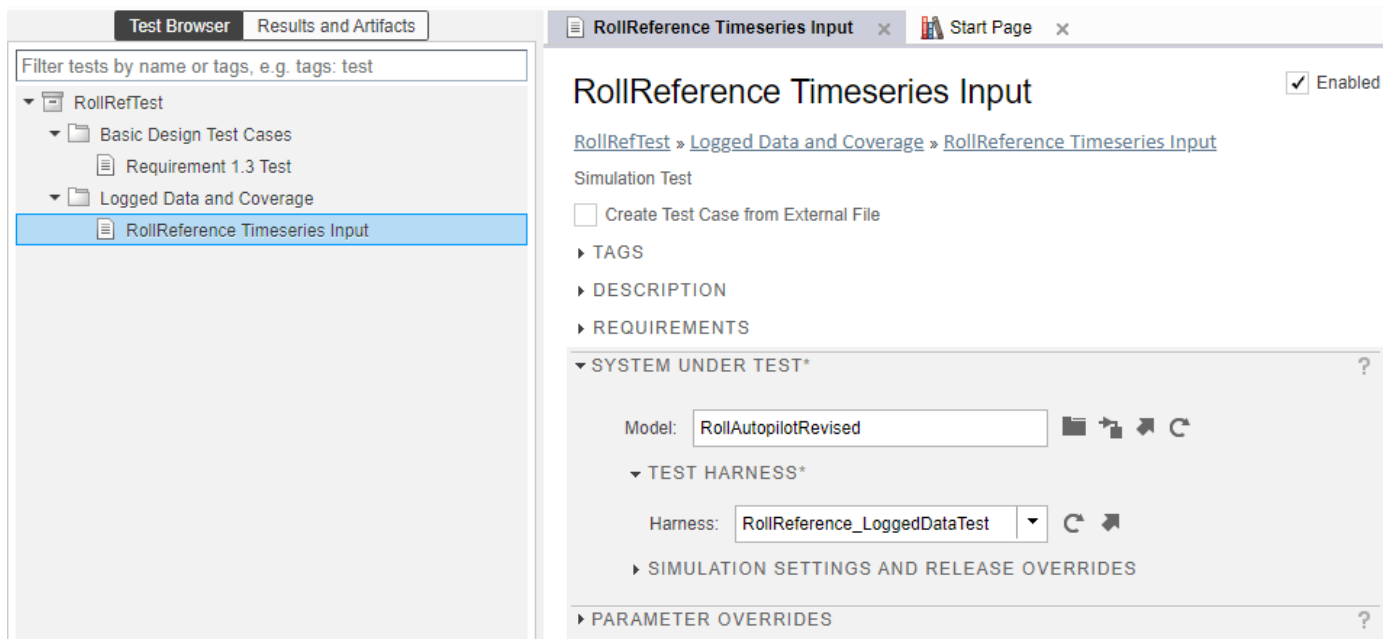
Set paths and filenames for the example.

```
rollModel = 'RollAutopilotRevised';  
testHarness = 'RollReference_LoggedDataTest';  
testFile = 'RollRefTest.mldatx';
```

Run the Initial Test and Review Coverage

1. Open the test file.

```
sltest.testmanager.load(testFile);  
sltest.testmanager.view;
```



2. Run the test. In the **Test Browser**, highlight the **Logged Data and Coverage** test suite. Click **Run**.

3. After the test completes, in the test results, expand the **Coverage Results** section. The test achieves partial coverage for the Roll Reference subsystem.

- Decision coverage: 80%
- Condition coverage: 70%
- MCDC 25%

▼ AGGREGATED COVERAGE RESULTS

Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.

ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	MCDC	EXECUTION
RollAutopilotRevised/Roll Reference		6	80%	70%	25%	100%

Generate Tests to Increase Model Coverage

Generate additional tests for missing coverage.

1. Select the RollAutopilotRevised/Roll Reference line in the Aggregated Coverage Results. Below the table, click **Add Tests for Missing Coverage**.

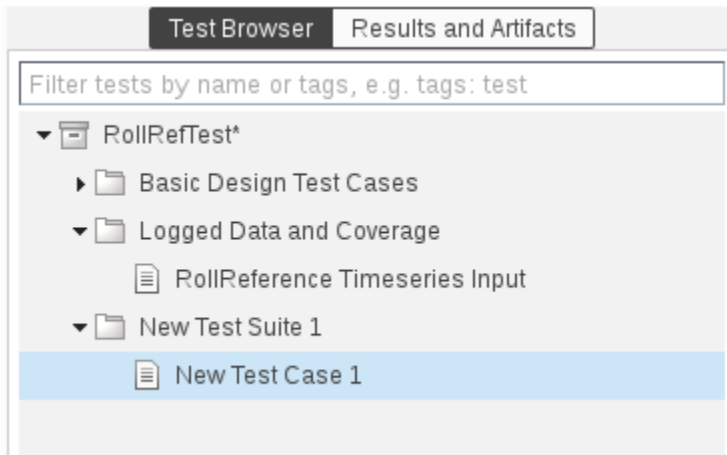
2. In the **Add Tests for Missing Coverage** dialog box, set these options:

- **Harness:** RollReference_LoggedDataTest. This maps the new test inputs to the existing test harness.
- **Test Case:** Create a new test case. This creates a new test case with the generated test inputs.
- **Test Type::** Baseline Test. This gives the option to capture baseline data output from the model for the generated tests.

- **Test File::** RollRefTest. This re-uses the existing test file.

3. Click **OK**. A dialog box shows progress of the test case generation. When test case generation is complete, a new test case appears in the Test Manager.

Alternatively, you can add tests programmatically by using the `sltest.testmanager.addTestsForMissingCoverage` function.



Run the New Test Case

1. Drag and Drop the new test case into the **Logged Data and Coverage** test suite.
2. Run the **Logged Data and Coverage** test suite again.
3. When simulation completes, in the **Results and Artifacts** section, select the result set and expand the **Aggregated Coverage Results**. The test suite achieves complete coverage:
 - Decision: 100%
 - Condition: 100%
 - MCDC: 100%

▼ AGGREGATED COVERAGE RESULTS

Create a coverage report from coverage results to justify or exclude missing coverage. The filters and updated coverage values will be displayed with this result.

ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	CONDITION	MCDC	EXECUTION
RollAutopilotRevised/Roll Reference		6	100%	100%	100%	100%

Cleanup

Clear variables and test results, and close the model.

```
clear reqDoc rollModel testFile testHarness topModel;
sltest.testmanager.clearResults;
```

```
sltest.testmanager.close;  
close_system('RollAutopilotRevised',0);
```

See Also

Related Examples

- “Collect Coverage in Tests” on page 6-151
- “Test Coverage for Requirements-Based Testing” on page 6-158
- “Run a Test Suite and Resolve Missing Model Coverage” (Simulink Coverage)
- “Generate Test Cases for Model Decision Coverage” (Simulink Design Verifier)

Run Tests Using Parallel Execution

In this section...

“When Do Tests Benefit from Using Parallel Execution?” on page 6-167

“Limitations of Running Tests Using Parallel Execution” on page 6-167

“Use Parallel Execution” on page 6-167

Running tests in parallel can speed up execution and decrease the amount of time it takes to get test results. If you have a Parallel Computing Toolbox license, you can execute tests in parallel on your local machine or cluster. If you have a MATLAB Parallel Server license, you can execute tests in parallel on a remote cluster, such as in the cloud.

When Do Tests Benefit from Using Parallel Execution?

In general, parallel execution can help reduce test execution time if you have

- A complex Simulink model that takes a long time to simulate
- Numerous long-running tests, such as iterations

Limitations of Running Tests Using Parallel Execution

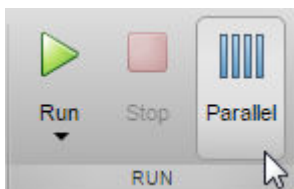
These limitations apply to running tests using parallel execution:

- Running tests in parallel is supported only on interactive parallel pools that you start by using `parpool`. Running tests on noninteractive parallel pools is not supported.
- Test harnesses for atomic subsystems are not supported for parallel test execution in software-in-the-loop (SIL) or processor-in-the-loop (PIL) mode.

Use Parallel Execution

To run tests in parallel:

- 1 Set up and open a parallel pool on the desired cluster, or set the desired cluster as the default. If you have a Parallel Computing Toolbox license, see “Discover Clusters and Use Cluster Profiles” (Parallel Computing Toolbox). If you have a MATLAB Parallel Server license, see “Run Code on Clusters and Cloud Platforms” (MATLAB Parallel Server). If you do not set your default cluster or have a parallel pool open, the Test Manager uses its default cluster, which is on the local machine.
- 2 Open the Test Manager.
- 3 On the Test Manager toolstrip, click the **Parallel** button.



- 4 Run a test file. The test file executes using parallel pool.

Note Baseline, equivalence, custom and assessments criteria evaluation occurs on the host MATLAB and not on the parallel MATLAB workers. Before parallel test execution begins, the base workspace variables from the host MATLAB are transferred to the base workspaces of the parallel MATLAB workers. However, after parallel test execution completes, the base workspace variables are not transferred back to the host MATLAB.

- 5 To turn off parallel execution, click the **Parallel** button to toggle it off.

Starting a parallel pool can take time, which would slow down test execution. To reduce time:

- Make sure that the parallel pool is already running before you run a test. By default, the parallel pool shuts down after being idle for a specified number of minutes. To change the setting, see “Specify Your Parallel Settings” (Parallel Computing Toolbox).
- Load Simulink on all the parallel pool workers.

See Also

`sltest.testmanager.run`

Related Examples

- “Clusters and Clouds” (Parallel Computing Toolbox)

Set Signal Tolerances

In this section...

“Modify Criteria Tolerances” on page 6-169

“Change Leading Tolerance in a Baseline Comparison Test” on page 6-169

You can specify tolerances in the **Baseline Criteria** or **Equivalence Criteria** sections of baseline and equivalence test cases. You can specify relative, absolute, leading, and lagging tolerances for a signal comparison. Leading and lagging tolerances allow you to compensate for differences in time between signals. The units for tolerances are seconds.

To learn about how tolerances are calculated, see “How the Simulation Data Inspector Compares Data”.

Modify Criteria Tolerances

To modify a tolerance, select the signal name in the criteria table, double-click the tolerance value, and enter a new value.

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL
▼ ☒ My_mat_base.mat	0	0.00%	0	0
☒ Ww	0	0.00%	0	0
☒ Vs	0	0.00%	0	0
☒ Sd	0	0.00%	0	0
☒ slp	0	0.00%	0	0

If you modify a tolerance after you run a test case, rerun the test case to apply the new tolerance value to the pass/fail results.

Change Leading Tolerance in a Baseline Comparison Test

Specify a tolerance when the difference between results falls in a range you consider acceptable. Suppose that your model under test uses a particular solver. Solvers are sometimes updated from one release to the next, and new solvers also become available. If you use an updated solver or change solvers, you can specify an acceptable tolerance for differences between your baseline and later tests. Leading and lagging tolerances allow you to re-evaluate criteria if there are differences in time, for example, due to solver the data is off by .04 seconds you can shift it left or right to account for this.

Generate the Baseline

Generate the baseline for the `sf_car` model, which uses the `ode-5` solver.

- 1 Open the `sf_car` model by using `openExample('sf_car')`.
- 2 Open the Test Manager and create a test file named `Solver Compare`. In the test case, set the system under test to `sf_car`.
- 3 Select the signal to log. Under **Simulation Outputs**, click **Add**. In the model, select the `shift_logic` output signal. In the Signal Selection dialog box, select the check box next to `shift_logic` and click **Add**.
- 4 Save the baseline. Under **Baseline Criteria**, click **Capture**. Set the file format to `MAT`. Name the baseline `solver_baseline` and click **Capture**.

After you capture the baseline MAT-file, the model runs and the baseline criteria appear in the table. Each default tolerance is 0.

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL
✓ solver_baseline.mat	0	0.00%	0	0
✓ shift_logic:1	0	0.00%	0	0

Change Solvers and Run the Test Case

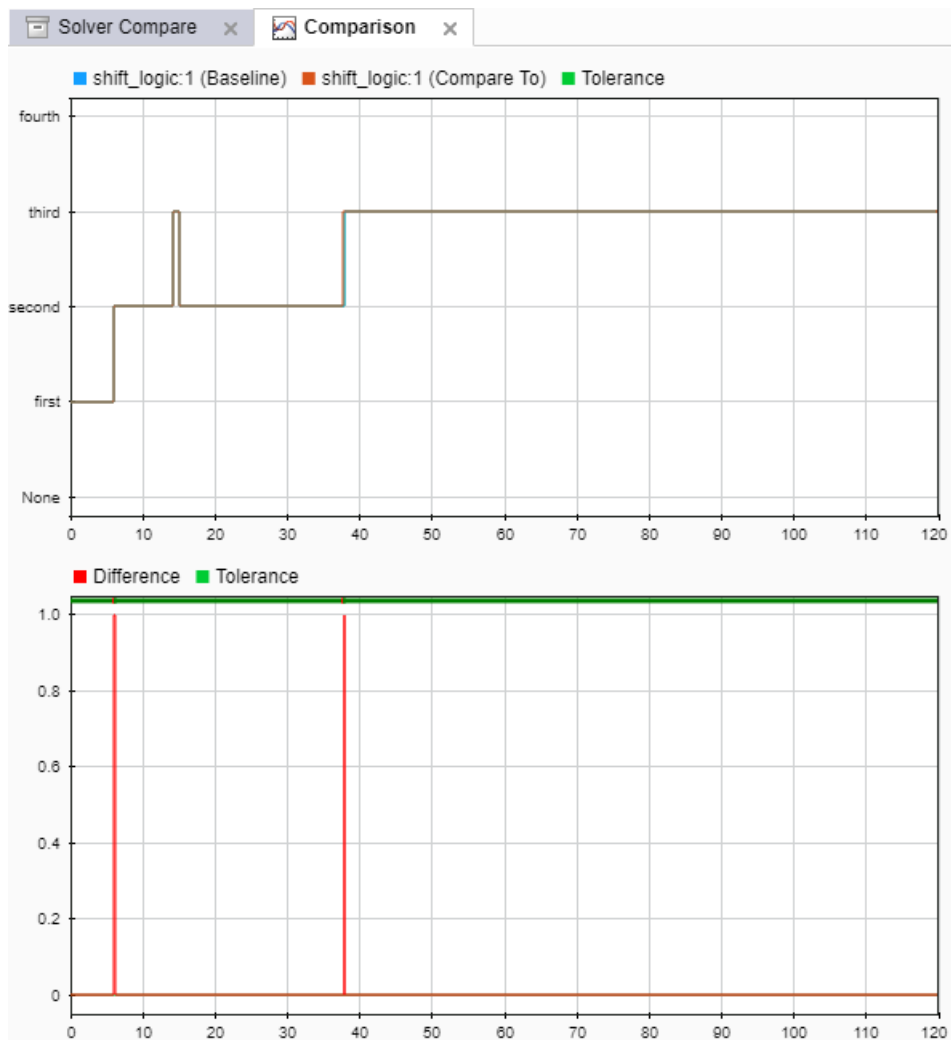
Suppose that you want to use a different solver with your model. You run a test to compare results using the new solver with the baseline.

- 1 In the model, change the solver to ode1.
- 2 In the Test Manager, with the Solver Compare test file selected, click **Run**.

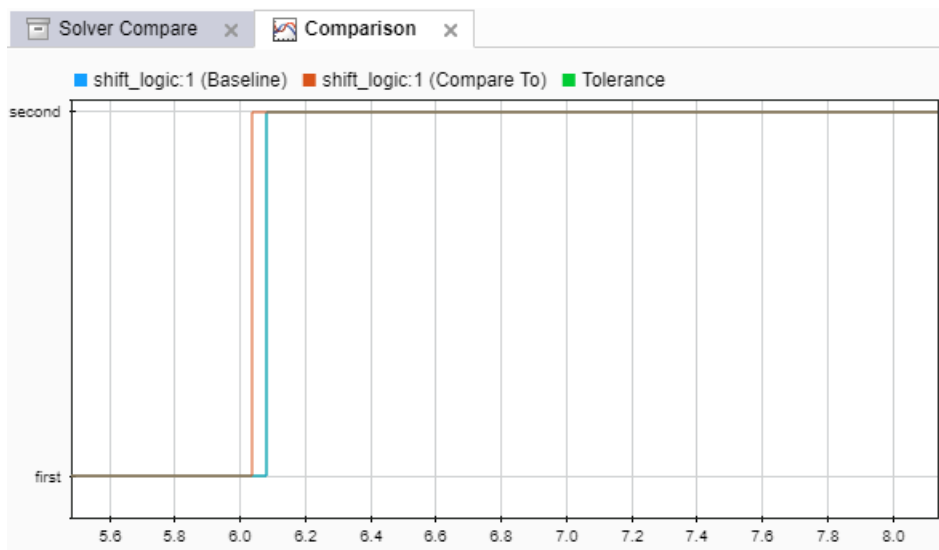
In the **Results and Artifacts** pane, notice that the test failed.

- 3 Expand the results of the failed test. Under **Baseline Criteria Result**, select the shift_logic signal.

The **Comparison** tab shows where the difference occurred.



- 4 Zoom the comparison chart where the results diverged. The comparison signal changes ahead of the baseline, that is, it leads the baseline signal.



Preview and Set a Leading Tolerance Value

You can use leading and lagging tolerances to allow for slight offsets in time between the simulation and baseline data. Suppose that your team determines that a tolerance the size of the simulation step size (.04 seconds in this case) is acceptable. In the Test Manager, set a leading tolerance value. Use a leading tolerance for the signal whose change occurs ahead of your baseline. Use a lagging tolerance for a signal whose change occurs after your baseline.

You can preview how the tolerance value affects the test to see if the test passes with the specified tolerance. Then set the tolerance on the baseline criteria and rerun the test.

- 1 Preview whether the tolerance you want to use causes the test to pass. With the result signal selected, in the property box, set **Leading Tolerance** to .04.

PROPERTY	VALUE
Name	☒ shift_logic:1
Status	✓
Absolute Tolerance	0
Relative Tolerance	0.00%
Leading Tolerance	0.04
Lagging Tolerance	0
Block Path	sf_car/shift_logic

When you change this value, the status changes to show that the failed tests pass.

- 2 When you are satisfied with the tolerance value, enter it in the baseline criteria so you can rerun the test and save the new pass-fail result. In the **Test Browser** pane, select the test case in the Solver Compare test.
- 3 Under **Baseline Criteria**, change the **Leading Tol** value for the solver_baseline.mat file to .04.

By default, each signal inherits this value from the baseline file. You can override the value for each signal.

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL
▼ ☒ solver_baseline.mat	0	0.00%	0.04	0
☒ shift_logic:1	0	0.00%	0.04	0

- 4 Run the test again. The test passes.
- 5 To store the tolerance value and the passed test with the test file, save the test file.

See Also

`sltest.testmanager.BaselineCriteria` | `sltest.testmanager.SignalCriteria`

Related Examples

- “Baseline Testing” on page 6-240

Specify Test Properties in the Test Manager

In this section...
“Test Case, Test Suite, and Test File Sections Summary” on page 6-173 “Create Test Case from External File” on page 6-175 “Tags” on page 6-175 “Description” on page 6-175 “Requirements” on page 6-175 “RoadRunner” on page 6-176 “System Under Test” on page 6-176 “Simulation 1 and Simulation 2” on page 6-178 “Parameter Overrides” on page 6-178 “Callbacks” on page 6-179 “Inputs” on page 6-181 “Simulation Outputs” on page 6-182 “Configuration Settings Overrides” on page 6-183 “Fault Settings” on page 6-184 “Sequence Diagram Assessment” on page 6-184 “Baseline Criteria” on page 6-184 “Equivalence Criteria” on page 6-186 “Iterations” on page 6-186 “Logical and Temporal Assessments” on page 6-187 “Custom Criteria” on page 6-188 “Coverage Settings” on page 6-188 “Test File Options” on page 6-190 “Test File Content” on page 6-190

The Test Manager has property settings that specify how test cases, test suites, and test files run. To open the Test Manager, use `sltest.testmanager.view`. For information about the Test Manager, see Simulink Test Manager

Test Case, Test Suite, and Test File Sections Summary

When you open a test case, test suite, or test file in the Test Manager, the test settings are grouped into sections. Test cases, test suites, and test files have different sections and settings. Click a test case, test suite, or test file in the **Test Browser** pane to see its settings.

Test Section	Test Case	Test Suite	Test File
“Tags” on page 6-175	✓	✓	✓
“Description” on page 6-175	✓	✓	✓

Test Section	Test Case	Test Suite	Test File
"Requirements" on page 6-175	✓	✓	✓
"RoadRunner" on page 6-176	✓		
"System Under Test" on page 6-176	✓		
"Simulation 1 and Simulation 2" on page 6-178	✓		
"Parameter Overrides" on page 6-178	✓		
"Callbacks" on page 6-179	✓	✓	✓
"Inputs" on page 6-181	✓		
"Simulation Outputs" on page 6-182	✓		
"Configuration Settings Overrides" on page 6-183	✓		
"Fault Settings" on page 6-184	✓		
"Sequence Diagram Assessment" on page 6-184	✓		
"Baseline Criteria" on page 6-184	✓		
"Equivalence Criteria" on page 6-186	✓		
"Iterations" on page 6-186	✓		
"Logical and Temporal Assessments" on page 6-187	✓		
"Custom Criteria" on page 6-188	✓		
"Coverage Settings" on page 6-188	✓	✓	✓
"Test File Options" on page 6-190			✓
"Test File Content" on page 6-190			✓

If you do not want to see all of the available test sections, you can use the Test Manager preferences to hide sections:

- 1 In the Test Manager toolstrip, click **Preferences**.
- 2 Select the **Test File**, **Test Suite**, or **Test Case** tab.
- 3 Select the sections to show, or clear the sections to hide. To show only the sections in which you have already set or changed settings, clear all selections in the **Preferences** dialog box.
- 4 Click **OK**.

Sections that you already modified appear in the Test Manager, regardless of the preference setting.

To set these properties programmatically, see `sltest.testmanager.getpref` and `sltest.testmanager.setpref`.

Create Test Case from External File

To use an existing Excel file that is in the supported Simulink Test format to create a test case, select **Create test case from external file**. Then, enter the path to the file. The corresponding API properties are `IsTestDataReferenced` and `TestDataPath`, which are described in the `setProperty` method of `sltest.testmanager.TestCase`.

To learn how to format the Excel file, see “Microsoft Excel Import, Export, and Logging Format” and “Create External Data Files to Use in Test Cases” on page 6-84.

To use an existing Excel that is not in the supported format, write an adapter function so you can use that file in the Test Manager. Then, register the file using the `sltest.testmanager.registerTestAdapter` function. If you have registered an adapter, when you select **Create test case from external file**, the Test Manager displays two fields, one for the path to the Excel file and one for the adapter function name. See `sltest.testmanager.registerTestAdapter` for information and an example.

Tags

Tag your test file, test suite, or test case with categorizations, such as **safety**, **logged-data**, or **burn-in**. Filter tests using these tags when executing tests or viewing results. See “Filter Test Execution, Results, and Coverage” on page 6-231.

For the corresponding API, see the `Tags` property of `sltest.testmanager.TestFile`, `sltest.testmanager.TestSuite`, or `sltest.testmanager.TestCase`, respectively.

Description

Add descriptive text to your test case, test suite, or test file.

For the corresponding API, see the `Description` property of `sltest.testmanager.TestFile`, `sltest.testmanager.TestSuite`, or `sltest.testmanager.TestCase`, respectively.

Requirements

If you have Requirements Toolbox installed, you can establish traceability by linking your test file, test suite, or test case to requirements. For more information, see “Link Test Cases to Requirements” (Requirements Toolbox). For more information on how to link iterations or assessments to requirements, see “Link Simulink Test Items to Requirements” (Requirements Toolbox).

To link a test case, test suite, or test file to a requirement:

- 1 Open the Requirements Editor. In the Simulink Toolstrip, on the **Apps** tab, under Model Verification, Validation, and Test, click **Requirements Editor**.
- 2 Highlight a requirement.
- 3 In the Test Manager, in the **Requirements** section, click the arrow next to the **Add** button and select **Link to Selected Requirement**.
- 4 The requirement link appears in the **Requirements** list.

For the corresponding API, see the Requirements property of `sltest.testmanager.TestFile`, `sltest.testmanager.TestSuite`, or `sltest.testmanager.TestCase`, respectively.

RoadRunner

This section appears only for a RoadRunner automated driving test case.

Console Mode — Whether the console mode is on or off. If the checkbox is selected, console mode is on and the RoadRunner user interface does not display when the simulation runs. Use console mode to run a RoadRunner test in a non-graphical environment or if you do not need to view the simulation as it runs.

Force Pacing Off — Whether pacing is on or off. Clear this checkbox to turn pacing on, which lets you change the speed at which the RoadRunner scenario simulation runs. If the checkbox is selected, pacing is off and you cannot control the speed of the RoadRunner simulation.

RoadRunner Scenario — Full path to the preconfigured RoadRunner scenario file. This file is usually stored in the Scenarios folder in the RoadRunner project folder.

Override RoadRunner Setup Settings — Whether the RoadRunner test case overrides the preferred path settings during test execution or uses the preferred path settings defined using `roadrunnerSetup`. Select this check box to override the preferred path settings.

RoadRunner Project Folder — Full path to the RoadRunner project folder. This field appears when you select **Override RoadRunner Setup Settings**.

RoadRunner Installation Folder — Full path to the folder where the RoadRunner application executable is installed. This field appears when you select **Override RoadRunner Setup Settings**.

System Under Test

Specify the model you want to test in the **System Under Test** section. To use an open model in the currently active Simulink window, click the **Use current model** button .

Note The model must be available on the path to run the test case. You can add the folder that contains the model to the path using the preload callback. See “Callbacks” on page 6-179.

Specifying a new model in the **System Under Test** section can cause the model information to be out of date. To update the model test harnesses, Signal Editor scenarios, and available configuration sets, click the **Refresh** button .

For a RoadRunner test case, you specify the model in System Under Test section that you set as a behavior of an actor in the RoadRunner scenario being tested.

For the corresponding API, see the `Model` name-argument pair of `setProperty`.

Test Harness

If you have a test harness in your system under test, then you can select the test harness to use for the test case. If you have added or removed test harnesses in the model, click the **Refresh** button  to view the updated test harness list.

For more information about using test harnesses, see “Refine, Test, and Debug a Subsystem” on page 2-22.

For the corresponding API, see the `HarnessName` name-argument pair of `setProperty`.

Simulation Settings and Release Overrides

To override the simulation mode of the model, select a mode from the list. If the model contains SIL/PIL blocks, and you need to run in normal mode, enable **Override model blocks in SIL/PIL mode to normal mode**. To change this setting programmatically, see the `OverrideSILPILMode` name-argument pair of `setProperty`. You can also override the simulation mode at test execution. For more information, see “Override the Simulation Mode During Test Execution” on page 6-32.

You can simulate the model and run tests in more than one MATLAB release that is installed on your system. Use **Select releases for simulation** to select available releases. You can use releases from R2011b forward.

To add one or more releases so they are available in the Test Manager, click **Add releases** in **Select releases for simulation** to open the **Release** pane in the Test Manager Preferences dialog box. Navigate to the location of the MATLAB installation you want to add, and click **OK**.

You can add releases to the list and delete them. You cannot delete the release in which you started the MATLAB session.

For more information, see “Run Tests in Multiple Releases of MATLAB” on page 6-102. For the corresponding API, see the `Release` name-argument pair of `setProperty`.

System Under Test Considerations

- The **System Under Test** cannot be in fast restart or external mode.
- To stop a test running in **Rapid Accelerator** mode, press **Ctrl+C** at the MATLAB command prompt.
- When running parallel execution in rapid accelerator mode, streamed signals do not show up in the Test Manager.
- The **System Under Test** cannot be a protected model.

Simulation 1 and Simulation 2

These sections appear in equivalence test cases. Use them to specify the details about the two simulations that you want to compare. Enter the system under test, the test harness if applicable, and simulation setting overrides under **Simulation 1**. You can then click **Copy settings from Simulation 1** under **Simulation 2** to use a starting point for your second set of simulation settings. You cannot compare more than two models in a single equivalence test case.

For the test to pass, Simulation 1 and Simulation 2 must log the same signals.

Use these sections with the **Equivalence Criteria** section to define the premise of your test case. For an example of an equivalence test, see “Back-to-Back Equivalence Testing” on page 6-36.

For the corresponding API, see the `SimulationIndex` name-argument pair of `setProperty`.

Parameter Overrides

Specify parameter values in the test case to override the parameter values in the model workspace, data dictionary, base workspace, or in a model reference hierarchy. Parameters are grouped into sets. Turn parameter sets and individual parameter overrides on or off by using the check box next to the set or parameter. To copy an individual parameter or parameter set and paste it into another parameter set, select the parameter and right-click to use **Copy** and **Paste** from the context menu.

For an example that uses parameter overrides, see “Override Model Parameters in a Test Case” on page 6-27. For the corresponding APIs, see the `sltest.testmanager.ParameterOverride` class, and the `OverrideStartTime`, `OverrideStopTime`, `OverrideInitialState`, `OverrideModelOutputSettings`, and `ConfigSetOverrideSetting` name-argument pairs of the `setProperty` method.

Add Parameter Override

- 1 Click **Add**.

A dialog box opens with a list of parameters. If needed, click **Refresh**  in the dialog box to update the parameter list.

- 2 Select the parameter you want to override.
- 3 To add the parameter to the parameter set, click **OK**.
- 4 Enter the override value in the parameter **Override Value** column.

Add Parameter Set

To add an empty parameter set, click **Add > Add Parameter Set**. The new parameter set appears in the **Parameter Overrides** table.

Add Parameter Overrides From a File

To add a set of parameter overrides from a MAT file, including one generated by Simulink Design Verifier, from a MATLAB script (.m file), or from an Excel file, click **Add > Add File**. To change an override value that is imported from a file, you must edit and save the file outside of MATLAB and add the overrides file again.

External files must be in a format that the Test Manager can read. See “Create External Data Files to Use in Test Cases” on page 6-84. The allowable formats are:

- Excel files — For information on how to format Excel files, see also “Microsoft Excel Import, Export, and Logging Format”
- MAT files — To create MAT files, see `matfile` and `save`.
- MATLAB scripts (.m files) — In MATLAB scripts, assign variables and values using MATLAB syntax.

Variant Configuration

If the model being tested uses variants and has defined variant configurations, select the variant configuration to use with this test case from the **Variant Configuration** field. For information, see “Variant Configurations”, and for an example that uses variant configurations, see “Run Tests for Variant Models Using Variant Configurations”.

For the corresponding API, use the `sltest.testmanager.testcase setProperty` method to set the `VariantConfiguration` property.

Restore Default Parameter Value

To delete an override and restore the default parameter value, right-click on the parameter and click **Delete** in the context menu.

Parameter Overrides Considerations

- The Test Manager displays only top-level system parameters from the system under test.
- Parameter overrides are evaluated only at test run time.
- You can export parameters to MATLAB script files or MAT files only.

Callbacks

Test-File Level Callbacks

Two callback scripts are available in each test file that execute at different times during a test:

- Setup runs before test file executes.
- Cleanup runs after test file executes.

For the corresponding test case APIs, see the `PreloadCallback`, `PostloadCallback`, `CleanupCallback`, and `PreStartRealTimeApplicationCallback` name-argument pairs of the `TestCase setProperty` method.

For the corresponding test file APIs, see the `SetupCallback` and `CleanupCallback` name-argument pairs of the test file `TestFile setProperty` method.

Test-Suite Level Callbacks

Two callback scripts are available in each test suite that execute at different times during a test:

- Setup runs before the test suite executes.

- Cleanup runs after the test suite executes.

If a test suite does not have any test cases, the test suite callbacks do not execute.

For the corresponding APIs, see the `SetupCallback` and `CleanupCallback` name-argument pairs of the `TestSuite` `setProperty` method.

Test-Case Level Callbacks

Three callback scripts are available in each test case that execute at different times during a test:

- **Pre-load** runs before the model loads and before the model callbacks.
- **Post-load** runs after the model loads and the `PostLoadFcn` model callback.
- **Cleanup** runs after simulations and model callbacks.

See “Test Execution Order” on page 6-227 for information about the order in which callbacks occur and models load and simulate.

To run a single callback script, click the **Run** button  above the corresponding script.

You can use predefined variables in the test case callbacks:

- `sltest_bdroot` available in **Post-Load**: The model simulated by the test case. The model can be a harness model.
- `sltest_sut` available in **Post-Load**: The system under test. For a harness, it is the component under test.
- `sltest_isharness` available in **Post-Load**: Returns true if `sltest_bdroot` is a harness model.
- `sltest_simout` available in **Cleanup**: Simulation output produced by simulation.
- `sltest_iterationName` available in **Pre-Load**, **Post-Load**, and **Cleanup**: Name of the currently executing test iteration.
- `sltest_testIteration` available in **Pre-Load**, **Post-Load**, and **Cleanup**: Current test iteration object.
- `sltest_testCase` available in **Pre-Load**, **Post-Load**, and **Cleanup**: Current test case object.
- `sltest_roadRunnerApp` available in **Pre-Load**, **Post-Load**, and **Cleanup**: RoadRunner application object. You can use this variable only for RoadRunner test cases or iterations.
- `sltest_roadRunnerSim` available in **Pre-Load**, **Post-Load**, and **Cleanup**: RoadRunner simulation object. You can use this variable only for RoadRunner test cases or iterations.

`disp` and `fprintf` do not work in callbacks. To verify that the callbacks are executed, use a MATLAB script that includes breakpoints in the callbacks.

The test case callback scripts are not stored with the model and do not override Simulink model callbacks. Consider the following when using callbacks:

- Do not modify `sltest_testIteration` or `sltest_testCase`. Use these callbacks to query aspects of the current iteration or test case, respectively.
- To stop execution of an infinite loop from a callback script, press **Ctrl+C** at the MATLAB command prompt.

- `sltest.testmanager` functions are not supported.
- To avoid unintended consequences, do not use "clear" APIs, such as `clear all`, in callbacks.

For the corresponding APIs, see the `PreloadCallback`, `PostloadCallback`, `CleanupCallback`, and `PreStartRealTimeApplicationCallback` name-argument pairs of the `TestCase.setProperty` method.

Assessment Callback

You can enter a callback to define variables and conditions used only in logical and temporal assessments by using the **Assessment Callback** section. See "Assessment Callback" on page 6-187 in the Logical and Temporal Assessments section for more information.

For the corresponding API, see `setAssessmentsCallback`.

Inputs

A test case can use input data from:

- A Signal Editor block in the system under test. Select **Signal Editor scenario** and select the scenario. The system under test can have only one Signal Editor block at the top level. See "Use Scenarios and Insert Signals in Signal Editor".
- An external data file. In the **External Inputs** table, click **Add**. Select a MAT file or Microsoft Excel file.

For more information on using external files as inputs, see "Use External File Data in Test Cases" on page 6-76. For information about the file format for using Microsoft Excel files in the Test Manager, see "Create External Data Files to Use in Test Cases" on page 6-84.

- Scenarios in a Test Sequence block. First, click the refresh arrow  next to the **Test Sequence Block** field, then select the Test Sequence block in the model that contains the scenarios. If you do not also select a scenario from **Override with Scenario** and do not use iterations, then the test runs the active scenario in the selected Test Sequence block. If you do not also select a scenario, but do use iterations, then the active scenario in the Test Sequence block is the default for all the iterations.

Use **Override with Scenario** to override the active scenario in the selected Test Sequence block. Click the refresh arrow next to the **Override with Scenario** field. Then, select the scenario to use instead of the active scenario or as the default for the iterations. In the **Iterations** section, you can change the scenario assigned to each iteration. For more information, see "Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager" on page 3-54.

- An input file template that you create and populate with data. See "Create External Data Files to Use in Test Cases" on page 6-84.

To include the input data in your test results set, select **Include input data in test result**.

If the time interval of your input data is shorter than the model simulation time, you can limit the simulation to the time specified by your input data by selecting **Stop simulation at last time point**.

For more information on test inputs, see the Test Authoring: Inputs page.

Edit Input Data Files in Test Manager

From the Test Manager, you can edit your input data files.

To edit a file, select the file and click **Edit**. You can then edit the data in the Signal Editor for MAT files or Microsoft Excel for Excel files.

To learn about the Signal Editor, see “Create and Edit Signal Data”.

To learn about the format for Excel files, see “Input, Baseline, and Parameter Override Test Case Data Formats in Excel” on page 6-89.

For the corresponding API, see `sltest.testmanager.TestInput`.

Simulation Outputs

Use the **Simulation Outputs** section to add signal outputs to your test results. Signals logged in your model or test harness can appear in the results after you add them as simulation outputs. You can then plot them. Add individual signals to log and plot or add a signal set.

Logged Signals — In the **Logged Signals** subsection, click **Add**. Follow the user interface.

To copy a logged signal set to another test case in the same or a different test file, select the signal set in **Logged Signals**, right-click to display the context menu, and click **Copy**. Then, in the destination test case, select the signal set in **Logged Signals**, right-click the signal, and click **Paste**. You can copy and paste more than one logged signal set at a time.

For a test case, you can use the **SDI View File** setting to specify the path to a Simulation Data Inspector (SDI) view file. You can assign a different view file to each test case. The view file configures which signals to plot and their layout in the test case results. The Test Manager does not support some configurations in the SDI view file, such as axes plot layouts other than time plots and axes layouts other than *N*-by-*M* grids. However, the Test Manager applies a similar configuration, if possible. You cannot save an SDI view file from the Test Manager, although when you save the test and results in an MLDATX test file, the file saves the current layout for that test. Use `Simulink.sdi.saveView` to create and save an SDI view file. For more information, see “Save and Share Simulation Data Inspector Data and Views”.

Other Outputs — Use the options in the **Other Outputs** subsection to add states, final states, model output values, data store variables, and signal logging values to your test results. To enable selecting one or more of these options, click **Override model settings**.

- **States** — Include state values between blocks during simulation.
- **Final states** — Include final state values.
- **Output** — Include model output values.
- **Data stores** — Include logged data store variables in Data Store Memory blocks in the model. This option is selected by default.
- **Signal logging** — Include logged signals specified in the model. This option is selected by default. If you selected **Log Signal Outputs** when you created the harness, all of the output signals for the component under test are logged and returned in test results, even though they are not listed in the **Simulation Outputs** section. To turn off logging for one of the signals, in the test harness, right-click a signal and select **Stop Logging Selected Signals**.

For more information, see “Capture Simulation Data in a Test Case” on page 6-93. For the corresponding API, see the `OverrideModelOutputSettings` name-argument pair of `setProperty`.

Output Triggers — Use the **Output Triggers** subsection to specify when to start and stop signal logging based on a condition or duration. Note that a test passes if it passes while triggered even if test fails outside of triggered time.

The **Start Logging** options are:

- **On simulation start** — Start logging data when simulation starts.
- **When condition is true** — Start logging when the specified condition expression is true. Click the edit symbol next to **Condition** to display an edit box, where you enter the condition.
- **After duration** — Start logging after the specified number of seconds have passed since the start of simulation. Click on the value next to **Duration(sec)** to display an edit box where you enter the duration in seconds.

The **Stop Logging** options are:

- **When simulation stops** — Stop logging data when simulation ends.
- **When condition is true** — Stop logging when the specified condition expression is true. Click the edit symbol next to **Condition** to display an edit box, where you enter the condition. Variables in the condition appear in the **Symbols** editor, where you can map them to a model element or expression, or rename them.
- **After duration** — Stop logging after the specified number of seconds have passed since logging started. Click on the value next to **Duration(sec)** to display an edit box where you enter the duration in seconds.

Shift time to zero — Shifts the logging start time to zero. For example, if logging starts at time 2, then selecting this option shifts all times back by 2 seconds.

Symbols — Click **Add** to map a signal from the model to a symbol name. You can use that symbol in a trigger condition. For information on using and mapping symbols, see “Assess Temporal Logic by Using Temporal Assessments” on page 3-85

Configuration Settings Overrides

For the test case, you can specify configuration settings that differ from the settings in the model. Setting the configuration settings in the test case enables you to try different configurations for a test case without modifying the model. The configuration settings overrides options are:

- **Do not override model settings** — Use the current model configuration settings
- **Name** — Name of active configuration set. A model can have only one active configuration set. Refresh the list to see all available configuration sets and select the desired one to be active. If you leave the default [Model Settings] as the name, the simulation uses the default, active configuration set of the model.
- **Attach configuration set in a file** — Path to the external file (**File Location**) that contains a configuration set variable. The variable you specify in **Variable Name** references the name of a configuration set in the file. For information on creating a configuration set, see `Simulink.ConfigSet` and “Save a Configuration Set”. For information on configuration set references, see “Share a Configuration with Multiple Models”.

For the corresponding API, see the `ConfigSetOverrideSetting`, `ConfigSetName`, `ConfigSetVarName`, `ConfigSetFileLocation`, and `ConfigSetOverrideSetting` name-argument pairs of `setProperty`.

Fault Settings

If you have Simulink Fault Analyzer, use this section to add faults to test cases and inject them into your models, create fault sets, retrieve the faults from the model under test, and specify active faults for simulation. For more information, see “Write and Run Tests that Inject Faults by Using the Simulink Test Manager” on page 6-232. To learn more about adding faults to models by using Simulink Fault Analyzer, see “Define and Model Faults” (Simulink Fault Analyzer).

For the corresponding APIs, see the `sltest.testmanager.FaultSet` and `sltest.testmanager.SpecifiedFault` classes, the `addFaultSet` and `getFaultSets` methods of the `sltest.testmanager.TestCase` class, the `sltest.testmanager.refreshFaults` function, and the `FaultSet` property of the `sltest.testmanager.TestIterationResult` class.

Sequence Diagram Assessment

Use this section to add a sequence diagram to your simulation or baseline tests. When you add a sequence diagram to your test, the sequence diagram simulates during test execution. To add a sequence diagram to your test case, in the **Sequence Diagram Assessment** section, select a sequence diagram. For more information about sequence diagrams, see “Describe System Behavior Using Sequence Diagrams” (System Composer).

You can select an architecture model or a test harness with defined sequence diagrams by using the **System Under Test** section of the Test Manager. See “System Under Test” on page 6-176.

To programmatically add a sequence diagram to your test case, use the `setProperty` method of `sltest.testmanager.TestCase` and set the `SequenceDiagram` property to the sequence diagram.

Sequence Diagram Assessment Considerations

- You can select only one sequence diagram per test case. To simulate more than one sequence diagram during test execution, use the **Iterations** section to create table or scripted iterations. See “Iterations” on page 6-186 and “Test Iterations” on page 6-139.
- Fast restart and multiple release testing are not supported.
- When you create a test harness that references an architecture model or component that has defined sequence diagrams, the sequence diagrams simulate only when you simulate the test harness as part of a test using the Test Manager.

Baseline Criteria

The **Baseline Criteria** section appears in baseline test cases. When a baseline test case executes, the Test Manager captures signal data from signals in the model marked for logging and compares them to the baseline data. To compare the test results of multiple models, you can compare the test results of each model to the same baseline data.

Include Baseline Data in Results and Reports

Click **Include baseline data in test result** to include baseline data in test result plots and test reports.

Capture Baseline Criteria

To capture logged signal data from the system under test to use as the baseline criteria, click **Capture**. Then follow the prompts in the Capture Baseline dialog box. Capturing the data compiles and simulates the system under test and stores the output from the logged signals to the baseline. For a baseline test example, see “Baseline Testing” on page 6-240.

Note If your model has multiple variant subsystems, capture independent baselines for each variant subsystem, instead of reusing a baseline.

For the corresponding API, see the `captureBaselineCriteria` method.

You can save the signal data to a MAT file or a Microsoft Excel file. To understand the format of the Excel file, see “Input, Baseline, and Parameter Override Test Case Data Formats in Excel” on page 6-89.

You can capture the baseline criteria using the current release for simulation or another release installed on your system. Add the releases you want to use in the Test Manager preferences. Then, select the releases you want available in your test case using the **Select releases for simulation** option in the test case. When you run the test, you can compare the baseline against the release you created the baseline in or against another release. For more information, see “Run Tests in Multiple Releases of MATLAB” on page 6-102.

When you select Excel as the output format, you can specify the sheet name to save the data to. If you use the same Excel file for input and output data, by default both sets of data appear in the same sheet.

If you are capturing the data to a file that already contains outputs, specify the sheet name to overwrite the output data only in that sheet of the file.

To save a baseline for each test case iteration in a separate sheet in the same file, select **Capture Baselines for Iterations**. This check box appears only if your test case already contains iterations. For more information on iterations, see “Test Iterations” on page 6-139.

Specify Tolerances

You can specify tolerances to determine the pass-fail criteria of the test case. You can specify absolute, relative, leading, and lagging tolerances for individual signals or the entire baseline criteria set.

After you capture the baseline, the baseline file and its signals appear in the table. In the table, you can set the tolerances for the signals. To see tolerances used in an example for baseline testing, see “Compare Simulation to Baseline Data” on page 6-246.

For the corresponding API, see the `AbsTol`, `RelTol`, `LeadingTol`, and `LaggingTol` properties of `sltest.testmanager.BaselineCriteria`.

Add File as Baseline

By clicking **Add**, you can select an existing file as a baseline. You can add MAT files and Microsoft Excel files as the baseline. Format Microsoft Excel files as described in “Input, Baseline, and Parameter Override Test Case Data Formats in Excel” on page 6-89.

For the corresponding API, see the `addInput` method.

Update Signal Data in Baseline

You can edit the signal data in your baseline, for example, if your model changed and you expect different values. To open the Signal Editor or the Microsoft Excel file for editing, select the baseline file from the list and click **Edit**. See “Manually Update Signal Data in a Baseline” on page 6-112.

You can also update your baseline when you examine test failures in the data inspector view. See “Examine Test Failures and Modify Baselines” on page 6-110.

Equivalence Criteria

This section appears in equivalence test cases. The equivalence criteria is a set of signal data to compare in Simulation 1 and Simulation 2. Specify tolerances to regulate pass-fail criteria of the test. You can specify absolute, relative, leading, and lagging tolerances for the signals.

To specify tolerances, first click **Capture** to run the system under test in Simulation 1 and add signals marked for logging to the table. Specify the tolerances in the table.

After you capture the signals, you can select signals from the table to narrow your results. If you do not select signals under **Equivalence Criteria**, running the test case compares all the logged signals in Simulation 1 and Simulation 2.

For an example of an equivalence test case, see “Back-to-Back Equivalence Testing” on page 6-36.

For the corresponding API, see the `captureEquivalenceCriteria` method.

Iterations

Use iterations to repeat a test with different parameter values, configuration sets, variant configurations, or input data.

- You can run multiple simulations with the same inputs, outputs, and criteria by sweeping through different parameter values in a test case.
- Models, external data files, and Test Sequence blocks can contain multiple test input scenarios. To simplify your test file architecture, you can run different input scenarios as iterations rather than as different test cases. You can apply different baseline data to each iteration, or capture new baseline data from an iteration set.
- You can iterate over different configuration sets, for example to compare results between solvers or data types. You can also iterate over different scenarios in a Test Sequence block, and over variants by using variant configurations.

To create iterations from defined parameter sets, Signal Editor scenarios, Test Sequence scenarios, external data files, or iterations that use configuration sets or variant configurations, use table iterations. You can select the configuration to use for each iteration in its column of the table.

To create a custom set of iterations from the available test case elements, add a MATLAB scripted iteration to the test case.

To run the iterations without recompiling the model for each iteration, enable **Run test iterations in fast restart**. When selected, this option reduces simulation time.

For more information about test iterations, including using predefined variables in scripted iterations, see “Test Iterations” on page 6-139. For more information about fast restart, see “How Fast Restart Improves Iterative Simulations”.

For the corresponding API, see `sltest.testmanager.TestIteration`.

Logical and Temporal Assessments

Create temporal assessments using the form-based editor that prompts you for conditions, events, signal values, delays, and responses. When you collapse the individual elements, the editor displays a readable statement summarizing the assessment. See “Assess Temporal Logic by Using Temporal Assessments” on page 3-85 and “Logical and Temporal Assessment Syntax” on page 3-99 for more information.

To copy and paste an assessment or symbol, select the assessment or symbol and right-click to display the context menu. You can select a single assessment or symbol or select multiple assessments or symbols. Alternatively, to copy or paste selected assessments or symbols, use **Ctrl+C** or **Ctrl+V**. Pasting the assessment adds it to the end of the assessment list in the current test case. You can also paste to a different test case. The assessments and their symbol names change to the default names in the pasted assessment. You can also use the context menu to delete assessments. To delete symbols, use the **Delete** button. If you delete an assessment or symbol, you cannot paste it even if you copied it before deleting it.

Assessment Callback

You can define variables and use them in logical and temporal assessment conditions and expressions in the **Assessment Callback** section.

Define variables by writing a script in the **Assessment Callback** section. You can map these variables to symbols in the **Symbols** pane by right-clicking the symbol, selecting **Map to expression**, and entering the variable name in the **Expression** field. For information on how to map variables to symbols, see **Map to expression** under “Resolve Assessment Parameter Symbols” on page 3-87.

The **Assessment Callback** section has access to the predefined variables that contain test, simulation, and model data. You can define a variable as a function of this data. For more information, see “Define Variables in the Assessment Callback Section” on page 3-102. For the corresponding API methods, see `setAssessmentsCallback` and `getAssessmentsCallback`.

If your assessments use `at least`, `at most`, `between`, or `until` syntax, select **Extend Results** to produce the minimum possible untested results. In some cases, none or not all untested results can be tested, so the results will still show some untested results. When you extend the test results,

previously passing tests might fail. Leave **Extend Results** checked unless you need to avoid an incompatibility with earlier test results.

Using `==` or `~=` operators in logical or temporal assessments might cause a test to issue a warning or fail when comparing floating-point data. If you use floating-point data, consider including a tolerance for the assessment. For example, instead of using `if(x == 5)` in a statement, check that `x` is within a tolerance of 0.001 by using `if(abs(x-5) < 0.001)`.

Symbol `t` (time)

The symbol `t` is automatically bound to simulation time and can be used in logical and temporal assessment conditions. This symbol does not need to be mapped to a variable and is not visible in the **Symbols** pane. For example, to limit an assessment to a time between 5 and 7 seconds, create a **Trigger-response** assessment and, in the trigger condition, enter `t < 5 & t > 7`. To avoid unexpected behavior, do not define a new symbol `t` in the **Symbols** pane.

Symbol Data Type

If you map a symbol to a discrete data signal that is linearly interpolated, the interpolation is automatically changed to zero-order hold during the assessment evaluation.

Custom Criteria

This section includes an embedded MATLAB editor to define custom pass/fail criteria for your test. Select **function customCriteria(test)** to enable the criteria script in the editor. Expand **Perform custom criteria analysis on test results** to see a list of available predefined variables. Custom criteria operate outside of model run time; the script evaluates after model simulation.

Common uses of custom criteria include verifying signal characteristics or verifying test conditions. MATLAB Unit Test qualifications provide a framework for verification criteria. For example, this custom criteria script gets the last value of the signal `PhiRef` and verifies that it equals 0:

```
% Get the last value of PhiRef from the dataset Signals_Req1_3
lastValue = test.sltest_simout.get('Signals_Req1_3').get('PhiRef').Values.Data(end);

% Verify that the last value equals 0
test.verifyEqual(lastValue,0);
```

See “Process Test Results with Custom Scripts” on page 6-197. For a list of MATLAB Unit Test qualifications, see “Table of Verifications, Assertions, and Other Qualifications”.

You can also define plots in the **Custom Criteria** section. See “Create, Store, and Open MATLAB Figures” on page 6-206.

For the corresponding API, see `sltest.testmanager.CustomCriteria`.

Coverage Settings

Use this section to configure coverage collection for a test file. The settings propagate from the test file to the test suites and test cases in the test file. You can turn off coverage collection or one or more coverage metrics for a test suite or test case, unless your test is a MATLAB-based Simulink test.

For MATLAB-based Simulink tests, you can change the coverage settings only at the test file level. If you change the coverage settings in the Test Manager, the changes are not saved to the MATLAB-

based Simulink test script file. If you also set the coverage using the `sltest.plugins.ModelCoveragePlugin` in a MATLAB-based Simulink test script (.m) file or at the command line, the Test Manager uses the coverage settings from the test script instead of the Test Manager coverage settings.

Coverage is not supported for SIL or PIL blocks.

The coverage collection options are:

- **Record coverage for system under test** — Collects coverage for the model or, when included, the component specified in the “System Under Test” on page 6-176 section for each test case. If you are using a test harness, the system under test is the component for which the harness is created. The test harness is not the system under test.
 - For a block diagram, the system under test is the whole block diagram.
 - For a Model block, the system under test is the referenced model.
 - For a subsystem, the system under test is the subsystem.
- **Record coverage for referenced models** — Collects coverage for models that are referenced from within the specified system under test. If the test harness references another model, the coverage results are included for that model, too.
- **Exclude inactive variants** — Excludes from coverage results these variant blocks that are not active at any time while the test runs:
 - Variant blocks in Simulink with **Variant activation time** set to startup
 - Variant configurations in Stateflow charts

When displaying the test results, if you select or clear this option in the **Aggregated Coverage Results** section, the coverage results update automatically. For information, see “Model Coverage for Variant Blocks” (Simulink Coverage).

Note Coverage settings, including coverage filter files, in the Test Manager override all coverage settings in the model configuration. In the Test Manager, **Do not override model settings** in the Configuration Settings section and **Override model settings** in the Simulation Outputs section do not apply to coverage.

By default the Test Manager includes external MATLAB functions and files in the coverage results. You can exclude external MATLAB functions and files by using `set_param(model, 'CovExternalEMLEnable', 'off', 'CovSFcnEnable', 'off');` at the command line. Alternatively, you can exclude MATLAB functions and files by using the **Include in analysis** setting in the Coverage Analyzer app from within the Simulink model.

For more information about collecting coverage, see “Collect Coverage in Tests” on page 6-151. For the corresponding API, see `sltest.testmanager.CoverageSettings`.

For information on the **Coverage Metrics** options, see “Types of Model Coverage” (Simulink Coverage).

For information about MATLAB-based Simulink tests, see “Using MATLAB-Based Simulink Tests in the Test Manager” on page 6-124.

Test File Options

Close open Figures at the end of execution

When your tests generate figures, select this option to clear the working environment of figures after the test execution completes.

For the corresponding API, see the `CloseFigures` property of `sltest.testmanager.Options`.

Store MATLAB figures

Select this option to store figures generated during the test with the test file. You can enter MATLAB code that creates figures and plots as a callback or in the test case **Custom Criteria** section. See “Create, Store, and Open MATLAB Figures” on page 6-206.

For the corresponding API, see the `SaveFigures` property of `sltest.testmanager.Options`.

Generate report after execution

Select **Generate report after execution** to create a report after the test executes. Selecting this option displays report options that you can set. The settings are saved with the test file.

Note To enable the options to specify the number of plots per page, select **Plots for simulation output and baseline**.

By default, the model name, simulation start and stop times, and trigger information are included in the report.

For the corresponding API, see the `GenerateReport` property of `sltest.testmanager.Options`.

For detailed reporting information, see “Export Test Results” on page 7-16 and “Customize Test Results Reports” on page 7-21.

Test File Content

For a MATLAB-based Simulink test, displays the contents of the M file that defines the test. This section appears only if you opened or created a new MATLAB-based Simulink test. See “Using MATLAB-Based Simulink Tests in the Test Manager” on page 6-124.

See Also

Simulink Test Manager | `sltest.testmanager.getpref` | `sltest.testmanager.setpref`

Preferences

Set test sections to display and releases to use for testing.

Display Tab - Select test sections to show when you select a test file, test suite, or test case in the Test Manager. Sections that you have changed from their default settings are always shown, even if you deselect them in this tab. To show only the sections in which you have already set or changed settings, clear all selections.

Release Tab - Select releases to use with tests. Only installed releases are available to add. Releases you select are available in **Selected releases for simulation list** in the System Under Test section.

Increase Coverage by Generating Test Inputs

In this section...

“Overall Workflow” on page 6-192

“Generate Test Cases Using Simulink Design Verifier” on page 6-193

Using Simulink Design Verifier, you can generate test inputs that replicate design errors, achieve test objectives, or meet coverage criteria. Simulink Test can create test cases that use test inputs and expected outputs from Simulink Design Verifier.

Overall Workflow

Test case generation follows this workflow.

- 1 Choose an existing Simulink Design Verifier results file, or generate new results by analyzing your model.
 - If you use an existing results file, you can load results by either:
 - Using the Simulink Test command `sltest.import.sldvData`.
 - Using Simulink Design Verifier menu items. In the model, on the **Apps** tab, under Model Verification, Validation, and Test, click **Design Verifier**. On the **Tests** tab, click **Simulink Test Manager**. In the **Review Results** section, click **Load Earlier Results**. Select the MAT file with the analysis results.
 - If you run a model analysis, the Simulink Design Verifier Results Summary window appears after the analysis completes.
- 2 In the results summary window, click **Export test cases to Simulink Test**. Test cases and iterations for Requirements Table blocks are linked automatically to the corresponding requirements.
- 3 Enter the name of an existing or new test harness.
- 4 Select a test harness source for the generated test inputs. You can select
 - **Inport**: The inputs are contained in the Simulink Design Verifier data file and mapped to Inport blocks in the test harness. The mapping is shown in the **Inputs** section of the test case. Using the **Inport** option allows you to map other inputs to the test harness Inport blocks, which can be useful for running multiple test cases or iterations using the same test harness. Both MAT and Excel files are supported when the source is **Inport**.
 - **Signal Editor**: The inputs are in scenarios in a Signal Editor block inside the test harness. The Signal Editor block supports MAT files that contain these inputs. You can edit these scenarios in the Signal Editor. See “Use Scenarios and Insert Signals in Signal Editor”.
- 5 Select a new or existing test file, and enter names for the test file and test case.
- 6 Click OK to export the test cases to Simulink Test. The test files and test cases are updated in the Test Manager. Simulink Design Verifier saves a MAT or Excel data file that also includes parameter settings. You can view or override these settings in the Parameter Overrides section of the Test Manager.

Note Another way to import test cases from Simulink Design Verifier is with the Create Test for Component wizard. For information, see “Generate Tests and Test Harnesses for a Model or Components” on page 6-20.

Generate Test Cases Using Simulink Design Verifier

This example shows how to generate test cases for a controller subsystem using Simulink Design Verifier, and export the test cases to a test file in Simulink Test. The example requires a Simulink Design Verifier license.

The model used in this example is a closed-loop heat pump system. The controller inputs are the measured room temperature and the set temperature. The controller outputs a bus of three signals that control the fan, heat pump, and direction of the heat pump (heat or cool). The model contains a harness that tests the heating and cooling scenarios.

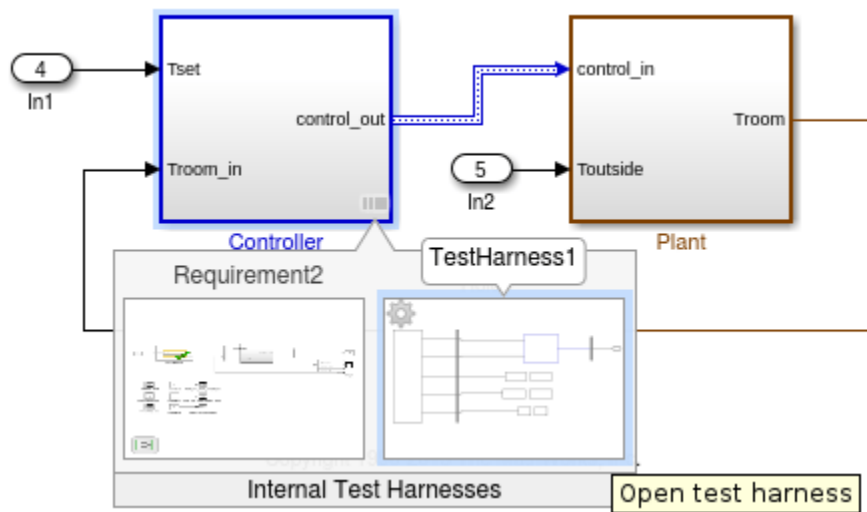
Open the Model

```
sltestTestCaseFromDVExample
```

Generate Tests and Export to Simulink Test

1. Right-click the Controller block and select **Design Verifier > Generate Tests for Subsystem**. Simulink Design Verifier generates the tests for the component.
2. In the results summary window, click **Export test cases to Simulink Test**.
3. In the Export Design Verifier Test Cases dialog box, enter:
 - **Test Harness:** TestHarness1
 - **Harness Source:** Signal Editor
 - Select **Use a new test file**
 - **Test File:** TestFile_GeneratedTests.mldatx
 - **Test Case:** <Create a new test case>
4. Click **OK**.

A new test file is created in the working folder, and a test harness, owned by the Controller subsystem, is added to the main model. Click the harness badge to preview the new test harness.

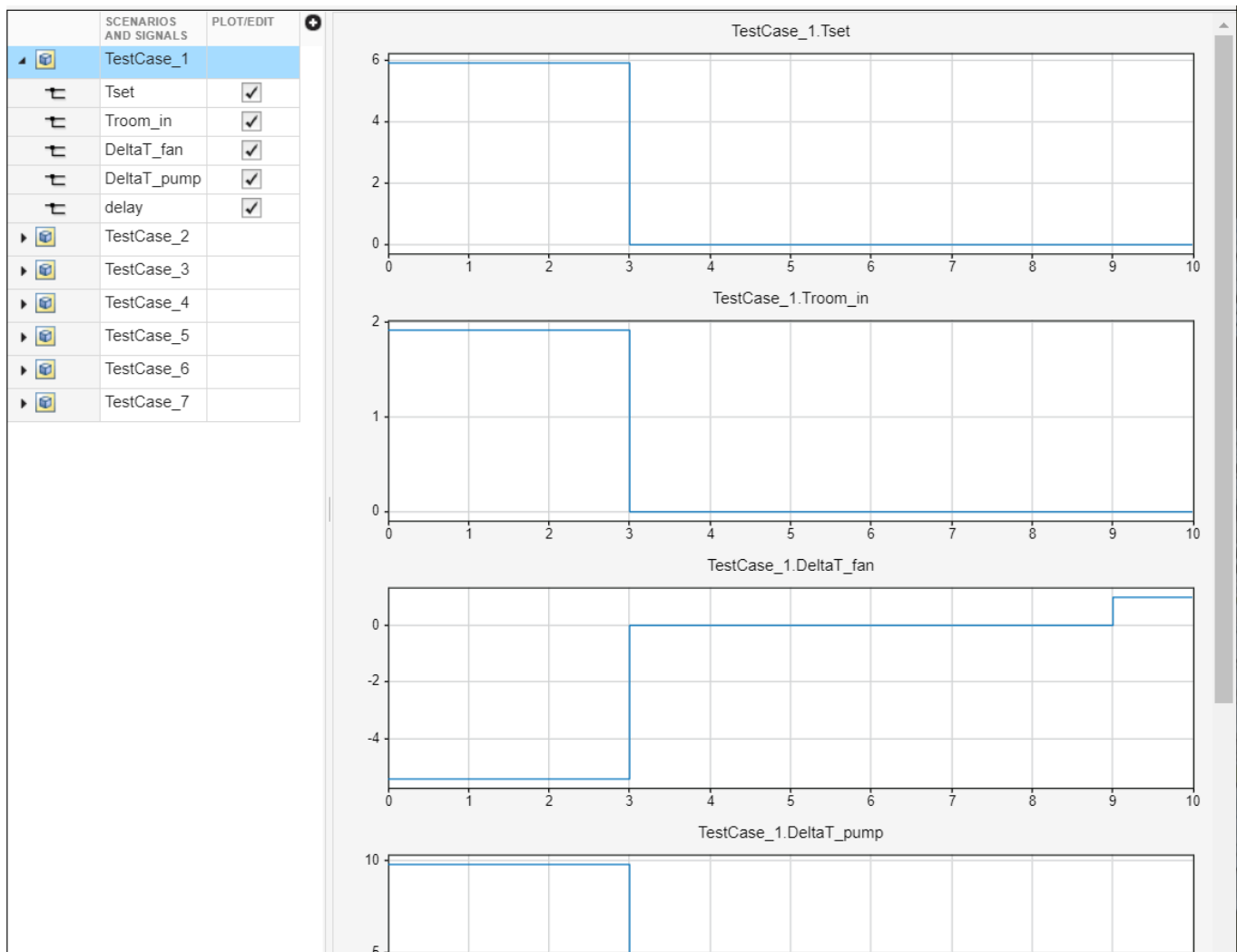


5. Click the **TestHarness1** thumbnail to open the harness. Then double-click the **Harness Inputs Signal Editor** block source.

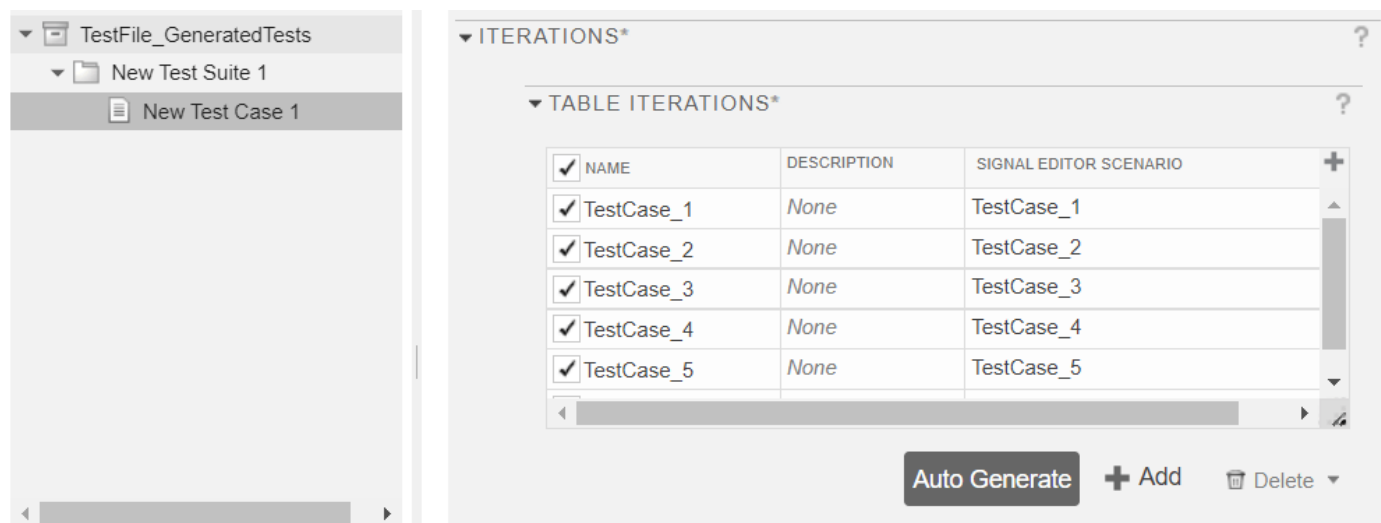
6. In the **Block Parameters: Harness Inputs** dialog box, click **Launch Signal Editor**.



7. To see the test inputs in the **Signal Editor**, expand a test case and select the inputs.



8. In the Test Manager, the new test case displays the system under test, and the test harness containing the generated inputs in the Signal Editor source. Expand the Iterations section to see the iterations corresponding to the signal scenarios.



See Also

sltest.import.sldvData

More About

- “Generate Tests and Test Harnesses for a Model or Components” on page 6-20

Process Test Results with Custom Scripts

In this section...

“MATLAB Testing Framework” on page 6-197
 “Define a Custom Criteria Script” on page 6-197
 “Reuse Custom Criteria and Debug Using Breakpoints” on page 6-198
 “Custom Criteria Programmatic Interface Example” on page 6-200

Testing your model often requires assessing conditions that ensure a test is valid, in addition to verifying model behavior. MATLAB Unit Test provides a framework for such assessments. In Simulink Test, you can use the test case custom criteria to author specific assessments, and include MATLAB Unit Test qualifications in your script.

Custom criteria apply as post-simulation criteria to the simulation output. See `Simulink.SimulationOutput`. If you require run-time verifications, use a `verify()` statement in a Test Assessment or Test Sequence block. See “Verify Model Simulation by Using when Decomposition” on page 3-14.

MATLAB Testing Framework

A custom criteria script is a method of `test`, which is a `matlab.unittest` test case object. To enable the function, in the test case **Custom Criteria** section of the Test Manager, select **function customCriteria(test)**. Inside the function, enter the custom criteria script in the embedded MATLAB editor.

The embedded MATLAB editor lists properties of `test`. Create test assessments using MATLAB Unit Test qualifications. Custom criteria supports verification and assertion type qualifications. See “Table of Verifications, Assertions, and Other Qualifications”. Verifications and assertions operate differently when custom criteria are evaluated:

- Verifications – Other assessments are evaluated when verifications fail. Diagnostics appear in the results. Use verifications for general assessments, such as checking simulation against expected outputs.

Example: `test.verifyEqual(lastValue,0)`

- Assertions – The custom criteria script stops evaluating when an assertion fails. Diagnostics appear in the results. Use assertions for conditions that render the criteria invalid.

Example: `test.assertEqual(lastValue,0).`

Define a Custom Criteria Script

This example shows how to create a custom criteria script for an autopilot test case.

1. Open the Test File

```
sltest.testmanager.load('AutopilotTestFile.mldatx');
sltest.testmanager.view
```

2. In the **Test Browser**, select **AutopilotTestFile > Basic Design Test Cases > Requirement 1.3 Test**.

3. In the test case, expand the **Custom Criteria** section.

4. Enable the custom criteria script by selecting function `customCriteria(test)`.

5. In the embedded MATLAB editor, add the following script, which gets and verifies the final values of the signals `Phi` and `APEng`. One of the test requirements is `Signals_Req1_3` and `sltest_simout` is the simulation output.

```
% Get the last values
```

```
lastPhi =  
test.sltest_simout.get('Signals_Req1_3').get('Phi').Values.Data(end);
```

```
lastAPEng =  
test.sltest_simout.get('Signals_Req1_3').get('APEng').Values.Data(end);
```

```
% Verify the last values equal 0
```

```
test.verifyEqual(lastPhi,0,['Final Phi value: ',num2str(lastPhi),'.']);
```

```
test.verifyEqual(lastAPEng,false,['Final APEng value: ',  
'num2str(lastAPEng),'.']);
```

6. To see how the logged signals are tested for the `Signals_Req1_3` test requirement, open the `RollRefAssessLib.slx` model and then open the Test Assessment Req 1-3 Subsystem block.

7. Run the test case.

8. In the **Results and Artifacts** pane, expand the **Custom Criteria Result**. Both criteria pass.

▼ Requirement 1.3 Test	✖
▶ Verify Statements	✖
▶ Sim Output (RollAutopilotMdlRef :	
▼ Custom Criteria Result	✔
Final Phi value: 0. { verifyEqual	✔
Final APEng value: 0. { verifyEq	✔

9. Clear and close the Test Manager.

```
sltest.testmanager.clear  
sltest.testmanager.clearResults  
sltest.testmanager.close
```

Reuse Custom Criteria and Debug Using Breakpoints

This example shows how to author custom criteria in a standalone function and add a breakpoint to that function. The custom criteria function `sltestCheckFinalRollRefValues.m` is provided as the starting point for this example.

Using a standalone function lets you:

- Reuse the custom criteria in multiple test cases.
- Set breakpoints in the criteria script for debugging.
- Investigate the simulation output using the command line.

1. Open the function file in the MATLAB Live Editor, and open the test file in the Test Manager

```
open('sltestCheckFinalRollRefValues.m')
```

```
sltest.testmanager.load('AutopilotTestFile.mldatx');
sltest.testmanager.view
```

2. In the **Test Browser** pane of the Test Manager, select **AutopilotTestFile > Basic Design Test Cases > Requirement 1.3 Test**.

3. Expand the **Custom Criteria** section, and enable the custom criteria script by selecting function `customCriteria(test)`. Then, in the embedded MATLAB editor, replace the content with the function call to the custom criteria:

```
sltestCheckFinalRollRefValues(test)
```

4. In `sltestCheckFinalRollRefValues.m` file in the MATLAB Live Editor, set a breakpoint by clicking on the 8 of line number 8.

5. In the Test Manager, run the test case. When the breakpoint is encountered, the test case pauses running, the command window displays the line where the breakpoint occurred, and the command line prompt changes to `K>>`, which indicates debugging mode.

6. Enter `test` at the command prompt to display the properties of the `STMCustomCriteria` object. The properties contain characteristics and simulation data output of the test case.

```
test =

  STMCustomCriteria with properties:

        TestResult: [1x1 sltest.testmanager.TestCaseResult]
      sltest_simout: [1x1 Simulink.SimulationOutput]
    sltest_testCase: [1x1 sltest.testmanager.TestCase]
      sltest_bdroot: {'RollReference_Requirement1_3'}
        sltest_sut: {'RollAutopilotMdlRef/Roll Reference'}
    sltest_isharness: 1
    sltest_iterationName: ''
```

7. The property `sltest_simout` contains the simulation data. To view the data `PhiRef`, at the command line enter

```
test.sltest_simout.get('Signals_Req1_3').get('PhiRef')
```

```
ans =

Simulink.SimulationData.Signal
Package: Simulink.SimulationData

Properties:
    Name: 'PhiRef'
    PropagatedName: ''
    BlockPath: [1x1 Simulink.SimulationData.BlockPath]
    PortType: 'outport'
    PortIndex: 1
    Values: [1x1 timeseries]
```

8. In the `sltestCheckFinalRollRefValues.m` file in the MATLAB Live Editor, click **Continue** to continue running the custom criteria script.

9. In the **Results and Artifacts** pane, expand the **Custom Criteria Result**. Both criteria pass.

▼ Requirement 1.3 Test	✖
▶ Verify Statements	✖
▶ Sim Output (RollAutopilotMdlRef :	
▼ Custom Criteria Result	✔
☒ Final Phi value: 0. { verifyEqual	✔
☒ Final APEng value: 0. { verifyEq	✔

To reuse the script in another test case, call the same function from the **Custom Criteria** of that test case.

Custom Criteria Programmatic Interface Example

This example shows how to set and get custom criteria using the programmatic interface.

Before running this example, temporarily disable warnings that result from verification failures.

```
warning off Stateflow:Runtime:TestVerificationFailed;
warning off Stateflow:cdr:VerifyDangerousComparison;
```

Load a Test File and Get Test Case Object

```
tf = sltest.testmanager.load('AutopilotTestFile.mldatx');
ts = getTestSuiteByName(tf, 'Basic Design Test Cases');
tc = getTestCaseByName(ts, 'Requirement 1.3 Test');
```


Create the Custom Criteria Object and Set Criteria

Create the custom criteria object.

```
tcCriteria = getCustomCriteria(tc)
```

```
tcCriteria =
  CustomCriteria with properties:
    Enabled: 0
    Callback: '↵'
```

Create the custom criteria expression. This script gets the last value of the signal Phi and verifies that it equals 0.

```
criteria = ...
  sprintf(['lastPhi = test.sltest_simout.get('Signals_Req1_3')',...
    '.get('Phi').Values.Data(end);\n',...
    'test.verifyEqual(lastPhi,0,['Final: ',num2str(lastPhi),'.'])']);])

criteria =
  'lastPhi = test.sltest_simout.get('Signals_Req1_3').get('Phi').Values.Data(end);
  test.verifyEqual(lastPhi,0,['Final: ',num2str(lastPhi),'.'])';
```

Set and enable the criteria.

```
tcCriteria.Callback = criteria;
tcCriteria.Enabled = true;
```

Run the Test Case and Get the Results

Run the test case.

```
tcResultSet = run(tc);
```

Get the test case results.

```
tcResult = getTestCaseResults(tcResultSet);
```

Get the custom criteria result.

```
ccResult = getCustomCriteriaResult(tcResult)
```

```
ccResult =
  CustomCriteriaResult with properties:
    Outcome: Failed
    DiagnosticRecord: [1x1 sltest.testmanager.DiagnosticRecord]
```

Restore warnings from verification failures.

```
warning on Stateflow:Runtime:TestVerificationFailed;
warning on Stateflow:cdr:VerifyDangerousComparison;
```

```
sltest.testmanager.clearResults  
sltest.testmanager.clear  
sltest.testmanager.close
```

See Also

Related Examples

- “Test Models Using MATLAB Unit Test” on page 6-209
- “Create, Store, and Open MATLAB Figures” on page 6-206

Assess the Damping Ratio of a Flutter Suppression System

This example shows how to use a custom criteria script to verify that wing oscillations are damped in multiple altitude and airspeed conditions.

The Test and Model

The model uses Simscape™ to simulate a Benchmark Active Controls Technology (BACT) / Pitch and Plunge Apparatus (PAPA) setup. It uses Aerospace Blockset™ to simulate aerodynamic forces on the wing.

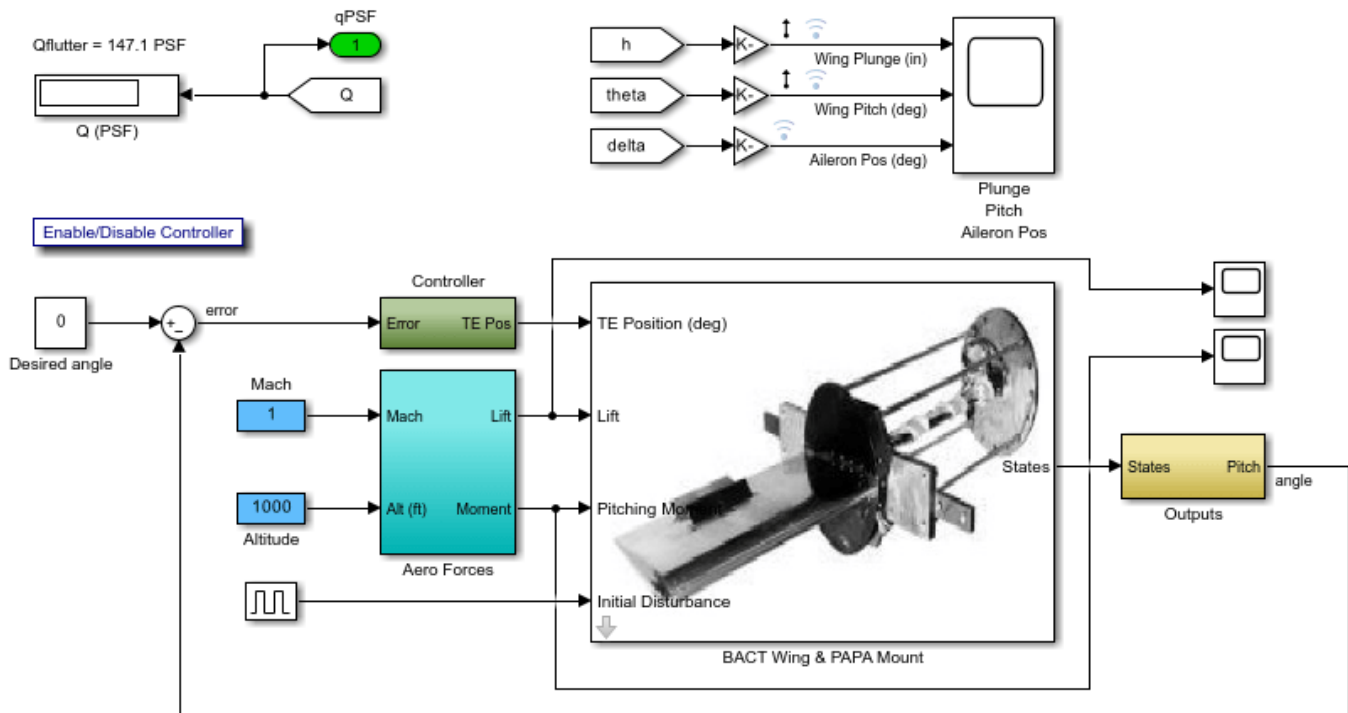
The test iterates over 16 combinations of Mach and Altitude. The test case uses custom criteria with Curve Fitting Toolbox™ to find the peaks of the wing pitch, and determine the damping ratio. If the damping ratio is not greater than zero, the assessment fails.

Open the test file in the Test Manager.

```
open('sltestFlutterCriteriaTest.mldatx')
```

In the **Test Browser** pane of the Test Manager, select **Altitude and mach iterations**. Open the model by clicking the arrow next to **Model** in the **System Under Test** section.

```
open_system('sltestFlutterSuppressionSystemExample.slx')
```



Copyright 2003-2021 The MathWorks, Inc.

Custom Criteria Script

The test case custom criteria uses this script to verify that the damping ratio is greater than zero.

```
% Get time and data for pitch
Time = test.sltest_simout.get('sigsOut').get('pitch').Values.Time(1:15000);
Data = test.sltest_simout.get('sigsOut').get('pitch').Values.Data(1:15000);

% Find peaks
[~, peakIds] = findpeaks(Data,'minpeakheight', 0.002, 'minpeakdistance', 50);
peakTime= Time(peakIds);
peakPos = Data(peakIds);
rn = peakPos(1)./peakPos(2:end);
L = 1:length(rn);

% Do curve fitting
fittedModel = exponentialFitAndPlot(L, rn);
delta = fittedModel.d;

% Find damping ratio
dRatio = delta/sqrt((2*pi)^2+delta^2);

% Make sure damping ratio is greater than 0
test.verifyGreaterThan(dRatio,0,'Damping ratio must be greater than 0');
```

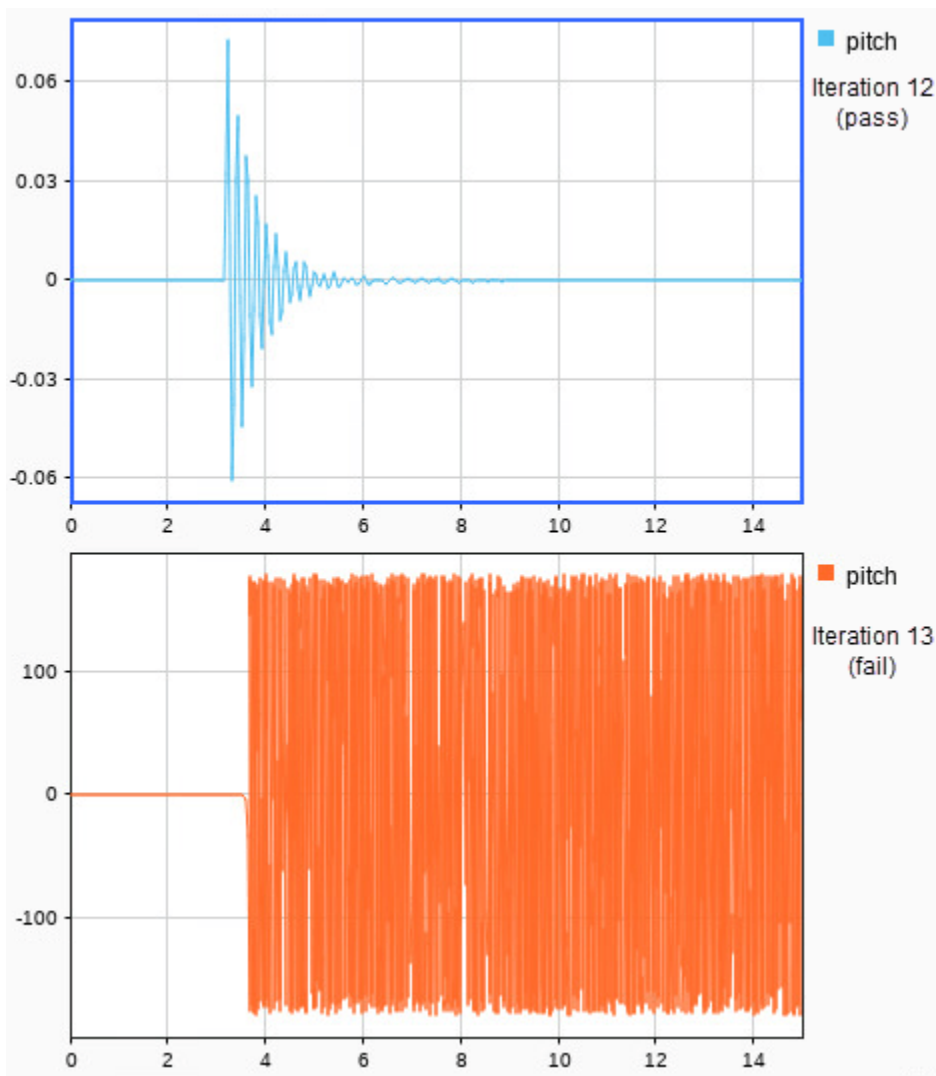
Test Results

Running the test case returns two conditions in which the damping ratio is greater than zero.

```
results = sltest.testmanager.run;
```

▼ I Scripted_Iteration13	✖
▶ 📊 Sim Output (sltestFlutterSuppressionSystemExa	
▶ 📄 Custom Criteria Result	✖
▼ I Scripted_Iteration14	✖
▶ 📊 Sim Output (sltestFlutterSuppressionSystemExa	
▶ 📄 Custom Criteria Result	✖
▶ I Scripted_Iteration15	✔

The wing pitch plots from iteration 12 and 13 show the difference between a positive damping ratio (iteration 12) and a negative damping ratio (iteration 13).



```
sltest.testmanager.clearResults  
sltest.testmanager.clear  
sltest.testmanager.close  
close_system('sltestFlutterSuppressionSystemExample.slx',0)
```

See Also

Related Examples

- “Test Iterations” on page 6-139
- “Process Test Results with Custom Scripts” on page 6-197

Create, Store, and Open MATLAB Figures

In this section...

"Create a Custom Figure for a Test Case" on page 6-206

"Include Figures in a Report" on page 6-207

You can create figures using MATLAB commands to include with test results and reports. Enter the commands in a test case section that accepts MATLAB code. These sections include the test case **Custom Criteria** section, and callbacks that can execute with your test case.

If you include code that creates figures with your test case, you can:

- Display the figures after the test runs
- Store the figures with your test case
- Include them in a report
- Access stored figures from your test results

To specify this behavior, use the **Test File Options** section under the **Test File** settings.

- Select **Close all open figures at the end of execution** if you do not need to see the figures right after the test executes, for example, if you are storing the figures or including them in a report. Clear this check box if you are not storing the figures and you want to view them after the test executes.
- Select **Store MATLAB figures** if you want to save the figures with the test results. This option also enables you to open the figures from the results and to include them in a report.

After you run the test, the figures appear under **MATLAB Figures** in the test case results.

Create a Custom Figure for a Test Case

In this example, add code that creates a figure to the **Custom Criteria** section of a test case. To access the figure from the test results, set options on the test file.

- 1 Open the model to test using `openExample('sldemo_absbrake')`.
- 2 In the Test Manager, create a test file and name it `custom_figures`.
- 3 In the default test case, under **System Under Test**, set the model to `sldemo_absbrake`.
- 4 Under **Custom Criteria**, select the **function customCriteria(test)** check box and paste this code in the text box.

```
h = findobj(0,'Name','ABS Speeds and Slip');
if isempty(h)
    h=figure('Position',[26 100 452 700],...
            'Name','ABS Speeds and Slip',...
            'NumberTitle','off');
end
figure(h)
set(h,'DefaultAxesFontSize',8)

% Log data object and store in sldemo_absbrake_output variable
out = test.sltest_simout.get('sldemo_absbrake_output');

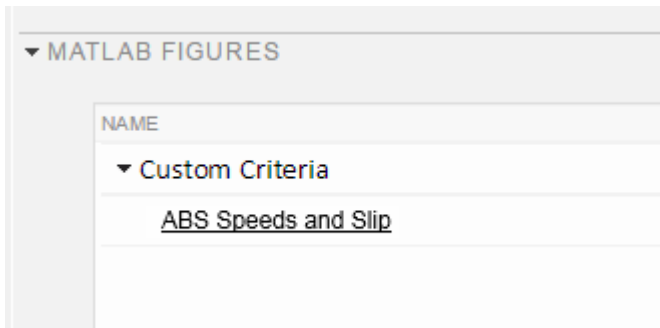
% Plot wheel speed and car speed
```

```

subplot(3,1,1);
plot(out.get('yout').Values.Vs.Time, ...
      out.get('yout').Values.Vs.Data);
grid on;
title('Vehicle speed'); ylabel('Speed(rad/sec)'); xlabel('Time(sec)');
subplot(3,1,2);
plot(out.get('yout').Values.Ww.Time, ...
      out.get('yout').Values.Ww.Data);
grid on;
title('Wheel speed'); ylabel('Speed(rad/sec)'); xlabel('Time(sec)');
subplot(3,1,3);
plot(out.get('slp').Values.Time, ...
      out.get('slp').Values.Data);
grid on;
title('Slip'); xlabel('Time(sec)'); ylabel('Normalized Relative Slip');

```

- 5 Set the figure options for the test file custom_figures. Under **Test File Options**:
 - Select **Close all open figures at the end of execution**. This option closes figures created by your Test Manager MATLAB code.
 - Select **Store MATLAB figures**.
- 6 With the test case or the test file selected, click **Run**.
- 7 In the **Results and Artifacts** pane, select the test case under the results for this test run. Click the links under **MATLAB Figures** to see the plots generated when the test ran. The plot generated by the code you entered appears under **Custom Criteria**.



Include Figures in a Report

You can select the **MATLAB Figures** option in the Create Test Results Report dialog box to include custom figures in your report. Alternatively, you can set report options under **Test File Options**. The **Test File Options** settings are saved with the test file.

- 1 Select the test file custom_figures.
- 2 Under **Test File Options**, select **Generate report after execution**. The section expands, displaying the same report options you can set using the dialog box.
- 3 To see the figures regardless of how the tests performed, set **Results for** to All Tests.
- 4 Select the **MATLAB figures** check box.
- 5 With the test file selected, run the test. Running the test generates the report and opens it in the PDF viewer.
- 6 Examine the report. The plot generated by the code you entered under **Custom Criteria** appears in the report section **Custom Criteria Plots**.

See Also

`sltest.testmanager.Options | getOptions (TestSuite) | getOptions (TestCase) |
getOptions (TestFile)`

Related Examples

- “Export Test Results” on page 7-16

Test Models Using MATLAB Unit Test

In this section...

“Overall Workflow” on page 6-209

“Considerations” on page 6-209

“Comparison of Test Nomenclature” on page 6-209

“Basic Workflow Using MATLAB® Unit Test” on page 6-210

You can use the MATLAB Unit Test framework to run tests authored in Simulink Test. Using the framework:

- Allows you to execute model tests together with Unit Test scripts, functions, and classes.
- Enables model and code testing using the same framework.
- Enables integration with continuous integration (CI) systems, such as Jenkins®.

Overall Workflow

To run tests with the Unit Test framework:

- 1 Create a `TestSuite` from the Simulink Test file.
- 2 Create a `TestRunner`.
- 3 Create plugin objects to customize the `TestRunner`. For example:
 - The `matlab.unittest.plugins.TAPPlugin` produces a results stream according to the Test Anything Protocol for use with certain CI systems.
 - The `sltest.plugins.ModelCoveragePlugin` specifies model coverage collection and lets you return coverage results at to the command line. If you set up coverage in the Test Manager, you do not need to use this plugin.
- 4 Add the plugins to the `TestRunner`.
- 5 Run the test using the `run` method, or run tests in parallel using the `runInParallel` method.

Considerations

When running tests using the Unit Test framework, consider the following:

- If you disable a test in the Test Manager, the test is filtered using Unit Test, and the result reflects a failed assumption.

Comparison of Test Nomenclature

The Unit Test framework has analogous properties to the functionality in Simulink Test. For example,

- If the test case contains iterations, the unit test contains parameterizations.
- If the test file or test suite contains callbacks, the unit test contains one or more callbacks fixtures.

Test Case Iterations and Unit Test Parameterizations

Parameterization details correspond to properties of the iteration.

Simulink Test	MATLAB Unit Test
Iteration type: Scripted	Parameterization property: ScriptedIteration
Iteration type: Table	Parameterization property: TableIteration
Iteration name	Parameterization Name
Test case iteration object	Parameterization Value

Test Callbacks and Unit Test Fixtures

Fixtures depend on callbacks contained in the test file. Fixtures do not include test case callbacks, which are executed with the test case itself.

Callbacks in Simulink Test	Fixtures in MATLAB Unit Test
Test file callbacks	FileCallbacksFixture
Test suite callbacks	SuiteCallbacksFixture
File and suite callbacks	Heterogeneous CallbacksFixture, containing FileCallbacksFixture and SuiteCallbacksFixture
No callbacks	No fixture

Basic Workflow Using MATLAB® Unit Test

This example shows how to create and run a basic MATLAB® Unit Test for a test file created in Simulink® Test™. You create a test suite, run the test, and display the diagnostic report.

Before running this example, temporarily disable warnings that result from verification failures.

```
warning off Stateflow:Runtime:TestVerificationFailed;
warning off Stateflow:cdr:VerifyDangerousComparison;
```

1. Author a test file in the Test Manager, or start with a preexisting test file. For this example, `AutopilotTestFile` tests a component of an autopilot system against several requirements, using `verify` statements.

2. Create a `TestSuite` from the test file.

```
apsuite = testsuite('AutopilotTestFile.mldatx');
```

3. Create the test runner.

```
import matlab.unittest.TestRunner
aprunner = TestRunner.withNoPlugins;
```

4. Add the plugin to produce Test Manager results.

```
import sltest.plugins.TestManagerResultsPlugin
tmr = TestManagerResultsPlugin;
addPlugin(aprunner,tmr)
```

5. Run the test.

```
apresults = run(aprunner,apsuite);
```

6. View the summary of the test, which shows that the test failed because of a verification failure.

```
apresults.Details.SimulinkTestManagerResults
```

```
ans =
```

```
  TestResultContainer with properties:
```

```
    TestResult: [1x1 sltest.testmanager.TestCaseResult]
```

Enable warnings.

```
warning on Stateflow:Runtime:TestVerificationFailed;
```

```
warning on Stateflow:cd:VerifyDangerousComparison;
```

See Also

Test | TestResult | TestRunner | TestSuite | matlab.unittest.plugins Package

Related Examples

- “Output Results for Continuous Integration Systems” on page 6-212
- “Run Tests for Various Workflows”

Output Results for Continuous Integration Systems

In this section...
"Test a Model for Continuous Integration Systems" on page 6-212
"Model Coverage Results for Continuous Integration" on page 6-214

You can create model tests that are compatible with continuous integration (CI) systems such as Jenkins. To create CI-compatible results, run your Simulink Test files using MATLAB Unit Test.

To run CI-compatible tests, follow this general procedure:

- 1 Create a test suite from the MLDATX test file.
- 2 Create a test runner.
- 3 Create plugins for the test output or coverage results.
 - For test outputs, use the `TAPPlugin` or `XMLPlugin`.
 - For model coverage, use the `ModelCoveragePlugin` and `CoberturaFormat`. When collecting model coverage in Cobertura format:
 - Only top model coverage is reflected in the Cobertura XML.
 - Only model Decision coverage is reflected, and it is mapped to Condition elements in Cobertura XML.
- 4 Create plugins for CI-compatible output.
- 5 Add the plugins to the test output or coverage results.
- 6 Add the test output plugins or coverage result plugins to the test runner.
- 7 Run the test.

Test a Model for Continuous Integration Systems

This example shows how to test a model, publish Test Manager results, and output results in TAP format with a single execution.

You use MATLAB® Unit Test to create a test suite and a test runner, and customize the runner with these plugins:

- `matlab.unittest.plugins.TestReportPlugin` produces a MATLAB Test Report.
- `sltest.plugins.TestManagerResultsPlugin` adds Test Manager results to the MATLAB Test Report.
- `matlab.unittest.plugins.TAPPlugin` outputs results to a TAP file.

The test case creates a square wave input to a controller subsystem and sweeps through 25 iterations of parameters `a` and `b`. The test compares the `alpha` output to a baseline with a tolerance of `0.0046`. The test fails on those iterations in which the output exceeds this tolerance.

1. Open the Simulink® Test™ test file.

```
testfile = fullfile('f14ParameterSweepTest.mldatx');
sltest.testmanager.view;
sltest.testmanager.load(testfile);
```

2. In the Test Manager, configure the test file for reporting.

Under **Test File Options**, select **Generate report after execution**. The section expands, displaying several report options. For more information, see “Save Reporting Options with a Test File” on page 7-17.

3. Create a test suite from the Simulink® Test™ test file.

```
import matlab.unittest.TestSuite

suite = testsuite('f14ParameterSweepTest.mldatx');
```

4. Create a test runner.

```
import matlab.unittest.TestRunner

f14runner = TestRunner.withNoPlugins;
```

5. Add the TestReportPlugin to the test runner.

The plugin produces a MATLAB Test Report F14Report.pdf.

```
import matlab.unittest.plugins.TestReportPlugin

pdfFile = 'F14Report.pdf';
trp = TestReportPlugin.producingPDF(pdfFile);
addPlugin(f14runner,trp)
```

6. Add the TestManagerResultsPlugin to the test runner.

The plugin adds Test Manager results to the MATLAB Test Report.

```
import sltest.plugins.TestManagerResultsPlugin

tmr = TestManagerResultsPlugin;
addPlugin(f14runner,tmr)
```

7. Add the TAPPlugin to the test runner.

The plugin outputs to the F14Output.tap file.

```
import matlab.unittest.plugins.TAPPlugin
import matlab.automation.streams.ToFile

tapFile = 'F14Output.tap';
tap = TAPPlugin.producingVersion13(ToFile(tapFile));
addPlugin(f14runner,tap)
```

8. Run the test.

Several iterations fail, in which the signal-baseline difference exceeds the tolerance criteria.

```
result = run(f14runner,suite);
```

```
Generating test report. Please wait.
```

```
    Preparing content for the test report.
```

```
        Adding content to the test report.
```

```
        Writing test report to file.
```

```
Test report has been saved to:
```

```
    C:\TEMP\tp96ff8d4f\simulinktest-ex40056435\F14Report.pdf
```

A single execution of the test runner produces two reports:

- A MATLAB Test Report that contains Test Manager results.
- A TAP format file that you can use with CI systems.

```
sltest.testmanager.clearResults
sltest.testmanager.clear
sltest.testmanager.close
```

Model Coverage Results for Continuous Integration

This example shows how to generate model coverage results for use with continuous integration. Coverage is reported in the Cobertura format. You run a Simulink® Test™ test file using MATLAB® Unit Test.

1. Import classes and create a test suite from the test file `AutopilotTestFile.mldatx`.

```
import matlab.unittest.TestRunner

aptest = sltest.testmanager.TestFile('AutopilotTestFile.mldatx');
apsuite = testsuite(aptest.FilePath);
```

2. Create a test runner.

```
trun = TestRunner.withNoPlugins;
```

3. Set the coverage metrics to collect. This example uses decision coverage. In the Cobertura output, decision coverage is listed as condition elements.

```
import sltest.plugins.coverage.CoverageMetrics

cmet = CoverageMetrics('Decision',true);
```

4. Set the coverage report properties. This example produces a file `R13Coverage.xml` in the current working folder. Ensure your working folder has write permissions.

```
import sltest.plugins.coverage.ModelCoverageReport
import matlab.unittest.plugins.codecoverage.CoberturaFormat

rptfile = 'R13Coverage.xml';
rpt = CoberturaFormat(rptfile)

rpt =
    CoberturaFormat with no properties.
```

5. Create a model coverage plugin. The plugin collects the coverage metrics and produces the Cobertura format report.

```
import sltest.plugins.ModelCoveragePlugin

mcp = ModelCoveragePlugin('Collecting',cmet,'Producing',rpt)

mcp =
    ModelCoveragePlugin with properties:
```

```
RecordModelReferenceCoverage: '<default>'
    MetricsSettings: [1x1 sltest.plugins.coverage.CoverageMetrics]
    ScopeToRequirements: 0
```

6. Add the coverage plugin to the test runner.

```
addPlugin(trun,mcp)
```

```
% Turn off command line warnings:
warning off Stateflow:cdr:VerifyDangerousComparison
warning off Stateflow:Runtime:TestVerificationFailed
```

7. Run the test.

```
APResult = run(trun,apsuite)
```

```
APResult =
    TResult with properties:
        Name: 'AutopilotTestFile > Basic Design Test Cases/Requirement 1.3 Test'
        Passed: 0
        Failed: 1
        Incomplete: 0
        Duration: 3.4542
        Details: [1x1 struct]

Totals:
    0 Passed, 1 Failed, 0 Incomplete.
    3.4542 seconds testing time.
```

8. Reenable warnings.

```
warning on Stateflow:cdr:VerifyDangerousComparison
warning on Stateflow:Runtime:TestVerificationFailed
```

See Also

TestRunner | TestSuite | sltest.plugins.ModelCoveragePlugin |
 sltest.plugins.TestManagerResultsPlugin |
 matlab.unittest.plugins.TestReportPlugin | matlab.unittest.plugins.TAPPlugin

More About

- “Test Models Using MATLAB Unit Test” on page 6-209

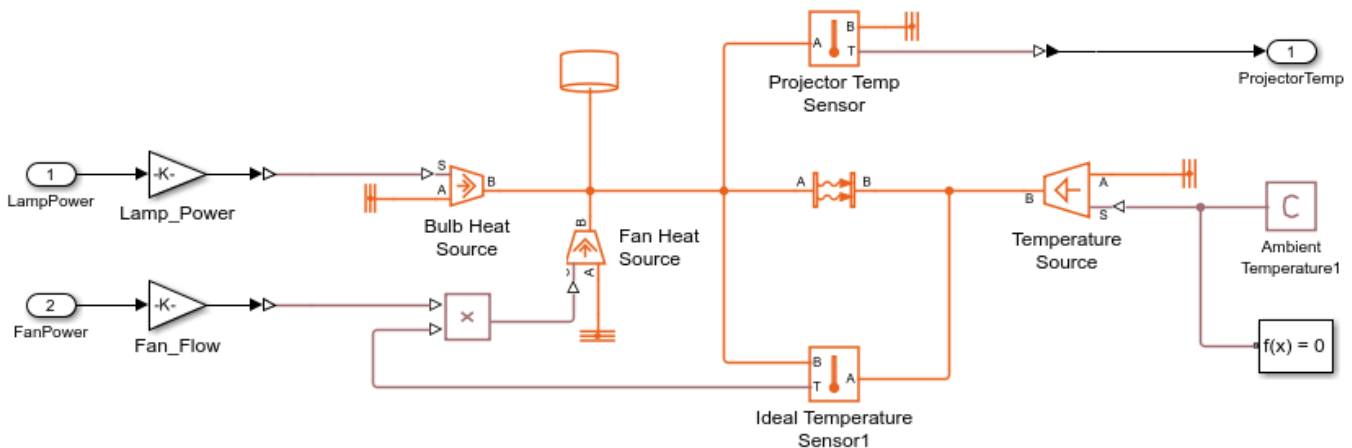
Parametric Sweep for a Simscape Thermal Model

This example shows how to test a physical system, and how to optimize a parameter using a test harness, test sequence, and the Test Manager. The example uses a system-level thermal model of a projector which includes Simscape® thermal blocks.

Set Up Variables

Set the required variables for the example.

```
Model = 'sltestProjectorFanSpeedExample';
Harness = 'FanSpeedTestHarness';
TestSuite = 'sltestProjectorFanSpeedTestSuite.mldatx';
open_system(Model);
```



Copyright 2015-2020 The MathWorks, Inc.

Test Plan and System Requirements

This test demonstrates sweeping through several fan speeds to determine the optimal value. In short, the optimal fan speed results in the fastest response without damaging the system. In detail, the optimal fan speed:

- Prevents the system from exceeding the specified maximum temperature.
- Minimizes the time for the system to reach the temperature at which the lamp emits visible light.

The document `sltestProjectorFanSpeedExampleRequirements.slreqx` captures these detailed requirements and the test procedure.

Test-specific model items reside in the test harness, keeping main models free of unnecessary blocks, suitable for code generation, and suitable for integration with other models.

Open the Test File

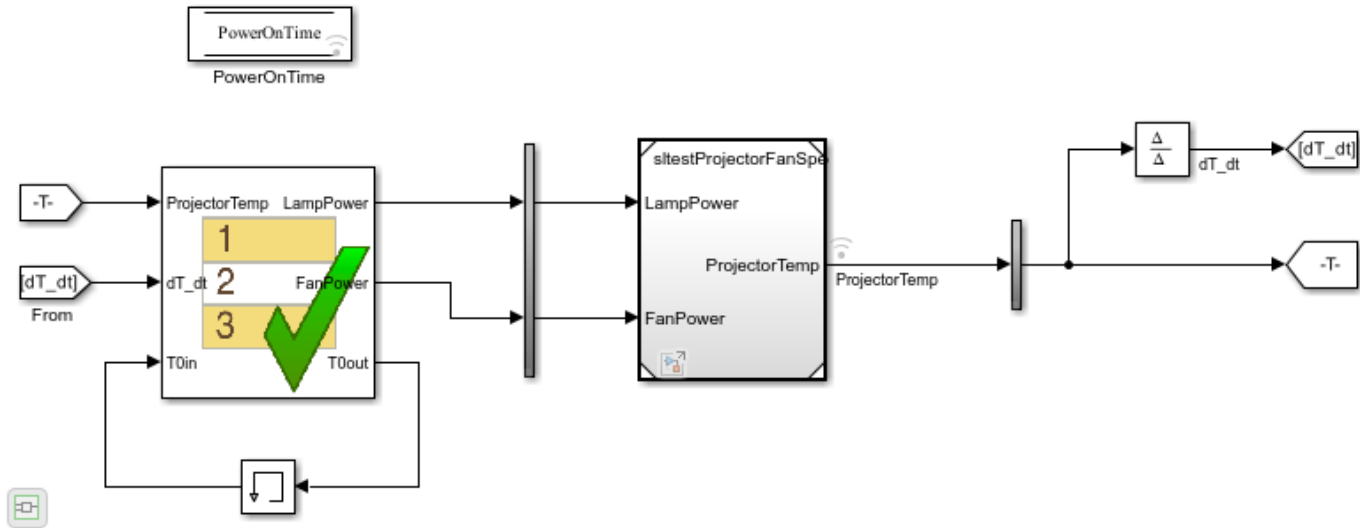
Open the Test Manager to view the test suite controlling the parameter sweep.

```
open(TestSuite)
```


Description of the Test

The test investigates the transient and steady-state thermal characteristics of the system. The test sequence initializes the system to ambient temperature, then powers the projector lamp. When the system reaches a steady-state condition, the lamp switches off. This test is modeled in a test harness using a Test Sequence block. Run the following to open the test harness:

```
sltest.harness.open(Model,Harness);
```



Requirements Linking

The test suite contains links to the requirements document. You can view the requirements links by selecting the **Fan Speed Parametric Study** test suite in the Test Browser pane of the Test Manager. Then, expand the **Requirements** section.

The Test Sequence

Double-click the Test Sequence block to open the test sequence editor.

Step	Transition	Next Step
InitializeSysToAmbient LampPower = 0; FanPower = 1; T0out = latch (ProjectorTemp); % latch captures the temp at the % start of the step	1. duration (abs(dT_dt/(ProjectorTemp-T0in)) < Threshold) >= DurationLimit % Transitions when the ratio of the temperature change to the initial temp % difference is less than a threshold (set to 0.1%) for a certain time. % Essentially seeks to transition when system reaches steady-state.	SystemHeatingTest ▼
SystemHeatingTest FanPower = latch (FanPower); LampPower = 1; T0out = latch (ProjectorTemp); PowerOnTime = latch (t);	1. duration (abs(dT_dt/(ProjectorTemp-T0in)) < Threshold) >= DurationLimit	SystemCoolingTest ▼
SystemCoolingTest FanPower = latch (FanPower); LampPower = 0; T0out = latch (ProjectorTemp);	1. duration (abs(dT_dt/(ProjectorTemp-T0in)) < Threshold) >= DurationLimit	Stop ▼
Stop LampPower = 0; FanPower = 0;		

The T0out and T0in signals store the initial projector temperature at each test step.

PowerOnTime stores simulation time when the lamp signal activates. This facilitates subsequent data analysis.

The transition condition detects the steady-state condition. At steady-state, the system temperature change is a small fraction (Threshold) of the difference between the current projector temperature and the initial projector temperature at each step. This condition must hold for a minimum time DurationLimit, in this case 10 seconds.

You may link the steps in the test sequence blocks against prepopulated requirements in the requirements document `sltestProjectorFanSpeedExampleRequirements.slreqx`.

Description of the Parameter Sweep

The pre-load callbacks contain the command to set the fan speed for each test case under the Fan Speed Parametric Study test suite. The parameter overrides contain the command to recalculate fan airflow from fan speed, and then override the test harness parameter. You can view these commands in the **Callbacks** and **Parameter Overrides** section of each test case.

Fan Speed = 1300

SIMULATION SETTINGS OVERRIDES

PARAMETER OVERRIDES

PARAMETER SET / WORKSPACE VARIABLE	OVERRIDE VALUE	SOURCE	MODEL ELEMENT
Parameter Set 1			
Fan_Flow	Fan_Speed*(Fan_Air_Displa...	base workspace	FanSpeedTestHarness/sltestProj...

+ Add

Refresh

Export

Delete

CALLBACKS

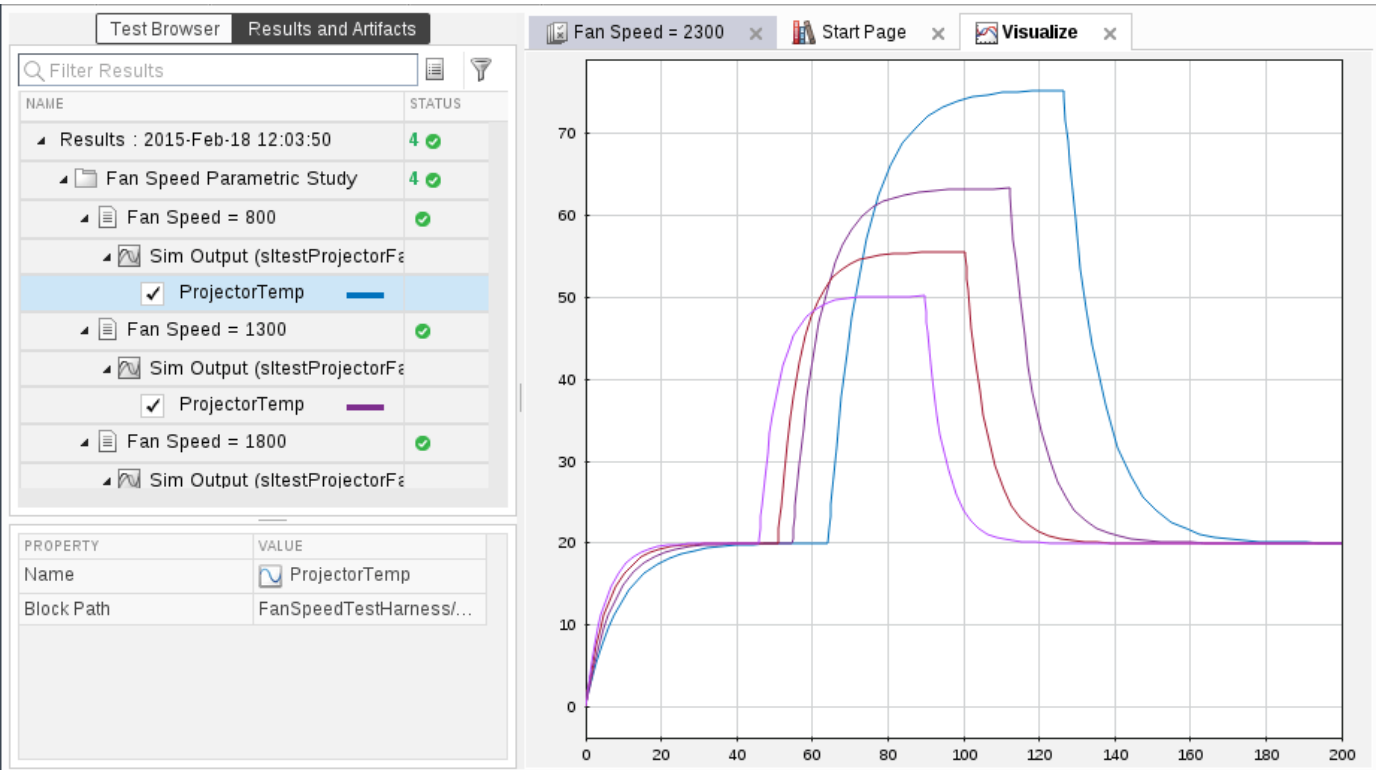
PRE-LOAD

1 % Runs before the model loads and any model callbacks

2 Fan_Speed = 1300;

Run the Test

In the **Test Browser**, highlight **Fan Speed Parametric Study** and click **Run**. When the test suite simulation completes, open the results for each test case and select **ProjectorTemp**. View the results in the Test Manager.



Export the Data

You can export data from the Test Manager for post-processing. To perform post-processing in MATLAB, in the **Results and Artifacts** pane of the Test Manager, right-click **Sim Output** for each test case and select **Export**.

This example includes the exported data in four MAT files:

```
ProjectorTempFanSpeed800.mat  
ProjectorTempFanSpeed1300.mat  
ProjectorTempFanSpeed1800.mat  
ProjectorTempFanSpeed2300.mat
```

Investigate Response Time and Maximum Projector Temperature

Since the test sequence transitions execute when the system reaches steady-state, and the fan speed changes the system response, the lamp activates at different simulation times for each of the four test cases. Simplify the graphical results analysis by plotting each response with the lamp activation at the same time.

Extract the lamp activation response data, and plot the system response for the four fan speeds. Evaluate the results against these criteria:

- The temperature shall not exceed 65 deg C.
- The lamp emits visible light above 45 deg C. Minimize the time to reach this temperature.

Load the results. At the command line, enter

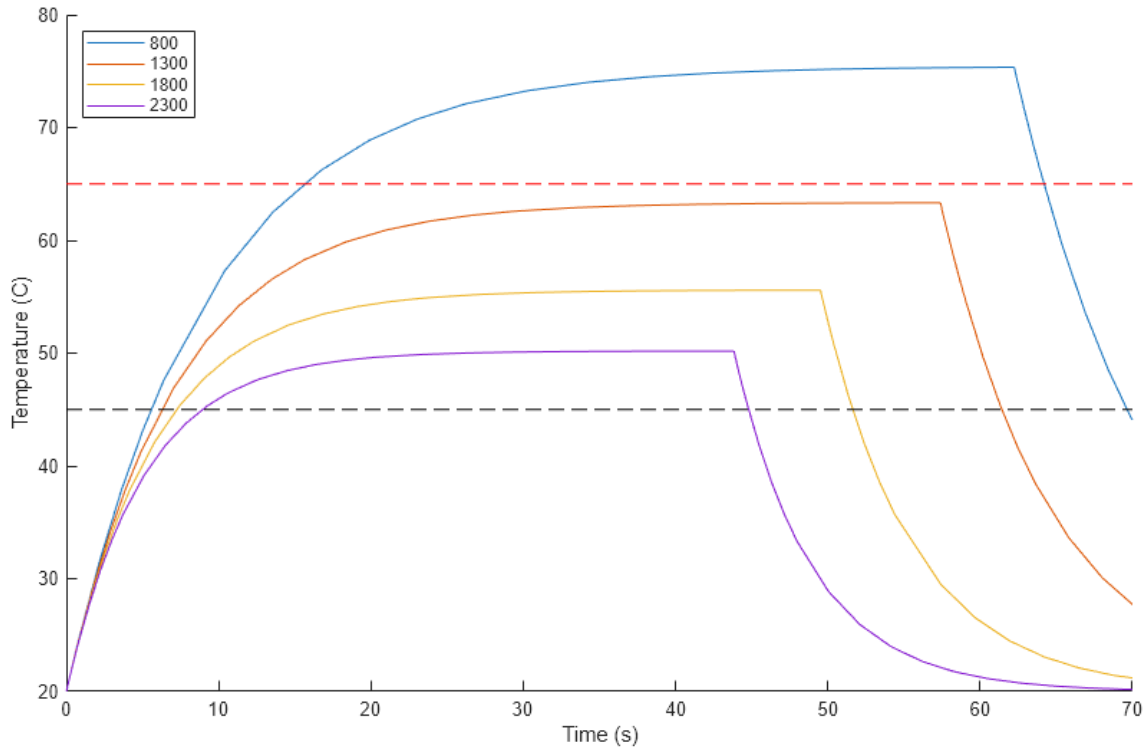
```
DataAt800 = load('ProjectorTempFanSpeed800.mat');  
DataAt1300 = load('ProjectorTempFanSpeed1300.mat');  
DataAt1800 = load('ProjectorTempFanSpeed1800.mat');  
DataAt2300 = load('ProjectorTempFanSpeed2300.mat');
```

The script `ArrangeProjectorData.m` arranges the temperature and power on data from the output for each run.

`ArrangeProjectorData`

The script `PlotProjectorThermalResponse.m` plots the thermal response of the projector after the lamp activates, for each of the fan speeds.

`PlotProjectorThermalResponse`



Results Interpretation

The results show that while the highest fan speed results in the lowest maximum temperature, it also takes the longest time to reach the lamp activation temperature. The lowest fan speed results in the fastest lamp activation, but the system exceeds the maximum specified temperature by a significant margin.

Fan speed = 1300 keeps the system under the maximum temperature spec, and the system also reaches lamp activation temperature approximately 3 seconds faster than with the highest fan speed.

```
close_system(Model,0);
```

```
clear Model;
clear Harness;
clear TestSuite;
close(figure(1));
```

See Also

Simulink Test Manager | Test Sequence

More About

- “Create or Import Test Harnesses and Select Properties” on page 2-13

Projector Controller Testing Using verify and Real-Time Tests

This example demonstrates testing a projector control system using model simulation and real-time execution on a target computer. The tests verify the controller by using test sequence scenarios that exercise the top-level controller model. The controller uses a push button input and a temperature sensor input, and outputs signals controlling the fan, fan speed, and projector lamp.

Set up the test file, model, and internal harness names for the example.

```
testFile = 'sltestProjectorCtrlTests.mldatx';
model = 'sltestProjectorController';
testharness = 'Test_Scenarios';
```

Open the model.

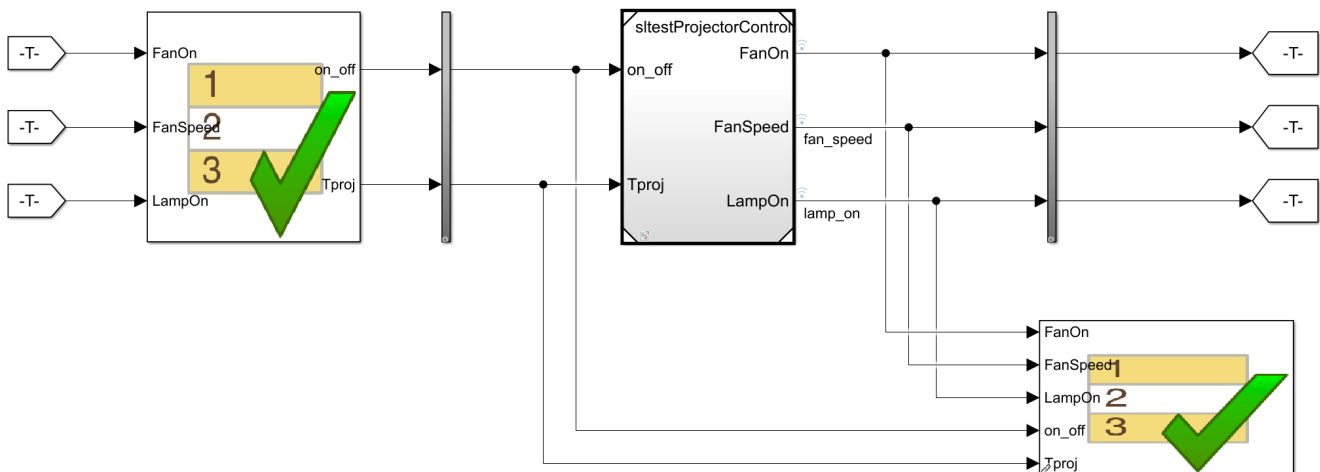
```
open_system(model)
```

View the Test Harness

Open the Test_Scenarios test harness.

```
sltest.harness.open(model, testharness);
```

The test harness uses a Test Sequence block to define the test scenarios and a Test Assessment block to verify the results.



In the test harness, open the Test Sequence block to view the scenarios, which are defined in tabs.

Nominal_Operation	On_Off_Button	Basic_Overheat	Overheat_Shutoff	+
Step	Transition			Next Step
initialize on_off = false; Tproj = single(0);	1. true			Normal_on_off ▼
Normal_on_off end_test = 0;				
On on_off = true;	1. FanOn == true			Wait ▼
Wait on_off = false; verify(FanOn == true,... 'Simulink:verify_scenario1',... 'Fan should be active');	1. after(20,sec)			Off ▼
Off on_off = true;	1. FanOn == false			End ▼
End on_off = false; end_test = 1;				

Open the Test Assessment block to view the verify statements.

Step	Transition	Next Step	Description
GlobalAssess <pre> verify(~LampOn == true FanOn == true,... 'Simulink:verify_lamp_implies_fan',... 'Fan must be on if lamp is on'); OverheatCondition when Tproj > 65 verify(LampOn == false &&... FanOn == true && FanSpeed == High,... 'Simulink:verify_overheat',... 'In overheat condition lamp must be off and fan high'); HighTempCondition when Tproj > 58 verify(FanOn == true && FanSpeed == High,... 'Simulink:verify_high_temp',... 'In high temp condition fan must be high'); NormalCondition when (Tproj < 58 && LampOn == true) verify(duration(FanSpeed == High) <= 10,... 'Simulink:verify_fan_normal',... 'In normal condition fan must return to normal within 10 s'); Else </pre>			The logical statement inside this verify statement is equivalent to LampOn -> FanOn. See requirement 1.4. This verifies that the fan is never at high for longer than 10 sec if the projector is at a normal temp. See requirement 1.8.

Open the Test File and Configure the Real-Time Target Computer

Open the test file in the Test Manager by entering:

```
open(testFile)
```

The test file contains a test suite with two test cases, each of which tests the four test scenarios. The **Simulation_Tests** test case simulates the model, and the **HIL_Tests** test case runs the tests on a real-time target computer.

Before running the example:

- 1 Configure your target computer using the Simulink Real-Time Explorer.
- 2 Connect to your target computer.
- 3 If your target computer is not the default target, update **Target Computer** in the **HIL_Tests** test case's **System Under Test** section.

For more information on real-time configuration see “System Configuration” (Simulink Real-Time).

Run the Model Simulation Tests

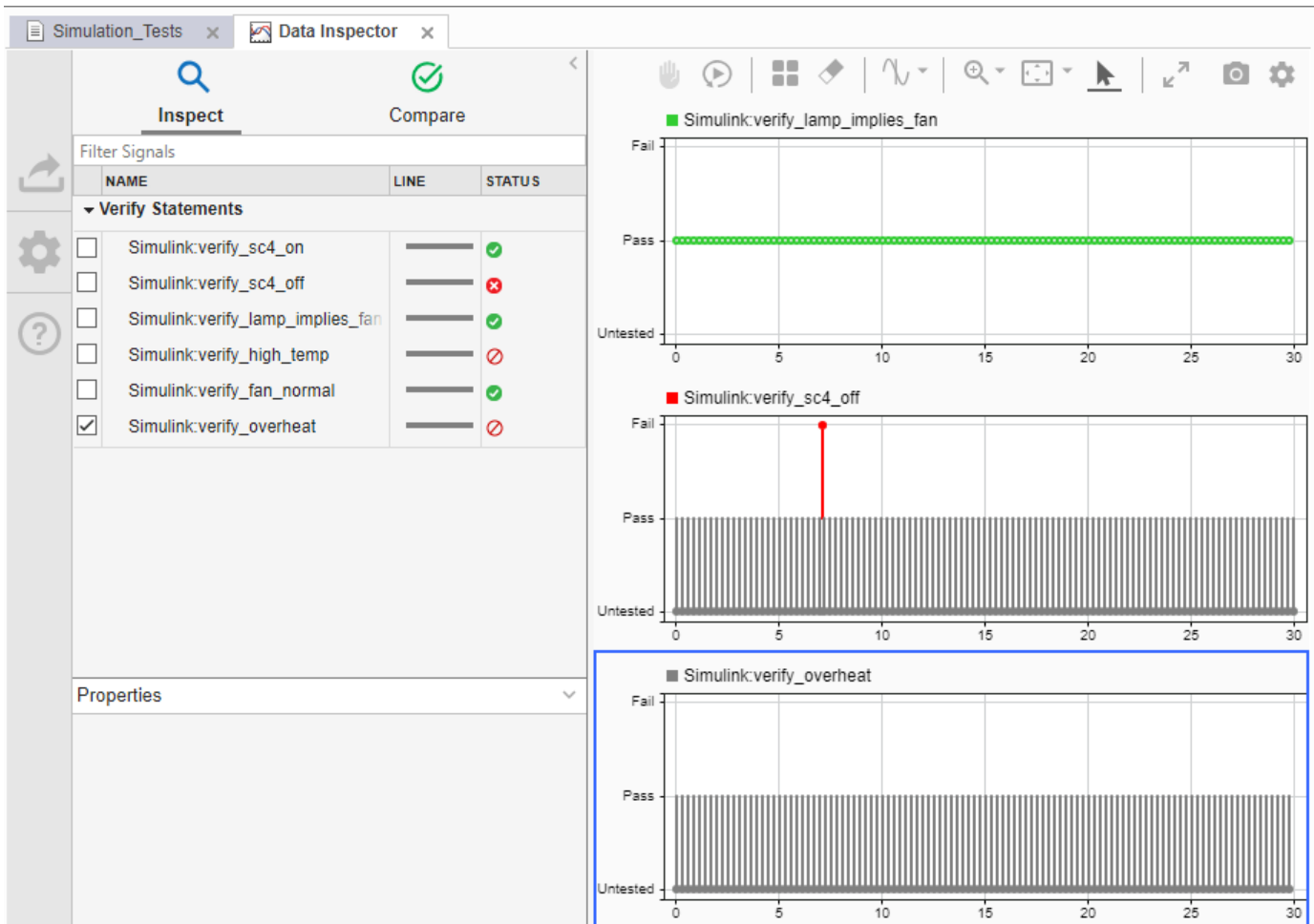
Run the **Simulation_Tests** test case. After simulation completes, click the **Results and Artifacts** pane in the Test Manager.

Expand the **Simulation_Tests** results and the four scenarios. Select one of the **Verify Statement** check boxes to view the results in the **Data Inspector** tab. The results in the **Verify Statements** section demonstrate fail, pass, and untested results:

- In all scenarios except **Basic_Overheat**, the controller does not operate in high-temperature or overheat mode, so the `verify_overheat` and `verify_high_temp` statements of the associated `verify` statements are untested.
- In all scenarios, the controller passes the test that if the lamp is on, the fan is also on: `verify_lamp_implies_fan`.
- In the **Overheat_Shutoff** scenario, the controller passes the test that the system stays off if the `on_off` button is pressed when the temperature is above a limit: `verify_sc4_on`. For other scenarios, `verify_sc4_on` is untested.
- In only the **Overheat_Shutoff** scenario, the controller fails the test that the system shuts off if the `on_off` button is pressed when the temperature is above a limit: `verify_sc4_off`. Resolving this failure requires modifying the `OnOff` Check subsystem in the main model.

For more information, see “Verify Model Simulation by Using when Decomposition” on page 3-14.

In the **Overheat_Shutoff** scenario, in the **Inspect** pane, select the `verify_sc4_off`, `verify_lamp_implies_fan`, and `verify_overheat` results.



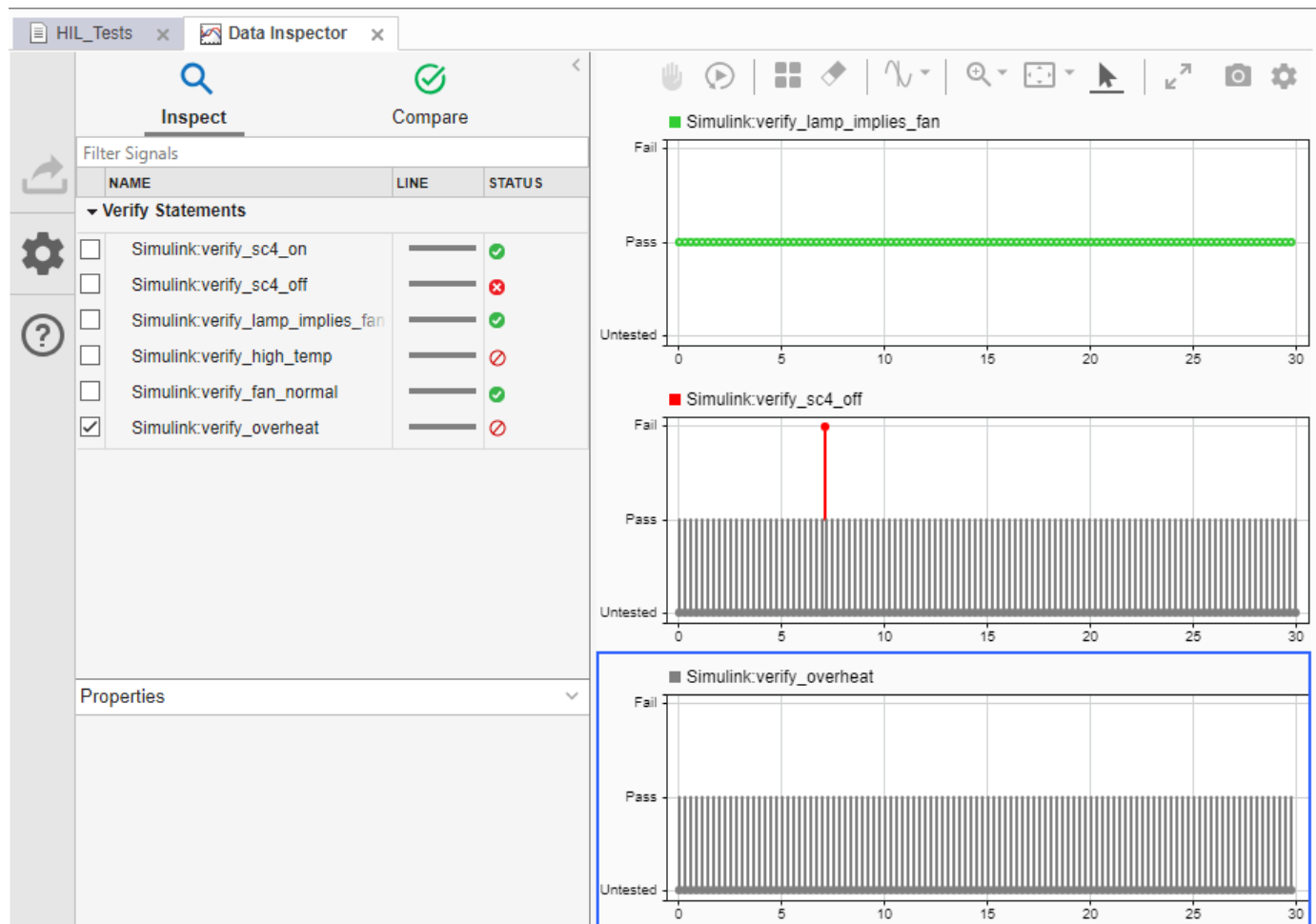
Execute the Real-Time Tests and Review the Results

The real-time test case (**HIL_Tests**) verifies that real-time execution results match model simulation results, and that the `verify` statements pass.

In the Test Manager, run the real-time test case (**HIL_Tests**).

The results of the `Simulation_Tests` and `HIL_Tests` show matching pass, fail, and untested statuses.

In the **Overheat Shutoff** scenario, in the **Inspect** pane, select **verify_sc4_off**, **verify_lamp_implies_fan**, and **verify_overheat** to visualize the results. The **Verify Statements** section shows similar results to the model simulation.



```
close_system(testharness,0)
close_system(model,0)
sltest.testmanager.clear;
sltest.testmanager.clearResults;
sltest.testmanager.close
clear testFile testHarness model;
```

See Also

Test Sequence | Test Assessment

Related Examples

- “Test Sequence Basics” on page 3-2
- “Test Sequence Editor” on page 3-29
- “Use Test Sequence Scenarios in the Test Sequence Editor and Test Manager” on page 3-54
- “Test Models in Real Time” on page 9-23

Test Execution Order

When you execute a test, Simulink Test opens the model to be tested, runs callback functions, closes the models, and cleans up. If you have scripted or table iterations, they run before the model is loaded. If your test uses callbacks, they run for each iteration. Simulink test evaluates custom criteria, temporal assessments, run comparisons, and equivalence checks after model simulation.

The order in which tests execute depends on:

- Whether you run a single test case or run a test suite containing one or more test cases
- The number of models tested
- The number of test cases

For serial simulations, the test cases run in the order they are listed in the Test Manager. To change the order in which test cases run in the Test Manager, drag and drop test cases into the desired order in the **Test Browser** pane. You cannot drag and drop test suites. If you run your simulations in parallel mode, the test cases might not run in the order displayed in the Test Manager.

If your test cases include callbacks, note that `disp` and `fprintf` do not work in callbacks. Instead, use the `ShowSimulationLogs` property of `sltest.testmanager.setpref`. To verify that the callbacks are executed, use a MATLAB script that includes breakpoints in the callbacks.

For test execution order for real-time tests, see “Test Models in Real Time” on page 9-23.

Single Test Case on a Single Model

If you select a specific test case to run on a single model and the model is not open before the test runs, the execution order is:

- 1 Run test case `Pre-Load` callback.
- 2 Run model `PreLoadFcn` callback.
- 3 Load model.
- 4 Run model `PostLoadFcn` callback.
- 5 Run test case `Post-Load` callback.
- 6 Simulate model.
- 7 Run test case `Cleanup` callback.
- 8 Run model `CloseFcn` callback.

If you run a test suite that contains a test case, the test suite `Setup` callback runs before the first step and the test suite `Cleanup` callback runs after the last step.

Multiple Test Cases on Multiple Models

If you run multiple test cases that run on separate models and the models are not open before the test runs, the execution order, which is shown for two test cases run on two models, is:

- 1 Run test case 1 `Pre-Load` callback.
- 2 Run model 1 `PreLoadFcn` callback.

- 3 Load model 1.
- 4 Run model 1 `PostLoadFcn` callback.
- 5 Run test case 1 `Post-Load` callback.
- 6 Simulate model 1 for test case 1.
- 7 Run test case 1 `Cleanup` callback.
- 8 Run test case 2 `Pre-Load` callback.
- 9 Run model 2 `PreLoadFcn` callback.
- 10 Load model 2.
- 11 Run model 2 `PostLoadFcn` callback.
- 12 Run test case 2 `Post-Load` callback.
- 13 Simulate model 2 for test case 2.
- 14 Run test case 2 `Cleanup` callback.
- 15 Run model 1 `CloseFcn` callback.
- 16 Run model 2 `CloseFcn` callback.

If you run a test suite that includes test cases, the test suite `Setup` callback runs before the first step and the test suite `Cleanup` callback runs after the last step.

The order in which models are closed using the `CloseFcn` might be different than the order in which they were opened or run. In the above example, steps 15 and 16 might be switched.

Multiple Test Cases in a Single Test Suite on a Single Model

If you run multiple test cases in a test suite on a single model and the model is not open before the test runs, the execution order, which is shown for two test cases, is:

- 1 Run test case 1 `Pre-Load` callback.
- 2 Run model `PreLoadFcn` callback.
- 3 Load model.
- 4 Run model `PostLoadFcn` callback.
- 5 Run test case 1 `Post-Load` callback.
- 6 Simulate model.
- 7 Run test case 1 `Cleanup` callback.
- 8 Run test case 2 `Pre-Load` callback.
- 9 Run test case 2 `Post-Load` callback.
- 10 Simulate model.
- 11 Run test case 2 `Cleanup` callback.
- 12 Run model `CloseFcn` callback.

If the model is open before the test runs, the execution order, which is shown for two test cases, is:

- 1 Run test case 1 `Pre-Load` callback
- 2 Run test case 1 `Post-Load` callback
- 3 Simulate model

- 4 Run test case 1 Cleanup callback
- 5 Run test case 2 Pre-Load callback
- 6 Run test case 2 Post-Load callback
- 7 Simulate model
- 8 Run test case 2 Cleanup callback

Notice that the model `PreLoadFcn` and `PostLoadFcn` callbacks do not execute because the model is already loaded before the test runs. The model `CloseFcn` callback does not execute either because the model is left open after test completion.

Multiple Test Cases in Multiple Test Suites on a Single Model

Suppose you have two test suites that each contain two test cases, such as:

- Test suite 1
 - Test case 1-1
 - Test case 1-2
- Test suite 2
 - Test case 2-1
 - Test case 2-2

The execution order of the callbacks is:

- 1 Run test suite 1 Setup callback.
- 2 Run test case 1-1 Pre-Load callback.
- 3 Run test case 1-1 PostLoad callback.
- 4 Simulate model.
- 5 Run test case 1-1 Cleanup callback.
- 6 Run test case 1-2 Pre-Load callback.
- 7 Run test case 1-2 Post-Load callback.
- 8 Simulate model.
- 9 Run test case 1-2 Cleanup callback.
- 10 Run test suite 1 Cleanup callback.
- 11 Run test suite 2 Setup callback.
- 12 Run test case 2-1 Pre-Load callback.
- 13 Run test case 2-1 Post-Load callback.
- 14 Simulate model.
- 15 Run test case 2-1 Cleanup callback.
- 16 Run test case 2-2 Pre-Load callback.
- 17 Run test case 2-2 Post-Load callback.
- 18 Simulate model.
- 19 Run test case 2-2 Cleanup callback.

20 Run test suite 2 Cleanup callback.

Test Case with Parameter Overrides

For a test case with parameter overrides, the execution order is:

- 1** Run test case Pre-Load callback.
- 2** Load model.
- 3** Read parameter overrides, which triggers a model update.
- 4** Run test case Post-Load callback.
- 5** Simulate model.
- 6** Run test case Cleanup callback.
- 7** Run model CloseFcn callback.

Filter Test Execution, Results, and Coverage

In this section...

“Add Tags” on page 6-231
 “Filter Tests and Results” on page 6-231
 “Run Filtered Tests” on page 6-231
 “Filter Coverage” on page 6-231

You can run a subset of tests or view a subset of test results by filtering test tags. Tags are a property of the test case, test suite, or test file.

Add Tags

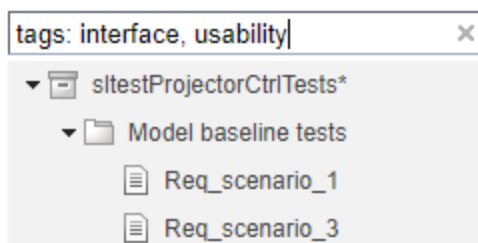
Add comma-separated tags to the **Tags** section in the Test Browser. Tags cannot contain spaces; spaces are corrected to commas.

▼ TAGS

safety, interface

Filter Tests and Results

In the text box at the top of the **Test Browser** or **Results and Artifacts** pane, filter tests by entering tags: id1, id2, ... where id1 and id2 are example test tags. Enter multiple tags separated by commas to return tests containing any tag in the list.



Run Filtered Tests

To run a subset of tests

- 1 Filter the tests using tags.
- 2 In the toolbar, click the down arrow below **Run** and select **Run Filtered**.

Filter Coverage

For information on filtering test coverage, see “Coverage Filtering Using the Test Manager” on page 6-156

Write and Run Tests that Inject Faults by Using the Simulink Test Manager

If you have Simulink Fault Analyzer, you can add faults to test cases and inject them into your models by creating fault sets. In each test case, the fault set specifies the active faults for that test. For more information on how to add faults to models, see “Define and Model Faults” (Simulink Fault Analyzer). You can create multiple test cases with different combinations of active faults in a test suite to simulate multiple combinations. After you inject faults, you can assess the signals to determine if the faults caused the desired outputs by using temporal assessments.

Create Fault Sets in the Simulink Test Manager

To create a fault set in a test case:

- 1 Open the Simulink Test app. In the **Apps** tab, click **Simulink Test**.
- 2 Open the Simulink Test Manager. In the **Tests** tab, click **Simulink Test Manager**.
- 3 Create a new test case or load an existing test case.
- 4 Expand the **Fault Settings** section and click **Add > Add Fault Set**.

The new fault set is empty. To add faults, select the fault set and click **Add > Add Fault**.

If you add faults to your model after you add the fault set, click **Refresh** to update the faults that the Simulink Test Manager identifies from your model. You can add more than one fault as long as each fault is on a different model element. Click **OK**. The fault set displays each fault name, their assigned model element, and their trigger.

▼ FAULT SETTINGS*

FAULT SET / MODEL ELEMENT	FAULT NAME	TRIGGER
▼ ☒ Fault Set		
☒ sldemo_fuelsys_fault_analyzer/To Controller/Inport/1	throttle_fault	Timed: 5
☐ sldemo_fuelsys_fault_analyzer/To Controller/Inport/2	speed_fault	Timed: 10
☐ sldemo_fuelsys_fault_analyzer/To Controller/Inport/3	ego_fault	Timed: 15
☐ sldemo_fuelsys_fault_analyzer/To Controller/Inport/4	map_fault	Timed: 20

+ Add ▼ ↻ Refresh 🗑 Delete

Select a fault in the **Fault Set/Model Element** column to simulate it. You can select each model element only once, and each model element can have only one selected fault. If the model element contains more than one fault, you can select a different fault by expanding the list in the **Fault Name** column.

When you run the test, the test overrides the activation status of the faults assigned to the model. If you do not select a fault set, the test runs the simulation by using the fault settings specified in the model. To run a test without faults, select the fault set and clear the faults.

To specify additional fault combinations, create a new fault set and select different faults. You can select one fault set for each simulation in a test. If you want to run more than one combination of

faults in a test, create a test iteration. See “Test Iterations” on page 6-139. If you want to adjust additional fault properties in each test, create a callback. See “Callbacks” on page 6-179.

After simulating, view the fault simulation results in the embedded Simulation Data Inspector.

Programmatically Create Fault Sets

You can also create and modify fault sets programmatically. The model that you test must already have faults. For more information on creating faults in models programmatically, see “Add and Configure Faults Programmatically” (Simulink Fault Analyzer).

To add a fault set programmatically:

- 1 Retrieve or create the `sltest.testmanager.TestCase` object. See, `createTestCase`, `getTestCases`, and `getTestCaseByName`.
- 2 Add a fault set to the test case by using the `addFaultSet` method. This method creates an instance of the `sltest.testmanager.FaultSet` class.
- 3 Update the available faults by using the `sltest.testmanager.refreshFaults` function on the model under test.
- 4 Add faults to the fault set by using the `addSpecifiedFault` method. This method creates an instance of the `sltest.testmanager.SpecifiedFault` class.

If you already have fault sets in a test case, you can retrieve the `sltest.testmanager.FaultSet` object by using the `getFaultSets` method on the `sltest.testmanager.TestCase` object. Similarly, retrieve the `sltest.testmanager.SpecifiedFault` object from a fault set by using the `getSpecifiedFaults` method on the `sltest.testmanager.FaultSet` object.

To remove fault sets and faults, use the `remove` method on the fault set or fault.

Run Fault Injection Tests on an Example Model

This example shows how to run tests that activate and inject faults by using fault sets. After you inject faults, you can assess the signals and determine if the faults caused the desired outputs by using temporal assessments.

Open the Model

Open the `sldemo_fuelsys_fault_analyzer` model and set up the faults by using the `fuelSysFaultTestSetup` helper function. The helper function deletes the existing Simulink® Fault Analyzer™ model artifacts and replaces them with the faults used in the example.

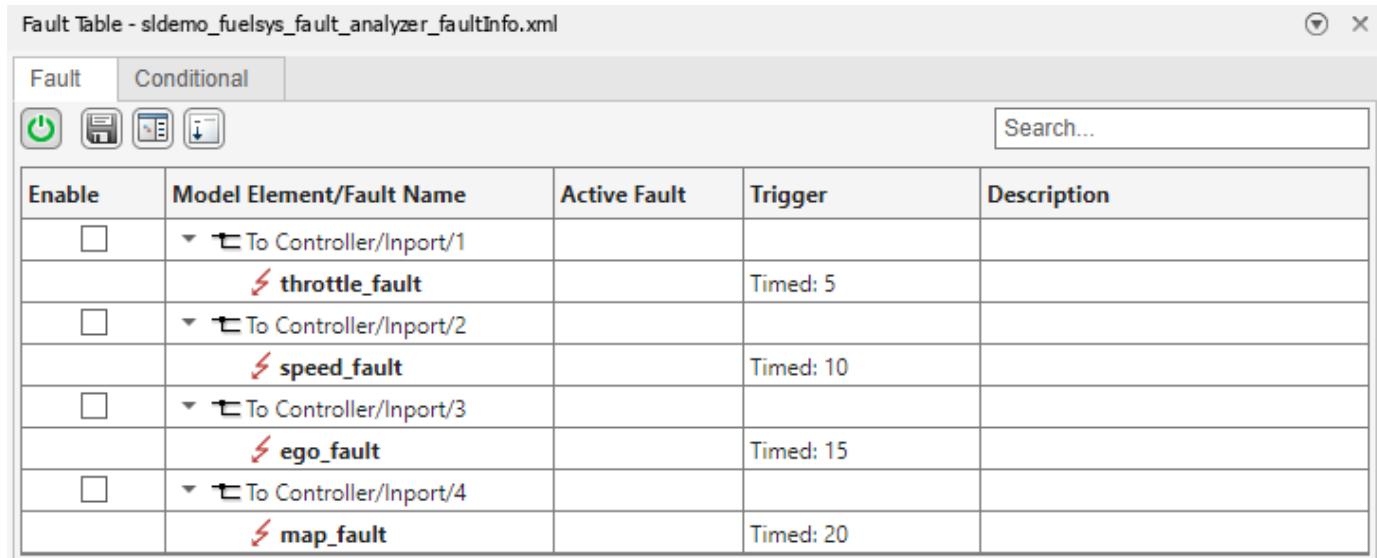
```
mdlName = "sldemo_fuelsys_fault_analyzer";
open_system(mdlName)
fuelSysFaultTestSetup
```

To view the code that sets up the faults, open the `fuelSysFaultTestSetup` MATLAB® file or go to the **Helper Function** section.

The model represents a fuel control system for a gasoline engine, and is a modified version of the model in “Model Fault-Tolerant Fuel Control System”. The model produces a fuel mode signal after detecting faults in the controller. When the controller detects a single fault, the controller changes the `fuel_mode` signal from `LOW`, which represents low emissions, to `RICH`, which represents a rich

mixture. When the controller detects a second fault, it sets `fuel_mode` to `DISABLED`. When the controller detects a fault, the model logs this behavior by setting the `normal_operation` signal to `false`.

The model contains four faults. To view the faults, open the **Fault Table** pane. In the **Apps** tab, click **Fault Analyzer**. In the **Fault Analyzer** tab, in the **View** section, click **Fault Table**. Expand each model element to view the faults. Each fault uses a timed trigger, where the fault is injected at the specified simulation time.



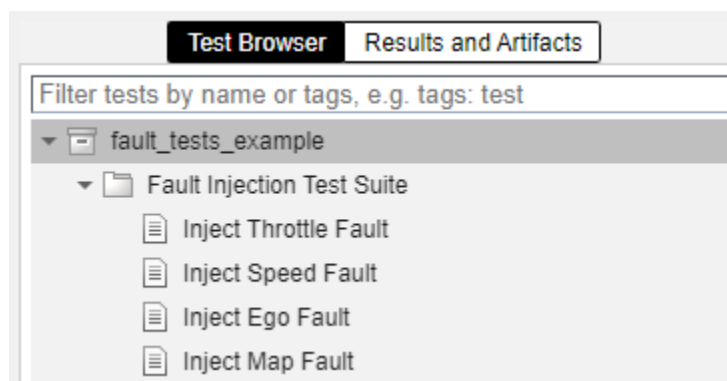
Enable	Model Element/Fault Name	Active Fault	Trigger	Description
☐	▼ To Controller/Inport/1			
	⚡ throttle_fault		Timed: 5	
☐	▼ To Controller/Inport/2			
	⚡ speed_fault		Timed: 10	
☐	▼ To Controller/Inport/3			
	⚡ ego_fault		Timed: 15	
☐	▼ To Controller/Inport/4			
	⚡ map_fault		Timed: 20	

View the Tests

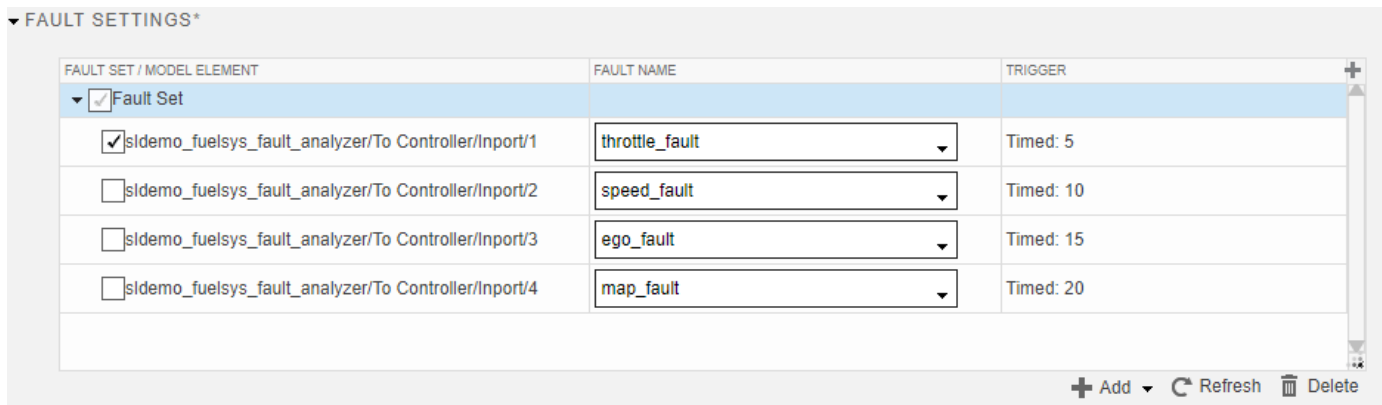
Each fault causes the `normal_operation` signal to output `false` after the fault is injected. In this example, you run tests to verify this behavior. Open the Simulink Test Manager and load the `fault_tests_example.mldatx` test file. In the **Apps** tab, click **Simulink Test**. In the **Tests** tab, click **Simulink Test Manager**. In the Simulink Test Manager, open the `fault_tests_example.mldatx` test file. Alternatively, enter this command:

```
tf = sltest.testmanager.load("fault_tests_example.mldatx");
sltest.testmanager.view;
```

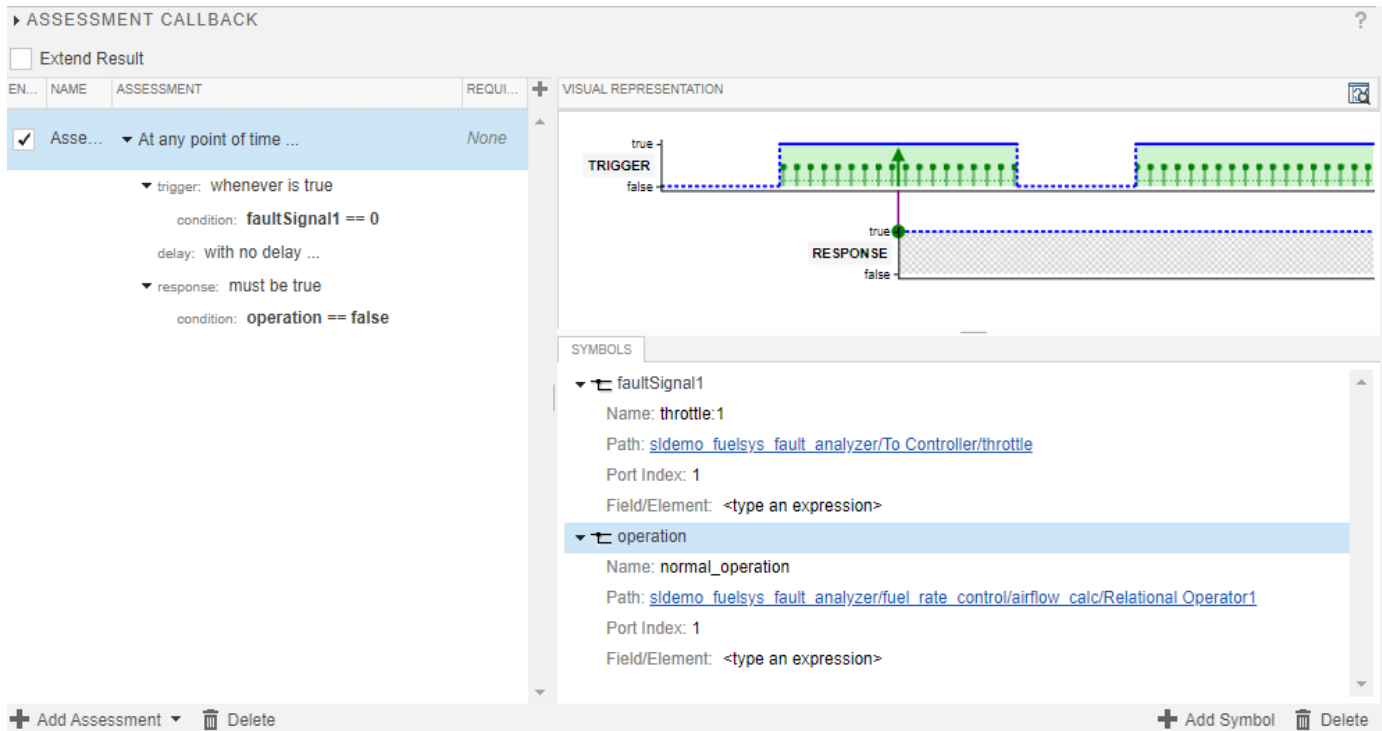
The test file has four test cases.



Each test case activates a fault by specifying a fault set with one selected fault. To view the fault set, in the Simulink Test Manager, in the left pane, click **Test Browser**. Expand the test file and the test suite, and select a test. Expand the **Fault Settings** section to view the fault set and the faults. This image shows the fault set for the test case that injects the `throttle_fault` fault.



To verify that the fault produces the desired behavior, each test case includes a temporal assessment. See “Assess Temporal Logic by Using Temporal Assessments” on page 3-85. In a test, expand the **Logical and Temporal Assessments** section. The assessment checks when the faulted signal equals the injected value. If the signal value equals the injected value from the fault, then the `normal_operation` signal must be false. This image shows the temporal assessment for the test case that injects the `throttle_fault` fault.

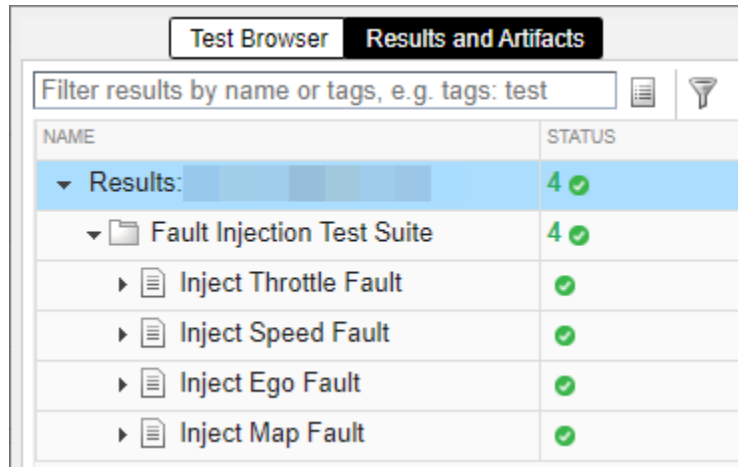


Run the Tests

Run the tests. Select **Fault Injection Test Suite** and click **Run**, or enter this command:

```
run(tf);
```

Select **Results and Artifacts**, then expand the results. Each test case passes, which means the controller detects when each fault is injected.



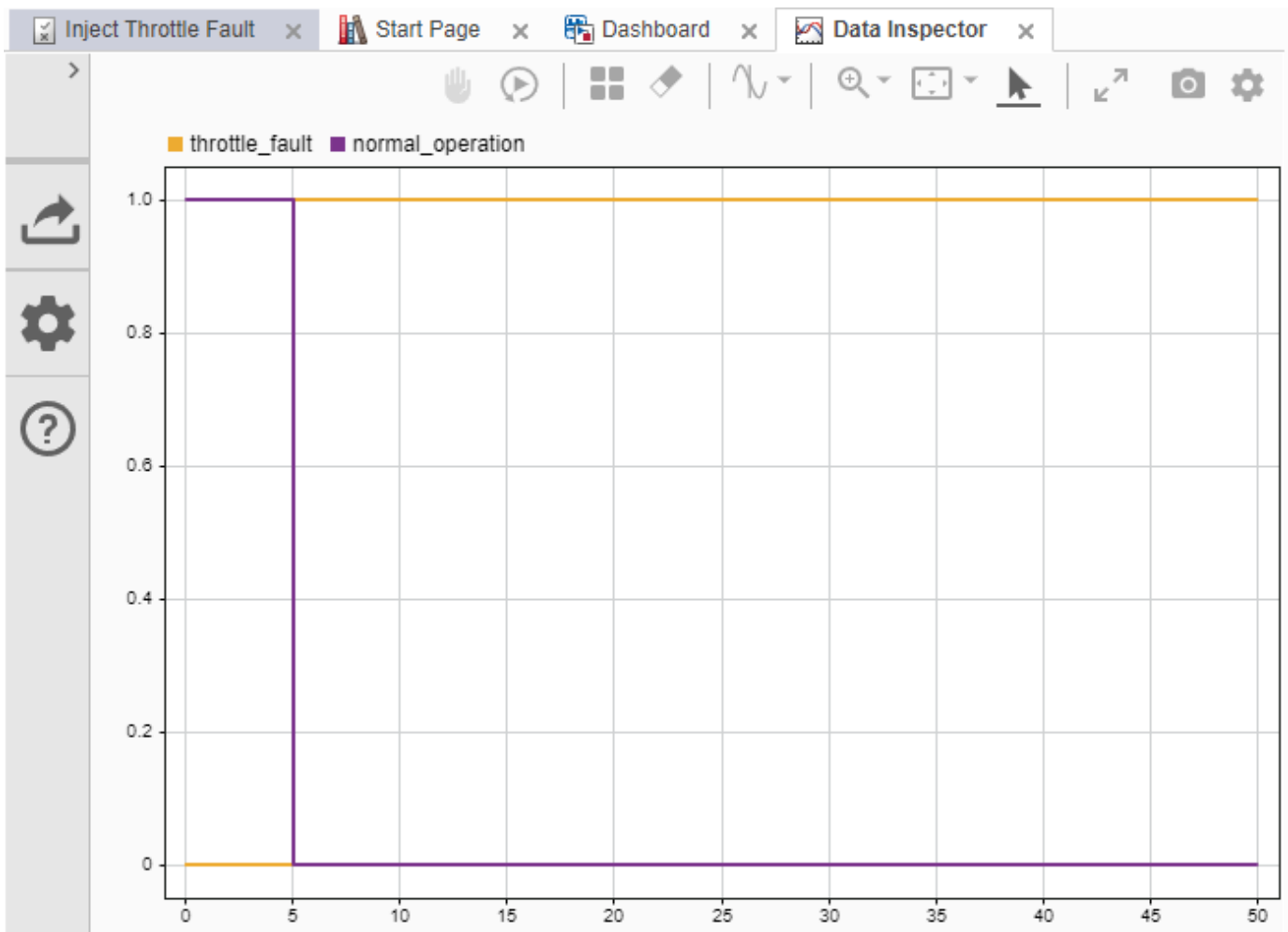
The screenshot shows the 'Test Browser' and 'Results and Artifacts' tabs. A search bar at the top allows filtering results by name or tags. Below the search bar is a table with two columns: 'NAME' and 'STATUS'. The table contains the following data:

NAME	STATUS
▼ Results:	4 ✓
▼ Fault Injection Test Suite	4 ✓
▶ Inject Throttle Fault	✓
▶ Inject Speed Fault	✓
▶ Inject Ego Fault	✓
▶ Inject Map Fault	✓

You can view the fault trigger status of the active faults of a test in the Simulink Test Manager.

- 1 Expand a test result and select **Sim Output**.
- 2 In **Data Inspector** tab, expand the **Faults** list.
- 3 Select the name of the fault to view the trigger status.

This image shows the fault trigger and the `normal_operation` signal status in the test that activated `throttle_fault`.



Helper Function

The `fuelSysFaultTestSetup` function adds the faults to the model, and adds fault sets and faults to the test cases. To see the example code that creates the fault sets, adds the faults, and enables the faults, see the [Create Fault Sets and Add Faults in Test Cases](#) section.

```
function fuelSysFaultTestSetup

%% Initializing
mdl = "sldemo_fuelsys_fault_analyzer";
bmdl = mdl + "_FaultModel";
load_system(mdl);
fault = Simulink.fault.Fault.empty(0,4);
ports = 4;
Simulink.fault.save(mdl)

% Load the test file, and get the test suite and test cases.
tf = sltest.testmanager.load("fault_tests_example.mldatx");
ts = getTestSuites(tf);
tc = getTestCases(ts);

%% Deleting existing faults, conditionals, and fault sets
```

```

for n = 1:numel(tc)
    initialFaultSets = getFaultSets(tc(n));
    if numel(initialFaultSets) > 0
        for m = 1:numel(initialFaultSets)
            remove(initialFaultSets(m))
        end
    end
end

warnID = "safetyanalyzer:faultinfo:DeletedEmptyFaultModelWarning";
warning("off",warnID)
Simulink.fault.deleteAll mdl
Simulink.fault.addConditional(mdl,"SampleConditional","x == true");
sltest.testmanager.refreshFaults(mdl)
warning("on",warnID)

%% Adding faults and behaviors
for n = 1:ports
    fault(n) = Simulink.fault.addFault(mdl ...
        + "/To Controller/Inport/" + num2str(n));
end

% Add behavior to each fault. Three of the faults use stuck-at-ground
% behavior and one of them uses stuck-at-constant behavior.
addBehavior(fault(1),bmdl,...
    FaultBehavior="mwfaultlib/Stuck-at-Ground");
addBehavior(fault(2),bmdl,...
    FaultBehavior="mwfaultlib/Stuck-at-Ground");
addBehavior(fault(3),bmdl,...
    FaultBehavior="mwfaultlib/Stuck-at-Constant");
addBehavior(fault(4),bmdl,...
    FaultBehavior="mwfaultlib/Stuck-at-Ground");
load_system(bmdl)
set_param(bmdl+"/ego_fault/Constant",Value="12")

%% Specify Fault Properties
% Define the trigger types. Each fault uses a timed trigger
% at a different time.
fault(1).TriggerType = "Timed";
fault(1).StartTime = 5;
fault(2).TriggerType = "Timed";
fault(2).StartTime = 10;
fault(3).TriggerType = "Timed";
fault(3).StartTime = 15;
fault(4).TriggerType = "Timed";
fault(4).StartTime = 20;

% Disable the faults.
Simulink.fault.enable(mdl + "/To Controller/Inport/1",0)
Simulink.fault.enable(mdl + "/To Controller/Inport/2",0)
Simulink.fault.enable(mdl + "/To Controller/Inport/3",0)
Simulink.fault.enable(mdl + "/To Controller/Inport/4",0)

save_system(bmdl);
Simulink.fault.save(mdl)

%% Create Fault Sets and Add Faults in Test Cases

```

```

% Update the available faults to include the faults in the model
% that you can select for the fault set.
sltest.testmanager.refreshFaults mdl)

% Create the fault sets and add the faults to each fault set.
% Activate only one fault for each fault set, and disable the others.
faultSets = sltest.testmanager.FaultSet.empty(0,ports);
testedFaults = sltest.testmanager.SpecifiedFault.empty(0,ports);
for i = 1:ports
    faultSets(i) = addFaultSet(tc(i),Name="Fault Set");
    for m = 1:ports
        testedFaults(m) = addSpecifiedFault(faultSets(i),...
            "sldemo_fuelsys_fault_analyzer/To Controller/Inport/" ...
            + num2str(m), fault(m).Name);
        if m == i
            testedFaults(m).IsActive = true;
        else
            testedFaults(m).IsActive = false;
        end
    end
end
end
end

```

See Also

Simulink Test Manager

Related Examples

- “Define and Model Faults” (Simulink Fault Analyzer)
- “Add and Configure Faults Programmatically” (Simulink Fault Analyzer)
- “Assess Temporal Logic by Using Temporal Assessments” on page 3-85
- “Evaluate Fault Combinations on a Fault-Tolerant Fuel System” (Simulink Fault Analyzer)
- “Simulate Models with Faults by Using the Multiple Simulations Panel” (Simulink Fault Analyzer)

Baseline Testing

This tutorial shows how to use Simulink® Test™ to create and run a baseline test to test for regression after you make changes to a model.

When you run a baseline test case, you compare simulation output data to baseline data. To run a baseline test using Simulink Test, you create a baseline test case and set the test properties. You can use the Test Manager to capture logged signal data from the system under test or add baseline data from an existing file and set the signal tolerances to specify the pass and fail criteria for the test. After you make changes to a model, you can run the baseline test case to check whether the new simulation output data satisfies the pass and fail criteria.

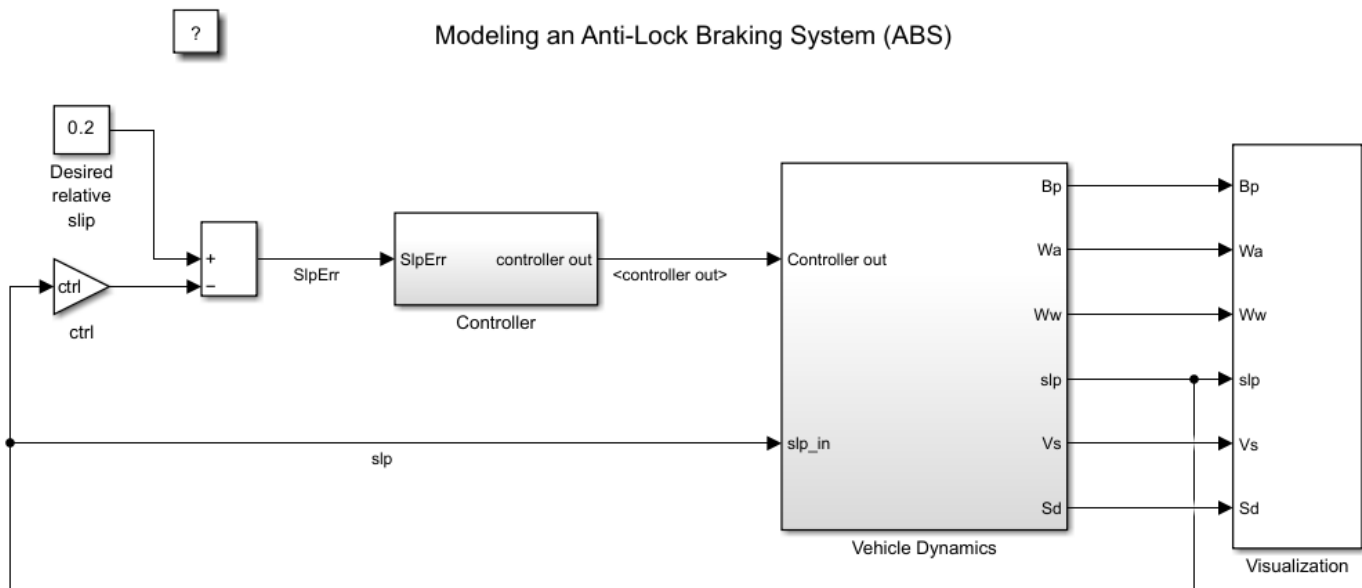
In this tutorial, you follow these steps to create a test case, capture baseline data, run the test and create a results report:

- 1 “Author a Baseline Test Case” on page 6-242: Create a new test file, specify test properties in the Test Manager, and capture and validate baseline data for the test.
- 2 “Compare Simulation to Baseline Data” on page 6-246: Make changes to the model, run the test case, and review the results.
- 3 “Create a Test Results Report” on page 6-249: Use the Test Manager to generate a test results report.

Examine the Model

This tutorial uses the `sldemo_absbrake` model. To open the model, enter:

```
open_system("sldemo_absbrake");
```



Copyright 1990-2023 The MathWorks, Inc.

The `sldemo_absbrake` model simulates the behavior of a wheel with an anti-lock braking system (ABS) and logs the signals in the MATLAB workspace. To simulate the model, on the **Simulation** tab,

click **Run**. The plots display the vehicle and wheel speed, slip, and wheel acceleration while braking. For more information on the model physics and analysis, see “Model an Anti-Lock Braking System”.

See Also

Simulink Test Manager

More About

- “Functional Testing for Verification”
- “Test Planning and Strategies”

Author a Baseline Test Case

In this example, you use the Simulink Test Manager to create a test file and author a test case. You specify the test properties by selecting the system under test and coverage collection settings. Then, you simulate the model to capture the baseline data and run the test case to validate the data and confirm that the signals are the same.

You can also create a test case programmatically or use the Test for Model Component wizard. For information see “Create and Run Test Cases with Scripts” on page 6-115 and “Generate Tests and Test Harnesses for a Model or Components” on page 6-20.

Open the Model and Test Manager

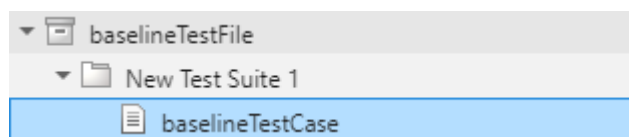
1. Open the `sldemo_absbrake` model.

```
open_system("sldemo_absbrake");
```

2. In the **Apps** tab, in the **Model Verification, Validation, and Test** section, click **Simulink Test**.
3. In the **Tests** tab, in the **Test Cases** section, click **Simulink Test Manager**.

Create the Test File and Specify Test Properties

1. In the Test Manager, click **New > Test File**.
2. Name the file `baselineTestFile` and save the file.
3. By default, the Test Manager creates a test suite with one baseline test case, `New Test Case 1`. In the **Test Browser**, right-click `New Test Case 1` and select **Rename**. Rename the test case to `baselineTestCase`.



4. Load the `sldemo_absbrake` model into the test case. In the **System Under Test** section, click the Use current model button .



5. To collect coverage in the test case, you must enable coverage collection in the test file. In the **Test Browser**, select `baselineTestFile`. Expand the **Coverage Settings** section. Select **Record coverage for system under test**. The **Coverage Settings** section of the test case updates to match

the coverage settings in the test file. For more information on coverage, see “Collect Coverage in Tests” on page 6-151.

Capture and Validate Baseline Data

1. In the **Test Browser**, select `baselineTestCase`. In the **Baseline Criteria** section, click **Capture** and specify a location in the **File** field. Name the file `absbrake_baselinedata` and click **Capture**. The test case simulates the model and captures the signals that are selected for signal logging. The **Baseline Criteria** section displays the logged signals with default tolerances of 0. For more information about how to mark signals for logging in your model, see “Save Signal Data Using Signal Logging”. Alternatively, you can select signals in the **Simulation Outputs** section of the Test Manager. See “Simulation Outputs” on page 6-182.

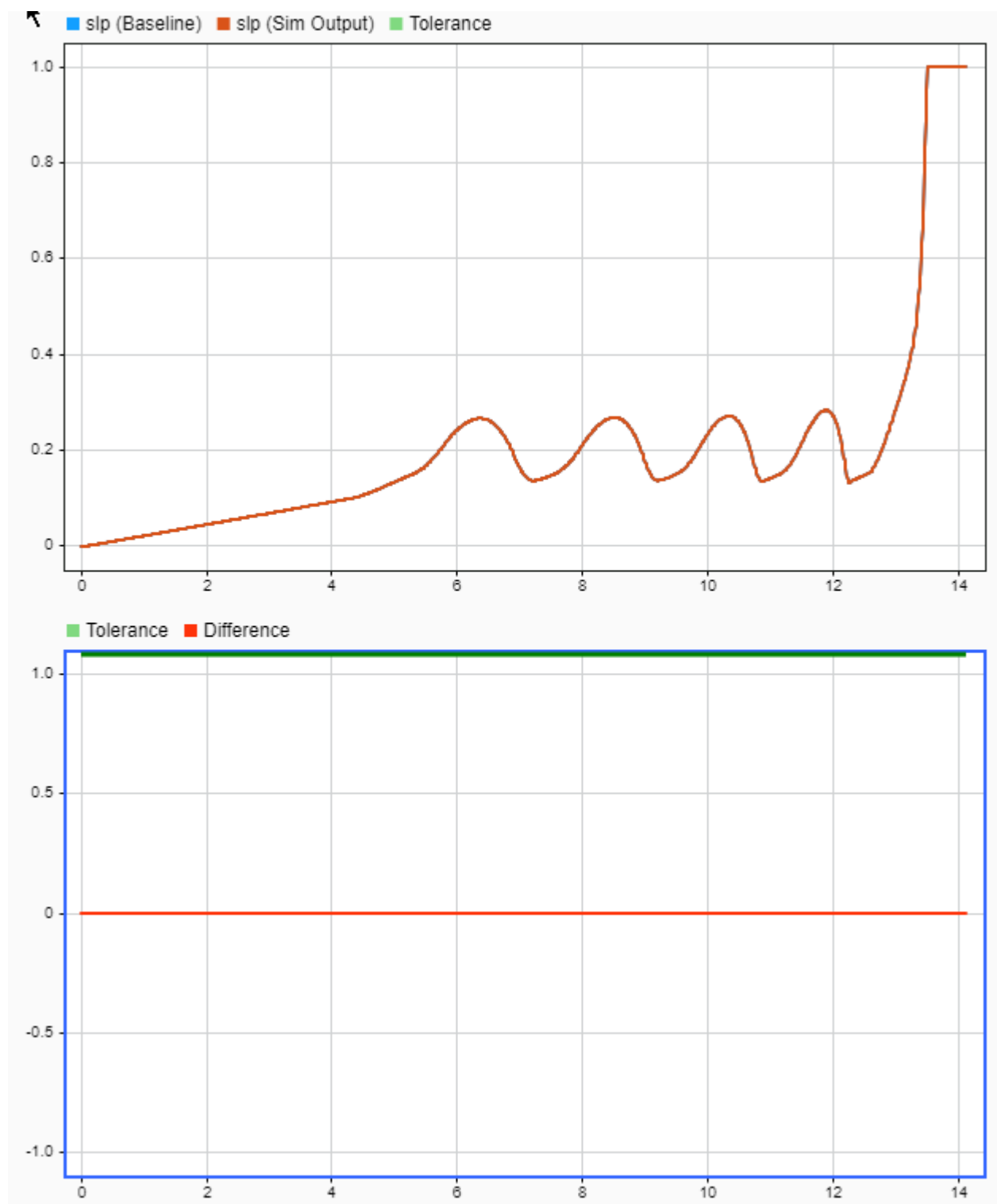
▼ **BASLINE CRITERIA***

☐ Include baseline data in test result

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL
▼ ☒ absbrake_baselinedata.mat	0	0.00%	0	0
☒ Ww	0	0.00%	0	0
☒ Vs	0	0.00%	0	0
☒ Sd	0	0.00%	0	0
☒ slp	0	0.00%	0	0
☒ Wa	0	0.00%	0	0
☒ Bp	0	0.00%	0	0
☒ WheelAngularVelocity	0	0.00%	0	0

+ Add...
 📁 Capture...
 ✎ Edit...
 ↻ Refresh
 📊 Visualize
 🗑 Delete

2. In the Test Manager, click **Run**. In the **Results and Artifacts** pane, expand the **Baseline Criteria Result** section and select the `slp` signal. The upper plot displays the simulation output data and baseline data and the lower plot displays the tolerance and difference between the simulation output data and baseline data. Review the signals to confirm that there is a zero difference between the simulation output and baseline data, as expected.



See Also

Simulink Test Manager

Related Examples

- “Model an Anti-Lock Braking System”

More About

- “Specify Test Properties in the Test Manager” on page 6-173
- “Capture Baseline Data from Iterations” on page 6-148
- “Collect Coverage in Tests” on page 6-151

Compare Simulation to Baseline Data

In this example, you use the Simulink Test Manager to add signal tolerances to baseline data. You make changes to the model and run the test to simulate the model. Then, you review the results to compare the simulation output to baseline data.

Open the Test File and Model

- 1. Load the test file and open the Test Manager.

```
sltest.testmanager.load("baselineTestFile.mldatx");
sltest.testmanager.view
```

- 2. Open the model.

```
open_system("sldemo_absbrake");
```

Add Signal Tolerances and Update Model

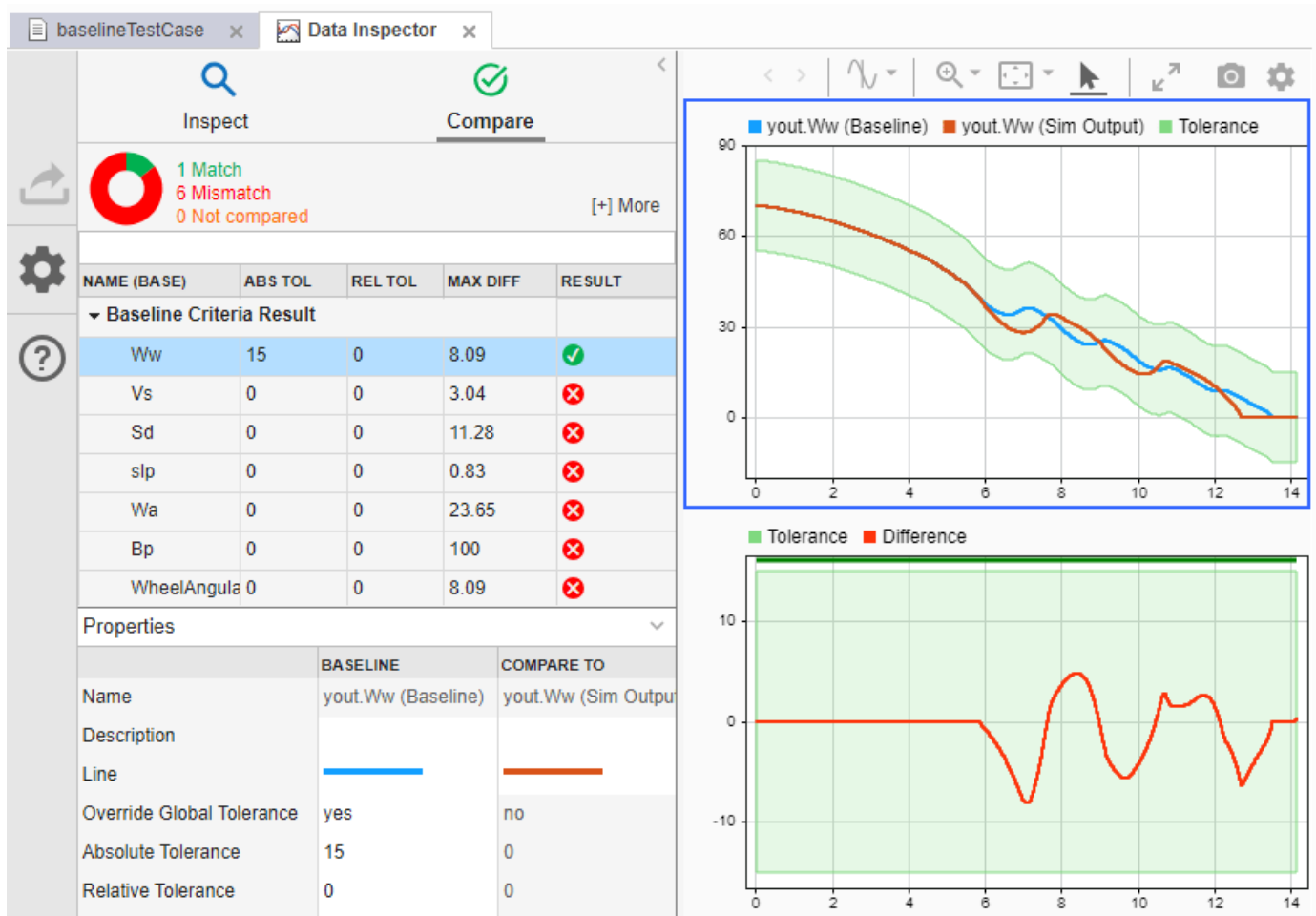
- 1. In the Test Manager, in the left pane, select **Test Browser**, then select the `baselineTestCase` test case. In the right pane, expand the **Baseline Criteria** section. Set the absolute tolerance of the `Ww` signal by setting the value in the **Abs Tol** column to 15. For more information, see “Set Signal Tolerances” on page 6-169

SIGNAL NAME	ABS TOL	REL TOL	LEADING TOL	LAGGING TOL	
▼ ☒ absbrake_baselinedata.mat	0	0.00%	0	0	
☒ Ww	15	0.00%	0	0	
☒ Vs	0	0.00%	0	0	
☒ Sd	0	0.00%	0	0	
☒ slp	0	0.00%	0	0	
☒ Wa	0	0.00%	0	0	
☒ Bp	0	0.00%	0	0	
☒ WheelAngularVelocity	0	0.00%	0	0	

- 2. In the `sldemo_absbrake` model, set the `Desired relative slip Constant` block to 0.22.

Run the Test and Review Results

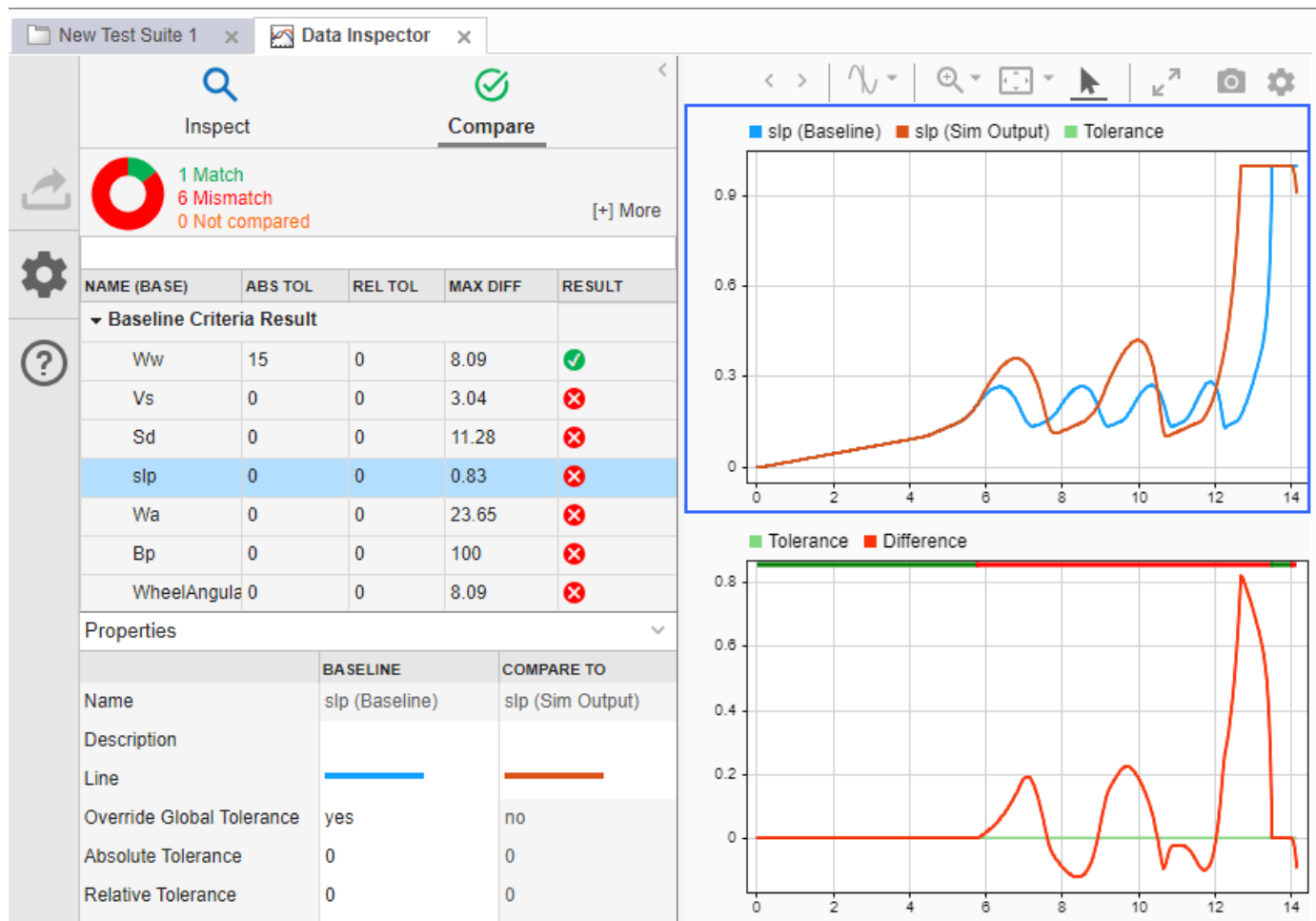
- 1. In the Test Manager, select `baselineTestCase` and click **Run**.
- 2. In the **Results and Artifacts** pane, for the most recent results, expand **baselineTestCase** and select **Baseline Criteria Result**. Select the `Ww` signal. In the **Data Inspector**, in the **Compare** pane, select the `Ww` signal. The upper plot displays the simulation output data, baseline data, and tolerance. The lower plot displays the tolerance and the difference between the simulation output and the baseline data. The `Ww` signal passes the test, because the signal is stays within the specified tolerance when compared to the baseline data.



3. The overall test fails because the other signal comparisons do not satisfy the tolerance values specified in the **Baseline Criteria** section of the test case. In the **Results and Artifacts** pane, the test case displays a failed status.

baselineTestCase	✗
Baseline Criteria Result	✗
Sim Output (slide...	

In the **Compare** pane of the **Data Inspector** tab, in the **Baseline Criteria Result** section, select the **slp** signal. The upper plot shows that the simulation output and baseline data are not the same. The lower plot displays a difference greater than the zero difference required to satisfy the baseline criteria.



See Also

Simulink Test Manager

Related Examples

- “Model an Anti-Lock Braking System”

More About

- “Specify Test Properties in the Test Manager” on page 6-173

Create a Test Results Report

This example shows how to generate a test results report for a baseline test by using the Simulink Test Manager.

Open the Test Results and Test Manager

1. Import the `baselineTestResults` file.

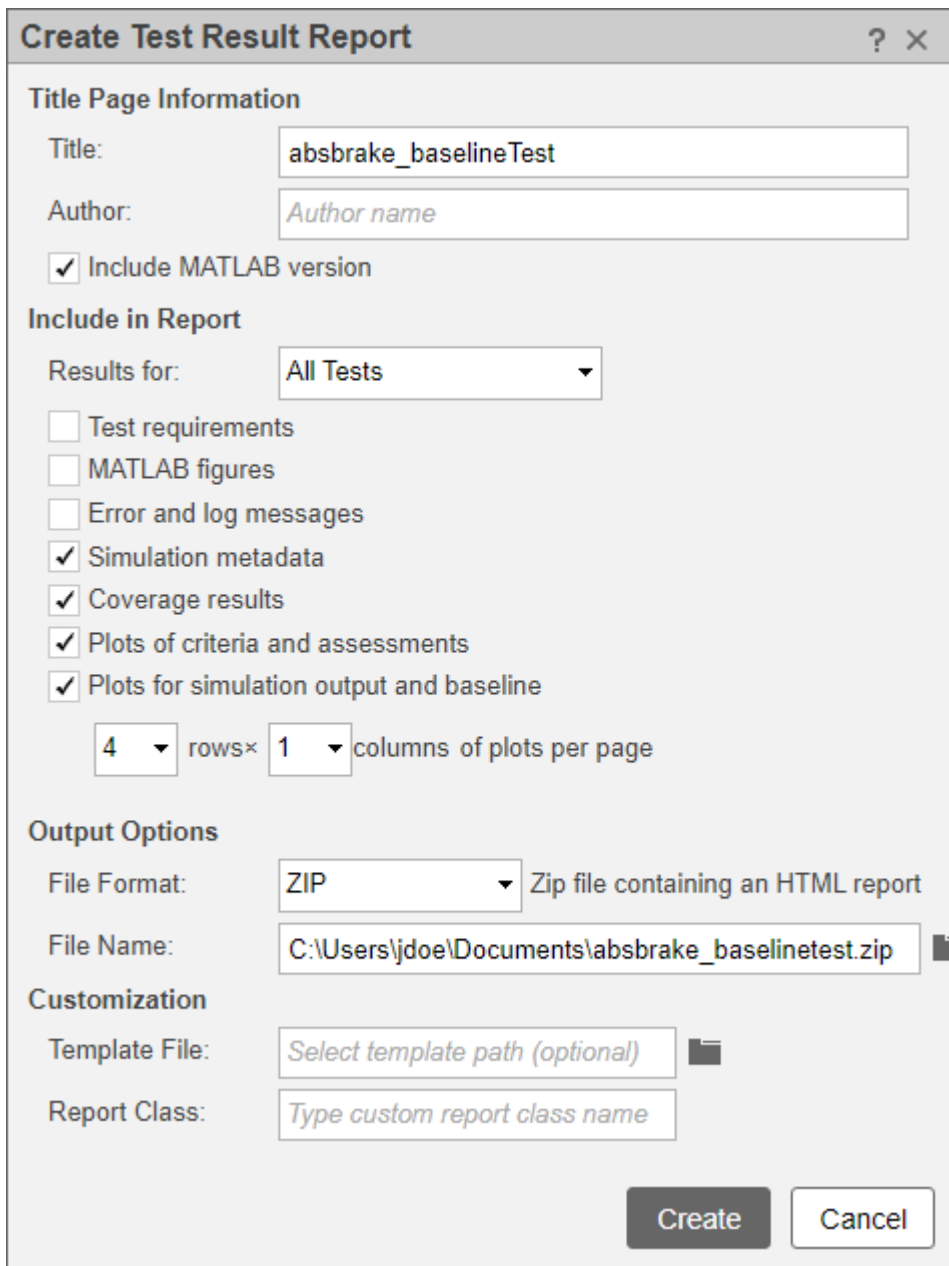
```
sltest.testmanager.importResults("baselineTestResults.mldatx");
```

2. Open the Test Manager.

```
sltest.testmanager.view
```

Generate Test Results Report

1. In the Test Manager, in the **Results and Artifacts** pane, for the most recent results, select `baselineTestCase`.
2. In the toolstrip, in the **Results** section, click **Report**. The Create Test Result Report dialog opens.
3. Set the **Title** to `absbrake_baselineTest`.
4. In the **Include in Report** section, set the **Results for** section to **All Tests** and select the information to include in the report. For more information on test results report options, see “Create, Set Options, and Use Templates for Test Result Reports” on page 7-17.
5. Set the **File Format** drop-down menu to **ZIP**.

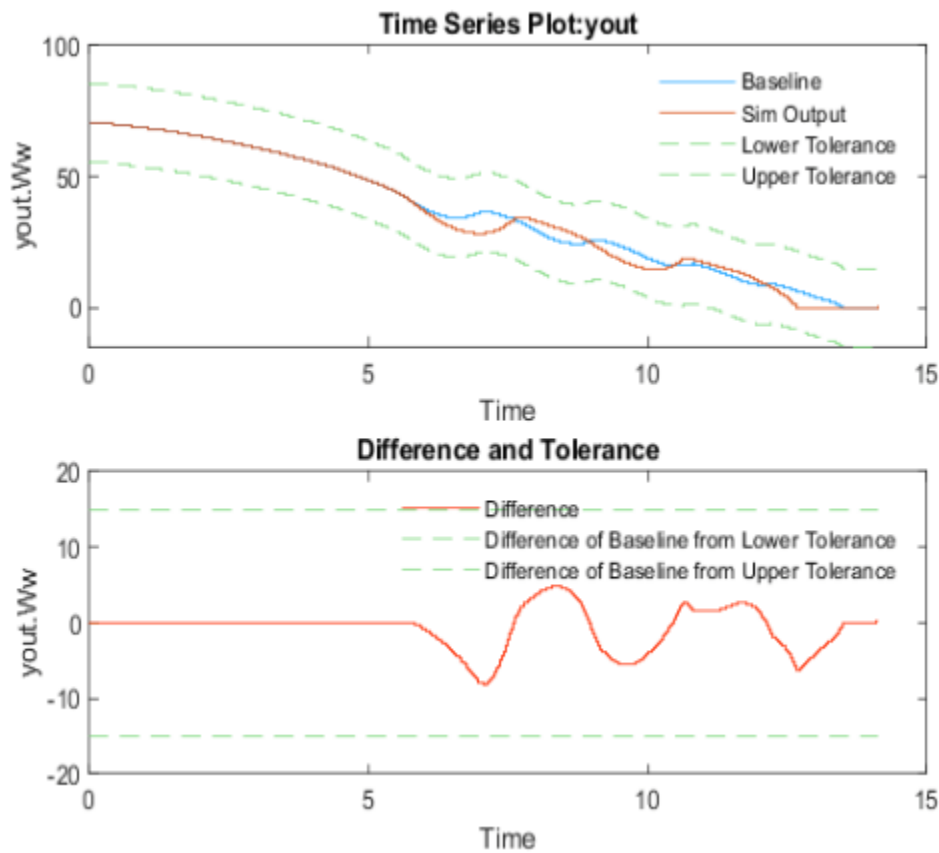


The image shows a 'Create Test Result Report' dialog box with the following sections and controls:

- Title Page Information**
 - Title:
 - Author:
 - ☒ Include MATLAB version
- Include in Report**
 - Results for:
 - ☐ Test requirements
 - ☐ MATLAB figures
 - ☐ Error and log messages
 - ☒ Simulation metadata
 - ☒ Coverage results
 - ☒ Plots of criteria and assessments
 - ☒ Plots for simulation output and baseline
 - rows × columns of plots per page
- Output Options**
 - File Format: Zip file containing an HTML report
 - File Name:
- Customization**
 - Template File:
 - Report Class:

Buttons: **Create** (highlighted), **Cancel**

5. Click **Create**. The Test Manager generates an HTML report. The report displays the test information selected in the Create Test Result Report dialog. Scroll to the section for the `yout.Ww` signal. The graph shows that difference between the `yout.Ww` signal and the baseline signal data meets the tolerance criteria.



Clean Up

Run these commands to close the model without saving, and to clear and close the Test Manager.

```
close_system("sldemo_absbrake",0)
sltest.testmanager.clear
sltest.testmanager.clearResults
sltest.testmanager.close
```

See Also

Simulink Test Manager

Related Examples

- “Model an Anti-Lock Braking System”

Test Manager Results and Reports

- “View Test Case Results” on page 7-2
- “Debugging Baseline Test Failures Using Model Slicer” on page 7-6
- “Export Test Results” on page 7-16
- “Create, Set Options, and Use Templates for Test Result Reports” on page 7-17
- “Generating a Test Results Report” on page 7-20
- “Customize Test Results Reports” on page 7-21
- “Append Code to a Test Report” on page 7-34
- “Results Sections” on page 7-37
- “Generate Test Specification Reports” on page 7-41
- “Customize Test Specification Reports” on page 7-45
- “Debugging Equivalence Test Failures Using Model Slicer” on page 7-54

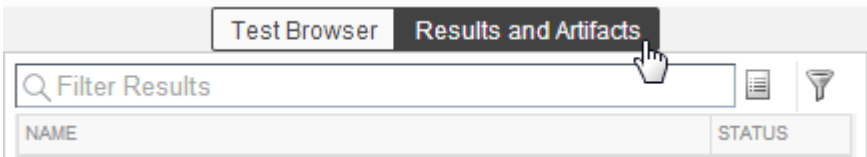
View Test Case Results

In this section...
“View Results Summary” on page 7-2
“Visualize Test Case Comparison and Criteria Results” on page 7-3

After a test case finishes running in the Test Manager, the test case result becomes available in the **Results and Artifacts** pane. This pane organizes test results by test case. Expand a test case to display its criteria result and output signals.

View Results Summary

- 1 Select the **Results and Artifacts** pane.



- 2 Click a test case.

▼ Results: 2024-Apr-08 08:48:07	1 ✖
▼ Equivalence Test	✖
○ Equivalence Criteria Result	✖
□ Sim Output 1(mForE...	
□ Sim Output 2(mForE...	

A tab opens containing the test case results information.

▼ SUMMARY

Name	Equivalence Test
Outcome	1 
Start Time	04/08/2024 08:48:07
End Time	04/08/2024 08:48:08
Type	Equivalence Test
Test File Location	C:\MATLAB\myEquivTest.mldatx
Test Case Definition	
Rerun Test Case	
Tags	
Cause of Failure	<u>Failed criteria: Equivalence</u>
▶ Execution Timing Information	
▶ Simulation 1 Metadata	
▶ Simulation 2 Metadata	

Visualize Test Case Comparison and Criteria Results

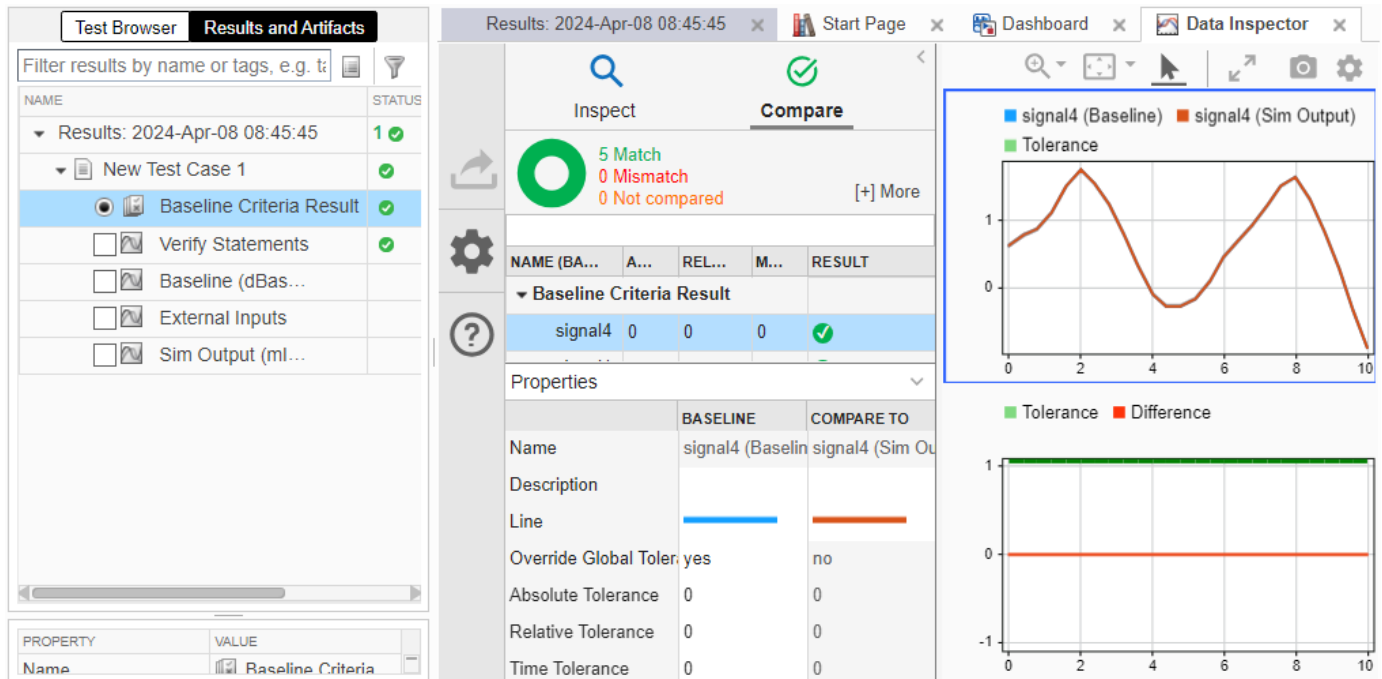
When you expand the Results to a test case or iteration, the results display radio buttons, which select comparison data, and check boxes, which select signal data. When you select a criteria result radio button or one of the test case signal check boxes, an embedded version of the Simulation Data Inspector opens and displays data plots of the results. Viewing comparison test results generated prior to R2019a opens a standalone version of the Simulation Data Inspector, and from R2019a to R2024b, the Test Manager included only a few Simulation Data Inspector options in a small toolbar.

▼ Results: 2024-Apr-08 08:45:45	1 
▼  New Test Case 1	
☒  Baseline Criteria Result	
☐  Verify Statements	
☐  Baseline (dBaseli...	
☐  External Inputs	
☐  Sim Output (mlns...	

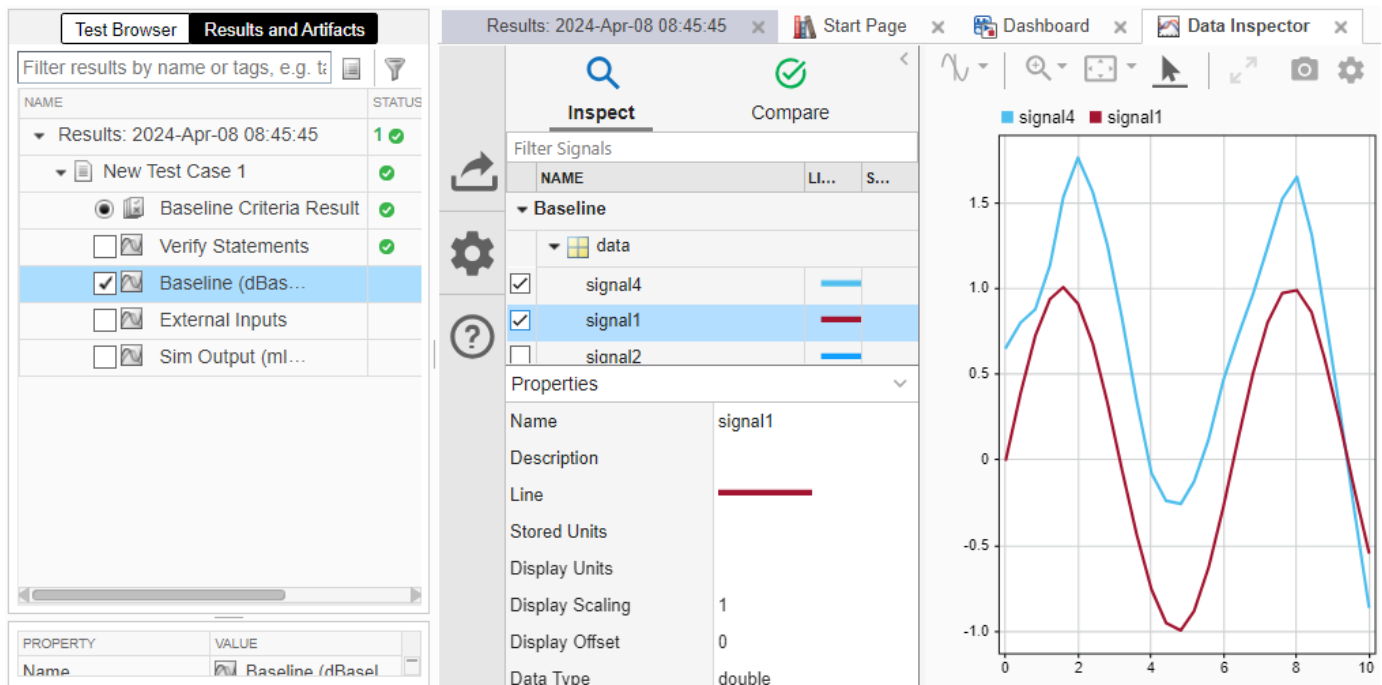
To compare results for baseline or equivalence tests,

- 1 Select the **Baseline** or **Equivalence Criteria Result** radio button. The embedded Simulation Data Inspector displays the **Compare** tab.

- 2 In the **Compare** tab, expand the **Baseline** or **Equivalence Criteria Result**.
- 3 Select a signal to display.



To view the signals for a run, click the check box in the **Results and Artifacts** pane. In the Simulation Data Inspector **Inspect** pane, expand the run and select the signal to plot.



Differences between the standalone Simulation Data Inspector and the Simulation Data Inspector embedded in the Simulink Test Manager include:

- The standalone Simulation Data Inspector uses a single time tolerance. If your test specifies lead and lag time tolerances, when you run the test, the results honor those times. However, if you change the time tolerance in the embedded Simulation Data Inspector, the new time tolerance applies to both the lead and lag tolerances.
- The embedded Simulation Data Inspector has fewer buttons in the side panel because the options are available in the Test Manager menus.

For more information about the Simulation Data Inspector, see “Inspect Simulation Data” and “Compare Simulation Data”.

See Also

Simulink Test Manager

Related Examples

- “Baseline Testing” on page 6-240
- “Back-to-Back Equivalence Testing” on page 6-36

Debugging Baseline Test Failures Using Model Slicer

This example shows how to debug Simulink Test baseline and verification failures by using the Model Slicer.

A predefined baseline test case is provided for use with the `sltestDemo_fuelsys` model. The baseline is captured from an earlier state of the model. After capturing the baseline, a design error is introduced in the model, which causes the baseline test to fail. Then, the Model Slicer is used to debug the failure and localize the design error.

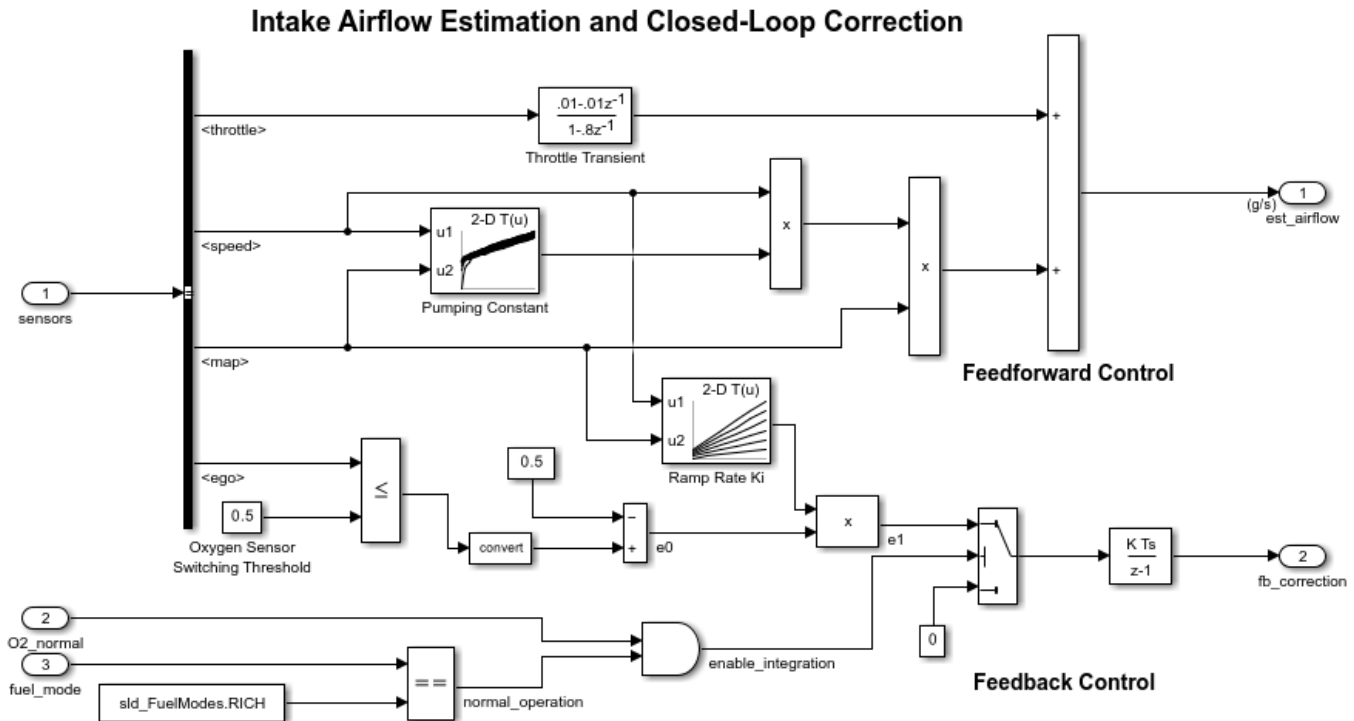
For information about the Model Slicer, see the Simulink Check documentation.

Step 1: Setting Up the Artifacts

This section describes how to run the test case and view the results.

1. Open the `sltestDemo_fuelsys` model.

```
open_system('sltestDemo_fuelsys');
```

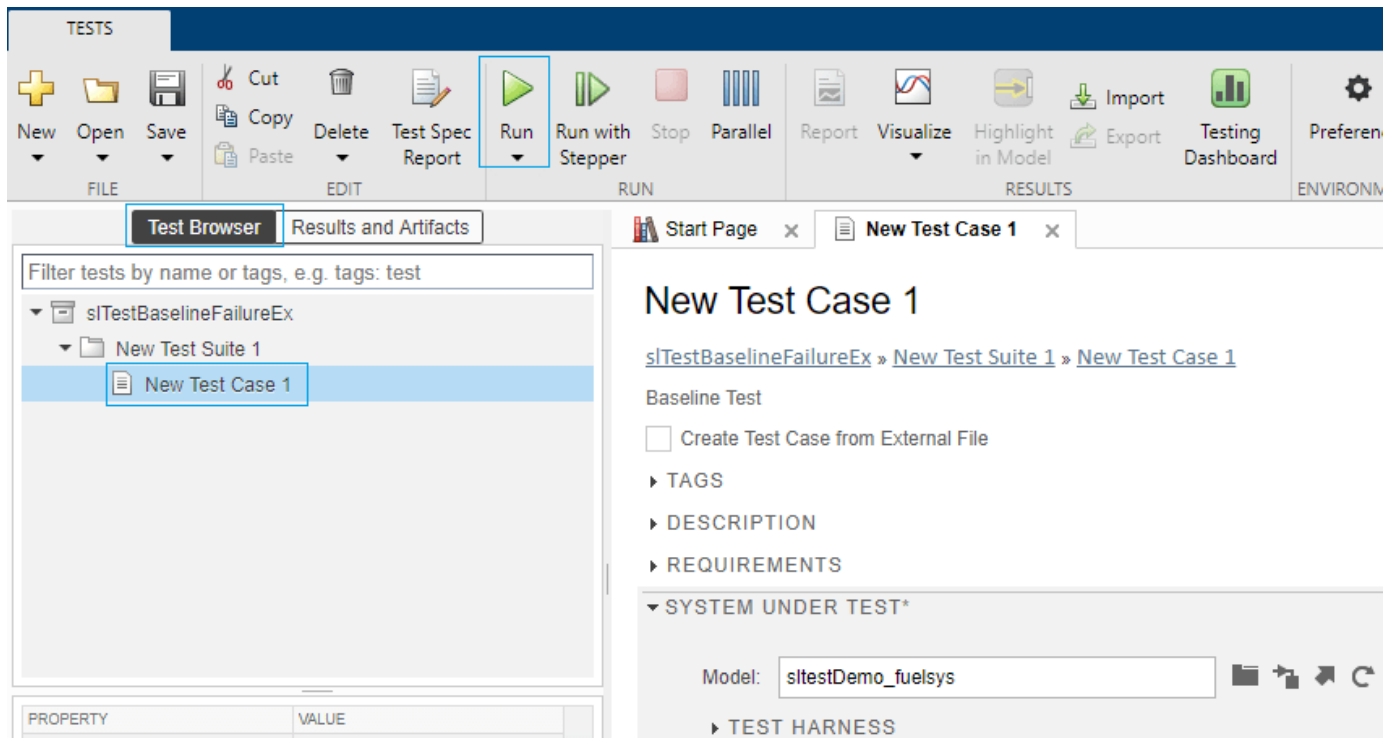


Copyright 1998-2020 The MathWorks, Inc.

2. Click **APPS** > **Model Verification, Validation, and Test** > **Simulink Test** to open the Simulink Test toolstrip.

3. Click **Tests** > **Simulink Test Manager** to open the Test Manager.

4. To open the existing test file, from the Test Manager toolbar, click **Open** and select `slTestBaselineFailureEx`.
5. After the test file loads, select **New Test Case1** in the **Test Browser** pane.
6. Click **Run**.



7. The new test results appear at the top of the **Results and Artifacts** pane. Right-click the result and select **Expand All Under**, so that you see the **Baseline Criteria Result** and the **Verify Statements**.

Test Browser Results and Artifacts

Filter results by name or tags, e.g. tags: tes

NAME	STATUS
Results: 2024-Oct-08 11:07:01	1 ✖
New Test Case 1	✖
Baseline Criteria Result	✖
Verify Statements	✖
Sim Output (sltestDem...	

New Test Case 1 Start Page Dashboard

Inspect Compare

3 Match
3 Mismatch
0 Not compared

[+] More

NAME (BASE)	ABS TOL	REL TOL	MAX ...	RESU...
Baseline Criteria Result				
air_fuel_ratio	0	0.00%	0.32	✖
speed	0	0.00%	0	✔
map	0	0.00%	0	✔
ego	0	0.00%	0.46	✖
throttle	0	0.00%	0	✔
fuel	0	0.00%	0.04	✖

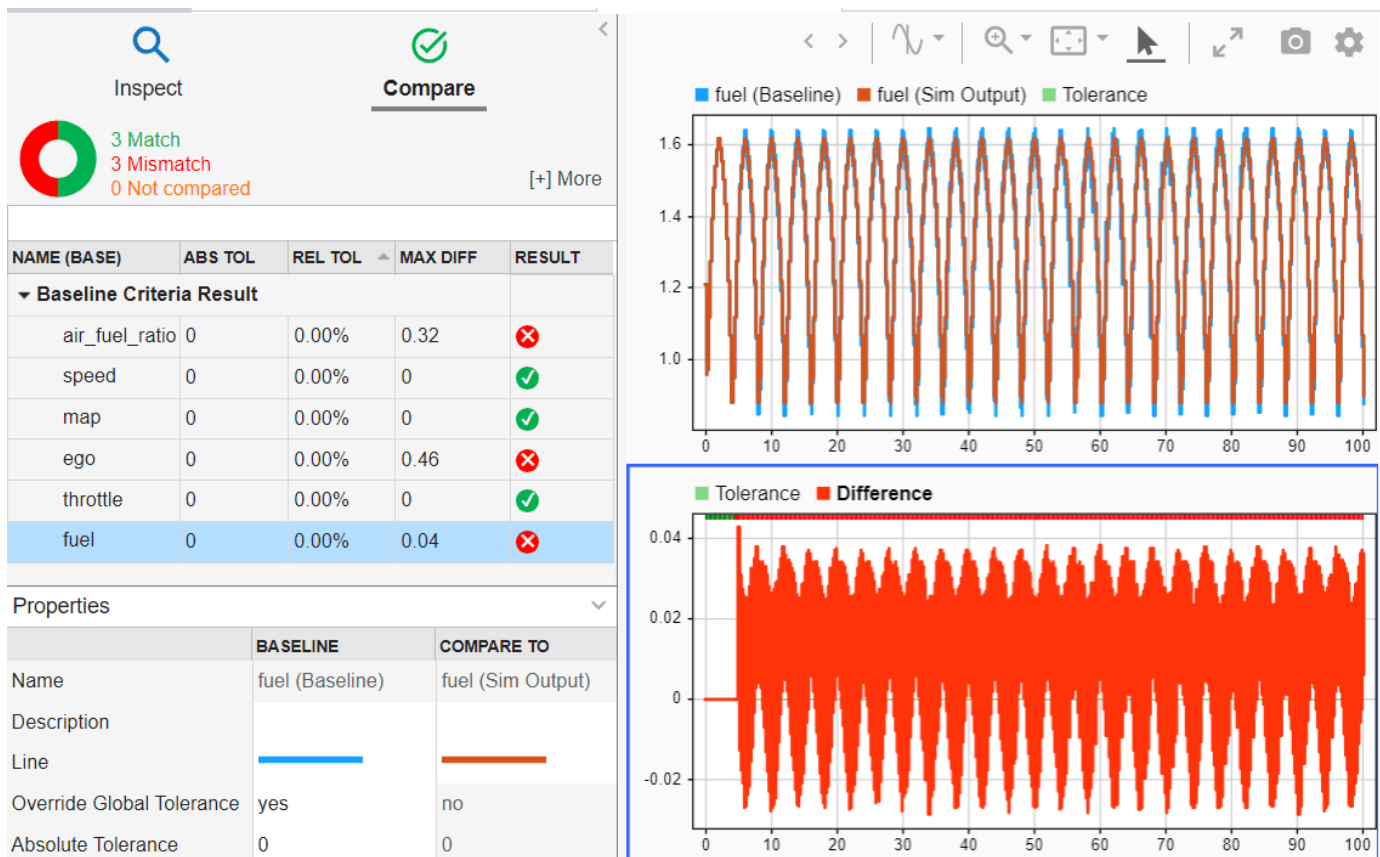
Observe that four signals have failed: `air_fuel_ratio`, `ego`, and `fuel`. This example uses the failed `fuel` signal to illustrate the debugging workflow.

Step 2: Entering Debug Session

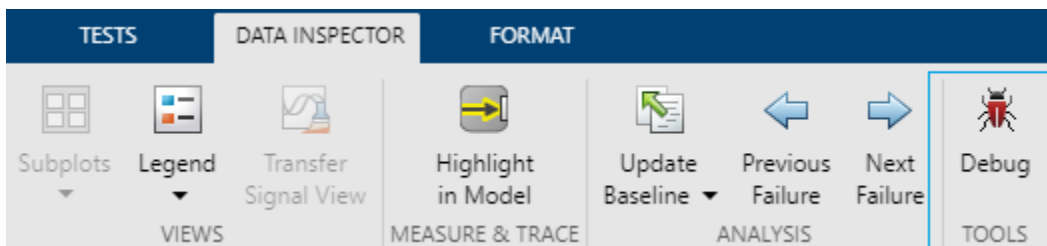
This section describes how to setup the Model Slicer for debugging the failed `fuel` signal.

1. To compare the `fuel` signals between the model and the baseline, select the **Baseline Criteria Result** and select the `fuel` signal in the Compare pane. Likewise, to debug a verify signal, expand the **Verify Statements** and select the failed verify signal. Another way to select a failed signal is from the **Signal to Debug** dropdown list in the toolbar.

In the plot area, compare the model output to the baseline data.

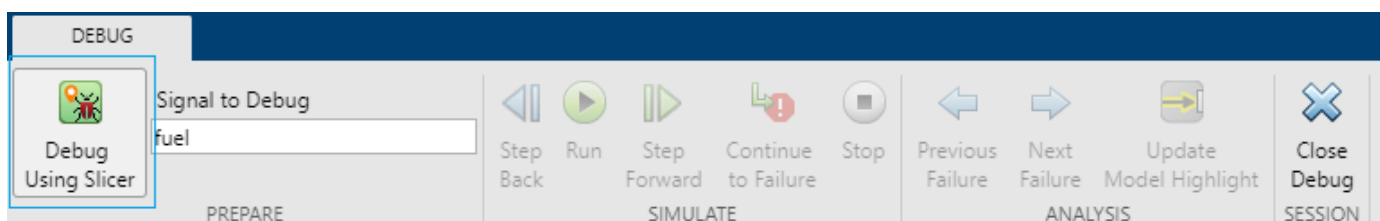


2. Click **Debug** in the TOOLS section of the toolbar. Note that the **Debug** option is enabled only when a failed baseline or verify signal is plotted.



The **DEBUG** tab replaces all existing toolbar tabs. Multiple Test Manager options are hidden or disabled to create the debug environment.

3. To set up the Model Slicer, click **Debug Using Slicer**.



Debug Using Slicer prepares the debugging session by:

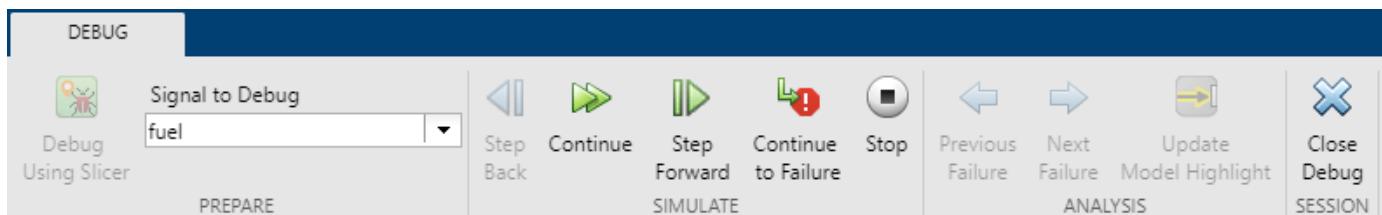
- 1** Rerunning the test case and creating new debugging results. This makes sure that the failure still exists in the current state of the test model.
- 2** Launching the Model Slicer on the test model.
- 3** Automatically plotting the selected failed signal in the debugging results, and setting the failed signal as the starting time point.
- 4** Pausing the simulation at the model start time to continue debugging.

Step 3: Debugging using Model Slicer

This section explains how to focus in on the reason for the failure by using the debugging features of the Model Slicer.

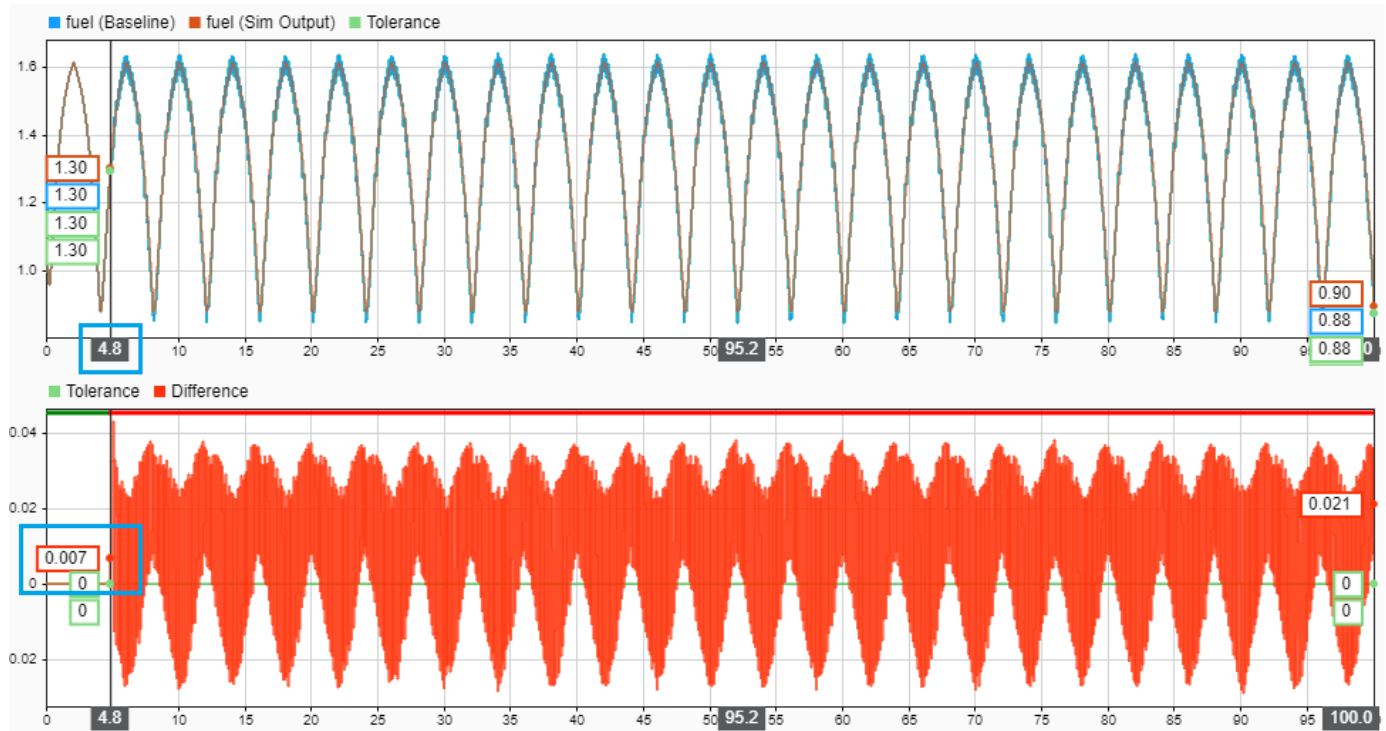
1. Use the **Step Back/Step Forward** buttons to move one step back or forward in simulation time. The left data cursor moves to the current simulation time. Observe the changes in the data dependencies by noting the changed model highlighting and port value labels for the active signals at every time stamp.

You can also use **Run/Continue/Stop** to run a new simulation, or complete or stop the current simulation.

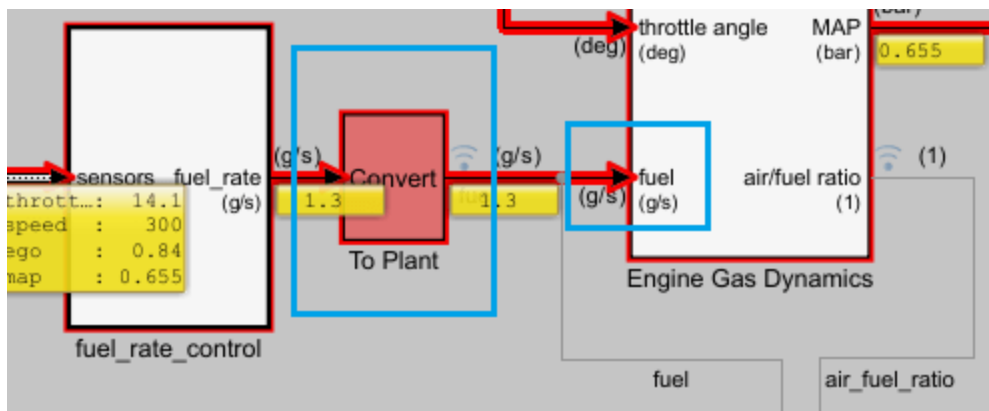


- Simulation pauses at $T = 4.81$.

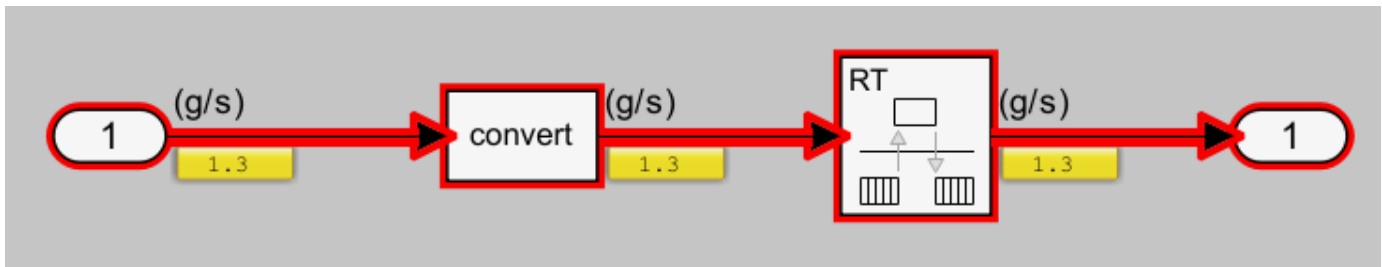
- Data cursors update accordingly.
- Difference between the Baseline and Sim Output is 0.007.



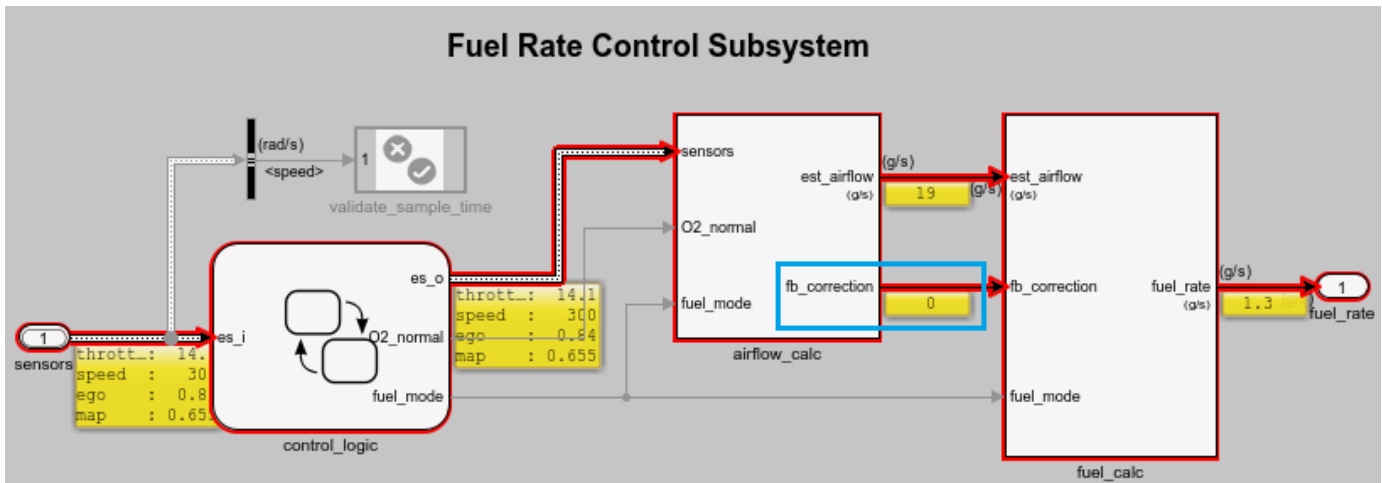
From the Model Slicer highlighting, you can find the cause of this difference, and see that the `sltestDemo_fuelsys/To Plant/fuel` value depends on `sltestDemo_fuelsys/To Plant`.



3. Open `sltestDemo_fuelsys/To Plant`. Notice that there is no change in the value being propagated.

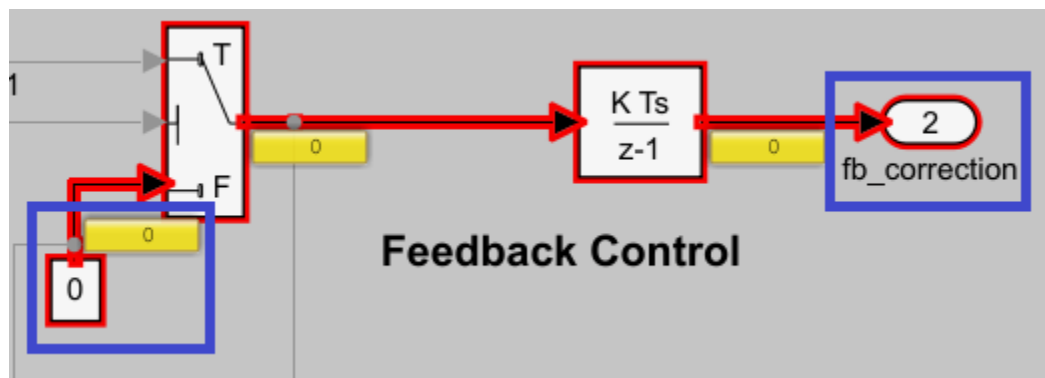


4. Open `sltestDemo_fuelsys/fuel_rate_control`.



Observe that the `fb_correction` value is 0. The difference between the Baseline and the Sim Output is 0.007, which is a small value. It might be that `fb_correction` is not calculated correctly.

5. Open `sltestDemo_fuelsys/fuel_rate_control/airflow_calc`, which computes the `fb_correction`, and observe the data dependencies.

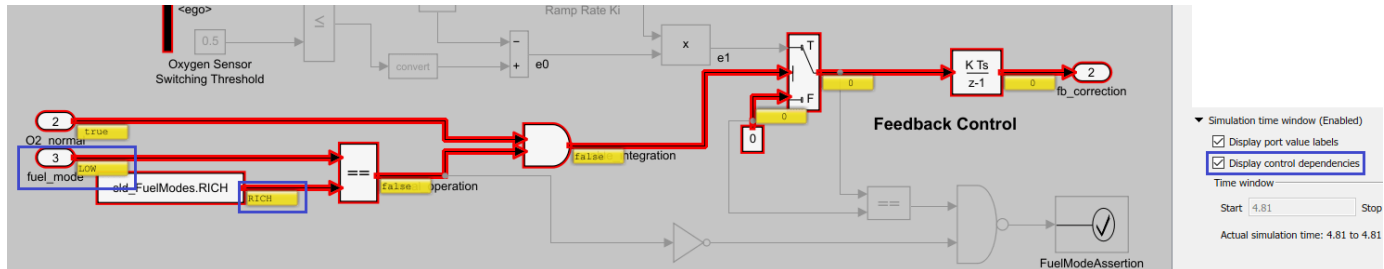


Notice that the constant, 0, is being passed through the `sltestDemo_fuelsys/fuel_rate_control/airflow_calc/hold` integrator switch block. To determine why the control port evaluates to false for the switch block, the control dependencies need to be highlighted on the model.

6. Enable **Display Control Dependencies** from the **Simulation Time Window** section in the Model Slicer Dialog docked on the model.

Observe that:

- `sltestDemo_fuelsys/fuel_rate_control/airflow_calc/fuel_mode` value is LOW, but `fb_correction` is still **zero**.
- `fuel_mode` is compared to `sltestDemo_fuelsys/fuel_rate_control/airflow_calc/Enumerated Constant`, which evaluates to false.



The Enumerated Constant value is set to `sld_FuelModes.RICH`. It should be checking against the `fuel_mode` value `sld_FuelModes.LOW`.

Step 4: Incorporating the fix

1. Exit the debugging session by clicking **SESSION > Close Debug**.
2. Open the model and update the `sltestDemo_fuelsys/fuel_rate_control/airflow_calc/Enumerated Constant` value to `sld_FuelModes.LOW`.
3. Save the model.
4. Run the test case and view the results.

Now, observe that the test results show the test as having passed.

Capabilities and Limitations

- If you use the Test Manager to set a simulation mode to one other than normal mode, such as SIL or PIL, you cannot use the Model Slicer for debugging.
- If the simulation mode is set in the model configuration, the Model Slicer changes the mode of the model and all referenced models to run in normal mode and then, you can use the Model Slicer for debugging.
- For models that do not support Fast Restart mode, the SIMULATION section of the toolbar is disabled. Use the ANALYSIS section to debug the failure.
- The ANALYSIS section is available only when the model is not simulating, such as when you click **Continue** or **Stop** in the SIMULATION section. It highlights a time region instead of a time step. To define a time region, move the data cursors manually, or use **Next Failure** or **Previous Failure**. Then, you can use **Update Slicer Highlight** to update the model highlighting for the defined time slice.

- The Results must be generated from the current release.

See Also

More About

- “Debugging Equivalence Test Failures Using Model Slicer” on page 7-54
- “Model Slicer Considerations and Limitations” (Simulink Check)
- “Refine, Test, and Debug a Subsystem” on page 2-22

Export Test Results

Once you have run test cases and generated test results, you can export results and generate reports. Test case results appear in the **Results and Artifacts** pane.

Test results are saved separately from the test file. To save results, select the result in the Test Manager, in the **Results and Artifacts** pane, and click **Export** on the toolbar.

- Select complete result sets to export to a MATLAB data export file (MLDATX).

NAME	STATUS
Results : 2015-Jan-16 11:18:26	1 ✖
Slip Baseline Test	✖
Baseline Criteria Result	✖
Sim Output (sldemo_absbrake :	

Note When you export the results into an MLDATX file, the corresponding test cases are not exported. When you reimport the result set in the Test Manager, the connections from the test case results to the original test cases are not preserved.

- Select criteria comparisons or simulation output to export signal data to the base workspace or to a MAT-file.

NAME	STATUS
Results : 2015-Jan-16 11:18:26	1 ✖
Slip Baseline Test	✖
Baseline Criteria Result	✖
Sim Output (sldemo_absbrake :	

See Also

Related Examples

- “Generating a Test Results Report” on page 7-20

More About

- “Create, Set Options, and Use Templates for Test Result Reports” on page 7-17

Create, Set Options, and Use Templates for Test Result Reports

Create a Test Results Report

Result reports contain report overview information, the test environment, results summaries with test outcomes, comparison criteria plots, and simulation output plots. You can customize the information included in the report, and you can save the report in three different file formats: ZIP (HTML), DOCX, and PDF.

- 1 In the Test Manager, in the **Results and Artifacts** pane, select results for a test file, test suite, or test case.

Note You can create a report from multiple result sets, but you cannot create a report from multiple test files, test suites, or test cases within results sets.

- 2 In the toolstrip, click **Report**.
- 3 Enter the title page information and select the information you want to include in the report. To enable the option to specify the number of plots per page, select **Plots for simulation output and baseline**.
- 4 Select the **File Format** to use for the generated file and, if desired, change the default **File Name** and path.
- 5 If you created a customized report template, enter the path and name of the **Template File**. See "Generate Reports Using Templates" on page 7-17.
- 6 If you created a customized report class to change fonts, add tables, add model images, etc., enter the **Report Class** name. See the "Generate Customized Test Results Report" section of "Customize Test Results Reports" on page 7-21.
- 7 Click **Create**.

Save Reporting Options with a Test File

You can generate a report every time you run a test case in a test file, using the same report settings each time. To generate a report each time you run the test, set options under **Test File Options**. These settings are saved with the test file.

- 1 In the **Test Browser** pane, select the test file whose report options you want to set.
- 2 Under **Test File Options**, select **Generate report after execution**. The section expands, displaying the same report options you can set using the dialog box.
- 3 Set the options. To include figures generated by callbacks or custom criteria, select **MATLAB figures**. For more information, see "Create, Store, and Open MATLAB Figures" on page 6-206.
- 4 Store the settings with your test file. Save the test file.
- 5 If you want to generate a report using these settings, select the test file and run the test.

Generate Reports Using Templates

Microsoft Word Format

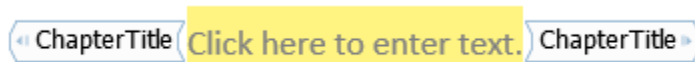
If you have a MATLAB Report Generator™ license, you can create reports from a Microsoft Word template. The resulting report is a Microsoft Word document.

The report generator in Simulink Test fills information into rich text content controls in your Microsoft Word template .dotx document. For more information on how to use rich text content controls or customize part templates, see the MATLAB Report Generator documentation, such as “Add Holes in Microsoft Word Templates” (MATLAB Report Generator).

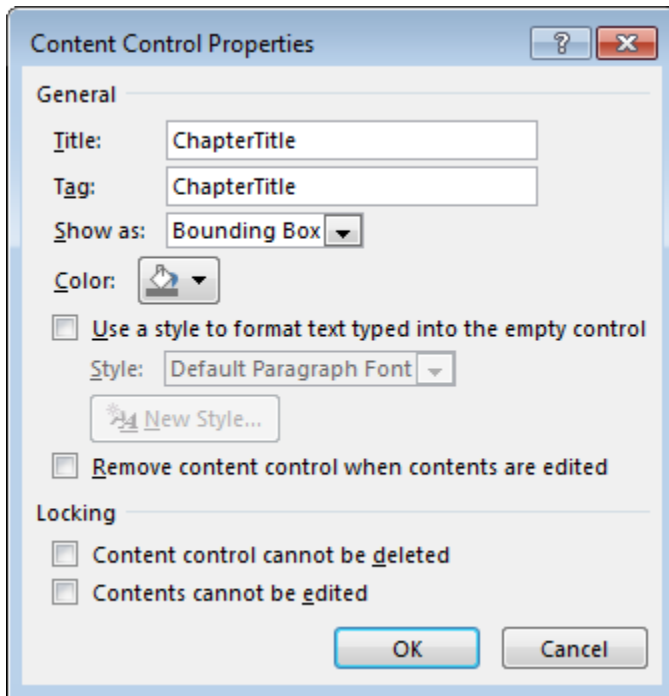
In a Microsoft Word template, you can add rich text content controls. Each Simulink Test report section can be inserted into the rich text content controls. The control names are:

- ChapterTitle — report title
- ChapterTestPlatform — version of MATLAB used to execute tests
- ChapterTOC — test results table of contents
- ChapterBody — test results

For example, the chapter title rich text content control appears in the Microsoft Word template as:



To change the control name, right-click the rich text content control and select **Properties**. Specify the control name, ChapterTitle or other name, in the **Title** and **Tag** field.



To generate a report from the Test Manager using a Microsoft Word template:

- 1 In the Test Manager, select the **Results and Artifacts** pane.
- 2 Select results for a test file, test suite, or test case.
- 3 In the toolbar, click **Report**.
- 4 Enter the title page information and specify the information you want to include in the report.

- 5 Select DOCX for the **File Format**.
- 6 Specify the full path and filename of your Microsoft Word template in the **Template File** field.
- 7 Click **Create**.

PDF or HTML Formats

If you have a MATLAB Report Generator license, you can create reports from a PDF or HTML template by using a PDFTX or HTMTX file. To generate a report from the Test Manager using a PDF or HTML template:

- 1 In the Test Manager, select the **Results and Artifacts** pane.
- 2 Select results for a test file, test suite, or test case.
- 3 In the toolbar, click **Report**.
- 4 Enter the title page information and specify the information you want to include in the report.
- 5 Select ZIP or PDF for the **File Format**. Selecting ZIP generates an HTML report.
- 6 Specify the full path and filename of your template in the **Template File** field. For PDF, use a PDFTX file. For HTML, use an HTMTX file. For more information on creating templates, see “Templates” (MATLAB Report Generator).
- 7 Click **Create**.

See Also

Related Examples

- “Generating a Test Results Report” on page 7-20
- “Templates” (MATLAB Report Generator)
- “Create, Store, and Open MATLAB Figures” on page 6-206

More About

- “Export Test Results” on page 7-16

Generating a Test Results Report

This example shows how to generate a test results report from the Test Manager using a baseline test case. The model used for this example is `sltestTestManagerReportsExample`.

Load and run the test file

Load and run the test file programmatically. The test file contains a baseline test case.

```
exampleFile = 'sltestTestManagerReportsTestSuite.mldatx';
sltest.testmanager.load(exampleFile);
baselineObj = sltest.testmanager.run;
```

Generate the report

You can generate the test results report programmatically or by using the Test Manager interface.

To generate the report programmatically:

Generate a report of the test case results using the results set object. The report is saved as a ZIP and shows all the test results. The report opens when it is completed.

```
sltest.testmanager.report(baselineObj, 'baselineReport.zip', ...
    'IncludeTestResults', 0, 'IncludeComparisonSignalPlots', true, ...
    'IncludeSimulationSignalPlots', true, 'NumPlotRowsPerPage', 3);
```

The `report.html` file is stored in the `baselineReport.zip` folder.

Alternatively, to generate the report using the Test Manager:

1. Open the Test Manager using `sltest.testmanager.view`.
2. In the **Test Browser** pane, select the Test Suite and click **Run**.
3. After the test runs, in the Results and Artifacts pane, select Results.
4. In the toolbar, click **Report**.
5. In the Create Test Result Report dialog box, select the items to include in the report and click **Create**.

To customize the report, see “Customize Test Results Reports” on page 7-21.

```
sltest.testmanager.clear;
```

See Also

`sltest.testmanager.report`

More About

- “Create, Set Options, and Use Templates for Test Result Reports” on page 7-17

Customize Test Results Reports

In this section...

"Customize Reports Using a Custom Class" on page 7-21

"Customize Reports Using a Custom Template File" on page 7-29

"Customize a Test Results Report" on page 7-29

You can choose how to format and aggregate test results by customizing reports. You can customize the report by using a custom class, a custom template file, or a combination of both. The custom class controls the content and the formatting of the output from Simulink® Test™, while the custom template file controls the content and the formatting of the report file. For example, you can use a custom class to only include failed tests in the report or to make the test results a certain font size. You can use a custom template file to add a cover page with custom text and control the font of that custom text.

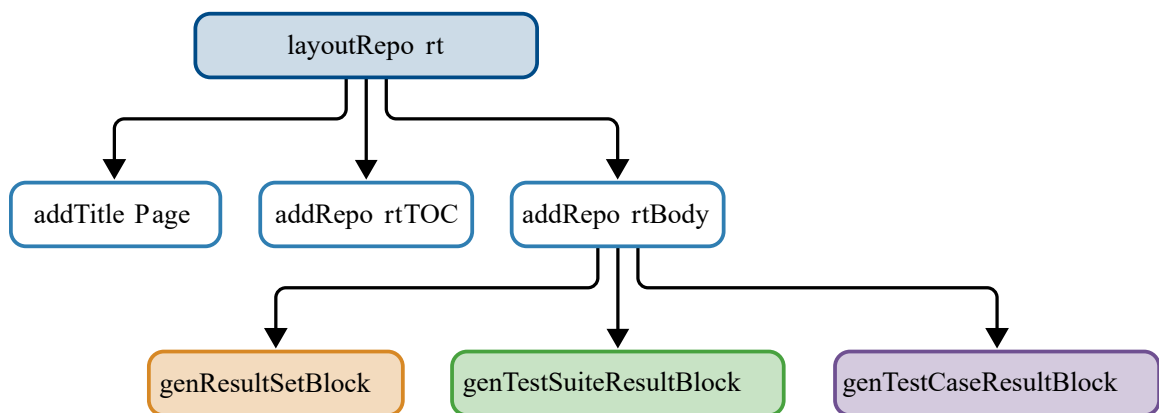
Customize Reports Using a Custom Class

Use the `sltest.testmanager.TestResultReport` class to create a subclass and then use the properties and methods to customize how the Test Manager generates the results report. You can change font styles, add plots, organize results into tables, include model images, and more. Using the custom class, requires a MATLAB Report Generator license.

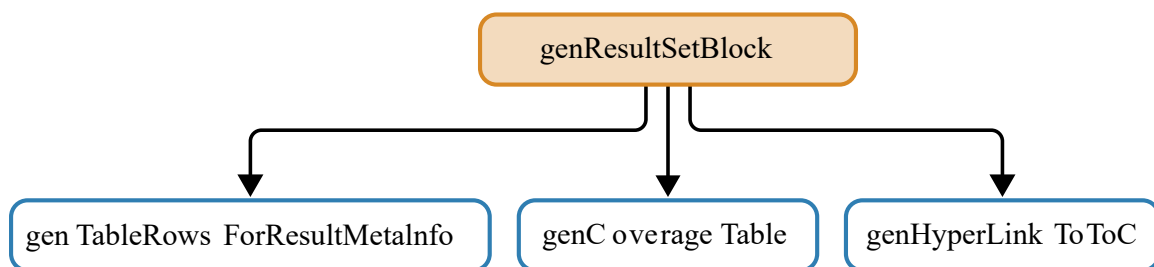
To customize the generated report, you must inherit from the `sltest.testmanager.TestResultReport` class. After you inherit from the class, you can modify the properties and methods. To inherit the class, add the class definition section to a new or existing MATLAB script. The subclass is your custom class name, and the superclass that you inherit from is `sltest.testmanager.TestResultReport`. For more information about creating subclasses, see "Design Subclass Constructors". Then, add code to the inherited class or methods to create your customizations.

Method Hierarchy

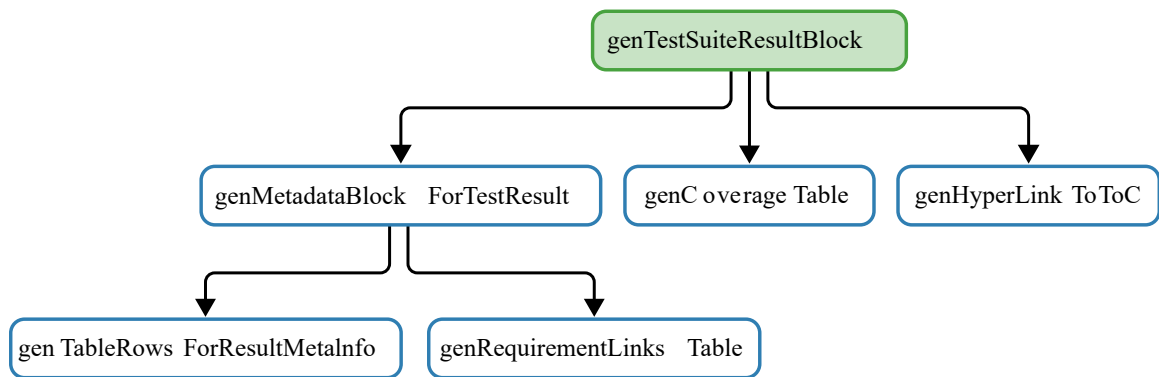
When you create the subclass, the derived class inherits methods from the `sltest.testmanager.TestResultReport` class. The body of the report is separated into three main groups: the result set block, the test suite result block, and the test case result block.



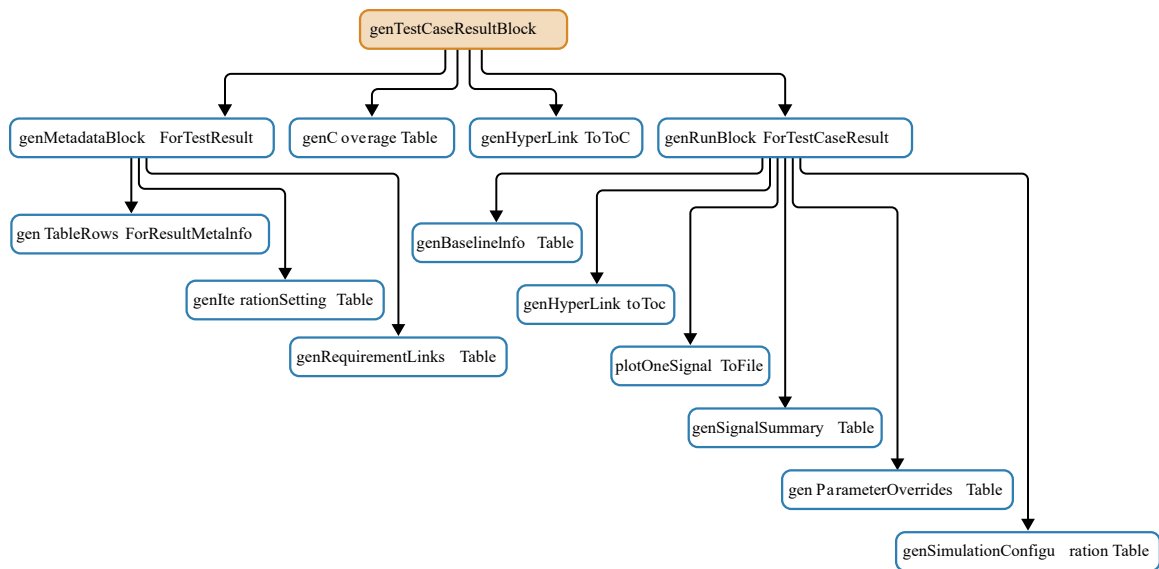
The result set block contains the result set table, the coverage table, and links to the table of contents.



The test suite result block contains the test suite results table, the coverage table, requirements links, and links to the table of contents.



The test case result block contains the test case and test iterations results table, the coverage table, requirements links, signal output plots, comparison plots, test case settings, and links to the table of contents.



Customize Reports Using a Custom Template File

You can generate reports as HTML, PDF, or DOCX files. You can customize the template files for the reports to add content, remove content, change the formatting, or change the section ordering of a report. For more information about customizing template files, see “Modify Styles in Microsoft Word Templates” (MATLAB Report Generator), “Modify Styles in HTML Templates” (MATLAB Report Generator), and “Modify Styles in PDF Templates” (MATLAB Report Generator).

Customize a Test Results Report

This example shows how to customize a test results report by using a custom class, a custom template file, or a combination of both.

Use a Custom Class to Customize a Test Results Report

You can customize your report by creating a subclass of the `sltest.testmanager.TestResultReport` and modifying the methods and properties. You can change the report title, plots, tables, headers, icons, and other aspects of the report. This example has a custom class file called `CustomReport` that includes modified properties and methods.

To customize the class, define a new class and inherit from the `sltest.testmanager.TestResultReport` class. In this example, the new class is `CustomReport`.

```
classdef CustomReport < sltest.testmanager.TestResultReport
```

To modify properties with protected `SetAccess`, add a class constructor method to the newly defined class. Then set the values of the properties you want to change. In this example, `CustomReport` changes the body font size to 11 pt.

```
methods
    function this = CustomReport(resultObjects, reportFilePath)
        this@sltest.testmanager.TestResultReport(resultObjects,reportFilePath);
        this.BodyFontSize = "11pt";
    end
end
```

To modify properties with public `SetAccess`, when generating the report programmatically, include the property name and new value in your `sltest.testmanager.report` command. Alternatively, when generating the report with the Test Manager, specify the values of these properties in the Create Test Results Report window.

To change or add content to the report, modify the inherited class methods. In this example, `CustomReport` modifies the `addTitlePage` method to include a custom label on the title page of the report.

```
methods(Access=protected)
    function addTitlePage(obj)
        import mlreportgen.dom.*;

        % Call the superclass method to get the default behavior
        addTitlePage@sltest.testmanager.TestResultReport(obj);

        % Add a custom message
        label = Text("Some custom content can be added here");
```

```
        append(obj.TitlePart,label);
    end
end
```

Additionally, lines 10 through 66 in `CustomReport` modify the `genTestCaseResultBlock` method to add a model screenshot.

Use a Custom Template File to Customize a Test Results Report

You can also use a custom template file to customize the report. To customize a PDF template file, follow these steps:

1. To access the template file, unzip the PDF template file.

```
unzip('dCustomReportTemplate_pdf_sample1.pdf');
```

Unzipping the file creates a `root.html` file and a `/stylesheets/root.css` file in the new `CustomReportTemplate_pdf_sample1` folder.

2. Open and edit the `root.html` file using a text editor. This file lists the content holes in the order in which the content appears in the report. In this file, you can reorder the report sections and delete template holes.

3. In the `stylesheets` folder, open and edit the `root.css` file using a text editor. In this file, you can change the table borders, font size, text color, and other styles for content added in the `root.html` file. To learn more about modifying report styles, see “Modify Styles in PDF Templates” (MATLAB Report Generator).

4. Zip the files into the `dCustomReportTemplate_pdf_sample1.pdf` file.

```
zipTemplate('dCustomReportTemplate_pdf_sample1.pdf');
```

For information on customizing Word or HTML templates, see “Modify Styles in Microsoft Word Templates” (MATLAB Report Generator) or “Modify Styles in HTML Templates” (MATLAB Report Generator), respectively.

The default PDF report uses this HTML code:

```
<html>
<head>
    <meta charset="utf-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <title>Report Template</title>
    <link rel="StyleSheet" href="./stylesheets/root.css" type="text/css" />
</head>

<body>
    <hole id="ChapterTitle"/>

    <pagebreak/>
    <hole id="ChapterTOC"/>

    <pagebreak/>
    <hole id="ChapterBody"/>

</body>
</html>
```

The template file in this example modifies the HTML code in the `<body>` section of the `root.html` file to add page numbers, a cover page with the MathWorks logo, a table, and custom text.

This code adds the page numbers:

```
<layout style="page-margin: 1in 1in 1in 1in 0.5in 0.5in 0in; page-size: 8.5in 11in portrait" >
    <pfooter template-name="Footer">
    </pfooter>
    <pnumber initial-value="1" format="1" />
</layout>
```

This code adds the logo:

```
<div align="center">
    
    
</div>
```

This code adds the title for the table:

```
<h1 align="center">Information captured in this report</h1>
```

The code in lines 23 through 82 adds the table. This table is hard-coded and does not reflect the content of the test results report. However, it is an example of how to create a table in the template file.

This code adds the custom text:

```
<br/><br/>
<span class="loremipsum">
    Lorem ipsum dolor sit amet.
</span>
```

Generate Customized Test Results Report

You can generate the test results report programmatically or by using the Test Manager interface.

To generate the report programmatically:

1. Load and run the tests in Test Manager.

```
exampleFile = 'sltestTestManagerReportsTestSuite.mldatx';
sltest.testmanager.load(exampleFile);
results = sltest.testmanager.run;
```

2. Specify the file name and extension for the report.

```
filePath = 'Report.pdf';
```

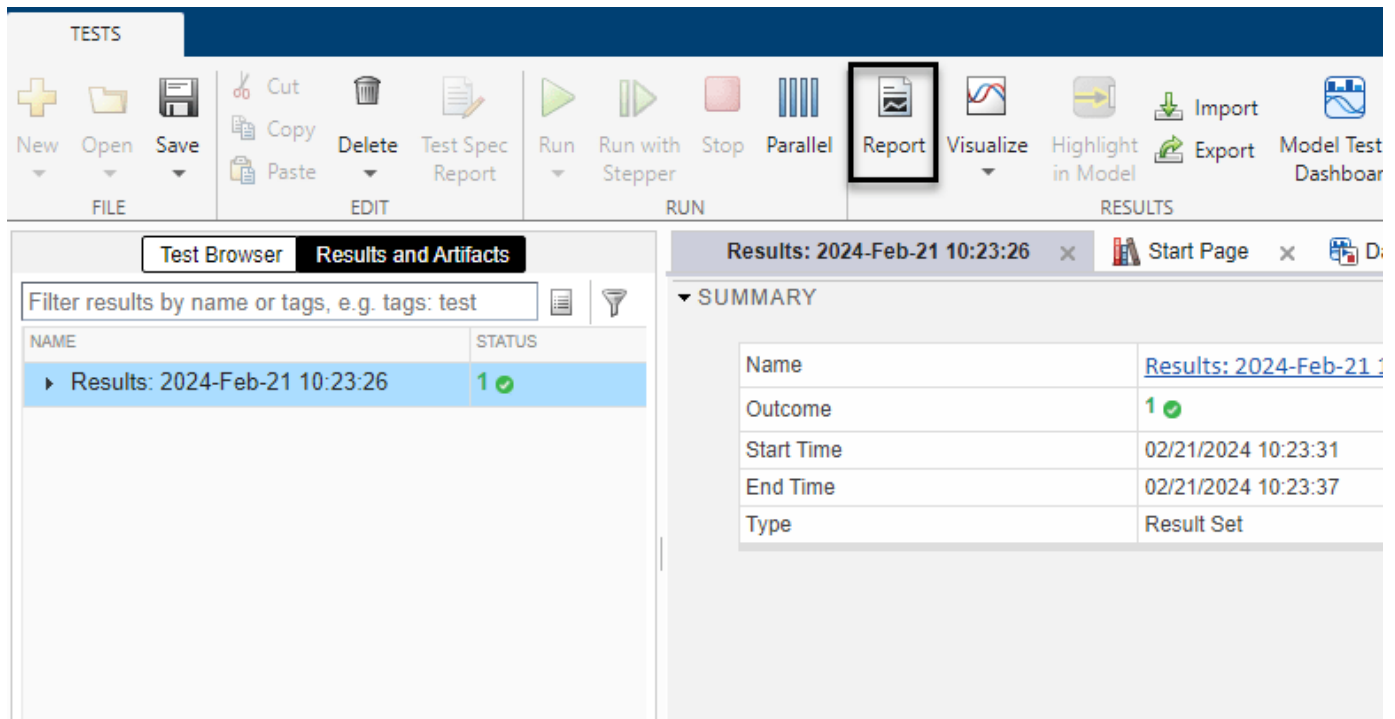
3. Generate the report using the custom class and custom template file. You can also modify other properties with public `SetAccess`, such as `IncludeCoverageResult`, as name-value arguments.

```
sltest.testmanager.report(results,filePath,...
    IncludeTestResults=0, ...
    IncludeComparisonSignalPlots=true, ...
    IncludeSimulationSignalPlots=true, ...
    NumPlotRowsPerPage=3, ...
    CustomReportClass="CustomReport", ...
```

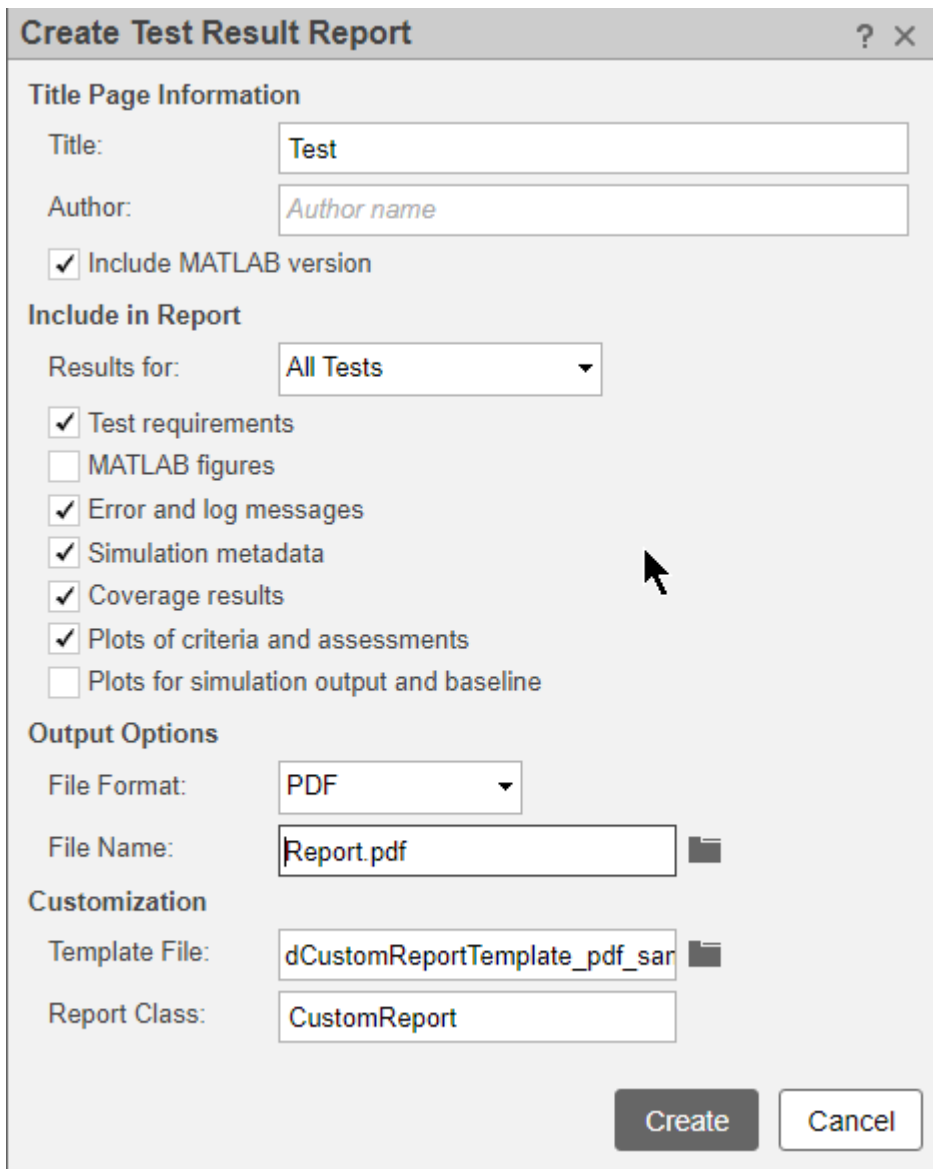
```
IncludeCoverageResult=true,...
CustomTemplateFile="dCustomReportTemplate_pdf_sample1.pdf tx")
```

To generate the report by using the Test Manager:

1. Double-click on the test file, `sltestTestManagerReportsTestSuite.mldatx`, in the current folder to open it in the Test Manager.
2. In the left pane, select the test suite. Click **Run**.
3. In the left pane, select the test results. In the toolstrip, click **Report**.



4. In the **Customization** section of the Create Test Result Report dialog box, click on the folder icon next to **Template File**.
5. Navigate to the 'dCustomReportTemplate_pdf_sample1.pdf tx' file and click **Open**.
6. Set **Report Class** to `CustomReport`, which is the name of the custom class. You can also modify properties with public `SetAccess`, such as **Coverage Results**, in this window.



The image shows a 'Create Test Result Report' dialog box with the following sections and controls:

- Title Page Information**
 - Title: Text box containing 'Test'
 - Author: Text box containing 'Author name'
 - ☒ Include MATLAB version
- Include in Report**
 - Results for: Dropdown menu set to 'All Tests'
 - ☒ Test requirements
 - ☐ MATLAB figures
 - ☒ Error and log messages
 - ☒ Simulation metadata
 - ☒ Coverage results
 - ☒ Plots of criteria and assessments
 - ☐ Plots for simulation output and baseline
- Output Options**
 - File Format: Dropdown menu set to 'PDF'
 - File Name: Text box containing 'Report.pdf' with a folder icon to its right
- Customization**
 - Template File: Text box containing 'dCustomReportTemplate_pdf_sar' with a folder icon to its right
 - Report Class: Text box containing 'CustomReport'

At the bottom right are two buttons: 'Create' and 'Cancel'.

7. Click **Create**.

See Also

`sltest.testmanager.TestResultReport` | `sltest.testmanager.report`

Related Examples

- “Design Subclass Constructors”

Append Code to a Test Report

This example shows how to use a customization class to print integrated code in a test results report. If you test models that include handwritten code, you can print the code to a report to be reviewed with the test results.

The cruise control model integrates handwritten C code using an S-Function builder block. The C code is a utility function that disregards simultaneous pressing of two buttons: `Accel/Res` and `Coast/Set`.

This example requires Simulink® Report Generator™ and Microsoft® Windows.

Example Files

Before running this example, set the filenames.

```
rptCustom = 'textAppendReport.m';  
resultsFile = 'DoublePressSfcnSimTestResults';  
filePath = fullfile(tempdir, 'textAppendedReport.zip');
```

Report Customization Class

The report customization class `textAppendReport.m` appends the S-Function code to the end of the report body.

```
open(rptCustom)
```

Load the Results and Create the Report

1. Load the test results file.

```
result = sltest.testmanager.importResults(resultsFile);
```

2. Create the test report using the customization.

```
sltest.testmanager.report(result, filePath, ...  
    'CustomReportClass', 'textAppendReport', ...  
    'IncludeTestResults', 0)
```

3. The report appends the S-Function wrapper code:

S-Function Wrapper

```

/*
 * Include Files
 *
 */
#ifdef(MATLAB_MEX_FILE)
#include "tmwtypes.h"
#include "simstruc_types.h"
#else
#include "rtwtypes.h"
#endif

/* %%%-SFUNWIZ_wrapper_includes_Changes_BEGIN --- EDIT HERE TO _END */
#include <math.h>
#include "RejectDoublePress.h"
/* %%%-SFUNWIZ_wrapper_includes_Changes_END --- EDIT HERE TO _BEGIN */
#define u_width 1
#define y_width 1
/*
 * Create external references here.
 *
 */
/* %%%-SFUNWIZ_wrapper_externs_Changes_BEGIN --- EDIT HERE TO _END */
/* extern double func(double a); */
/* %%%-SFUNWIZ_wrapper_externs_Changes_END --- EDIT HERE TO _BEGIN */

/*
 * Output functions
 *
 */
void RejectDoublePress_sfund_outputs_wrapper(const boolean_T *AccelResSwIn,
                                             const boolean_T *CoastSetSwIn,
                                             boolean_T *AccelResSwOut,
                                             boolean_T *CoastSetSwOut)

```

For more information on report customization, see “Customize Test Results Reports” on page 7-21.

```
sltest.testmanager.clearResults;  
sltest.testmanager.close;
```

See Also

`sltest.testmanager.importResults` | `sltest.testmanager.report`

Results Sections

In this section...
"Summary" on page 7-38 "Test Requirements" on page 7-38 "Iteration Settings" on page 7-38 "Errors" on page 7-39 "Logs" on page 7-39 "Description" on page 7-39 "Variant Configuration Details" on page 7-39 "Fault Settings" on page 7-39 "Parameter Overrides" on page 7-40 "Coverage Results" on page 7-40 "Aggregated Coverage Results" on page 7-40 "Scope coverage results to linked requirements" on page 7-40 "Add Tests for Missing Coverage" on page 7-40 "Applied Coverage Filters" on page 7-40

Double-click a test case results in the **Results and Artifacts** pane to open a results tab and view the test case result sections. A baseline test case result is shown as an example.

Baseline Test Case ✕

SUMMARY

Name	Baseline Test Case
Outcome	1
Start Time	01/05/2016 21:38:14
End Time	01/05/2016 21:38:18
Type	Baseline Test
Test File Location	C:\MATLAB\Test File.mldatx
Test Case Definition	
Rerun Test Case	
▶ Simulation Metadata	

▶ TEST REQUIREMENTS

▶ ITERATION SETTINGS

▶ ERRORS

▶ LOGS

▶ DESCRIPTION

▶ COVERAGE RESULTS

Summary

For a selected test case, the **Summary** section includes the basic test information and the test outcome. For more information about the simulation, toggle the **Simulation Metadata** arrow to expand the section.

For a selected Results item, the **Summary** section includes information for the Result Set, which applies to all of its child test suites and test cases.

Test Requirements

A list of test requirements linked to the test case. See “Requirements” on page 6-175 for more information on linking requirements to test cases.

Iteration Settings

If you are using iterations to run test cases, then this section appears in the results. For more information about test iterations, see “Test Iterations” on page 6-139.

Errors

This section displays simulation errors captured from the Simulink Diagnostic Viewer. Errors from incorrect information defined in the test case and callback scripts are also shown here. To view simulation logs in the command window, enter this command at the command line before running the test:

```
sltest.testmanager.setpref('ShowSimulationLogs',...
    'IncludeOnCommandPrompt',true)
```

Logs

This section displays simulation warnings captured from the Simulink Diagnostic Viewer. To view simulation logs in the command window, enter this command at the command line before running the test:

```
sltest.testmanager.setpref('ShowSimulationLogs',...
    'IncludeOnCommandPrompt',true)
```

Description

You can include notes about the test results here. These notes are saved with the results.

Variant Configuration Details

Information about the variant configuration applied during the test appears in this section. The applied variant configuration is identified in the **Name** and **Description** columns.

The **Control Variables** table contains information about the variant control variables that constitute the applied variant configuration.

- **Name** — Name of the variable
- **Value** — Variable value used for the test
- **Variant Activation Time** —
 Variant Activation Time — If the variable is a `Simulink.VariantControl` object, this field shows the value of the `ActivationTime` property of the object.
- **Source** — Data source where the variable is defined

If a variant configuration is specified for a test case or iteration, then this section appears in the results summary.

For information, see “Variant Configurations”.

Fault Settings

List of the fault sets used in the test case or test iteration. Expand a fault set to see the name of the model elements that have faults, the active fault on each model element, and the trigger that each fault uses. For more information, see [Define and Model Faults](#).

Parameter Overrides

List of the parameter overrides specified in the test case under **Parameter Overrides**. If parameter overrides are not specified, then this section is not shown in the results summary.

Coverage Results

If you collect coverage in your test, then the coverage results for the selected test case results appear in this section. Coverage results are aggregated at the test file level. For more information about coverage, see “Collect Coverage in Tests” on page 6-151.

Aggregated Coverage Results

At the Results level, lists the model analyzed for test coverage and includes a link to generate a coverage report. This section also reports the complexity level and the decision and execution percentages.

Scope coverage results to linked requirements

Controls whether coverage results include all executed items or only executed items that are directly linked to requirements. If not selected, coverage results include all executed items. If selected and the results set contains more than one simulation, such as multiple test cases or iterations, coverage results are shown for tests explicitly linked to requirements in those test cases or iterations. Requirements linked to steps in a Test Sequence block are not included in the coverage results.

Add Tests for Missing Coverage

Generate tests for missing coverage using Simulink Design Verifier. To add an iteration to an existing test case, select the test case name in Test Case. To create a new test case, select <Create a new test case> and specify the Test Type and Test Filename. See “Increase Test Coverage for a Model” on page 6-163 and “Increase Coverage by Generating Test Inputs” on page 6-192.

Applied Coverage Filters

At the Results level, lists the filter files applied to the coverage results shown in the Aggregated Coverage Results section.

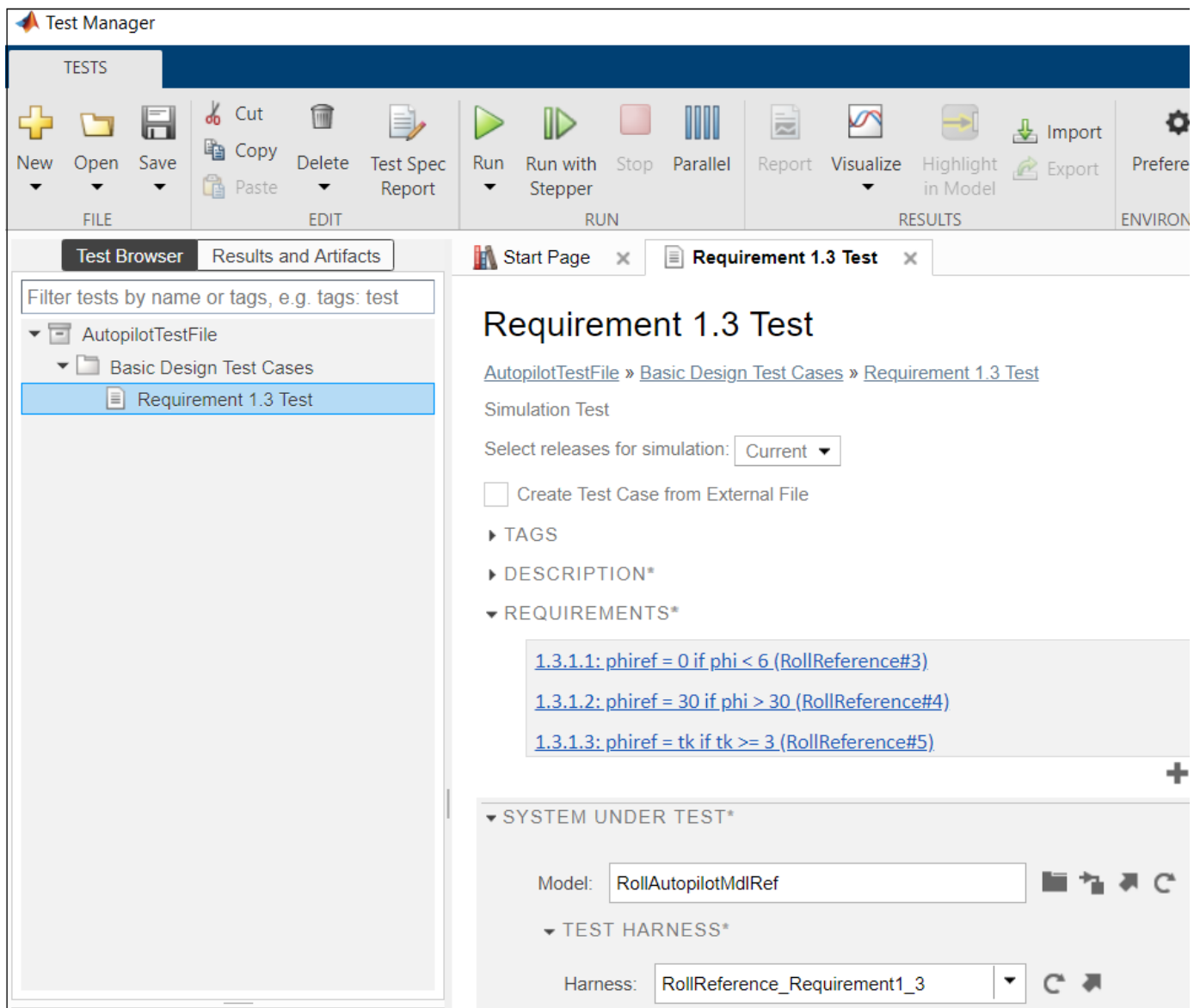
Generate Test Specification Reports

Test specification reports are reports of the test settings and parameters used for test cases, test suites, or test files. Common uses for these reports are capturing information for test procedure design reviews and archiving test information. You can create the report before or after running a test. In addition to using the Test Manager to create the report, you can create the report programmatically. See the `sltest.testmanager.TestSpecReport` reference page for examples.

For a test specification report, all of the items you select must be of the same type, either test files, test suites, or test cases. If you select a mixture of test files, suites, and cases, the **Test Spec Report** button and the context menu **Create Report** option are dimmed. If you select a test file, the report includes all of its test suites and test cases. If you select a test suite, the report includes all of its test cases.

This example uses an existing test file (`AutopilotTestFile.mldatx`), which was created for the `RollAutopilotMdlRef.slx` model and its `RollReference_Requirement1_3` test harness.

- 1 To change to the folder that contains the `RollAutopilotMdlRef.slx` model and `AutopilotTestFile.mldatx` test file, enter `openExample('RollAutopilotMdlRef')` on the MATLAB command line.
- 2 To open the Test Manager, enter `sltest.testmanager.view` on the MATLAB command line.
- 3 Click **Open**.
- 4 In the Open File dialog box, open the `matlab/examples/simulink/main` folder and select `AutopilotTestFile.mldatx`.
- 5 Highlight the Requirement 1.3 Test test case and click **Test Spec Report**.



- 6 In the Create a Test Specification Report dialog box, specify the **Title** as RollAutopilot Model Test Specification Report and the **Author** as John Smith.

Create a Test Specification Report ? x

Title Page Information

Title: RollAutopilot Model Test Specification Report

Author: John Smith

Include in Report

☒ Test Details	☒ Iterations
☒ Logged Signals	☒ External Inputs
☒ Callback Scripts	☒ Parameter Overrides
☒ Coverage Settings	☒ Logical and Temporal Assessments
☒ System Under Test	☒ Configuration Settings
☒ Custom Criteria	

Output Options

File Format: PDF

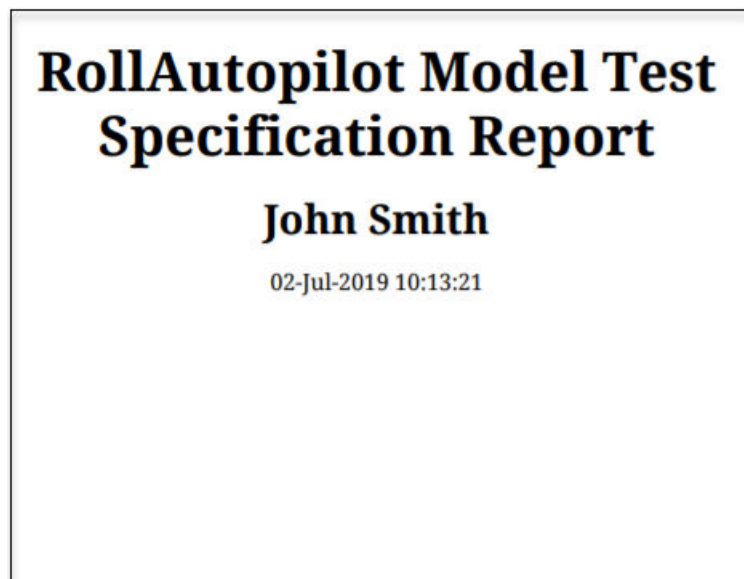
File Name: mynewReport.pdf

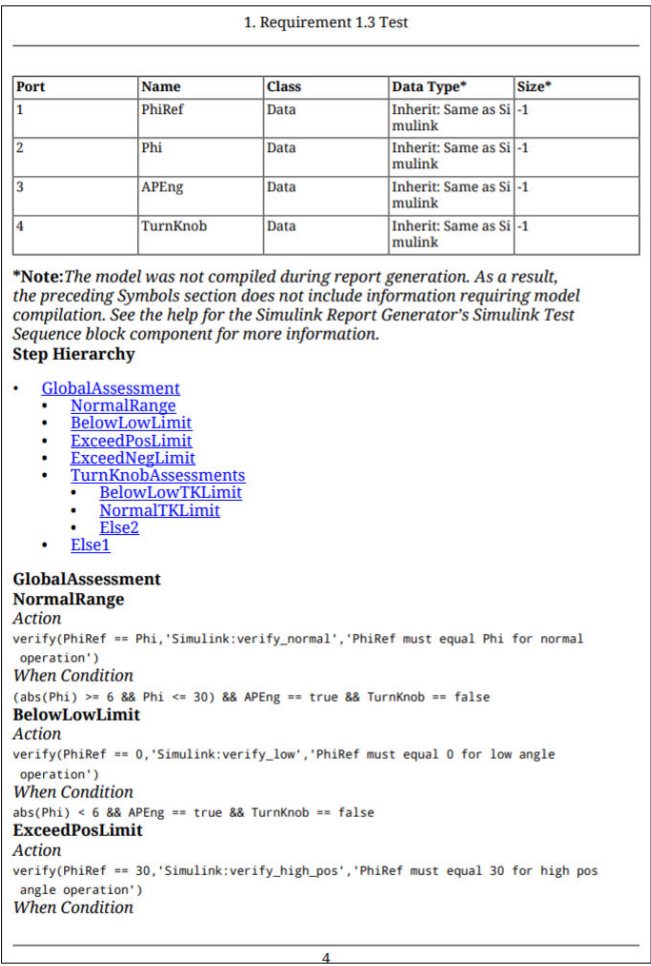
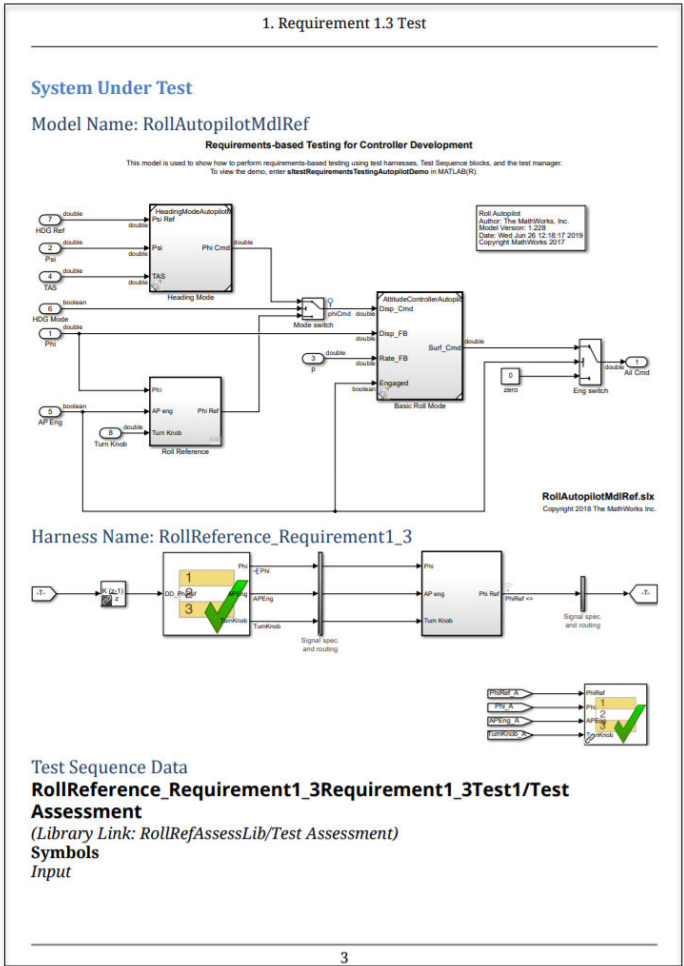
Customization Templates

Test Case Reporter: Select TestCaseReporter Template (optional)

Create Cancel

- 7 Leave all the report sections selected by default in the **Include in Report** section.
- 8 Leave PDF as the default output format.
- 9 Specify the filename for the saved report as `mynewReport.pdf` in a writable folder. If your current working folder is not writable, use a full pathname to a writable folder.
- 10 Leave the Test Case Reporter field blank because this report uses the default test case template.
- 11 Click **Create** to generate the report and open it automatically. These images from the report show the title page, images of the model and harness, and test inputs and assessment information.





See Also

`sltest.testmanager.TestSpecReport`

More About

- “Customize Test Specification Reports” on page 7-45

Customize Test Specification Reports

In this section...

“Update PDF or HTML Reports” on page 7-45

“Update Word Reports” on page 7-48

“Add Content to Reports” on page 7-49

To customize the formatting, section ordering, or to remove content from a test specification report, you must first create a copy of the test case or test suite template. You can then modify the template and apply it to your test specification report. You use test suite templates for both test suites and test files. To add new content, create a new reporter and then, specify new holes to hold that content.

Note To customize a report, you must have a Simulink Report Generator license.

Update PDF or HTML Reports

This example creates a new test case reporter template for a PDF report. Use this process to remove content or change the formatting or order of a test specification report. The process is the same to create templates for HTML output and test suite reporter templates.

- 1 Create a copy of the default `TestCaseReporter` PDF template in the current working folder. This folder must be writable.

```
sltest.testmanager.TestCaseReporter.createTemplate(...
    'myCustomTCTemplate', 'pdf');
```

For PDF and HTML output, `createTemplate` creates a zipped file that contains the template files and other supporting files. For PDF output, the file extension is `.pdftx` and for HTML, it is `.htmltx`.

- 2 To access the separate template files, unzip the PDF template file.

```
unzipTemplate('myCustomTCTemplate.pdftx');
```

Unzipping the file creates a `docpart_templates.html` file and a `/stylesheets/root.css` file in the new `myCustomTCTemplate` folder. Both PDF and HTML reports use HTML template files.

- 3 Open and edit the `docpart_templates.html` file using a text editor. This file lists the content holes in the order in which the content appears in the report. Content holes are filled with the associated data when the report is generated. In this template file, you can reorder the report sections and delete template holes. This image shows a portion of the default `docpart_templates.html` file.

```

<html>
  <head>
    <meta charset="utf-8" />
    <title>Document Part Templates</title>
    <link rel="StyleSheet" href="./stylesheets/root.css" type="text/css" />
  </head>

  <body>
    <dplibrary>
      <dptemplate name="TestCaseReporter">
        <!-- filled with test details table -->
        <hole id="TestDetails">TEST_DETAILS</hole>

        <!-- filled with System Under Test -->
        <hole id="SystemUnderTest">SYSTEM_UNDER_TEST</hole>

        <!-- filled with Parameter Overrides -->
        <hole id="ParameterOverrides">PARAMETER_OVERRIDES</hole>

        <!-- filled with Callback scripts -->
        <hole id="Callbacks">CALLBACKS</hole>

        <!-- filled with External inputs table -->
        <hole id="ExternalInputs">EXTERNAL_INPUTS</hole>

        <!-- filled with Simulation Outputs -->
        <hole id="SimulationOutputs">SIMULATION_OUTPUTS</hole>

        <!-- filled with Config Settings Overrides -->
        <hole id="ConfigSettingsOverrides">CONFIG_SETTINGS_OVERRIDES</hole>
      </dptemplate>
    </dplibrary>
  </body>
</html>

```

For information on adding new content holes, see “Add Content to Reports” on page 7-49.

- 4 In the `stylesheets` folder, open and edit the `root.css` file using a text editor. This file controls the formatting of the report sections. You can change the table borders, font size, text color, and other styles. For example, to change the font size to 14 pixels, use `font-size: 14px;`. The font-size setting is shown in the second section of the file image. This image shows a portion of the default `root.css` file.

```

root.css * X
table.TestCase_TestDetailsTable, table.TestCase_CoverageSettingsTable, table.BaselineC
table.ExternalInputsTable, table.ParameterSetsTable, table.TargetSettingsTable, table.
    table-layout: fixed;
    width: 100%;
    border-collapse: collapse;
    border-style:solid;
    margin-top: 15px;
    margin-bottom: 25px;
}

table.TestCase_TestDetailsTable td, table.TestCase_CoverageSettingsTable td, table.Bas
table.ExternalInputsTable td, table.ParameterSetsTable td, table.TargetSettingsTable t
table.SymbolsMetadataTable td {
    border-collapse: collapse;
    border-style:solid;
    white-space: normal;
    text-align: start;
    font-size: 16px;
    font-style: normal;
    font-variant: normal;
    font-weight: normal;
    line-height: normal;
    padding: 10px;
    hyphenation: hyphen;
}

table.BaselineCriteriaTable th, table.EquivalenceCriteriaTable th, table.IterationsTab
table.AssessmentsTable th, table.SymbolsTable th {
    padding: 5px;
    border-style:solid;

```

To learn more about modifying report styles, see “Modify Styles in PDF Templates” (MATLAB Report Generator). For information on HTML styles, see “Modify Styles in HTML Templates” (MATLAB Report Generator).

- 5 Zip the files from the folder you used in Step 1 into the `myCustomTCTemplate.pdf` file.

```
zipTemplate('myCustomTCTemplate.pdf');
```

- 6 Use the custom template for your test specification PDF report by using either of these processes:

- Use `sltestmgr` to open the Test Manager and click **Test Spec Report** to open the Create a Test Specification Report dialog box. Add `myCustomTCTemplate.pdf` to the **Test Case Reporter** field.
- Use the `sltest.testmanager.TestSpecReport` function and specify the `myCustomTCTemplate.pdf` filename as an input argument.

For example, use this code to create a test specification report that uses your custom template for the `AutopilotTestFile.mldatx` test file,

```

openExample('GenerateATestSpecificationReportExample')
testfile = 'AutopilotTestFile.mldatx';
sltest.testmanager.load(testfile);

```

```
tfile = sltest.testmanager.TestFile(testfile);
tsuites = getTestSuites(tfile);
tcases = getTestCases(tsuites);

sltest.testmanager.TestSpecReport(tcases, 'testReport.pdf', ...
    'Author', 'John Smith', 'Title', 'Autopilot Test Spec Report', ...
    'LaunchReport', true, ...
    'TestCaseReporterTemplate', 'MyCustomTCTemplate.pdftx')

sltest.testmanager.clear;
sltest.testmanager.clearResults;
```

Update Word Reports

This example creates a new test case reporter template for a Microsoft Word report. Use this process to remove content or change the formatting or order of a test specification report content. Use the same process to create a test suite reporter template.

- 1 Create a copy of the default `TestCaseReporter` Word template in the current working folder. The new file has a DOTX file extension. This example creates a `myCustomTCTemplate.dotx` test case reporter template file in the current working directory.

```
sltest.testmanager.TestCaseReporter.createTemplate(...
    'myCustomTCTemplate', 'docx');
```

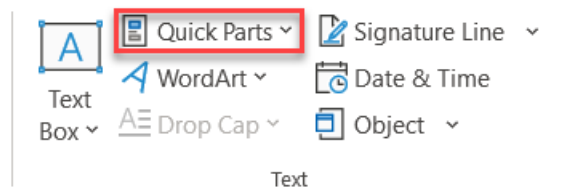
To create a `TestSuiteReporter` use

```
sltest.testmanager.TestSuiteReporter.createTemplate instead.
```

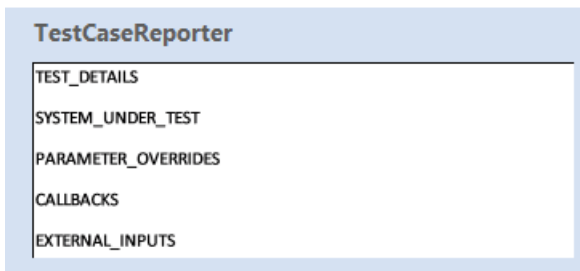
- 2 In the current working folder outside of MATLAB, right-click the `myCustomTCTemplate.dotx` file and click **Open**. The file appears to be blank but has defined holes that are not visible until you open the **Quick Parts**.

Note If you double-click on the file name to open the file, it opens a Word document file and not the template file.

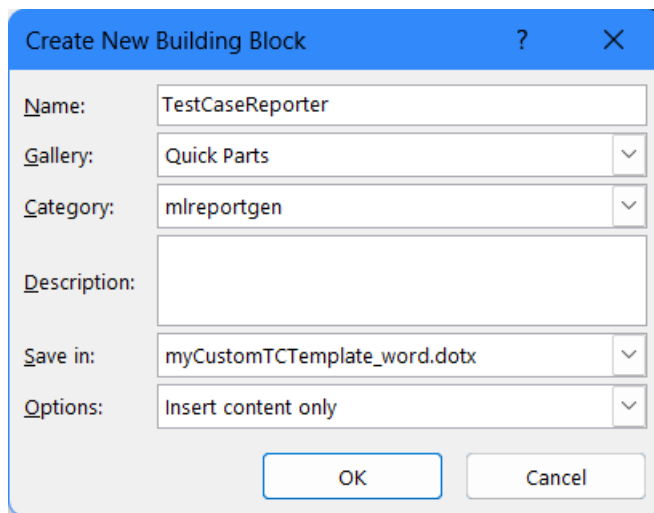
- 3 In the **Insert** tab, in the **Text** section, click **Quick Parts**.



- 4 In the Quick Parts library, click the **Test Case Reporter** section to populate the report with the default holes for the report.



- 5 Delete or move holes as desired. To change the formatting of the hole, see “Specify Default Styles for Text That Fill Holes” (MATLAB Report Generator).
- 6 Select all content in the template and reopen the Quick Parts library as described in step 3.
- 7 At the bottom of the panel, click **Save Selection to Quick Part Gallery**. The Create New Building Block dialog box opens.
- 8 Enter this required information:
 - **Name** — Name of the Quick Part library block. For this example, enter `TestCaseReporter`.
 - **Gallery** — Type of the selected content. Select `Quick Parts`.
 - **Category** — Category of the content. Select `mlreportgen`.
 - **Save in** — Name of the template file. For this example, select `myCustomTCTemplate.dotx`.
 - **Options** — How to insert the content. Select `Insert Content only`.



- 9 Click **OK** and then click **Yes** to confirm your changes.
- 10 Save the file as a DOTX file. You can then use the file as the template for your test case specification report.

For information on Word styles, see “Modify Styles in Microsoft Word Templates” (MATLAB Report Generator).

Add Content to Reports

To add new content to a Word, PDF, or HTML test specification report or override how content is added, create a subclass of the `sltest.testmanager.TestCaseReporter` or

`sltest.testmanager.TestSuiteReporter` class. Then add properties and methods for the new content in its class definition file. Add holes to hold that content in the test suite or test case templates.

This example describes how to create a new test case reporter for a test specification report. Use the same process to create a new test suite reporter.

- 1 To create a new test case reporter class, use the `customizeReporter` method of the `TestCaseCreate` reporter class. This command creates a new class folder in the current working folder. This new reporter inherits from the `TestCaseReporter` class.

```
customTCRptr = ...
    sltest.testmanager.TestCaseReporter.customizeReporter...
    ('@myTCReporter');
```

See “Subclass Reporter Definitions” (MATLAB Report Generator).

The `@myTCReporter` folder has a `myTCReporter.m` class definition file and a `resources` folder. The `resources` folder contains a `templates` folder, which contains folders and files for the report output types:

- `pdf` folder
 - `default.pdf.tx` — Zipped PDF template file. Unzip this file using `unzipTemplate` and then open the template file using a text editor. After editing, use `zipTemplate`.
- `docx` folder
 - `default.dotx` — Word template file. Outside of MATLAB open this template file by right-clicking and selecting **Open** from the context menu. If you click the filename to open it, the Word file associated with the template opens instead of the template file. See “Open Template Files” (MATLAB Report Generator).
- `html` folder
 - `default.html` — Single-file HTML template. Open this file using a text editor.
 - `default.html.tx` — Zipped HTML template file. Unzip this file using `unzipTemplate` and then open the template file using a text editor. After editing, zip the files using `zipTemplate`.

For information on templates, see “Templates” (MATLAB Report Generator).

- 2 In the `@myTCReporter` folder, open the class definition file `myTCReporter.m` in a text editor.

```

myTCReporter.m ×
classdef myTCReporter < sltest.testmanager.TestCaseReporter

    properties
    end

    methods
        function obj = myTCReporter(varargin)
            obj = obj@sltest.testmanager.TestCaseReporter(varargin{:});
        end
    end

    methods (Hidden)
        function templatePath = getDefaultTemplatePath(~, rpt)
            path = myTCReporter.getClassFolder();
            templatePath = ...
                mlreportgen.report.ReportForm.getFormTemplatePath(...
                    path, rpt.Type);
        end
    end

    methods (Static)
        function path = getClassFolder()
            [path] = fileparts(mfilename('fullpath'));
        end

        function createTemplate(templatePath, type)
            path = myTCReporter.getClassFolder();
            mlreportgen.report.ReportForm.createFormTemplate(...
                templatePath, type, path);
        end

        function customizeReporter(toClasspath)
            mlreportgen.report.ReportForm.customizeClass(...
                toClasspath, "myTCReporter");
        end
    end
end
end

```

- 3 To add new content, add a property and use the corresponding `get` method for that property in the customized class definition file. Then add the hole to the output type templates. For example, for a new section named References, this `myTCReporter.m` class definition file shows an added References property and a defined `getReferences` method.

```

myTCReporter.m
classdef myTCReporter < sltest.testmanager.TestCaseReporter

    properties
        References;
    end

    methods
        function obj = myTCReporter(varargin)
            obj = obj@sltest.testmanager.TestCaseReporter(varargin{:});
        end

        function content = getReferences(h,rpt)
            import mlreportgen.dom.*;
            heading = mlreportgen.dom.Heading4('References');
            heading.StyleName = 'ReferencesHeading';
            ol = OrderedList;
            listItem = ExternalLink(
                'https://www.mathworks.com/products/simulink-test.html',...
                'Simulink Test');
            append(ol,listItem);
            listItem = ExternalLink(
                'https://www.mathworks.com/products/SL_reportgenerator.html',...
                'Simulink Report Generator');
            append(ol,listItem);
            content = [{heading},{ol}];
        end
    end
end

```

Add `<hole id="References">REFERENCES</hole>` to the template files in the desired location to include the hole content in the generated report for each output type. See “Add Holes in HTML and PDF Templates” (MATLAB Report Generator) and “Add Holes in Microsoft Word Templates” (MATLAB Report Generator).

- 4 To override an existing method, add a function in the customized class definition file that defines the `get` method for the hole.

For example, for the `TestDetails` hole in the `TestCaseReporter`, create a method called `getTestDetails` in the customized `TestCaseReporter` class definition file. You do not need to add a property or hole because they are already specified in the `TestCaseReporter` class from which the customized reporter inherits.

- 5 To generate a report using the custom reporter, use Simulink Report Generator commands. See “Define New Reporters” (MATLAB Report Generator).

These sample commands create a PDF report for a test case. It uses the `myTCReporter` reporter, which takes a test case array (`test_cases`) as the input object. Then, the code adds the test case reporter object to the report and uses `rptview` to display the report. The code saves the report in the `myCustomTestSpecRpt.pdf` file.

```

myrpt = slreportgen.report.Report('myCustomTestSpecRpt.pdf');
testcaseRptr = myTCReporter('Object',test_cases);

```



```
add(myrpt,testCaseRptr);  
close(myrpt);  
rptview(myrpt);
```

See Also

More About

- “Templates” (MATLAB Report Generator)
- “Open Template Files” (MATLAB Report Generator)
- “Subclass Reporter Definitions” (MATLAB Report Generator)
- “Define New Reporters” (MATLAB Report Generator)
- “Customize Test Results Reports” on page 7-21

Debugging Equivalence Test Failures Using Model Slicer

This example shows how to debug Simulink Test equivalence failures by highlighting functional dependencies in the Model Slicer. For more information, see “Highlight Functional Dependencies” (Simulink Check). The example files include a predefined equivalence test to use with the `sldemo_fuelsys_dd_controller` model. The sample test case compares the model simulation in normal and software-in-the-loop (SIL) mode. The model includes a numerical discrepancy. In this example, you use Model Slicer to trace the faults and identify the discrepancy. For more information about debugging test failures, see the **Capabilities and Limitations** section of “Debugging Baseline Test Failures Using Model Slicer” on page 7-6.

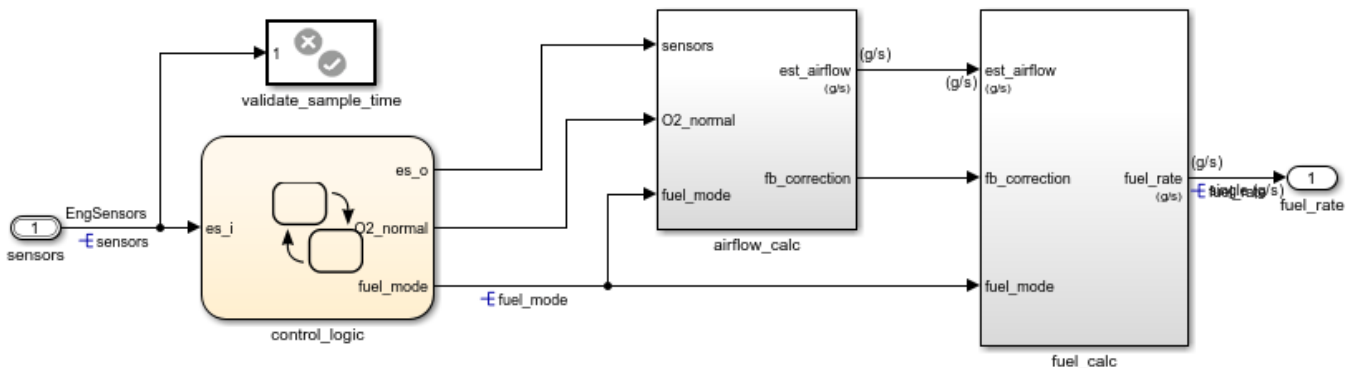
Setting Up the Artifacts

Run the test case and view the results.

1. Open the `sldemo_fuelsys_dd_controller` model:

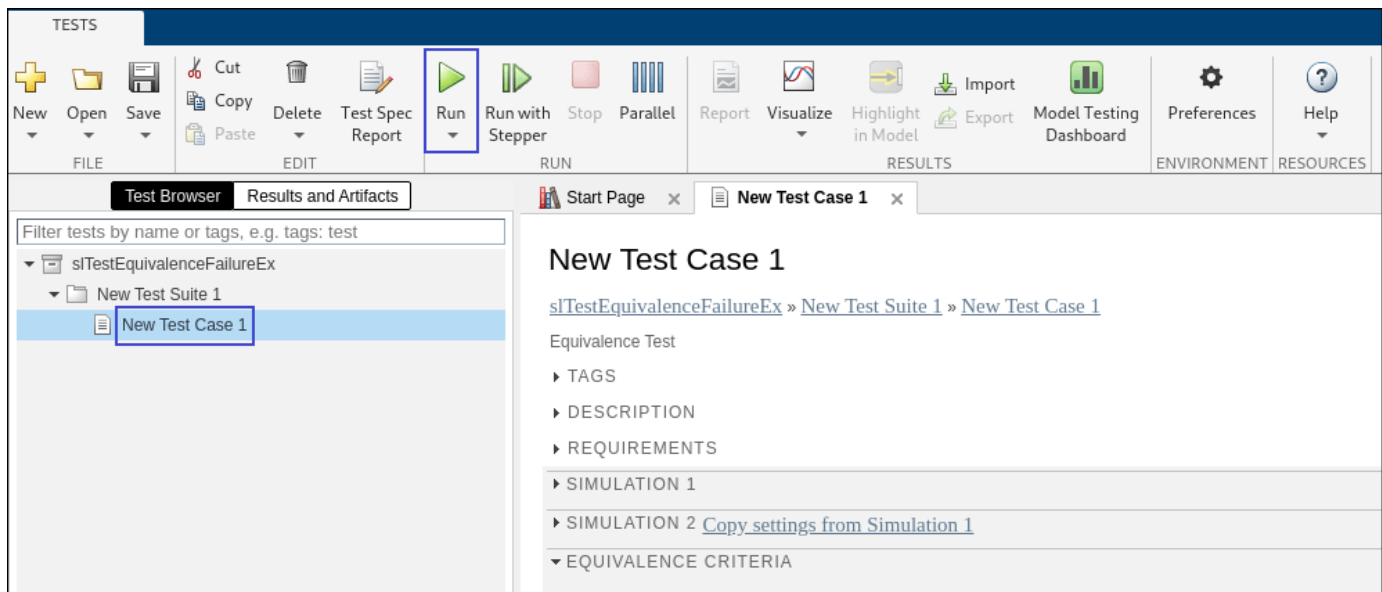
```
open_system('sldemo_fuelsys_dd_controller');
```

Fuel Rate Controller



Copyright 1990-2022 The MathWorks, Inc.

2. Log the output by the signals coming out of `airflow_calc` and `fuel_calc` subsystems and the Stateflow chart `control_logic` to generate a visualization after the analysis. To log signals, click the signal and, in the action bar, click Log Selected Signal.
3. In the **Apps** tab, in the **Model Verification, Validation, and Test** section, click **Simulink Test** to open the Simulink Test toolstrip.
4. In the **Tests** tab, click **Simulink Test Manager** to open the Test Manager.
5. To open the test file, click **Open** and select `slTestEquivalenceFailureEx` from the example folder.
6. After the test file loads, select **New Test Case 1** in the **Test Browser** pane.
7. Click **Run**.



8. The new test results appear in the **Results and Artifacts** pane. Right-click the result and select **Expand All Under** to see the **Equivalence Criteria Result** and **Verify Statements 1** section.

NAME	STATUS
▼ Results: 2021-Nov-16 16:44:40	1 ✖
▼ New Test Case 1	✖
▼ Equivalence Criteria Result	✖
○ airflow_calc:1	✓
● airflow_calc:2	✖
○ control_logic:1.throttle	✓
○ control_logic:1.speed	✓
○ control_logic:1.ego	✓
○ control_logic:1.map	✓
○ control_logic:2	✓
○ fuel_mode	✓
○ fuel_rate	✓
▶ Verify Statements 1	✓
▶ Sim Output 1(sldemo_fuelsys_c	
▶ Sim Output 2(sldemo_fuelsys_c	

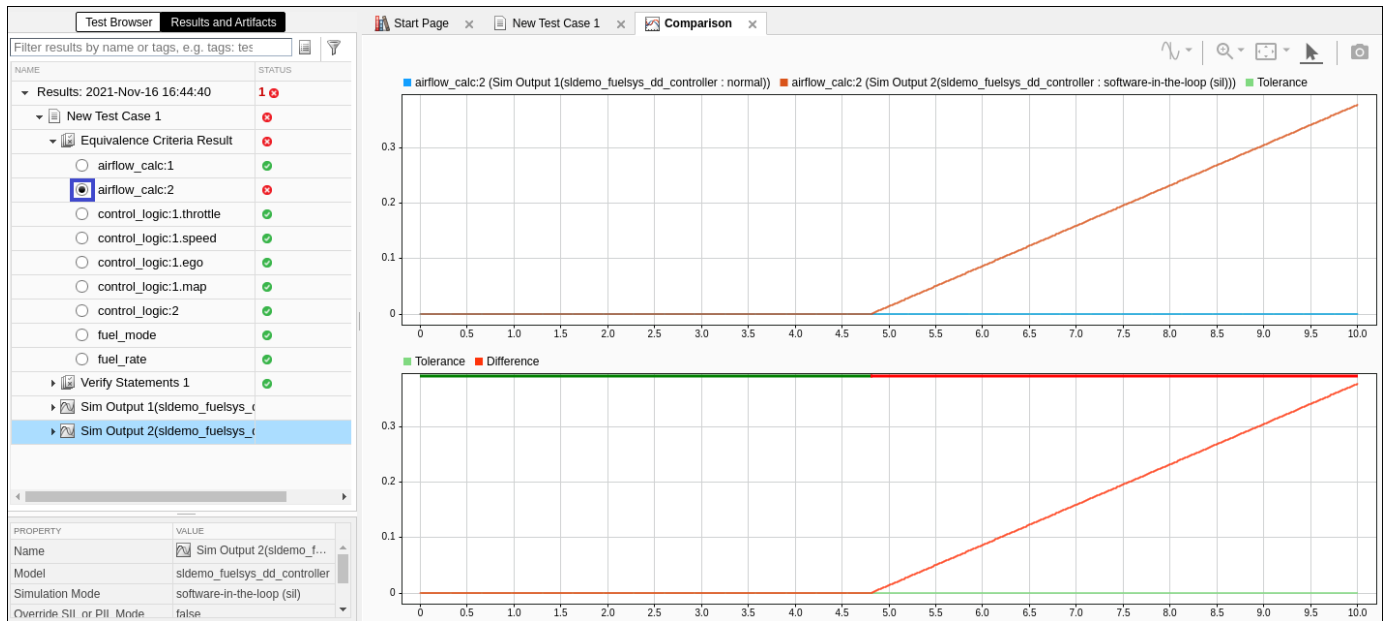
Observe that only one signal failed: `airflow_calc`. Consider changing the **Device details** from 32 bits to 64 bits from **Hardware and Implementation** pane after clicking on **Model Settings** from Simulation tab, if you are not able to see the failure.

Entering Debug Session

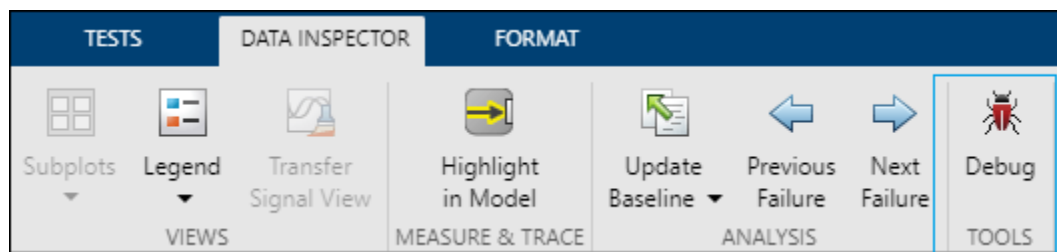
Set up the Model Slicer to debug the failed `airflow_calc` signal.

1. To compare the `airflow_calc` signal in different simulation modes, select the radio button next to the `airflow_calc` signal. Another way to select a failed signal is from the **Signal to Debug** dropdown list in the toolbar.

In the plot area, compare the outputs across simulations.

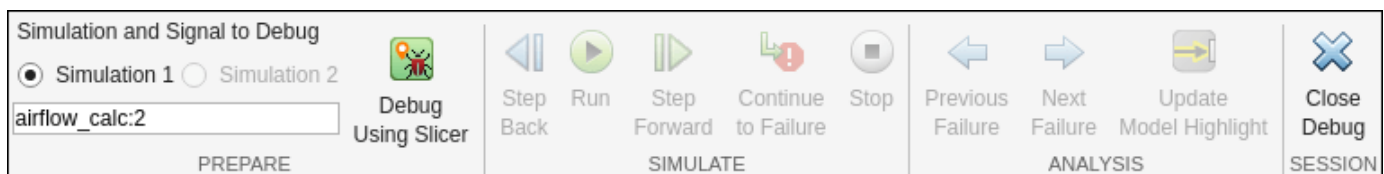


2. In the **Tools** section, click **Debug**. Note that you must plot a failed equivalence or verify signal to enable the **Debug** button.



The **Debug** tab opens and hides. The debug tab hides multiple Test Manager options.

3. To set up the Model Slicer, click **Debug Using Slicer**.



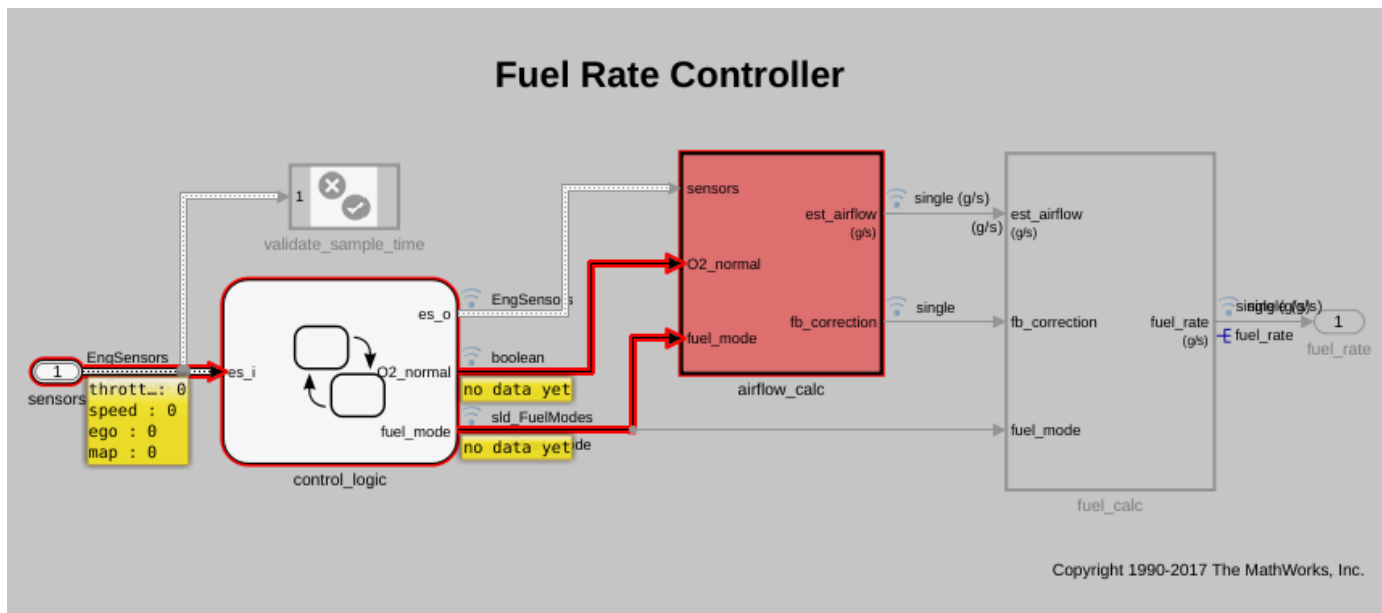
Model Slicer prepares the debugging session by:

- 1 Rerunning the test case and creating new debugging results. This action ensures that the failure still exists in the current state of the test model.
- 2 Launching the Model Slicer on the test model.
- 3 Automatically plotting the selected failed signal in the debugging results and setting the failed signal as the starting time point.
- 4 Pausing the simulation at the model start time to continue debugging.

Debugging Using Model Slicer

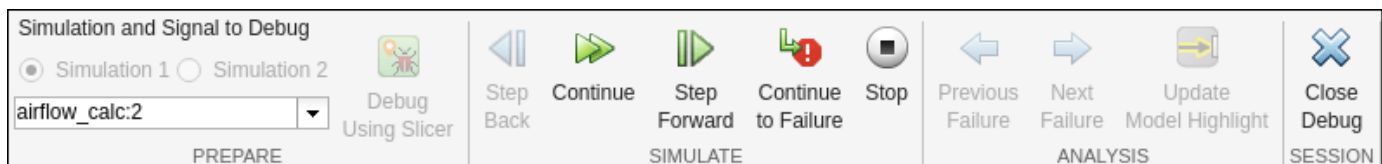
Identify the reason for the failure by using the debugging features of the Model Slicer.

1. Use the **Step Back** or **Step Forward** buttons to move one step back or forward in simulation time. The left data cursor moves to the current simulation time. Observe the changes in the data dependencies by noting the changed model highlighting and port value labels for the active signals at every time stamp.



You can also use **Run**, **Continue**, or **Stop** buttons to run, complete, or stop the current simulation, respectively.

2. In the Test Manager, click **Continue to Failure** to continue the model simulation to the beginning of the next failure region. The data cursors show the bounds of the failure region.



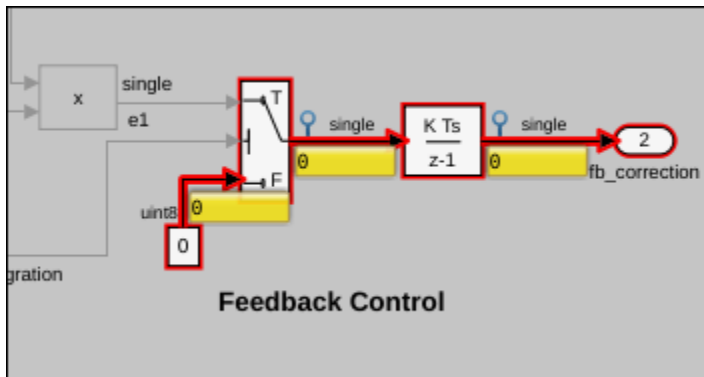
3. Observe these changes at the failure:

- Simulation pauses at $T = 4.81$.

- The data cursors update accordingly.

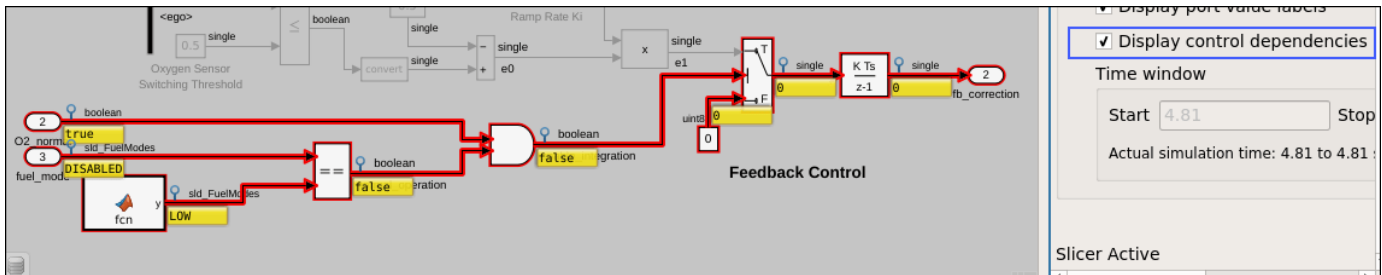


From the Model Slicer highlighting, you can observe the branch of the model that causes the error.



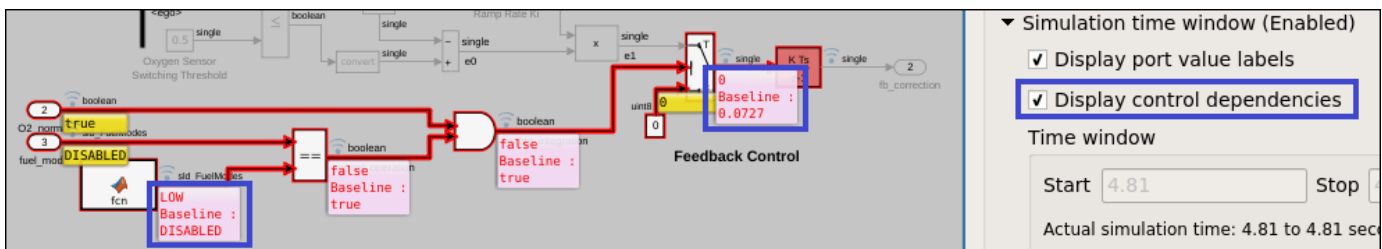
Notice that the constant, 0 passes through the `sldemo_fuelsys_dd_controller/airflow_calc/hold` integrator Switch block. To determine why the control port evaluates to false, highlight the control dependencies in the model.

4. Enable **Display Control Dependencies** from the **Simulation Time Window** section in the Model Slicer Dialog pane. Observe the chain of blocks that the Model Slicer highlights as the probable cause of the discrepancy. To further visualize the numerical differences between the simulation modes, end the debugging session, log the signals in the active chain, and debug the model again.



Refining the Debugging Results

1. Exit the debugging session by clicking **Session > Close Debug**.
2. Open the model and log all signals in the observed path before.
3. Save the model.
4. Repeat steps 1 to 3 from the Debug Using Model Slicer section.



Observe that the Enumerated Constant value is being set in the MATLAB function block based on the simulation mode.

Incorporating the Fix

1. Exit the debugging session by clicking **SESSION > Close Debug**.
2. Open the model and update `sldemo_fuelsys_dd_controller/airflow_calc/MATLAB` function to return the same value irrespective of the simulation mode.
3. Save the model.
4. Run the test case and view the results. Observe that the test results test results pass.

See Also

More About

- "Model Slicer Considerations and Limitations" (Simulink Check)
- "Refine, Test, and Debug a Subsystem" on page 2-22

Process Automation for Simulink Test

Use Process Advisor to Streamline Requirements-Based Testing of Models and Generated Code

Note The information below introduces using the Process Advisor to run the model testing process tasks and review the assessment results. For detailed information, see document *Model Testing Process in Process Advisor* ([lifecycle_automation.pdf](#)). To view the document, in the Process Advisor, click the ellipses () for any task in the **Tasks** column and then click the **Help** button.

When testing models in a project that uses a Model-Based Design (MBD) approach, using a model testing process streamlines testing of Simulink models to help you confirm that models used to generate production code accurately represents the system and meets the project requirements. Following a model testing workflow helps to ensure the accuracy, reliability, and performance of the designed system before deployment.

The Process Automation for Simulink Test add-on is an interactive workflow that uses verification and validation tools to assess the completeness of model testing for a project. The add-on uses the Process Advisor to systematically execute pre-defined tasks in the model testing process. These tasks are used to:

- Assess specification of units and components in the project.
- Verify that the model design does not introduce unintended functionality.
- Verify that each requirement has a test.
- Generate code.
- Run tests in normal and SIL modes and use aggregate results to verify that the desired level of model and code coverage is achieved.
- Achieve translation validation through back-to-back testing, which uses both requirements-based and non-requirements-based tests to compare the behavior of the model and generated code. Translation validation provides a high degree of confidence that, for the design, the output of the code generator, compiler, and linker tool chain exhibits equivalent input-output behavior as the model used for production code generation.

The Process Advisor uses data collected from the tasks to identify potential issues in the tests. You can review the results and use the interactive functionality in the model testing process to address assessment issues, such as:

- Creating test files and tests and link them to existing requirements to resolve gaps in the requirements-based tests.
- Modifying tests to resolve test failures.
- Resolving model and code coverage gaps in the tests to achieve the desired level of coverage.

Use the Process Automation for Simulink Test add-on to support your work across all stages of the software development lifecycle, including during the planning and development stages.

After performing the initial assessment, the Process Advisor detects additional changes to your project and determines the impact of the change on your existing task results. For example, when you add a new requirement, the Process Advisor displays a warning in the banner that indicates changes are detected. When you click **Refresh Tasks**, the Process Advisor rescans the project and marks the

affected results as outdated. Use the **Run All** option to rerun only the tasks with stale results, as well as any tasks that are not executed.

Note You can use the Process Automation for Simulink Test add-on with R2024a and later.

This table identifies the MathWorks® products required to open the Process Advisor and run the model testing process tasks. Use the add-on manager to review your installed MathWorks products, including support packages and add-ons. During installation, the add-on manager informs you of dependencies that you may need to install before using the product.

MathWorks Products	Usage
CI Support Package for Simulink support package.	Process Advisor app. Install the support package from the Add-On Explorer.
Process Automation for Simulink Test	Define tasks in the model testing workflow. Download the installer from File Exchange.
Simulink Check™	Required to install the CI Support Package for Simulink support package.
Simulink Test	Author and execute tests and collect assessment information. Open the Model Testing Process by using Simulink Test Manager.

Set Up the Model Testing Process

Install the tools needed to assess your project and set up the model testing process in the Process Advisor.

- 1 Use the add-on manager to check that these products are installed:
 - Simulink Check
 - Embedded Coder
 - Requirements Toolbox
 - Simulink Coverage
 - CI Support Package for Simulink support package (available from Add-On Explorer).
 - Process Automation for Simulink Test add-on (available from File Exchange).

Note MATLAB installs add-ons in a default installation folder specific to the platform. For more information, see “Default Add-On Installation Folder”.

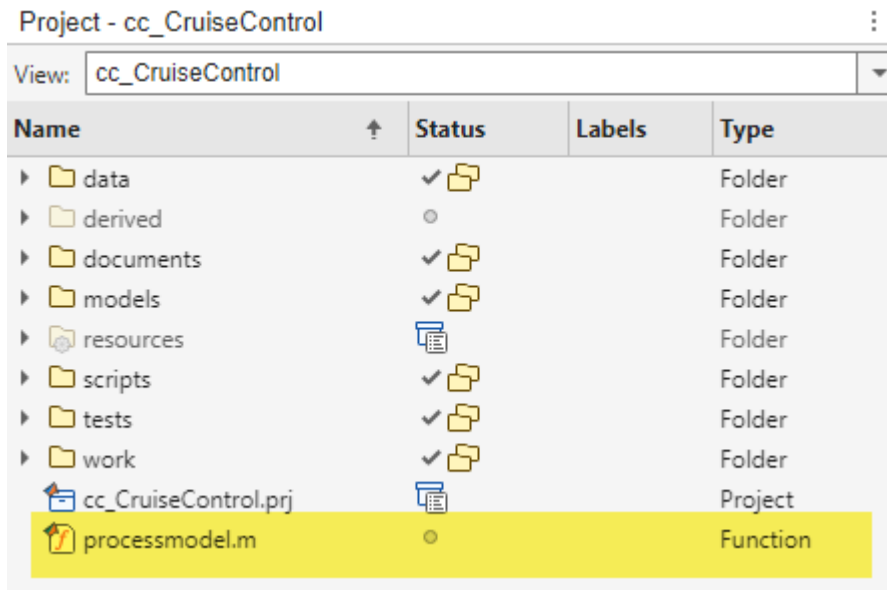
- 2 If prompted, restart MATLAB.
- 3 Set up the model testing process in the Process Advisor. For more information, see the document *Getting Started with Process Automation for Simulink Test*. This document is available on File Exchange, under **Examples**.

During setup, the Process Advisor automatically creates a default process model file (`processmodel.m`) for the project. The Process Advisor reads the file and determines which tasks to run, how the tasks perform their actions, and the order in which tasks run. For more information, see “Overview of Process Model” (Simulink Check).

Open the Process Advisor by Using Simulink Test Manager

Use Simulink Test Manager to open the model testing process in the Process Advisor.

- 1 Open a project that has a defined process model file (`processmodel.m`). The process model file is available in the project files.



If the project does not contain a process model, then you need to create the file by setting up the model testing process on page 8-3.

- 2 Open the existing process model file and verify that the `processmodel` function contains these code lines.

```
%Adds Model Testing Process from Process Automation for Simulink Test Add-On
if(~isempty(which('pasltest.addProcess')))
```

pasltest.addProcess(pm);

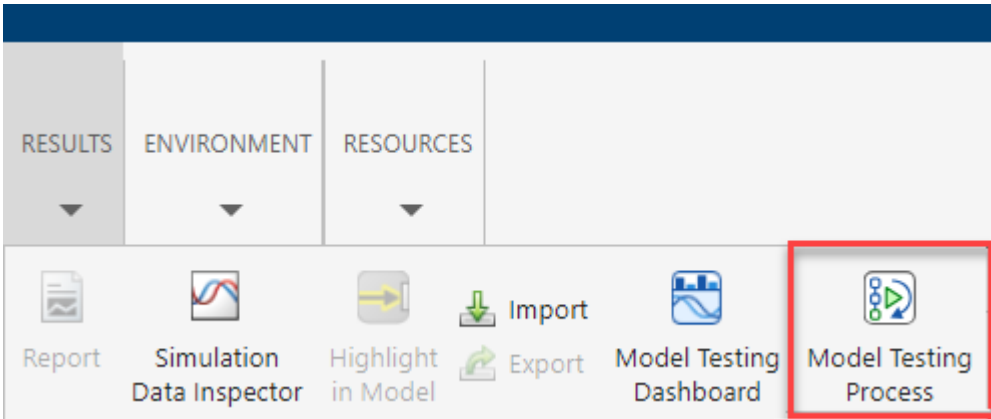
```
end
```

If missing, then add the code to the bottom of the file and save the file. Open the Process Advisor and review the model testing process tasks. Close the Process Advisor.

- 3 Open the Simulink Test Manager.

```
sltest.testmanager.view
```

- 4 In the **Results** gallery, select **Model Testing Process**. This option is available only after you install both the Process Automation for Simulink Test add-on and CI Support Package for Simulink support package.



The Process Advisor opens and displays the tasks in the model testing process.

Tasks
▼ ○ Model Testing Process
▼ ○ Setup
○ Configure units and components
▼ ○ Test Authoring
▼ ○ Create tests
○ cc_ControlMode
○ cc_DriverSwRequest
○ cc_LightControl
○ cc_ThrottleController
▼ ○ Configure tests
○ cc_ControlMode
○ cc_DriverSwRequest

Run Tasks and View Results in the Model Testing Process

Note For detailed information about running the model testing process and reviewing the results, see *Model Testing Process in Process Advisor* (lifecycle_automation.pdf). To open the document, click the ellipses (⋮) for any task in the **Tasks** column and then click the **Help** button.

The tasks in the model testing process appear in the **Tasks** column. Hover over a task and observe these icons.

- ▶ — Run that task and outdated dependent tasks.
- ⓘ — Review the instructions and guidelines for the task.
- ⋮ — Lists additional task functionality. Click the **Help** button to open the *Model Testing Process in Process Advisor* (lifecycle_automation.pdf) document.

Run all tasks together by using these buttons:

- **Run All** — Run all tasks except those with up-to-date results.
- **Run All > Force Run All** — Run all tasks, including those with up-to-date results.
- **Run All > Clean All** — Clear task results and delete task outputs for each of the tasks.

As you run the tasks in the model testing process, the Process Advisor:

- Detects the changes, displays a warning banner, and runs tasks with stale results.
- Displays a **Stop** button, which ends the task currently running and prevents the next queued tasks from running.

Use icons to quickly review the results:

- The **Tasks** column displays these icons for each task. Point to the status icon and view details about the status, including the task status and how long the task took to run.
 -  — Pass
 -  — Fail
 -  — Error
 -  — Not run
- Hover over the icon in the **I/O** column to review the output artifacts from a task.
- The Process Advisor aggregates the results of each task. In the **Details** column, Process Advisor shows the number of passing, warning, or failing results:
 -  — Pass
 -  — Warning
 -  — Fail
- Grey icons indicate that tasks results are outdated and the task must be rerun.

See Also

More About

- “Get and Manage Add-Ons”
- “Overview of Process Model” (Simulink Check)
- “Run Tasks with Process Advisor” (Simulink Check)
- “Automate and Run Tasks with Process Advisor” (Simulink Check)
- “Specify Settings for Process Advisor and Build System” (Simulink Check)
- Process Advisor

External Websites

- Process Automation for Simulink Test on File Exchange

Real-Time Testing

- “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2
- “Create and Run Tests Using the ASAM XIL Standard” on page 9-4
- “Create Tests That Use Third-Party Test Benches” on page 9-9
- “Create Tests Using Simulink Model Simulation as a Test Bench” on page 9-14
- “ASAM XIL Classes and Methods Supported by Simulink Test” on page 9-17
- “Test Models in Real Time” on page 9-23
- “Reuse Desktop Test Cases for Real-Time Testing” on page 9-27
- “Prepare for Real-Time Testing” on page 9-31
- “Create and Run Real-Time Tests” on page 9-34

Install and Set Up the Simulink Test Support Package for ASAM XIL Standard

The ASAM® XIL standard defines communication between test automation tools and test benches. You can use the Simulink Test Support Package for ASAM XIL Standard to use Simulink Test to control a test bench.

If you are using Simulink Real-Time™ and switching between test benches, or if you are using a third-party test bench or a Simulink model simulation as the test bench, you must install the support package. If you are using Simulink Real-Time as your test bench and do not need to switch between test benches, you do not need to install the support package. For Simulink Real-Time, use the real-time test case provided with Simulink Test.

To use the ASAM XIL API, first install the Simulink Test Support Package for ASAM XIL Standard, and then set up your test bench and Simulink model.

Install the Support Package

- 1 On the MATLAB **Home** tab, in the **Environment** section, click **Add-Ons > Get Hardware Support Packages**.
- 2 In the Add-On Explorer window, browse or search for the Simulink Test Support Package for ASAM XIL Standard.
- 3 Select the support package and click **Install**.

If you are using Simulink Real-Time as your test bench, you must also install the Simulink Real-Time Support Package for ASAM XIL Standard. See “Install the Simulink Real-Time Support Package for ASAM XIL Standard” (Simulink Real-Time).

- 4 After the support package installation completes, set up your test bench and build the model you want to run on it.

Set Up the Test Bench

- 1 Install and set up your test bench.
 - If you are using a third-party test bench, refer to the information supplied with that test bench.
 - If you are using Simulink Real-Time as your test bench, see “Install the Simulink Real-Time Support Package for ASAM XIL Standard” (Simulink Real-Time) for setup information.
 - If you are using Simulink model simulation as your test bench, append `matlabroot\extern\bin\win64` to your Windows® PATH variable.
- 2 If you are using Simulink Real-Time or a third-party product as your test bench, install the test bench compiler toolchain supplied with your test bench on your Windows computer. You use the compiler toolchain when building your Simulink model.

Configure and Build Your Model

These steps are required only if you are using Simulink Real-Time or a third-party product as your test bench. You do not need to perform these steps if you are using Simulink model simulation as your test bench.

Note Use the same version of MATLAB that you verified as compatible with your test bench.

- 1 In your Simulink model, in the **Modeling** tab, click **Model Settings** to open the Configuration Parameters dialog box. In the left pane, click **Code Generation**, then set **Toolchain** to the test bench toolchain.
- 2 Build the model for your test bench, creating a binary image that is runnable on the test bench.

After you have installed the support package and set up the test bench, you can build additional models and configurations at any time without having to set up the test bench again.

- 3 Create a Simulink Real-Time or vendor-specific port configuration file. See “Create and Run Tests Using the ASAM XIL Standard” on page 9-4 for information and examples of setting up configuration files.

See Also

`sltest.xil.framework.Framework`

More About

- “Create and Run Tests Using the ASAM XIL Standard” on page 9-4
- “Create Tests That Use Third-Party Test Benches” on page 9-9
- “Create Tests Using Simulink Model Simulation as a Test Bench” on page 9-14
- “ASAM XIL Classes and Methods Supported by Simulink Test” on page 9-17

External Websites

- ASAM XIL

Create and Run Tests Using the ASAM XIL Standard

The Simulink Test Support Package for ASAM XIL Standard implements the ASAM XIL API, which is a standard that defines communication between test automation tools, such as Simulink Test, and test benches, such as Simulink Real-Time, third-party test benches, and Simulink model simulation test benches. You can use the ASAM XIL API to run real-time hardware-in-the-loop (HIL), software-in-the-loop (SIL), and model-in-the-loop test cases and Simulink simulations created in Simulink Test using its XIL framework. The framework includes methods for mapping variables from the test code to the test bench, configuring the ports to use, specifying the test bench startup and shutdown order, and other commands to query and control the test bench.

The steps for creating a test that uses the Simulink Test Support Package for ASAM XIL Standard are:

- Install the support package and set up the test bench and your model. See “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2.
- “Configure the Test Bench” on page 9-4.
- “Create the Test Body” on page 9-6.
- “Run the Test” on page 9-7.

To use more than one test bench, repeat the set up process and configure the test bench tasks for each test bench. For detailed examples, see “Create ASAM XIL Test with Two Test Benches” on page 9-11.

Configure the Test Bench

Follow these steps to configure the test bench port, add the port to the Simulink Test ASAM XIL framework, and map variables from the model to the test bench variables. You can include the code for these steps in the same file as the test body. See “Create the Test Body” on page 9-6.

- 1 Create an instance of the `sltest.xil.framework.Framework` class. Use only one `Framework` object at a time.

```
frm = sltest.xil.framework.Framework;
```

- 2 For tests using third-party test benches, use the `displayAvailableTestbenches` method of `sltest.xil.framework.Framework` to obtain the names of the available test benches.
- 3 Create an XML port configuration file. You can use any name for the file, but it must use the `.xml` file extension. The configuration options depend on the test bench. Replace the product version number shown in the sample port configuration file with the version you have. Sample port configuration files are:

- Simulink Real-Time

Use `createPortConfigureFile` to create the file.

- NI™ VeriStand

```
<?xml version="1.0" encoding="UTF-8"?>
<NIVSPortConfig>
  <Version Major="2020" Minor="4" Fix="0" Build="0"/>
  <Project>C:\NIProjects\Project.nivproj</Project>
</NIVSPortConfig>
```

- dSPACE®

```
<?xml version="1.0" encoding="utf-8"?>
<PortConfigurations
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  <MAPortConfig>
    <SystemDescriptionFile>
      C:\DSPACEDProjects\smd_1104_sl.sdf
    </SystemDescriptionFile>
    <PlatformName>DS1104</PlatformName>
  </MAPortConfig>
</PortConfigurations>
```

- Simulink model simulation

Use `sltest.xil.testbench.createMAPortConfigFile` to create the file.

- 4 Add the port to use with the Simulink Test ASAM XIL framework. See “Create Tests That Use Third-Party Test Benches” on page 9-9 or “Create Tests Using Simulink Model Simulation as a Test Bench” on page 9-14 for where to include this code in the MATLAB code file that contains the test body. Replace the product version number shown in the appropriate sample port file with the version you have.

You can use `frm.Configuration` to display a summary of your configuration.

- Simulink Real-Time

To use all of the Simulink Real-Time functionality, add all three ports.

```
frm.Configuration.addModelAccessPort(...
  'MAPort', ...
  'asamxil.v2_1', ...
  'VendorName', 'MathWorks', ...
  'ProductName', 'XIL API', ...
  'ProductVersion', '1.0', ...
  'PortConfigFile', fullfile(pwd, 'myConfigureFile.xml'));
frm.Configuration.addECUCalibrationPort(...
  'ECUCPort', ...
  'asamxil.v2_1', ...
  'VendorName', 'MathWorks', ...
  'ProductName', 'XIL API', ...
  'ProductVersion', '1.0', ...
  'PortConfigFile', fullfile(pwd, 'myConfigureFile.xml'), ...
  'TargetState', 'started');
frm.Configuration.addECUMeasurementPort(...
  'ECUMPort', ...
  'asamxil.v2_1', ...
  'VendorName', 'MathWorks', ...
  'ProductName', 'XIL API', ...
  'ProductVersion', '1.0', ...
  'PortConfigFile', fullfile(pwd, 'myConfigureFile.xml'));
```

- NI™ VeriStand

```
frm.Configuration.addModelAccessPort(...
  'MAPort1', ...
  'asamxil.v2_1', ...
  'VendorName', 'National Instruments', ...
  'ProductName', 'NI VeriStand ASAM XIL Interface', ...
```

```
'ProductVersion','2020', ...  
'PortConfigFile',fullfile(pwd,'NIVeriStandPortConfig.xml')));
```

- dSPACE

```
frm.Configuration.addModelAccessPort(...  
    'MAPort1', ...  
    'asamxil.v2_1', ...  
    'vendorName','dSPACE GmbH', ...  
    'productName','XIL API', ...  
    'productVersion','2021-A', ...  
    'portConfigFile',fullfile(pwd,'dSpaceConfig.XML'));
```

- Simulink model simulation

```
frm.Configuration.addModelAccessPort...  
    ('MAPort1','asamxil.v2_1',...  
     'VendorName','MathWorks',...  
     'ProductName','Simulation Server XIL API',...  
     'ProductVersion','1.0.0.0', ...  
     'PortConfigFile',MAPortConfigFile);
```

- 5 Map the test variable names used in the test to the test bench variable names for the specified test bench. Optionally, also specify the task (that is, the logging rate) for the variables. To display the available test bench variable IDs, use the `displayAllTestbenchVariables` method of the `sltest.xil.framework.Framework` class. To view the associated tasks, use the `displayAllTaskInfo` method. See “Create Tests That Use Third-Party Test Benches” on page 9-9 for where to include this code in the MATLAB code file with the test body.

- This example maps the RPM test variable name to the Targets/Controller/Simulation Models/Models/simple_R2020a_2/Outputs/Out3 test bench variable name.

```
frm.Configuration.addTestVariableMapping(...  
    'RPM','MAPort1',...  
    ['Targets/Controller/Simulation Models/Models'...  
     '/simple_R2020a_2/Outputs/Out3']);
```

Create the Test Body

Write the test body in a MATLAB code file. If you write the test body as a function, you can call it from functions that define different configurations. The test body steps are independent of a test bench. After you define the test body, you use it as-is with any test bench configuration.

The test body typically includes these steps:

- 1 Use `init` to initialize the test bench.
- 2 Instantiate the mapped test variables so you can use them in the rest of the test body.
- 3 Tune parameters in the model to desired values for your test. You can change these values at any time, but if you change values in the middle of the test, the timing is not deterministic.
- 4 Set up the acquisition and start it.
- 5 At any point, if desired, call the `sltest.TestCase` methods, such as `sltest.TestCase.verifyThat`, to determine the pass or fail status of the test case.
- 6 Wait for acquisition or simulation to complete, or stop the simulation using the `stop` method.
- 7 Fetch the logged data.

- 8 Push the logged data results to the Test Manager. Only push the logged data if you plan to run your test using the Test Manager.

Run the Test

After you configure the test bench and write the test body, run your test in the Test Manager or at the command line.

- The MATLAB code file inherits from `sltest.TestCase`, which enables you to open and run the MATLAB code test file in the Test Manager. In the test body code, you can include pushing the logged data results to the Test Manager, and then load the file into the Test Manager using **Open > Open MATLAB-Based Simulink Test (.m)**. Then, run the test like you run other test cases in the Test Manager.
- You can run your code at the MATLAB command line, however you cannot push data to the Test Manager from the command line.

Perform additional analysis on the logged data, such as using the `verifySignalsMatch` method to compare the results to the baseline data.

Limitations

These limitations of the ASAM XIL standard apply when doing real-time testing using the Simulink Test Support Package for ASAM XIL Standard.

- The ASAM XIL standard does not provide support for
 - Setting the stop time of the model. On some test benches, simulation stops at the stop time the model was built with. Other test benches force the stop time to be `Inf`.
 - Verifying results. Use the `verify` methods from `sltest.TestCase` to analyze your captured data.
 - Enumerated, fixed point, or bus data types.
- On some test benches, logging does not start at the same time as simulation starts. This timing difference might cause your tests to not be precisely repeatable. Use an `Acquisition` trigger to repeatedly capture a region of interest in the logged data. The trigger corresponds to `t = 0`.

When using a Simulink model simulation as the test bench, these additional limitations apply:

- Variables in data dictionaries, cell arrays, bus signals, structures, and `Simulink.VariantExpression` objects do not appear in the `MAPort` variable list.
- If your model logs a large amount of data for an extended period of time, a memory overflow might occur, especially if simulation pacing is off. One way to avoid this issue is to turn off logging for large signals, such as matrices, and only log the minimum amount of signal data. Additionally, you can specify a simulation stop time, or if your model stop time is set to infinity, turn simulation pacing on.

Another way to control memory overflows is to limit the number of logged data points by using `<MaxLoggedDataPoints>`. When you set a value for the maximum number of logged data points, the `MaxDataPoints` and `DataLoggingMaxPoints` model settings update to that value. Any data lost due to these settings is not available using ASAM XIL capture or acquisition.

Memory overflows can occur when logging large amounts of data in any Simulink model. These overflows are not specific to using ASAM XIL with Simulink as the test bench.

- Models that use variable-step solvers are not supported.

Troubleshooting

You can display information about available test benches, variables, and tasks by using these `sltest.xil.framework.Framework` methods:

- `displayAllAvailableTestbenches`
- `displayAllTestbenchVariables`
- `displayAllTaskInfo`

See Also

`sltest.xil.framework.Framework` | `sltest.xil.framework.FrameworkConfiguration` |
`sltest.xil.framework.Acquisition` | `sltest.xil.framework.Stimulation` |
`sltest.xil.framework.TestVariable`

Related Examples

- “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2
- “Create Tests That Use Third-Party Test Benches” on page 9-9
- “Create Tests Using Simulink Model Simulation as a Test Bench” on page 9-14
- “ASAM XIL Classes and Methods Supported by Simulink Test” on page 9-17

External Websites

- ASAM XIL

Create Tests That Use Third-Party Test Benches

These examples show how you can use MATLAB to develop tests that run on third-party test benches using the Simulink Test Support Package for ASAM XIL Standard. One example configures a third-party test bench and creates a test that controls data acquisition by setting conditions to trigger the start and stop of data logging. The other example has two test points, that each configure a different test bench, but use the same shared test body. For more information on the Simulink Test ASAM XIL workflow, see “Create and Run Tests Using the ASAM XIL Standard” on page 9-4.

The examples inherit from the `sltest.TestCase` class, so you can load them into the Test Manager using **Open > Open MATLAB-Based Simulink Test (.m)**. Then, run them in the Test Manager. Alternatively, you can run the example files at the MATLAB command line, but you cannot push data to the Test Manager from the command line.

Note To run these examples, you must install the Simulink Test Support Package for ASAM XIL Standard. For more information, see “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2.

Create ASAM XIL Test with Trigger that Controls Data Acquisition

The `xilexample_trigger` file demonstrates how to configure a third-party test bench and run a test that uses a trigger to control the start and stop of data logging, push the logged data to the Test Manager and compare the results to baseline data.

The `ABCCoTestPoint` function creates a `Framework` object, configures a test bench port and maps test variables to test bench variables. For more information, see `sltest.xil.framework.Framework` and `sltest.xil.framework.FrameworkConfiguration`.

The `testBody` function initializes the framework to connect to the configured test bench and creates instances of the test variables `rpm`, `temperature`, `target_rpm`, and `input1`. Then, the function sets up logging of the `rpm` and `temperature` variables to start when `rpm` becomes greater than 10 and to stop after 15 seconds. Then, the function uses a `Stimulation` object to set up an external input to the test bench, runs the simulation, pushes the logged data to the Test Manager and verifies that the logged data and data in `baseline.mat` are equivalent. For more information on setting up inputs and data acquisition controls, see `sltest.xil.framework.Stimulation` and `sltest.xil.framework.Acquisition`.

```
classdef xilexample_trigger < sltest.TestCase

    methods (Test)
        function ABCCoTestPoint(testCase)
            import sltest.xil.framework.*;

            frm = Framework;

            frm.Configuration.addModelAccessPort(...
                'MAPort1', ...
                'asamxil.v2_1', ...
                'VendorName', 'ABC Co.', ...
                'ProductName', 'ABC Test Bench', ...
                'ProductVersion', '1.7', ...
                'PortConfigFile', fullfile(pwd, 'myConfigureFile.xml'));
        end
    end
end
```

```
frm.Configuration.addTestVariableMapping(...
    'rpm', 'MAPort1', ...
    ['Targets/Controller/Simulation Models/'...
     'Models/simpleXIL/Outports/Out3']);
frm.Configuration.addTestVariableMapping(...
    'temperature', 'MAPort1', ...
    ['Targets/Controller/Simulation Models/'...
     'Models/simpleXIL/Outports/Out4']);
frm.Configuration.addTestVariableMapping(...
    'target_rpm', 'MAPort1', ...
    ['Targets/Controller/Simulation Models/'...
     'Models/simpleXIL/Parameters/K']);
frm.Configuration.addTestVariableMapping(...
    'input1', 'MAPort1', ...
    ['Targets/Controller/Simulation Models/'...
     'Models/simpleXIL/Inports/Inport']);

% Beyond this point the configuration is done and
% the test is generic with no test bench specifics

testBody(testCase, frm);
end
end

methods (Access = 'private')
function testBody(testCase, frm)
    import sltest.xil.framework.*;

    frm.init;

    rpm = frm.createVariable('rpm');
    temperature = frm.createVariable('temperature');
    target_rpm = frm.createVariable('target_rpm');
    input1 = frm.createVariable('input1');

    target_rpm.write(50);

    % Start acquisition when rpm reaches more than 10 and
    % stop after 15 seconds.
    frm.Acquisition.setupWithVariables([rpm, temperature], ...
        'triggerVariables', rpm, ...
        'startTriggerType', 'condition', ...
        'startTriggerVal', 'rpm > 10', ...
        'stopTriggerType', 'duration', ...
        'stopTriggerVal', 15);
    frm.Acquisition.start;

    % Set up a stimulation (external input) for the model.
    % Waveform defined here lasts 5 seconds and LoopCount
    % of 2 doubles its duration to 10 seconds.
    tseries = timeseries(cos(2*pi*(0:1000)/200)*10, (0:1000)/200);
    frm.Stimulation.setupWithVariablesAndData(...
        {{input1, tseries}}, 'LoopCount', 2);
    frm.Stimulation.start;

    frm.start;
    disp(temperature.read);
```

```

        frm.Acquisition.wait;
        frm.stop;

        result = frm.Acquisition.fetch;
        frm.pushDataToSimulinkTestManager(testCase,result);
        testCase.verifySignalsMatch(result,'baseline1.mat');
    end
end
end

```

Create ASAM XIL Test with Two Test Benches

The `xilexample_polling` code file shows how to configure two test benches that can use the same test body.

The `ABCCoTestPoint` function creates a `Framework` object, configures a test bench port and maps test variables to test bench variables. For more information, see `sltest.xil.framework.Framework` and `sltest.xil.framework.FrameworkConfiguration`.

The `SimulinkRealTimeTestPoint` function follows the same steps to configure Simulink Real-Time as a second test bench.

The `testBody` function initializes the framework to connect to the configured test bench and creates instances of the test variables `rpm`, `temperature`, `target_rpm`, and `input1`. Then, the function sets up data acquisition for `rpm` and `temperature` variables without a trigger control, configures the `Stimulation` object to control input to the test bench, and starts the simulation. See `sltest.xil.framework.Acquisition` and `sltest.xil.framework.Stimulation`. The function starts the simulation and waits for the `temperature` variable to be less than 50. When the `temperature` variable value is less than or equal to 50, the function verifies that the `rpm` variable value is greater than 100, stops the simulation, and pushes the results to the Test Manager.

```

classdef xilexample_polling < sltest.TestCase

    methods (Test)

        function ABCCoTestPoint(testCase)
            import sltest.xil.framework.*;

            frm = Framework;

            % Add the ports
            frm.Configuration.addModelAccessPort(...
                'MAPort1', ...
                'asamxil.v2_1', ...
                'VendorName','ABC Co.', ...
                'ProductName','ABC Test Bench', ...
                'ProductVersion','1.7', ...
                'PortConfigFile',fullfile(pwd,'myConfigureFile.xml'));

            % Create the mapping from test variables to
            % test bench port variables
            frm.Configuration.addTestVariableMapping(...
                'rpm','MAPort1',...
                ['Targets/Controller/Simulation Models/'...
                'Models/simpleXIL/Outports/Out3']);
            frm.Configuration.addTestVariableMapping(...)
        end
    end
end

```

```

        'temperature', 'MAPort1', ...
        ['Targets/Controller/Simulation Models/' ...
         'Models/simpleXIL/Outports/Out4']);
frm.Configuration.addTestVariableMapping(...
    'target_rpm', 'MAPort1', ...
    ['Targets/Controller/Simulation Models/' ...
     'Models/simpleXIL/Parameters/K']);
frm.Configuration.addTestVariableMapping(...
    'input1', 'MAPort1', ...
    ['Targets/Controller/Simulation Models/' ...
     'Models/simpleXIL/Inports/Inport']);

% Beyond this point the configuration is done and
% the test is generic with no test bench specifics

% Call the generic test body
testBody(testCase, frm);
end

function SimulinkRealTimeTestPoint(testCase)
import sltest.xil.framework.*;

frm = Framework;

% Create ports
frm.Configuration.addModelAccessPort(...
    'MAPort', ...
    'asamxil.v2_1', ...
    'VendorName', 'MathWorks', ...
    'ProductName', 'XIL API', ...
    'ProductVersion', '1.0', ...
    'PortConfigFile', fullfile(pwd, 'myConfigureFile.xml'));

% Create the mapping from test variables to
% test bench port variables
frm.Configuration.addTestVariableMapping(...
    'rpm', 'MAPort', 'simpleXIL/Gain:1', 'TaskName', 'SubRate1');
frm.Configuration.addTestVariableMapping('temperature', ...
    'MAPort', 'simpleXIL/Gain2:1', 'TaskName', 'SubRate2');
frm.Configuration.addTestVariableMapping('target_rpm', ...
    'MAPort', 'simpleXIL/Gain/Gain');
frm.Configuration.addTestVariableMapping('input1', ...
    'MAPort', 'simpleXIL/Inport:1');

% Beyond this point the configuration is done and
% the test is generic with no test bench specifics

% Call the generic test body
testBody(testCase, frm);
end
end

methods (Access = 'private')
function testBody(testCase, frm)
import sltest.xil.framework.*;

frm.init;

```

```

rpm = frm.createVariable('rpm');
temperature = frm.createVariable('temperature');
target_rpm = frm.createVariable('target_rpm');
input1 = frm.createVariable('input1');

target_rpm.write(100);

frm.Acquisition.setupWithVariables([rpm, temperature]);
frm.Acquisition.start;

% Set up a stimulation (external input) for the model.
% Waveform defined here lasts 5 seconds and LoopCount
% of 2 doubles its duration to 10 seconds.
tseries = timeseries(cos(2*pi*(0:1000)/200)*10,(0:1000)/200);
frm.Stimulation.setupWithVariablesAndData(...
    {{input1,tseries}}, 'LoopCount',2);
frm.Stimulation.start;

frm.start;
while(temperature.read > 50)
    pause(1);
end

testCase.verifyTrue(rpm.read >= 100);

frm.stop;
result = frm.Acquisition.fetch;
frm.pushDataToSimulinkTestManager(testCase,result);
end
end
end

```

See Also

[sltest.xil.framework.Framework](#) | [sltest.xil.framework.FrameworkConfiguration](#) |
[sltest.xil.framework.Acquisition](#) | [sltest.xil.framework.Stimulation](#) |
[sltest.xil.framework.TestVariable](#) | [sltest.TestCase](#)

Related Examples

- “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2
- “Create and Run Tests Using the ASAM XIL Standard” on page 9-4
- “ASAM XIL Classes and Methods Supported by Simulink Test” on page 9-17

External Websites

- [ASAM XIL](#)

Create Tests Using Simulink Model Simulation as a Test Bench

This example shows how to develop tests that use Simulink model simulation as a test bench with the Simulink® Test™ Support Package for ASAM® XIL Standard. The example configures a Simulink model that simulates in normal mode as a test bench and creates a test that simulates the model, logs data, and then compares the logged data to the expected data to confirm that the results are within acceptable limits.

To run this example, you must install the Simulink Test Support Package for ASAM XIL Standard. For more information, see “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2. Then, you can load the file into the Test Manager using **Open > Open MATLAB-Based Simulink Test (.m)** and run the test. Alternatively, you can run the example files at the MATLAB command line, but you cannot push data to the Test Manager.

The ASAMXILSimulinkSimExampleTest.m file defines the SimulinkModelTestPoint function, which configures the model ASAMXILSimulinkSim as the test bench. The function creates a Framework object and then generates a model access port configuration file using sltest.xil.testbench.createMAPortConfigFile. Then, the function adds a model access port that specifies the generated configuration file and maps the test variables to test bench variables. For more information on how to configure test benches and map variables, see sltest.xil.framework.FrameworkConfiguration.

The testbody function initializes the framework and creates instances of the mapped variables, sets up the test to log the Sine, Cosine, and Result variables, enables logging, and simulates the model. Then, the function stops data acquisition, compares the simulation results to the expected results, and pushes the test case and results to the Test Manager. For more information signal data logging, see sltest.xil.framework.Acquisition.

```
classdef ASAMXILSimulinkSimExampleTest < sltest.TestCase

    methods (Test)

        function SimulinkModelTestPoint(testCase)
            import sltest.xil.framework.*;

            portConfigFileName = fullfile(pwd, ...
                'ASAMXILSimulinkSimExampleTestConfiguration.xml');
            sltest.xil.testbench.createMAPortConfigFile(portConfigFileName, ...
                "ASAMXILSimulinkSim")

            frm = Framework;

            frm.Configuration.addModelAccessPort(...
                'aPort', ...
                'asamxil.v2_1', ...
                'VendorName', 'MathWorks', ...
                'ProductName', 'Simulation Server XIL API', ...
                'ProductVersion', '1.0.0.0', ...
                'PortConfigFile', portConfigFileName);

            frm.Configuration.addTestVariableMapping('Sine', ...
                'aPort', 'ASAMXILSimulinkSim/Sine wave:1');
            frm.Configuration.addTestVariableMapping('Cosine', [' ' ...
                'aPort'], 'ASAMXILSimulinkSim/Cosine wave:1');
```

```

frm.Configuration.addTestVariableMapping('Result', ...
    'aPort', 'ASAMXILSimulinkSim/Product:1');
frm.Configuration.addTestVariableMapping('SineFreq', ...
    'aPort', 'SineFrequency');
frm.Configuration.addTestVariableMapping('CosineFreq', ...
    'aPort', 'CosineFrequency');

% Beyond this point the configuration is done and
% the test is generic with no test bench specifics

% Call the generic test body
testBody(testCase, frm);
end
end
methods (Access = 'private')
function testBody(testCase, frm)

    frm.init;

    Sine = frm.createVariable('Sine');
    Cosine = frm.createVariable('Cosine');
    Result = frm.createVariable('Result');
    SineFreq = frm.createVariable('SineFreq');
    CosineFreq = frm.createVariable('CosineFreq');

    sineFrequency = sqrt(8);
    cosineFrequency = sqrt(6);
    SineFreq.write(sineFrequency);
    CosineFreq.write(cosineFrequency);

    frm.Acquisition.setupWithVariables([Sine, Cosine, Result]);
    frm.Acquisition.start;

    frm.start;
    frm.waitForSimulationComplete;
    frm.stop;

    fetchedResult = frm.Acquisition.fetch;

    vals = fetchedResult.find('Result').Values;
    time = vals.Time;
    data = vals.Data;
    expectedData = sin(time*sineFrequency).*cos(time*cosineFrequency);

    import matlab.unittest.constraints.IsEqualTo
    import matlab.unittest.constraints.AbsoluteTolerance
    testCase.verifyThat(data, IsEqualTo(expectedData, ...
        'Within', AbsoluteTolerance(1.0e-12)))

    frm.pushDataToSimulinkTestManager(testCase, fetchedResult);
end
end
end

```

See Also

`sltest.xil.framework.Framework` | `sltest.xil.framework.FrameworkConfiguration` |
`sltest.xil.framework.Acquisition` | `sltest.xil.framework.Stimulation` |
`sltest.xil.framework.TestVariable` | `sltest.TestCase` |
`sltest.xil.testbench.createMAPortConfigFile`

Related Examples

- “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2
- “Create and Run Tests Using the ASAM XIL Standard” on page 9-4
- “ASAM XIL Classes and Methods Supported by Simulink Test” on page 9-17

External Websites

- [ASAM XIL](#)

ASAM XIL Classes and Methods Supported by Simulink Test

In this section...
“Testbench Class” on page 9-17 “MAPort Class” on page 9-18 “MAPortVariableInfo Class” on page 9-18 “MAPortConfig Class” on page 9-19 “MAPortFactory Class” on page 9-19 “Capture Class” on page 9-19 “CaptureEvent Class” on page 9-20 “CaptureResult Class” on page 9-20 “CaptureResultMemoryWriter Class” on page 9-20 “CaptureResultMDFWriter Class” on page 9-20 “CycleNumberDuration Class” on page 9-20 “CapturingFactory Class” on page 9-21 “DurationWatcher Class” on page 9-21 “DurationFactory Class” on page 9-21 “CycleDurationNumber Class” on page 9-21 “TimeSpanDuration Class” on page 9-21 “WatcherFactory Class” on page 9-22 “TaskInfo Class” on page 9-22 “VendorSpecificConfiguration Class” on page 9-22

You can use the Simulink Test Support Package for ASAM XIL Standard to interface with Simulink test cases using the ASAM XIL API. The tables lists the API methods that you can use with the support package.

Testbench Class

Class	Method	Introduced in Simulink Test Version
Testbench	WatcherFactory	24.2 (2024b)
Testbench	DurationFactory	24.2 (2024b)
Testbench	MAPortFactory	24.2 (2024b)
Testbench	ValueFactory	24.2 (2024b)
Testbench	CapturingFactory	24.2 (2024b)
Testbench	BuildNumber	24.2 (2024b)
Testbench	MajorNumber	24.2 (2024b)
Testbench	VendorName	24.2 (2024b)
Testbench	ProductName	24.2 (2024b)

Class	Method	Introduced in Simulink Test Version
Testbench	ProductVersion	24.2 (2024b)
Testbench	AvailablePortTypes	24.2 (2024b)
Testbench	MinorNumber	24.2 (2024b)
Testbench	RevisionNumber	24.2 (2024b)

MAPort Class

Class	Method	Introduced in Simulink Test Version
MAPort	CheckVariableNames	24.2 (2024b)
MAPort	Configure	24.2 (2024b)
MAPort	GetDataType	24.2 (2024b)
MAPort	GetVariableInfo	24.2 (2024b)
MAPort	IsReadable	24.2 (2024b)
MAPort	IsWritable	24.2 (2024b)
MAPort	LoadConfiguration	24.2 (2024b)
MAPort	StartSimulation	24.2 (2024b)
MAPort	StopSimulation	24.2 (2024b)
MAPort	TaskInfos	24.2 (2024b)
MAPort	TaskNames	24.2 (2024b)
MAPort	VariableNames	24.2 (2024b)
MAPort	Dispose	24.2 (2024b)
MAPort	Read	24.2 (2024b)
MAPort	Write	24.2 (2024b)
MAPort	CreateCapture	24.2 (2024b)
MAPort	SimulationStepSize	24.2 (2024b)
MAPort	State	24.2 (2024b)
MAPort	Configuration	24.2 (2024b)
MAPort	Name	24.2 (2024b)
MAPort	Disconnect	24.2 (2024b)

MAPortVariableInfo Class

Class	Method	Introduced in Simulink Test Version
MAPortVariableInfo	AvailableTasks	24.2 (2024b)
MAPortVariableInfo	IsReadable	24.2 (2024b)

Class	Method	Introduced in Simulink Test Version
MAPortVariableInfo	IsWriteable	24.2 (2024b)
MAPortVariableInfo	DataType	24.2 (2024b)
MAPortVariableInfo	Name	24.2 (2024b)
MAPortVariableInfo	Readable	24.2 (2024b)
MAPortVariableInfo	Writeable	24.2 (2024b)
MAPortVariableInfo	Xsize	24.2 (2024b)
MAPortVariableInfo	Ysize	24.2 (2024b)

MAPortConfig Class

Class	Method	Introduced in Simulink Test Version
MAPortConfig	ModelFile	24.2 (2024b)
MAPortConfig	VendorSpecificConfigurat ion	24.2 (2024b)

MAPortFactory Class

Class	Method	Introduced in Simulink Test Version
MAPortFactory	CreateMAPort	24.2 (2024b)

Capture Class

Class	Method	Introduced in Simulink Test Version
Capture	Fetch	24.2 (2024b)
Capture	Start	24.2 (2024b)
Capture	Stop	24.2 (2024b)
Capture	State	24.2 (2024b)
Capture	Variables	24.2 (2024b)
Capture	Port	24.2 (2024b)
Capture	ClearConfiguration	24.2 (2024b)
Capture	CaptureResult	24.2 (2024b)

CaptureEvent Class

Class	Method	Introduced in Simulink Test Version
CaptureEvent	TimeStamp	24.2 (2024b)
CaptureEvent	Type	24.2 (2024b)

CaptureResult Class

Class	Method	Introduced in Simulink Test Version
CaptureResult	CaptureStartTime	24.2 (2024b)
CaptureResult	Events	24.2 (2024b)
CaptureResult	SignalGroupNames	24.2 (2024b)
CaptureResult	ExtractSignalValue	24.2 (2024b)
CaptureResult	GetSignalGroupValue	24.2 (2024b)
CaptureResult	GetVariableNames	24.2 (2024b)

CaptureResultMemoryWriter Class

Class	Method	Introduced in Simulink Test Version
CaptureResultMemoryWriter	Save	24.2 (2024b)
CaptureResultMemoryWriter	Type	24.2 (2024b)

CaptureResultMDFWriter Class

Class	Method	Introduced in Simulink Test Version
CaptureResultMDFWriter	Filename	24.2 (2024b)
CaptureResultMDFWriter	Version	24.2 (2024b)
CaptureResultMDFWriter	Overwrite	24.2 (2024b)
CaptureResultMDFWriter	Save	24.2 (2024b)

CycleNumberDuration Class

Class	Method	Introduced in Simulink Test Version
CycleNumberDuration	Save	24.2 (2024b)
CycleNumberDuration	Type	24.2 (2024b)

CapturingFactory Class

Class	Method	Introduced in Simulink Test Version
CapturingFactory	CreateCaptureResult	24.2 (2024b)
CapturingFactory	CreateCaptureResultMemoryWriter	24.2 (2024b)
CapturingFactory	CreateCaptureResultMDFWriter	24.2 (2024b)
CapturingFactory	CreateCaptureResultMDFWriterByFileName	24.2 (2024b)

DurationWatcher Class

Class	Method	Introduced in Simulink Test Version
DurationWatcher	Duration	24.2 (2024b)
DurationWatcher	Duration2	24.2 (2024b)

DurationFactory Class

Class	Method	Introduced in Simulink Test Version
DurationFactory	CreateCycleNumberByDuration	24.2 (2024b)
DurationFactory	CreateTimeSpanDuration	24.2 (2024b)

CycleDurationNumber Class

Class	Method	Introduced in Simulink Test Version
CycleDurationNumber	CycleNumber	24.2 (2024b)
CycleDurationNumber	Type	24.2 (2024b)

TimeSpanDuration Class

Class	Method	Introduced in Simulink Test Version
TimeSpanDuration	TimeSpan	24.2 (2024b)
TimeSpanDuration	Type	24.2 (2024b)

WatcherFactory Class

Class	Method	Introduced in Simulink Test Version
WatcherFactory	CreateDurationWatcher	24.2 (2024b)
WatcherFactory	CreateDurationWatcherByTimeSpan	24.2 (2024b)

TaskInfo Class

Class	Method	Introduced in Simulink Test Version
TaskInfo	Name	24.2 (2024b)
TaskInfo	Period	24.2 (2024b)
TaskInfo	Type	24.2 (2024b)

VendorSpecificConfiguration Class

Class	Method	Introduced in Simulink Test Version
VendorSpecificConfiguration	WorkingDirectory	24.2 (2024b)
VendorSpecificConfiguration	EnablePacing	24.2 (2024b)
VendorSpecificConfiguration	PacingRate	24.2 (2024b)
VendorSpecificConfiguration	MaxLoggedDataPoints	24.2 (2024b)
VendorSpecificConfiguration	CheckMAPortStates	24.2 (2024b)

See Also

More About

- “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2
- “Create and Run Tests Using the ASAM XIL Standard” on page 9-4
- “Create Tests That Use Third-Party Test Benches” on page 9-9
- “Create Tests Using Simulink Model Simulation as a Test Bench” on page 9-14

External Websites

- ASAM XIL

Test Models in Real Time

In this section...
“Prepare, Create, and Run Real-Time Tests” on page 9-23
“Real-Time Testing Considerations” on page 9-25
“Real-Time Testing Limitations” on page 9-25

You can test your system in environments that resemble your application by using Simulink Real-Time and Simulink Test. You begin with model simulation on a development computer, then use software-in-the-loop (SIL) and processor-in-the-loop (PIL) simulation. You can use Simulink Test to create baseline, equivalence, and simulation real-time tests or convert desktop test cases to real-time tests. Real-time tests execute an application on a standalone target computer that connects to a physical system. Real-time testing can include effects of timing, signal interfaces, system response, and production hardware.

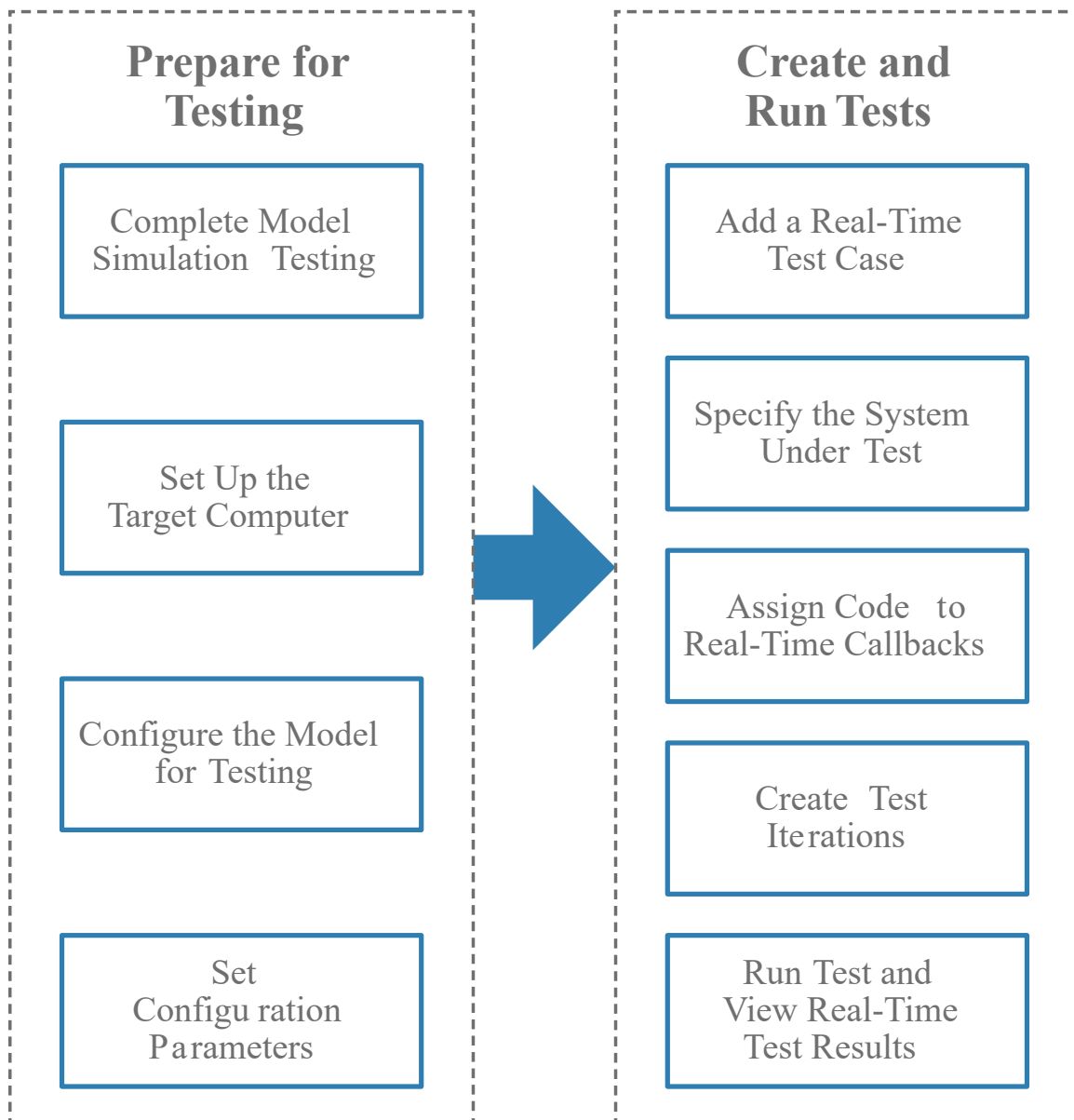
You can use real-time tests to perform:

- Rapid prototyping — Iterative testing on a standalone target connected to plant hardware. You evaluate the system by testing in real time and verifying your results against requirements and model results. Using rapid prototyping results, you can change your model and update your requirements. After refining your design, you retest on the standalone target.
- Hardware-in-the-loop (HIL) testing — Testing a system that has passed several stages of verification, typically SIL and PIL simulations. You can use Simulink Test on a Windows or Linux® computer to run real-time test cases using Simulink Real-Time.

Note To perform tests using ASAM XIL, see “Install and Set Up the Simulink Test Support Package for ASAM XIL Standard” on page 9-2 and “Create and Run Tests Using the ASAM XIL Standard” on page 9-4.

Prepare, Create, and Run Real-Time Tests

To create a real-time test, you must set up your model and hardware for testing on real-time applications, then design, run, and report on the real-time test results.



Learn about each step in the process:

- 1 “Prepare for Real-Time Testing” on page 9-31 — Prepare your model, tests, and components for real time testing by:
 - Creating test cases that verify the model against requirements and running the model simulation tests.
 - Configuring the real-time target settings.
 - Creating test harnesses and marking signals for logging. You can also add `verify` statements to Test Assessment blocks to evaluate logical expressions that assess the real-time execution.
 - Configuring model parameters.
- 2 “Create and Run Real-Time Tests” on page 9-34 — Create real-time test cases, specify test properties, run the test and view results by:
 - Using the Test Manager to create a baseline, equivalence, or simulation real-time test case.
 - Specifying the model or test harness, the simulation setting overrides, and where to load the target application from.
 - Using the **Pre-start real-time application** callback to execute commands before the real-time application executes.
 - Setting up test iterations to execute the test case with different data.
 - Executing the tests, analyzing the results in the Test Manager, and reporting the results.

Real-Time Testing Considerations

- If you want to reuse a desktop simulation test case on a real-time target, use `verify` statements. In a real-time environment, `verify` statement failures only appear in the **Verify Statements** section of the test case results in the Test Manager. You can also access information about the `verify` runs using `getVerifyRuns`.
- Your target computer must have a file system to use `verify` statements and test case logging.
- When you use `verify` statements, you can choose to log only `pass` and `fail` results to reduce the transfer of data, which leads to greater simulation efficiency. To log only `pass` and `fail` results, on the **Tests** or **Harness** tab, in the **Test Cases** section, click **Suppress Untested Results**.
- Baseline or equivalence comparisons can fail because of missing data or time-shifted data from the real-time target computer. When investigating real-time test failures, look for time shifts or missing data points.

Real-Time Testing Limitations

- In Test Sequence blocks or Stateflow charts, `assert` statements are not supported for code generation and are ignored, so no error occurs if the assert condition fails during a Simulink Real-Time simulation. However, `verify` statements are supported for Simulink Real-Time code generation and automatically log results for a test case in the Test Manager. The same logging behavior is available when using a Simulink Assert block.
- You cannot use variant configurations.
- You cannot log states or, on some platforms, output ports.
- You cannot override the real-time execution sample time for applications built from models containing a Test Sequence block. The code generated for the Test Sequence block contains a

hard-coded sample time. Overriding the target computer sample time can produce unexpected results.

See Also

Related Examples

- “Reuse Desktop Test Cases for Real-Time Testing” on page 9-27
- “Projector Controller Testing Using verify and Real-Time Tests” on page 6-222
- “Use Simulink Real-Time UI to Create and Execute a Real-Time Application” (Simulink Real-Time)
- “Create and Run Tests Using the ASAM XIL Standard” on page 9-4

Reuse Desktop Test Cases for Real-Time Testing

Convert Desktop Test Cases to Real-Time

In the Test Manager, you can reuse test cases for real-time testing by converting desktop test cases to real-time test cases. For convenience, data can be stored externally so that each test case accesses common inputs and baseline data. The overall workflow is as follows:

- 1 Create a baseline, equivalence, or simulation test case with external inputs. For baseline tests, add baseline data from external files.
- 2 In the Test Manager, select the test case in the **Test Browser**.
- 3 Copy the test case. Right-click the test case and select **Copy**.
- 4 Paste the new test case into a test suite.
- 5 Rename the new test case.
- 6 Right-click the new test case, and select **Convert to > Real-Time Test**. For equivalence tests, select which simulation (simulation 1 or simulation 2) to run in real time.
- 7 Select the **Target Computer** and **Load Application From** options.
- 8 Ensure that the model settings are compatible with real-time test execution. For more information, see “Development Computer Requirements” (Simulink Real-Time).

Use External Data for Real-Time Tests

You can simplify test input data management by defining the input data in an external MAT or Excel file. Map the data to root inports in your model or test harness for desktop simulation. When you convert the desktop simulation test case into a real-time test, the test case uses the same inport mapping.

Using external data depends on how your test case loads the real-time application:

Load Real-Time Application from Model

If you are using external data for a real-time test, loading the real-time application from the model gives you the option of using an Excel file, MAT file, or CSV file. The external data is built into the application, and you can rerun the application from the target application or target computer.

In the **System Under Test** section, set the application to load from **Model**. In the **Inputs** section of the test case, click **Add**, and select an Excel file, MAT file, or CSV file. Map the data to your model inports. For more information on input mapping, see “Use External File Data in Test Cases” on page 6-76.

Load Real-Time Application from Target Application or Target Computer

After running the test from the model, you can run the test from the target application or target computer without recompiling. The application uses the input mapping from when the test ran from the model.

You can map external data to a test case loaded from the target application or target computer, without first running from the model. The external data must be in a MAT file, in the same format used if the test is loaded from the model. In the **System Under Test** section, select to load the application from the **Target Application** or **Target Computer**. In the **Inputs** section, click **Add** and select a MAT file. The Input string is not editable.

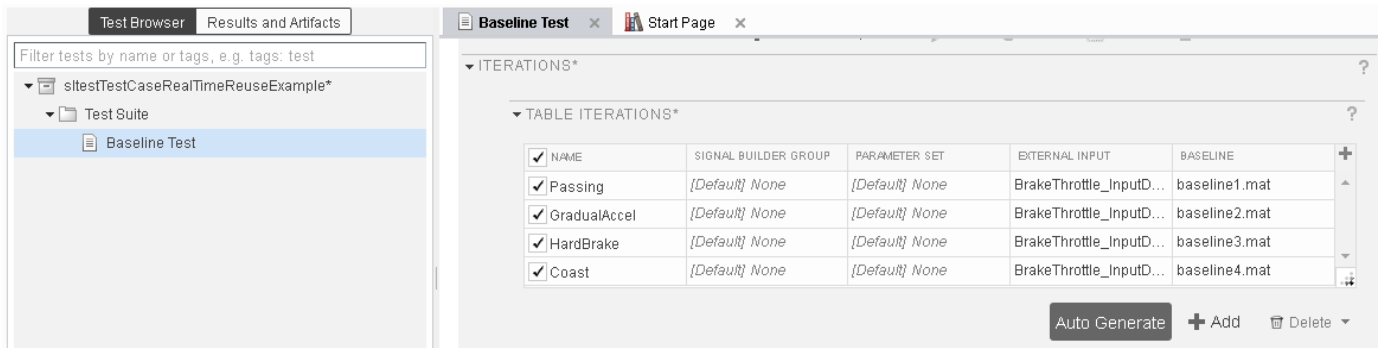
Reuse Desktop Test Case for Real-Time Testing

This example shows a basic desktop test case reuse workflow using external input data defined in an Excel file. You run the baseline test case on the desktop, update the baseline data, convert a copy of the test case to a real-time test, then run the test case on a target computer. The test file, baseline data, and Excel input data file are provided. This example runs only on Windows systems.

Open the Test Manager and Test File

The test file runs a transmission shift controller algorithm through four iterations, each corresponding to a different test scenario: passing, gradual acceleration, hard braking, and coasting. Baseline data associated with each scenario for the signals vehicle speed and output torque.

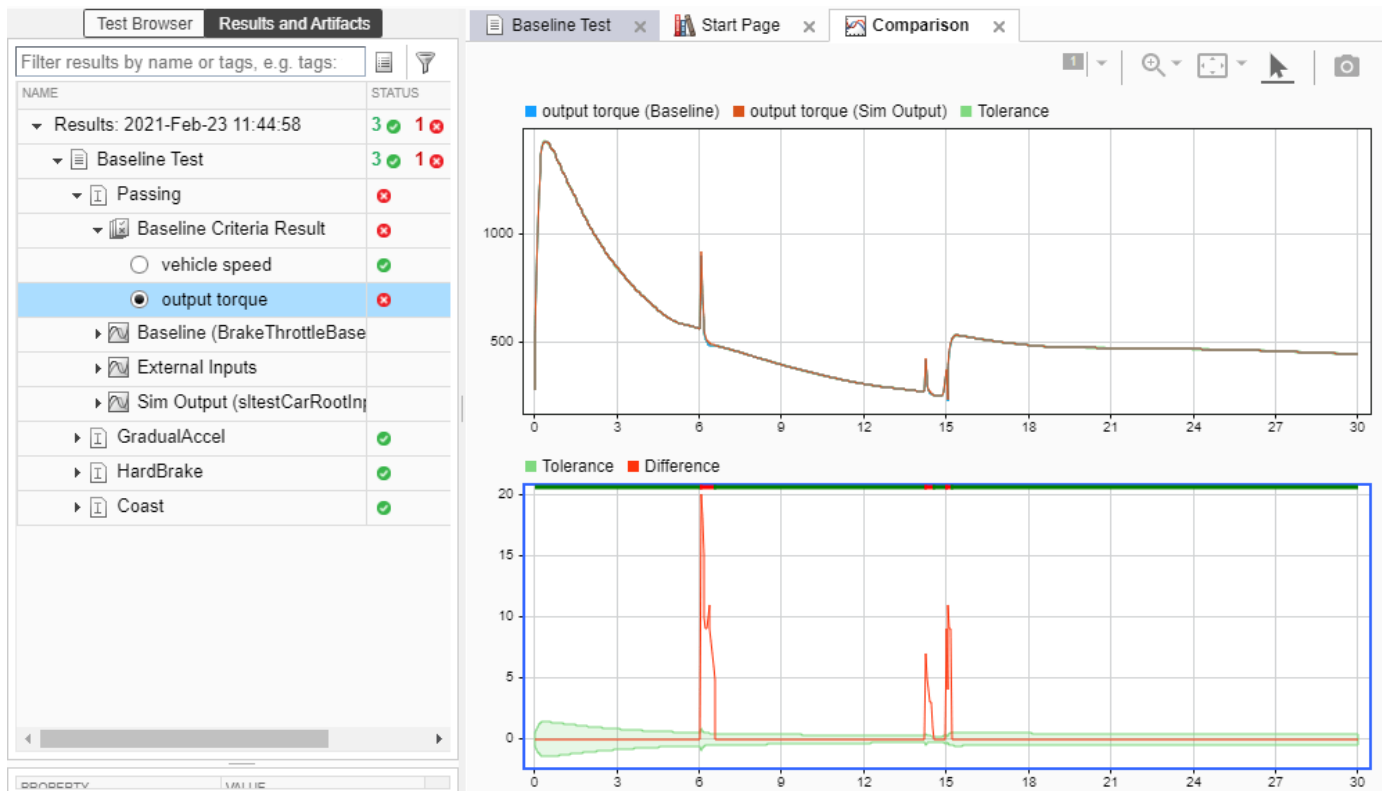
```
tf = sltest.testmanager.TestFile( ...  
    "sltestTestCaseRealTimeReuseExample.mldatx");  
sltest.testmanager.load(tf.Name);  
sltest.testmanager.view;
```



Run the Baseline Test and View Results

Click **Run** in the toolbar.

When the test finishes running, select **output torque** under **Baseline Criteria Result** to view the comparison. The Passing result fails due to transient signals that fall outside the relative tolerance.



Update the Baseline

Assume that the transient signals are not significant, and update the baseline data:

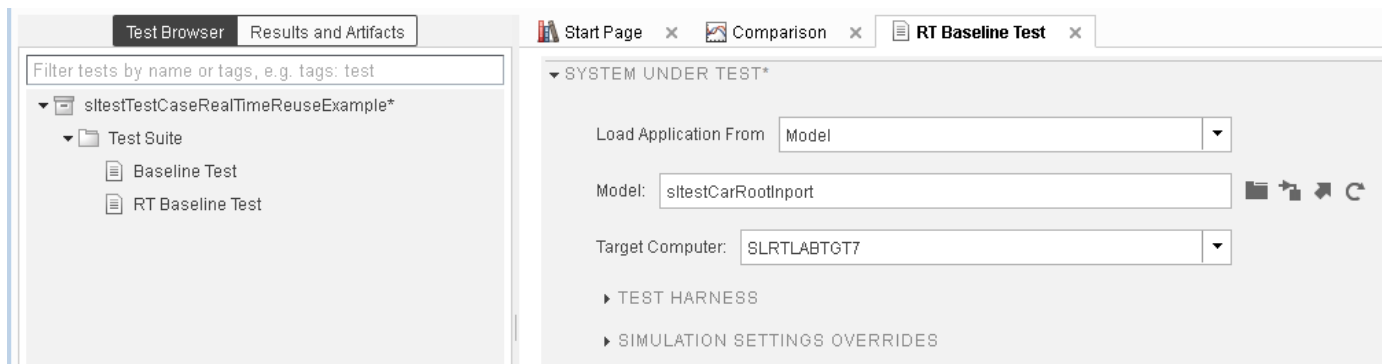
- 1 Click **Next Failure**. The first failure region is bounded by data cursors.
- 2 Click **Update Baseline > Replace Signal Segment in Baseline File** from the toolbar, and confirm that you want to overwrite the data.
- 3 Repeat this process for the other two failure regions.

Convert Baseline Test to Real-Time Test

- 1 In the Test Browser, right-click **Baseline Test** and select **Copy**.
- 2 Paste the new test case under the test suite.
- 3 Rename the new test case **RT Baseline Test**.
- 4 Right-click **RT Baseline Test** and select **Convert to > Real-Time Test**.

Run the Real-Time Test Case

- 1 Set the **Target Computer**.



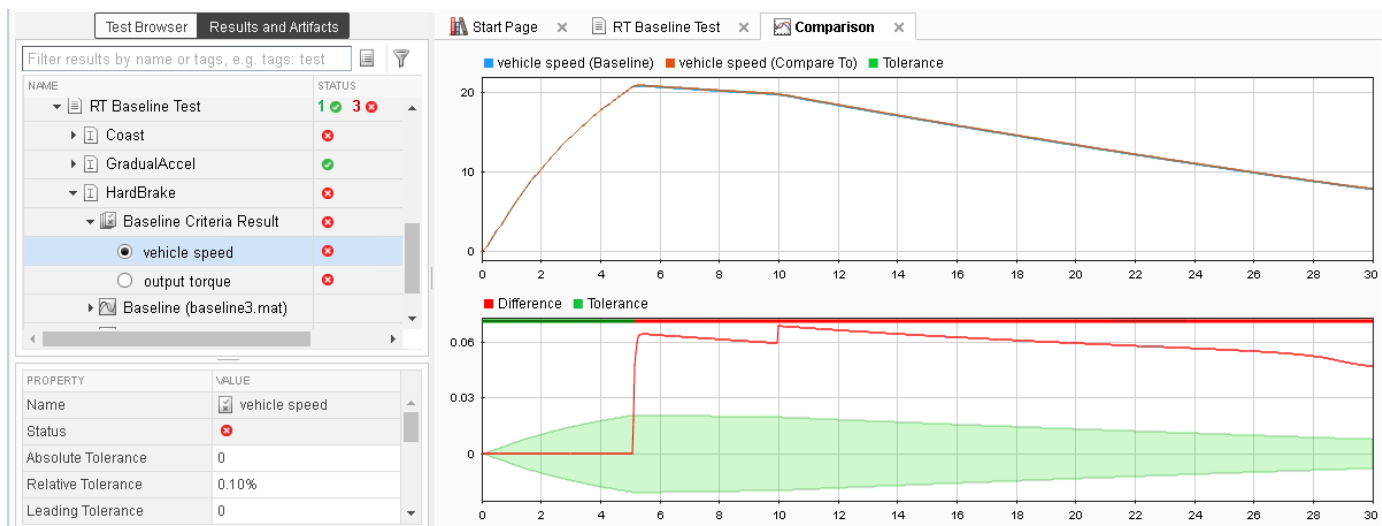
2. Set the system under test to load from **Model**.

3. Run the **RT Baseline Test** test case.

Examine and Resolve Test Failures

In this example, several of the scenarios fail due to timing impacts on the data output. For example, in the **HardBrake** iteration, the **vehicle speed** output falls outside the relative tolerance after the brake is applied. To resolve this failure, you could:

- Increase the relative tolerance for the real-time test.
- Create a separate set of baseline data for the real-time test.



See Also

Related Examples

- “Use Simulink Real-Time UI to Create and Execute a Real-Time Application” (Simulink Real-Time)
- “Projector Controller Testing Using verify and Real-Time Tests” on page 6-222

Prepare for Real-Time Testing

Use Simulink Test to execute simulation tests and prepare your model for real-time tests. By testing your model using desktop simulation, you can refine and debug your design before you run real-time tests, which typically take longer to execute. After you complete your simulation tests, you can configure your target computer, and the model or test harness for testing with Simulink Real-Time. You can create baseline, equivalence, or simulation test cases to test your system in real-time. If you choose a simulation test case, you can add `verify` and `assert` statements to blocks in your model to assess real-time execution and define test criteria.

Complete Model Simulation Testing

Real-time testing often takes longer than comparative model testing, especially if you execute a suite of real-time tests that cover several scenarios. Before executing real-time tests, complete testing using desktop simulation. Then, use the desktop simulation results to:

- Debug your model or make changes to your design to meet requirements.
- Debug your test sequence. Use the debugging features in the Test Sequence Editor. See “Debug a Test Sequence” on page 3-67.
- Update your requirements and add corresponding test cases.

When you create the simulation test cases, you can define the test input data in external files and map the data to inputs in the model. Later, when you convert the test cases to real-time test cases, the test cases use the same mapping as the original inputs. For more information, see “Use External Data for Real-Time Tests” on page 9-27. Alternatively, you can create real-time test cases and specify the test properties in the Test Manager after you configure your model. See “Create and Run Real-Time Tests” on page 9-34.

Set Up the Target Computer

Real-time testing requires a target computer or external hardware test bench. running Simulink Real-Time. Use the Simulink Real-Time Explorer to configure the target computer configuration settings. For more information, see Simulink Real-Time Explorer and “Target Computer Settings” (Simulink Real-Time). Alternatively, you can configure the settings programmatically. See “Simulink Real-Time Command-Line Interface in MATLAB” (Simulink Real-Time).

Configure the Model for Testing

Before you create a real-time test case, you can configure your model to fit your testing needs. You can create a test harness to test your model in an isolated environment, select signals for logging, and add assessments to your model or test harness to execute in real time.

Create a Test Harness

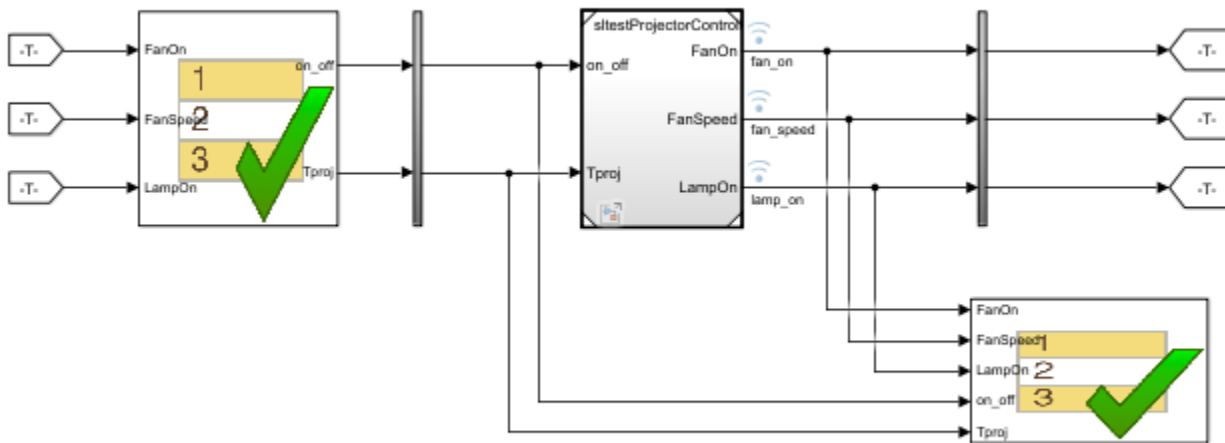
You can create test harnesses for your top-level model or one or more components in your model. You can also reuse test harnesses from model simulation testing. For more information about test harnesses, see “Test Harness and Model Relationship” on page 2-2 and “Create or Import Test Harnesses and Select Properties” on page 2-13.

Configure Signals for Logging

Before you create a real-time test case, configure logged signals:

- 1 In the Configuration Parameters dialog box, in the **Data Import/Export** pane enable the **Signal logging** parameter.
- 2 In your model, select a signal to log.
- 3 In the Property Inspector, select **Log signal data**.
- 4 Enter a name in the **Signal name** field.

You can assign unnamed signals a default name that does not match the name of the baseline or equivalence signal. In this example test harness, the logged signals are `fan_on`, `fan_speed`, and `lamp_on`. The logging badge indicates that the signal is configured for logging.



Add Run-Time Assessments

When you create a baseline or equivalence real-time test case, you compare signal data to baseline or equivalence criteria that you specify in your test. If you plan to create simulation real-time test case, you can assess behavior by adding `verify` statements in Test Assessment blocks in your model or test harness. For more information about test case type, see “Add a Real-Time Test Case” on page 9-34.

`verify` statements evaluate logical expressions and return `pass`, `fail` or `untested` results. For more information about adding `verify` statements to Test Assessment blocks, see “Verify Model Simulation by Using when Decomposition” on page 3-14. For more information about using `verify` statements in real-time tests, see “Real-Time Testing Considerations” on page 9-25.

Set Configuration Parameters

Before you perform real-time testing, you must configure the code generation and data import and export parameters in your model or test harness.

Code Generation

A real-time test case requires a real-time system target file. To confirm or set the system target file for your model:

- 1 In the **Apps** tab, click **Simulink Coder**. In the **Prepare** section, click **Settings**.
- 2 In the Configuration Parameters dialog box, click **Code Generation** and verify that the **System target file** parameter is set to a Simulink Real-Time model. If the file is incorrect, next to **System target file** click Browse and select the Simulink Real-Time system target file.

If your model requires a different system target file, you can set the parameter using a test case or test suite callback. For more information, see “Assign Code to Real-Time Callbacks” on page 9-36.

For more information, see **System target file**.

Data Import/Export Format

To use your model in a real-time test case, your model must use a data format other than `dataset`. To set the data format:

- 1 Open the Configuration Parameters dialog box. In the **Modeling** tab, click **Model Settings**.
- 2 In the left pane of the Configuration Parameters dialog box, select **Data Import/Export**.
- 3 In the **Format options** section of the right pane, set the **Save format** field.

See Also

Apps

Simulink Real-Time Explorer

Functions

`set_param` | `setVariable` | `verify` | `assert`

Related Examples

- “Reuse Desktop Test Cases for Real-Time Testing” on page 9-27
- “Projector Controller Testing Using `verify` and Real-Time Tests” on page 6-222

More About

- “Manage Test Harnesses” on page 2-29

Create and Run Real-Time Tests

You can use the Simulink Test Manager to convert desktop simulation test cases or to create real-time baseline, equivalence, or simulation tests. You can then define how the tests run by specifying the test settings in the Test Manager. Some settings that appear in the Test Manager are only available in the real-time test cases. For information on general Test Manager settings, see “Specify Test Properties in the Test Manager” on page 6-173.

For information about configuring your model for real-time testing and setting up the target computer, see “Prepare for Real-Time Testing” on page 9-31.

Add a Real-Time Test Case

To add a real-time test case to an existing test file:

- 1 In the Simulink Toolstrip, on the **Apps** tab, under **Model Verification, Validation, and Test**, click **Simulink Test**.
- 2 In the **Tests** tab, click **Simulink Test Manager**.
- 3 In the Test Manager, in the **File** section, click **Open** and select a test file.
- 4 Select **New > Test Case > Real-Time Test**.
- 5 In the New real-time test dialog box, set the **Test Type** parameter and click **Create**. You can select a baseline, equivalence, or simulation real-time test:
 - **Baseline** — Compares the signal data returned from the target computer to the baseline data in the test case. To compare a real-time execution result to a model simulation result, add the model baseline result to the real-time test case and apply optional tolerances to the signals in the **Baseline Criteria** section. See “Baseline Criteria” on page 6-184 and “Set Signal Tolerances” on page 6-169.
 - **Equivalence** — Compares the signal data from a simulation to a real-time test, or compares two real-time tests. When you select this type of test, you can specify the **Simulation 1 on target** and **Simulation 2 on target** parameters. To run a real-time test on the target computer and compare results to a model simulation, select **Simulation 1 on target** and clear **Simulation 2 on target**. The test case displays two simulation sections, **Simulation 1 (Real-Time)** and **Simulation 2**. To compare the signal data from two real-time tests, select **Simulation 1 on target** and **Simulation 2 on target**. The test case displays two simulation sections, **Simulation 1 (Real-Time)** and **Simulation 2 (Real-Time)**.

You can apply optional tolerances to the signals. See “Equivalence Criteria” on page 6-186 and “Set Signal Tolerances” on page 6-169.
 - **Simulation** — Assesses the test result using only **verify** statements and real-time execution. If no **verify** statements fail, and the real-time test executes, the test case passes.

For information about converting a desktop test case, see “Reuse Desktop Test Cases for Real-Time Testing” on page 9-27.

Specify the System Under Test

In the Test Manager, expand the **System Under Test** section and specify the model and how to load the real-time application. You can also optionally specify a test harness or simulation overrides.

Select Model and Test Harness

You can specify the model you want to test by selecting the model in the currently active Simulink window or by browsing for the model. If your model has a test harness, you can select the test harness for the test case. For information on how to specify the model or test harness for your test, see “System Under Test” on page 6-176.

Load Application

The real-time application is built from your model or test harness. To choose the application to load, set the **Load Application From** parameter to:

- **Model** - This setting loads the application from the model, builds the real-time application, downloads it to the target computer and executes it on the target computer. Use this setting if you are running the real-time test for the first time, or if your model changed since the last real-time execution. This setting typically takes the longest to run.

Note If your test executes table iterations from Signal Editor scenarios or Test Sequence scenarios, select the **Model** settings.

- **Target Application** - Use this setting to send the target application from the host to a target computer, and execute the application. Use this setting if you want to load an already-built application on multiple targets.
- **Target Computer** - This setting updates the parameters in the test case and executes the test. Use this setting to execute an application that is already loaded on the real-time target computer.

This table summarizes which steps and callbacks execute for each option. If your test uses callbacks and test iterations, the callbacks run for each iteration.

Test Case Execution Step	Load Application From Parameter Setting		
	Model	Target Application	Target Computer
Executes pre-load callback	Yes	Yes	Yes
Loads Simulink model	Yes	No	No
Executes post-load callback	Yes	No	No
Sets Signal Editor scenario	Yes	No	No
Builds real-time application from model	Yes	No	No
Downloads real-time application to target computer	Yes	Yes	No
Sets runtime parameters	Yes	Yes	Yes
Runs Test Sequence scenarios	Yes	No	No

Test Case Execution Step	Load Application From Parameter Setting		
	Model	Target Application	Target Computer
Executes pre-start real-time callback	Yes	Yes	Yes
Executes real-time application	Yes	Yes	Yes
Executes cleanup callback	Yes	Yes	Yes

Override Simulation Settings

For real-time tests, you can use the **Simulation Settings Overrides** section to override the simulation stop time, which can be useful in debugging real-time test failures. Consider a 60-second test that returns a `verify` statement failure at 15 seconds due to a bug in the model. After debugging your model, you execute the real-time test to verify the fix. You can override the stop time to terminate the execution at 20 seconds, which reduces the time it takes to verify the fix.

Assign Code to Real-Time Callbacks

Use the **Pre-start real-time application** callback to execute commands before the application executes on the target computer. Real-time test callbacks execute in a sequence with the model load, build, download, and execute steps. Callbacks and step execution depend on how the test case loads the application.

Sequence	Load application from: Model	Load application from: Target application	Load application from: Target computer
Executes first	Preload callback	Preload callback	Preload callback
	Post-load callback	—	—
	Pre-start real-time callback	Pre-start real-time callback	Pre-start real-time callback
Executes last	Cleanup callback	Cleanup callback	Cleanup callback

If your test requires a different system target file, you can set the parameter using a test case or test suite callback. After the real-time test executes, set the parameter to its original setting with a cleanup callback. For example, this callback opens the `sltestProjectorController` model and sets the `SystemTargetFile` to `speedgoat.tlc`.

```
openExample("sltestProjectorController");
set_param("sltestProjectorController",...
    "SystemTargetFile","speedgoat.tlc");
```

For more information about callbacks, see “Callbacks” on page 6-179. For information about using callbacks to set the system target file, see “Code Generation” on page 9-33.

Create Test Iterations

You can use the **Iterations** section in the Test Manager to execute iterations in real-time tests. Use iterations to execution real-time tests that sweep through parameter values or Signal Editor

scenarios. The Test Manager groups the results by iteration. For more information on setting up iterations, see “Test Iterations” on page 6-139. You can create:

- Table iterations from a parameter set — Define several parameter sets in the **Parameter Overrides** section of the test case. Under **Table Iterations**, click **Auto Generate** and select **Parameter Set**.
- Table iterations from Signal Editor scenarios — If your model or test harness uses a Signal Editor input, below the **Table Iterations** table, click **Auto Generate** and select **Signal Editor Scenario**. If you use a Signal Editor scenario, you must set **Load Application From** to Model.
- Table iterations from Test Sequence scenarios — If your test harness uses Test Sequence block scenarios, you can create an iteration for each scenario in the Test Manager. Below the **Table Iterations** table, click **Auto Generate** and select **Test Sequence Scenario**. If you use Test Sequence scenarios, load the application from the model.
- Scripted iterations — Use scripts to iterate using model variables or parameters. For example, assume you are testing an oscillator system and use a Test Sequence block to create a square wave test signal using the parameter frequency.

Symbols	Step	Transition	Next Step
Input Output 1. waveform Local Constant Parameter frequency	Initialize waveform = 0; step_2 waveform =square(et*frequency)*0.5 + 0.5;	1. true	step_2 ▼

In the Test Manager, in the **Scripted Iterations** section, you can use real-time test scripted iterations to cover a frequency sweep from 5 Hz to 35 Hz. The script iterates the value of frequency in the Test Sequence block.

```
%% Iterate over frequencies to determine best oscillator settings
```

```
% Create parameter sets
```

```
freq = 5.0:1.0:35.0;
```

```
for i_iter = 1:length(freq)
```

```
    % Create iteration object
```

```
    testItr = sltestiteration();
```

```
    % Set parameters
```

```
    setVariable(testItr, 'Name', 'frequency', 'Source', ...  
                'Test Sequence', 'Value', freq(i_iter));
```

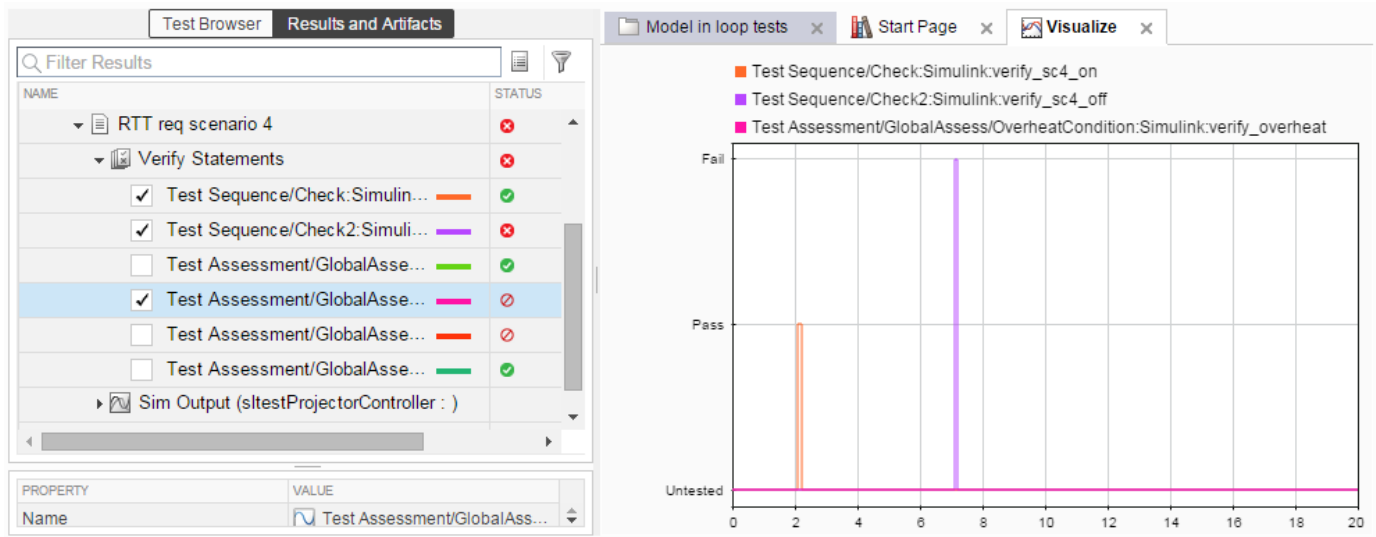
```
    % Register iteration
```

```
    addIteration(sltest_testCase, testItr);
```

```
end
```

Run Test and View Real-Time Test Results

After you define the test, you can execute the test by clicking **Run** in the Test Manager. You can access results in the **Results and Artifacts** pane. If your test is a baseline or equivalence test, the right pane compares the test signal data to the data and tolerances you defined in the baseline or equivalence criteria section. If your test includes **verify** statements, in the **Results and Artifacts** pane, expand the **Verify Statements** section to select an assessment and view results. The right pane displays the pass, fail, and untested results.



You can create a report from the test results. See “Generating a Test Results Report” on page 7-20 and “Customize Test Results Reports” on page 7-21.

See Also

Apps

Simulink Test Manager

Functions

setVariable | verify | assert

Related Examples

- “Reuse Desktop Test Cases for Real-Time Testing” on page 9-27
- “Projector Controller Testing Using verify and Real-Time Tests” on page 6-222

More About

- “Specify Test Properties in the Test Manager” on page 6-173

Testing Custom C/C++ Code

- “Importing and Testing Custom C/C++ Code” on page 10-2
- “Import Custom Code for Unit Testing Using API Commands” on page 10-5
- “Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard” on page 10-11

Importing and Testing Custom C/C++ Code

You can test custom C or C++ code by importing it into Simulink using the Code Importer wizard in the Test Manager or API commands at the MATLAB command line. You can perform unit testing to test a subset of your C code or integration testing to test your complete C or C++ code. When you import your code, the code importer:

- Converts the C code functions to Simulink C Caller blocks and saves those blocks in a Simulink library
- Creates an internal harness for each Simulink C Caller block
- Generates a test file

For unit tests, the code importer additionally creates a sandbox to isolate the imported functions.

Import Code Using the Wizard or the API

To import and test custom C or C++ code using the Code Importer wizard, open the Test Manager and select **New > Test for C/C++ Code**. The wizard steps are shown in the “Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard” on page 10-11 example. After you import the code using the wizard, the **Test Browser** pane of the Test Manager shows the generated test file, test suites, and test cases, and automatically fills the library model and test harness fields and coverage settings for each test case. You can customize the test cases in the Test Manager by adding inputs, assessments, links to requirements, or other options.

The “Import Custom Code for Unit Testing Using API Commands” on page 10-5 example shows the classes and methods for importing code. The code importer sets the property values for the generated library, test file, test suites, test cases, and coverage. You can customize the test cases by using API commands to add inputs, assessments, links to requirements, or other options.

Before you run the test cases, either change the current folder to the folder that contains the generated artifacts or add the generated data dictionary to the path. Then run the test cases and view the coverage and other test results.

Code Importer Generated Artifacts

The code importer creates these artifacts:

- A Simulink library with C Caller blocks for each imported custom code function.
- An internal test harness for each C Caller block. For each generated harness, the solver is `FixedStepDiscrete`, and coverage is enabled.
- An MLDATX test file. The test file includes a test suite and test case for each C Caller block. The code importer also sets these coverage types:
 - Decision coverage
 - Condition coverage
 - MCDC coverage
 - Lookup table coverage
 - Signal range coverage
 - Coverage for Simulink Design Verifier blocks

- Relational boundary coverage
- Signal range coverage
- Simulink data dictionary

Additionally, for unit tests only, the code importer creates:

- A sandbox to isolate the functions being tested
- Stubs, if any source files have undefined symbols

Limitations and Workarounds

These limitations and workarounds apply to using the custom C or C++ Code Importer.

General Limitations and Workarounds

- For integration tests, if your code includes C++ functions, add wrappers around them to make them C-compatible before importing the functions into Simulink.
- These C types, and functions with formal arguments that use these types, cannot be imported. Global variables that use these types are not exposed as ports on the C Caller block:
 - Structures with unions or pointer members
 - Functions with inputs that have more than one level of pointer indirection (for example, `>=**`)
 - Functions that return a pointer
 - Types with names longer than 2048 characters, and functions and variables that use those types
- If your code has many global variables, use a Stateflow chart instead of an Initialize Function block to set the variables to their initial values in your Simulink model.
- If a header file or C or C++ file contains assembly code that is defined in the function body, the code importer does not import that function. This limitation applies only if the assembly code is not compatible with the host computer. To import the function,
 - For integration tests, replace the assembly code by using an `#ifdef` directive.
 - For unit tests, the function is moved automatically to the `auto_stub.c` file with an empty body. To import the function into Simulink, manually stub the function in `man_stub.c`.
- For target-specific code, if the code accesses absolute memory addresses, comment out that code to prevent the simulation from failing.

Header Files

- If the same header file is included multiple times and each inclusion is preceded by a different preprocessor directive (such as, `#define X 1`, `#define X 2`), the code might not import correctly.
- If assembly code is defined in the header file, define a compatible macro by using an `#ifdef` directive. For example, if your code is:

```
#define XYZ(K,L) {\
asm("MOVLW " __mkstr(K) ); \
asm("MOVLW " __mkstr(L) ); \
}
```

replace it with:

```
#ifndef IS_SL_IMPORT
#define XYZ(K,L) {\
asm("MOVLW " ____mkstr(K) ); \
asm("MOVLW " ____mkstr(L) ); \
}
#else
// a valid implementation
#endif
```

Then, add `IS_SL_IMPORT` to the list of defines when you import code.

Unit Tests

These limitations and workarounds apply only to unit tests.

- Only C code is supported for importing.
- If a source or header file contains a function definition that is included multiple times in the source file being imported, update the code so the function definition appears only once.
- All included files, using `#include`, should be self-contained, that is, they compile on their own. Specifically, a header should have header guard and include all other headers that it needs.

See Also

`sltest.CodeImporter` | `sltest.CodeImporter.SandboxSettings` | `createSandbox`

Related Examples

- “Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard” on page 10-11
- “Import Custom Code for Unit Testing Using API Commands” on page 10-5

Import Custom Code for Unit Testing Using API Commands

This example shows how to use the API to import custom C code into Simulink for unit testing. Unit tests test one or more functions in isolation from the custom code library. For unit tests, the Simulink Test code importer generates a test sandbox and a library containing a C Caller block from the specified custom code.

Heat Pump Controller Custom Code Files

The complete code for the heat pump controller is in these C code source and header files:

The source files are in the `src` directory:

- `tempController.c`
- `utils.c`

The header files are in the `include` directory:

- `tempController.h`
- `utils.h`
- `controllerTypes.h`

The `tempController.c` file contains the algorithm for the custom C code for a heat pump unit. The `heatpumpController` function in that file uses the room temperature (`Troom_in`) and the set temperature (`Tset`) as inputs. The output is `pump_control_bus` type structure with signals that control the fan, heat pump, and the direction of the heat pump (heat or cool). The `pump_control_bus` structure has these fields: `fan_cmd`, `pump_cmd`, and `pump_dir`. The `pump_control_bus` structure type is defined in the `controllerTypes.h` file. The output of the `heatpumpController` function is:

Temperature Condition	System State	Fan Command	Pump Command	Pump Direction
$ T_{room_in} - T_{set} < \Delta T_{fan}$	Idle	0	0	IDLE
$\Delta T_{fan} \leq T_{room_in} - T_{set} < \Delta T_{pump}$	Fan only	1	0	IDLE
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} < T_{room_in}$	Cooling	1	1	COOLING
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} > T_{room_in}$	Heating	1	1	HEATING

The `heatpumpController` function uses two utility functions, `absoluteTempDifference` and `pumpDirection`, which are defined in the `utils.c` file. The `absoluteTempDifference` function returns the absolute difference between `Tset` and `Troom_in` as a double. The `pumpDirection` function returns one of these `PumpDirection` type enum values:

Temperature Condition	Pump Direction
$T_{set} < T_{room_in}$	COOLING
$T_{set} > T_{room_in}$	HEATING

The `PumpDirection` enum type is defined in the `controllerTypes.h` file.

Import Heat Pump Controller Code and Automatically Create Stubs

This example uses only `tempController.c` to create and import a test sandbox into Simulink. You use the sandbox for performing unit testing only on the `heatpumpController` function and not on

the complete code. Generating the sandbox automatically creates stubs for the utility functions used by the `heatpumpController` function, `absoluteTempDifference` and `pumpDirection`. Since the utility functions are not defined in the `tempController.c` file and the `utils.c` and `utils.h` files are not included, the code importer creates stubs so the code does not error.

Set Up the CodeImporter Object

Create an instance of a `CodeImporter` object for the heat pump controller custom code. Setting the `OutputFolder` property to `$pwd$` evaluates the string in between the `$` symbols as a MATLAB expression. Set `OutputFolder` to `$pwd$` to specify the current folder as the output folder. Set the `SourceFiles` property to the `tempController.c` file in the `src` directory. Use the `$` symbols to specify the file location for the `CustomCode` property, too.

```
obj = sltest.CodeImporter('heatpumpController');  
  
obj.OutputFolder = "$pwd$";  
  
obj.CustomCode.SourceFiles = "$fullfile('src','tempController.c')$";  
obj.CustomCode.IncludePaths = fullfile('include');  
obj.CustomCode.GlobalVariableInterface = true;
```

Create the Test Sandbox

Configure the `CodeImporter` object with the desired test type and sandbox settings.

To create a test sandbox for the specified `heatpumpController` function in the custom code, set the `TestType` property `UnitTest`. For this example, use the `GenerateAggregatedHeader` sandbox mode. For information about the different sandbox modes, see `sltest.CodeImporter.SandboxSettings`.

Setting `SandboxSettings.CopySourceFiles` to `true` copies the specified source file into the test sandbox.

Note that you can use `GenerateAggregatedHeader` sandbox mode only with a single source file.

```
obj.TestType = "UnitTest";  
  
obj.SandboxSettings.Mode = "GenerateAggregatedHeader";  
obj.SandboxSettings.CopySourceFiles = true;
```

Create the sandbox. This test sandbox is isolated from the original custom code library. Setting `Overwrite` to `on` overwrites the existing test sandbox, if one exists. By default, `Overwrite` is `off`.

```
obj.createSandbox('Overwrite','on');
```

The `createSandbox` method creates the `heatpumpController_sandbox` directory in the specified output folder, which in this example is the current working folder.

The sandbox directory contains the following subdirectories:

- `src`: This directory contains the copied source file, `tempController.c`.
- `include`: This directory contains the required include files to compile `tempController.c` in the sandbox `src` directory. This directory also contains the `aggregatedHeader.h` file, which contains all the required symbols to compile `tempController.c`.

- **autostub:** This directory contains the `auto_stub.c` and `auto_stub.h` files, which hold the automatically generated stubs for `absoluteTempDifference` and `pumpDirection` utility functions.
- **manualstub:** This directory contains the `man_stub.c` and `man_stub.h` files, which define any manually specified stubs. By default, these files do not define any functions.

Import the Test Sandbox

Import the sandbox code into Simulink.

```
obj.import('Functions','heatpumpController');
```

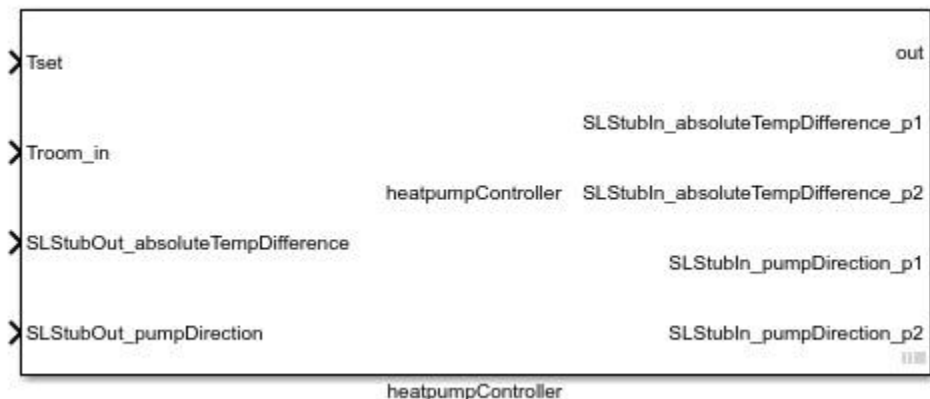
The `import` function creates the sandbox. It also creates a library that contains a C Caller block called `heatpumpController`, which contains an internal test harness that you can use to perform unit testing on the `heatpumpController`. The library is attached to a Simulink Data Dictionary that defines `pump_control_bus` and `PumpDirection` as a `Simulink.Bus` object and a `Simulink` enumeration signal, respectively.

The C Caller block is attached with an internal test harness for unit testing the `heatpumpController` function.

Input and Output Ports on the C Caller Block

Because you set `CustomCode.GlobalVariableInterface` to `true` before importing, the `import` function creates stubs for the `absoluteTempDifference` and `pumpDirection` global variables in `auto_stub.c` and creates ports for them. For information, see “Automatically infer global variables as function interfaces”.

This is the C Caller block, `heatpumpController`, generated from the `heatpumpController` function:



These are the automatically generated global variables in the `auto_stub.c` file for `absoluteTempDifference` and `pumpDirection`:

```

/*****
/* Generated Global Variables for Stubbed Functions Interface */
/*****
double SLStubIn_absoluteTempDifference_p1;
double SLStubIn_absoluteTempDifference_p2;

```

```

double SLStubOut_absoluteTempDifference;
double SLStubIn_pumpDirection_p1;
double SLStubIn_pumpDirection_p2;
PumpDirection SLStubOut_pumpDirection;

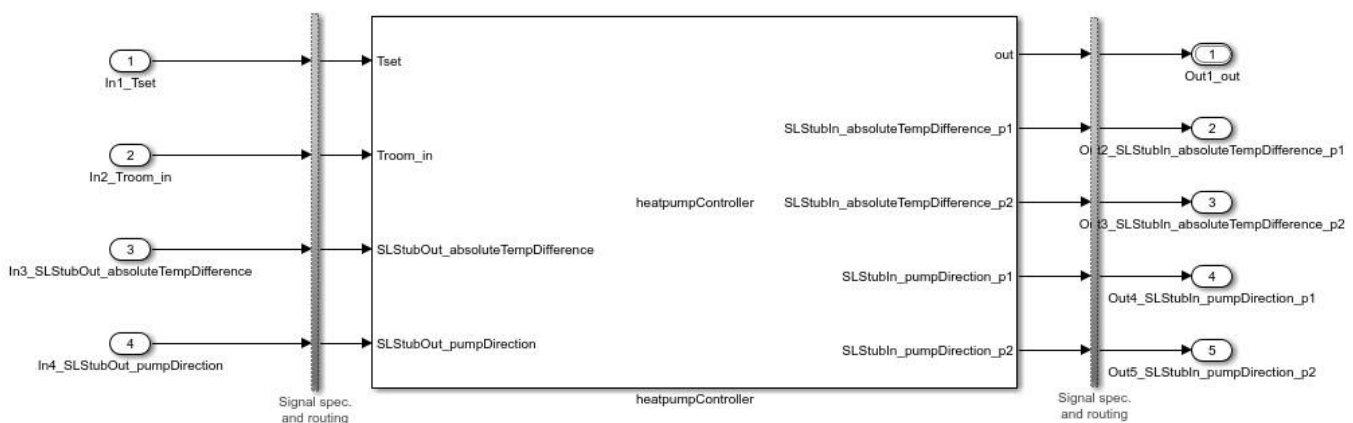
double absoluteTempDifference( double absoluteTempDifference_p1, double absoluteTempDifference_p2)
{
    SLStubIn_absoluteTempDifference_p1 = absoluteTempDifference_p1;
    SLStubIn_absoluteTempDifference_p2 = absoluteTempDifference_p2;
    return SLStubOut_absoluteTempDifference;
}

PumpDirection pumpDirection( double pumpDirection_p1, double pumpDirection_p2)
{
    SLStubIn_pumpDirection_p1 = pumpDirection_p1;
    SLStubIn_pumpDirection_p2 = pumpDirection_p2;
    return SLStubOut_pumpDirection;
}

```

In the automatically generated stubs for the `absoluteTempDifference` function, the global variables `SLStubIn_absoluteTempDifference_p1` and `SLStubIn_absoluteTempDifference_p2` save the input arguments of the function. The function returns the value stored in `SLStubOut_absoluteTempDifference`. Similarly, `pumpDirection` saves the input arguments and returns `SLStubOut_pumpDirection`.

To use the test harness created using automatically created stubs, refer to the next figure. Add buses for the inputs and outputs. To enable connecting a Simulink signal for simulation, connect inputs for `Tset`, `Troom_in` and the expected outputs from the global variables, `SLStubOut_absoluteTempDifference` and `SLStubOut_pumpDirection`. Likewise, connect outputs as shown in the figure. You can use the inputs and outputs to observe the internal values passed by `heatpumpController` to the `absoluteTempDifference` and `pumpDirection` subfunction calls.



Change Automatically Created Stubs to Manual Stubs

In cases where, for example, you want to generate the intended output of the stubs as input to the automatically generated ports, you can substitute manual stubs for the stubs automatically generated

when the sandbox was created. After you switch to using manual stubs, you update the existing sandbox and import it again.

Manually Modify an Automatically Generated Stub Function

You can manually provide definitions for the automatically generated stub functions by updating the `man_stub.c` and `man_stub.h` files in the `manualstub` directory.

```
manualstubpath = fullfile([obj.LibraryFileName.char '_sandbox'], 'manualstub');
helperFunctionToUpdateManualStubs(manualstubpath);
```

The `helperFunctionToUpdateManualStubs` function updates the manual stub files in the test sandbox.

The updated function definition of `absoluteTempDifference` is:

```
double absoluteTempDifference(double Tset, double Troom_in){
    return (double)fabs(Tset - Troom_in);
}
```

The updated function definition of `pumpDirection` is:

```
PumpDirection pumpDirection(double Tset, double Troom_in){
    return Tset > Troom_in ? HEATING : COOLING;
}
```

Update the Existing Test Sandbox

To use the manual stub functions, update the sandbox to reflect your changes. Setting the `Overwrite` option to `off` preserves the changes made to the `manualstub` directory in the test sandbox.

```
obj.createSandbox('Overwrite', 'off');
```

After updating the test sandbox, the `autostub` directory is empty because you defined all of the undefined symbols in the specified custom code.

Import the Updated Test Sandbox

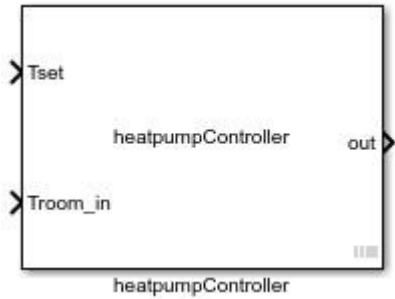
After updating the test sandbox, import the sandbox code into Simulink.

```
obj.import('Functions', 'heatpumpController');
```

The library contains a C Caller block called `heatpumpController` with updated ports, and The internal test harness attached to the C Caller block is also updated.

The import function updates the existing library and imports the `heatpumpController` function. Like when you used automatically generated stubs, the library is attached to a Simulink Data Dictionary that defines `pump_control_bus` and `PumpDirection` as a `Simulink.Bus` object and a Simulink enumeration signal, respectively.

The C Caller block ports reflect the changes made to the stub files. Because the manual implementation of the `absoluteTempDifference` and `pumpDirection` functions does not use any global variables, the C Caller block only has ports for the input arguments and return argument of the `heatpumpController` function. The internal test harness attached to the C Caller block is also updated.



You can use the created MLDATX test file to test the code. See “Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard” on page 10-11 for a testing example.

See Also

`sltest.CodeImporter` | `sltest.CodeImporter.SandboxSettings` | `createSandbox` | `Simulink.CodeImporter.CustomCode` | `import` | `Simulink.CodeImporter` | `Simulink.CodeImporter.ParseInfo` | `Simulink.CodeImporter.Options`

More About

- “Importing and Testing Custom C/C++ Code” on page 10-2

Conduct Unit Testing on Imported Custom Code by Using the Code Importer Wizard

This example shows how to use the Code Importer wizard to import custom C code for a heat pump controller into Simulink for unit testing. Unit tests test one or more functions in isolation from the custom code library. For unit tests, the Simulink Test Code Importer wizard generates a test sandbox and a library containing a C Caller block for each specified function from the custom code. For more information see “Importing and Testing Custom C/C++ Code” on page 10-2.

This example uses only `tempController.c` file to create and import a test sandbox into Simulink. You use the sandbox for performing unit testing only on the `heatpumpController` function in the `tempController.c` file and not on the complete code. Generating the sandbox automatically creates stubs for the utility functions used by the `heatpumpController` function, `absoluteTempDifference` and `pumpDirection`. Since the utility functions are not defined in the `tempController.c` file, and the `utils.c` and `utils.h` files are not included, the code importer creates stubs so the code does not error.

Heat Pump Controller Custom Code

The complete code for the heat pump controller is in these C code source and header files:

The source files are in the `src` directory:

- `tempController.c`
- `utils.c`

The header files are in the `include` directory:

- `tempController.h`
- `utils.h`
- `controllerTypes.h`

The `tempController.c` file contains the algorithm for the custom C code for a heat pump unit. The `heatpumpController` function in that file uses the room temperature (`Troom_in`) and the set temperature (`Tset`) as inputs. The output is the `pump_control_bus` type structure with signals that control the fan, heat pump, and the direction of the heat pump (heat or cool). The fields in the `pump_control_bus` structure are `fan_cmd`, `pump_cmd`, and `pump_dir`. The `pump_control_bus` structure type is defined in the `controllerTypes.h` file. The output of the `heatpumpController` function is:

The output of the `heatpumpController` algorithm is summarized in the following table:

Temperature Condition	System State	Fan Command	Pump Command	Pump Direction
$ T_{room_in} - T_{set} < \Delta T_{fan}$	Idle	0	0	IDLE
$\Delta T_{fan} \leq T_{room_in} - T_{set} < \Delta T_{pump}$	Fan only	1	0	IDLE
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} < T_{room_in}$	Cooling	1	1	COOLING
$ T_{room_in} - T_{set} \geq \Delta T_{pump}$ and $T_{set} > T_{room_in}$	Heating	1	1	HEATING

The `heatpumpController` function uses two utility functions, `absoluteTempDifference` and `pumpDirection`, which are defined in the `utils.c` file. The `absoluteTempDifference` function

returns the absolute difference between `Tset` and `Troom_in` as a double. The `pumpDirection` function returns one of these `PumpDirection` type enum values:

Temperature Condition Pump Direction

<code>Tset < Troom_in</code>	COOLING
<code>Tset > Troom_in</code>	HEATING

The `PumpDirection` enum type is defined in the `controllerTypes.h` file.

Open the Code Importer Wizard

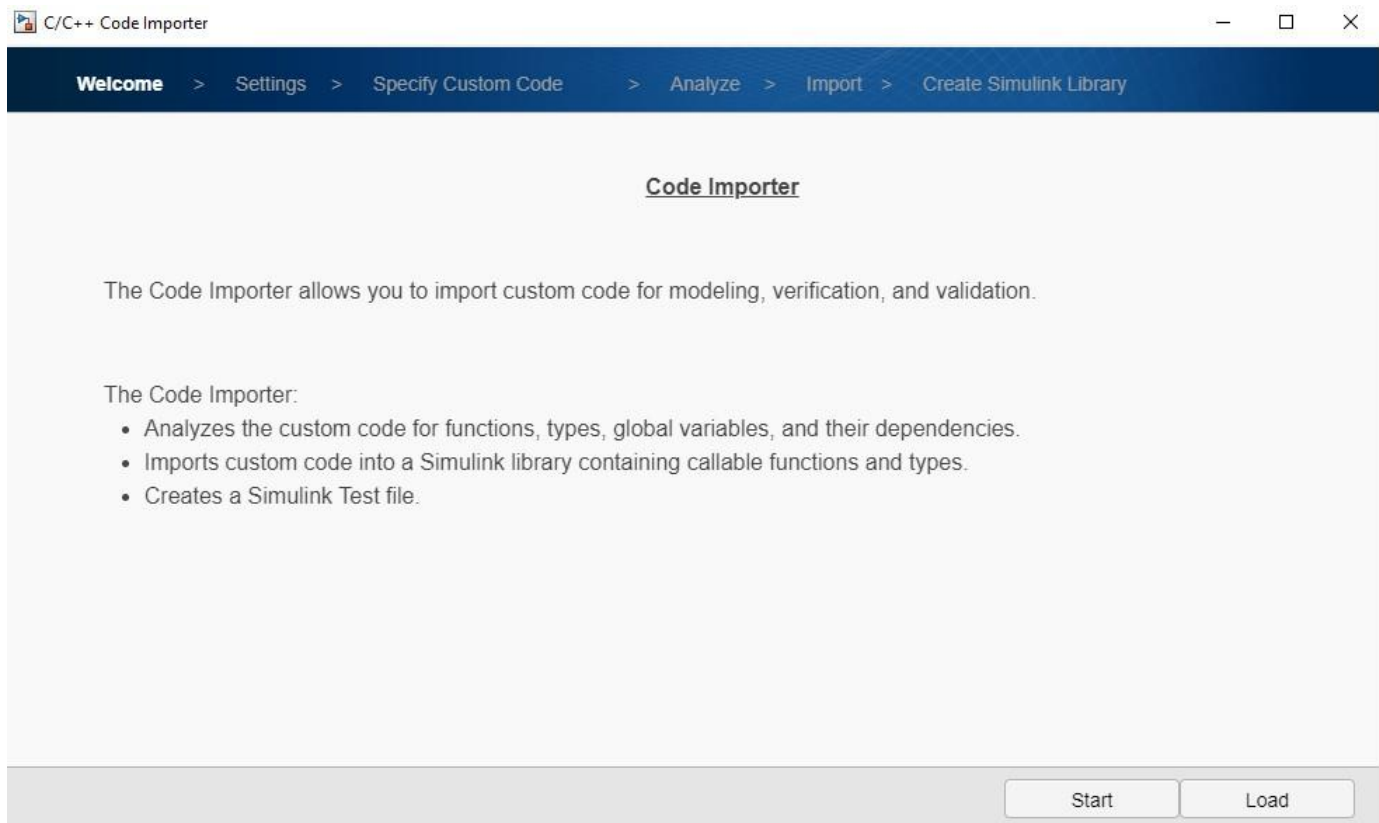
To open the Simulink Test C/C++ Code Importer wizard, first open the Simulink Test Manager.

`sltest.testmanager.view`

Then, select **New > Test for C/C++ code**.

Specify the Simulink Library and Testing Method

The wizard opens and displays the **Welcome** tab. Click **Start** to begin the import process.



On the **Settings** tab:

- 1 Enter the **Simulink library file name**. The generated Simulink library, test sandbox directory, and test file (MLDATX) use this name. Enter `heatpumpController`.
- 2 Specify the **Output folder** in which to save the generated artifacts. Enter the name of a writable folder.

3 Select **C Code Unit Testing** as the testing method.

The C Code Unit Testing method tests one C file or a subset of your custom code in isolation. This method creates a test sandbox from the specified source file or files, and imports the test sandbox into Simulink. Because the test sandbox only contains a subset of the C files, the Code Importer wizard automatically creates stubs for all of the undefined symbols used in the C files. This method supports testing C code only. For information on Integration Testing, see the `TestType` property of `sltest.CodeImporter`.

C/C++ Code Importer

Welcome > **Settings** > Specify Custom Code > Analyze > Import > Create Simulink Library

Specify the output Simulink library file name and output folder:

Simulink library file name:

Output folder:

Specify testing method:

☒ C Code Unit Testing

☐ Integration Testing

What to consider

Simulink library file name: This name will be used to name the Simulink library file and generated artifacts.

Output folder: The Simulink library and artifacts will be generated in this folder.

Select **C Code Unit Testing** if you want to isolate and test a subset of your custom code.

Select **Integration Testing** if you want to test your entire custom code.

About the selected option

The option you selected creates a sandbox from the specified source files. The sandbox will consist of:

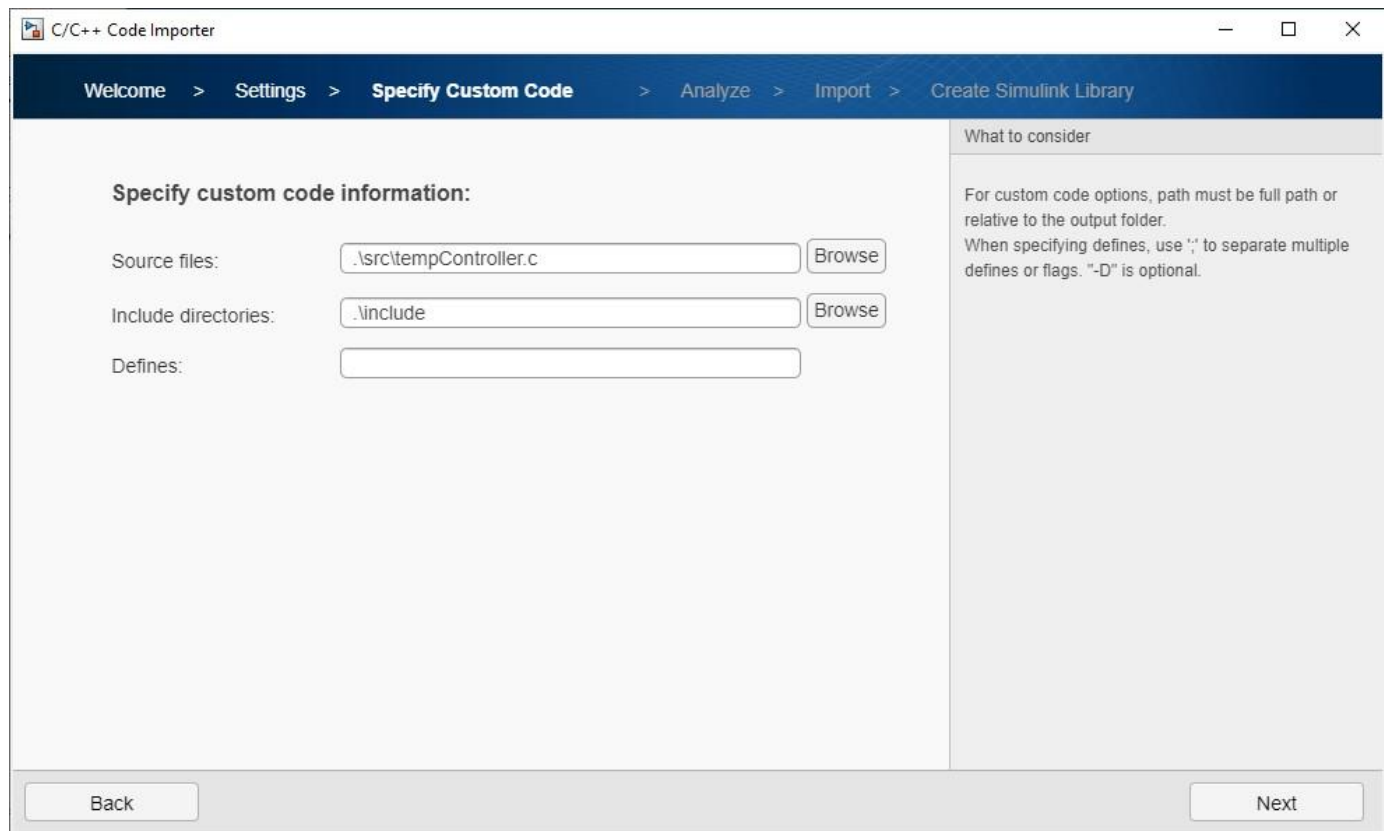
- Interface header file.
- Auto-stub files for undefined symbols.
- Manual stub files to allow manually editing undefined symbols.

Click **Next**.

Specify the Custom Code to Import

On the **Specify Custom Code** tab:

- 1 In **Source files**, specify the source file that contains the function to import for unit testing. Enter `.\src\tempController.c`.
- 2 In **Include directories**, specify the directories on which the specified source files depend. Enter `.\include`.
- 3 **Defines** specifies the compiler-specific defines. For this example, leave this field blank.



Click **Next**.

Specify the Test Sandbox Settings

On the **Analyze** tab, specify the sandbox settings.

Select the output test sandbox mode

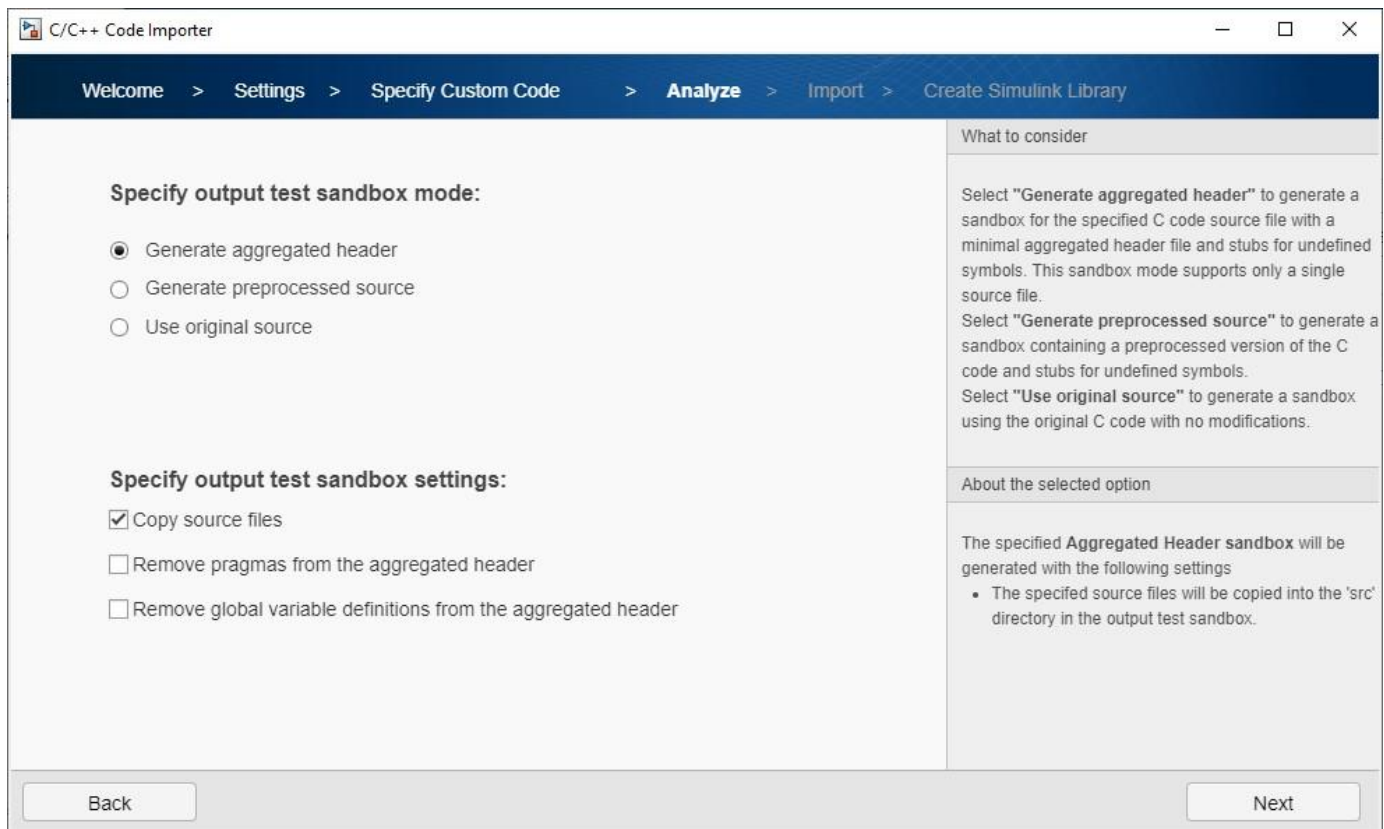
Each of the three test sandbox modes has settings that determine how the Code Importer wizard generates the sandbox and the sandbox artifacts.

Select **Generate aggregated header**. This test sandbox mode generates a minimal aggregated header file for the specified source file. The aggregated header file contains all the declarations of the symbols used by the specified source file.

Select the output test sandbox settings

Select only **Copy source files**. For this option, the Code Importer wizard copies the specified source file to the sandbox `src` directory.

For information on the other output test sandbox modes and settings, see the properties of `sltest.CodeImporter.SandboxSettings`.



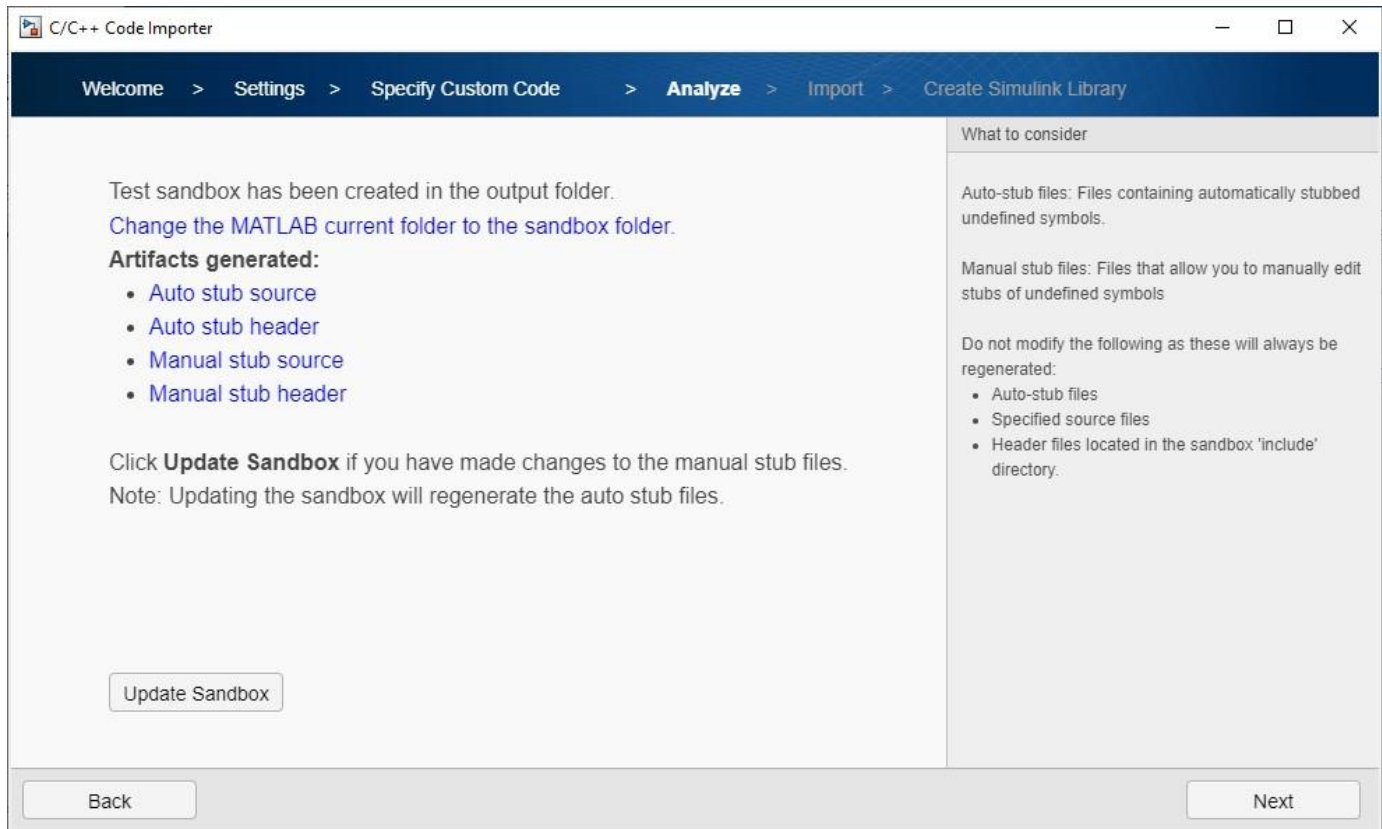
Click **Next** to create the test sandbox with the specified settings.

Create the Test Sandbox

The specified library file name determines the name of the generated sandbox. For this example, the sandbox name is `heatpumpController_sandbox`.

This example uses `tempController.c` and performs unit testing only on the `heatpumpController` function. Generating the sandbox automatically creates stubs for the `absoluteTempDifference` and `pumpDirection` utility functions used by the `heatpumpController` function. Since the utility functions are not defined in the `tempController.c` file, the stubs prevent the code from erroring.

When the sandbox is created, the confirmation screen displays.



The output sandbox folder contains the following subfolders:

- **src:** This folder contains the copied source file or files, which for this example is `tempController.c`.
- **include:** Depending on the test sandbox mode, this folder contains either the `aggregatedHeader.h` or `interfaceHeader.h` file. This example uses the `aggregatedHeader.h` file. The folder also contains and copies of the other header files required for the compilation of the specified source file or files. The `aggregatedHeader.h` file contains all the declarations of the symbols used by the specified source file. The `interfaceHeader.h` contains the declarations for functions, global variables, and types used by the specified source files. Simulink uses the generated interface header file during the import process.
- **autostub:** This folder contains the `auto_stub.c` and `auto_stub.h` files, which hold the automatically generated stubs for `absoluteTempDifference` and `pumpDirection` utility functions.
- **manualstub:** This folder contains the `man_stub.c` and `man_stub.h` files, which define any manually specified stubs. By default, these files do not define any functions.

NOTE: Do not modify the files in the `src`, `include`, or `autosub` folders of the generated sandbox.

After the sandbox has been created, you can view the created files:

- Click **Auto stub source** and **Auto stub header** to open the `auto_stub.c` and `auto_stub.h` files, respectively. In `auto_stub.c`, you can find the stubs for the utility functions used by `heatpumpController`, `absoluteTempDifference` and `pumpDirection`. In `auto_stub.h`, you can find the extern declarations of the stubbed functions.

- Click **Manual stub header** and **Manual stub source** to open the `man_stub.h` and `man_stub.c` files, respectively. Because you selected **Generate aggregated header** for the sandbox mode, `man_stub.h` includes the `aggregatedHeader.h` file. By default, the manual stub files do not have any function definitions.

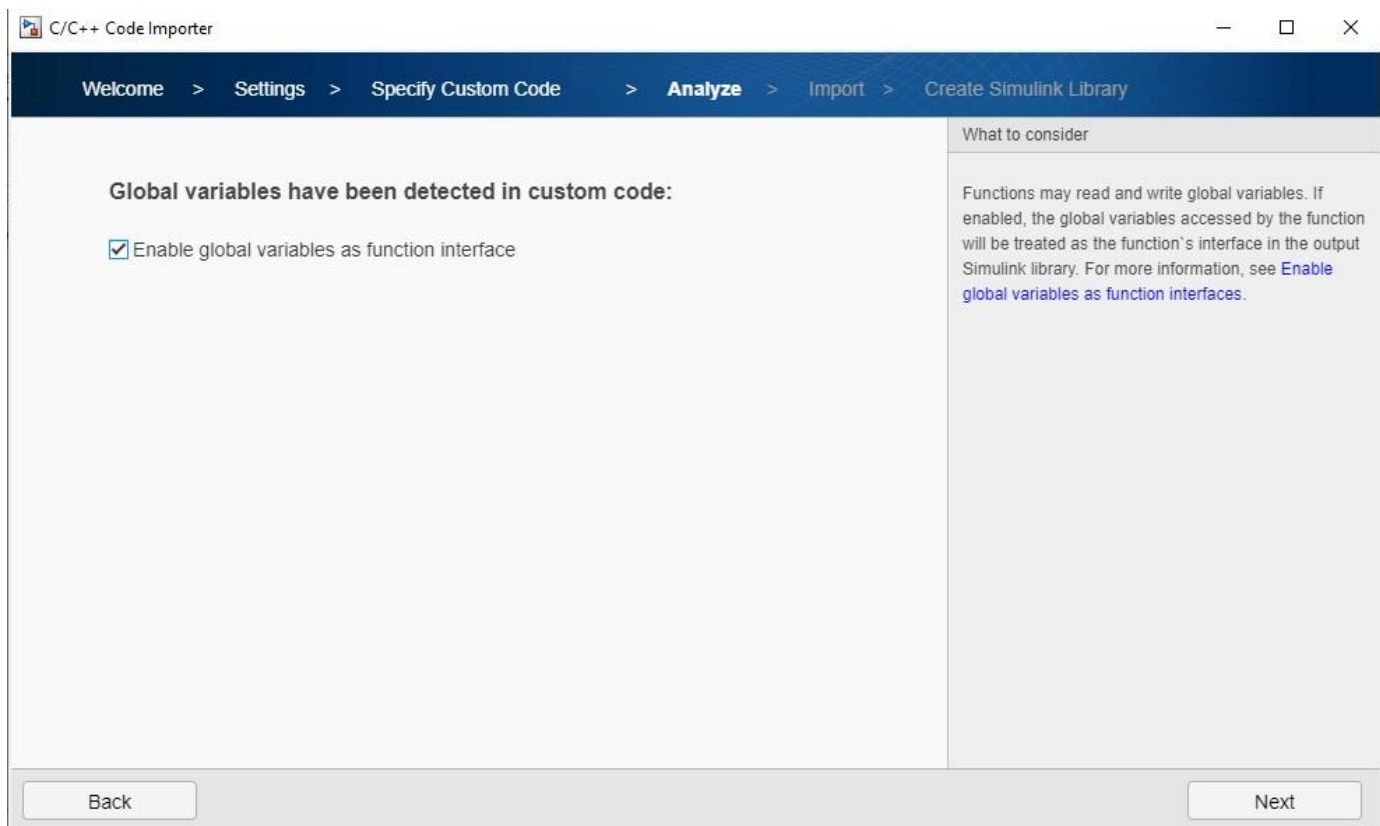
Click **Next**.

Specify Import Settings

Specify Using Global Variables as Function Interfaces

When the wizard detects global variables in the sandbox or custom code, you have the option to use those variables as input or outputs for the function and ports on the created C Caller block. For more information, see “Automatically infer global variables as function interfaces”.

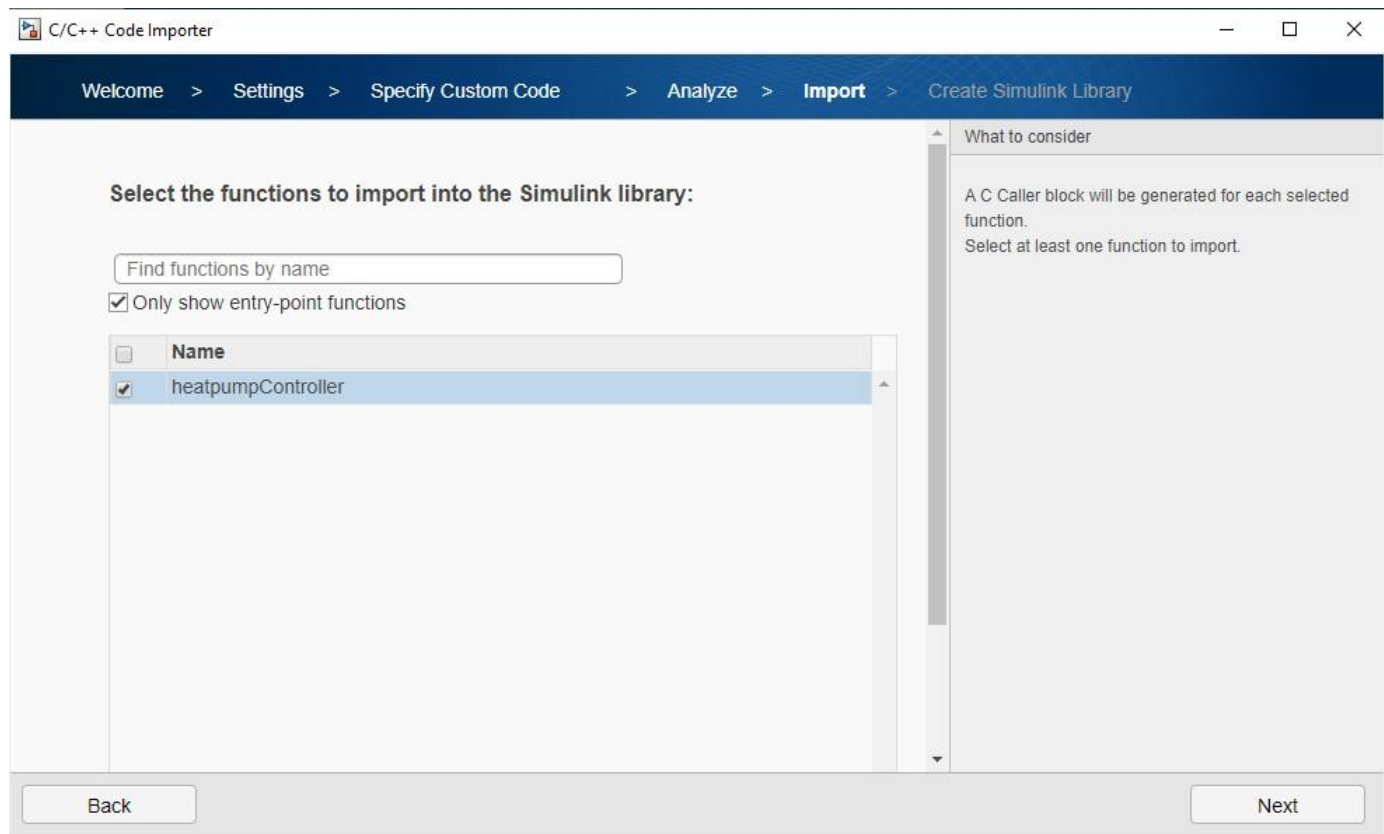
Select **Enable global variables as function interface** option.



Click **Next**.

Select Functions to Import

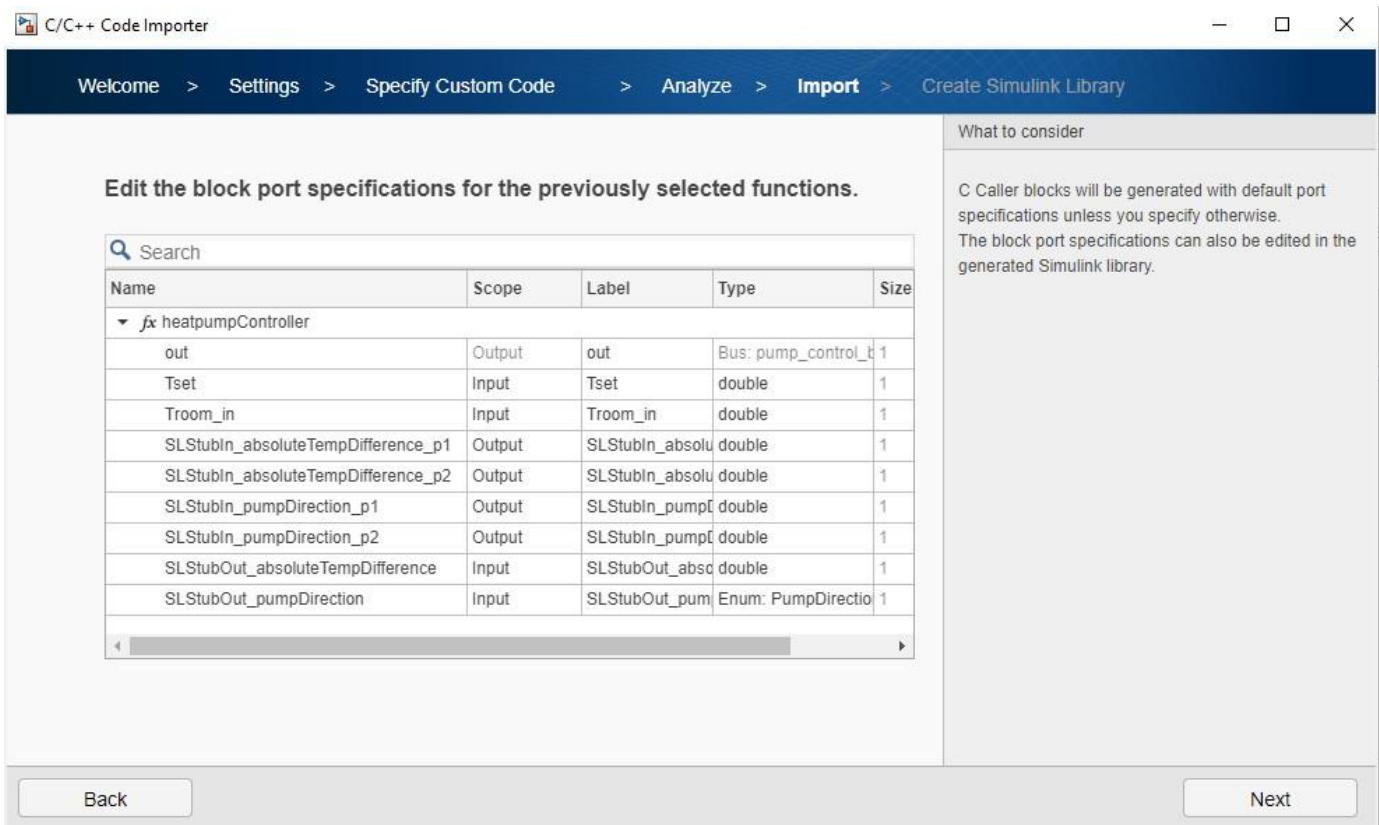
On the **Import** page, select the `heatpumpController` function to import into the Simulink library.



Click **Next**.

Set the Block Port Specifications

For the function you selected on the previous page, the wizard generates a function port specification. The selected ports are used for the generated C Caller block. Note that If the code to import had a function with a pointer output, you would need to specify the size of the output port on this page of the wizard.



In this example, the port specification table lists the formal arguments, `Tset`, `Troom_in`, and `out`, for the `heatpumpController` function, and six more from the automatically generated stubs. Because you selected **Enable global variables as function interface**, the wizard creates ports that are generated from the stubs for the `absoluteTempDifference` and `pumpDirection` functions. The stubs are in `auto_stub.c`.

These are the automatically generated global variables in the `auto_stub.c` file:

```

/*****
/* Generated Global Variables for Stubbed Functions Interface */
/*****
double SLStubIn_absoluteTempDifference_p1;
double SLStubIn_absoluteTempDifference_p2;
double SLStubOut_absoluteTempDifference;
double SLStubIn_pumpDirection_p1;
double SLStubIn_pumpDirection_p2;
PumpDirection SLStubOut_pumpDirection;

double absoluteTempDifference(double absoluteTempDifference_p1, double absoluteTempDifference_p2)
{
    SLStubIn_absoluteTempDifference_p1 = absoluteTempDifference_p1;
    SLStubIn_absoluteTempDifference_p2 = absoluteTempDifference_p2;
    return SLStubOut_absoluteTempDifference;
}

PumpDirection pumpDirection(double pumpDirection_p1, double pumpDirection_p2)
{

```

```

SLStubIn_pumpDirection_p1 = pumpDirection_p1;
SLStubIn_pumpDirection_p2 = pumpDirection_p2;
return SLStubOut_pumpDirection;
}

```

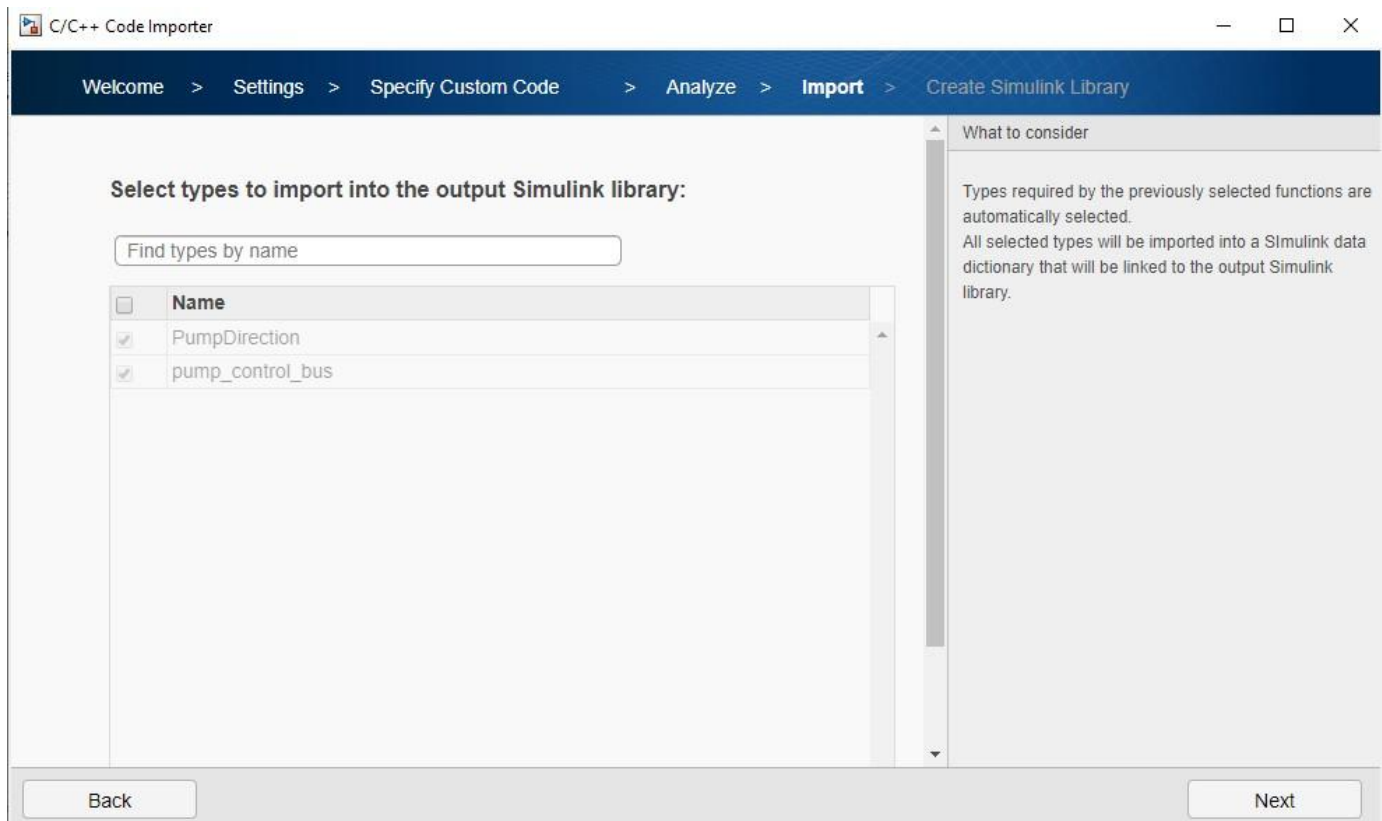
In the automatically generated stubs for `absoluteTempDifference`, the global variables `SLStubIn_absoluteTempDifference_p1` and `SLStubIn_absoluteTempDifference_p2` save the input arguments. The function returns the value stored in `SLStubOut_absoluteTempDifference` as its output. Similarly, `pumpDirection` saves the input arguments and returns `SLStubOut_pumpDirection`.

In this example, `absoluteTempDifference`, `SLStubIn_absoluteTempDifference_p1` and `SLStubIn_absoluteTempDifference_p2` are outputs. The global variable `SLStubOut_absoluteTempDifference` is an input.

Do not make any changes to the port specification. Click **Next**.

Specify Types to Import

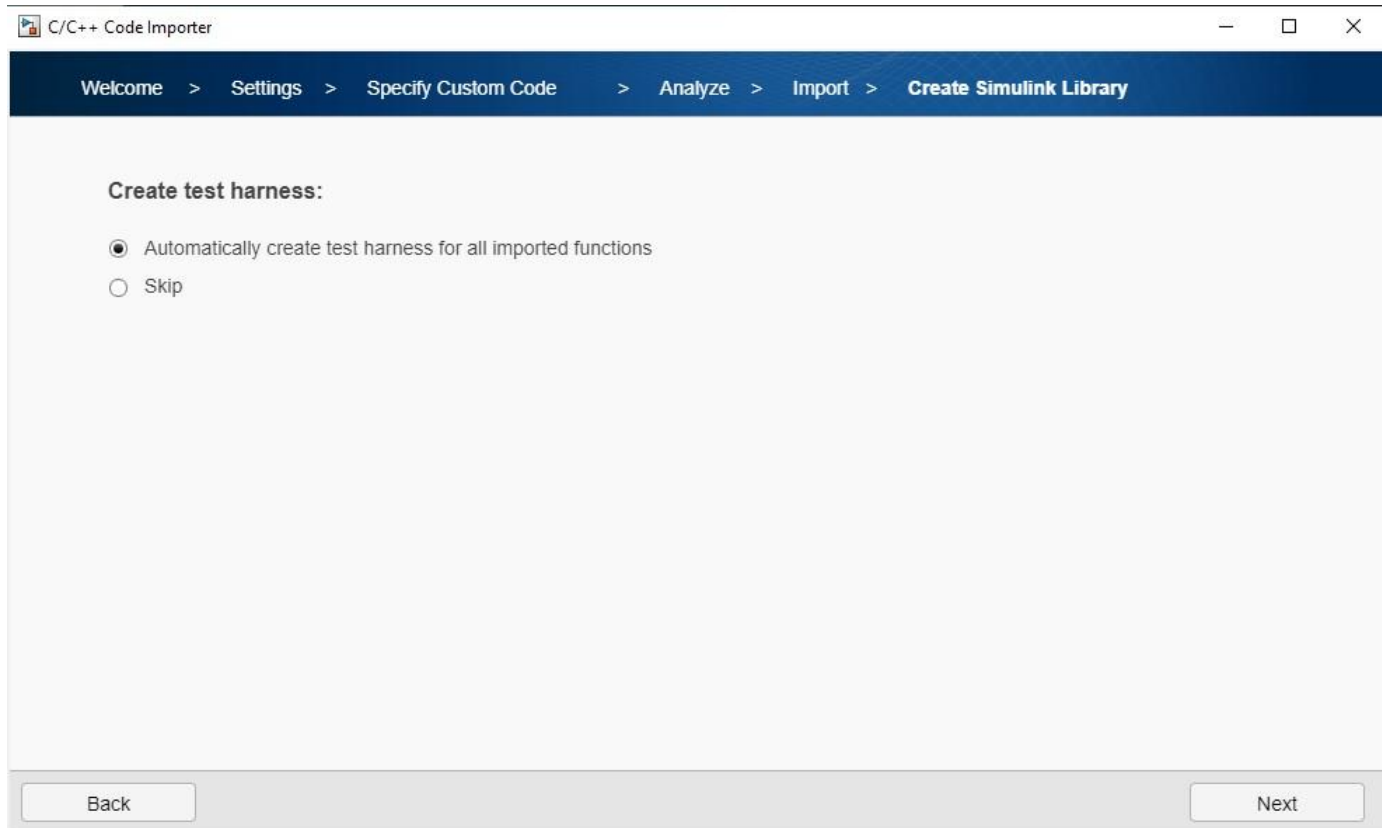
Select the types to import into Simulink. Because the `pump_control_bus` and `PumpDirection` types are required by the `heatpumpController` function, they are selected and dimmed. The wizard creates a Simulink data dictionary containing these types and links the dictionary to the generated library.



Click **Next** to display a summary of the generated library. Click **Next** again to continue.

Create a Test Harness

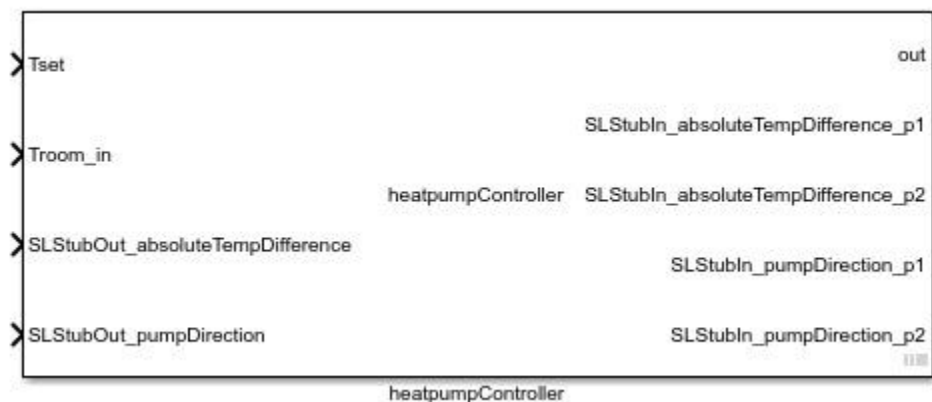
Select **Automatically create test harness for all imported functions**.



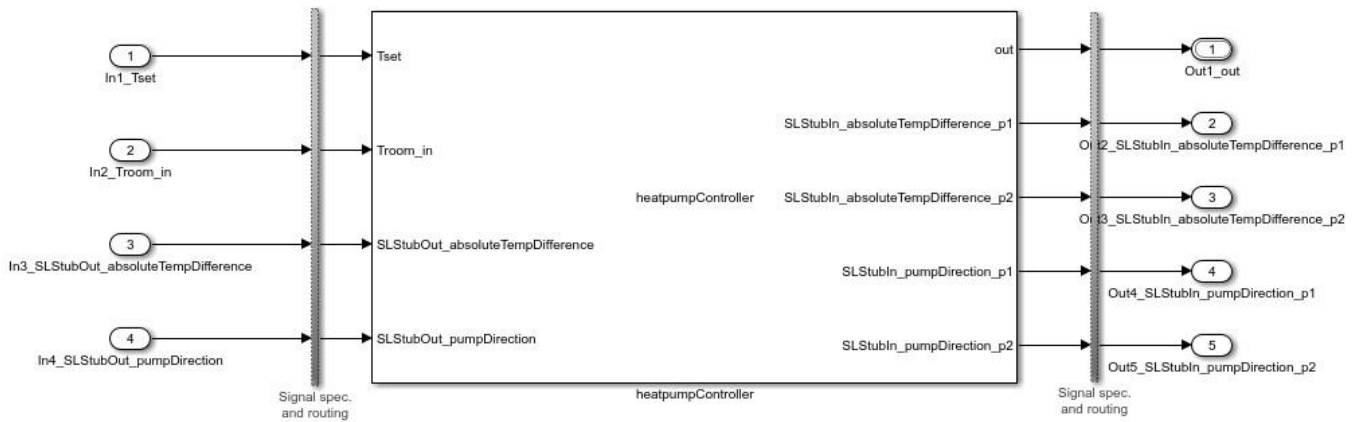
Click **Next** to generate the Simulink library.

After the code imports, the wizard creates a library attached to a Simulink data dictionary that defines `pump_control_bus` and `PumpDirection` as a `Simulink.Bus` object and a Simulink enumeration signal, respectively.

The C Caller block created in the Simulink library is:

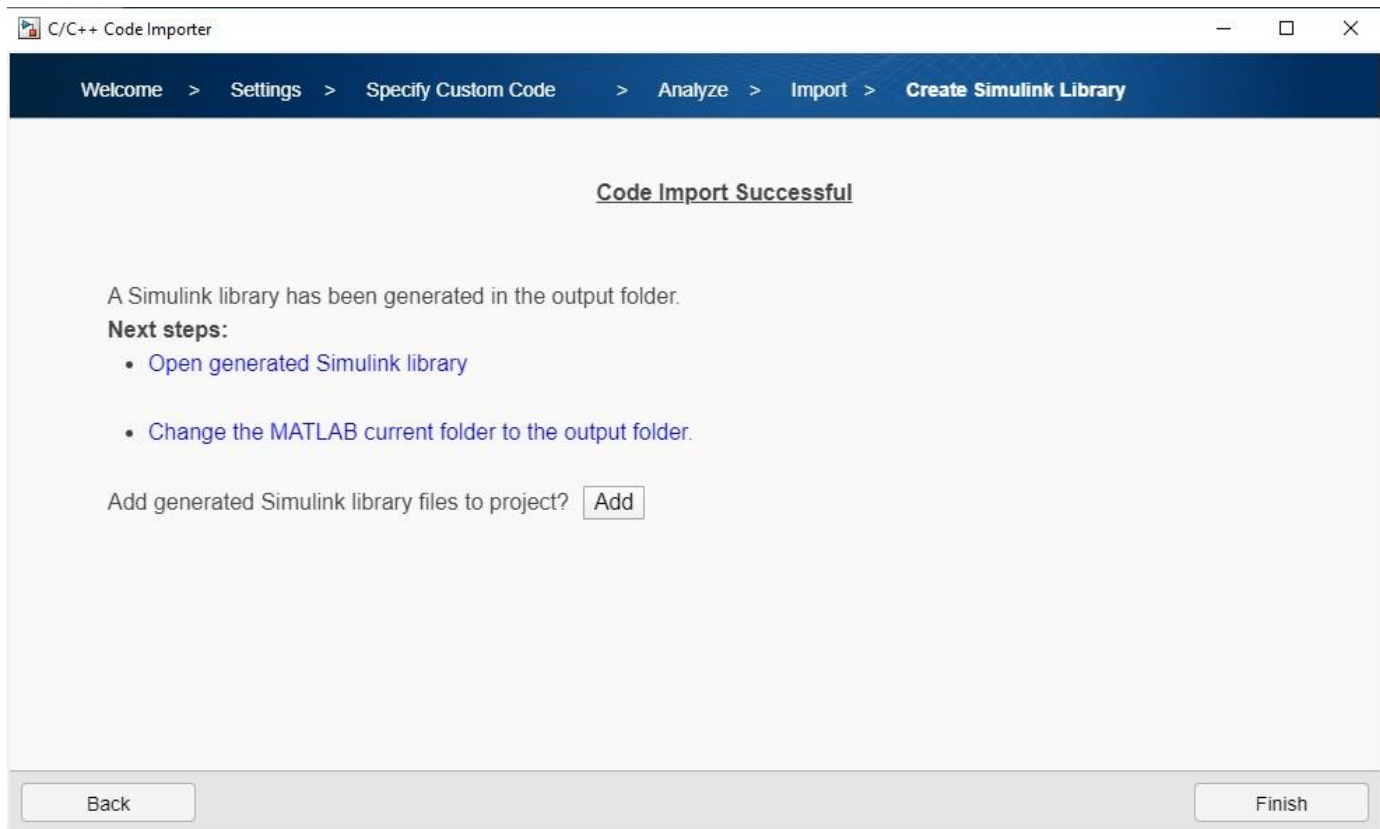


Click in the lower right corner of the block to open its internal test harness:



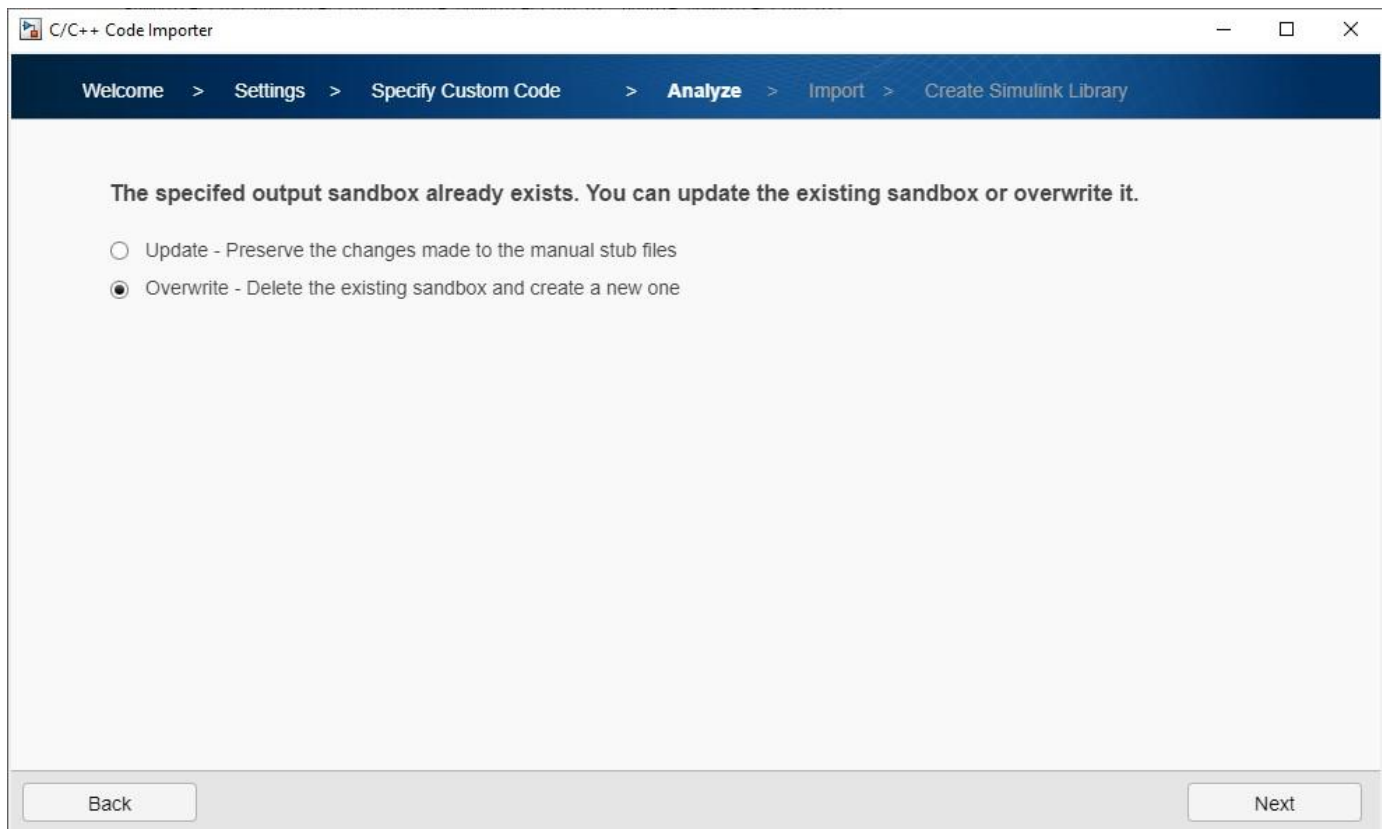
On the Code Import Successful page, click **Change the MATLAB folder to the output folder**.

Do not click **Finish**. Continue to the following section to manually update the stubs for undefined symbols.



Update Test Sandbox with Manual Stubbing

To manually create the stub files for `absoluteTempDifference` and `pumpDirection`, click the **Analyze** tab in the wizard toolbar and then click **Next** twice to display this page:



The Code Importer detects the sandbox you created. Select **Overwrite** and then, click **Next**.

Outside of the wizard, but leaving the wizard open, copy the files in the `updated_manualstub` directory and paste them into the `manualstub` directory of the sandbox.

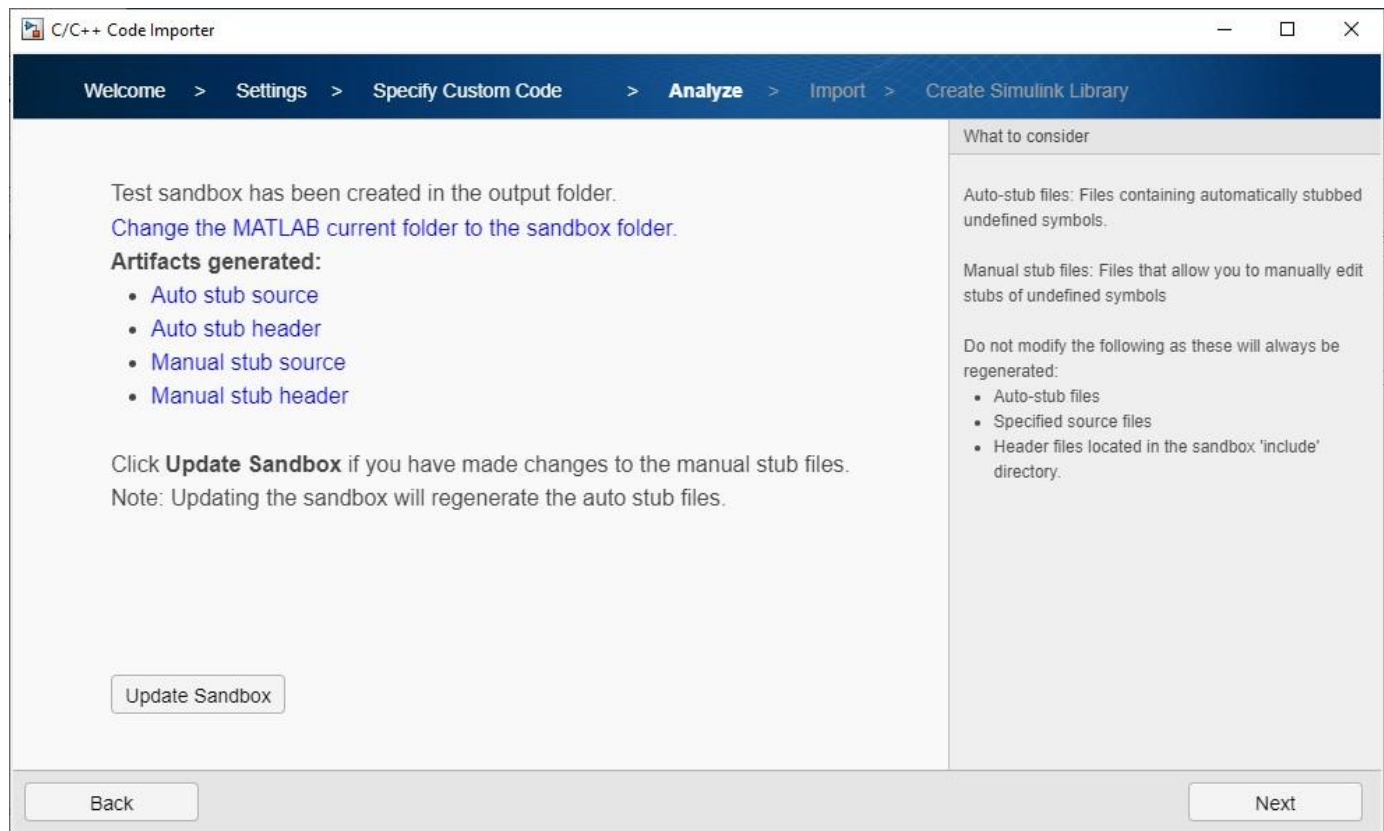
In the `man_stub.c` file, edit the function definition of `absoluteTempDifference`:

```
double absoluteTempDifference(double Tset, double Troom_in){
    return (double)fabs(Tset - Troom_in);
}
```

and the function definition of `pumpDirection`:

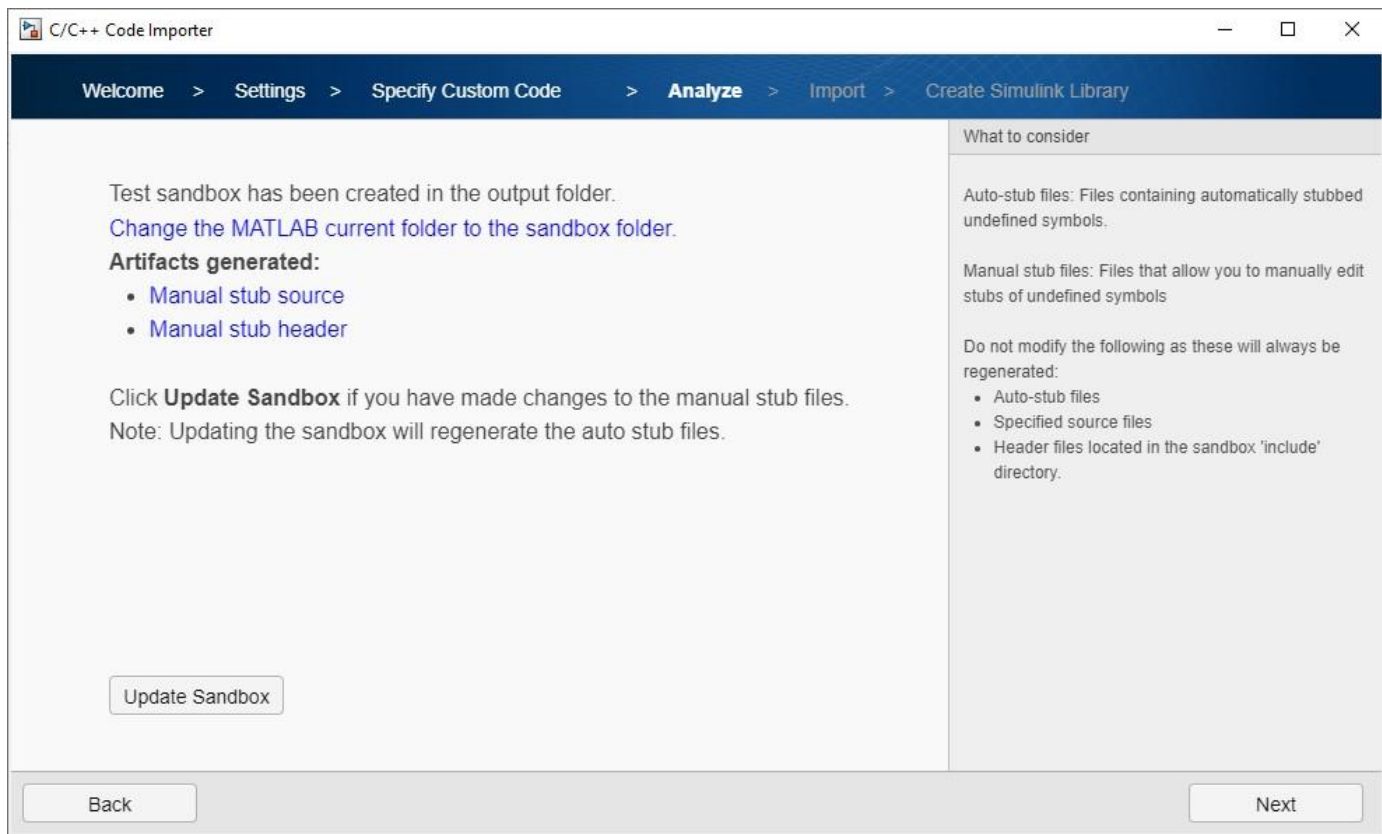
```
PumpDirection pumpDirection(double Tset, double Troom_in){
    return Tset > Troom_in ? HEATING : COOLING;
}
```

Return to the Code Importer wizard.

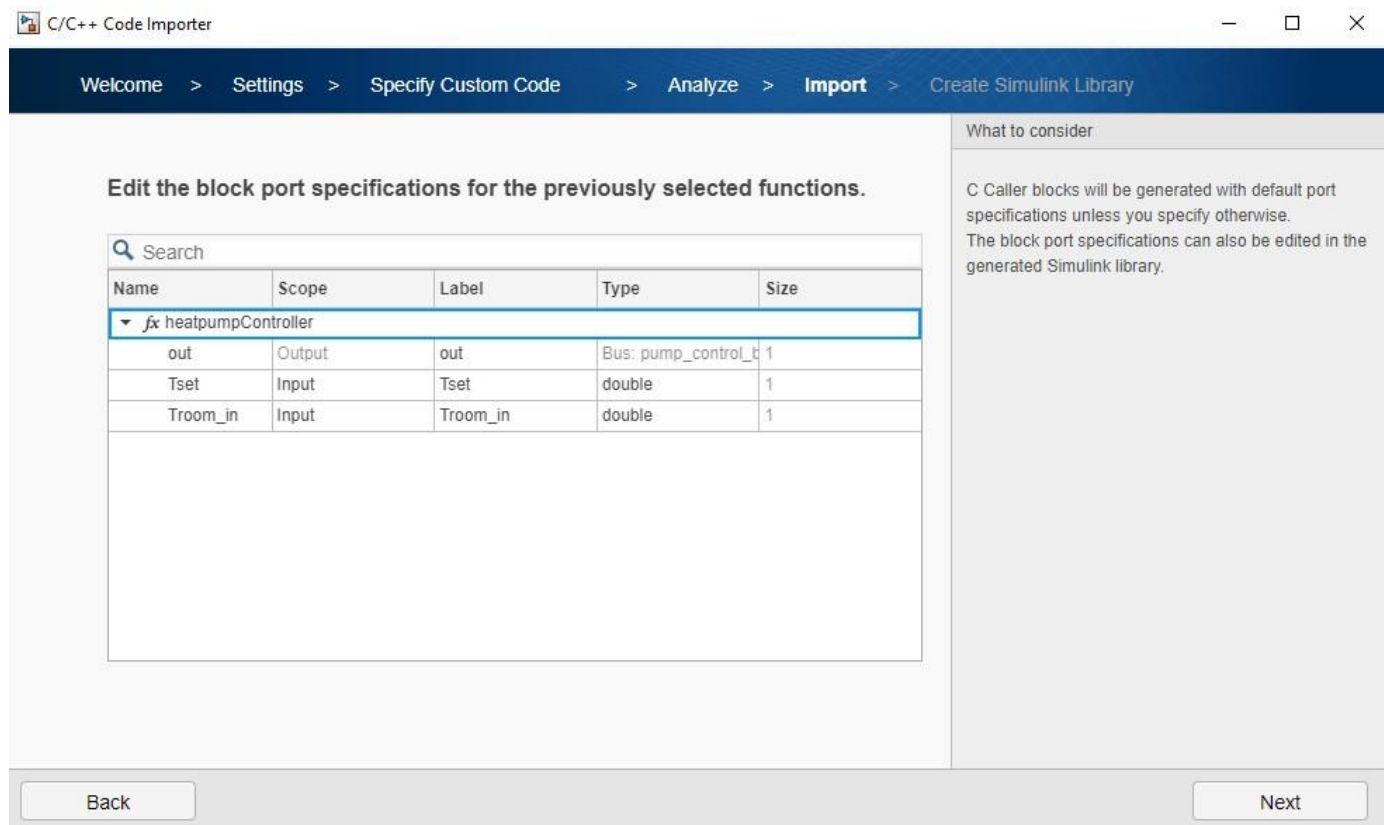


After modifying the manual stub files, return to the wizard and click **Update Sandbox**.

Because you manually defined the definitions for the `absoluteTempDifference` and `pumpDirection` functions, there are no symbols to automatically stub, no artifacts are generated for the `autostub` directory in the sandbox and it is empty.



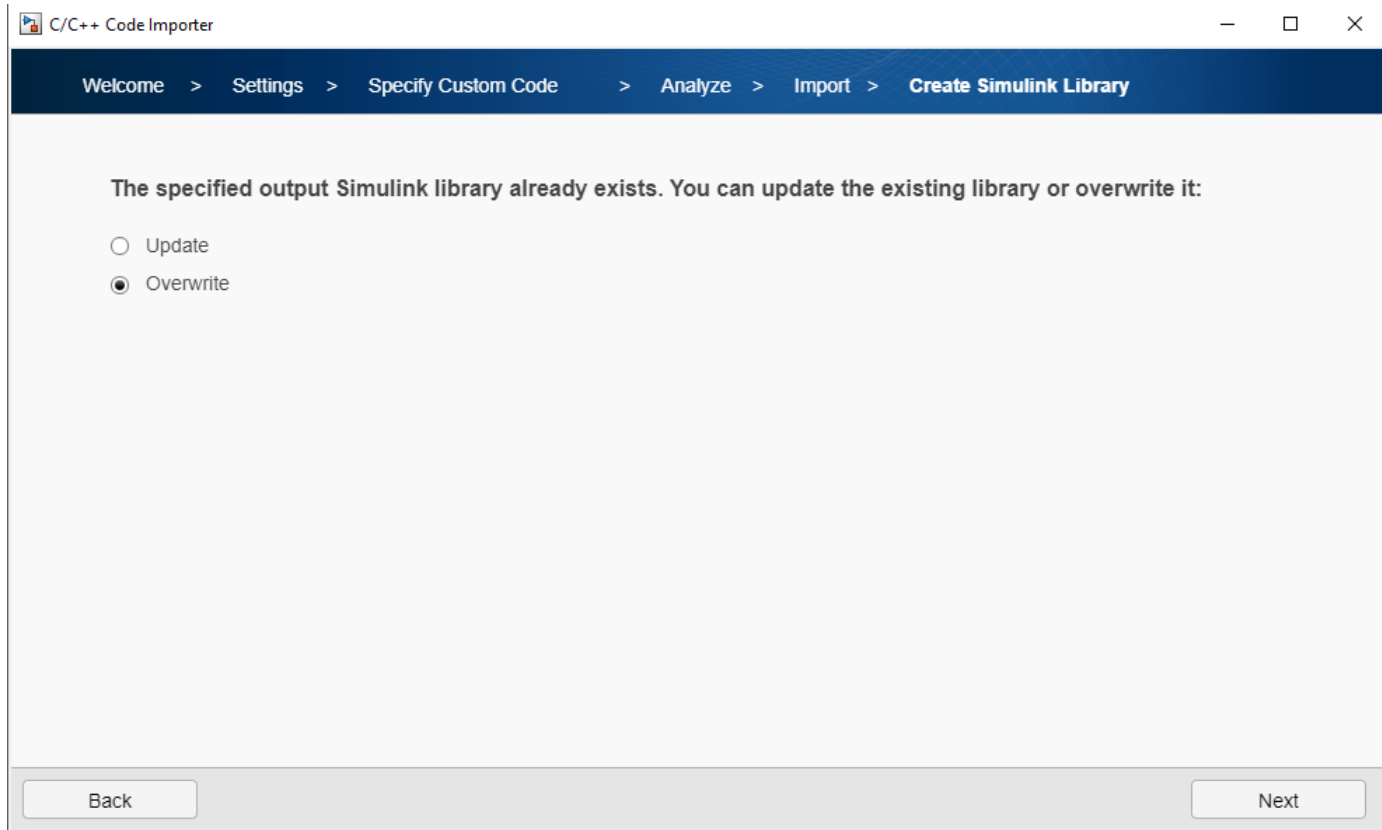
Click **Next** until you reach the block port specification page:



Because the manual implementation of `absoluteTempDifference` and `pumpDirection` does not have any global variables, only the formal arguments and the return argument appear as ports.

Click **Next**.

On the **Create Simulink Library** tab, select **Overwrite**.



Click **Next** through the remaining pages until you reach the **Code Import Successful** page.

Then click **Finish**. If desired, click **Yes** in the dialog box that opens to save the current import settings as a JSON file, which you use to reload the settings into another Code Importer wizard session.

Test the Imported Code

In the Test Manager, in the **Test Browser** pane expand the **heatpumpController** test file and **heatpumpController/heatpumpController** test suite. Then select the **heatpumpController_Harness1** test case.

Specify Simulation Stop Time

In the **System Under Test** section, expand the **Simulation Setting and Release Overrides** section and set the **Stop Time** to 200.

SYSTEM UNDER TEST*

Model:

TEST HARNESS*

Harness:

SIMULATION SETTINGS AND RELEASE OVERRIDES*

Simulation Mode:

☐ Override model blocks in SIL/PIL mode to normal mode

Select releases for simulation:

☐ Start Time:

☒ Stop Time:

☐ Initial State:

Add Inputs

In the **Inputs** section, at the bottom of the **External Inputs** table, click **Add** to open the Add Input dialog box.

In the **Input File Specification**, enter `heatpumpControllerHarnessInput.xlsx`.

Under **Input Mapping**, select Block Name as the **Mapping Mode** and click **Map Inputs**. After the inputs appear in the Mapping Status table, click **OK**.

INPUT FILE SPECIFICATION

File: 

☐ Add iterations to run this input

▶ SHEETS AND/OR RANGE SPECIFICATION

▼ INPUT MAPPING

Mapping Mode: 

☒ Compile the system under test

▼ MAPPING STATUS

Some inports were not assigned any values during mapping and have been set to ground values to enable the model to simulate

PORT	BLOCK NAME	MAPPED SIGNAL	STATUS
1	heatpumpController_Harness1/In...		✓
2	heatpumpController_Harness1/In...		✓
3	heatpumpController_Harness1/In...		!
4	heatpumpController_Harness1/In...		!

Add Assessments

In the **Logical and Temporal Assessments** section, add assessments for each temperature condition:

EN...	NAME	ASSESSMENT	REQUIREMENTS
☒	Assessment1	▶ At any point of time, whenever <code>abs(Troom_in - Tset) < 2</code> is true then, with no delay, <code>fan_cmd == uint8(0) & pump_cmd == uint8(0) & pump_dir == IDLE</code> must be true	None
☒	Assessment2	▶ At any point of time, whenever <code>abs(Troom_in - Tset) >= 2 & abs(Troom_in - Tset) < 4</code> is true then, with no delay, <code>fan_cmd == uint8(1) & pump_cmd == uint8(0) & pump_dir == IDLE</code> must be true	None
☒	Assessment3	▶ At any point of time, whenever <code>abs(Troom_in - Tset) >= 4 & Tset < Troom_in</code> is true then, with no delay, <code>fan_cmd == uint8(1) & pump_cmd == uint8(1) & pump_dir == COOLING</code> must be true	None
☒	Assessment4	▶ At any point of time, whenever <code>abs(Troom_in - Tset) >= 4 & Tset > Troom_in</code> is true then, with no delay, <code>fan_cmd == uint8(1) & pump_cmd == uint8(1) & pump_dir == HEATING</code> must be true	None

For information on assessments, see “Assess Temporal Logic by Using Temporal Assessments” on page 3-85.

In the Symbols pane, add the symbols definitions:

- ▼  **Troom_in**
 - Name: Input Conversion Subsystem:2
 - Path: heatpumpController_Harness1/Input Conversion Subsystem
 - Port Index: 2
 - Field/Element: <type an expression>
 - ▼  **Tset**
 - Name: Input Conversion Subsystem:1
 - Path: heatpumpController_Harness1/Input Conversion Subsystem
 - Port Index: 1
 - Field/Element: <type an expression>
 - ▼  **fan_cmd**
 - Name: heatpumpController:1
 - Path: heatpumpController_Harness1/heatpumpController
 - Port Index: 1
 - Field/Element: .fan_cmd
 - ▼  **pump_cmd**
 - Name: heatpumpController:1
 - Path: heatpumpController_Harness1/heatpumpController
 - Port Index: 1
 - Field/Element: .pump_cmd
 - ▼  **pump_dir**
 - Name: heatpumpController:1
 - Path: heatpumpController_Harness1/heatpumpController
 - Port Index: 1
 - Field/Element: .pump_dir
 - ▼  **HEATING**
 - Expression: PumpDirection.HEATING
 - ▼  **COOLING**
 - Expression: PumpDirection.COOLING
 - ▼  **IDLE**
 - Expression: PumpDirection.IDLE
-

Run the Test and View Results

Click **Run** to run the test.

When the test completes, in the **Results and Artifacts** pane, expand the **Results**. All of the assessments pass.

The screenshot shows the 'Results and Artifacts' pane. On the left, a list of results is shown under the heading 'Results: 2021-Feb-14 16:31:11'. The list includes 'heatpumpController_Harness1' (status 1), 'Sim Output (heatpumpController_Harness1)' (status 1), 'Logical and Temporal Assessment' (status 1), and 'Assessment1' (status 1, selected). Below this are 'Assessment2', 'Assessment3', and 'Assessment4', all with status 1. The main pane displays 'Assessment1' details. It includes an 'ASSESSMENT' section with a description: 'At any point of time, whenever $\text{abs}(\text{Troom_in} - \text{Tset}) < 2$ is true then, with no delay, $\text{fan_cmd} == \text{uint8}(0)$ & $\text{pump_cmd} == \text{uint8}(0)$ & $\text{pump_dir} == \text{IDLE}$ must be true'. A 'SYMBOLS' list on the right shows 'Troom_in', 'Tset', and 'fan_cmd'. Below this is an 'EXPRESSION TREE' section with the same assessment description. To the right of the expression tree is a 'PLOTS' section showing a coverage plot. The plot has a horizontal axis from 0 to 180. The vertical axis has three categories: 'Fail' (top), 'Pass' (middle), and 'Untested' (bottom). The plot shows a green bar at the 'Pass' level from 0 to approximately 30, and another green bar at the 'Pass' level from approximately 90 to 120. The rest of the plot is in the 'Untested' category.

To view the coverage results, select **heatpumpController_Harness1** under **Results** and expand the **Coverage Results** section. The coverage is 100% for both the `tempController.c` and `man_stub.c` files.

COVERAGE RESULTS				
ANALYZED MODEL	REPORT	COMPLEXI...	DECISION	EXECUTION
man_stub.c		3	100%	100%
tempController.c		3	100%	100%

See Also

More About

- “Importing and Testing Custom C/C++ Code” on page 10-2

Verification and Validation

- “Verify and Validate a Model and Code” on page 11-2
- “Test Model Against Requirements and Report Results” on page 11-4
- “Analyze Models for Standards Compliance and Design Errors” on page 11-9
- “Perform Functional Testing and Analyze Test Coverage” on page 11-11
- “Analyze Code and Perform Software-in-the-Loop Testing” on page 11-14

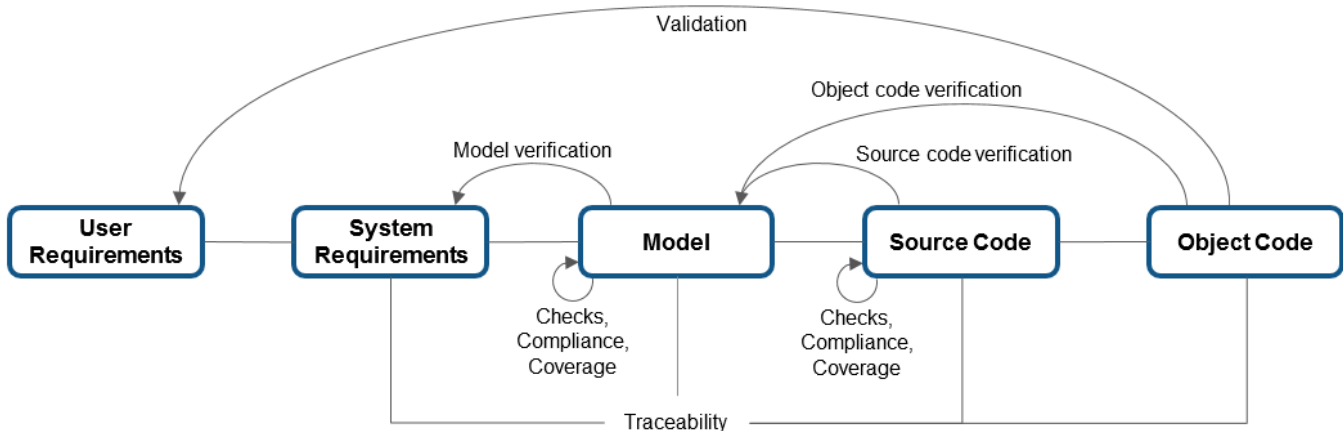
Verify and Validate a Model and Code

This tutorial shows how to establish traceability for requirements-based model testing, check a model for compliance, measure test coverage, generate tests to increase coverage, and generate and test code. Verifying your model and code throughout development increases confidence that your implemented system meets design requirements and specifications.

Early in the development process, you can create a high-level system model to serve as a basis for more detailed models. You can link items in the model to system requirements to establish traceability, and then use the model as an executable specification. As you verify the model, you can check for standards compliance, design errors, and run simulations to demonstrate that the results match the expected output. You can author tests, perform coverage analysis to measure test completeness, and generate additional tests. Finally, you can validate requirements by connecting your system prototype to user interface or plant hardware and executing it.

You can also verify generated and handwritten code. You can check your code for standards compliance, design errors, and analyze code metrics. You can then create tests and measure code coverage to determine the extent that the code is tested. For generated code, you can run the same tests to verify your model to demonstrate code-model equivalence. Finally, you can add tests, refine the system, and refine requirements until the design is fully functional, demonstrates no unintended behavior, and is fully tested.

This image shows how you can use verification and validation techniques throughout your development process.



To verify and validate a model, follow these steps in the tutorial:

- 1 "Test Model Against Requirements and Report Results" (Requirements Toolbox)
- 2 "Analyze Models for Standards Compliance and Design Errors" (Requirements Toolbox)
- 3 "Perform Functional Testing and Analyze Test Coverage" (Requirements Toolbox)
- 4 "Analyze Code and Perform Software-in-the-Loop Testing" (Requirements Toolbox)

See Also

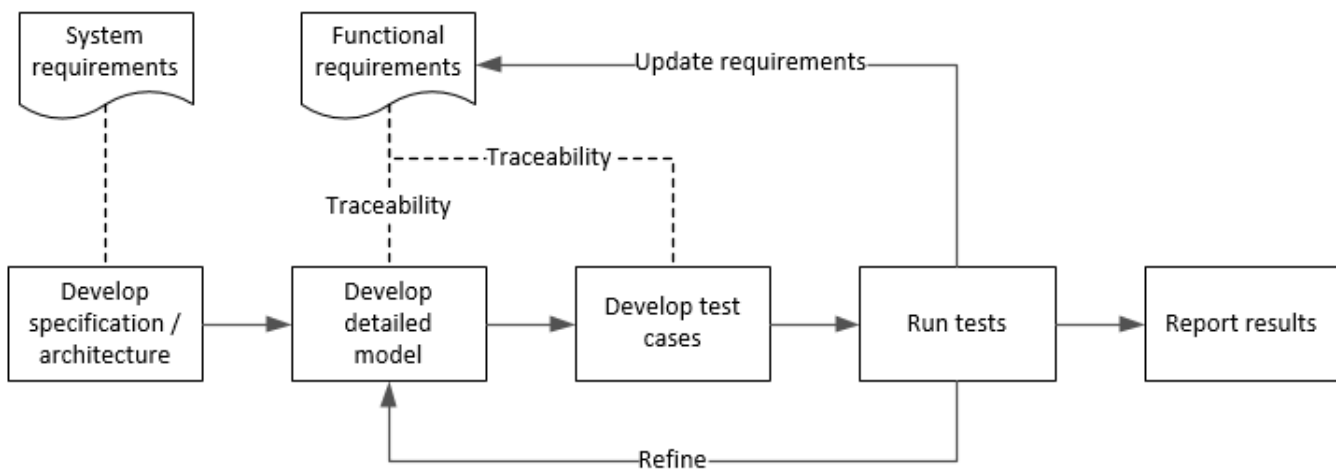
More About

- “Model-Based Design with Simulink”

Test Model Against Requirements and Report Results

Establishing traceability between requirements and test cases helps you interpret the test results and see the extent to which your model verifies your requirements. You can establish traceability by linking a requirement to the elements that verify it, such as the test cases in the Test Manager, verify statements in a Test Sequence block, or model verification blocks. When you run tests, a pass/fail summary appears in your requirements set.

This example demonstrates how to test the requirements of a cruise control system against a model. You start with a requirements set, a model, and a test case. You add traceability between the tests and the safety requirements, run the test, summarize the verification status, and report the results.



In this example, you test of two requirements in the requirement set:

- That the cruise control system transitions from engaged to disengaged when a braking event occurs
- That the cruise control system transitions from engaged to disengaged when the current vehicle speed is outside the range of 20 mph to 90 mph

Display the Requirements

- 1 Open the example project.

```
openExample("shared_vnv/CruiseControlVerificationProjectExample");
pr = openProject("SimulinkVerificationCruise");
```

- 2 Open the simulinkCruiseAddReqExample model.

```
open_system("simulinkCruiseAddReqExample");
```

- 3 Display the requirements. Click the  icon in the lower-right corner of the model canvas, and select **Requirements**. The **Requirements** pane opens.
- 4 Display the verification and implementation status. Right-click the requirement set and select **Verification Status** and **Implementation Status**.

The screenshot displays the MATLAB/Simulink environment. The main workspace shows a Simulink model titled 'simulinkCruiseAddReqExample'. The model includes several input blocks: 'CruiseOnOff' (boolean), 'Brake' (boolean), 'Speed' (single), 'CoastSetSw' (boolean), and 'AccelResSw' (boolean). These inputs feed into a central block labeled 'Compute target speed'. The outputs of the model are 'engaged' (boolean) and 'tspeed' (single). The 'engaged' output is connected to a block labeled '1', and the 'tspeed' output is connected to a block labeled '2'.

The 'Requirements' pane at the bottom left shows a list of requirements. The requirement 'A 1.2' is highlighted, which is 'Set Speed / Decelerate Button'. The 'Implemented' column for this requirement shows a green bar, indicating it is implemented. The 'Verified' column shows a yellow bar, indicating it has not been verified.

The 'Property Inspector' on the right shows the details for requirement 'A 1.2'. The 'Type' is 'Functional', the 'Index' is '1.2', and the 'Custom ID' is 'A 1.2'. The 'Summary' is 'Set Speed / Decelerate Button'. The 'Description' tab is active, showing the following text:

Set Speed/Decelerate Button

The controller shall have an input button to:

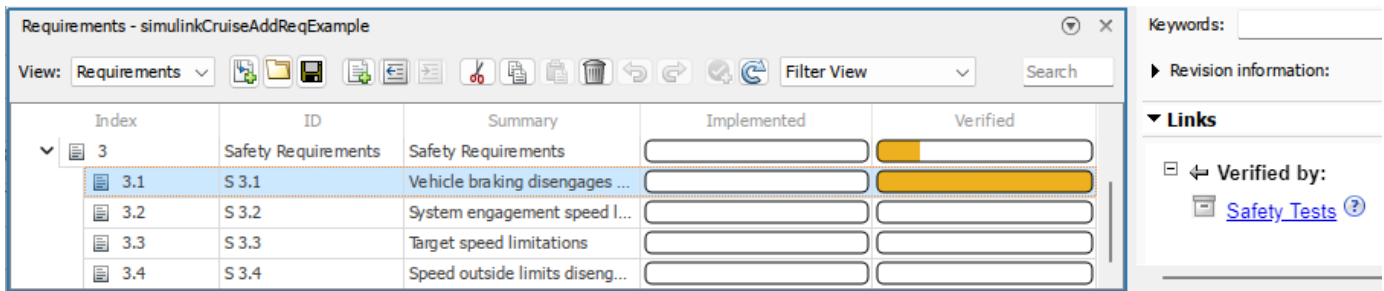
- set the target speed to the current vehicle speed when the cruise control is **not engaged (active)**
- decelerate (reduce) the target speed when the cruise control is **engaged (active)**

The 'Keywords' field is empty. The 'Revision information' section is empty. The 'Links' section shows 'Implemented by:' with a link to 'CoastSetSw'. The 'Comments' section is empty.

Link Requirements to Tests

Link the requirements to the test case.

- 1 In MATLAB, in the **Project** pane, in the tests folder, open the Simulink Test file `slReqTests.mldatx`. The test file opens in the Test Manager.
- 2 In the model, click on requirement S 3.1.
- 3 In the Test Manager, select the **Safety Tests** test case. In the right pane, in the **Requirements** section, click **Add > Link to Selected Requirement**. In the model, in the Property Inspector, the **Verified by** section indicates a link to the **Safety Tests**. In the **Requirements** pane, the yellow bars in the **Verified** column indicate that the linked tests have not been executed.



- Repeat the previous step to add a link for requirement S 3.4.

Run the Test

The test case uses a test harness named **SafetyTest_Harness1**. To open the test harness, click the  icon in the lower-right corner of the model canvas and select **SafetyTest_Harness1**. In the test harness, open the Test Sequence block. The test sequence sets the input conditions and checks the model behavior:

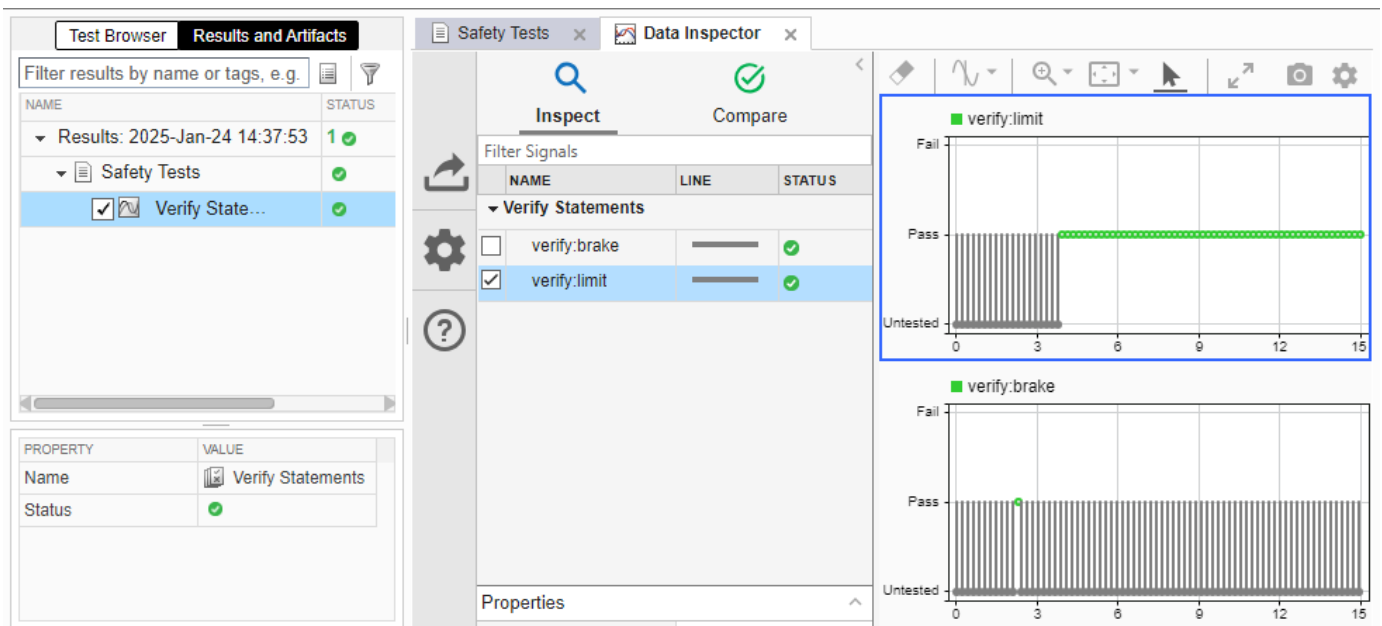
- The BrakeTest sequence engages the cruise control, then applies the brake. It includes this verify statement:

```
verify(engaged == false,...
    'verify:brake',...
    'system must disengage when brake applied')
```

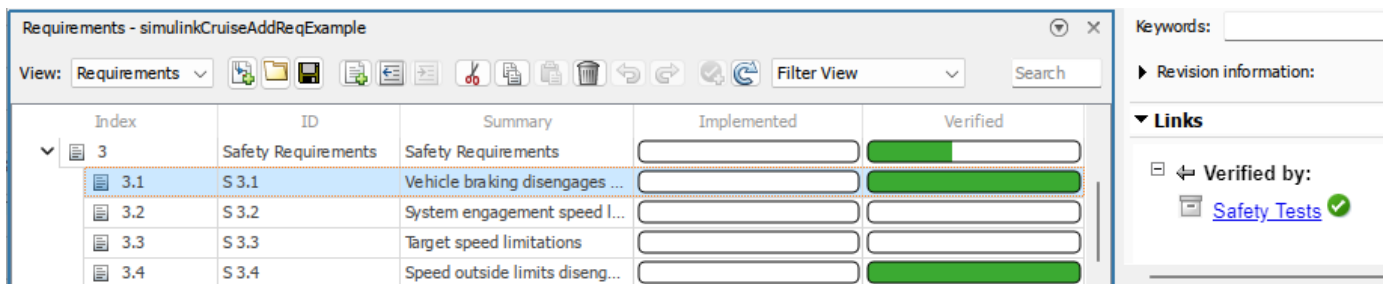
- The LimitTest sequence engages the cruise control, then ramps up the vehicle speed until it exceeds the upper limit. It includes this verify statement:

```
verify(engaged == false,...
    'verify:limit',...
    'system must disengage when limit exceeded')
```

- To run the test case, return to the Test Manager and click **Run**.
- When the test finishes, review the results. In the Test Manager, in the **Results and Artifacts** pane, expand the test case and select **Verify Statements**. Then, in the **Data Inspector** tab, in the **Inspect** tab, select the verify statements. The plots provide the detailed results of each verify statement and show that both assessments pass.



- Return to the model and click the Refresh button in the **Requirements** pane to refresh the requirements. The green bar in the **Verified** column indicates that the test case successfully verified requirements S 3.1 and S 3.4.



Report the Results

Create a report using a custom Microsoft Word template.

- In the Test Manager, in the **Results and Artifacts** pane, right-click the test case name. Select **Create Report**.
- In the Create Test Result Report dialog box, set these options:
 - Title** — SafetyTest
 - Results for** — All Tests
 - File Format** — DOCX
- Enter a file name and select a location for the report.
- Set **Template File** to the file ReportTemplate.dotx file in the documents project folder.
- Click **Create**.
- Review the report.

- The **Test Case Requirements** section specifies the associated requirements.
- The **Verify Result** section contains details of the two assessments in the test and links to the simulation output.

See Also

Related Examples

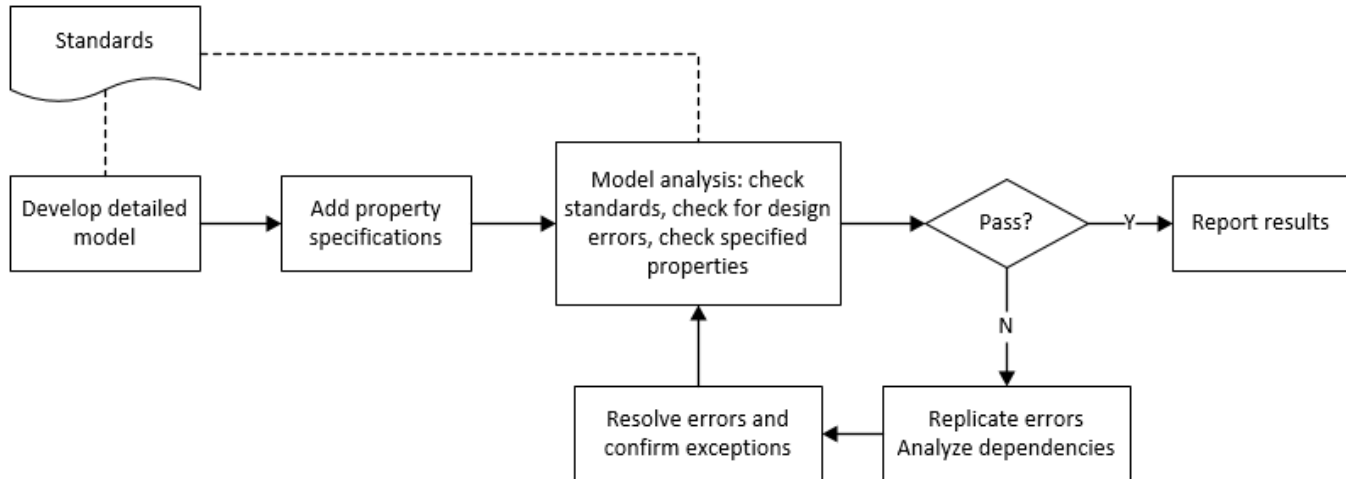
- “Link to Requirements” on page 1-2
- “Validate Requirements Links in a Model” (Requirements Toolbox)
- “Customize Requirements Traceability Report for Model” (Requirements Toolbox)

External Websites

- Requirements-Based Testing Workflow

Analyze Models for Standards Compliance and Design Errors

During model development, you can check and analyze your model against standards such as MAB style guidelines and high-integrity system design guidelines such as DO-178 and ISO 26262. When you analyze your model, you identify errors, dead logic, and conditions that violate required properties. You can then use the analysis results to update your model and document exceptions and report the results using customizable templates.



This example shows how to use the Model Advisor to check a cruise control model for MathWorks Advisory Board (MAB) style guideline violations and design errors. You select checks and run the analysis on the model and iteratively debug issues using the Model Advisor and rerun checks to verify that it is in compliance. When the model passes the selected checks, you report the results.

Check Model for MAB Style Guideline Violations

Check that your model complies with MAB guidelines by using the Model Advisor.

- 1 Open the example project.

```
openExample("shared_vnv/CruiseControlVerificationProjectExample");
pr = openProject("SimulinkVerificationCruise");
```

- 2 Open the `simulinkCruiseErrorAndStandardsExample` model.

```
open_system("simulinkCruiseErrorAndStandardsExample");
```

- 3 In the **Modeling** tab, select **Model Advisor**.

- 4 From the System Selector dialog, select `simulinkCruiseErrorAndStandardsExample` and click **OK**.

- 5 Check your model for MAB style guideline violations by using Simulink Check:

- a In the left pane, in the **By Product > Simulink Check > Modeling Standards > MAB Checks** folder, select:

- **Check Indexing Mode**
- **Check model diagnostic parameters**

- b** Right-click the **MAB Checks** node and select **Run Checks**.
- c** In the left pane, select **Check model diagnostic parameters**. The analysis results appear in the right pane, on the **Report** tab. The report displays the violation details and the recommended action.
- d** To open the Configuration Parameters dialog box, click the parameter hyperlinks, update the model diagnostic parameters, and save the model.
- e** To verify that your model passes, rerun the check. Repeat steps from **c** to **e**, if necessary, until both checks pass.
- f** To generate a results report of the Simulink Check checks, select the **MAB Checks** node, and click **Report** in the toolbar.

Check Model for Design Errors

In the Model Advisor, you can also check your model for hidden design errors by using Simulink Design Verifier.

- 1** In the left pane, in the **By Products > Simulink Design Verifier** folder, click the check box for **Design Error Detection**. All checks in the folder are selected.
- 2** From the toolbar, click **Run Checks**.
- 3** When the analysis is complete, generate an HTML report of the analysis. In the toolbar, click **Report**.
- 4** In the generated report, click a **Simulink Design Verifier Results Summary** hyperlink. The dialog box helps you diagnose errors and warnings in your model. You can:
 - a** Review the analysis results for the model. Click the **Compute target speed** subsystem. The Simulink Design Verifier Results Inspector window provides derived ranges that can help you understand the source of an error by identifying the possible signal values.
 - b** Review the harness model or create one if it does not already exist.
 - c** View tests and export test cases.
 - d** Review the analysis report. To see a detailed analysis report, click **HTML** or **PDF**.

See Also

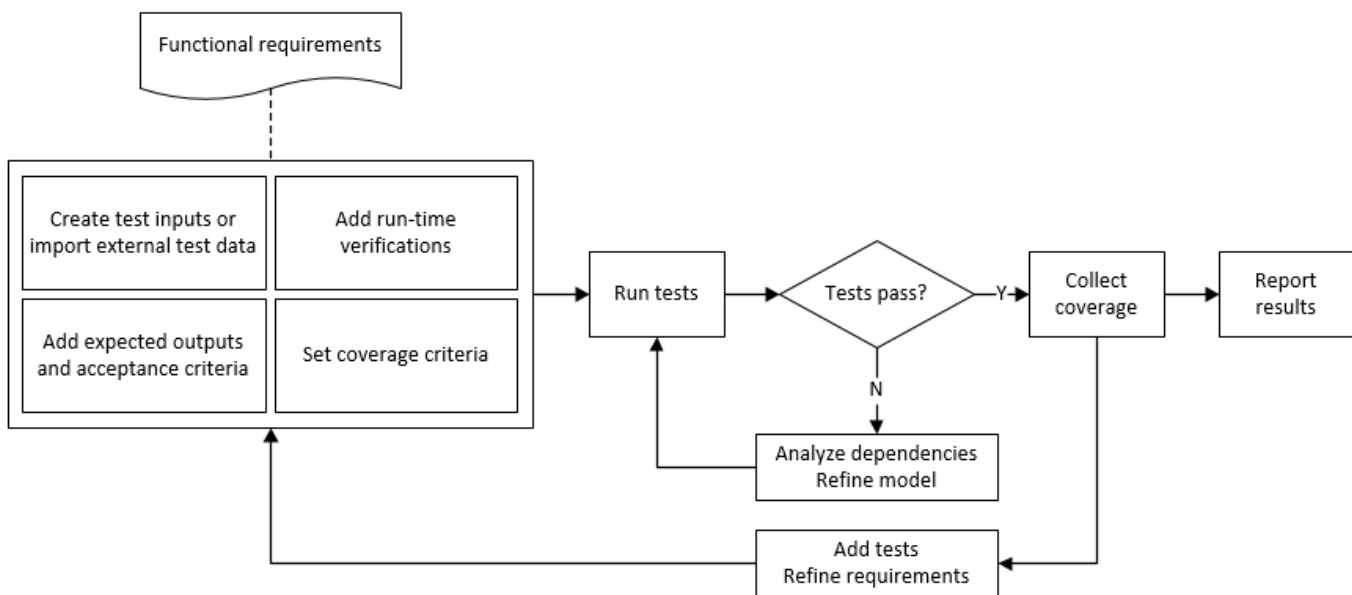
Related Examples

- “Check Model Compliance by Using the Model Advisor” (Simulink Check)
- “Collect Model Metrics Using the Model Advisor” (Simulink Check)
- “Analyze Models for Design Errors” (Simulink Design Verifier)
- “Prove Properties in a Model” (Simulink Design Verifier)

Perform Functional Testing and Analyze Test Coverage

Functional testing begins with building test cases based on requirements. These tests cover key aspects of your design and verify that individual model components meet requirements. Test cases made for functional testing include inputs, expected outputs, and acceptance criteria. Additionally, if you store your functional test cases in test suites, you can run functional tests systematically.

You can also run regression testing and measure different levels of coverage to further refine your tests. To check for regression, you add baseline criteria to the test cases and compare the model outputs to a set of baseline data. You can then do coverage measurement on your model to identify the extent to which the tests exercised the model and identify if you need to add tests or requirements to meet your coverage targets.



This example shows how to perform requirements-based tests for a cruise control model. In this example, you:

- 1 Run the tests.
- 2 Determine test coverage by using Simulink Coverage.
- 3 Increase coverage by generating additional tests using Simulink Design Verifier.
- 4 Report the results.

Open the Test Harness and Model

- 1 Open the example project:

```
openExample("shared_vnv/CruiseControlVerificationProjectExample");
pr = openProject("SimulinkVerificationCruise");
```

- 2 Open the model and the test harness.

```
open_system("simulinkCruiseAddReqExample");
sltest.harness.open("simulinkCruiseAddReqExample", "SafetyTest_Harness1")
```

- 3 Open the test file `slReqTests.mldatx` in the Simulink Test Manager.

```
pf = fullfile(pr.RootFolder,"tests","slReqTests.mldatx");
tf = sltest.testmanager.TestFile(pf);
sltest.testmanager.view
```

- 4 In the test harness, open the Test Sequence block. The sequence verifies system disengagement when either:
 - The brake pedal is pressed.
 - The speed exceeds a limit.

Measure Model Coverage

- 1 In the Simulink Test Manager, select the `slReqTests` test file.
- 2 To enable coverage collection, in the right pane, expand **Coverage Settings**, then expand **Coverage to Collect**:
 - Select **Record coverage for referenced models**.
 - Select **Decision**, **Condition**, and **MCDC**.
- 3 On the Test Manager toolstrip, click **Run**.
- 4 After the test completes, click **Results and Artifacts**. In the **Results and Artifacts** pane, select the results. The right pane displays the coverage results in the **Aggregated Coverage Results** section. The test achieves 50% decision coverage, 41% condition coverage, and 25% MCDC coverage.

ANALYZED MODEL	REPORT	CO...	DECISION	CONDITION	MCDC
simulinkCruiseAddReqExample		31	50%	41%	25%

+ Add Tests for Missing Coverage Export

Generate Tests to Increase Model Coverage

Use Simulink Design Verifier to generate additional tests to increase model coverage.

- 1 In **Results and Artifacts**, select the `slReqTests` test file and, in the right pane, open the **Aggregated Coverage Results** section.
- 2 Right-click the test results and select **Add Tests for Missing Coverage**.
- 3 Set **Harness** to **Create a new harness**.
- 4 Click **OK**. The model being tested must either be on the MATLAB path or in the working folder.
- 5 To execute the updated test suite, on the Test Manager toolstrip, click **Run**. The test results include coverage for the combined test case inputs and achieve increased model coverage.

See Also

`sltest.testmanager.addTestsForMissingCoverage` | `sltest.testmanager.TestOptions`

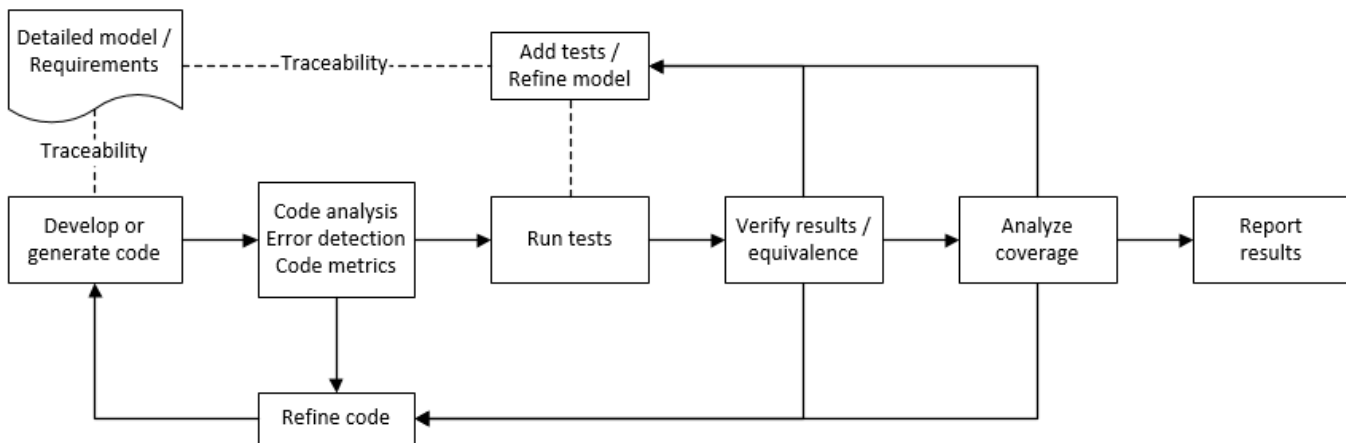
Related Examples

- “Link to Requirements” on page 1-2
- “Verify Model Simulation by Using when Decomposition” on page 3-14
- “Baseline Testing” on page 6-240
- “Generate Test Cases for Model Decision Coverage” (Simulink Design Verifier)
- “Increase Test Coverage for a Model” on page 6-163

Analyze Code and Perform Software-in-the-Loop Testing

You can analyze your code to detect errors, check for standards compliance, and evaluate key metrics such as length and cyclomatic complexity. When you create your own code, you typically check for run-time errors by using static code analysis and run test cases that evaluate the code against requirements and evaluate code coverage. Based on the results, you refine the code and add tests.

In this example, you generate code and demonstrate that the code execution produces equivalent results to the model by using the same test cases and baseline results. Then you compare the code coverage to the model coverage. Based on the test results, you add tests and modify the model to regenerate the code.



Analyze Code for Defects, Metrics, and MISRA C:2012

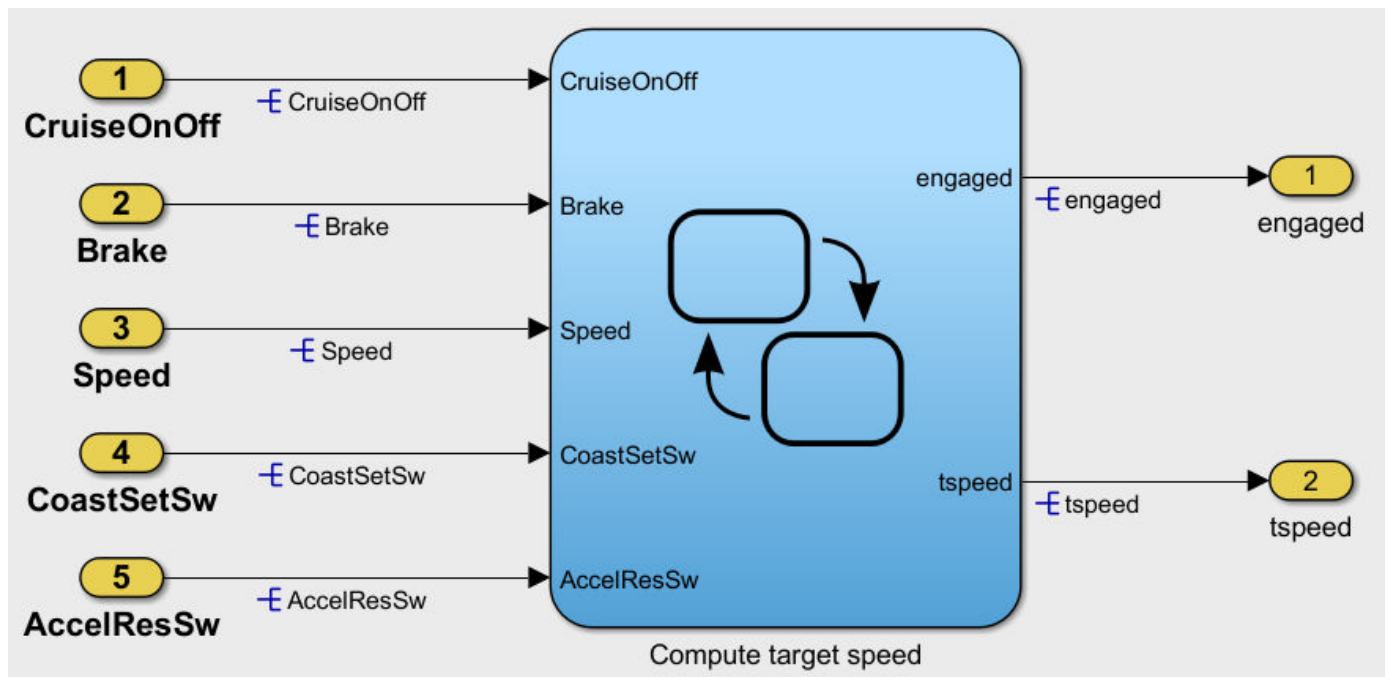
First, check that the model produces MISRA™ C:2012 compliant code and analyze the generated code for code metrics and defects. To produce code compliant with MISRA, you use the Code Generation Advisor and Model Advisor. To check whether the code is MISRA compliant, you use the Polyspace MISRA C:2012 checker.

- 1 Open the example project.

```
openExample("shared_vnv/CruiseControlVerificationProjectExample");
pr = openProject("SimulinkVerificationCruise");
```

- 2 Open the simulinkCruiseErrorAndStandardsExample model.

```
open_system("simulinkCruiseErrorAndStandardsExample");
```

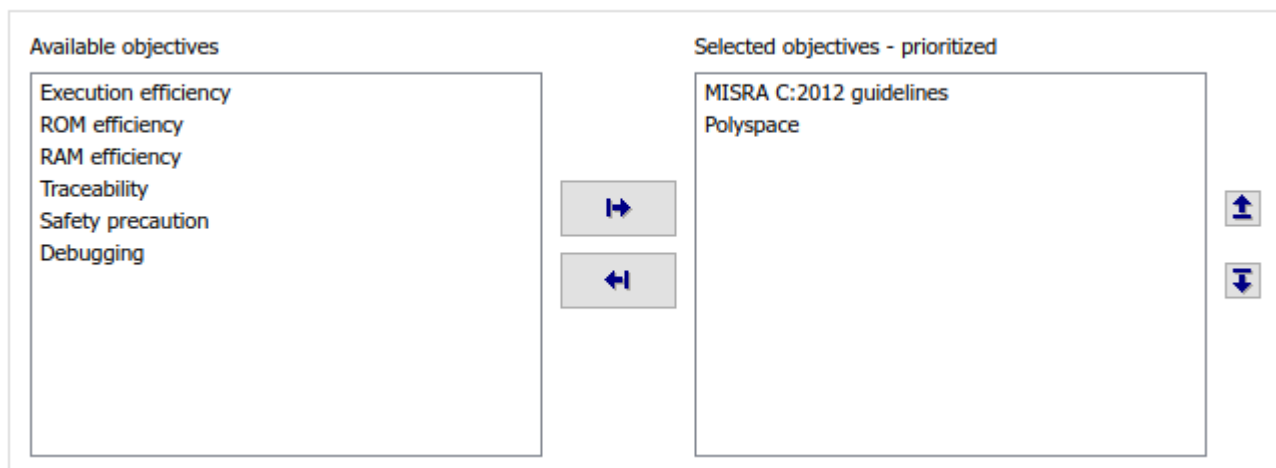


Run Code Generator Checks

Check your model by using the Code Generation Advisor. Configure the code generation parameters to generate code more compliant with MISRA C and more compatible with Polyspace.

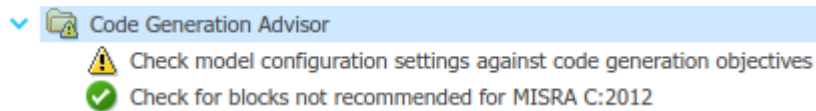
- 1 Right-click the **Compute target speed** subsystem and select **C/C++ Code > Code Generation Advisor**.
- 2 In the left pane, select the **Code Generation Advisor** folder. In the right pane, under **Available objectives** select **Polyspace** and click the right arrow. The **MISRA C:2012 guidelines** objective is already selected.

Code Generation Objectives (System target file: ert.tlc)



- 3 Click **Run Selected Checks**.

The Code Generation Advisor checks whether the model includes blocks or configuration settings that are not recommended for MISRA C:2012 compliance and Polyspace code analysis. For this model, the check for incompatible blocks passes, but some configuration settings are incompatible with MISRA compliance and Polyspace checking.



- 4 Click **Check model configuration settings against code generation objectives**. Accept the parameter changes by selecting **Modify Parameters**.
- 5 To rerun the check, click **Run This Check**.

Run Model Advisor Checks

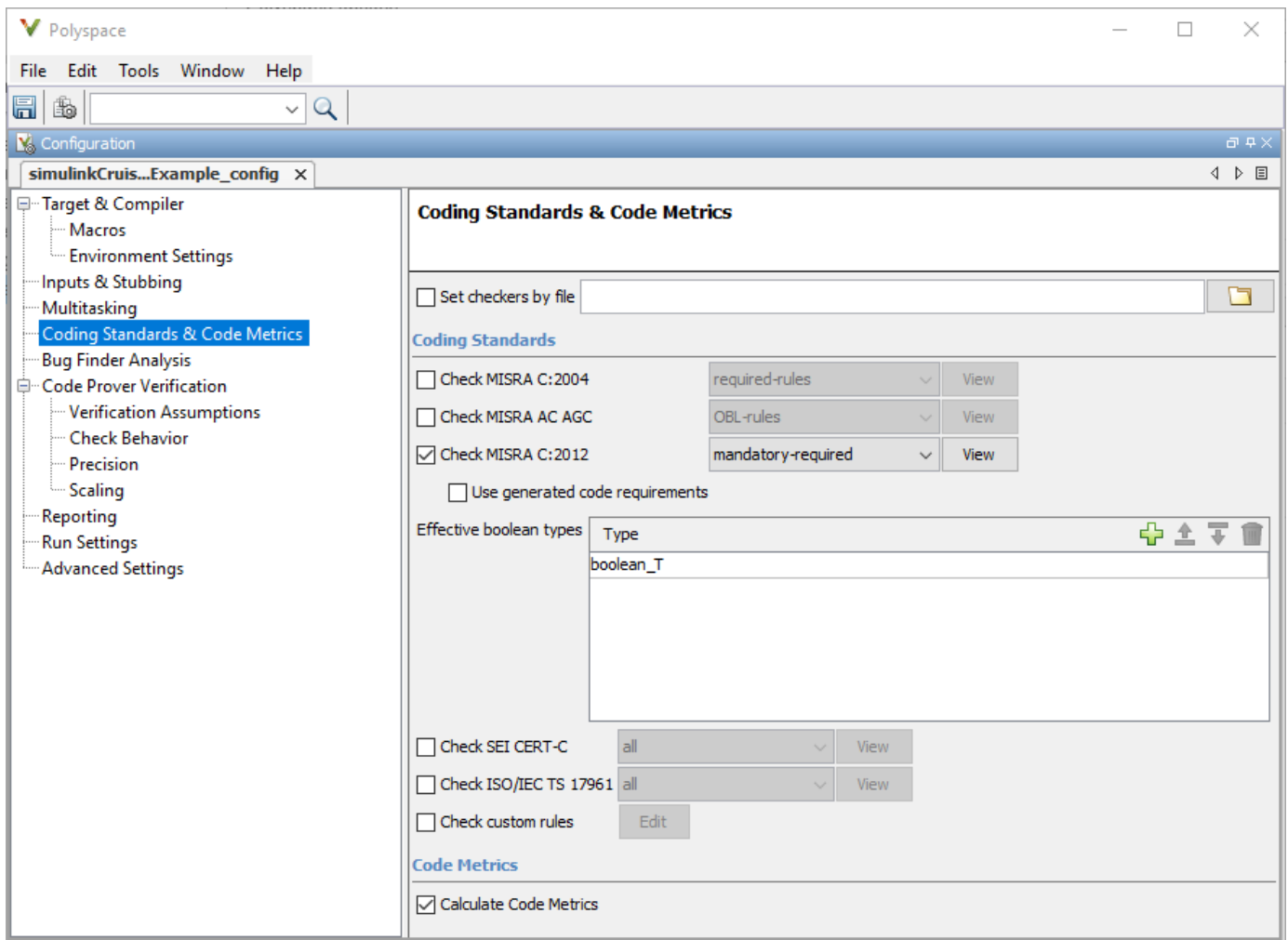
Before you generate code from your model, use the Model Advisor to check your model for MISRA C and Polyspace compliance.

- 1 At the bottom of the Code Generation Advisor window, click **Model Advisor**.
- 2 In the Model Advisor, under the **By Task** folder, select **Modeling Standards for MISRA C:2012**.
- 3 Click **Run Checks** and review the results.
- 4 If any of the tasks fail, make the suggested modifications and rerun the checks until the MISRA modeling guidelines pass.

Generate and Analyze Code

After you check the model for compliance, you can generate the code. After you generate the code, you can use Polyspace to check the code for compliance with MISRA C:2012 and generate reports to demonstrate compliance with MISRA C:2012.

- 1 In the Simulink model, right-click the `Compute target speed` subsystem and click **C/C++ Code > Build This Subsystem**.
- 2 After the code generates, in the Simulink Editor, right-click the `Compute target speed` subsystem and select **Polyspace > Options**.
- 3 Click **Configure** to choose more advanced Polyspace analysis options. The Polyspace window opens.
- 4 In the left pane, click **Coding Standards & Code Metrics**, then select **Calculate Code Metrics** to enable code metric calculations for your generated code. Select **Check MISRA C:2012**.
- 5 Save and close the Polyspace window.

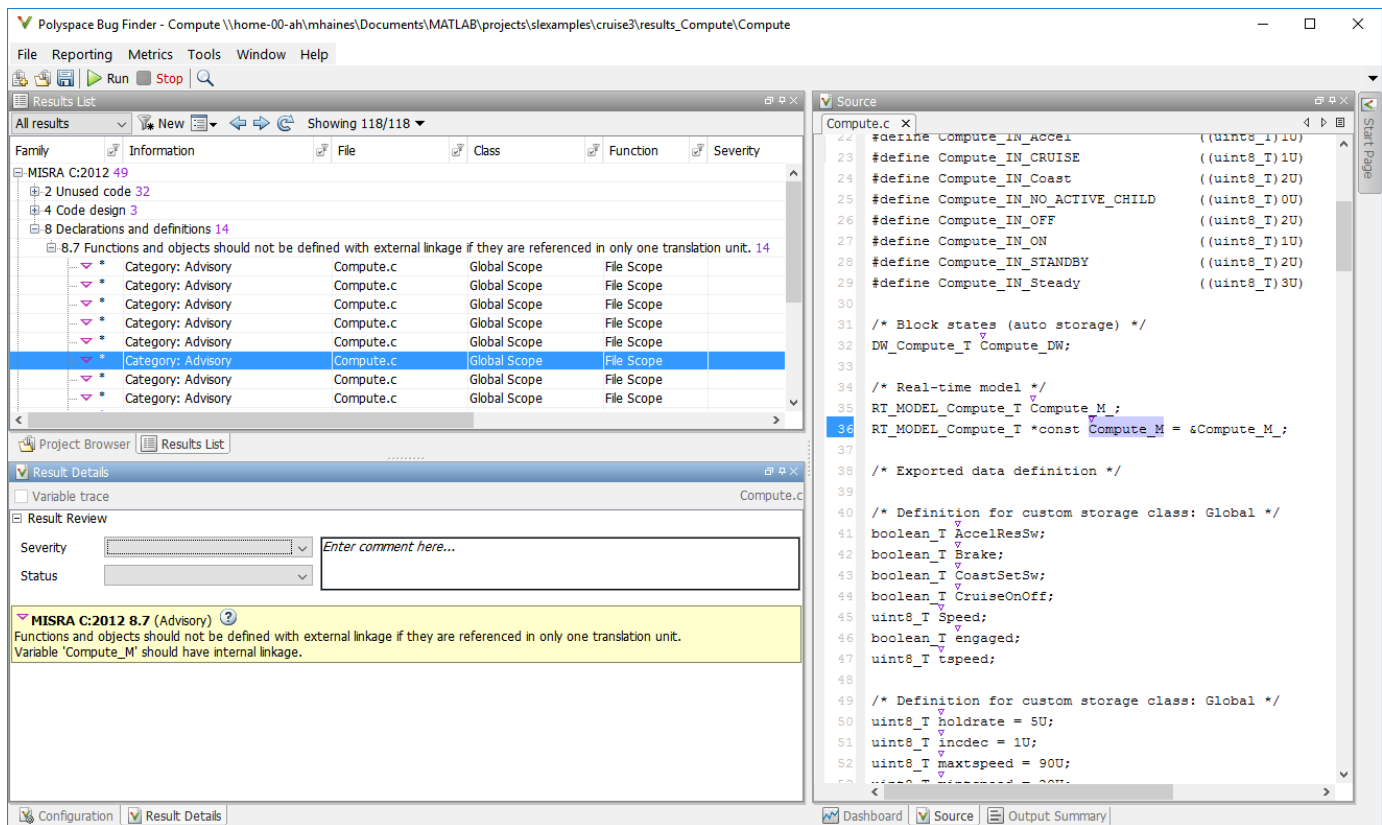


- 6 In the model, right-click the Compute target speed subsystem and select **Polyspace > Verify > Code Generated For Selected Subsystem**.

Polyspace Bug Finder analyzes the generated code for a subset of MISRA checks. You can see the progress of the analysis in the MATLAB Command Window. After the analysis finishes, the Polyspace environment opens.

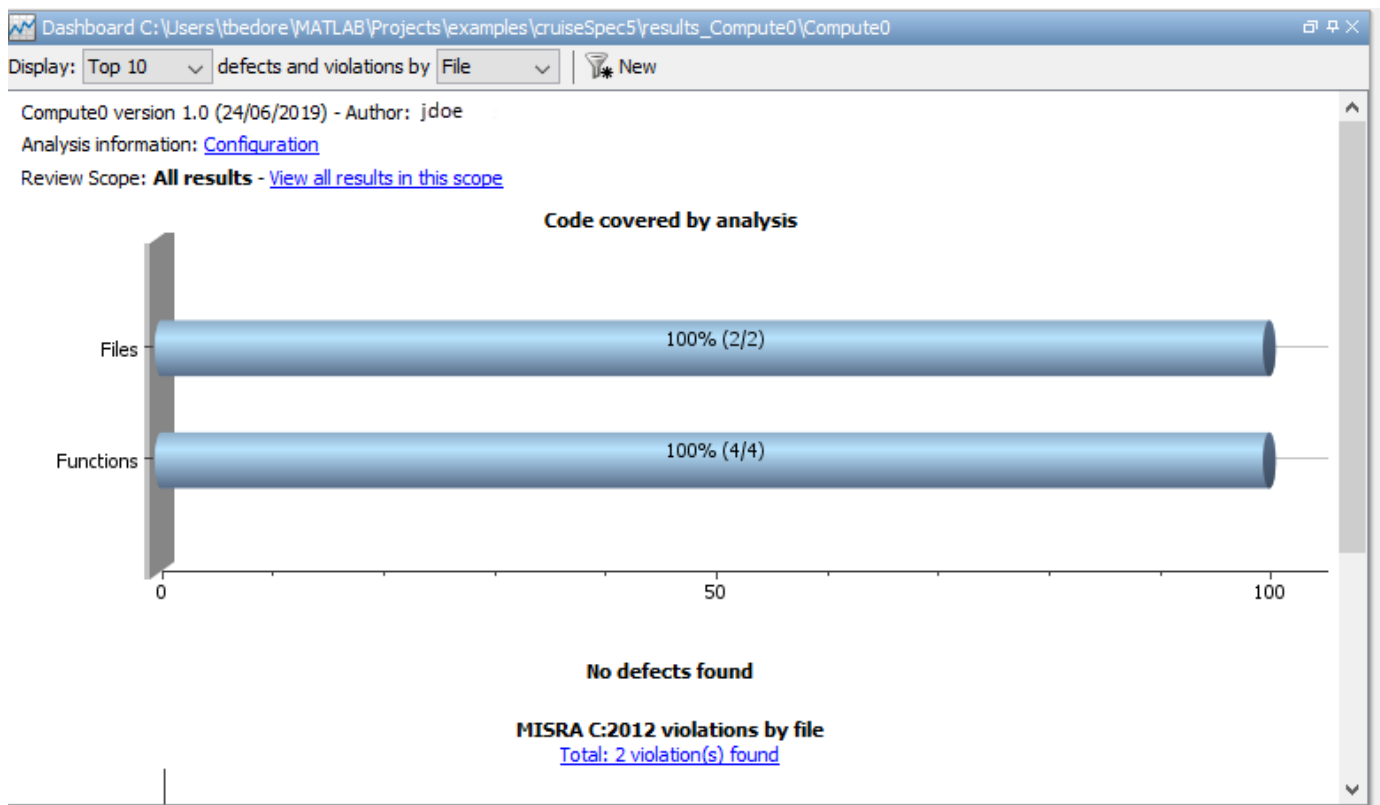
Review Results

The Polyspace environment shows you the results of the static code analysis. For example, expand the tree for rule 8.7 and click through the results. Rule 8.7 states that functions and objects should not be global if the function or object is local. These results refer to variables that other components also use, such as `CruiseOnOff`. You can annotate your code or your model to justify every result.



To configure the analysis to check only a subset of MISRA rules:

- 1 In your model, right-click the **Compute** target speed subsystem and select **Polyspace > Options**.
- 2 In the Configuration Parameters dialog, set **Settings from** to **Project configuration**.
- 3 Click **Apply**.
- 4 Click **Configure**.
- 5 In the Polyspace window, in the left pane, click **Coding Standards & Code Metrics**. Then select **Check MISRA C:2012** and, from the drop-down list, select **single-unit-rules**. Now Polyspace checks only the MISRA C:2012 rules that are applicable to a single unit.
- 6 Save and close the Polyspace window.
- 7 Rerun the analysis with the new configuration. When you limit the rules Polyspace checks to the single-unit subset, Polyspace finds only two violations.



Generate Report

To demonstrate compliance with MISRA C:2012 and report on your generated code metrics, you must export your results. If you want to generate a report every time you run an analysis, see [Generate report](#) (Polyspace Bug Finder).

- 1 If they are not open already, open your results in the Polyspace environment.
- 2 From the toolbar, select **Reporting > Run Report**.
- 3 Select **BugFinderSummary** as your report type.
- 4 Click **Run Report**. The report is saved in the same folder as your results.
- 5 To open the report, select **Reporting > Open Report**.

Test Code Against Model Using Software-in-the-Loop Testing

Next, run the same test cases on the generated code to show that the code produces the equivalent results to the original model and fulfills the requirements. Then compare the code coverage to the model coverage to see the extent to which the tests exercised the generated code.

- 1 In MATLAB, in the **Project** pane, in the tests folder, open `SILTests.mldatx`. The file opens in the Test Manager.
- 2 Review the test case. In the **Test Browser** pane, click **SIL Equivalence Test Case**. This equivalence test case runs two simulations for the `simulinkCruiseErrorAndStandardsExample` model using a test harness:

- Simulation 1 is a model simulation in normal mode.
- Simulation 2 is a software-in-the-loop (SIL) simulation. For the SIL simulation, the test case runs the code generated from the model instead of running the model.

The equivalence test logs one output signal and compares the results from the simulations. The test case also collects coverage measurements for both simulations.

- 3 To run the equivalence test, select the test case and click **Run**.
- 4 Review the results in the Test Manager. In the **Results and Artifacts** pane, select **SIL Equivalence Test Case**. The test case passes and the code produces the same results as the model for this test case.

The screenshot displays the MATLAB/Simulink Test Manager interface. The left pane shows the 'Test Browser' with a tree view of test results. The 'SIL Equivalence Test Case' is selected, showing a status of '1' (pass) and a green checkmark. The right pane shows the 'Results and Artifacts' for the selected test case, including a 'SUMMARY' tab, 'LOGS', 'DESCRIPTION', and 'COVERAGE RESULTS'.

The 'COVERAGE RESULTS' section contains a table with the following data:

ANALYZED MODEL	REPORT	SIM MODE	COM...	DECISION	CONDITION	MCDC	EXECUTION	FUNCTION	FUNCTION...
div_su32.c		SIL	2	0%	--	--	0%	0%	--
simulinkCruiseErrorAndStandardsExample		ModelRe...	22	54%	44%	17%	70%	100%	33%
simulinkCruiseErrorAndStandardsExample		Normal	31	50%	41%	25%	--	--	--

At the bottom of the right pane, there are buttons for 'Add Tests for Missing Coverage' and 'Export'.

- 5 In the right pane, expand the **Coverage Results** section. The coverage measurements show the extent to which the test case exercises the model and the code.

When you run multiple test cases, you can view aggregated coverage measurements in the results for the whole run. Use the coverage results to add tests and meet coverage requirements, as shown in "Perform Functional Testing and Analyze Test Coverage" (Simulink Check).

You can also test the generated code on your target hardware by running a processor-in-the-loop (PIL) simulation. By adding a PIL simulation to your test cases, you can compare the test results and coverage results from your model to the results from the generated code as it runs on the target hardware. For more information, see "Code Verification Through Software-in-the-Loop and Processor-in-the-Loop Execution" (Embedded Coder).

See Also

Related Examples

- “Run Polyspace Analysis on Code Generated with Embedded Coder” (Polyspace Bug Finder)
- “Back-to-Back Equivalence Testing” on page 6-36
- “Export Test Results” on page 7-16

